Além do código

Backend de verdade para quem vai assumir a liderança técnica

Claude

Além do código: Backend de verdade para quem vai assumir a liderança técnica

Texto escrito por Claude, um modelo de IA da Anthropic.

Versões, valores padrão e recomendações de ferramentas e padrões mudam com o tempo. Confira a documentação oficial antes de tomar decisões com base neles.

Edição gerada em 06/11/2026.

Sumário

Antes do show

Parte I: O núcleo: dinheiro no banco de dados

1 **Modelagem relacional**

Um centavo, um sábado e um financeiro que nunca mais foi o mesmo
Ponto flutuante, ou como errar com dezessete dígitos de convicção
Arredondamento: o centavo órfão precisa de um tutor legal
Partidas dobradas: dinheiro não nasce nem morre, só muda de conta
Chaves, ou quem é quem quando todo mundo tem dois documentos
Formas normais: cada fato num lugar só, que nem segredo de família
Desnormalização: toda cópia é uma promessa, e promessa se quebra
Restrições: o banco como último adulto na sala
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

2 **Concorrência na aplicação**

Dois saques, um saldo e um vendedor com duas abas abertas
Condição de corrida, o bug que só aparece quando você não está olhando
Seção crítica e mutex, ou ciúme com suporte do sistema operacional
Operações atômicas: o check e o act no mesmo instante
Deadlock e as quatro condições de Coffman
O lock do Carlos e os processos que não se conheciam
Locks distribuídos, ou o mutex com prazo de validade
Na Lastro: o lock vai morar com o dinheiro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

3 **Transações e isolamento**

A quarta aba
Uma transação, quatro contas e zero meio-termo
O padrão SQL define isolamento pelos defeitos
MVCC: o banco que guarda todas as versões da sua vida
Write skew: dois médicos de plantão e nenhum no hospital
Serializable de verdade, com nota fiscal
Na Lastro: um nível para cada operação
Armadilhas e casos reais

Resumo
A Lastro agora
Referências

4 Locks, contenção e pools de conexão
A conta que todo mundo quer tocar
Onde o PostgreSQL guarda um lock
A matemática da fila indiana
FOR UPDATE, NOWAIT e SKIP LOCKED
Otimista, ou "trava nada e reza"
Deadlock no banco
Uma conexão é um processo
Pools: guardar conexões abertas
Quantas conexões, afinal
Na Lastro: a conta quente
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

5 Índices
O extrato de seis segundos e o índice de quarenta e oito
A B-tree por dentro, ou a busca binária que foi para a academia
O que o índice guarda, e a visita obrigatória à tabela
A ordem das colunas, ou o índice lê da esquerda para a direita
Seletividade, cardinalidade e o dia em que o planner ignora o seu índice
Parciais, de expressão e os índices que não são árvore
Cada índice é um imposto sobre a escrita
Os índices do ledger, escolhidos pelo que se lê
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

6 Planos de execução
O painel das 400 consultas e a estagiária na página 4.000
EXPLAIN, ou como fazer o banco confessar
Três jeitos de juntar duas tabelas
Estatísticas, ou o planner decide com o que viu há 19 dias
N+1, o ORM que pergunta uma coisa de cada vez
OFFSET, o banco que conta até 199.950 em voz alta
O painel e o extrato da Lastro
Armadilhas e casos reais
Resumo
A Lastro agora

Referências

7 **Migrações de schema sem downtime**

"É só uma coluna", às 3h11 de um domingo
Todo ALTER TABLE quer a tabela só para ele
Seguras, perigosas e as que mentem sobre o que são
Expand/contract, a migração em prestações
Backfill em lotes, ou como atualizar 45 milhões de linhas sem ninguém perceber
O ledger da Lastro aprende espanhol
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

Parte II: A API no mundo

8 **TCP/IP e DNS**

Camadas, ou como 300 bytes de JSON viajam com mais cabeçalho do que conteúdo
IP, portas e o endereço que ninguém tem de verdade
O handshake, ou como dois computadores precisam de três mensagens para dizer oi
Controle de fluxo e de congestionamento, ou o TCP pisando no freio por dois motivos diferentes
Fim de conexão e TIME_WAIT, o luto obrigatório
UDP, o protocolo que manda e vai embora
RTT, ou a velocidade da luz não aceita chamado com "URGENTE!!!!"
DNS, o banco de dados distribuído que todo mundo esquece que é um banco de dados
Na Lastro: 200 ms recuperados e uma troca de IP de duas semanas
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

9 **HTTP em profundidade**

O link que estornava sozinho
HTTP/1.1, ou texto puro com pretensão de protocolo
HTTP/2: cem conversas numa conexão só, e todas morrem juntas
HTTP/3: o QUIC desistiu do TCP e foi morar no UDP
Métodos, ou a promessa que todo robô da internet cobra do seu GET
Status, ou três dígitos para dizer o que aconteceu
Cache e validação condicional, ou como guardar dinheiro velho
CORS, a regra de segurança que só o navegador respeita
A API de cobranças da Lastro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

10 **TLS e certificados**

Domingo, 3h12, ou a hora exata em que o fundador marcou o próprio funeral
O que o TLS garante, em três propriedades e nenhuma a mais
O handshake do TLS 1.3, ou dois estranhos combinando um segredo em voz alta numa praça
O certificado, um documento que só vale porque alguém que você nunca viu assinou
Cadeia de confiança, ou o golpe da pirâmide que funciona
SNI e terminação TLS, ou onde a criptografia acaba e a confiança começa
ACME, o robô que não esquece aniversário
mTLS, quando o servidor também pede documento
Na Lastro: o vencimento sai das mãos humanas
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

11 **Contratos de API**

"É só um campo", ou como um JSON educado derrubou um marketplace inteiro
O contrato é tudo o que o cliente consegue ver, inclusive o que você nunca prometeu
Versionar é escolher onde mora a mentira
Paginação e filtros, o contrato escrito nas entrelinhas
Erros com formato: RFC 9457, o fim do {"erro": "deu ruim"}
OpenAPI e testes de contrato, ou o contrato com firma reconhecida
As regras da Lastro, por escrito e com testemunha
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

12 **Estilos de comunicação**

Três pedidos, três protocolos e um gerente que acha tudo igual
REST, o estilo que já pagava as contas
gRPC, um contrato binário com HTTP/2 obrigatório e nenhum ser humano no meio
GraphQL, ou o cliente escreve a consulta e você paga a conta
Empurrar do servidor: WebSocket, SSE e webhooks
Na Lastro, cada canal recebe o estilo que pede
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

13 **Processos, threads e I/O assíncrono**

A CPU a 4% e ninguém atendendo
Processo e thread, ou a árvore genealógica mais mórbida da computação
A troca de contexto, ou o kernel que vive de interromper

I/O bloqueante e não bloqueante, ou a arte de não esperar sentado
O event loop, ou uma thread que não pode piscar
O GIL, ou o cadeado que o Python pôs no próprio pé
Modelos de servidor, ou quantas requisições cabem num processo
Quantos workers, ou a conta que o fundador fez de cabeça
Na Lastro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

14 Desempenho mensurável

A média do Carlos
Latência, throughput e por que a média mente na cara dura
A Lei de Little, ou a equação que entrega o teste do Carlos
Utilização e filas, ou por que 90% de CPU é uma emergência
A Lei de Amdahl, ou otimizar o que não importa com muita competência
Números de latência para fazer conta de cabeça
Teste de carga, ou como o gerador mente para você
Na Lastro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

Parte III: Dinheiro entre sistemas

15 Falhas parciais

Recusada, segundo o cronômetro
As formas criativas de uma chamada remota dar errado
Timeout, ou o momento exato de desistir de alguém
Retry, ou insistir com quem já disse que está ocupado
Circuit breaker e bulkhead, ou a arte de largar quem não te merece
Deadlines e degradação, ou quanto tempo ainda resta de vida
Na Lastro: um "não sei" honesto
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

16 Idempotência e semântica de entrega

Duas vezes, com jitter
At-most-once, at-least-once e o unicórnio
Idempotência, a palavra que todo mundo usa e quase ninguém conjuga
A chave de idempotência, ou como dar RG para uma intenção

Deduplicação do lado de quem recebe
Na Lastro: cada efeito com um RG
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

17 Mensageria
Quinta, 14h02: o Hornero ficou lento e a Lastro foi junto, por solidariedade
O broker, ou o primeiro componente da Lastro cuja função é segurar coisa dos outros
Na Lastro: um log para os fatos, uma fila para as entregas
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

18 Transações distribuídas
Terça, 16h00min03s: o Pix existe, o evento não
A escrita dupla, ou duas linhas de código que não podem ser atômicas
Two-phase commit, o protocolo que faz todo mundo esperar de mão levantada
Sagas, ou desfazer em vez de não fazer
Outbox e CDC: o evento entra na mesma transação
Conciliação: a verificação que não confia em ninguém
Na Lastro: outbox, duas sagas e um arquivo às 6h
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

19 Modelos de consistência
Sexta, 17h02: dois números certos, com onze minutos de diferença
"Consistência", a palavra que significa três coisas e briga nas três
Linearizabilidade, ou fingir que existe uma cópia só
Descendo a escada: sequencial, causal, eventual
CAP com a definição certa, e o PACELC que diz o resto
Na Lastro: cada leitura com o contrato de que precisa
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

20 Tempo, ordem e consenso
Meia-noite, duas vezes, com seis minutos de intervalo
Relógios físicos, ou cada máquina tem a sua opinião sobre que horas são
Ordem sem relógio: Lamport e os vetores

Consenso, ou como três máquinas concordam sem confiar no relógio de nenhuma
Leases e fencing tokens: o líder que não sabe que morreu
Na Lastro: um dono, um token e um dia que não depende de relógio
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

21 Event sourcing e CQRS

O comprador tem memória, a Lastro tem amnésia
Event sourcing, ou parar de dar UPDATE na própria biografia
O event store, um banco que só aceita confissão
Projeções e snapshots, ou como ler o passado sem reler a vida inteira
Versionar eventos, porque o passado não aceita migração
CQRS, a sigla que a Lastro já praticava escondida
O que a Lastro guarda como evento (e o que continua sendo foto)
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

Parte IV: Escala

22 Cache

A tarifa por sorteio
Por que o cache funciona, e por que ele mente
Os padrões de leitura e escrita, ou quem tem a obrigação de mentir
Invalidação, a coisa difícil que tem nome de coisa fácil
Despejo, ou quem morre quando a memória acaba
Cache stampede, ou a manada no portão
Camadas, e por que o atraso soma
O que a Lastro põe em cache
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

23 Replicação

Primário e réplica, ou quem manda e quem copia
Síncrona ou assíncrona, ou quanto tempo o COMMIT espera pela réplica
Lag de replicação, ou a réplica que vive no passado
Ler o que você acabou de escrever
Failover, split-brain e como aposentar um primário à força
Multi-líder e sem líder, ou democracia na escrita
Na Lastro: três cópias, um primário e nenhuma dúvida sobre quem manda

Armadilhas e casos reais
Resumo
A Lastro agora
Referências

24 Particionamento e sharding
Particionar sem sair da máquina
Sharding, ou como transformar um banco em vários problemas independentes
Hot spots, ou o hash que espalha as chaves e não a carga
Entre shards: consultas, transações e mudanças de casa
Na Lastro: o tempo particiona, o marketplace fragmenta
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

25 Modelos não relacionais
O padrão de acesso escolhe o modelo, nunca o contrário
Na Lastro: três perguntas, três modelos, um ledger intocado
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

26 Decomposição de sistemas
Três formas de empacotar o mesmo código
Acoplamento, coesão e a pergunta que decide a fronteira
Um banco por serviço, e o que você perde com ele
Quanto custa trocar uma função por uma requisição
Disponibilidade composta, ou por que cinco noves de vizinhos dão menos que um
Lei de Conway, a observação que ninguém revogou
Na Lastro: a fronteira segue a transação
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

27 Balanceamento e proxies
Camada 4 e camada 7, ou quanto da conversa o balanceador tem coragem de ler
Os algoritmos, ou como dividir trabalho sem saber quanto trabalho é
Health checks, ou o exame médico que pode matar o paciente e a família
Sticky sessions, o casamento por conveniência
Reverse proxy, API gateway, service mesh e CDN, o mesmo programa com quatro crachás
Na Lastro: uma porta de entrada, um proxy por máquina e um IP para os parceiros
Armadilhas e casos reais

Resumo
A Lastro agora
Referências

28 Controle de carga
Por que todo sistema tem um limite e quase nenhum sabe qual é
Os algoritmos de limitação, ou quatro jeitos de contar até cem
Limitar com vários gateways, onde o contador mora
A resposta: 429, Retry-After e um corpo que explica
Contrapressão e filas limitadas
Descarte de carga, ou quem sai do bote primeiro
Na Lastro: limites por marketplace e uma ordem de sacrifício
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

29 Escalabilidade e capacidade
Multiplicar tudo por dez, ou a arte de errar a conta com confiança
Vertical, horizontal e o pedágio que cada máquina nova cobra
Stateless, ou onde você escondeu o corpo
Autoscaling, um laço de controle com atraso
A conta do Tuiuiú
O autoscaling da Lastro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

Parte V: Segurança

30 Autenticação
Quem é você, e por que o servidor acredita
Sessão com cookie, a tecnologia que ninguém acha moderna e quase todo mundo deveria usar
Tokens opacos e JWT, ou onde mora a verdade
OAuth 2.0 e OpenID Connect
MFA e passkeys, ou como parar de confiar na memória humana
As credenciais dos marketplaces
O painel dos vendedores
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

31 Autorização e multitenancy

Autorização, a pergunta que vem depois do crachá
RBAC, ou como cinco papéis viram quatrocentos
ABAC e ReBAC, ou a política que olha o objeto
IDOR, o ataque que cabe numa tecla
Isolamento entre tenants
Row-level security, o WHERE que o Carlos não consegue esquecer
Na Lastro: marketplaces, vendedores e quem pode o quê
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

32 Vulnerabilidades de backend
As listas da OWASP, ou o ranking anual das mesmas dez formas de apanhar
SSRF, o servidor que faz o que o atacante pede
Injeção, a mais velha da casa
Deserialização insegura, ou o pickle que executa
Autorização por propriedade e por função
Dependências, o código que você não escreveu e que roda com seus privilégios
Na Lastro: a superfície de ataque revisada
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

33 Criptografia aplicada e segredos
Hash de senha, ou a arte de ser lento de propósito
AES-GCM, ou cifrar sem autenticar é deixar a porta encostada
Assimétrica e assinaturas, ou a chave que pode ir para o outdoor
HMAC, ou como provar que foi você sem mostrar o RG
Envelope encryption e KMS, ou chave dentro de chave dentro de caixa
Segredos, ou onde a senha do banco passa as noites
Tokenização, ou o cartão no programa de proteção a testemunhas
Na Lastro: o cofre, a fronteira e o resto do lixo
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

Parte VI: Infraestrutura e operação

34 Contêineres
Um contêiner é um processo com antolhos
cgroups, ou o kernel cobrando a conta no ato
Imagens e camadas, ou o sistema de arquivos que nunca esquece

Build em múltiplos estágios, ou a obra sem o entulho
PID 1, ou o processo que o kernel não deixa morrer de susto
Registries e a tríade OCI
Na Lastro: imagens, sinais e limites
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

35 Kubernetes
O control plane, ou um banco de dados cercado de laços infinitos
Pods, ou o menor imóvel que o Kubernetes aluga
Deployments e rolling update, ou trocar o pneu com o carro andando e 37 autorizações no banco de trás
Services e Ingress, ou um IP que não pertence a ninguém
Secrets, ou base64 com crachá de criptografia
Requests, limits e a ordem de quem morre primeiro
Probes, ou o médico com licença para matar
HPA, ou o autoscaler do capítulo 29 com outro crachá
Na Lastro: o cluster
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

36 Redes em nuvem e infraestrutura como código
Regiões, zonas e a rede virtual, ou o datacenter que você aluga em fatias
Tabelas de rota, ou onde "pública" e "privada" são decididas
Firewalls com estado e sem estado, ou a diferença entre ter memória e ter regulamento
Infraestrutura como código, ou o fim do "quem clicou nisso?"
Na Lastro: a rede refeita, e o enterro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

37 CI/CD e estratégias de deploy
O pull request que não mudava nada
O pipeline: compile uma vez, promova o mesmo artefato
Credenciais de curta duração: o pipeline sem chave no bolso
Estratégias de deploy: quanto tráfego a versão nova machuca primeiro
Feature flags: deploy é uma coisa, lançamento é outra
Rollback, roll forward e o dinheiro que já foi gravado
Na Lastro
Armadilhas e casos reais

Resumo
A Lastro agora
Referências

38 Observabilidade
O dashboard verde e o Bazar vermelho
Monitorar o que você sabe, observar o que você não sabe
Logs estruturados: o evento com campos
Métricas: contadores, gauges, histogramas e a conta da cardinalidade
Tracing distribuído: a cobrança com rastro
OpenTelemetry: um padrão para os três
RED, USE e a métrica que o negócio sente
Na Lastro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

39 SLI, SLO e SLA
O anexo 3, ou como um nove a mais virou cláusula de multa
Três siglas, três níveis de compromisso
Os noves, ou quanto tempo você pode passar morto por mês
Error budget, ou a licença para quebrar
Alertas por burn rate
Na Lastro: o que medir, quanto prometer
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

40 Backup e recuperação de desastres
O mês em que tudo aconteceu duas vezes
RPO e RTO, ou quanto você perde e quanto você espera
Replicação copia o erro com perfeição
Tipos de backup
Estratégias de recuperação de desastres
Na Lastro
Armadilhas e casos reais
Resumo
A Lastro agora
Referências

Antes do show

Você é sênior, escreve código há anos e sempre recebeu a tarefa mastigada. Agora o backend é seu: o lock que trava o banco na Black Friday, o certificado que expira às 3h, o deploy que erra um centavo em cada cobrança e o cliente que descobre tudo antes do monitoramento.

Este livro ensina o que mora abaixo do framework, do ledger à recuperação de desastres, em 40 capítulos lidos em ordem. O fio condutor é a Lastro, uma fintech fictícia de pagamentos que quebra e é consertada a cada capítulo. Você só lê. O `temp-01`, por enquanto, continua rodando.

PARTE I

O núcleo: dinheiro no banco de dados

CAPÍTULO 1

Modelagem relacional

Um centavo, um sábado e um financeiro que nunca mais foi o mesmo

O financeiro da Lastro achou uma divergência de R$ 0,01 às 23h de um sábado, e desde então ele fala desse centavo com a voz baixa de quem voltou de uma guerra.

Contexto rápido, porque hoje é o seu primeiro dia. A Lastro é uma plataforma de pagamentos para marketplaces: o marketplace cria uma **cobrança** na API, o comprador paga por Pix ou cartão, a Lastro divide o valor entre marketplace e vendedores (o *split*), desconta a tarifa, guarda os saldos e repassa o dinheiro. Um cliente, o Bazar Bem-te-vi: 1.200 vendedores, umas 3.000 cobranças por dia. E você, o novo líder técnico do backend, contratado no dia em que o fundador admitiu que "a gente escala depois" tem data de vencimento.

O fundador escreveu a primeira versão num fim de semana, num servidor chamado `temp-01`. O nome diz "temporário". O *uptime* diz dois anos e três meses.

O saldo de cada vendedor mora numa coluna `saldo double precision`. A cada pagamento, `UPDATE vendedores SET saldo = saldo + 87.01 WHERE id = 42`. A cada saque, o mesmo com o sinal trocado. Não existe histórico, existe um número que muda.

Todo mês, o financeiro compara esses saldos com a planilha de conciliação, a fonte oficial da verdade da Lastro, digam o que disserem. Neste mês, 37 vendedores divergem: 34 por um centavo, ao acaso; três por valores como R$ 412,30, uma venda somada duas vezes por um *retry*. Provar é impossível: a coluna tem a memória de um peixinho dourado e a responsabilidade de um banco.

```
Financeiro: Por que o vendedor 42 tem R$ 412,30 a mais?
Banco:      O saldo dele é 9871.440000000001.
Financeiro: E de onde veio isso?
Banco:      Eu não guardo "de onde". Eu guardo float.
Fundador:   No meu tempo a gente arredondava na tela e pronto.
Carlos:     (já está editando o saldo na mão, em produção)
```

A pergunta deste capítulo: como guardar dinheiro num banco relacional de modo que todo centavo tenha origem conhecida, o saldo seja incapaz de discordar da própria história e o banco recuse dado errado, inclusive quando quem tenta gravar é o Carlos?

Ponto flutuante, ou como errar com dezessete dígitos de convicção

O `double precision` do PostgreSQL e o `float` do Python são o formato *binary64* da IEEE 754: 1 bit de sinal, 11 de expoente e 52 de fração, 53 bits de precisão efetiva, todo valor na forma m × 2^e. Um decimal só tem representação exata se for soma finita de potências de 2. O 0,25 tem. O 0,1, em binário, é a dízima periódica 0,0001100110011001100..., e o hardware guarda o binary64 mais próximo. Quase todo valor com centavos nasce errado. Pouquinho, mas nasce.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> sum([0.1] * 10)
0.9999999999999999
>>> round(2.675, 2)
2.67
```

O último é o mais cruel: o valor guardado é 2,67499999999999982236..., um número que você nunca digitou.

O erro por operação é minúsculo, mas se acumula ao longo de dois anos de somas. E a adição em ponto flutuante perde a associatividade: (a + b) + c pode diferir de a + (b + c). O `SUM` sobre `float8` depende da ordem das linhas, que depende do plano; com agregação paralela, a mesma consulta sobre os mesmos dados pode variar nas últimas casas. O saldo passa a depender de quantos *workers* acordaram de bom humor.

A documentação do PostgreSQL manda usar `numeric` para valores monetários. O ponto flutuante foi desenhado para física e computação gráfica, onde errar na décima sexta casa é ruído e ninguém processa ninguém por um pixel torto.

As alternativas exatas:

- **Inteiro em unidades mínimas**: centavos num `BIGINT`. Oito bytes, aritmética exata, máximo de 9.223.372.036.854.775.807 centavos (uns R$ 92 quatrilhões, o bastante até para o fundador pedir aumento). A escala fica implícita: a coluna diz `10000` e só a convenção diz que isso é R$ 100,00.
- **`NUMERIC(p, s)`**: decimal exato, aritmética em base 10 feita em software. Ocupa 2 bytes por grupo de 4 dígitos mais 3 a 8 de cabeçalho, e cada soma custa bem mais que uma soma de inteiros.

```sql
SELECT 0.1::float8 + 0.2::float8 AS flutuante,
       0.1::numeric + 0.2::numeric AS decimal,
       10 + 20 AS centavos;
--       flutuante      | decimal | centavos
-- --------------------+---------+----------
--  0.30000000000000004 |     0.3 |       30
```

No Python, o par exato do `numeric` é o `decimal.Decimal`, que o psycopg 3 e o SQLAlchemy devolvem por padrão. O `float` aparece quando alguém declara a coluna como `Float` "porque o tutorial fazia assim". O tutorial não tinha um financeiro.

Arredondamento: o centavo órfão precisa de um tutor legal

Tipo exato nenhum salva você da divisão. R$ 100,00 entre três vendedores dão R$ 33,33 para cada um, a soma dá R$ 99,99 e um centavo fica órfão, vagando pelo sistema até ser adotado pela planilha do financeiro.

Meio para cima (*half-up*) leva 2,345 a 2,35; todo empate sobe, e milhões de operações acumulam viés. **Meio para o par** (*half-even*, o "arredondamento bancário") leva 2,345 a 2,34 e 2,355 a 2,36, e o viés some em média. **Truncamento** descarta as casas, com viés sempre na mesma direção; adiante você vê o que isso fez com uma bolsa de valores. E o `round()` do Python usa meio para o par, enquanto o `round(numeric)` do PostgreSQL arredonda empates para longe do zero: dois componentes, duas regras, um centavo por empate.

A regra para dinheiro: calcular exato, arredondar uma única vez, num ponto e com regra definidos, e fechar por diferença: a última parcela recebe o total menos as outras, e a soma bate por construção. Em rateios maiores, o método dos maiores restos dá os centavos que sobram a quem mais perdeu no arredondamento. Arredondar cada parte e torcer para a soma bater é uma loteria de centavos em que o único apostador é o financeiro e o prêmio é um sábado à noite.

Partidas dobradas: dinheiro não nasce nem morre, só muda de conta

O ledger de partidas dobradas foi descrito por Luca Pacioli em 1494. É o padrão mais antigo deste livro: mais de cinco séculos sem migração de schema, e nenhum incidente atribuído a ele, porque o incidente sempre é atribuído a quem implementou.

O modelo tem três entidades. A **conta** é onde dinheiro fica: o disponível do vendedor 42, a comissão do Bazar, a receita de tarifas, o banco liquidante. A **transação** é um evento de negócio (pagamento, estorno, saque). O **lançamento** movimenta um valor com sinal numa conta, dentro de uma transação.

E três invariantes. Toda transação tem pelo menos dois lançamentos, com soma zero. Lançamentos são imutáveis: entram por `INSERT` e nunca saem por `UPDATE` ou `DELETE`. O saldo de uma conta é a soma dos lançamentos dela. Consequência global: a soma de todos os lançamentos do sistema é zero, sempre, e um centavo que surge do nada vira uma consulta em vez de um sábado à noite.

Convenção de sinal: positivo é débito, negativo é crédito. Contas de ativo (o banco liquidante) ficam positivas; de passivo (o que a Lastro deve a vendedores e marketplace) e de receita, negativas. Sim, no razão o vendedor mais rico do Bazar tem o saldo mais negativo da empresa. Contabilidade é a única área em que dever dinheiro a alguém se chama crédito e ninguém acha estranho. O extrato do vendedor inverte o sinal, por caridade.

Um pagamento Pix de R$ 100,00 no Bazar, com comissão de 12% e tarifa Pix da Lastro de 0,99% descontada do vendedor:

```
Transação 7781 · pagamento Pix · cobrança 01J9ZB4Q...
conta                          valor (centavos)
banco_liquidante                        +10000
vendedor_42:disponivel                   -8701
bazar:comissao                           -1200
lastro:receita_tarifas                     -99
                                 soma:       0
```

A parcela do vendedor foi fechada por diferença (10000 − 1200 − 99), e a soma zero vem de graça. Um estorno vira uma transação nova, com cada lançamento invertido e referência à original. O passado da Lastro fica imutável, igual ao do Carlos, só que documentado.

```
Carlos: Esse lançamento tá errado. Vou dar um UPDATE.
Ledger: Aqui não tem UPDATE.
Carlos: DELETE, então.
Ledger: Também não.
Carlos: E como eu conserto?
Ledger: Com um lançamento novo que desfaz o antigo. Com data e motivo.
Carlos: Isso é praticamente uma confissão assinada.
Ledger: Agora você entendeu o modelo.
```

De brinde, o saldo de qualquer instante passado vira um `SUM` com filtro de data, e um pagamento duplicado aparece como duas transações da mesma cobrança.

Chaves, ou quem é quem quando todo mundo tem dois documentos

Uma **chave candidata** é um conjunto mínimo de atributos que identifica uma linha; a primária é a escolhida, as demais viram `UNIQUE`.

Uma **chave natural** tem significado fora do banco: CNPJ, CPF, o EndToEndId do Pix. Como `UNIQUE`, ela impede duplicar a coisa real. Como chave primária, ela amarra o seu schema às regras de outra pessoa. A Instrução Normativa RFB nº 2.229, de 2024, deu letras aos CNPJs emitidos a partir de julho de 2026. Todo sistema que usou CNPJ em `BIGINT` como chave em quarenta tabelas ganhou, por decreto, uma migração de quarenta tabelas.

Uma **chave substituta** é gerada pelo banco, sem significado, e nunca muda. Duas famílias:

- **Sequência** (`GENERATED ALWAYS AS IDENTITY`): 8 bytes, crescente, cada inserção no fim do índice B-tree (capítulo 5). O `nextval` nunca é desfeito por *rollback*, então há buracos, e o ID exposto conta segredos. Se a cobrança de hoje é a 310.200 e a de amanhã é a 313.180, qualquer concorrente sabe que você fez 2.980 cobranças no intervalo, sem nem precisar subornar o Carlos.
- **UUID**: 128 bits, gerado em qualquer lugar sem coordenação. A versão 4 tem 122 bits aleatórios, e cada inserção cai numa página aleatória do índice: mais páginas sujas, mais escrita no WAL. A versão 7 (RFC 9562) começa com 48 bits de *timestamp* Unix em milissegundos, então as inserções voltam para o fim do índice, ao custo de revelar o instante de criação. O PostgreSQL ganhou `uuidv7()` na versão 18.

Entidade	Chave primária	Outras chaves	Motivo
Contas, transações, lançamentos	`BIGINT` identity	Nenhuma	Internas; maior volume, 8 bytes por entrada de índice e FK
Cobranças	UUIDv7, também o ID público	Nenhuma por ora	Exposto na API sem revelar volume; ordenado por tempo
Marketplaces, vendedores	`BIGINT` identity	CNPJ/CPF `UNIQUE`, em `TEXT`	Documento muda de formato
Transações Pix	Ligadas à cobrança	EndToEndId `UNIQUE`	Mesmo Pix registrado uma vez só (capítulo 16)

UUID em tudo seria a escolha certa com vários bancos ou clientes *offline* gerando IDs sem coordenação. Chave natural primária serve quando o valor é estável e seu, o que exclui qualquer coisa emitida pela Receita Federal.

Formas normais: cada fato num lugar só, que nem segredo de família

Quando o mesmo fato mora em vários lugares, aparecem as **anomalias**: **de atualização** (muda uma cópia, as outras ficam velhas), **de inserção** (só dá para registrar um fato inventando outro) e **de exclusão** (apagar um fato leva junto outro). A coluna `saldo` do `temp-01` é a anomalia de atualização em estado puro: a Lastro guardava a cópia e jogava fora o original.

As formas normais são condições sobre **dependências funcionais**: X → Y quando o valor de X determina o valor de Y.

- **1FN**: atributos atômicos, sem grupos repetidos. Uma coluna `recebedores` com `42:8701;77:1200` viola: é um CSV escondido numa célula, esperando um vendedor com ponto e vírgula no nome.
- **2FN**: 1FN, e nenhum atributo fora da chave depende de só parte de uma chave composta. Com chave (`cobranca_id`, `vendedor_id`), uma coluna `vendedor_nome` viola. O vendedor muda o nome fantasia e 40 mil linhas mentem.
- **3FN**: 2FN, e nenhum atributo fora da chave depende de outro atributo fora da chave. Uma `tarifa_pix_do_marketplace` na tabela de cobranças viola: depende do marketplace.
- **BCNF** (Boyce-Codd): para toda dependência não trivial X → Y, X é superchave. É mais estrita que a 3FN.

Sutileza que vale dinheiro: a tarifa *contratada* depende do marketplace, mas a tarifa *aplicada* no dia do pagamento é um fato da cobrança e fica gravada nela. O contrato muda em março; a cobrança de fevereiro continua tendo pago 0,99%. Valor histórico copiado registra outro fato.

3FN e BCNF divergem com chaves candidatas sobrepostas. Na tabela (`marketplace`, `bandeira`, `contrato`), {`marketplace`, `bandeira`} → `contrato` e `contrato` → `bandeira`. Todo atributo pertence a uma chave candidata, então está na 3FN; `contrato` falha em ser superchave, então viola a BCNF, e "o contrato C é da Visa" se repete por marketplace. Decompor em (`contrato`, `bandeira`) e (`marketplace`, `contrato`) elimina a repetição e perde a garantia de um contrato por bandeira por marketplace, que deixa de caber num `UNIQUE`. A 3FN sempre admite decomposição que preserva as dependências; a BCNF nem sempre. Ela entrega a pureza prometida e cobra em restrição perdida, igual consultor por hora.

Desnormalização: toda cópia é uma promessa, e promessa se quebra

Desnormalizar é guardar de propósito um dado derivável, para ler mais rápido. A conta chega na escrita: cada mudança na origem atualiza a cópia na mesma transação, ou um processo corrige a diferença depois. Cópia sem esse mecanismo é a coluna `saldo` do `temp-01` com outro nome e a mesma ficha criminal.

Estratégia	Custo da leitura	Custo da escrita	Risco
Saldo derivado (`SUM`)	Todos os lançamentos da conta	Só `INSERT`	Lento com volume (capítulo 5)
Materializado na mesma transação	Uma linha	`INSERT` + `UPDATE` da conta	Contenção na linha (capítulo 4); bug grava cópia errada
Projeção assíncrona	Uma linha	`INSERT` + consumidor	Leitura atrasada (capítulos 19 e 21)

A Lastro fica com o saldo derivado. Quem decide é a conciliação: o saldo precisa ser explicável lançamento por lançamento, e um valor derivado é incapaz de divergir do histórico porque ele é o histórico somado. O volume permite: uns 12 mil lançamentos por dia, e a conta mais movimentada, a comissão do Bazar, recebe cerca de 1,1 milhão por ano. O materializado vira a escolha certa quando ler o saldo vira gargalo mesmo com índices, ou quando uma verificação precisa travar o saldo numa linha só. Mesmo aí ele é cache: se discordar do `SUM`, o `SUM` ganha e a cópia vai para o paredão.

Restrições: o banco como último adulto na sala

Regra validada só no Python protege só as escritas que passam pelo Python. Na Lastro, escrevem no banco a API, o job `repasse_final_v2_agora_vai.py`, as migrações e o Carlos, com o `psql` aberto e a confiança de quem nunca foi demitido. Restrição declarada no banco vale para os quatro.

`NOT NULL` e `CHECK` validam uma linha por vez (`valor <> 0`). Chaves estrangeiras com `ON DELETE RESTRICT` impedem lançamento órfão e conta com histórico apagada. `UNIQUE` barra o EndToEndId repetido.

A soma zero envolve várias linhas, e o `CHECK` enxerga uma só. Uma saída é um *constraint trigger* `DEFERRABLE INITIALLY DEFERRED`: no `COMMIT`, soma os lançamentos de cada transação tocada e, se a soma der diferente de zero, levanta erro e desfaz tudo, ao custo de uma consulta a mais por *commit*. A outra é uma função SQL de escrita que valida antes de inserir, e o Carlos vai achar o caminho que não passa por ela.

A imutabilidade usa duas camadas: o papel (*role*) da aplicação recebe só `INSERT` e `SELECT` nos lançamentos, e um *trigger* `BEFORE UPDATE OR DELETE` recusa a operação até para o dono da tabela. Contra um superusuário, que pode desligar *triggers*, nada resolve; o fundador tem esse acesso, e o capítulo 31 trata de quem pode o quê.

```
lastro=> INSERT INTO lancamentos (transacao_id, conta_id, valor) VALUES (7781, 42, 0);
ERROR:  new row for relation "lancamentos" violates check constraint
"lancamentos_valor_check"
lastro=> UPDATE lancamentos SET valor = -870100 WHERE id = 991203;
ERROR:  lancamentos: append-only, registre um estorno
```

A primeira mensagem é do PostgreSQL; a segunda, do *trigger*. O `-870100` é o Carlos "corrigindo" R$ 87,01 com dois zeros a mais, e o banco salvou a Lastro de dar R$ 8.701,00 de presente a um vendedor.

Armadilhas e casos reais

> **Na vida real:** a Bolsa de Valores de Vancouver lançou um índice em janeiro de 1982, valendo 1.000, recalculado a cada negociação e truncado em três casas, com o valor truncado alimentando o cálculo seguinte. Em novembro de 1983 ele fechou uma sexta em 524,811; recalculado corretamente no fim de semana, abriu em 1.098,892. O mercado dobrou sem uma única negociação, a melhor semana da história para quem não fez nada.
>
> **Armadilha:** a RFC 8259 avisa que implementações de JSON costumam usar binary64, e o JavaScript usa: acima de 2^53, inteiros perdem precisão (`9007199254740993` chega como `9007199254740992`), e `87.01` cai no ponto flutuante do outro lado. Valor monetário trafega como inteiro em centavos ou string, senão o `Number` do JavaScript repete o erro do `temp-01` no navegador do comprador.
>
> **Armadilha:** `NUMERIC(12, 2)` recebe `10.005` e grava `10.01`, sem aviso. Arredondamento silencioso: a gentileza que o financeiro chama de fraude.
>
> **Armadilha:** sequência tem buraco. O auditor pergunta pelo lançamento 1.042, e a resposta honesta é "morreu num *rollback* de terça-feira".

Resumo

- Ponto flutuante erra decimais, acumula o erro e perde a associatividade; dinheiro vai em inteiro de unidades mínimas ou `NUMERIC`.
- Arredondar uma vez, com regra definida, e fechar a última parcela por diferença.
- Ledger: lançamentos imutáveis com soma zero por transação, saldo derivado, estorno como transação nova.
- Chave natural traz regras de fora; sequência delata volume; UUIDv7 ordena por tempo.
- 1FN, 2FN, 3FN e BCNF eliminam anomalias; a BCNF pode perder dependências.
- Desnormalizar é assinar um contrato de manutenção de cópia.
- Restrição no banco vale até para o Carlos.

A Lastro agora

A coluna `saldo double precision` morreu, e ninguém foi ao velório. O dinheiro da Lastro mora num ledger de partidas dobradas no PostgreSQL: lançamentos em centavos `BIGINT`, soma zero por *constraint trigger* adiado, imutabilidade por privilégio e *trigger*, estorno como transação nova, saldo derivado por `SUM`. Taxas em `NUMERIC` com quatro casas, tarifa arredondada uma vez (meio para o par), parcela do vendedor fechada por diferença, tarifa aplicada gravada na cobrança. Cobranças com UUIDv7 público; o resto com `BIGINT` identity.

Na conciliação seguinte, planilha e ledger bateram até o último centavo, e o financeiro dormiu num sábado pela primeira vez em dois anos. Durou até quarta. Um vendedor com R$ 500,00 de saldo disparou dois saques de R$ 500,00 no mesmo milissegundo, e cada um gerou uma transação perfeita, soma zero, registrada com a precisão de um tabelião. Partida dobrada, saque dobrado, e o Bazar já abriu o chamado: "URGENTE!!!!".

Referências

- IEEE. *IEEE Standard for Floating-Point Arithmetic* (IEEE 754-2019). https://standards.ieee.org/ieee/754/6210/
- GOLDBERG, David. *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys, 1991. https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html
- Python. *decimal: Decimal fixed-point and floating-point arithmetic*. https://docs.python.org/3/library/decimal.html

- PostgreSQL. *Numeric Types*. https://www.postgresql.org/docs/current/datatype-numeric.html
- PostgreSQL. *Mathematical Functions and Operators* (`round`). https://www.postgresql.org/docs/current/functions-math.html
- PostgreSQL. *CREATE TRIGGER* (constraint triggers). https://www.postgresql.org/docs/current/sql-createtrigger.html
- RFC 9562. *Universally Unique IDentifiers (UUIDs)*. https://www.rfc-editor.org/rfc/rfc9562
- RFC 8259. *The JavaScript Object Notation (JSON) Data Interchange Format*. https://www.rfc-editor.org/rfc/rfc8259
- CODD, E. F. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 1970. https://doi.org/10.1145/362384.362685
- Receita Federal. Instrução Normativa RFB nº 2.229, de 15 de outubro de 2024 (CNPJ alfanumérico).
- QUINN, Kevin. *Ever Had Problems Rounding Off Figures? This Stock Exchange Has*. The Wall Street Journal, 8 nov. 1983, p. 37 (caso do índice da Bolsa de Vancouver).

CAPÍTULO 2

Concorrência na aplicação

Dois saques, um saldo e um vendedor com duas abas abertas

O vendedor 42 tinha R$ 500,00 de saldo disponível, duas abas do painel abertas e o dedo mais rápido do Bazar Bem-te-vi. Clicou em "Sacar R$ 500,00" nas duas, e a Lastro, com a honestidade contábil do capítulo 1, registrou dois saques perfeitos, soma zero em cada um, dignos de tabelião. O banco parceiro mandou R$ 1.000,00 por Pix, e o saldo do vendedor ficou em R$ −500,00, que o ledger guarda com o mesmo carinho dos positivos.

O código do saque fazia o que qualquer pessoa sensata faria:

```
1. saldo = SUM dos lançamentos da conta disponível do vendedor
2. se saldo >= valor: insere a transação de saque
3. senão: recusa
```

Três linhas e nenhum bug, desde que só exista um clique no universo. As duas requisições chegaram com 3 ms de diferença, as duas leram R$ 500,00 e as duas estavam certas no momento em que olharam.

O Carlos corrigiu em vinte minutos com um lock em memória por conta. Nos testes, perfeito. Na semana seguinte, o vendedor 42, que tinha descoberto um hobby, repetiu a façanha. Por que o lock funcionou no notebook do Carlos e falhou em `temp-01`, e onde a verificação de saldo precisa morar para que dois cliques nunca virem dois saques?

Condição de corrida, o bug que só aparece quando você não está olhando

Uma **condição de corrida** (*race condition*) acontece quando o resultado depende da ordem em que execuções concorrentes intercalam seus passos, e alguma dessas ordens produz um resultado errado. Cada execução, sozinha, está correta. O erro mora entre elas, numa ordem que nenhum teste unitário produz, porque teste unitário faz uma coisa de cada vez, igual ao dev que escreveu.

O padrão do saque é o **check-then-act**: verificar uma condição (saldo suficiente) e agir com base nela (debitar). Entre as duas coisas existe um intervalo, e a condição pode deixar de valer dentro dele. Em segurança, o nome é TOCTOU (*time of check to time of use*, CWE-367), porque atacante adora um intervalo.

```
tempo   requisição A                requisição B
t0      lê saldo: 50000
t1                                  lê saldo: 50000
t2      50000 >= 50000, debita
t3                                  50000 >= 50000, debita
t4      commit                      commit           saldo final: -50000
```

O primo é o **read-modify-write**: duas execuções leem 10, calculam 11, gravam 11, e um incremento some sem deixar bilhete. É o *lost update*, que volta no capítulo 3 de roupa de gala.

Agora o bit favorito de quem programa em Python: "mas tem o GIL". Tem. O GIL (*global interpreter lock*) deixa uma thread por vez executar bytecode no CPython e protege as estruturas internas do interpretador. O seu saldo fica fora dessa proteção, por mais que o fundador trate `temp-01` como extensão do próprio corpo. Um `contador += 1` vira várias instruções de bytecode, e a troca de thread pode cair entre elas. E o Python 3.13 trouxe o build *free-threaded* (PEP 703), sem GIL, para quem achava que concorrência estava fácil demais.

O `asyncio` tem a sua versão do crime, numa thread só. Uma corrotina cede o controle em cada `await`. O saque fazia `await` na consulta do saldo e `await` na inserção, e entre os dois o event loop atendeu a outra aba do vendedor 42. Zero paralelismo e mesmo assim uma corrida: o equivalente técnico de bater o carro sozinho num estacionamento vazio.

E corridas dependem de *timing*: um `print` muda o tempo e a corrida some; você tira o `print` e ela volta. O mercado chama isso de *heisenbug*, e o plantão chama de "não consigo reproduzir", frase que já encerrou mais chamados do que qualquer correção.

Seção crítica e mutex, ou ciúme com suporte do sistema operacional

O trecho que acessa um estado compartilhado e não pode ser intercalado com outro acesso ao mesmo estado é a **seção crítica**; no saque, vai da leitura do saldo à gravação do débito. A solução clássica é a **exclusão mútua**: uma execução por vez lá dentro.

O instrumento é o **mutex** (*mutual exclusion lock*, ou só *lock*), com duas operações: adquirir (`acquire`) e liberar (`release`). Quem tenta adquirir um lock ocupado espera o dono liberar. Um lock correto garante segurança (*safety*: nunca duas execuções dentro da seção crítica) e vivacidade (*liveness*: se ninguém está dentro, quem quer entrar entra). Falhar na primeira dá saque duplo; falhar na segunda dá um sistema que nunca erra porque nunca faz nada, o sonho de todo auditor.

O lock só protege se **todo** acesso ao estado passar por ele. Basta um caminho sem lock para tudo desabar, e esse caminho sempre existe: chama-se "o script de correção que o Carlos rodou direto no banco".

O custo é a serialização: a seção crítica é parte serial por definição, e a Lei de Amdahl (capítulo 14) diz que a parte serial limita qualquer paralelismo. Daí o lock fino, um por conta. Um lock global poria os 1.200 vendedores do Bazar sacando em fila indiana, o que resolve a corrida e cria uma reclamação por minuto.

Operações atômicas: o check e o act no mesmo instante

Uma operação é **atômica** quando ninguém observa um estado intermediário dela. Fazer o check e o act numa única operação atômica elimina o intervalo, e a corrida morre de fome.

O processador oferece isso em hardware. **Compare-and-swap** (CAS) grava um valor novo num endereço só se ele ainda contém o valor esperado, numa instrução só (`CMPXCHG` no x86). Com CAS se constroem contadores, filas e o próprio mutex sem ninguém esperar ninguém, técnica chamada *lock-free*, que funciona perfeitamente até você descobrir o problema ABA e passar a dormir de luz acesa.

Servidores de dados oferecem a mesma ideia mais acima. No Redis, `INCR` é atômico. No SQL, um `UPDATE` com a condição no `WHERE` é um compare-and-set sobre uma linha:

```
UPDATE saldos SET disponivel = disponivel - 50000
 WHERE conta_id = 42 AND disponivel >= 50000;
-- primeira sessão:  UPDATE 1
-- segunda sessão:   UPDATE 0   (esperou a primeira, reavaliou, recusou)
```

No PostgreSQL, em *read committed* (o nível padrão), a segunda sessão espera o lock da linha e, depois do *commit* da primeira, reavalia o `WHERE` contra a versão nova. Vê 0 e não grava nada. O `UPDATE 0` é a forma mais educada que um banco tem de dizer "seu vendedor é um golpista". Repare que isso exige uma linha com o saldo, e a Lastro, desde o capítulo 1, deriva o saldo dos lançamentos.

Deadlock e as quatro condições de Coffman

Quem usa lock descobre o **deadlock**: execuções esperando umas pelas outras, para sempre. A tem o lock da conta 7 e quer o da 9; B tem o da 9 e quer o da 7.

```
A: Me passa a conta 9?
B: Passo, assim que você me passar a 7.
A: Passo, assim que terminar. E só termino com a 9.
B: Então ficamos assim.
(eternidade)
Bazar: URGENTE!!!!
```

Consistência perfeita: ninguém escreve nada errado porque ninguém escreve nada.

Coffman, Elphick e Shoshani mostraram em 1971 que um deadlock só acontece se quatro condições valerem ao mesmo tempo:

1. **Exclusão mútua**: o recurso só pode ser usado por um de cada vez.
2. **Posse e espera** (*hold and wait*): quem tem um recurso pode esperar por outro sem soltar o primeiro.
3. **Sem preempção**: ninguém tira um recurso de quem o tem; só o dono libera.
4. **Espera circular**: existe um ciclo de execuções, cada uma esperando um recurso que a próxima tem.

Quebrar qualquer uma elimina o deadlock. A mais usada é a quarta, com **ordem global de aquisição**: se todos adquirem locks em ordem crescente de ID, ninguém segura a 9 esperando a 7. Quebrar a segunda é adquirir tudo de uma vez. Quebrar a terceira é *timeout*: quem espera demais solta tudo e tenta de novo, e aí nasce o *livelock*, todo mundo desistindo e tentando no mesmo ritmo, muita atividade e nenhum progresso, igual a uma reunião de planejamento.

O PostgreSQL ataca depois. Quem espera um lock por mais que `deadlock_timeout` (1 segundo por padrão) dispara a busca por ciclos no grafo de espera; o servidor escolhe uma vítima e aborta a transação dela, sem direito a últimas palavras:

```
ERROR:  deadlock detected
DETAIL:  Process 4120 waits for ShareLock on transaction 91822; blocked by process 4188.
Process 4188 waits for ShareLock on transaction 91821; blocked by process 4120.
```

A aplicação recebe o erro e repete a transação. O capítulo 4 volta aos deadlocks no banco.

O lock do Carlos e os processos que não se conheciam

A correção do Carlos era um dicionário de `asyncio.Lock`, um por conta, na memória do processo. No teste, um processo, um lock, os dois saques em fila. Correto.

Só que `temp-01` roda o monólito em três processos, porque o fundador configurou `--workers 3` há dois anos "para aproveitar os núcleos" (o capítulo 13 explica esses *workers*). Cada processo tem a própria memória, logo o próprio dicionário, logo o próprio lock da conta 42. Com as conexões distribuídas por igual entre três processos, a chance de os dois saques caírem em processos diferentes é de 2/3. O lock do Carlos funcionava em um terço dos casos. Tecnicamente é um lock. Juridicamente é um problema.

Cada primitiva de exclusão mútua tem um escopo, e fora dele ela não existe:

Primitiva	Exclui	Não enxerga
`asyncio.Lock`	corrotinas do mesmo event loop	outras threads, outros processos
`threading.Lock`	threads do mesmo processo	outros processos
`multiprocessing.Lock`	processos que herdaram o lock do mesmo pai, na mesma máquina	outras máquinas
`flock` (arquivo local)	processos da mesma máquina	outras máquinas
lock no banco de dados	qualquer sessão no mesmo banco	quem escreve sem pegar o lock
lock distribuído (Redis, etcd, ZooKeeper)	clientes do serviço de lock	o recurso protegido, que nem sabe que o lock existe

E a primitiva errada cobra caro: um `threading.Lock` em código `async` trava o event loop inteiro enquanto espera, e um `asyncio.Lock` ignora outras threads. Deadlock ou corrida, você escolhe.

Três processos numa máquina ainda são o caso fácil. Quando existir `temp-02` atrás de um balanceador (capítulo 27; o fundador já falou em "duplicar a máquina pra Black Friday"), nem o `multiprocessing.Lock` alcança. Réplicas em máquinas diferentes só compartilham o que está fora delas: o banco ou um serviço de lock.

Locks distribuídos, ou o mutex com prazo de validade

Um **lock distribuído** é exclusão mútua entre processos em máquinas diferentes, coordenada por um serviço externo. O exemplo mais comum usa o Redis:

```
SET lock:conta:42 9f3c1a77 NX PX 10000
OK          (primeiro cliente: adquiriu)
SET lock:conta:42 b81e0d42 NX PX 10000
(nil)       (segundo cliente: a chave já existe)
```

`NX` só grava se a chave não existir, o que torna a aquisição atômica. O valor aleatório identifica o dono, e a liberação apaga a chave só se o valor ainda for o seu. `PX 10000` dá 10 segundos de vida à chave, e é aqui que a ciência acaba e a fé começa.

O TTL existe porque processos morrem segurando lock, e sem ele a vítima do OOM killer deixaria a conta 42 travada até alguém apagar a chave na mão. Com ele, o lock vira um **lease**: posse temporária, que expira sozinha. E aí vem a falha de todo lock baseado em tempo:

```
t=0s    processo A adquire o lease (TTL 10s)
t=1s    A entra numa pausa de GC de 12 s (ou swap, ou migração da VM)
t=10s   lease expira; processo B adquire e debita a conta 42
t=13s   A acorda, ainda convencido de que é o dono, e debita a conta 42
```

O processo A morreu aos 10 segundos e ninguém avisou. Acorda zumbi, com acesso de escrita, e faz o que zumbis com acesso de escrita fazem. Nenhum TTL resolve: pausas de processo não têm limite superior garantido.

Martin Kleppmann descreveu o problema em 2016, ao analisar o Redlock (o algoritmo sobre vários nós Redis proposto pelo criador do Redis): lock baseado em lease serve para **eficiência** (evitar trabalho duplicado que só custa CPU) e, sozinho, falha para **correção** (impedir um efeito que não pode acontecer duas vezes). Para correção, o recurso protegido precisa recusar o dono antigo com um *fencing token*, número crescente a cada aquisição que o recurso confere. O capítulo 20 constrói isso para o `repasse_final_v2_agora_vai.py`, que vai precisar.

E tem a replicação: um Redis com réplica assíncrona pode confirmar o `SET` e cair antes de replicar. O lock foi replicado em pensamento. etcd e ZooKeeper, baseados em consenso, evitam essa perda e fornecem números monotônicos para o fencing, ao custo de mais uma peça com quórum. E o PostgreSQL, onde mora o dinheiro, continua sem fazer ideia de quem o Redis acha que é o dono da conta 42.

Na Lastro: o lock vai morar com o dinheiro

O requisito é uma invariante: o saldo disponível de um vendedor nunca fica negativo por causa de um débito, venha ele de qualquer processo, de qualquer máquina ou de qualquer script do Carlos.

Opção	Garante	Custo	Serve quando
Lock em memória	exclusão num processo	falha com 2+ processos	um processo, para sempre
Lock distribuído (Redis)	exclusão enquanto o lease vale	peça nova; sem fencing, o PostgreSQL ignora	evitar trabalho duplicado
`pg_advisory_xact_lock(conta_id)`	exclusão no banco até o fim da transação	chave numérica por convenção	recurso sem linha própria
`UPDATE` condicional em saldo materializado	check e act num comando	segunda fonte da verdade	saldo materializado já existe
`FOR UPDATE` na conta, `SUM` e débito na mesma transação	exclusão por conta até `COMMIT` ou `ROLLBACK`	fila por conta; `SUM` cresce	saldo derivado, como na Lastro

A Lastro escolhe a última, porque o lock passa a viver no mesmo sistema que o dado, dentro da transação que grava o débito. Se o processo dorme segurando o lock, a transação continua aberta e a conta continua travada: o lock pertence à transação, e relógio nenhum o expira. Se o processo morre, a conexão cai, o PostgreSQL faz *rollback* e solta o lock. Sem lease, não existe dono antigo para recusar. E o saldo continua derivado, como o capítulo 1 exigiu pela conciliação explicável; o `UPDATE` condicional espera o dia em que houver saldo materializado.

O fluxo do saque fica assim:

```
BEGIN                                    (read committed)
  SELECT ... FROM contas WHERE id = :conta FOR UPDATE   -- espera aqui, se preciso
  SELECT SUM(valor) FROM lancamentos WHERE conta_id = :conta
  saldo suficiente?  não -> ROLLBACK, saque recusado
  INSERT transação de saque + 2 lançamentos (soma zero)
COMMIT                                   (lock liberado)
depois do COMMIT: pedido de Pix ao banco parceiro
```

O segundo saque do vendedor 42 espera uns 4 ms no `FOR UPDATE`, recebe o lock depois do *commit* do primeiro, e o `SUM` dele roda num comando novo, com *snapshot* novo, que já enxerga o débito. Saldo zero, saque recusado, hobby encerrado.

Falta garantir que todo débito passe pelo `FOR UPDATE`, e caminho alternativo é a especialidade do Carlos. Por isso a verificação sai do código Python e vai para o banco: um *trigger* `BEFORE INSERT` nos lançamentos que debitam uma conta disponível trava a linha da conta, recalcula o saldo e recusa o lançamento que o deixaria negativo. Monólito, job ou `psql` do Carlos, todos passam por ele. Créditos dispensam o lock, porque só aumentam o saldo. Ao Python sobrou traduzir o erro em "saldo insuficiente".

Duas regras vêm junto. Pela quarta condição de Coffman, uma transação que debita várias contas disponíveis (um estorno que debita o vendedor e a comissão do Bazar) insere os lançamentos em ordem crescente de `conta_id`. E o pedido de Pix ao banco parceiro sai depois do `COMMIT`, porque centenas de milissegundos de rede dentro da seção crítica fariam todo saque da conta esperar a rede de outra empresa. Se esse pedido falha depois do débito, os capítulos 15 e 18 assumem.

Os números: cerca de 400 saques por dia no Bazar, quase nunca dois da mesma conta no mesmo segundo, lock segurado por uns 4 ms. A fila por conta vive vazia. Uma conta debitada centenas de vezes por segundo encheria essa fila, e o capítulo 4 trata dessa conta.

Armadilhas e casos reais

> **Armadilha:** a mesma receita falha em *repeatable read*. Nesse nível, o `SUM` usa o snapshot do início da transação, e o `FOR UPDATE` numa linha só travada (sem alteração) pela outra transação passa sem erro: o segundo saque espera, recebe o lock e soma um saldo velho. A corrida volta fantasiada de correção. O capítulo 3 explica por quê.
>
> **Armadilha:** o dicionário do Carlos criava um `asyncio.Lock` por conta e nunca apagava nenhum. Além de falhar entre processos, vazava memória no formato exato do cadastro de vendedores. Um bug que falha e ainda paga aluguel.

Teste de uma requisição por vez nunca acha corrida; o de concorrência dispara tudo junto, muitas vezes, e confere a invariante, porque a corrida de uma em mil aparece todo dia em produção.

> **Na vida real:** em março de 2014, a Flexcoin, que se apresentava como banco de bitcoin, perdeu 896 BTC e fechou. Segundo a empresa, o atacante abriu uma conta e disparou milhares de transferências simultâneas entre usuários; o saldo era verificado antes de ser atualizado, e todas passaram pela mesma verificação. Na

> mesma semana, a Poloniex relatou ataque semelhante nos saques e perdeu 12,3% dos bitcoins que custodiava. O vendedor 42 fez a mesma coisa com duas abas; o atacante só tinha mais abas.

Resumo

- Condição de corrida: resultado dependente da intercalação; check-then-act e read-modify-write são as formas comuns.
- O GIL deixa `+=` sem atomicidade; no `asyncio`, cada `await` é ponto de intercalação.
- Seção crítica pede exclusão mútua, e o lock só protege se todo acesso passar por ele.
- Operações atômicas (CAS, `INCR`, `UPDATE` condicional) eliminam o intervalo entre verificar e agir.
- Deadlock exige as quatro condições de Coffman; ordem de aquisição quebra a espera circular.
- Toda primitiva de lock tem escopo; lock em memória fica preso ao processo.
- Lock distribuído com TTL é lease; correção exige fencing token.
- Lock no banco, na transação que grava o dado, dispensa lease e fencing.

A Lastro agora

O saque trava a linha da conta disponível com `SELECT ... FOR UPDATE`, recalcula o saldo pelo `SUM` e debita na mesma transação, em *read committed*; um trigger impõe a verificação a qualquer caminho de escrita, débitos em várias contas seguem a ordem de `conta_id`, e o Pix sai só depois do commit. O lock em memória foi apagado, e os três processos de `temp-01` agora podem discordar de tudo, menos do saldo.

O vendedor 42 tentou com três abas e recebeu "saldo insuficiente" em duas. Saque único, partida dobrada, do jeito que Pacioli queria. Aí o gerente de produto pediu um limite diário de saque ("mas é só somar os saques do dia, não?"), alguém sugeriu *repeatable read* "por segurança", e o vendedor 42 abriu a quarta aba.

Referências

- Coffman, E. G.; Elphick, M.; Shoshani, A. *System Deadlocks*. ACM Computing Surveys 3(2), 1971. https://doi.org/10.1145/356586.356588
- PostgreSQL. *Explicit Locking*. https://www.postgresql.org/docs/current/explicit-locking.html

- PostgreSQL. *Transaction Isolation*. https://www.postgresql.org/docs/current/transaction-iso.html
- PostgreSQL. *deadlock_timeout*. https://www.postgresql.org/docs/current/runtime-config-locks.html
- Python. *threading: Lock Objects*. https://docs.python.org/3/library/threading.html#lock-objects
- Python. *asyncio: Synchronization Primitives*. https://docs.python.org/3/library/asyncio-sync.html
- PEP 703. *Making the Global Interpreter Lock Optional in CPython*. https://peps.python.org/pep-0703/
- MITRE. *CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition*. https://cwe.mitre.org/data/definitions/367.html
- Redis. *Distributed Locks with Redis*. https://redis.io/docs/latest/develop/use/patterns/distributed-locks/
- Kleppmann, M. *How to do distributed locking*, 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Computerworld. *Withdrawal vulnerabilities enabled bitcoin theft from Flexcoin and Poloniex*, 2014. https://www.computerworld.com/article/1383775/withdrawal-vulnerabilities-enabled-bitcoin-theft-from-flexcoin-and-poloniex.html

CAPÍTULO 3

Transações e isolamento

A quarta aba

Segunda-feira, 9h12, e o vendedor 42 voltou ao hobby, agora com quatro abas, porque ele aprende rápido e a Lastro aprende por chamado. Saldo disponível de R$ 6.000,00, R$ 1.000,00 já sacados no dia, e o limite diário novo do gerente de produto ("mas é só somar os saques do dia, não?") é de R$ 5.000,00. Duas abas pediram R$ 4.000,00 no mesmo segundo. As duas passaram.

Resultado: R$ 9.000,00 sacados num dia com limite de R$ 5.000,00 e saldo de R$ −2.000,00. O saldo negativo que o capítulo 2 tinha enterrado saiu do caixão de terno, gravata e chave Pix cadastrada.

O *pull request* subiu numa sexta, claro. Ele somava os saques do dia dentro do *trigger* que já verificava o saldo e trazia uma linha do Carlos trocando o isolamento para *repeatable read*, com o comentário `# por segurança`. Nenhum teste falhou, porque nenhum teste abre duas abas. O Bazar abriu o chamado às 9h40: "URGENTE!!!! VENDEDOR COM SALDO NEGATIVO DE NOVO". O "de novo" doeu mais que as exclamações.

O financeiro, que não dorme desde o capítulo 1, emendou: "e se o servidor cair no meio de um split? O vendedor recebe e o Bazar não?". O que garante que um pagamento que escreve em quatro contas nunca fica pela metade? E como um isolamento mais rígido quebrou o que o mais frouxo protegia?

Uma transação, quatro contas e zero meio-termo

Um Pix de R$ 100,00 no Bazar vira uma transação do ledger com quatro lançamentos (sinal do capítulo 1: positivo é débito, negativo é crédito):

```
conta                                valor (centavos)
banco liquidante (ativo)                   +10000
vendedor 42, disponível                     -8701
comissão do Bazar (12%)                     -1200
receita de tarifas da Lastro (0,99%)          -99
soma                                            0
```

Os R$ 87,01 do vendedor voltam, desta vez sem os dois zeros a mais do Carlos. "Transação" agora tem dois sentidos, a linha em `transacoes` e a transação do banco, e a primeira só existe direito se couber inteira na segunda.

Uma **transação** de banco de dados é um grupo de operações com as garantias da sigla ACID (Härder e Reuter, 1983, quando o maior risco de um banco era alguém tropeçar no cabo).

Atomicidade: ou todas as escritas valem, ou nenhuma. No PostgreSQL, cada linha gravada carrega o identificador da transação que a criou (`xmin`). O *commit* é um registro no WAL (*write-ahead log*, o log que vai para o disco antes dos dados) e uma marca de "confirmada" no `pg_xact`. Se o processo morre antes, a transação conta como abortada. Abortar não apaga nada: as linhas ficam no disco, mortas e sem atestado de óbito, invisíveis, até o `VACUUM` passar recolhendo os corpos. O *rollback* do PostgreSQL é o luto mais barato da computação: ele só para de olhar.

Consistência é a letra intrusa: fala das invariantes da aplicação, e o banco só impõe as que você declarou. O *constraint trigger* do capítulo 1 confere a soma zero no `COMMIT`, e só consegue porque as quatro inserções estão na mesma transação. Fora de um `BEGIN` explícito, cada comando é uma transação sozinha; com quatro *commits*, o primeiro lançamento chegaria ao trigger somando +10000 e morreria ali.

Durabilidade: depois que o `COMMIT` retorna, a transação sobrevive a uma queda. Com `synchronous_commit = on` (o padrão), o `COMMIT` só responde depois do `fsync` do WAL. Com `off`, responde antes, e uma queda pode perder as últimas transações confirmadas, até três vezes o `wal_writer_delay` (600 ms com o padrão de 200 ms), sem corromper nada. Seiscentos milissegundos de pagamentos confirmados e esquecidos são uma tarde inteira de planilha. Perda de disco: capítulos 23 e 40.

Isolamento: transações concorrentes não enxergam o trabalho pela metade umas das outras. Quanto cada uma enxerga é o nível de isolamento, e foi aí que o Carlos tropeçou "por segurança".

O padrão SQL define isolamento pelos defeitos

O padrão SQL (ISO/IEC 9075) define quatro níveis pelos fenômenos que cada um proíbe, a forma mais honesta já inventada de vender um produto: "este modelo vem sem *dirty read*; o *phantom* é opcional de fábrica".

- **Dirty read**: ler o que outra transação escreveu e não confirmou. Se ela aborta, você decidiu com dado que nunca existiu.

- **Non-repeatable read**: ler a mesma linha duas vezes e receber valores diferentes, porque outra transação alterou e confirmou no meio.
- **Phantom**: repetir uma consulta com `WHERE` e receber outro conjunto de linhas, porque outra transação inseriu ou apagou linhas do predicado.

Nível	Dirty read	Non-repeatable read	Phantom
Read uncommitted	possível	possível	possível
Read committed	não	possível	possível
Repeatable read	não	não	possível
Serializable	não	não	não

Serializable ainda tem uma exigência a mais: o resultado de transações concorrentes precisa ser igual ao de alguma execução delas uma de cada vez, em alguma ordem.

Em 1995, Berenson, Bernstein, Gray, Melton e os dois O'Neil publicaram "A Critique of ANSI SQL Isolation Levels": as definições do padrão são ambíguas, e evitar os três fenômenos não basta para ser serializable. O artigo nomeou anomalias que a tabela esquece, o **lost update** (duas transações leem, calculam e gravam por cima, e uma escrita some) e o **write skew**, que logo será apresentado ao vendedor 42. E definiu o **snapshot isolation**, que não cabe em nenhuma linha da tabela e acabou implementado em metade do mercado com o nome de outra linha.

O "repeatable read" do PostgreSQL é snapshot isolation. O "serializable" do Oracle também, então lá o write skew passa até no nível mais alto, de crachá. O padrão do InnoDB (MySQL) é repeatable read, com semântica própria; o do PostgreSQL, read committed. O nome do nível é rótulo de embalagem; as anomalias ficam na documentação, em letra de bula.

MVCC: o banco que guarda todas as versões da sua vida

O PostgreSQL implementa isolamento com **MVCC** (*multiversion concurrency control*). O `UPDATE` nunca altera a linha no lugar: grava uma versão nova (*tuple*) com `xmin` da transação atual e marca a velha com `xmax`. O `DELETE` só preenche o `xmax`. O banco é aquele ex que guarda todas as versões de você e escolhe qual mostrar conforme quem pergunta.

Quem pergunta carrega um **snapshot**: o maior identificador de transação já iniciado e a lista das que estavam em andamento naquele instante. Uma versão é visível se a transação do `xmin` confirmou antes do snapshot e a do `xmax`, se existir, não. Leitor não

bloqueia escritor nem o contrário; só duas escritas na mesma linha se esperam, pelo lock de linha. O preço é o lixo: versões mortas se acumulam até o `VACUUM`, que só remove o que nenhum snapshot aberto ainda enxerga.

Os dois níveis diferem em quando tiram o snapshot:

- **Read committed**: um snapshot novo a cada comando. Por isso o `SUM` depois do `FOR UPDATE` do capítulo 2 enxergava o débito recém-confirmado.
- **Repeatable read**: um snapshot só, tirado no primeiro comando da transação (no primeiro comando, não no `BEGIN`), usado até o fim. Retrato congelado, que não muda por mais saques que aconteçam lá fora.

Nas escritas também. Em read committed, o `UPDATE` que encontra uma linha alterada por transação concorrente espera e, se a outra confirma, reavalia o `WHERE` na versão nova (o `UPDATE` condicional do capítulo 2). Em repeatable read, "o primeiro a atualizar vence": a segunda espera e, se a primeira confirmou, recebe isto:

```
lastro=*> UPDATE contas SET bloqueada = true WHERE id = 42;
ERROR:  could not serialize access due to concurrent update
```

Isso mata o lost update: ler, somar no Python e gravar por cima, que em read committed apaga a escrita do vizinho, vira erro. O Carlos acertou metade do comentário. A outra metade custou R$ 2.000,00.

O detalhe que derruba a Lastro: a regra só dispara quando a outra transação **alterou ou apagou** a linha. Linha apenas travada com `FOR UPDATE` passa sem erro.

Write skew: dois médicos de plantão e nenhum no hospital

O exemplo clássico da literatura: regra de pelo menos um médico de plantão, dois escalados, os dois pedem para sair ao mesmo tempo. Cada transação conta dois de plantão, conclui que pode sair e altera a própria linha. Nenhuma escrita colide, o snapshot isolation confirma as duas, e o hospital fica sem ninguém. Eu já fui esse plantão, e o monitoramento também tinha ido pra casa.

A definição: **write skew** acontece quando duas transações concorrentes leem dados que se sobrepõem, cada uma decide com base no que leu e cada uma escreve em linhas diferentes. Sozinha, cada uma preserva a invariante; juntas, violam. O snapshot isolation só detecta conflito de escrita na mesma linha, e aqui nenhuma linha foi escrita pelas

duas. Quando a leitura é um predicado ("os saques do dia da conta 42"), as linhas do problema nem existiam na hora: são inserções novas, fantasmas que o snapshot nunca vai mostrar.

Na segunda-feira, foi isto:

```
T1 (aba 2, R$ 4.000,00)                  T2 (aba 3, R$ 4.000,00)
BEGIN ISOLATION LEVEL REPEATABLE READ    BEGIN ISOLATION LEVEL REPEATABLE READ
primeiro comando: snapshot S1            primeiro comando: snapshot S2 (= S1)
INSERT saque -> trigger:                 INSERT saque -> trigger:
  FOR UPDATE conta 42 (obtém)              FOR UPDATE conta 42 (espera...)
  saldo 6.000, sacado hoje 1.000: ok
COMMIT
                                           (obtém: linha só travada, sem erro)
                                           saldo 6.000, sacado hoje 1.000 (S2): ok
                                         COMMIT
resultado: saldo -2.000,00; sacado hoje 9.000,00
```

O trigger roda dentro da transação, com o snapshot dela. O `FOR UPDATE` fez as duas passarem em fila indiana pela porta, e o snapshot fez a segunda passar de olhos vendados. O débito de T1 já estava confirmado, mas T2 tinha tirado o retrato antes, e repeatable read é um juramento solene de nunca olhar a foto nova. Era a armadilha anunciada no capítulo 2, com limite diário de brinde.

Dentro do snapshot isolation há duas saídas. Uma é **materializar o conflito**: fazer as duas transações escreverem na mesma linha (um `UPDATE` na conta), e o write skew vira conflito de escrita que o "primeiro vence" detecta. A outra é voltar ao read committed com o lock do capítulo 2, em que o comando depois do lock lê dados frescos. As duas exigem descobrir, para cada regra, qual linha concentra o conflito, e que ela exista; quando não existe, alguém inventa uma linha só para brigar por ela.

Serializable de verdade, com nota fiscal

Desde a versão 9.1, o serializable do PostgreSQL é **SSI** (*Serializable Snapshot Isolation*), algoritmo de Cahill, Röhm e Fekete (2008), implementado por Ports e Grittner. Ele roda sobre o snapshot isolation e acrescenta vigilância: cada leitura deixa um **SIRead lock**, um lock de predicado que não bloqueia ninguém e só anota "esta transação leu isto". Quando uma transação escreve algo que outra leu sem enxergar, o banco registra uma dependência leitura-escrita (*rw-antidependency*) entre as duas.

A base teórica: toda execução não serializável sob snapshot isolation tem um ciclo com duas arestas leitura-escrita consecutivas entre transações concorrentes. O PostgreSQL não procura o ciclo inteiro; ao ver a estrutura perigosa (uma transação "pivô" com uma

aresta entrando e outra saindo), aborta uma das envolvidas, mesmo que o ciclo nunca se fechasse. Daí os falsos positivos, abortos por excesso de cautela, como o gerente que cancela o deploy de sexta que ia funcionar. Na conversa:

```
App:        COMMIT.
PostgreSQL: ERROR: could not serialize access due to read/write
            dependencies among transactions
            DETAIL: Reason code: Canceled on identification as a pivot,
            during commit attempt.
            HINT: The transaction might succeed if retried.
App:        Mas eu fiz algo errado?
PostgreSQL: Você ia fazer. Tenta de novo.
```

O código do erro é SQLSTATE `40001` (`serialization_failure`). O preço do serializable fica todo aqui:

- **Retry obrigatório.** A aplicação repete a transação inteira, do `BEGIN`, refazendo leituras e decisões. Repetir só o `COMMIT` é inútil, e reaproveitar valores da tentativa anterior traz o write skew de volta pela porta dos fundos. O deadlock (`40P01`, capítulo 2) entra no mesmo tratamento.
- **Granularidade.** Leituras por índice travam linhas e páginas; um *seq scan* trava a tabela inteira para fins de SSI, e locks finos demais são promovidos a grossos (`max_pred_locks_per_transaction`, padrão 64). Mais grosso, mais falsos positivos: índice vira também questão de taxa de abort (capítulo 5).
- **Só vale entre serializables.** Uma transação em read committed ao lado fica fora da vigilância, como o `psql` do Carlos. Em troca, protege qualquer invariante entre transações serializable, inclusive as que ninguém se deu ao trabalho de proteger, que são a maioria.

Na Lastro: um nível para cada operação

Os requisitos: o pagamento com split grava tudo ou nada; todo débito de conta disponível respeita saldo e limite diário, venha de onde vier; extrato e conciliação leem um estado coerente ao longo de várias consultas. Para os débitos:

Opção	Saldo e limite	Custo	Serve quando
Repeatable read (o PR de sexta)	write skew	nenhum, até segunda-feira	nunca, com regra sobre conjunto de linhas
Read committed + `FOR UPDATE` (capítulo 2)	corretos se a regra cabe sob o lock da conta	achar a linha que cobre cada regra nova	regras sobre uma linha; muitos débitos por conta

Opção	Saldo e limite	Custo	Serve quando
Repeatable read + conflito materializado	corretos	versão nova e WAL por débito; mesma análise por regra	sistema preso a repeatable read
Serializable	corretos para qualquer regra	`40001`, retry, falsos positivos	conflitos raros, regras que mudam

Saldo e limite diário cabem sob o lock da conta, e o read committed do capítulo 2 resolveria os dois. A Lastro escolhe serializable pela fila do gerente de produto: limite noturno para Pix e limite menor para chave Pix cadastrada há menos de 24 horas, que lê a tabela de chaves, fora do lock da conta. Em read committed, cada regra nova exige descobrir qual linha travar, análise feita por quem estiver de plantão naquela sexta; em serializable, o SSI detecta o ciclo seja qual for a regra. O custo é pequeno: uns 400 saques por dia, conflito só quando a mesma conta debita duas vezes no mesmo instante, ou seja, quando o vendedor 42 está acordado.

Operação	Nível	Motivo
Pagamento com split	read committed, uma transação	só insere; nenhuma leitura decide escrita; soma zero no `COMMIT`
Débito de conta disponível (saque, estorno)	serializable, com retry	regras sobre conjuntos de linhas
Extrato, conciliação	repeatable read, `READ ONLY`	saldo inicial, lançamentos e saldo final no mesmo snapshot

O fluxo do saque passa a ser:

```
tentativa 1 a 3:
  BEGIN ISOLATION LEVEL SERIALIZABLE
    INSERT transação de saque + 2 lançamentos
      trigger: FOR UPDATE na conta; saldo e saques do dia
  COMMIT
  40001 ou 40P01 -> espera aleatória de 5 a 50 ms, próxima tentativa
  3 falhas ou outro erro -> saque recusado com erro temporário
depois do COMMIT: pedido de Pix ao banco parceiro
```

A espera aleatória evita nova colisão no mesmo milissegundo (capítulo 15). O Pix fica fora do laço: transação abortada não deixa rastro, Pix enviado não volta. Reenvio do saque pelo marketplace é assunto do capítulo 16.

O trigger mantém o `FOR UPDATE` do capítulo 2: em serializable, só organiza a fila (a segunda transação ainda recebe `40001` e repete); em read committed, segue protegendo os caminhos que escapam da regra, como o `psql` do Carlos. E passa a recusar débito em repeatable read, o único nível em que a receita falha:

```
lastro=*> INSERT INTO lancamentos (transacao_id, conta_id, valor)
          VALUES (88231, 42, 400000);
ERROR:  débito em conta disponível recusado em repeatable read
HINT:   use serializable ou read committed (capítulo 3, vendedor 42)
```

A segunda-feira reprisada: T2 recebe `40001`, repete, tira um snapshot que já contém o débito de T1, encontra R$ 2.000,00 de saldo e R$ 5.000,00 sacados no dia, e recusa.

Com centenas de débitos por segundo na mesma conta, a taxa de `40001` cresce, cada abort joga trabalho fora, e o lock em read committed (mais as técnicas de conta quente do capítulo 4) custa menos. Num banco cujo serializable é snapshot isolation com outro nome, só o conflito materializado protege.

Armadilhas e casos reais

> **Armadilha:** o retry que repete efeitos colaterais. Se o laço manda e-mail, chama o banco parceiro ou publica evento, cada `40001` gera um efeito a mais, e o vendedor recebe três "seu saque foi aprovado" para um saque recusado. Dentro do laço, só o que o `ROLLBACK` desfaz.
>
> **Armadilha:** a transação longa em repeatable read. Uma conciliação de 40 minutos segura um snapshot por 40 minutos, e o `VACUUM` não remove nenhuma versão morta que ele ainda possa ver, em tabela nenhuma. O ledger incha de cadáveres que ninguém pode enterrar porque um relatório ainda está olhando para eles. A sessão `idle in transaction` faz o mesmo sem nem trabalhar; `idle_in_transaction_session_timeout` executa a sessão parada.
>
> **Na vida real:** em 2017, Todd Warszawski e Peter Bailis publicaram "ACIDRain" (SIGMOD), sobre aplicações de e-commerce populares nos níveis de isolamento padrão dos bancos: 22 ataques em 12 aplicações, entre eles gastar o mesmo vale-presente várias vezes com requisições concorrentes. O vendedor 42 tem concorrência acadêmica.
>
> **Na vida real:** em 2020, o Jepsen testou o PostgreSQL 12.3 e encontrou anomalias do tipo write skew (G2-item, na nomenclatura de Adya) no nível serializable, por um bug no SSI corrigido na 12.4. Até o banco que vigia leituras teve um dia de vista cansada.

Resumo

- ACID: atomicidade pelo *commit* no WAL; consistência são as invariantes declaradas; durabilidade pelo `fsync` do WAL.
- Níveis do padrão SQL definidos por fenômenos; lost update, write skew e snapshot isolation vêm de Berenson et al. (1995).
- MVCC: versões com `xmin` e `xmax`; snapshot por comando (read committed) ou por transação (repeatable read).
- Repeatable read do PostgreSQL é snapshot isolation: impede lost update, permite write skew.
- Serializable do PostgreSQL é SSI: abort com `40001`, falsos positivos, retry da transação inteira, efeitos externos fora do laço.

A Lastro agora

O pagamento com split grava os quatro lançamentos numa transação só, em read committed, com a soma zero conferida no `COMMIT`. Débitos de conta disponível (saque e estorno) rodam em serializable, com até três tentativas para `40001` e `40P01`, espera aleatória de 5 a 50 ms e o Pix só depois do `COMMIT`. O trigger mantém o `FOR UPDATE`, confere saldo e limite diário de R$ 5.000,00 e recusa débito em repeatable read. Extrato e conciliação leem em repeatable read, `READ ONLY`. A linha `# por segurança` foi apagada, e foi a mudança mais segura do PR.

O vendedor 42 abriu a quinta aba, ganhou um `40001` particular e, na segunda tentativa, "saldo insuficiente". Saque único, partida dobrada, agora com nota fiscal do PostgreSQL. Enquanto isso, toda venda do Bazar escreve na mesma conta de comissão, e o Bazar acabou de anunciar a campanha "Black Friday o mês inteiro". Por enquanto ninguém trava essa conta. Por enquanto.

Referências

- ISO/IEC 9075-2, *SQL/Foundation*, níveis de isolamento.
- Berenson, H. et al. *A Critique of ANSI SQL Isolation Levels*. SIGMOD 1995. https://arxiv.org/abs/cs/0701157
- Cahill, M. J.; Röhm, U.; Fekete, A. D. *Serializable Isolation for Snapshot Databases*. SIGMOD 2008.
- Ports, D. R. K.; Grittner, K. *Serializable Snapshot Isolation in PostgreSQL*. VLDB 2012. https://arxiv.org/abs/1208.4179

- Härder, T.; Reuter, A. *Principles of Transaction-Oriented Database Recovery*. ACM Computing Surveys, 1983.
- PostgreSQL. *Transaction Isolation*. https://www.postgresql.org/docs/current/transaction-iso.html
- PostgreSQL. *Serialization Failure Handling*. https://www.postgresql.org/docs/current/mvcc-serialization-failure-handling.html
- PostgreSQL. *Asynchronous Commit*. https://www.postgresql.org/docs/current/wal-async-commit.html
- Warszawski, T.; Bailis, P. *ACIDRain: Concurrency-Related Attacks on Database-Backed Web Applications*. SIGMOD 2017.
- Jepsen. *PostgreSQL 12.3*. https://jepsen.io/analyses/postgresql-12.3

CAPÍTULO 4

Locks, contenção e pools de conexão

A conta que todo mundo quer tocar

Às 21h04 de uma terça, no quarto dia da campanha "Black Friday o mês inteiro", a API da Lastro parou de responder, e quem avisou foi o Bazar, num chamado com o assunto "URGENTE!!!!" e o corpo "?????". O monitoramento só soube às 21h11. O monitoramento soube pelo chamado.

O log tinha uma linha só, repetida 3.800 vezes:

```
sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 10
reached, connection timed out, timeout 30.00
```

Saque, extrato, consulta de cobrança: tudo esperava 30 segundos e morria. Um vendedor que só queria o extrato de março olhou meio minuto para um spinner e ganhou um `500`. O ledger estava íntegro, a soma dava zero, e ninguém conseguia entrar no prédio.

A causa tinha três semanas. O gerente de produto pediu "a comissão do Bazar em tempo real no painel, é só um número". O `SUM` sobre os 2,1 milhões de lançamentos da conta de comissão levava 1,4 segundo, e o Carlos criou a coluna `contas.saldo_cache` e um trigger que, a cada lançamento, faz `UPDATE contas SET saldo_cache = saldo_cache + valor`. O commit dizia `saldo instantâneo, zero risco`. Dois erros em quatro palavras, uma taxa que nem o vendedor 42 alcança.

Toda venda do Bazar grava quatro lançamentos (capítulo 3): liquidante, vendedor, comissão e receita da Lastro. Só o vendedor muda. Com o trigger, toda venda passou a dar `UPDATE` na mesma linha da comissão, e o handler que confirma o Pix só faz `COMMIT` depois de mandar o webhook para o Bazar, que em campanha respondia em 250 ms.

Por que uma linha derrubou a API inteira, inclusive o que nem passa perto dela?

Onde o PostgreSQL guarda um lock

Os locks de tabela do PostgreSQL ficam em memória compartilhada. São oito modos, de `ACCESS SHARE` (todo `SELECT`; só conflita com `ACCESS EXCLUSIVE`) até `ACCESS EXCLUSIVE` (quase todo `ALTER TABLE`, `DROP`, `TRUNCATE`; conflita com tudo, até com o `SELECT` mais humilde). `INSERT`, `UPDATE` e `DELETE` pegam `ROW EXCLUSIVE`, que, apesar do nome, não conflita com outro `ROW EXCLUSIVE`. Quem batizou odiava a gente. Locks de DDL ficam para o capítulo 7.

Os locks de linha ficam na própria linha: quem trava grava o seu ID de transação no `xmax` da tupla (o do MVCC, capítulo 3) e marca nos bits de informação que aquilo é lock, e não exclusão. Por isso o PostgreSQL trava 10 milhões de linhas sem gastar memória de lock. O custo é uma escrita em cada linha travada, inclusive no `SELECT ... FOR UPDATE`, que tem cara de leitura e alma de `UPDATE`.

São quatro modos de lock de linha:

Modo	Quem pega	Conflita com
`FOR KEY SHARE`	checagem de chave estrangeira	`FOR UPDATE`
`FOR SHARE`	pedido explícito	`FOR NO KEY UPDATE`, `FOR UPDATE`
`FOR NO KEY UPDATE`	`UPDATE` que não muda chave	`FOR SHARE`, `FOR NO KEY UPDATE`, `FOR UPDATE`
`FOR UPDATE`	`DELETE`, `UPDATE` de chave, pedido explícito	todos

Repare na primeira linha. Todo `INSERT` em `lancamentos` confere a chave estrangeira para `contas` e, para isso, pega `FOR KEY SHARE` na linha da conta. O `FOR UPDATE` do trigger de débito (capítulo 2) bloqueia, portanto, todo crédito na conta enquanto o saque não termina. Um `FOR NO KEY UPDATE` daria a mesma exclusão entre débitos e deixaria os créditos passarem. Duas palavras a mais no SQL, uma fila a menos na conta.

E quem espera, espera o quê? A transação B encontra o ID da transação A no `xmax`, enfileira-se pelo lock da tupla e depois pede um `ShareLock` sobre o próprio ID de transação de A. Esse lock A segura desde que nasceu e só solta no `COMMIT` ou no `ROLLBACK`. Ou seja: B espera A **morrer**. O lock de linha do PostgreSQL é uma herança, e só se recebe no velório.

A matemática da fila indiana

Contenção é tempo esperando lock em vez de trabalhando. Para uma linha: se cada transação segura o lock por `t` segundos, aquela linha atende no máximo `1 / t` transações por segundo, com qualquer número de núcleos, de workers ou de orações do fundador. O tempo de posse vai do comando que trava até o fim da transação, com tudo que o código faz no meio. Na Lastro de 21h04:

```
BEGIN
  INSERT lançamentos ... trigger: UPDATE contas (comissão)  -- trava aqui
  UPDATE cobrancas SET estado = 'paga'
  POST https://bazar.../webhooks         -- 250 ms, Bazar em campanha
COMMIT                                   -- solta aqui
```

Posse de ~260 ms dá teto de 3,8 vendas por segundo na comissão. O pico da campanha: 12 por segundo. Chegam 12, saem 3,8: a fila cresce 8 transações por segundo, para sempre, ou até algo quebrar. Algo quebrou.

Cada transação na fila segura uma conexão, e conexão esperando lock está tão ocupada quanto uma trabalhando. As 45 do pool (15 por worker) encheram em menos de seis segundos, e daí em diante toda requisição, inclusive o extrato de março, esperava 30 segundos e morria. O lock transforma espera em conexões presas, e o pool transforma conexões presas em fila para todo mundo. A contenção foi numa linha; a indisponibilidade, no sistema inteiro.

O próprio banco mostra a cena:

```
  pid  |        state        | wait_event_type |  wait_event   | query
-------+---------------------+-----------------+---------------+---------------
 51120 | idle in transaction | Client          | ClientRead    | UPDATE cobranc
 51133 | active              | Lock            | transactionid | INSERT INTO la
 51134 | active              | Lock            | tuple         | INSERT INTO la
 51137 | active              | Lock            | tuple         | INSERT INTO la
 (44 rows)
```

O processo 51120 segura o lock e está `idle in transaction`: parado, de transação aberta, esperando o Python, que espera o Bazar. O 51133 é o primeiro da fila e espera o `transactionid` dele; os outros 42 esperam o lock da tupla, atrás do 51133. Quarenta e quatro conexões, uma trabalhando, e ela nem está trabalhando. É a reunião de planejamento do banco de dados.

`FOR UPDATE`, `NOWAIT` e `SKIP LOCKED`

O `SELECT ... FOR UPDATE` trava as linhas que retorna até o fim da transação e, por padrão, espera quem chegou antes. Dois modificadores mudam isso.

`NOWAIT` falha na hora com erro `55P03` (`lock_not_available`) se a linha estiver travada: serve quando esperar é pior que desistir. `SET lock_timeout = '2s'` dá o mesmo erro depois de dois segundos. Fila, sim, mas com prazo de validade, igual a iogurte.

`SKIP LOCKED` pula as linhas travadas e devolve as livres. O resultado omite linhas que existem, então ele serve para uma coisa só: vários consumidores pegando tarefas da mesma tabela sem pisar uns nos outros.

```
BEGIN;
SELECT id, payload FROM tarefas
 WHERE estado = 'pendente'
 ORDER BY id
 LIMIT 10
 FOR UPDATE SKIP LOCKED;
-- worker A recebe 1..10; worker B, no mesmo instante, recebe 11..20
```

Se o worker morre no meio, o `ROLLBACK` solta as linhas e outro worker herda as tarefas do falecido. Fila sem broker, transacional com o resto do banco. O limite é o volume: cada tarefa é escrita, lock e, depois, uma tupla morta para o VACUUM enterrar (o capítulo 17 conta quando a fila sai do banco).

Otimista, ou "trava nada e reza"

Tudo até aqui é controle **pessimista**: trava antes, porque alguém vai mexer. O controle **otimista** não trava: lê a linha com um número de versão, faz o que tem de fazer e grava só se a versão ainda for a mesma.

```
UPDATE cobrancas
   SET estado = 'estornada', versao = versao + 1
 WHERE id = '01936f3e-...' AND versao = 7;
-- UPDATE 0   (alguém mudou a cobrança depois que você leu)
```

`UPDATE 0` é conflito: relê e decide de novo, ou devolve erro. É o `UPDATE` condicional do capítulo 2 com versão no lugar do saldo, e o que o HTTP faz com `ETag` e `If-Match` (capítulo 9).

A régua é a taxa de conflito. Com conflito raro, o otimista não segura lock nenhum, e o pessimista paga escrita de lock à toa. Com conflito frequente, como na conta de comissão a 12 vendas por segundo, o otimista também faz fila, só que em loop no Python, perdendo, repetindo e queimando CPU. E o otimista alcança o que o pessimista não alcança: leitura e escrita em requisições diferentes. O operador abre a cobrança às 14h e clica em "estornar" às 14h07. Nenhum lock de banco atravessa sete minutos e duas requisições HTTP. Uma coluna de versão atravessa até o almoço.

Deadlock no banco

O PostgreSQL não previne deadlock (capítulo 2): detecta. Quem espera um lock por mais de `deadlock_timeout` (1 segundo por padrão) dispara a busca de ciclo no grafo de espera; achando um, aborta uma das transações com `40P01`, e a outra segue.

Com o `saldo_cache` do Carlos, o ciclo veio de brinde. O pagamento atualizava o cache na ordem de inserção (liquidante, vendedor, comissão, receita). O estorno, na ordem crescente de `conta_id` do capítulo 2, atualizava a comissão, criada antes de quase todo vendedor, primeiro.

```
ERROR:  deadlock detected
DETAIL:  Process 51190 waits for ShareLock on transaction 9182733;
         blocked by process 51188.
         Process 51188 waits for ShareLock on transaction 9182730;
         blocked by process 51190.
CONTEXT: while updating tuple (8812,4) in relation "contas"
```

O estorno retenta `40P01` (capítulo 3). O pagamento não retentava: o comprador pagou, o banco parceiro confirmou, e a Lastro respondeu erro. E como a busca do ciclo só começa depois de 1 segundo, todo deadlock custa pelo menos 1 segundo de lock segurado, com a fila inteira da conta quente atrás, batendo o pé.

Uma conexão é um processo

O PostgreSQL atende cada conexão com um processo do sistema operacional, criado por `fork`. Abrir uma custa handshake TCP, TLS, autenticação SCRAM-SHA-256 e o `fork`: alguns milissegundos na mesma rede, mais entre máquinas distantes (capítulo 8). Cada processo ocupa alguns megabytes, mais `work_mem` por ordenação ou hash, e cada conexão existente pesa em contas internas como a montagem de snapshots (o PostgreSQL 14 baratiou esse custo, sem zerá-lo).

Por isso `max_connections` vem com 100 por padrão. Passar disso compra memória gasta, troca de contexto e um banco mais lento com 500 conexões ativas do que com 20. Conexão é cara de abrir e cara de manter, como todo relacionamento que você já teve.

Pools: guardar conexões abertas

Um **pool de conexões** mantém conexões abertas e as empresta: pega, usa, devolve, e ninguém paga handshake nem `fork` de novo.

O pool **na aplicação** vive dentro de cada processo. O `QueuePool` do SQLAlchemy usa `pool_size=5` (conexões mantidas), `max_overflow=10` (extras sob demanda) e `pool_timeout=30` (segundos de espera antes do erro que abriu o capítulo); o `psycopg_pool` tem equivalentes. O problema é a multiplicação: o banco recebe processos × (pool + overflow). Três workers dão 45. Trinta réplicas com três workers dão 1.350, e o `max_connections` ri por último.

O pool **externo**, como o PgBouncer, fica entre as aplicações e o banco: aceita milhares de conexões de clientes e as multiplexa sobre poucas conexões reais. O modo define quando a conexão real volta para o pool:

Modo	Conexão real devolvida	Quebra
`session`	quando o cliente desconecta	nada; e multiplexa pouco
`transaction`	no fim de cada transação	`SET` de sessão, advisory lock de sessão, `LISTEN`, tabelas temporárias que sobrevivem à transação
`statement`	a cada comando	transações com mais de um comando

O modo `transaction` multiplexa de verdade e cobra o preço: estado de sessão some ou, pior, aparece para outro cliente. Prepared statements quebravam nesse modo até o PgBouncer 1.21, que passou a rastreá-los com `max_prepared_statements`. No modo `transaction`, a conexão real vira motel: entra, resolve, sai, e o próximo hóspede não pode achar nada seu na gaveta.

Quantas conexões, afinal

Pool pequeno demais enfileira requisições na aplicação; grande demais enfileira processos no banco, disputando CPU. O número sai de duas contas.

A demanda vem da Lei de Little (capítulo 14): conexões em uso = taxa de requisições × tempo médio de posse. Com 40 requisições por segundo segurando conexão por 25 ms, a média é 1 conexão ocupada. O pool cobre picos acima disso, não 45 vezes isso.

O teto vem do banco. Um ponto de partida empírico, publicado pelo projeto HikariCP, é `conexões = núcleos × 2 + discos efetivos`: acima disso, cada conexão a mais só disputa CPU e disco. Para o PostgreSQL da Lastro, com 8 núcleos e SSD, dá perto de 17 a 20 conexões ativas úteis. Ponto de partida para medir, não lei.

E o `pool_timeout` de 30 segundos? Aquele timeout de 30 segundos que ninguém sabe quem configurou? Ninguém configurou. É o padrão. O culpado era o silêncio.

Na Lastro: a conta quente

Uma **conta quente** recebe lançamento numa fração grande de todas as transações. A Lastro tem três: liquidante, receita e comissão de cada marketplace; com dezenas de marketplaces (Parte IV), liquidante e receita entram em todas as vendas de todos. São as contas mais populares da empresa, e popularidade, no PostgreSQL, se mede em gente esperando na porta.

O ledger do capítulo 1 já aguentava: créditos são `INSERT`, e `INSERT` na mesma conta não conflita. Quem criou a fila foi o cache. E o requisito do painel (saldo da comissão em tempo real, sem o `SUM` de 1,4 s) continua de pé, porque o gerente de produto já mostrou o número para o Bazar. As opções:

Estratégia	Espera por lock	Saldo no painel	Custo
`saldo_cache` numa linha	1 fila por conta; teto `1/t`	exato	o incidente
Só `SUM`, sem cache	nenhuma	exato, 1,4 s e crescendo	leitura O(n); índices no capítulo 5
Agregação assíncrona (fila com `SKIP LOCKED`)	nenhuma no pagamento	atrasado segundos	job novo; saldo defasado
Saldo em N baldes (subcontas de cache)	1 fila por balde; teto `N/t`	exato, soma de N linhas	N linhas por conta; ordem de travamento

A Lastro escolhe os baldes, só para contas marcadas como quentes. A tabela `saldos_parciais (conta_id, balde, saldo)` tem 16 linhas por conta quente; cada pagamento soma o valor no balde `hash(cobranca_id) % 16`, e o saldo é a soma das 16, exata no snapshot de quem lê. Contas de vendedor seguem sem cache: lá ninguém disputa. A agregação assíncrona perdeu porque o financeiro concilia o saldo exibido com o ledger, e saldo atrasado vira divergência às 23h de sábado. Ela seria a escolha se o painel aceitasse defasagem de segundos (consistência eventual, capítulo 19).

```
pagamento (read committed)
  INSERT 4 lançamentos            (FK: FOR KEY SHARE nas contas)
  UPDATE cobrancas estado = paga
  UPDATE saldos_parciais           ordem (conta_id, balde)   -- último comando
COMMIT                             posse do balde: ~2 ms
  └─> webhook ao Bazar             depois do COMMIT, fora da transação
```

Junto vieram três regras. O webhook sai depois do `COMMIT` (o capítulo 17 o tira da requisição também). O trigger de débito troca `FOR UPDATE` por `FOR NO KEY UPDATE`, e créditos param de esperar saques. A ordem de `(conta_id, balde)` vale para toda escrita em conta, e o pagamento ganha a retentativa de `40P01` que o estorno já tinha.

Com posse de ~2 ms, um balde aguenta perto de 500 pagamentos por segundo; 16, perto de 8.000. O pico da campanha caberia em 2,4% de um balde só. Os 16 são para a Black Friday de verdade (capítulo 14) e para o liquidante, que recebe as vendas de todos os marketplaces. O Carlos sugeriu 1.024 baldes, "por segurança". O `# por segurança` tem currículo neste livro, e foi recusado.

O pool ficou em `pool_size=5`, `max_overflow=0` e `pool_timeout=3`: 15 conexões previsíveis e falha em 3 segundos em vez de 30. Cada transação ganha `lock_timeout = '2s'`, e o `idle_in_transaction_session_timeout` do capítulo 3 fica em 5 segundos. PgBouncer fica fora: com 15 conexões, multiplexaria quase nada e acrescentaria um salto de rede e um componente para operar. Ele vira a escolha quando processos × pool passar do teto útil do banco, com o monólito dividido em serviços (capítulo 26) e réplicas em Kubernetes (capítulo 35).

Armadilhas e casos reais

> **Armadilha:** chamada de rede dentro de transação com lock. O lock dura o p99 do serviço externo, não o seu. A sua conta quente passa a ter o SLA do Bazar, e o Bazar não tem SLA.
>
> **Armadilha:** aumentar o pool para "resolver" a falta de conexões. Com fila de lock, mais conexões só põem mais transações na fila. O Carlos subiu `pool_size` para 50 às 21h20. Às 21h24 eram 150 conexões esperando a mesma linha, e o `max_connections` recusou o resto. Ele chamou de teste de carga.
>
> **Armadilha:** `SKIP LOCKED` em consulta que precisa de resposta completa. Um relatório que "pula o que está travado" é um relatório que mente em silêncio, e o financeiro descobre a mentira na planilha.

`SET statement_timeout` de sessão atrás de PgBouncer em modo `transaction` fica na conexão real e vaza para o próximo cliente; `SET LOCAL` dura só a transação.

> **Na vida real:** antes do PgBouncer 1.21, o erro `prepared statement "S_1" does not exist` era a forma mais comum de descobrir que o driver usava prepared statements e o pooler estava em modo `transaction`. Drivers como o JDBC do

> PostgreSQL passam a preparar comandos automaticamente depois de algumas execuções (`prepareThreshold` , 5 por padrão), então o erro aparecia na quinta chamada, nunca no teste com quatro.

Resumo

- Locks de tabela em memória (8 modos); locks de linha no `xmax` (4 modos); esperar uma linha é esperar a transação dona terminar.
- FK pega `FOR KEY SHARE` , que só conflita com `FOR UPDATE` .
- Linha travada por `t` segundos: teto de `1/t` transações por segundo, e `t` inclui tudo até o `COMMIT` .
- `NOWAIT` e `lock_timeout` limitam a espera; `SKIP LOCKED` serve a filas de tarefas.
- Otimista com versão: conflito raro ou requisições diferentes; colapsa com conflito alto.
- Deadlock detectado após `deadlock_timeout` (1 s), vítima com `40P01` ; ordem fixa previne.
- Conexão = processo; pool na aplicação multiplica por processo; PgBouncer multiplexa (session, transaction, statement).
- Pool: Lei de Little para a demanda, núcleos × 2 + discos para o teto.
- Conta quente: I/O fora da transação, travar por último, baldes.

A Lastro agora

O `saldo_cache` saiu de `contas` . Contas quentes (liquidante, receita, comissão de cada marketplace) têm saldo em 16 baldes em `saldos_parciais` , atualizados no último comando, em ordem de `(conta_id, balde)` . O webhook sai depois do `COMMIT` , o trigger de débito usa `FOR NO KEY UPDATE` , o pagamento retenta `40P01` , `lock_timeout` é 2 s, e o pool tem 5 conexões por worker, sem overflow, 3 segundos de espera.

O commit do Carlos dizia `saldo instantâneo, zero risco` . O saldo voltou a ser instantâneo, em 16 pedaços, e o risco virou uma linha do postmortem com o nome dele. O painel ganhou fôlego e uma ambição nova: o extrato do vendedor por período, que filtra 40 milhões de lançamentos e já leva 6 segundos. O gerente de produto acha que é só um índice. Dessa vez ele quase acertou.

Referências

- PostgreSQL 16 Documentation, *Explicit Locking*. https://www.postgresql.org/docs/16/explicit-locking.html
- PostgreSQL 16 Documentation, *SELECT: The Locking Clause*. https://www.postgresql.org/docs/16/sql-select.html#SQL-FOR-UPDATE-SHARE
- PostgreSQL 16 Documentation, *Client Connection Defaults*. https://www.postgresql.org/docs/16/runtime-config-client.html
- PostgreSQL 16 Documentation, *The Cumulative Statistics System*. https://www.postgresql.org/docs/16/monitoring-stats.html
- SQLAlchemy 2.0 Documentation, *Connection Pooling*. https://docs.sqlalchemy.org/en/20/core/pooling.html
- PgBouncer, *Configuration*. https://www.pgbouncer.org/config.html
- HikariCP Wiki, *About Pool Sizing*. https://github.com/brettwooldridge/HikariCP/wiki/About-Pool-Sizing
- Freund, A. *Improving Postgres Connection Scalability: Snapshots*. Microsoft Tech Community, 2020.

CAPÍTULO 5

Índices

O extrato de seis segundos e o índice de quarenta e oito

O gerente de produto disse que era só um índice, e o Carlos, que nunca deixou um gerente sem resposta, criou um índice às 15h40 de uma quinta-feira, em produção, sem perguntar a ninguém. Foi o único momento da história da Lastro em que o gerente e o Carlos concordaram, e o banco puniu os dois.

O vendedor 7310, a Capinhas Hiper Ltda., maior do Bazar, espera seis segundos pela primeira página do extrato de outubro. `lancamentos` passou de 40 milhões de linhas, 1,1 milhão delas da Capinhas, e o extrato filtra por conta e por `criado_em` (`timestamptz`, *default* `now()`). Os índices eram três: chave primária, `transacao_id` e `conta_id`, este desde o trigger de saque do capítulo 2. O Carlos:

```
CREATE INDEX lancamentos_criado_em_idx ON lancamentos (criado_em);
-- 48 segundos. "Rapidinho." (Carlos, 15h40)
```

`CREATE INDEX` sem `CONCURRENTLY` pega um lock `SHARE` na tabela, que conflita com o `ROW EXCLUSIVE` de qualquer escrita (capítulo 4). O `lock_timeout` de 2 segundos, aquele que salvou o pool, cumpriu o dever: os pagamentos receberam `55P03` e morreram limpos, sem fila. Quarenta e oito segundos de Pix recusado. O Carlos chama isso de "rapidinho"; o Bazar chamou de "URGENTE!!!!", assunto que já tem template próprio no helpdesk.

E o extrato foi de 6,0 para 5,8 segundos. Por que um índice em `criado_em` quase não ajudou uma consulta que filtra por `criado_em`? E qual índice resolve sem cobrar pedágio em cada lançamento de cada pagamento?

A B-tree por dentro, ou a busca binária que foi para a academia

O índice padrão do PostgreSQL é a *B-tree* (`nbtree`, variante de Lehman e Yao): chaves ordenadas em páginas de 8 KB, em níveis. A busca é uma busca binária em que cada página interna divide o intervalo em centenas, e não em dois.

```
              [raiz: ... | 5000 | 9100 | ...]
              /             |              \
[interno: ...|7300|7320|...]   ...           ...
          |
[folha: (7310, out/01 00:02) ... (7310, out/01 00:09)] <-> [folha seguinte]
          |
        TID (bloco 811204, item 7) -> linha na tabela (heap)
```

As **páginas internas** (a raiz é uma delas) guardam chaves separadoras e ponteiros para as páginas de baixo. As **folhas** guardam as chaves, cada uma com um **TID** (*tuple identifier*: bloco e posição da linha na tabela), e são encadeadas às vizinhas: uma busca por faixa desce até o primeiro valor e depois anda de lado. O ponteiro para a irmã da direita é o truque de Lehman e Yao que deixa a leitura seguir enquanto outra transação divide uma página.

A conta do fan-out: uma entrada `BIGINT` ocupa uns 20 bytes, e uma página comporta perto de 400. Para 40 milhões de chaves, cerca de 110 mil folhas, umas 280 páginas internas e uma raiz. Três níveis. Com 400 vezes mais dados, quatro. Raiz e internos ficam em memória; a folha custa no máximo uma leitura de disco.

Folha cheia se divide (*page split*): metade das entradas vai para uma página nova e o pai ganha uma separadora. Se o pai enche, divide também, até a raiz, que ganha uma raiz nova acima dela. A árvore só cresce para cima, e todas as folhas ficam à mesma distância da raiz. Única hierarquia do planeta em que todo mundo lá embaixo está a exatamente três reuniões do topo.

Desde o PostgreSQL 13, a **deduplicação** guarda uma chave repetida uma vez só, com a lista de TIDs: o índice em `conta_id`, com 1,1 milhão de entradas `7310`, agradece. A B-tree responde a `=`, faixas, `IN`, `IS NULL`, prefixo de `LIKE` (com ressalva de *collation*, nas armadilhas) e `ORDER BY` nos dois sentidos, porque as folhas são percorridas para a frente e para trás. Dado que sai ordenado do índice dispensa o `Sort`, a etapa do plano que mais ama memória e a sua espera.

O que o índice guarda, e a visita obrigatória à tabela

A folha guarda a chave e o TID. Só isso. Ela não sabe se a linha apontada está viva.

Pelo MVCC (capítulo 3), a visibilidade mora no `xmin` e no `xmax` da linha, e o índice aponta para todas as versões, vivas, mortas e não nascidas. É um necrotério com a ficha de todo mundo e sem atestado de óbito de ninguém. Saber se a linha existe para você exige ir à tabela, o *heap*, e olhar: um *index scan* que devolve 96 mil linhas faz até 96 mil visitas a blocos espalhados.

A saída é o **index-only scan**: se todas as colunas de que a consulta precisa estão no índice, o banco pula o heap, desde que a página esteja marcada como *all-visible* no **visibility map** (um bit por página, mantido pelo `VACUUM`). Página não marcada, visita obrigatória, contada no plano como `Heap Fetches`. Para pôr colunas no índice sem fazê-las parte da chave, existe o **índice de cobertura** com `INCLUDE`: as colunas extras vão só nas folhas e servem para devolver, nunca para buscar. Custa tamanho, e índice com `INCLUDE` não usa deduplicação.

Tabela append-only é o caso ideal: linha sem `UPDATE` nem `DELETE` fica visível para sempre depois do `VACUUM`, que desde o PostgreSQL 13 também dispara por volume de inserções. O ledger imutável do capítulo 1 ganha o elogio do VACUUM. Única tabela da Lastro com ficha limpa, e só porque é proibida de fazer qualquer coisa.

A ordem das colunas, ou o índice lê da esquerda para a direita

Um **índice composto** em `(a, b)` ordena por `a` e, dentro de cada `a`, por `b`:

Condição	Índice `(conta_id, criado_em)`	Índice `(criado_em, conta_id)`
`conta_id = 7310`	faixa contígua	varre o índice inteiro
`conta_id = 7310 AND criado_em` em outubro	faixa contígua e exata	faixa de outubro inteira, filtrando conta
`criado_em` em outubro, todas as contas	varre o índice inteiro	faixa contígua
`ORDER BY criado_em DESC` com `conta_id = 7310`	já ordenado	precisa ordenar

A regra: igualdade primeiro, faixa ou ordenação por último. Com a igualdade na frente, a Capinhas em outubro ocupa folhas contíguas; com a faixa na frente, aparece salpicada no outubro de todo mundo, uma entrada a cada quarenta.

O índice do Carlos, só em `criado_em`, é a faixa na frente sem nem ter a conta atrás. Outubro tem 5,4 milhões de lançamentos; o índice acha todos, o heap é visitado para cada um, e 98% vão para o lixo por serem de outro vendedor. O planner fez a conta e voltou para o índice em `conta_id`, com a cara de quem recebeu um presente de amigo secreto e já guardou a nota para trocar. O Carlos indexou o calendário e ficou magoado

quando o calendário não sabia quem era a Capinhas. Em defesa dele, o índice sabia exatamente quando ela vendeu. Junto com outros 9 mil vendedores. No mesmo milissegundo.

`(conta_id, criado_em)` serve também a buscas só por `conta_id` (o prefixo da esquerda está ordenado), e o índice antigo vira redundante. Sem condição na primeira coluna, o composto fica quase inútil. Quase: o PostgreSQL 18 trouxe o *skip scan*, que pula entre os valores distintos da primeira coluna quando eles são poucos. Com uns 9 mil valores distintos de `conta_id`, o skip scan desceria a árvore 9 mil vezes para achar outubro, o que dá mais trabalho do que o Carlos teve para derrubar o Pix.

Seletividade, cardinalidade e o dia em que o planner ignora o seu índice

Cardinalidade é o número de valores distintos de uma coluna; **seletividade**, a fração das linhas que uma condição devolve: `conta_id = 7310` devolve 2,75%; `estado = 'paga'` em `cobrancas`, uns 95%. Índice em coluna de 95% é decoração.

O planner pesa I/O: por padrão, `seq_page_cost = 1` e `random_page_cost = 4`, ou seja, pular de bloco em bloco seguindo TIDs custa quatro vezes mais por página do que ler em sequência. Com fração grande, o *seq scan* ganha. No meio fica o *bitmap scan*: junta os TIDs num mapa de blocos e lê cada bloco do heap uma vez, em ordem física.

As frações vêm das estatísticas do `ANALYZE`: valores mais comuns, histograma e a **correlação** entre a ordem física da tabela e a do índice. `criado_em` numa tabela append-only tem correlação perto de 1; `conta_id`, perto de zero, porque a Capinhas vende o dia todo junto com todo mundo e tem lançamento em quase toda página. Leitura de planos é o capítulo 6; aqui basta o extrato antes da correção:

```
Limit (actual time=6011.4..6011.5 rows=50 loops=1)
  ->  Sort (actual time=6011.4..6011.4 rows=50 loops=1)
        Sort Key: criado_em DESC
        ->  Bitmap Heap Scan on lancamentos (actual time=401.2..5968.0 rows=96211
loops=1)
              Recheck Cond: (conta_id = 7310)
              Filter: ((criado_em >= '2026-10-01 03:00:00+00') AND (criado_em < '2026-
11-01 03:00:00+00'))
              Rows Removed by Filter: 1004127
              Heap Blocks: exact=287412
              ->  Bitmap Index Scan on lancamentos_conta_id_idx (rows=1100338 loops=1)
Execution Time: 6012.9 ms
```

O índice achou 1,1 milhão de linhas, o banco visitou 287 mil dos 330 mil blocos da tabela para jogar fora um milhão delas por serem de outro mês, e ordenou o resto para mostrar cinquenta. Leu 2,2 GB para exibir uma página. Dava para baixar o extrato em forma de

filme.

Parciais, de expressão e os índices que não são árvore

Um **índice parcial** tem `WHERE`: só guarda as linhas do predicado, e só serve à consulta cuja condição o implica. Um **índice de expressão** indexa o resultado de uma expressão, como `lower(email)`, e só serve para consultas com a mesma expressão; a função precisa ser `IMMUTABLE`, porque o valor no índice não pode mudar sem a linha mudar. Fora da B-tree:

Tipo	Estrutura	Responde a	Custo e limite
Hash	buckets do hash de 32 bits	só `=`	sem `UNIQUE`, multicoluna nem ordem; WAL só desde o 10
GIN	invertido: elemento → lista de linhas	arrays, `jsonb` (`@>`, `?`), texto	escrita cara; `fastupdate` gera lista pendente que a leitura varre
GiST	árvore de "caixas" que contêm as de baixo	geometria, faixas (`&&`), vizinho mais próximo, exclusão	busca desce por ramos sobrepostos
BRIN	mínimo e máximo por faixa de blocos	faixas em coluna correlacionada	minúsculo; inútil sem correlação

O hash guarda 4 bytes por chave e ganha em tamanho com chave longa. O GIN existe porque uma B-tree num `jsonb` indexa o documento inteiro como um valor, e ninguém procura cobrança pelo JSON completo. O BRIN indexa `criado_em` dos 40 milhões de lançamentos em algumas dezenas de KB. O índice do Carlos tinha 500 MB e foi ignorado; um BRIN do tamanho de um GIF de gatinho teria servido melhor à conciliação.

Cada índice é um imposto sobre a escrita

Cada `INSERT` grava a linha no heap e uma entrada em cada índice, cada uma com seu registro no WAL. Como o índice do PostgreSQL aponta para a posição física, um `UPDATE` que põe a versão nova em outro lugar precisa de entrada nova em todos os índices, inclusive nos das colunas que nem mudaram. A linha mudou um centavo, e cinco índices precisaram ser avisados, por escrito, com cópia para o WAL.

A exceção é o **HOT** (*heap-only tuple*): sem coluna indexada alterada e com espaço na página, a versão nova é encadeada à antiga, sem tocar índice. Indexar uma coluna que vive mudando, como `cobrancas.estado`, desliga o HOT de toda transição de estado

(coluna de predicado de índice parcial conta como indexada). Um índice "inofensivo" dobra o custo do `UPDATE` mais frequente do sistema, e a fatura chega no fim do mês junto com a do disco.

E há localidade. Um índice em `criado_em` recebe toda inserção na folha mais à direita, sempre quente; um em `(conta_id, criado_em)`, no fim da faixa de cada conta, espalhada por milhares de folhas. Depois de cada *checkpoint*, a primeira alteração de cada página grava a página inteira no WAL (*full-page write*): mais páginas distintas, mais WAL. O imposto da escrita é pago por todo mundo, inclusive por quem nunca abriu o extrato na vida.

Os índices do ledger, escolhidos pelo que se lê

Uma semana de produção medida em `lancamentos`, no pico: ~50 inserções/s; ~20 extratos/s (conta mais faixa de `criado_em`, ordenado); ~5 buscas/s por `transacao_id` (estorno); ~1 `SUM` por conta/s no trigger de débito; e a conciliação, uma vez por dia, lendo o dia inteiro. As alternativas para o extrato:

Opção	Extrato da Capinhas	Escrita	Tamanho estimado
Manter `(conta_id)`	6 s	1 entrada	~300 MB (deduplicado)
`(criado_em)` do Carlos	5,8 s	1 entrada, página quente	~500 MB
`(conta_id, criado_em)`	~3 ms, visita o heap	1 entrada	~1,2 GB
`(conta_id, criado_em) INCLUDE (valor, transacao_id)`	~0,3 ms, index-only	1 entrada	~2 GB

A Lastro escolhe a última e apaga a do Carlos e a de `conta_id`. O requisito: extrato de qualquer vendedor abaixo de 100 ms, incluindo o saldo anterior ao período, um `SUM(valor)` sobre a mesma faixa de chave. Com `valor` nas folhas, listagem e soma viram index-only scan, e o `SUM` do trigger de débito (capítulos 2 e 3) ganha o mesmo atalho. Sendo append-only, `Heap Fetches` fica perto de zero:

```
Limit (actual time=0.041..0.198 rows=50 loops=1)
  ->  Index Only Scan Backward using lancamentos_conta_criado_idx on lancamentos
        Index Cond: ((conta_id = 7310) AND (criado_em >= '2026-10-01 03:00:00+00')
                     AND (criado_em < '2026-11-01 03:00:00+00'))
        Heap Fetches: 0
Execution Time: 0.231 ms
```

Seis segundos viraram 0,23 milissegundo, e o `Backward` entrega o `ORDER BY criado_em DESC` lendo as folhas de trás para a frente, sem `Sort`.

O imposto: continuam três índices e três entradas por inserção. O composto espalha escrita, e em homologação o `INSERT` de lançamento foi de 0,11 para 0,13 ms, com 15% a mais de WAL, folga larga no orçamento de posse de 2 ms do capítulo 4. A conciliação, que lê um dia inteiro de todas as contas, ganha um BRIN em `criado_em`: dezenas de KB, escrita desprezível, até as partições por tempo do capítulo 24.

Fora do ledger, em `cobrancas`:

- `(expira_em) WHERE estado = 'pendente'`, parcial, para o job que expira Pix não pagos. Só 0,3% das cobranças estão pendentes, e o HOT perdido é o de uma transição que acontece uma vez por cobrança.
- `(marketplace_id, (metadata->>'pedido_id'))`, de expressão, para o Bazar buscar pelo pedido que manda no `metadata` (`jsonb`). Com uma chave só buscada, a B-tree basta; se os marketplaces passarem a buscar chaves arbitrárias, um GIN com `jsonb_path_ops`, mais caro na escrita, vira a escolha certa.

E as chaves de identificação (CNPJ, chave Pix, EndToEndId) passam a `COLLATE "C"`, pelo motivo da primeira armadilha.

O Carlos pediu para manter o índice dele "por segurança". O `pg_stat_user_indexes` mostrou `idx_scan = 0`. Foi enterrado com honras: 500 MB, zero leituras, uma vida inteira dedicada a atrasar `INSERT`. Deixa viúva (a fatura do disco) e nenhum herdeiro.

Armadilhas e casos reais

> **Na vida real:** a glibc 2.28, de 2018, mudou a ordenação de texto de quase todas as *locales*, sobretudo pontuação e espaços. Quem atualizou o sistema operacional sem `REINDEX` ficou com B-trees ordenadas pela regra velha e consultadas pela nova: buscas que não acham linhas existentes e `UNIQUE` deixando passar duplicatas (a wiki do PostgreSQL documenta). CPF com pontos e traço é a vítima típica. Com `COLLATE "C"` a ordem é a dos bytes, e o `LIKE 'prefixo%'` ainda usa a B-tree sem `text_pattern_ops`. Seu índice pode corromper sem um bit mudar no disco: basta mudarem o alfabeto por baixo dele.
>
> **Na vida real:** em 2016, a Uber explicou a troca de PostgreSQL por MySQL num sistema central, e o primeiro item era amplificação de escrita: os índices do PostgreSQL apontam para a posição física, então um `UPDATE` sem HOT gera

> entradas em todos os índices, enquanto os secundários do InnoDB apontam para a chave primária. A comunidade PostgreSQL contestou parte das conclusões; a conta por índice é a deste capítulo.
>
> **Armadilha:** `WHERE date(criado_em) = '2026-10-15'` ignora o índice em `criado_em`: o índice guarda `criado_em`, e a consulta pede outra coisa. Índice de expressão também falha, porque `date(timestamptz)` depende do fuso da sessão e é só `STABLE`. A forma certa é a faixa, `criado_em >= '2026-10-15 03:00:00+00' AND criado_em < '2026-10-16 03:00:00+00'`. Função na coluna é o jeito mais discreto de dar `DROP INDEX` sem deixar digital.
>
> **Armadilha:** `CREATE INDEX` sem `CONCURRENTLY` trava as escritas durante o build inteiro, como o Carlos demonstrou ao vivo. O `CONCURRENTLY` evita isso com duas passadas na tabela e pode deixar um índice `INVALID` se falhar; detalhes no capítulo 7, onde o Carlos ganha outra chance de virar exemplo. Índice sem uso (`idx_scan = 0` depois de semanas) continua cobrando em toda escrita; antes do enterro, confira se ele garante um `UNIQUE`, porque esse trabalha sem nunca ser lido.

Resumo

- B-tree: páginas de 8 KB, fan-out de centenas, três ou quatro níveis, folhas encadeadas para faixas e ordenação.
- O índice guarda chave e TID; index-only scan exige as colunas no índice (`INCLUDE`) e páginas all-visible.
- Composto: igualdade primeiro, faixa ou ordenação por último; o prefixo da esquerda serve sozinho.
- Seletividade e correlação decidem entre index, bitmap e seq scan.
- Parcial e de expressão; hash, GIN, GiST e BRIN para o que a B-tree não cobre.
- Cada índice cobra em toda escrita e pode desligar o HOT.

A Lastro agora

O extrato caiu de 6 segundos para 0,23 milissegundo com `(conta_id, criado_em) INCLUDE (valor, transacao_id)`, que substitui o de `conta_id` e serve ao trigger de débito. O índice do Carlos morreu, um BRIN atende a conciliação, `cobrancas` ganhou um parcial e um de expressão, e as chaves de identificação passaram a `COLLATE "C"`. O ledger segue com três B-trees, agora todas lidas.

O gerente de produto disse que era só um índice, e era: só não era o do Carlos. Animado, pediu um painel com as vendas do dia do Bazar, e o ORM entregou um painel que dispara 400 consultas por tela, cada uma usando o índice com perfeição, 400 vezes. E a estagiária da Capinhas acaba de clicar na página 4.000 do extrato completo. O índice acha a primeira linha em três níveis; o `OFFSET` manda pular 199.950 delas, uma por uma, cada uma encontrada com toda a eficiência do mundo antes de ser jogada fora.

Referências

- PostgreSQL 16, documentação, capítulo "Indexes": https://www.postgresql.org/docs/16/indexes.html
- PostgreSQL 16, "B-Tree Indexes" (implementação, deduplicação, remoção bottom-up): https://www.postgresql.org/docs/16/btree.html
- PostgreSQL 16, "Index-Only Scans and Covering Indexes": https://www.postgresql.org/docs/16/indexes-index-only-scans.html
- PostgreSQL 16, "Heap-Only Tuples (HOT)": https://www.postgresql.org/docs/16/storage-hot.html
- PostgreSQL 16, `CREATE INDEX`: https://www.postgresql.org/docs/16/sql-createindex.html
- PostgreSQL 18, notas de lançamento (skip scan em B-tree): https://www.postgresql.org/docs/18/release-18.html
- Lehman, P. L.; Yao, S. B. "Efficient Locking for Concurrent Operations on B-Trees". ACM TODS, 1981.
- PostgreSQL Wiki, "Locale data changes": https://wiki.postgresql.org/wiki/Locale_data_changes
- Uber Engineering, "Why Uber Engineering Switched from Postgres to MySQL" (2016): https://www.uber.com/blog/postgres-to-mysql-migration/

CAPÍTULO 6

Planos de execução

O painel das 400 consultas e a estagiária na página 4.000

O gerente de produto ganhou o painel que pediu, e o painel ganhou o banco de dados inteiro de brinde.

A tela mostra as últimas 100 vendas do dia do Bazar, com vendedor, comprador, split e estornos. O ORM fez uma consulta para as 100 cobranças e, depois, uma por relação, por cobrança: cem compradores, cem splits, cem listas de estornos e 99 vendedores, porque o *identity map* do SQLAlchemy já tinha a Capinhas na sessão e poupou uma ida ao banco. Total: 400 consultas por tela. O gerente chamou isso de "tempo real". O PostgreSQL chamou de assédio.

Cada ida e volta, somando rede e ORM, custa perto de 1 ms. A tela demora 420 ms com uma conexão do pool de 15 (capítulo 4) presa o tempo todo, e se atualiza a cada 15 segundos em 40 abas abertas no Bazar: mais de mil consultas por segundo para mostrar 100 linhas, num banco que recebe 50 lançamentos por segundo no pico. O painel virou o maior cliente da Lastro e não paga tarifa.

Enquanto isso, a estagiária da Capinhas clicou na página 4.000 do extrato completo: `OFFSET 199950`, 3,4 segundos, com o índice perfeito do capítulo 5 sendo usado. O Bazar abriu o chamado. O helpdesk tem template.

Às 18h, o cabeçalho do painel, com o total vendido por vendedor, que às 9h saía em meio segundo, leva 7,4 segundos. Ninguém mudou nada. O banco ficou mais burro ao longo do dia, igual a todos nós.

Como o banco decide o que fazer com uma consulta, e como você descobre isso antes do Bazar?

EXPLAIN, ou como fazer o banco confessar

O SQL diz o que você quer; o *planner* (planejador) decide como buscar: gera planos alternativos, estima o custo de cada um e executa o mais barato. O plano é uma árvore de nós, cada um entregando linhas ao pai.

`EXPLAIN` mostra o plano sem executar. `EXPLAIN ANALYZE` executa e mostra, ao lado de cada estimativa, o que aconteceu: é o banco tirando a roupa na sua frente, e ninguém fica bonito. `BUFFERS` acrescenta quantas páginas de 8 KB cada nó achou no cache (`shared hit`) e quantas pediu ao sistema operacional (`read`); no PostgreSQL 18, vem ligado por padrão com `ANALYZE` .

Cada nó traz dois grupos de números:

- `cost=0.43..8.45 rows=1 width=16` : custo de partida (até a primeira linha), custo total, linhas estimadas e largura média em bytes. O custo é uma unidade arbitrária, calibrada em leituras de página: `seq_page_cost = 1` , `random_page_cost = 4` , `cpu_tuple_cost = 0.01` , `cpu_index_tuple_cost = 0.005` , `cpu_operator_cost = 0.0025` . Milissegundo, nunca. Custo é a fé do planner expressa em páginas.
- `actual time=0.019..21.330 rows=11802 loops=1` : tempo real até a primeira linha e até a última, em milissegundos, linhas produzidas e quantas vezes o nó rodou. Tempo e linhas são médias **por loop**. Um nó com `actual time=0.6` e `loops=11802` custou sete segundos, e o plano não soma isso para você, porque o PostgreSQL acha que você é adulto.

As folhas são os nós de leitura do capítulo 5 (`Seq Scan` , `Index Scan` , `Index Only Scan` , `Bitmap Heap Scan`); acima ficam `Sort` , `Limit` , `HashAggregate` e as junções. A pergunta que resolve metade dos planos ruins: em que nó `rows` estimado e `rows` real se separam por uma ordem de grandeza? Dali para cima, tudo foi escolhido com base numa mentira.

> **Armadilha:** `EXPLAIN ANALYZE` executa a consulta. Inclusive `UPDATE` e `DELETE` . O Carlos descobriu isso com um `EXPLAIN ANALYZE DELETE FROM lancamentos WHERE ...` "só para ver o plano", e quem salvou o ledger foi o trigger de imutabilidade do capítulo 1, que recusou o `DELETE` com a dignidade de um tabelião. Para medir escrita, o comando vai dentro de `BEGIN` e termina em `ROLLBACK` .

Três jeitos de juntar duas tabelas

O `JOIN` que você escreve é uma intenção. O banco tem três algoritmos para cumpri-la, e cada um tem o seu fetiche.

Nested loop. Para cada linha do lado externo, procura as correspondentes no lado interno: N buscas de O(log M) com índice no interno, N varreduras completas sem ele. Tem custo de partida quase zero (combina com `LIMIT`) e aceita qualquer condição, inclusive desigualdade. É a relação de muitos para muitos atendida um de cada vez, e o algoritmo que transforma uma estimativa de 1 linha em 11.802 varreduras. Já chegamos lá.

Hash join. Lê o lado menor inteiro (o *build*), monta uma tabela hash pela chave e percorre o lado maior (o *probe*) consultando a tabela. Custa O(N + M), só serve para igualdade e paga a construção antes da primeira linha. A tabela cabe em `work_mem × hash_mem_multiplier` (4 MB × 2,0 por padrão desde o PostgreSQL 15); se não couber, o plano mostra `Batches: 8` e o resto vai para disco.

Merge join. Recebe os dois lados ordenados pela chave e avança em paralelo, o passo de intercalação do merge sort. Custa O(N + M) mais as ordenações que faltarem; com os dois lados saindo ordenados de índices, sai quase de graça.

Algoritmo	Custo típico	Bom quando	Ruim quando
Nested loop	N × busca no interno	Externo pequeno, índice no interno, `LIMIT`	Externo grande e interno sem índice
Hash join	N + M, memória do lado menor	Igualdade, nenhum lado pequeno, sem ordem útil	Lado de build maior que a memória
Merge join	N + M + ordenações	Entradas grandes já ordenadas	Ordenar tudo só para juntar

A escolha depende de quantas linhas cada lado tem, e o planner nunca conta linhas: ele estima.

Estatísticas, ou o planner decide com o que viu há 19 dias

As estimativas vêm de `pg_statistic` (legível pela view `pg_stats`), preenchida pelo `ANALYZE`. Por coluna: fração de nulos, número estimado de valores distintos (`n_distinct`), os valores mais comuns com suas frequências (MCV), um histograma de faixas com o mesmo número de linhas cada para o resto, e a `correlation` do capítulo 5. Com `default_statistics_target = 100`, são até 100 valores comuns e 100 faixas, tirados de uma amostra de 300 × 100 = 30.000 linhas, tenha a tabela mil linhas ou um bilhão. O planner decide o destino de 40 milhões de lançamentos com base numa pesquisa de opinião.

Igualdade usa o MCV, faixa interpola no histograma, e duas condições juntas multiplicam as seletividades, assumindo colunas independentes. Em `cidade = 'Campinas' AND uf = 'SP'`, isso divide a estimativa pela fração de SP sem motivo, porque toda Campinas do cadastro já está em SP. `CREATE STATISTICS` (tipos `dependencies`, `ndistinct` e `mcv`) coleta estatística sobre combinações de colunas.

O autovacuum roda `ANALYZE` quando as linhas alteradas desde o último passam de `autovacuum_analyze_threshold + autovacuum_analyze_scale_factor × linhas`: por padrão, 50 + 10% da tabela. Em `cobrancas`, com 9 milhões de linhas, são 900 mil alterações; a umas 12 mil cobranças por dia, um `ANALYZE` a cada 75 dias. O último foi há 19.

O histograma de `criado_em` termina 19 dias atrás. Para ele, hoje não existe: a estimativa para `criado_em >= current_date` cai para quase zero e vira `rows=1`. Quando há uma B-tree que começa pela coluna, o planner lê no índice o valor máximo real e corrige o extremo do histograma; o índice do painel é `(marketplace_id, criado_em)` e começa pela coluna errada. O banco acredita, com a convicção de um sistema determinístico, que o Bazar vendeu uma coisa hoje. Uma. Provavelmente uma capinha.

O cabeçalho do painel junta as cobranças do dia com a conta disponível de cada vendedor, e `contas.vendedor_id` não tem índice (o PostgreSQL nunca indexa sozinho a coluna que referencia numa chave estrangeira). Com 1 linha prevista do lado de fora, o mais barato é um nested loop que varre `contas` uma vez. Uma vez por venda, e foram 11.802:

```
HashAggregate  (cost=214.02..214.03 rows=1 width=40) (actual time=7409.80..7410.21
rows=1187 loops=1)
  Group Key: k.id
  ->  Nested Loop  (cost=0.43..214.01 rows=1 width=16) (actual time=0.08..7398.10
rows=11802 loops=1)
        Join Filter: (k.vendedor_id = c.vendedor_id)
        Rows Removed by Join Filter: 14150598
        ->  Index Scan using cobrancas_marketplace_criado_idx on cobrancas c
              (cost=0.43..8.45 rows=1 width=16) (actual time=0.02..21.33 rows=11802
loops=1)
              Index Cond: ((marketplace_id = 1) AND (criado_em >= CURRENT_DATE))
        ->  Seq Scan on contas k  (cost=0.00..202.50 rows=1200 width=16) (actual
time=0.01..0.62 rows=1200 loops=11802)
              Filter: (tipo = 'disponivel'::text)
              Rows Removed by Filter: 7800
Execution Time: 7410.63 ms
```

De baixo para cima: `rows=1` contra `rows=11802` em `cobrancas`, ali nasceu a mentira. O `Seq Scan` em `contas` leva 0,62 ms, ótimo até você ver `loops=11802`, e o nested loop jogou fora 14 milhões de pares para achar 11.802. Às 9h, com 500 vendas, eram 500 varreduras e meio segundo. O banco punia o sucesso comercial com uma precisão que o financeiro respeitaria.

Depois de um `ANALYZE cobrancas`, que lê 30 mil linhas de amostra e termina em menos de um segundo:

```
HashAggregate  (actual time=29.95..30.41 rows=1187 loops=1)
  ->  Hash Join  (rows=11650) (actual time=1.92..26.07 rows=11802 loops=1)
        Hash Cond: (c.vendedor_id = k.vendedor_id)
        ->  Index Scan using cobrancas_marketplace_criado_idx on cobrancas c
```

```
(rows=11650) (actual rows=11802 loops=1)
        ->  Hash  (actual rows=1200 loops=1)
              ->  Seq Scan on contas k  (actual rows=1200 loops=1)
Execution Time: 30.88 ms
```

Mesma consulta, mesmos dados. A estimativa certa trocou o algoritmo, `contas` passou a ser lida uma vez, e 7,4 segundos viraram 31 ms sem mudar uma linha de código.

N+1, o ORM que pergunta uma coisa de cada vez

A lista do painel tem outro defeito, e esse nenhum `ANALYZE` conserta. N+1: uma consulta traz N objetos, e o acesso a uma relação em cada um dispara mais uma. A culpa mora num inocente `cobranca.vendedor.nome` dentro de um laço do template, que o *lazy loading* transforma em ida ao banco. Cada consulta tem plano ótimo e índice certo. O problema é haver 400 delas.

O log do SQLAlchemy entrega o crime em forma de poesia concreta:

```
SELECT ... FROM cobrancas WHERE marketplace_id = 1 AND criado_em >= ... ORDER BY
criado_em DESC LIMIT 100
SELECT ... FROM compradores WHERE compradores.id = 991827
SELECT ... FROM recebedores_split WHERE recebedores_split.cobranca_id = 8812340
SELECT ... FROM estornos WHERE estornos.cobranca_id = 8812340
SELECT ... FROM vendedores WHERE vendedores.id = 7310
SELECT ... FROM compradores WHERE compradores.id = 1004512
... (mais 394)
```

Em produção, quem denuncia é o `pg_stat_statements`, que agrega por texto normalizado: `SELECT ... FROM estornos WHERE cobranca_id = $1` com milhões de `calls` e 0,1 ms de média. Média ótima, total obsceno. É o bandido que só rouba um centavo por vez e já levou a empresa inteira.

O custo mora fora do banco: ida e volta na rede, serialização, um `await` e o ORM montando objetos. A 1 ms por ida e volta, 400 consultas custam 400 ms mesmo com o banco respondendo instantaneamente.

A cura, em qualquer ORM, é dizer antes o que vai ser lido:

- Junção na própria consulta (`joinedload` no SQLAlchemy, `select_related` no Django): uma consulta só. Perfeita para muitos-para-um. Em coleções, multiplica: 100 cobranças × 4 recebedores × 2 estornos viram 800 linhas para montar 100 objetos, o produto cartesiano do amor.
- Uma consulta por relação com `IN` (`selectinload`, `prefetch_related`): `WHERE cobranca_id IN (...)` com os 100 IDs de uma vez, sem multiplicação.

- SQL escrito para a tela, com agregação no banco, quando a tela mostra um resumo e não objetos.

Para o N+1 não voltar quando alguém acrescentar uma coluna ao template, o SQLAlchemy aceita `lazy="raise"` na relação: acessar algo não carregado levanta exceção em vez de consultar em silêncio. A lentidão em produção vira erro no teste. Crime desses merece flagrante.

OFFSET, o banco que conta até 199.950 em voz alta

A documentação do PostgreSQL avisa: as linhas puladas por `OFFSET` ainda são computadas no servidor. O nó `Limit` puxa linhas do filho e descarta as primeiras, e tudo abaixo dele (índice, heap, a junção com `transacoes` para buscar a descrição) roda 200.000 vezes. É uma chacina de 199.950 linhas para entregar 50, com atestado de óbito:

```
Limit  (actual time=3021.40..3022.17 rows=50 loops=1)
  Buffers: shared hit=402118 read=187441
  ->  Nested Loop  (actual time=0.05..3008.62 rows=200000 loops=1)
```

O custo cresce linear com a página: a 1 sai em 1 ms, a 4.000 em 3 segundos, e a 22.012 (a última da Capinhas) leva o tempo de a estagiária ser efetivada. Some o `COUNT(*)` que o paginador roda para escrever "página 4.000 de 22.012", contando 1,1 milhão de lançamentos a cada clique.

E tem um defeito de correção: se entra um lançamento entre a página 1 e a 2, tudo escorrega uma posição, e o último item da 1 reaparece no topo da 2. Em extrato financeiro, linha repetida vira chamado com o assunto que você conhece.

A paginação por cursor (*keyset pagination*) troca "pule N linhas" por "continue de onde parou": o cliente devolve a chave da última linha vista, e ela vira limite de faixa no índice:

```sql
SELECT criado_em, id, valor, transacao_id
FROM lancamentos
WHERE conta_id = $1
  AND (criado_em, id) < ($2, $3)   -- última linha da página anterior
ORDER BY criado_em DESC, id DESC
LIMIT 50;
```

A comparação de linha é lexicográfica: `criado_em` e, só no empate, `id`. O desempate é obrigatório, porque lançamentos da mesma transação têm o mesmo `criado_em` (`now()` devolve o início da transação), e um cursor só por `criado_em` pularia ou repetiria linhas. Com a chave de índice `(conta_id, criado_em, id)`, o banco desce três níveis da B-tree até o cursor e lê 50 entradas, na página 1 ou na 4.000.

O preço: nada de pular direto para a página 4.000, nada de "página X de Y", e a ordenação precisa casar com um índice. Quem quer outubro usa o extrato por período do capítulo 5.

O painel e o extrato da Lastro

A lista do painel passa a carregar vendedor e comprador por junção (muitos-para-um) e split e estornos por `selectinload`: 3 consultas por tela no lugar de 400, com todas as relações em `lazy="raise"`. O total por vendedor segue como agregação no banco.

Opção para a lista	Consultas por tela	Linhas transferidas	Custo
Lazy loading (antes)	400	~800	420 ms, conexão presa o tempo todo
`joinedload` em tudo	1	~800, com repetição	Produto cartesiano entre split e estornos
Junção + `selectinload` (escolhida)	3	~900, sem repetição	~6 ms
SQL próprio com `json_agg`	1	100	Menos linhas; mapeamento à mão para cada tela

O requisito é atualizar 40 abas a cada 15 segundos sem prender o pool de 15 conexões: 3 idas e voltas por tela resolvem, e o código segue no ORM. SQL próprio com `json_agg` vira a escolha certa quando a tela tiver relações aninhadas em vários níveis, onde até o `selectinload` empilha consultas.

Para as estimativas, `cobrancas` e `lancamentos` ganham autovacuum próprio: `autovacuum_analyze_scale_factor = 0` e `autovacuum_analyze_threshold = 5000`, um `ANALYZE` a cada 5 mil linhas alteradas, algumas vezes por dia em `cobrancas` e de hora em hora em `lancamentos` no pico. O requisito é o painel do dia: consulta que filtra "hoje" precisa de um histograma que saiba que hoje existe, e cada `ANALYZE` custa menos de um segundo. `contas.vendedor_id` ganha índice com `CONCURRENTLY` (o Carlos foi consultado e disse que dessa vez quer só assistir): 9 mil linhas, escrita rara, custo quase nulo.

O extrato completo pagina por cursor: a API devolve 50 lançamentos e um `proximo_cursor` opaco, a codificação de `(criado_em, id)` da última linha, validado como qualquer entrada e restrito à conta que o cliente já pode ver (o contrato fica para o capítulo 11). Sem `COUNT(*)` e sem número de página.

Isso muda o índice do capítulo 5 para `(conta_id, criado_em, id) INCLUDE (valor, transacao_id)`. O `id` entra na chave porque precisa participar da ordem e do limite de faixa; são 8 bytes por entrada, de 2 GB para perto de 2,3 GB, e o prefixo `conta_id` segue servindo ao extrato por período e ao `SUM` do trigger de débito. O novo nasce com `CONCURRENTLY` antes de o antigo sair com `DROP INDEX CONCURRENTLY` (detalhes no capítulo 7). Qualquer página sai em 0,4 ms. A estagiária nem percebeu: agora não existe página 4.000, existe "carregar mais", e ela está clicando nele há vinte minutos.

Armadilhas e casos reais

> **Armadilha:** otimizar pela média do `pg_stat_statements`. A consulta de 0,1 ms com 30 milhões de `calls` por dia gasta mais banco que o relatório de 4 segundos que roda duas vezes.
>
> **Armadilha:** confiar no plano do ambiente de teste. Com mil linhas, o seq scan ganha de tudo e qualquer plano parece genial; as estatísticas de uma base pequena descrevem outro banco. O plano que importa é o de produção, que o `auto_explain` grava no log quando a consulta passa de `auto_explain.log_min_duration`.
>
> **Armadilha:** cursor por `criado_em` para sincronização. Uma transação que começou antes e confirmou depois grava um `criado_em` mais antigo que o cursor que o cliente já passou, e a linha some da listagem. Para exibir extrato, serve; para um consumidor que precisa ver todas as linhas, o assunto volta nos capítulos 17 e 18.
>
> **Na vida real:** o Rails tem a gem Bullet só para avisar quando uma página dispara carregamento preguiçoso em laço. Cada ORM popular reinventou a mesma cura (`includes`, `prefetch_related`, `selectinload`) para a mesma doença, o que diz muito sobre a doença.
>
> **Na vida real:** Markus Winand mantém a página "No Offset" no *Use The Index, Luke!* com o argumento de custo do OFFSET. Os frameworks continuam gerando `OFFSET` por padrão, porque "página 4.000 de 22.012" fica bonito na tela até alguém clicar.

Resumo

- `EXPLAIN` mostra o plano, `EXPLAIN ANALYZE` executa e mostra o real; tempo e linhas são por loop.

- Nested loop para externo pequeno; hash join para igualdade entre entradas grandes; merge join para entradas ordenadas.
- Estimativas vêm de `pg_stats`, amostradas pelo `ANALYZE`, com colunas independentes salvo `CREATE STATISTICS`; em tabela grande, o histograma envelhece.
- N+1: carregamento explícito (junção ou `IN`) e `lazy="raise"`.
- OFFSET custa linear na página e repete linhas sob inserção; cursor custa constante e exige ordem total casada com um índice.

A Lastro agora

O painel caiu de 400 consultas para 3; o total por vendedor, de 7,4 s para 31 ms, com `ANALYZE` a cada 5 mil alterações e índice em `contas.vendedor_id`. O extrato pagina por cursor `(criado_em, id)` sobre o índice `(conta_id, criado_em, id) INCLUDE (valor, transacao_id)`, em 0,4 ms por página.

O banco passou 19 dias convencido de que o Bazar vendia uma coisa por dia, e a única pessoa da empresa com o mesmo nível de certeza sobre um dado errado é o fundador. Agora o gerente de produto voltou de uma reunião com um marketplace argentino e diz que suporte a outra moeda "é só uma coluna" em `lancamentos`. Uma coluna, numa tabela que não para de receber Pix nem às 3h da manhã. O Carlos já se ofereceu para o `ALTER TABLE`, rapidinho.

Referências

- PostgreSQL 16, "Using EXPLAIN": https://www.postgresql.org/docs/16/using-explain.html
- PostgreSQL 16, "Statistics Used by the Planner": https://www.postgresql.org/docs/16/planner-stats.html
- PostgreSQL 16, "Row Estimation Examples": https://www.postgresql.org/docs/16/row-estimation-examples.html
- PostgreSQL 16, "Planner Cost Constants" e demais parâmetros de consulta: https://www.postgresql.org/docs/16/runtime-config-query.html
- PostgreSQL 16, "CREATE STATISTICS": https://www.postgresql.org/docs/16/sql-createstatistics.html
- PostgreSQL 16, "Routine Vacuuming" (autovacuum e `ANALYZE`): https://www.postgresql.org/docs/16/routine-vacuuming.html
- PostgreSQL 16, "LIMIT and OFFSET": https://www.postgresql.org/docs/16/queries-limit.html

- PostgreSQL 16, `pg_stat_statements`: https://www.postgresql.org/docs/16/pgstatstatements.html
- PostgreSQL 16, `auto_explain`: https://www.postgresql.org/docs/16/auto-explain.html
- SQLAlchemy 2.0, "Relationship Loading Techniques": https://docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html
- Django, `select_related` e `prefetch_related`: https://docs.djangoproject.com/en/stable/ref/models/querysets/
- Markus Winand, "No Offset", Use The Index, Luke!: https://use-the-index-luke.com/no-offset

CAPÍTULO 7

Migrações de schema sem downtime

"É só uma coluna", às 3h11 de um domingo

Às 3h11 de um domingo, dia 1º, o Carlos aplicou uma migração de duas linhas e a Lastro passou dezoito minutos sem registrar um centavo. Uma empresa de pagamentos que não registra pagamento tem nome técnico: ONG.

Na sexta, o gerente de produto tinha voltado de uma reunião com o Mercadito Hornero, um marketplace argentino, com a frase de sempre: "suporte a peso é só uma coluna". O Carlos escolheu domingo de madrugada "porque não tem ninguém usando". O Pix funciona 24x7. Às 3h de domingo, quem usa é exatamente o tipo de pessoa que vai reclamar.

O SQL que o Alembic gerou:

```
-- 0042_multimoeda.py  (autor: Carlos; descrição: "rapidinho")
ALTER TABLE lancamentos ADD COLUMN moeda text NOT NULL DEFAULT 'BRL';
ALTER TABLE lancamentos ALTER COLUMN valor TYPE numeric(20,4);
```

A segunda linha era o plano do Carlos para "caber qualquer moeda". Nem chegou a vez dela: a primeira travou antes, e travou tudo. A sessão de diagnóstico mostrava isto:

```
 pid  | wait_event | blocked_by | query
------+------------+------------+-------------------------------------------
 8812 |            | {}         | SELECT ... -- fechamento mensal, desde 03:00
 9120 | relation   | {8812}     | ALTER TABLE lancamentos ADD COLUMN moeda ...
 9133 | relation   | {9120}     | INSERT INTO lancamentos ...
 9134 | relation   | {9120}     | INSERT INTO lancamentos ...
 9141 | relation   | {9120}     | SELECT ... FROM lancamentos -- extrato
```

O fechamento mensal do financeiro, uma transação repeatable read `READ ONLY` que começa às 3h do dia 1º e passa 38 minutos lendo o mês inteiro, segurava a tabela. O `ALTER` esperava por ele. E todo `INSERT` de pagamento esperava pelo `ALTER`, estourava o `lock_timeout` de 2 segundos (capítulo 4) e voltava com `55P03`. Às 3h19 o Bazar abriu chamado: "URGENTE!!!!". O template do helpdesk já preenche sozinho. Às 3h29 o Carlos viu o canal e apertou Ctrl+C. Na segunda, o financeiro conciliou 47 Pix à mão, com a calma de quem já perdeu tudo que tinha para perder.

O melhor: no PostgreSQL 16, a primeira linha leva milissegundos. Uma operação instantânea parou a empresa por dezoito minutos. A segunda reescreveria 45 milhões de linhas com a tabela trancada.

A pergunta do capítulo: como mudar o schema de uma tabela com 45 milhões de lançamentos, que recebe Pix a qualquer hora, sem parar nada?

Todo ALTER TABLE quer a tabela só para ele

O capítulo 4 apresentou os oito modos de lock de tabela e prometeu os locks de DDL para cá. Promessa cumprida, com juros.

Quase todo `ALTER TABLE` pede `ACCESS EXCLUSIVE`, que conflita com todos os modos, inclusive com o `ACCESS SHARE` do `SELECT` mais inocente. Concedido, ninguém lê nem escreve na tabela. Mas ele dura o tempo da operação, e operação que só mexe no catálogo dura milissegundos.

O que assusta é a fila. O gerenciador de locks do PostgreSQL atende os pedidos em ordem: um pedido novo que conflita com um pedido **que está esperando** entra na fila atrás dele, mesmo que fosse compatível com o lock que está concedido agora. Isso evita que o `ALTER` morra de fome com uma procissão infinita de `SELECT`s passando na frente. O efeito colateral é o domingo do Carlos:

```
Fechamento: (lendo o mês inteiro, ACCESS SHARE, sem pressa)
ALTER:      Preciso da tabela inteira. Espero.
INSERT:     Só quero inserir um Pix. Sou compatível com o fechamento.
Banco:      É, mas o ALTER chegou antes. Fila.
INSERT:     E se eu esperar dois segundos?
Banco:      55P03. Próximo.
Bazar:      (abre chamado)
```

O `ALTER` em si não fez nada por dezoito minutos. Ficou parado na porta, de braços abertos, impedindo a passagem. Um segurança de balada que nem trabalha na balada.

A defesa tem três partes:

1. **`lock_timeout` curto na sessão da migração**, de 1 a 2 segundos: sem lock, a migração desiste e libera a fila. A sessão do Carlos tinha o padrão do PostgreSQL, zero, que significa esperar para sempre. Paciência infinita é virtude em monge e defeito em migração.
2. **Retentativa com espera.** Uma hora a transação longa termina.
3. **Olhar antes de entrar.** `pg_stat_activity.xact_start` mostra as transações longas. O fechamento mensal estava lá desde 3h00, com o nome escrito no comentário.

```
SET lock_timeout = '1s';
ALTER TABLE lancamentos ADD COLUMN moeda text NOT NULL DEFAULT 'BRL';
-- ERROR:  canceling statement due to lock timeout
```

O pior caso vira "pagamentos esperaram 1 segundo, algumas vezes". Ninguém abre chamado por um segundo. (Mentira, o Bazar abre. Mas por um segundo ele abre com um ponto de exclamação só.)

Alguns subcomandos pegam só `SHARE UPDATE EXCLUSIVE`, que convive com leituras e escritas: `VALIDATE CONSTRAINT`, `SET STATISTICS` e parâmetros de armazenamento, como os de autovacuum. Por isso o `ALTER TABLE ... SET (autovacuum_analyze_scale_factor = 0)` do capítulo 6 passou sem ninguém notar. Único DDL do ano sem chamado, e ninguém comemorou.

Seguras, perigosas e as que mentem sobre o que são

Uma operação de DDL pode custar três coisas, em ordem crescente de dor:

- **Só catálogo:** muda metadados em `pg_class` e `pg_attribute`. Milissegundos.
- **Varredura:** lê a tabela inteira para verificar uma restrição, com o lock segurado durante a leitura.
- **Reescrita:** copia a tabela linha a linha e reconstrói todos os índices, com disco dobrado e lock até o fim. É a migração que manda o plantonista atualizar o currículo.

No PostgreSQL 16:

Operação	Custo	Lock	Alternativa sem parar
`ADD COLUMN` sem default	catálogo	`ACCESS EXCLUSIVE` breve	já é segura (com `lock_timeout`)
`ADD COLUMN ... DEFAULT` constante	catálogo (desde a versão 11)	idem	já é segura
`ADD COLUMN ... DEFAULT` volátil (`clock_timestamp()`, `gen_random_uuid()`)	reescrita	`ACCESS EXCLUSIVE` longo	coluna sem default + backfill em lotes
`ALTER COLUMN TYPE`	reescrita + índices (salvo conversões binárias, como `varchar(n)` para `text`)	`ACCESS EXCLUSIVE` longo	coluna nova + expand/contract

Operação	Custo	Lock	Alternativa sem parar
`SET NOT NULL`	varredura	`ACCESS EXCLUSIVE` longo	`CHECK (col IS NOT NULL) NOT VALID`, `VALIDATE`, então `SET NOT NULL`
`ADD CONSTRAINT ... CHECK` / `FOREIGN KEY`	varredura	`ACCESS EXCLUSIVE` / `SHARE ROW EXCLUSIVE`	`NOT VALID` + `VALIDATE CONSTRAINT`
`CREATE INDEX`	leitura completa	`SHARE` (bloqueia escritas)	`CREATE INDEX CONCURRENTLY`
`ADD CONSTRAINT ... UNIQUE`	leitura + índice	`ACCESS EXCLUSIVE`	índice único `CONCURRENTLY` + `UNIQUE USING INDEX`
`DROP COLUMN`	catálogo	`ACCESS EXCLUSIVE` breve	segura para o banco; perigosa para o código antigo
`RENAME COLUMN`	catálogo	`ACCESS EXCLUSIVE` breve	segura para o banco; fatal para o código antigo

A primeira linha do Carlos estava na segunda linha da tabela. Desde o PostgreSQL 11, um default constante não toca nas linhas existentes: o valor fica guardado no catálogo e o banco o devolve para toda linha antiga que não tem a coluna fisicamente.

```
SELECT attname, atthasmissing, attmissingval
FROM pg_attribute
WHERE attrelid = 'lancamentos'::regclass AND attname = 'moeda';
```

```
 attname | atthasmissing | attmissingval
---------+---------------+---------------
 moeda   | t             | {BRL}
```

Quarenta e cinco milhões de linhas passam a ter moeda `BRL` sem que nenhuma tenha sido escrita. A maior mentira coletiva já contada por um banco de dados, e perfeitamente consistente. Default volátil não tem esse atalho: `clock_timestamp()` precisa de um valor por linha, e o banco reescreve a tabela para calcular cada um.

A segunda linha, `bigint` para `numeric(20,4)`, reescreve ~3 GB de heap e reconstrói ~4,5 GB de índices sob `ACCESS EXCLUSIVE`: pela conta da equipe, seis a dez minutos de Lastro fechada, fora a fila. De brinde, desfazia a decisão do capítulo 1 de guardar dinheiro em

inteiro. Uma migração com dois defeitos, e o menor era derrubar a produção.

`ADD CONSTRAINT ... NOT VALID` registra a restrição e a aplica a toda linha nova ou alterada, sem verificar as existentes; o lock é breve. Depois, `VALIDATE CONSTRAINT` varre a tabela com `SHARE UPDATE EXCLUSIVE` (mais `ROW SHARE` na tabela referenciada, se for chave estrangeira), que não bloqueia `INSERT` nem `UPDATE`. A varredura demora o mesmo tanto, só que ninguém espera por ela. E, desde o PostgreSQL 12, `SET NOT NULL` pula a varredura se houver um `CHECK (col IS NOT NULL)` válido: o banco aceita a palavra de uma restrição já verificada, coisa que nenhum auditor faz.

`CREATE INDEX CONCURRENTLY` já apareceu no capítulo 5, quando o índice do Carlos derrubou o Pix numa quinta. Dois detalhes importam aqui: ele não aceita bloco de transação (no Alembic, exige o `autocommit_block()`) e, se falhar no meio, deixa para trás um índice `INVALID`, que não serve consultas e continua sendo atualizado em toda escrita. Um índice zumbi: morto para a leitura, vivo para a fatura.

Expand/contract, a migração em prestações

Mesmo com DDL instantâneo, sobra o código. Em `temp-01`, o reload troca os 3 workers um por vez, e por uns 40 segundos v1 e v2 atendem juntas. Se o deploy der errado, v1 volta para o schema que v2 deixou. O schema precisa servir duas versões de código ao mesmo tempo, sempre.

O padrão expand/contract (também chamado *parallel change*) quebra uma mudança incompatível numa sequência de mudanças compatíveis:

1. **Expand:** acrescentar a estrutura nova sem tirar a velha. Coluna nova, tabela nova, restrição `NOT VALID`.
2. **Migrar escritas:** deploy de código que escreve nos dois lugares.
3. **Backfill:** copiar o histórico para a estrutura nova.
4. **Migrar leituras:** deploy de código que lê da estrutura nova.
5. **Contract:** quando nenhuma versão viva usa mais a estrutura velha, removê-la.

É a mesma sequência de deprecar um parâmetro de função numa biblioteca: aceitar os dois nomes, avisar, migrar os chamadores, e só então remover o antigo. Ninguém remove um parâmetro e publica a biblioteca na mesma versão, a não ser quem quer receber issues em caixa alta.

Renomear `cliente_email` para `email` vira cinco passos em três deploys. O `RENAME COLUMN` direto também leva três: um para quebrar, um para tentar consertar e um de rollback às 4h. A compatibilidade:

	Schema antigo	Schema expandido	Schema contraído
Código v1	funciona	funciona	quebra
Código v2	quebra	funciona	funciona

A ordem segura sai daí: expand antes do deploy de v2, contract depois que v1 morreu e não pode voltar por rollback. Nenhuma combinação viva pode cair num "quebra".

Backfill em lotes, ou como atualizar 45 milhões de linhas sem ninguém perceber

O passo 3 é onde a migração costuma ficar gorda. A tentação é um `UPDATE` só:

```
UPDATE clientes SET email = cliente_email;  -- uma transação, 45 milhões de linhas
```

No MVCC (capítulo 3), cada linha atualizada ganha uma versão nova e a antiga vira tupla morta. Numa transação só, são 45 milhões de versões novas: a tabela quase dobra, o WAL sai numa enxurrada, o lock de cada linha fica até o `COMMIT` e o VACUUM não limpa nada mais novo que ela enquanto durar. Se falhar no minuto 40, o rollback é instantâneo, e os 40 minutos de tuplas mortas ficam lá, como cadáveres que ninguém vai reclamar.

O backfill em lotes troca uma transação gigante por milhares de transações pequenas:

```
-- lote de 5.000 pela chave primária; o próximo começa em 1210000
UPDATE clientes
SET email = cliente_email
WHERE id >= 1205000 AND id < 1210000
  AND email IS NULL;
-- UPDATE 4987
```

As propriedades que importam:

- **Lote pequeno por faixa da PK.** Cada lote termina em milissegundos e segura poucos locks. É a paginação por cursor do capítulo 6 aplicada a escrita.
- **Idempotente e retomável.** Com `email IS NULL`, repetir um lote não faz nada; o processo pode morrer na faixa 1.205.000 e recomeçar dali. Morrer é normal; ressuscitar sem duplicar é o que se pede.
- **Ritmo controlado.** A pausa entre lotes deixa o autovacuum limpar e as réplicas acompanharem o WAL (lag de replicação, capítulo 23). Uma hora ou uma noite custam as mesmas escritas; só uma aparece no gráfico de latência.
- **Escrita dupla antes do backfill.** Senão, toda linha alterada depois que o lote passou fica com o valor velho, a verificação acha 3 mil buracos e alguém descobre que a ordem dos passos tinha motivo.

Se a coluna não tem índice e a página tem espaço, o `UPDATE` é HOT (capítulo 5) e nem toca nos índices.

O ledger da Lastro aprende espanhol

O Mercadito Hornero vai cobrar em pesos argentinos, e o ledger precisa parar de supor que todo número é real. As restrições:

- Pix 24x7: não há janela de manutenção, nem às 3h de domingo, como foi demonstrado empiricamente.
- `lancamentos` tem ~45 milhões de linhas, ~3 GB de heap, ~4,5 GB de índices, e é imutável: o trigger "tabelião" do capítulo 1 recusa `UPDATE`. Backfill em `lancamentos` morre na porta do cartório.
- Valores seguem em `bigint` na unidade mínima (capítulo 1). O ISO 4217 define as casas decimais: BRL e ARS têm 2, o iene 0, o dinar do Bahrein 3. Uma tabela `moedas (codigo, casas_decimais)` guarda isso; o `numeric(20,4)` do Carlos não resolvia nem o dinar.

As alternativas:

Alternativa	Lock pesado	Disco extra	Pix durante	Custo
Migração do Carlos numa janela	`ACCESS EXCLUSIVE` por 6 a 10 min, mais a fila	~7,5 GB temporários	parado	proibida pelo 24x7
Tabela nova + cópia + troca de nome (o método do gh-ost e do pt-online-schema-change no MySQL)	segundos, na troca	~7,5 GB	segue	sincronizar 45 milhões de linhas com a tabela mais quente da empresa; troca ainda pede `ACCESS EXCLUSIVE`
Expand/contract com default constante	milissegundos por passo	quase zero	segue	duas migrações em dois deploys e disciplina de ordem

A Lastro fica com expand/contract: o 24x7 exige que nenhum passo segure `ACCESS EXCLUSIVE` por mais que o `lock_timeout`, e o default constante entrega isso. E encaixa na imutabilidade: todo lançamento histórico é em real, e o `attmissingval` diz isso sem `UPDATE` nenhum. O tabelião nem acorda.

A moeda vai para `contas` (uma moeda por conta; o vendedor argentino ganha contas em ARS) e é copiada em `lancamentos`. O capítulo 1 cobrou o custo de manter cópias; aqui o banco paga: a chave estrangeira composta `lancamentos (conta_id, moeda) → contas (id, moeda)` torna impossível um lançamento em pesos numa conta em reais. Em troca, o trigger de soma zero agrupa por `(transacao_id, moeda)` sem juntar com `contas`, a tabela mais disputada do ledger (capítulo 4). Câmbio, quando existir, serão duas pernas com uma conta de câmbio em cada moeda.

```
FASE 1  EXPAND  (migração 0043, antes do deploy)      serve v1 e v2
  moedas (codigo, casas_decimais): BRL 2, ARS 2
  contas.moeda       NOT NULL DEFAULT 'BRL'  FK -> moedas   (~9 mil linhas)
  índice único (id, moeda) em contas: CONCURRENTLY + UNIQUE USING INDEX
  lancamentos.moeda  NOT NULL DEFAULT 'BRL'                 (só catálogo)
  FK (conta_id, moeda) -> contas (id, moeda)  NOT VALID
  VALIDATE CONSTRAINT                          (~4 min, sem bloquear)
  trigger de soma zero por (transacao_id, moeda)

FASE 2  DEPLOY v2                                       v1 e v2 convivem
  v2 grava moeda explicitamente; janela de rollback: 7 dias

FASE 3  USO
  contas ARS do Mercadito Hornero e de seus vendedores

FASE 4  CONTRACT  (migração 0044, só depois que v1 não pode voltar)
  lancamentos.moeda  DROP DEFAULT
  DROP da FK simples conta_id -> contas (redundante com a composta)
```

Cada passo usa `lock_timeout = '1s'`, até cinco tentativas com espera crescente e `statement_timeout` próprio. O `VALIDATE` lê os 45 milhões de linhas por uns 4 minutos (estimativa) com pagamentos entrando, porque a FK `NOT VALID` já vale para toda linha nova. Nenhuma migração começa com transação longa em `lancamentos`, e o calendário do fechamento mensal entrou no checklist.

O `DROP DEFAULT` da fase 4 existe porque, com contas ARS, um caminho esquecido que insere sem moeda gravaria `BRL` por omissão; a FK composta recusaria, mas o erro de `NOT NULL` é mais claro. O histórico não muda: o `attmissingval` continua lá. O caminho esquecido tem nome: `repasse_final_v2_agora_vai.py`, que roda pelo cron com a própria cópia do código, atualizada quando alguém pensa nele. Na fase 4, ele quebra em vez de pagar um argentino em reais. Pela primeira vez na história, o job vai falhar do jeito certo.

Com janela de manutenção (um produto só de cartão em horário comercial, por exemplo), a migração direta seria mais simples e o risco, aceitável. Se fosse preciso mudar o tipo de `valor` de verdade, o default não ajudaria, o backfill esbarraria na imutabilidade, e a cópia para tabela nova com troca de nome passaria a ser a opção viável, com todo o seu custo.

Armadilhas e casos reais

> **Armadilha:** o autovacuum comum cede: se ele bloqueia o lock de um DDL, o PostgreSQL o cancela. O autovacuum de prevenção de *wraparound* de ID de transação não cede. Sua migração entra na fila atrás dele, os pagamentos entram na fila atrás da migração, e o log mostra `(to prevent wraparound)`, a única mensagem do PostgreSQL que soa como ameaça.
>
> **Armadilha:** o SQLAlchemy lista as colunas do model em todo `SELECT` e `INSERT`. Remova uma coluna que v1 ainda mapeia e cada requisição de v1 morre com `column "x" does not exist`, em ordem de chegada. Primeiro o deploy que deixa de mapear, depois o `DROP`.
>
> **Armadilha:** o `CREATE INDEX CONCURRENTLY` que falhou por violação de unicidade deixa o índice `INVALID` atualizado em toda escrita, sem servir consulta nenhuma. `\d tabela` mostra a palavra `INVALID` no fim da linha. Ninguém olha. O disco olha.
>
> **Na vida real:** o GitHub criou o gh-ost, migração online para MySQL, porque ferramentas baseadas em triggers, como o pt-online-schema-change da Percona, somavam carga e disputa de locks às tabelas mais movimentadas. O gh-ost lê o log binário em vez de usar triggers. Os dois copiam a tabela e trocam o nome no fim, a alternativa que a Lastro descartou.
>
> **Na vida real:** a gem strong_migrations, do Rails, recusa por padrão operações perigosas no PostgreSQL (default volátil, `ALTER COLUMN TYPE`, índice sem `CONCURRENTLY`) e indica a sequência segura de cada uma. A documentação dela vira checklist de revisão em qualquer linguagem, inclusive em Python, que ela ignora com toda razão.

O Carlos, para constar, propôs aplicar a migração 0043 "na hora do almoço, que é tranquilo". O almoço é o pico de Pix do Bazar.

Resumo

- `ALTER TABLE` quase sempre pede `ACCESS EXCLUSIVE`; o DDL esperando bloqueia quem chega depois. `lock_timeout` curto e retentativa.
- DDL custa catálogo, varredura ou reescrita; default constante é catálogo (PostgreSQL 11+); mudança de tipo reescreve.
- `NOT VALID` + `VALIDATE`; `SET NOT NULL` via `CHECK`; índices `CONCURRENTLY`.
- Expand/contract: schema compatível com duas versões de código.

- Backfill em lotes pela PK, idempotente, retomável, depois da escrita dupla.

A Lastro agora

O ledger tem moeda: `moedas` com as casas decimais do ISO 4217, `contas.moeda` e `lancamentos.moeda` com default `'BRL'` só no catálogo, FK composta `(conta_id, moeda)` e soma zero por moeda. Toda migração usa `lock_timeout` de 1 segundo, retentativa, checagem de transações longas, e expand e contract em deploys separados; a 0044 espera 7 dias de rollback. O Pix não percebeu nada.

O gerente acertou: era só uma coluna. Foram duas migrações, dois deploys, uma FK composta, um trigger novo e 47 Pix conciliados à mão, mas era só uma coluna. Agora o Mercadito Hornero reclama que cada chamada à API leva 300 ms a mais do que o processamento, e o fundador quer trocar `temp-01` por um servidor novo: "é só mudar o IP no DNS". É só um registro.

Referências

- PostgreSQL 16, `ALTER TABLE` (locks por subcomando, `NOT VALID`, `VALIDATE CONSTRAINT`, `SET NOT NULL`): https://www.postgresql.org/docs/16/sql-altertable.html
- PostgreSQL 16, locks explícitos e conflitos entre modos: https://www.postgresql.org/docs/16/explicit-locking.html
- PostgreSQL 16, `CREATE INDEX` (construção concorrente e índices inválidos): https://www.postgresql.org/docs/16/sql-createindex.html
- PostgreSQL 16, parâmetros `lock_timeout` e `statement_timeout`: https://www.postgresql.org/docs/16/runtime-config-client.html
- PostgreSQL 16, autovacuum e prevenção de wraparound: https://www.postgresql.org/docs/16/routine-vacuuming.html
- PostgreSQL 11, notas de versão (`ADD COLUMN` com default não volátil sem reescrita): https://www.postgresql.org/docs/release/11.0/
- Danilo Sato, "ParallelChange": https://martinfowler.com/bliki/ParallelChange.html
- ISO 4217, códigos de moeda: https://www.iso.org/iso-4217-currency-codes.html
- Alembic, `MigrationContext.autocommit_block`: https://alembic.sqlalchemy.org/en/latest/api/runtime.html
- gh-ost: https://github.com/github/gh-ost
- strong_migrations: https://github.com/ankane/strong_migrations

PARTE II

A API no mundo

CAPÍTULO 8

TCP/IP e DNS

O Mercadito Hornero mandou um gráfico. Cliente que manda gráfico já desistiu de ser educado e está montando o processo.

A linha de baixo é o processamento que a API da Lastro informa em cada resposta: 40 ms. A de cima é o que o cliente do Hornero mede para a mesma chamada: 340 ms. Trezentos milissegundos somem no caminho, e o Bazar Bem-te-vi, na mesma API, nunca reclamou. O Bazar reclama de tudo. Quando o Bazar não reclama, o problema existe só para os outros.

Na mesma reunião, o fundador anunciou que vai trocar `temp-01` por um servidor novo, `api-01`: "é só mudar o IP no DNS, coisa de cinco minutos". O Carlos concordou, e o financeiro começou a separar café para o fim de semana.

Para onde vão 300 ms que nenhum código da Lastro gasta, e por que "só mudar o IP no DNS" pode deixar o Pix parado por um dia inteiro?

Camadas, ou como 300 bytes de JSON viajam com mais cabeçalho do que conteúdo

O `POST /v1/cobrancas` do Hornero sai como um JSON de 300 bytes e desce pelas quatro camadas do modelo TCP/IP, cada uma com seu cabeçalho: aplicação (HTTP, DNS; o TLS fica logo abaixo), transporte (TCP, UDP: dados entre processos), internet (IP: pacotes entre máquinas) e enlace (Ethernet: quadros entre vizinhos). O modelo OSI tem sete; as camadas 5 e 6 só existem em entrevista de emprego, onde ninguém lembra o nome delas e todo mundo finge.

Um quadro Ethernet carrega até 1.500 bytes de pacote IP (a *MTU*). Tirando 20 bytes de cabeçalho IPv4 e 20 de TCP, sobram 1.460 de dados por segmento (o *MSS*). Os 300 bytes do Hornero cabem num segmento só, com folga para mais quatro JSONs e um pedido de desculpas.

IP, portas e o endereço que ninguém tem de verdade

O IP entrega pacotes entre endereços em regime de *best effort*: o pacote pode se perder, duplicar ou chegar fora de ordem, e o IP não pede desculpa por nenhum dos três. Cada roteador decrementa o campo TTL do pacote (*hop limit* no IPv6); em zero, o pacote é descartado e a origem recebe um ICMP *Time Exceeded*. O `traceroute` manda pacotes com TTL 1, 2, 3 e anota quem assinou cada atestado de óbito. O DNS tem outro TTL, que mata registro em vez de pacote; os dois se encontram neste capítulo.

O IPv4 tem 32 bits, uns 4,3 bilhões de endereços, e o estoque central da IANA acabou em 2011. O IPv6 tem 128 bits, o suficiente para o Carlos criar ambientes de teste até a morte térmica do universo sem apagar nenhum.

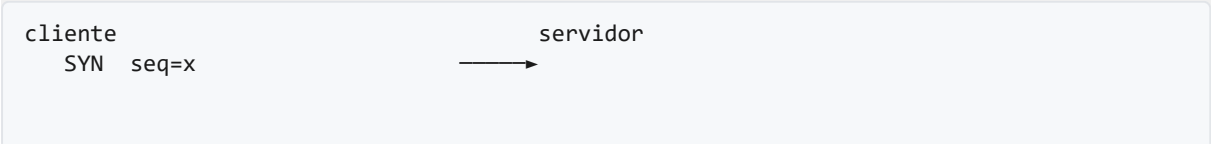
Faixas se escrevem em **CIDR**: endereço e quantos bits iniciais formam o prefixo. `10.0.1.0/24` deixa 8 bits livres, 256 endereços; `/32` é um endereço só, a notação que todo firewall de parceiro pede. O fundador uma vez sugeriu ao banco liberar `0.0.0.0/0` "pra simplificar". É a internet inteira. O banco respondeu com um silêncio que durou duas semanas e uma auditoria.

A RFC 1918 reserva três faixas **privadas**, que a internet não roteia: `10.0.0.0/8`, `172.16.0.0/12` e `192.168.0.0/16`. Máquina privada sai por **NAT**: a borda troca IP e porta de origem pelo seu IP **público** e anota o par numa tabela para traduzir a resposta. De fora, ninguém inicia conexão com quem está atrás dela. E a tabela esquece: mapeamento ocioso é removido e o próximo pacote leva RST (no NAT Gateway da AWS, após 350 segundos). A conexão estava viva para as duas pontas e morta para o meio, um casamento em que o cartório perdeu a certidão.

Dentro da máquina, quem recebe é uma **porta** de 16 bits: o servidor escuta numa conhecida (443), o cliente ganha uma **efêmera** (no Linux, de 32768 a 60999). Uma conexão TCP é a quíntupla protocolo, IP e porta de origem, IP e porta de destino. Troque qualquer um dos cinco e nasce outra conexão, com outro handshake, que é para onde foram os 300 ms.

O handshake, ou como dois computadores precisam de três mensagens para dizer oi

O TCP entrega bytes confiáveis e ordenados sobre um IP que perde e embaralha. Antes, as pontas combinam números de sequência:



```
                                ◄──────  SYN-ACK  seq=y  ack=x+1
  ACK  ack=y+1  (já pode levar dados) ──────►
```

O número inicial é aleatório (RFC 6528), para que ninguém de fora adivinhe a sequência e injete bytes na conversa. Cada byte tem número; o receptor confirma com ACK cumulativo ("recebi tudo até o byte N"). Segmento sem confirmação dentro do *retransmission timeout* (RTO, calculado do RTT medido, RFC 6298) é reenviado; três ACKs duplicados disparam a retransmissão antes do timeout.

O custo é um RTT inteiro antes do primeiro byte de dados. É o protocolo mais inseguro do mundo: precisa de três mensagens para ter certeza de que o outro está a fim, e ainda assim às vezes o outro não estava ouvindo.

Sem resposta, o Linux reenvia o SYN 6 vezes (`tcp_syn_retries`), dobrando a espera a partir de 1 segundo: 127 segundos até desistir. Dois minutos mandando mensagem para quem já bloqueou você. Cliente sem *connect timeout* espera isso tudo (capítulo 15).

Efeito colateral da ordem: perdido o segmento 4, os de 5 a 40 esperam no buffer do kernel pela retransmissão. Esse *head-of-line blocking* é o motivo de o HTTP/3 ter largado o TCP (capítulo 9).

Controle de fluxo e de congestionamento, ou o TCP pisando no freio por dois motivos diferentes

O **controle de fluxo** protege o receptor: cada ACK anuncia a janela de recepção (*rwnd*), quantos bytes ainda cabem no buffer de quem recebe (16 bits, 64 KiB; até 1 GiB com *window scaling*, RFC 7323). O **controle de congestionamento** protege a rede, e a janela dele (*cwnd*) mora só na cabeça do emissor, que mantém em trânsito no máximo `min(cwnd, rwnd)` bytes sem confirmação:

```
vazão máxima ≈ janela / RTT
64 KiB / 100 ms ≈ 655 KB/s ≈ 5,2 Mbit/s   (com um link de 1 Gbit/s, tanto faz)
```

Conexão nova começa em *slow start*: janela inicial de 10 segmentos (RFC 6928), uns 14,6 KB, dobrando a cada RTT até uma perda ou um limiar. Uma resposta de 200 KB numa conexão recém-aberta leva quatro idas e voltas: 14,6 KB, mais 29,2, mais 58,4, mais 116,8. Quem batizou crescimento exponencial de "slow start" também chamaria o Carlos de "cauteloso".

Na perda, o CUBIC (RFC 9438), padrão do Linux, multiplica a janela por 0,7 e volta a crescer por uma função cúbica do tempo; o BBR estima banda e RTT mínimo em vez de esperar perda. E conexão reaproveitada nem sempre é rápida: com

`tcp_slow_start_after_idle = 1`, o padrão do Linux, a janela volta ao valor inicial após um RTO de ociosidade. Conexão parada esquece que já foi rápida, como o Carlos depois das férias.

Fim de conexão e TIME_WAIT, o luto obrigatório

Para fechar, cada direção manda o seu FIN e recebe ACK: quatro segmentos. Quem fecha primeiro entra em **TIME_WAIT** por 2 × MSL (vida máxima de um segmento; a RFC 9293 sugere MSL de 2 minutos, o Linux usa 60 segundos fixos de TIME_WAIT). Serve para reenviar o último ACK se ele se perder e para que um segmento atrasado da conexão morta não seja aceito por uma nova com a mesma quíntupla. O corpo fica velado um minuto, caso algum parente distante apareça com bytes atrasados.

Cada conexão em TIME_WAIT segura uma porta efêmera. Com 28.232 portas e 60 segundos de luto, um cliente que abre conexão nova a cada chamada para o mesmo destino sustenta umas 470 por segundo antes de o `connect()` falhar com `EADDRNOTAVAIL`. O cemitério lota antes da fila de vivos. Já o `RST` é morte sem velório: aborta na hora, e o kernel o manda para segmento de conexão que ele não conhece, como aquela que o NAT esqueceu.

UDP, o protocolo que manda e vai embora

O UDP põe 8 bytes de cabeçalho (portas, tamanho, checksum) no datagrama e entrega ao IP. Sem conexão, retransmissão, ordem ou controle de congestionamento; datagrama perdido é problema da aplicação. É a rapidinha da pilha de rede: zero preliminar, zero garantia, e ninguém liga no dia seguinte. Serve a DNS, NTP (capítulo 20), mídia em tempo real e ao QUIC, que refaz confiabilidade e congestionamento por cima do UDP para escapar do head-of-line blocking (capítulo 9).

RTT, ou a velocidade da luz não aceita chamado com "URGENTE!!!!"

Luz em fibra anda a uns 200 mil km/s, dois terços da velocidade no vácuo: 1 ms de RTT a cada 100 km de fibra, no mínimo físico. A rota real é mais longa que a linha reta, e cada roteador soma fila.

Os servidores do Hornero ficam numa região de nuvem na Virgínia, nos Estados Unidos. `temp-01` está num datacenter em São Paulo, a uns 7.600 km em linha reta: pelo menos 76 ms de RTT, mesmo esticando um cabo reto por cima da Amazônia e do Caribe. O

Hornero mede 100 ms. O Bazar, em São Paulo, mede 2 ms. E o cliente do Hornero abre conexão nova em toda chamada:

```
RTT 100 ms        Hornero (Virgínia)          temp-01 (São Paulo)
   0 ms  SYN                          ──────────►
 100 ms                               ◄──────────  SYN-ACK          TCP: 1 RTT
         ACK + ClientHello            ──────────►
 200 ms                               ◄──────────  ServerHello...   TLS 1.3: 1 RTT
         Finished + POST              ──────────►
 300 ms                               ◄──────────  201 Created      HTTP: 1 RTT
         (fecha, e a próxima chamada recomeça do SYN)
```

Três RTTs, 300 ms, mais 40 de processamento: o gráfico, centavo por centavo (o handshake TLS fica para o capítulo 10). O Bazar comete o mesmo pecado, mas a 2 ms de distância o pecado custa 6 ms e ninguém confessa.

DNS, o banco de dados distribuído que todo mundo esquece que é um banco de dados

O DNS traduz nomes em registros, e a resolução atravessa uma hierarquia de servidores:

```
aplicação ─► stub resolver (getaddrinfo, no SO)
               ─► resolver recursivo (do provedor, da nuvem, 1.1.1.1...)
                    ─► raiz       "para .br, pergunte a estes"
                    ─► TLD .br    "para lastro.com.br, pergunte a estes"
                    ─► autoritativo de lastro.com.br   "198.51.100.7, TTL 86400"
```

O **autoritativo** guarda a zona e responde por ela. O **recursivo** faz a caminhada, guarda cada resposta em cache e devolve ao stub do sistema operacional. Consultas vão por UDP na porta 53; resposta grande demais volta truncada e é refeita por TCP, que todo servidor DNS precisa suportar (RFC 7766).

Os registros que um backend usa: A e AAAA (endereço IPv4 e IPv6), CNAME (apelido para outro nome), MX (e-mail), TXT (texto livre: SPF, verificação de domínio), NS (quem é autoritativo pela zona) e SOA (metadados da zona, inclusive o TTL das respostas negativas).

Cada registro tem **TTL** em segundos: quanto tempo um cache pode reutilizar a resposta. Dois minutos depois, o mesmo recursivo devolve o registro com o TTL descontado:

```
$ dig +noall +answer api.lastro.com.br
api.lastro.com.br.  86400   IN  A   198.51.100.7

$ dig +noall +answer api.lastro.com.br     # 2 minutos depois
api.lastro.com.br.  86280   IN  A   198.51.100.7
```

Esse é o número que o fundador escolheu, um dia, "pra não ficar consultando à toa". Mudar o registro muda o autoritativo e só ele; cada recursivo do planeta com a resposta em cache segue entregando o IP antigo até o seu TTL acabar.

O mercado chama isso de "propagação de DNS", como se o registro novo saísse em turnê pelo mundo. Nada propaga. Caches vencem, cada um no seu tempo. A única propagação real é do autoritativo primário para os secundários, por transferência de zona guiada pelo serial do SOA.

E há quem guarde além do TTL. A JVM guarda por 30 segundos por padrão e, com *security manager*, para sempre, porque o Java nunca soube largar nada. Processo que resolve o nome só ao iniciar usa aquele IP até reiniciar, e conexão TCP aberta não resolve nada de novo.

Na Lastro: 200 ms recuperados e uma troca de IP de duas semanas

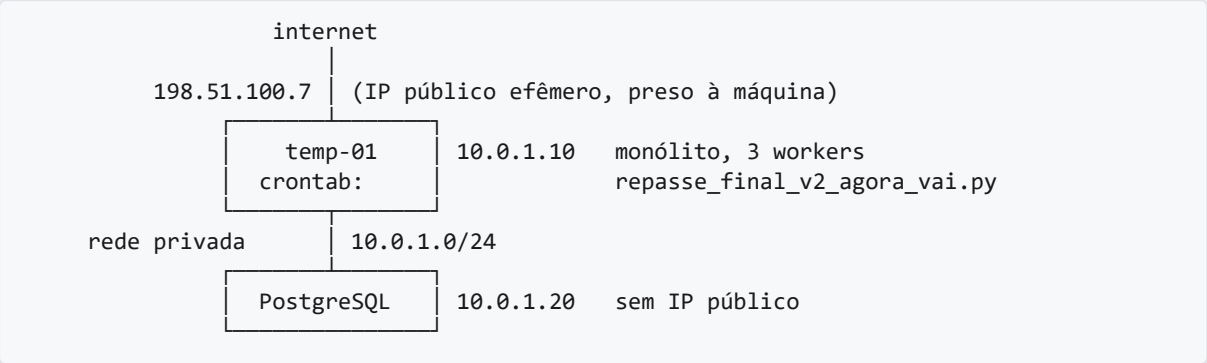
Os 300 ms do Hornero

Opção	Latência por chamada	Custo
Reuso de conexão no cliente, *keep-alive* longo no servidor	3 RTTs → 1 (300 → 100 ms)	conexões ociosas ocupam memória e descritor no servidor
TLS 0-RTT na retomada	3 → 2 RTTs	dados 0-RTT podem ser repetidos por atacante; inaceitável para criar cobrança (capítulo 10)
API numa região perto do Hornero	~100 ms a menos até a API	o banco fica em São Paulo: cada consulta passa a pagar 100 ms

A Lastro adota a primeira. O Hornero passa a usar pool de conexões, e o servidor ASGI de `temp-01` sobe o *keep-alive* de 5 segundos (padrão do Uvicorn) para 65. O guia de integração pede que o cliente feche conexões ociosas antes de 60 segundos. O cliente fecha primeiro de propósito: no caso contrário, o servidor fecha no instante em que o cliente manda uma requisição, o cliente leva um RST e não sabe se a cobrança foi criada (capítulo 16).

Resultado: 140 ms por chamada, 100 de física e 40 de processamento. O gerente perguntou se dava para pedir ao provedor "uma fibra mais rápida". O Carlos anotou o pedido.

A troca de `temp-01`



O PostgreSQL mora em máquina própria, só com IP privado, e o `pg_hba.conf` aceita `10.0.1.0/24`. `api-01` entra na mesma rede e fala com o banco a 0,3 ms. Como o capítulo 2 pôs a verificação de saldo no banco, duas máquinas atendendo juntas é seguro; o vendedor 42 vai precisar de uma sexta aba.

O IP fixo do banco parceiro. O banco que dá acesso ao Pix só aceita conexões de IPs cadastrados no firewall dele: hoje, `198.51.100.7/32`. Esse IP é efêmero, atribuído pelo provedor à máquina, intransferível e perdido se ela for destruída. O medo coletivo de desligar `temp-01` acaba de ganhar fundamento técnico. A decisão: reservar um IP público fixo e transferível entre máquinas (um *Elastic IP* na AWS ou um IP externo estático no Google Cloud, por exemplo), `203.0.113.10`, e pedir ao banco que o cadastre **ao lado** do antigo. Prazo do banco: até 10 dias úteis e um formulário com firma reconhecida. Depois disso, trocar de máquina é mover o IP, sem cadastro novo.

O TTL de um dia. Mudar o registro A com TTL 86400 deixa até 24 horas de clientes divididos entre as duas máquinas, sem controle de quem. A sequência:

Quando	Passo
D−14	reserva de `203.0.113.10`; pedido ao banco parceiro
D−2	TTL de `api.lastro.com.br` cai para 300; espera de 24 h, o TTL antigo
D−1	banco confirma; `api-01` sobe com o IP reservado; Pix de teste
D0	registro A → `203.0.113.10`; em 5 minutos os caches vencem; `temp-01` responde com `Connection: close` para que conexões keep-alive (as do Hornero, recém-convertido) reconectem e resolvam o nome de novo

O TTL fica em 300 segundos: o requisito é apontar a API para outra máquina em minutos se `api-01` cair, ao custo de uma consulta ao autoritativo a cada 5 minutos por recursivo. TTL de horas serve a registros estáveis (MX, TXT) e sobrevive a uma queda do autoritativo.

Uma semana depois, `temp-01` ainda recebe tráfego, de duas fontes. Uma integração do Bazar que resolveu o nome ao iniciar, há 200 dias, e nunca reiniciou. E o `repasse_final_v2_agora_vai.py`, no crontab de `temp-01`, com sua cópia particular do código, chamando o banco a partir de `198.51.100.7`. O IP antigo fica no cadastro do banco, e `temp-01` fica ligado. O fundador disse "eu avisei". Tecnicamente avisou, só que de outra coisa.

Armadilhas e casos reais

> **Armadilha:** consultar o nome antes de criá-lo. "Não existe" (`NXDOMAIN`) também vai para cache, pelo tempo do SOA da zona (RFC 2308). O Carlos testou `api-01.lastro.com.br` dez minutos antes de criar o registro, o resolvedor guardou a inexistência por uma hora, e ele passou essa hora reiniciando a máquina errada.
>
> **Armadilha:** CNAME no ápice da zona. Nome com CNAME não pode ter outros registros (RFC 1034), e o ápice sempre tem SOA e NS. Provedores oferecem registros proprietários (ALIAS, *CNAME flattening*) que resolvem o alvo no servidor e respondem com A.
>
> **Na vida real:** em 4 de outubro de 2021, um comando de manutenção desconectou o *backbone* do Facebook dos datacenters. Os servidores DNS, feitos para retirar seus anúncios de rota (BGP) quando perdem contato com os datacenters, fizeram exatamente isso, e Facebook, Instagram e WhatsApp sumiram da resolução de nomes por cerca de seis horas. Um mecanismo de segurança que funcionou com perfeição, contra a própria empresa.
>
> **Na vida real:** em 19 e 20 de outubro de 2025, uma condição de corrida na automação que gerencia o DNS do DynamoDB, na região us-east-1 da AWS, deixou vazio o registro de `dynamodb.us-east-1.amazonaws.com`. O serviço estava de pé; ninguém achava o endereço. A condição de corrida do capítulo 2 sacou o dobro de um vendedor; esta sacou o registro inteiro e deixou saldo zero: um nome com nenhum IP.

Resumo

- IP *best effort*; CIDR; faixas privadas; NAT esquece conexões ociosas.
- TCP: quíntupla, handshake de 1 RTT, retransmissão, head-of-line blocking.
- Fluxo protege o receptor, congestionamento protege a rede; vazão ≤ janela / RTT.
- TIME_WAIT (60 s no Linux) prende portas de quem fecha primeiro.
- UDP: sem conexão nem garantias.
- RTT: ~1 ms a cada 100 km de fibra; conexão nova com TLS 1.3 custa 3 RTTs.
- DNS: recursivo, autoritativo, TTL e cache (inclusive negativo); nada propaga, caches vencem.

A Lastro agora

O Hornero reutiliza conexões e mede 140 ms por chamada. A API responde em `api-01`, com o IP reservado `203.0.113.10` cadastrado no banco parceiro e TTL de 300 segundos. `temp-01` continua ligado, com o IP antigo no firewall do banco, servindo uma integração do Bazar que não reinicia e o job de repasse que ninguém migrou.

O fundador queria trocar de servidor em cinco minutos. Levou duas semanas, um formulário com firma reconhecida, e a Lastro terminou com dois servidores. `temp-01` tem agora dois motivos técnicos para viver, nenhum para morrer, e ganhou um irmão. Enquanto isso, o Carlos descobriu que o estorno do painel é um `GET` e achou prático: "dá pra estornar colando o link no navegador". Qualquer coisa que siga links também acha.

Referências

- RFC 791. *Internet Protocol*. https://www.rfc-editor.org/rfc/rfc791
- RFC 8200. *Internet Protocol, Version 6 (IPv6) Specification*. https://www.rfc-editor.org/rfc/rfc8200
- RFC 9293. *Transmission Control Protocol (TCP)*. https://www.rfc-editor.org/rfc/rfc9293
- RFC 768 (UDP). https://www.rfc-editor.org/rfc/rfc768
- RFC 6298 (RTO). https://www.rfc-editor.org/rfc/rfc6298
- RFC 6528 (números de sequência). https://www.rfc-editor.org/rfc/rfc6528
- RFC 7323 (window scaling). https://www.rfc-editor.org/rfc/rfc7323
- RFC 6928 (janela inicial). https://www.rfc-editor.org/rfc/rfc6928
- RFC 9438 (CUBIC). https://www.rfc-editor.org/rfc/rfc9438

- RFC 1918 (endereços privados). https://www.rfc-editor.org/rfc/rfc1918
- RFC 4632 (CIDR). https://www.rfc-editor.org/rfc/rfc4632
- RFC 1034 e RFC 1035 (DNS). https://www.rfc-editor.org/rfc/rfc1034, https://www.rfc-editor.org/rfc/rfc1035
- RFC 2308 (cache negativo). https://www.rfc-editor.org/rfc/rfc2308
- RFC 7766 (DNS sobre TCP). https://www.rfc-editor.org/rfc/rfc7766
- Linux, "IP Sysctl". https://docs.kernel.org/networking/ip-sysctl.html
- Uvicorn, "Settings". https://www.uvicorn.org/settings/
- AWS, "Troubleshoot NAT gateways". https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-troubleshooting.html
- Meta, "More details about the October 4 outage". https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
- AWS, resumo da interrupção do DynamoDB em us-east-1 (outubro de 2025). https://aws.amazon.com/message/101925/

CAPÍTULO 9

HTTP em profundidade

O link que estornava sozinho

Na quarta-feira, às 10h12, o Carlos colou um link no canal do suporte para provar ao financeiro como o painel da Lastro é prático. O link era `https://painel.lastro.com.br/cobrancas/0190c3a1.../estornar?valor=129900`. Três segundos depois, o robô de pré-visualização do chat abriu o link para gerar a miniatura, e a Capinhas Hiper Ltda. perdeu R$ 1.299,00 de uma venda de R$ 2.598,00 (um kit de 200 capinhas, porque a Capinhas nunca vende uma capinha só).

O financeiro encaminhou o link à contabilidade com o assunto "isso é normal?". O antivírus do e-mail abriu o link para ver se tinha malware. Tinha: estornou os outros R$ 1.299,00.

O Carlos abriu o link de novo para mostrar que estava tudo sob controle, levou "sem valor estornável" e chamou isso de "a validação funcionando". O tabelião do ledger registrou os dois estornos com perfeição, e nenhum foi pedido por um ser humano. O Bazar abriu o chamado às 10h31, "URGENTE!!!!", com o corpo "quem estornou?". A resposta honesta era "um robô e um antivírus", e ninguém teve coragem de mandar.

E a Lastro está abrindo a API de cobranças para mais marketplaces, começando a aceitar cartão por um adquirente e levando o checkout para o navegador do comprador. O que o HTTP promete sobre cada requisição, e como uma API de dinheiro se apoia nessas promessas em vez de ser atropelada por elas?

HTTP/1.1, ou texto puro com pretensão de protocolo

O HTTP/1.1 (RFC 9112 no fio, RFC 9110 na semântica) é texto sobre TCP: linha de pedido, cabeçalhos, linha em branco, corpo opcional.

```
POST /v1/cobrancas HTTP/1.1
Host: api.lastro.com.br
Content-Type: application/json
Content-Length: 59

{"valor": 259800, "metodo": "pix", "pedido_id": "BZ-88412"}
```

O fim da mensagem vem de `Content-Length`, com o número exato de bytes, ou de `Transfer-Encoding: chunked`, com pedaços prefixados pelo tamanho e um pedaço final de tamanho zero. Quando proxy e servidor discordam sobre qual vale, nasce o *request smuggling*: uma requisição escondida no corpo da outra. Um protocolo de 1997 com ambiguidade no "onde acaba". Igual reunião de planejamento.

A conexão é persistente por padrão (o *keep-alive* do capítulo 8), porém monogâmica: uma requisição por vez. O *pipelining* exige respostas na ordem dos pedidos, e uma resposta lenta segura as de trás. Esse *head-of-line blocking* fez os navegadores desligarem o pipelining e abrirem seis conexões por host. Uma geração de engenheiros otimizou páginas criando subdomínios falsos (`img1`, `img2`, `img3`) para enganar o limite, e ninguém foi preso.

HTTP/2: cem conversas numa conexão só, e todas morrem juntas

O HTTP/2 (RFC 9113) mantém a semântica e troca o formato no fio: *frames* binários com cabeçalho de 9 bytes (tamanho, tipo, flags e o ID do *stream*, uma troca requisição-resposta). Streams do cliente têm IDs ímpares crescentes, e os frames de vários se intercalam. Isso é a **multiplexação**: cem requisições em voo numa conexão TCP, cada uma terminando quando terminar.

`SETTINGS` negocia limites, como `SETTINGS_MAX_CONCURRENT_STREAMS` (a RFC recomenda não menos que 100); `WINDOW_UPDATE` faz controle de fluxo por stream; `RST_STREAM` cancela um stream sem matar a conexão; `GOAWAY` encerra a conexão com o ID do último stream que o servidor processou ou vai processar. Streams com ID maior nunca foram tocados e podem ir para outra conexão, mesmo um `POST` de cobrança. É o único momento do HTTP em que o servidor diz com certeza "isso eu não fiz". No capítulo 16 você vai sentir saudade dele.

Os cabeçalhos são comprimidos com **HPACK** (RFC 7541): tabela estática de 61 entradas, tabela dinâmica por conexão (4.096 bytes por padrão) e Huffman. Na segunda requisição, o `Authorization` de 200 bytes vira um ou dois. O HPACK evita DEFLATE porque a compressão genérica vazou cookies no ataque CRIME, em 2012. O HTTP/2 aprendeu com a morte dos outros, o único método de ensino desta profissão.

A parte triste: todos os streams viajam num fluxo TCP, que entrega bytes em ordem, e um pacote perdido segura todos até a retransmissão. O HTTP/2 trocou seis conexões que travavam sozinhas por uma que trava todo mundo. Coletivismo de rede.

```
Stream 1 (autorização R$ 89,90):  pronta, esperando
Stream 3 (autorização R$ 12,00):  pronta, esperando
```

```
Stream 5 (estorno R$ 1.299,00):   pronta, esperando
TCP:     perdi o segmento 4.812. ninguém sai daqui até ele chegar.
Stream 5: mas eu não estava no segmento 4.812.
TCP:     solidariedade.
```

HTTP/3: o QUIC desistiu do TCP e foi morar no UDP

O HTTP/3 (RFC 9114) roda sobre o QUIC (RFC 9000), um transporte confiável sobre UDP. O QUIC refaz no espaço do usuário o que o TCP faz no kernel (confirmação, retransmissão, controle de congestionamento), com streams independentes no transporte: pacote perdido no stream 5 segura só o stream 5. Os cabeçalhos usam QPACK (RFC 9204).

O TLS 1.3 vem embutido no handshake: conexão nova custa 1 RTT, em vez dos 2 de TCP mais TLS. A primeira chamada do Hornero cairia de 3 RTTs para 2 (capítulo 8). O 0-RTT continua com o risco de *replay* que a Lastro já recusou, e reenviar criação de cobrança o vendedor 42 já faz sem ajuda de protocolo.

A conexão QUIC é identificada por um *connection ID*. O celular do comprador troca do Wi-Fi para o 4G no meio do checkout e a conexão sobrevive; no TCP, a quíntupla mudou, a conexão morreu, e o comprador desistiu da compra e do marketplace. O cliente descobre o HTTP/3 pelo cabeçalho `Alt-Svc` ou pelo registro DNS `HTTPS` (RFC 9460), e volta para TCP quando a rede bloqueia UDP na porta 443, porque o firewall de 2009 acha que todo UDP é torrent.

Métodos, ou a promessa que todo robô da internet cobra do seu GET

A RFC 9110 define duas propriedades de método que explicam o estorno do Carlos inteiro. Um método é **seguro** (*safe*) quando a semântica é só leitura: o cliente não pede mudança de estado. É **idempotente** quando o efeito pretendido de N requisições idênticas no servidor é o mesmo de uma. A resposta pode variar: um `DELETE` repetido devolve `204` e depois `404`, e o recurso está apagado nos dois casos.

Método	Seguro	Idempotente
`GET`, `HEAD`, `OPTIONS`	sim	sim
`PUT`, `DELETE`	não	sim
`POST`, `PATCH`	não	não

Essas propriedades são um contrato com o resto da internet. Robôs de pré-visualização, scanners de e-mail e o *prefetch* do navegador fazem `GET` em qualquer link; clientes e proxies reenviam sozinhos requisições idempotentes quando a conexão cai, porque a RFC autoriza. O Carlos quebrou a primeira promessa, e a internet cobrou a parte dela com a eficiência de um oficial de justiça.

A RFC aceita efeitos colaterais num `GET` (log, contador), desde que o cliente não os tenha pedido nem seja responsabilizado por eles. Estornar R$ 1.299,00 foi um efeito colateral pelo qual o Carlos foi muito responsabilizado.

O `POST` não promete nada. Se a conexão caiu depois do envio, o cliente não sabe se a cobrança foi criada; a chave de idempotência, que dá ao `POST` a propriedade que o método não tem, é o capítulo 16.

Status, ou três dígitos para dizer o que aconteceu

O status é a parte da resposta que intermediários e bibliotecas entendem sem ler o corpo, e ele dirige comportamento automático: um `503` com `Retry-After` convida a tentar de novo, um `422` avisa que repetir dá o mesmo erro. Os que uma API de pagamentos usa, na RFC 9110:

- `201 Created`, com o recurso novo em `Location`; `202 Accepted`, aceito para um processamento que ainda não terminou. A RFC chama o `202` de "intencionalmente evasivo", o status mais honesto do protocolo.
- `307` e `308` mantêm o método; `301` e `302` deixam, por histórico, o cliente trocar `POST` por `GET`, e a criação de cobrança vira consulta. Silenciosamente.
- `401` (sem credencial válida), `403` (sem permissão), `405` (com `Allow`), `409` (conflito com o estado do recurso), `412`, `422`, `429` (capítulo 28); `502` (o proxy recebeu lixo de trás), `503`, `504` (o proxy cansou de esperar).

E o `402 Payment Required` está "reservado para uso futuro" desde a RFC 2068, de 1997. Quase trinta anos de uso futuro. Mais antigo que o `temp-01` e com menos responsabilidade.

O pecado capital é o `200` com `{"erro": "saldo insuficiente"}` no corpo: o Bazar vê `200`, marca o pedido como pago e despacha o kit de capinhas. É o log `Erro: sucesso` promovido a contrato público.

Cache e validação condicional, ou como guardar dinheiro velho

A RFC 9111 define quem guarda uma resposta e por quanto tempo: caches privados (o do navegador) ou compartilhados (proxies, CDNs). O `Cache-Control` manda neles: `max-age=N` declara a resposta fresca por N segundos (`s-maxage` só para os compartilhados); `private` proíbe os compartilhados; `public` libera; `no-store` proíbe guardar; e `no-cache` permite guardar, desde que revalide antes de cada uso. Sim, "no-cache" significa "cache, sim, com fiscalização". O nome foi escolhido por alguém que também batiza variáveis de `temp`.

Sem `Cache-Control`, um cache pode inventar frescor por heurística a partir do `Last-Modified`, e um proxy de terceiros decide quanto tempo o saldo do vendedor fica velho. A validação condicional evita reenviar o que não mudou: o servidor manda um `ETag`, identificador opaco da versão, e o cliente o devolve em `If-None-Match`:

```
GET /v1/cobrancas/0190c3a1-7e2b-7c41-9d0f-5b2a8e6f1c33 HTTP/1.1
If-None-Match: "v4"

HTTP/1.1 304 Not Modified
ETag: "v4"
Cache-Control: private, no-cache
```

O `304` não tem corpo e economiza bytes; o RTT continua. O `ETag` pode ser forte (`"v4"`, idêntico byte a byte) ou fraco (`W/"v4"`, equivalente em significado).

O irmão de escrita é o `If-Match`: "aplique só se a versão ainda for `v4`", senão `412`. É o controle otimista do capítulo 4, e o perdedor recebe um status em vez de um `UPDATE` que afetou zero linhas. O `428` (RFC 6585) obriga o cliente a mandar a condição.

O `Vary` diz quais cabeçalhos da requisição mudam a resposta (`Vary: Origin`). Sem ele, um cache compartilhado serve a resposta de um cliente para outro. Num cadastro de gatos, é um bug. Num sistema de pagamentos, é um incidente com nome, data e advogado.

CORS, a regra de segurança que só o navegador respeita

O navegador aplica a *same-origin policy*: um script de uma origem (esquema, host e porta) não lê respostas de outra. Pelo CORS (Fetch Standard, do WHATWG), o servidor autoriza exceções, e quem obedece é o navegador. O `curl`, o Python e o atacante não estão nem aí para CORS, e nunca estiveram.

Na requisição simples (`GET`, `HEAD` ou `POST`, com cabeçalhos da lista segura e `Content-Type` só `text/plain`, `multipart/form-data` ou `application/x-www-form-urlencoded`), o navegador manda direto, com `Origin`, e só entrega a resposta ao script se vier `Access-Control-Allow-Origin` compatível. O detalhe que derruba gente experiente: a requisição foi processada. O CORS bloqueou a leitura, e o estorno aconteceu do mesmo jeito (requisição forjada é CSRF, capítulo 32).

Qualquer outra coisa (`application/json`, `Authorization`, `DELETE`) dispara o *preflight*, um `OPTIONS` perguntando se pode:

```
OPTIONS /v1/checkout/cobrancas/0190c3a1-7e2b-7c41-9d0f-5b2a8e6f1c33 HTTP/1.1
Origin: https://www.bazarbemtevi.com.br
Access-Control-Request-Method: GET
Access-Control-Request-Headers: authorization

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://www.bazarbemtevi.com.br
Access-Control-Allow-Headers: authorization
Access-Control-Max-Age: 7200
Vary: Origin
```

O `Access-Control-Max-Age` diz quanto tempo o navegador guarda a resposta (o Chromium limita a 7.200 segundos), por origem e URL: cobrança nova, um RTT inteiro de "posso?" antes do "quero". Com cookies, o servidor precisa de `Access-Control-Allow-Credentials: true` e origem explícita, porque o navegador recusa o curinga `*` com credenciais. A especificação proibiu exatamente a configuração que todo mundo tenta primeiro.

A API de cobranças da Lastro

Os métodos e os status

O endpoint antigo do painel passa a responder `405` com `Allow: POST`, e o estorno vira um formulário com confirmação. O robô do chat agora só leva a página de confirmação, que lê com a atenção de um gerente lendo postmortem. A API pública:

Operação	Requisição	Resposta
Criar cobrança Pix	`POST /v1/cobrancas`	`201`, `Location`, estado `pendente`
Criar cobrança cartão	`POST /v1/cobrancas`	`201`, `autorizada` ou `recusada` (adquirente síncrono, ~800 ms)
Consultar	`GET /v1/cobrancas/{id}`	`200` ou `304`, `ETag`

Operação	Requisição	Resposta
Capturar	`POST /v1/cobrancas/{id}/captura`	`200`
Estornar	`POST /v1/cobrancas/{id}/estornos`	`202`, `Location: /v1/estornos/{id}`
Consultar estorno	`GET /v1/estornos/{id}`	`200`: `processando`, `concluido` ou `falhou`
Listar moedas	`GET /v1/moedas`	`200`, `public, max-age=86400` (muda uma vez por argentino)

Cobrança recusada pelo emissor volta `201` com estado `recusada`: recusa é resultado de negócio, e um `4xx` faria a biblioteca do cliente lançar exceção no dia mais normal de uma fintech. `422` é corpo válido com conteúdo impossível (split que soma 103%); `409`, conflito com o estado (estornar cobrança `pendente` ou mais do que resta). O formato do erro é o capítulo 11.

O estorno usa `202` porque o débito no ledger é imediato (serializable, capítulo 3), mas a devolução depende do banco parceiro ou do adquirente; o marketplace acompanha pelo `Location`, e um webhook avisa a conclusão.

A consulta leva `ETag` da coluna nova `versao` de `cobrancas`, incrementada a cada mudança de estado, e `private, no-cache`. O Hornero consulta a cobrança Pix a cada 2 segundos, e 95% das consultas voltam `304` (tempo real é o capítulo 12). O saldo responde `no-store`: nenhum cache guarda o número que decide um saque (capítulo 22).

O adquirente e a conexão compartilhada

A autorização é uma chamada HTTPS ao adquirente de ~800 ms. No pico atual de 12 pagamentos por segundo, a Lei de Little (capítulo 14) dá 12 × 0,8 ≈ 10 autorizações simultâneas. O contrato limita cada subcredenciadora a 20 conexões simultâneas, e a Lastro tem 6 processos (3 workers em `temp-01`, 3 em `api-01`).

Opção	Concorrência	Teto/s	Custo
Conexão nova por chamada	20	25	handshakes em toda autorização; TIME_WAIT
HTTP/1.1 com pool	20	25	a Black Friday não cabe

Opção	Concorrência	Teto/s	Custo
HTTP/2, 1 conexão por processo	6 × 100 = 600	750	perda de pacote atrasa 1/6 das autorizações em voo

A Lastro usa HTTP/2, negociado por ALPN: o teto de 25 por segundo do HTTP/1.1 é só o dobro do pico atual, e a Black Friday vem aí (capítulo 14). No `GOAWAY`, streams acima do último ID processado vão para uma conexão nova; os abaixo não são reenviados sem chave de idempotência. HTTP/1.1 com pool seria a escolha com adquirente sem limite de conexões ou sem HTTP/2, ou num caminho com muita perda, em que conexões independentes isolam o estrago. Para os marketplaces nada muda: o Uvicorn fala HTTP/1.1, e com o keep-alive do capítulo 8 basta; HTTP/2 na borda depende de um proxy (capítulos 10 e 27).

O checkout no navegador

O checkout da Lastro roda na página do marketplace (`https://www.bazarbemtevi.com.br`) e consulta `https://api.lastro.com.br/v1/checkout/...` para mostrar o QR Code e o estado do Pix. Origens diferentes: CORS. O Carlos propôs `Access-Control-Allow-Origin: *` com `Allow-Credentials: true`, "pra não dar problema". O navegador recusou, e o Chromium virou o primeiro revisor da Lastro a barrar um PR do Carlos.

Só `/v1/checkout/` responde CORS; a chave secreta do marketplace nunca vai ao navegador. O checkout usa um token de curta duração, restrito a uma cobrança, no `Authorization` (capítulo 30). Na URL, o token dispensaria o preflight e terminaria impresso em log de proxy e `Referer`, então a Lastro paga o RTT. A origem é conferida contra a lista cadastrada do marketplace e devolvida como veio, com `Vary: Origin`, sem cookies e com `Max-Age` de 7.200 segundos.

Armadilhas e casos reais

> **Na vida real:** em 2005, o Google Web Accelerator fazia prefetch dos links das páginas visitadas, e aplicações que apagavam registros com links `GET` viram dados sumir sem ninguém clicar. É o estorno do Carlos com vinte anos de antecedência. O Carlos ainda não estava na indústria; o espírito dele já estava.
>
> **Na vida real:** em outubro de 2023, a CVE-2023-44487 (*HTTP/2 Rapid Reset*) permitiu ataques recordes; o Google mitigou um pico de 398 milhões de requisições por segundo. O atacante abre um stream e manda `RST_STREAM` logo depois: o stream

> sai da conta do `MAX_CONCURRENT_STREAMS`, com o trabalho do servidor já começado. A multiplexação virou uma maternidade de requisições natimortas.
>
> **Armadilha:** desde a versão 1.9.13, o nginx não repassa ao próximo servidor um `POST` já enviado, a menos que `proxy_next_upstream` inclua `non_idempotent`. Alguém acrescenta o parâmetro para "resolver" um `502` e passa a cobrar o comprador duas vezes. O `502` sumiu. O comprador também.
>
> **Armadilha:** `no-cache` em dado pessoal, achando que nada é guardado. O dado fica no disco do proxy, e o próximo erro de `Vary` entrega o extrato de um vendedor a outro. Dado que não pode ser guardado leva `no-store`.

Resumo

- HTTP/1.1: texto, uma requisição por vez por conexão; HTTP/2: streams multiplexados, HPACK, `GOAWAY`, head-of-line blocking do TCP; HTTP/3: QUIC, streams independentes, 1 RTT, connection ID.
- Seguro é só leitura; idempotente é N requisições com efeito de uma. Robôs e retentativas automáticas contam com isso.
- O status dirige o comportamento automático dos clientes; `202` para o que termina depois.
- `Cache-Control`, `ETag` com `If-None-Match` (`304`) e `If-Match` (`412`), `Vary`.
- CORS: o navegador aplica, o servidor autoriza; requisição simples é processada mesmo sem permissão de leitura.

A Lastro agora

A API de cobranças da Lastro segue a semântica do HTTP: `201` para criar, `202` para estornos, `409` e `422` para erros do cliente, `ETag` nas consultas, `no-store` no saldo, estorno só por `POST`. O adquirente recebe HTTP/2 com 600 streams contra o limite de 20 conexões, e o checkout usa CORS com origens cadastradas. A Capinhas recebeu os R$ 2.598,00 de volta em dois lançamentos novos, porque o tabelião não apaga nada, nem constrangimento.

O Carlos colou o link antigo de novo "só pra testar a correção", e o robô do chat recebeu um `405` educado. Ninguém conferiu a data do certificado TLS de `api.lastro.com.br`, emitido à mão pelo fundador num sábado e com vencimento numa madrugada de

domingo. Às 3h, todas essas respostas vão continuar perfeitas, com o status certo, o `ETag` certo e o CORS certo, e nenhum cliente vai conseguir abrir a conexão para receber nenhuma delas.

Referências

- RFC 9110, *HTTP Semantics*. https://www.rfc-editor.org/rfc/rfc9110
- RFC 9111, *HTTP Caching*. https://www.rfc-editor.org/rfc/rfc9111
- RFC 9112, *HTTP/1.1*. https://www.rfc-editor.org/rfc/rfc9112
- RFC 9113, *HTTP/2*. https://www.rfc-editor.org/rfc/rfc9113
- RFC 7541, *HPACK: Header Compression for HTTP/2*. https://www.rfc-editor.org/rfc/rfc7541
- RFC 9000, *QUIC: A UDP-Based Multiplexed and Secure Transport*. https://www.rfc-editor.org/rfc/rfc9000
- RFC 9114, *HTTP/3*. https://www.rfc-editor.org/rfc/rfc9114
- RFC 9204, *QPACK: Field Compression for HTTP/3*. https://www.rfc-editor.org/rfc/rfc9204
- RFC 6585, *Additional HTTP Status Codes*. https://www.rfc-editor.org/rfc/rfc6585
- WHATWG, *Fetch Standard*. https://fetch.spec.whatwg.org/
- NVD, CVE-2023-44487. https://nvd.nist.gov/vuln/detail/CVE-2023-44487
- Google Cloud, "How it works: The novel HTTP/2 'Rapid Reset' DDoS attack". https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack
- nginx, `proxy_next_upstream`. https://nginx.org/en/docs/http/ngx_http_proxy_module.html#proxy_next_upstream
- Wikipedia, "Google Web Accelerator". https://en.wikipedia.org/wiki/Google_Web_Accelerator

CAPÍTULO 10

TLS e certificados

Domingo, 3h12, ou a hora exata em que o fundador marcou o próprio funeral

Às 3h12 de um domingo, a API da Lastro estava perfeita: `201` com `Location`, `ETag` com aspas, CORS com `Vary: Origin`. E ninguém no planeta conseguia falar com ela, porque um arquivo de texto de 2 KB tinha passado da validade, igual iogurte.

O certificado TLS de `api.lastro.com.br` foi comprado pelo fundador, à mão, num sábado de madrugada, depois de uma pizza. Validade de um ano. Um ano depois de um sábado cai num domingo:

```
$ openssl x509 -in api.lastro.com.br.crt -noout -subject -dates
subject=CN = api.lastro.com.br
notBefore=Nov  1 06:12:44 2025 GMT
notAfter=Nov  1 06:12:44 2026 GMT
```

São 6h12 em GMT, 3h12 em Brasília. O certificado morreu em inglês, sem velório. Quando `api-01` nasceu (capítulo 8), o Carlos copiou para ele o mesmo arquivo e a mesma chave privada, e as duas máquinas morreram no mesmo segundo, com a sincronia que nenhum cluster da Lastro jamais teve. Cada cliente recebeu:

```
httpx.ConnectError: [SSL: CERTIFICATE_VERIFY_FAILED]
  certificate verify failed: certificate has expired
```

O Bazar abriu chamado às 3h14, assunto "URGENTE!!!!", com um print do navegador: "Sua conexão não é particular". Pior: o banco parceiro, que notifica cada Pix recebido chamando `api.lastro.com.br`, recusou o certificado três vezes e desistiu. O dinheiro chegava e a Lastro não sabia. Uma fintech que recebe dinheiro sem saber é, juridicamente, um cofrinho.

A conta da autoridade certificadora estava no e-mail pessoal do fundador, com MFA no celular dele, em modo avião. Às 5h47 o Carlos emitiu um certificado pela Let's Encrypt em `api-01` e copiou para `temp-01`, chave junto, reincidente em menos de três horas. Saldo: 2h35 fora, 214 cobranças não criadas e 61 Pix pagos sem notificação, que o financeiro conciliou à mão, de novo, agora com a calma de quem já não sente nada.

Na segunda, o fundador disse: "no meu tempo o certificado durava três anos". É verdade: até 2018 valia até 39 meses, e ele perderia o vencimento do mesmo jeito, só que com menos frequência. A pergunta: o que esse arquivo faz em cada conexão, e como tirar o vencimento das mãos de qualquer ser humano, principalmente desse?

O que o TLS garante, em três propriedades e nenhuma a mais

O TLS (versão 1.3 na RFC 8446) fica entre o TCP e o HTTP e entrega três coisas sobre um canal que qualquer roteador pode ler e alterar:

- **Confidencialidade:** só as duas pontas leem os dados.
- **Integridade:** qualquer byte alterado no caminho é detectado e a conexão cai.
- **Autenticação do servidor:** prova criptográfica de que o outro lado tem a chave privada de um certificado válido para `api.lastro.com.br`. Opcionalmente, o cliente também se autentica (mTLS, mais adiante).

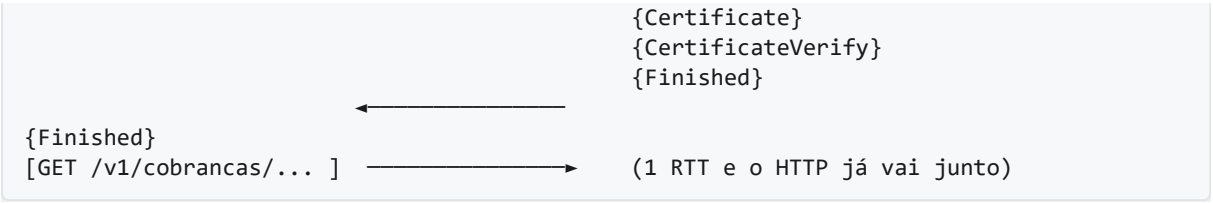
E só. O cadeado garante que você fala em particular com alguém, sem dizer se esse alguém é honesto. Golpista também tem cadeado, e o dele está em dia.

Os dados viajam em *records* cifrados com AEAD, que cifra e autentica ao mesmo tempo. O TLS 1.3 aceita cinco *cipher suites*, todas AEAD; o 1.2 aceitava centenas, algumas ensinando o atacante a decifrar o tráfego. O 1.0 e o 1.1 foram aposentados pela RFC 8996 e seguem vivos em servidor de prefeitura, de pijama.

O handshake do TLS 1.3, ou dois estranhos combinando um segredo em voz alta numa praça

As duas pontas precisam de chaves de sessão que ninguém conheça, trocando mensagens que todo mundo vê. A ferramenta é o Diffie-Hellman efêmero (ECDHE): cada lado gera um par de chaves descartável, envia a parte pública e combina a própria privada com a pública do outro. Os dois chegam ao mesmo segredo; quem só viu as públicas não chega.

```
Cliente                                              Servidor
ClientHello
  + key_share (X25519, pública do cliente)
  + server_name: api.lastro.com.br   (SNI)
  + alpn: h2, http/1.1
  + signature_algorithms
                          ──────────────►
                                                ServerHello
                                                  + key_share (pública do servidor)
                                                {EncryptedExtensions: alpn=h2}
```



As mensagens entre `{}` já vão cifradas. Passo a passo:

1. O **ClientHello** já leva a parte pública do ECDHE, apostando no grupo que o servidor vai aceitar (quase sempre X25519). Aposta errada, o servidor pede outra com `HelloRetryRequest`, e lá se vai mais um RTT.
2. O **ServerHello** traz a parte pública do servidor. Os dois derivam as chaves com HKDF, e tudo o que segue é cifrado.
3. **Certificate** manda a cadeia. **CertificateVerify** é uma assinatura, com a chave privada do certificado, sobre o hash de todas as mensagens até ali: a prova de que o servidor tem a chave, e não só o certificado, que é público e qualquer um baixa.
4. **Finished** é um HMAC sobre o histórico inteiro. Se alguém no caminho rebaixou a versão ou mexeu na lista de algoritmos, os hashes não batem e a conexão morre.

Uma ida e volta, contra duas do TLS 1.2: o RTT que o capítulo 8 cobrou do Hornero, enfim explicado.

A chave privada do certificado só assina; nunca cifra as chaves de sessão. Isso é **forward secrecy**: quem gravar o tráfego da Lastro hoje e roubar a chave daqui a dois anos não decifra nada, porque as chaves ECDHE de cada conexão já foram jogadas fora. No TLS 1.2 com troca de chaves RSA, a chave roubada abria todo o arquivo gravado, de trás pra frente. O 1.3 removeu essa opção, a primeira vez na história que um comitê de padronização apagou alguma coisa.

A retomada de sessão usa um *ticket* de conexão anterior e permite dados no primeiro voo, o 0-RTT, que um atacante pode gravar e reenviar: a mesma cobrança, duas vezes, com criptografia impecável. A Lastro recusou o 0-RTT no capítulo 8, e o vendedor 42 ficou ofendido de não ser consultado.

O certificado, um documento que só vale porque alguém que você nunca viu assinou

Um certificado X.509 (RFC 5280) é uma estrutura assinada que liga uma chave pública a nomes. Os campos que derrubam produção:

- **Subject Alternative Name (SAN):** os nomes DNS para os quais o certificado vale; o cliente compara o nome que pediu com essa lista (RFC 9525). O `CN` do subject é

ignorado pelos navegadores e segue preenchido por tradição, igual campo "fax" em formulário.

- **Validade:** `notBefore` e `notAfter`, em UTC. Fora disso, lixo.
- **Chave pública** e **assinatura do emissor** sobre tudo isso.
- **Extensões:** `basicConstraints` (pode assinar outros certificados?) e `extendedKeyUsage` (`serverAuth`, `clientAuth`).

Um curinga `*.lastro.com.br` cobre um nível: `api.lastro.com.br` sim, `x.api.lastro.com.br` e `lastro.com.br` não. Cobertura no padrão do plano de saúde do Carlos.

Cadeia de confiança, ou o golpe da pirâmide que funciona

Ninguém confia no certificado da Lastro. O cliente confia em **raízes**: certificados autoassinados de autoridades certificadoras (CAs), no *trust store* do sistema, do navegador ou, em Python, do pacote `certifi`. A raiz fica offline e assina **intermediárias**, que assinam o certificado do servidor (a folha).

```
ISRG Root X1        (no trust store do cliente; o servidor não envia)
  └── E7            (intermediária; o servidor ENVIA)
        └── api.lastro.com.br   (folha; o servidor envia)
```

Validar é subir a cadeia: assinaturas conferem com a chave de cima, todos no prazo, os de cima com `basicConstraints CA:TRUE`, o nome no SAN da folha, o caminho terminando numa raiz confiável. Um elo podre e a conexão cai.

As regras do CA/Browser Forum encurtam a vida do certificado em escada: no máximo 200 dias a partir de 15 de março de 2026, 100 a partir de março de 2027 e 47 a partir de março de 2029. O certificado de um ano do fundador nasceu pouco antes da lei, como quase tudo que o fundador fez. Em 2029, quem renova à mão renova mais vezes do que corta o cabelo.

Revogação. Chave vazada, a CA revoga e publica em CRLs (listas assinadas) ou responde por OCSP (consulta online). Navegadores tratam falha de OCSP como "deixa passar", e quem bloqueia a consulta anula a revogação; a Let's Encrypt desligou seu OCSP em agosto de 2025. A revogação que funciona é o certificado curto: morre de causas naturais antes que alguém precise matá-lo.

Certificate Transparency. Toda CA confiável publica cada certificado em logs públicos append-only (RFC 6962). Dá para monitorar quem emitiu para o seu domínio, e todo nome com certificado público fica público para sempre. Se o Carlos emitir um

certificado para `banco-legado-nao-mexer.lastro.com.br`, o atacante recebe o convite com endereço. O registro DNS **CAA** (RFC 8659) lista as CAs autorizadas a emitir para o domínio, e elas são obrigadas a conferir.

SNI e terminação TLS, ou onde a criptografia acaba e a confiança começa

Um IP, vários nomes, e o `Host` só chega depois do handshake. O **SNI** (RFC 6066) põe o nome pedido no ClientHello, em claro: o caminho inteiro vê que você acessou `api.lastro.com.br`, só não vê o quê. A **ALPN** (RFC 7301), no mesmo ClientHello, negocia `h2` ou `http/1.1`; foi por ela que a Lastro chegou ao HTTP/2 com o adquirente no capítulo 9.

Terminar TLS é decidir onde a conexão cifrada acaba:

Arranjo	Quem guarda a chave	Custo
No servidor de aplicação	cada processo	trocar certificado = recarregar a aplicação
No proxy	só o proxy	atrás dele, HTTP em claro; o trecho precisa ser confiável
Proxy re-cifra	proxy e backend	dois handshakes, dois certificados
Passthrough pelo SNI	só o backend	o proxy não lê HTTP nem roteia por caminho

Terminando no proxy, a aplicação perde o IP do cliente e o esquema; o proxy repassa em `X-Forwarded-For` e `X-Forwarded-Proto` (ou `Forwarded`, RFC 7239), e a aplicação só acredita neles vindos do proxy. Acreditar em `X-Forwarded-For` de qualquer um é deixar o suspeito preencher o próprio boletim de ocorrência.

ACME, o robô que não esquece aniversário

O ACME (RFC 8555) é o protocolo com que um cliente pede certificado, prova controle do domínio e recebe a cadeia sem humano no caminho. Let's Encrypt e Google Trust Services são exemplos de CAs que o falam.

1. O cliente registra uma conta, identificada por um par de chaves próprio, e cria uma **ordem** para os nomes; a CA devolve uma **autorização** por nome, com desafios.

2. O cliente cumpre um **desafio**: - **HTTP-01:** servir um token em `/.well-known/acme-challenge/<token>` na porta 80; a CA resolve o DNS e busca. - **DNS-01:** publicar um TXT em `_acme-challenge.<nome>`. Único que emite curinga; exige credencial de DNS. - **TLS-ALPN-01** (RFC 8737): responder na 443 com um certificado especial, pela ALPN `acme-tls/1`.
3. Com a autorização válida, o cliente envia um CSR (a chave pública, assinada pela privada) e baixa a cadeia.

O cliente renova sozinho, em geral com um terço da validade restante (aos 60 dias num certificado de 90); a extensão ARI deixa a CA antecipar a janela quando precisa revogar em massa. A chave não sai da máquina, ninguém compra nada e ninguém precisa do celular do fundador.

mTLS, quando o servidor também pede documento

No **mTLS** (*mutual TLS*), o servidor manda `CertificateRequest` no handshake, e o cliente responde com o próprio `Certificate` e um `CertificateVerify`, assinando o histórico com a sua chave privada. O servidor valida a cadeia do cliente contra as CAs que **ele** escolhe, quase sempre uma CA privada, e a identidade do cliente passa a ser o subject ou o SAN do certificado, provada antes do primeiro byte de HTTP.

Uma chave de API trafega em cada requisição e vaza no log que a imprimir; a chave do mTLS nunca sai da máquina. O custo é operar certificados de cliente com prazos do parceiro. É a relação em que os dois exigem exame antes, e alguém sempre esquece de renovar o exame.

Na Lastro: o vencimento sai das mãos humanas

Requisitos. Pix 24x7 (capítulo 7): troca de certificado sem janela. Duas máquinas respondem por `api.lastro.com.br`: `api-01`, alvo do DNS, e `temp-01`, com a integração do Bazar que resolveu o DNS há 200 dias (capítulo 8). E o banco parceiro avisou, por ofício, que sua API Pix exige mTLS em 60 dias.

Terminação. Hoje o Uvicorn termina TLS, e trocar certificado é reiniciar os workers. A Lastro põe um proxy reverso em cada máquina (nginx e Caddy são exemplos) na porta 443, falando HTTP com o Uvicorn em `127.0.0.1:8000`; o Uvicorn aceita `X-Forwarded-*` só de `127.0.0.1`. O proxy recarrega certificado sem derrubar conexão e oferece `h2` pela ALPN, o HTTP/2 na borda que o capítulo 9 deixou pendente. TLS 1.3 e 1.2 (só ECDHE e

AEAD, porque um marketplace ainda chega em 1.2); 1.0 e 1.1 desligados. Um balanceador gerenciado, com certificado gerenciado, dispensaria o proxy e é a escolha quando houver balanceador (capítulo 27).

Emissão. O HTTP-01 só alcança a máquina para onde o DNS aponta, e `temp-01` ficaria sem certificado, com o Bazar dentro. Por isso, DNS-01 nas duas máquinas, cada uma com **sua própria chave** e seu certificado. O risco do DNS-01 é a credencial: quem edita a zona aponta `api.lastro.com.br` para onde quiser. A Lastro delega o desafio para uma zona que só contém desafios:

```
api.lastro.com.br.                    300  IN A      203.0.113.10
_acme-challenge.api.lastro.com.br.    300  IN CNAME  api.desafios-acme.lastro.com.br.
lastro.com.br.                       3600  IN CAA    0 issue "letsencrypt.org"
lastro.com.br.                       3600  IN CAA    0 issue "pki.goog"
lastro.com.br.                       3600  IN CAA    0 iodef
"mailto:seguranca@lastro.com.br"
```

A credencial das máquinas só escreve em `desafios-acme.lastro.com.br`: se vazar, o atacante publica TXT, e só. O CAA autoriza duas CAs, para a segunda emitir se a primeira estiver fora. Chave ECDSA P-256, certificado de 90 dias, renovação aos 60.

Vigilância. Automação que falha em silêncio é só um vencimento com etapas. Um verificador externo conecta uma vez por dia nos dois IPs, com SNI, e lê o certificado **servido** (o arquivo no disco pode estar renovado e o proxy, sem recarregar). Alerta abaixo de 21 dias: nove dias seguidos de renovação quebrada. Um monitor de Certificate Transparency avisa de todo certificado novo para `lastro.com.br`, inclusive os que ninguém pediu.

mTLS com o banco parceiro. Em duas direções, com dois nomes no mesmo IP, separados pelo SNI e ambos no SAN do certificado ACME de `api-01`:



Para chamar o banco, cada máquina gera seu par de chaves, manda um CSR pelo portal e recebe um certificado de cliente assinado pela CA privada do banco, válido por um ano. A chave fica em arquivo `0600`, legível só pelo usuário do serviço, e nunca sai da máquina

(KMS no capítulo 33). Sem ACME, a renovação é manual: o verificador acompanha esses certificados com alertas a 60, 30 e 14 dias, e a troca tem sobreposição, com o banco aceitando o novo antes de o velho morrer.

Na outra direção, as notificações de Pix passam para `webhooks-pix.lastro.com.br`, onde o proxy exige certificado de cliente e confia só na CA do banco. O proxy repassa o subject verificado num cabeçalho e **apaga** qualquer cabeçalho de mesmo nome vindo de fora; a aplicação só aceita o subject do banco. Sem isso, qualquer um mandaria um "Pix recebido de R$ 50.000,00" com um `curl`, e a Lastro creditaria com a maior boa vontade. Exigir mTLS dos marketplaces é decisão de autenticação (capítulo 30).

Armadilhas e casos reais

> **Na vida real:** em 6 de dezembro de 2018, um certificado vencido no software SGSN-MME da Ericsson derrubou operadoras em 11 países; cerca de 30 milhões de clientes da O2, no Reino Unido, ficaram quase um dia sem serviço. Ninguém atacou nada. O calendário atacou sozinho, e calendário não tem CVE.
>
> **Na vida real:** em 30 de setembro de 2021 venceu a raiz DST Root CA X3, que assinava em cadeia cruzada os certificados da Let's Encrypt. Clientes com OpenSSL 1.0.x falharam mesmo confiando na raiz nova, por uma peculiaridade na construção do caminho: APIs, IoT, o servidor esquecido no canto. Toda empresa tem um. A Lastro deu nome ao dela.
>
> **Armadilha:** a intermediária que não foi enviada. O Chrome completa a cadeia buscando a intermediária pela URL do certificado e tudo "funciona"; o cliente Python, sobre o OpenSSL, não busca e falha com `unable to get local issuer certificate`. O dev abre no navegador e fecha o chamado. O marketplace reabre. Os dois envelhecem.
>
> **Armadilha:** `verify=False` para "resolver" erro de certificado. A conexão segue cifrada, só que até o atacante no meio, que decifra, lê e cifra de novo. Camisinha com furo de fábrica. O Carlos usou na madrugada do incidente, no script de reenvio de notificações, e o código está em revisão, com o Carlos dentro.

Resumo

- TLS: confidencialidade, integridade, autenticação do servidor. TLS 1.3 em 1 RTT, ECDHE com forward secrecy, só AEAD; 0-RTT sujeito a replay.

- Certificado: nomes no SAN, validade em UTC, chave pública, assinatura do emissor. Cadeia até uma raiz do trust store; o servidor envia as intermediárias.
- Revogação é frágil; validade curta é a que funciona. CT publica todo certificado; CAA limita as CAs.
- SNI escolhe o certificado, ALPN o protocolo. Terminação no proxy exige confiar em `X-Forwarded-*` só vindo dele.
- ACME automatiza emissão e renovação (HTTP-01, DNS-01, TLS-ALPN-01); monitora-se o certificado servido.
- mTLS autentica o cliente por certificado, contra a CA que o servidor escolhe.

A Lastro agora

Cada máquina tem um proxy na 443 terminando TLS 1.3 e 1.2, com HTTP/2 para os marketplaces e o Uvicorn escondido em `127.0.0.1`. Os certificados de `api.lastro.com.br` vêm por ACME com DNS-01 delegado, um por máquina, 90 dias, renovados aos 60, duas CAs no CAA e alerta externo com 21 dias. Com o banco parceiro, mTLS nas duas direções: certificados de cliente de um ano em `api-01` e `temp-01`, alertas a 60, 30 e 14 dias, e notificações de Pix em `webhooks-pix.lastro.com.br`. O certificado do fundador foi revogado e mora num pendrive na gaveta dele, etiquetado "backup".

O fundador perguntou se agora o certificado "nunca mais vence". Vence a cada 90 dias, só que sem plateia. Enquanto isso, o gerente de produto mandou incluir na resposta da cobrança um campo novo, `parcelas_detalhe`, porque "é só um campo, quem não usa ignora". O desserializador do Bazar não ignora nada, e o próximo "URGENTE!!!!" vai chegar por uma conexão impecável: TLS 1.3, forward secrecy, certificado válido por mais 89 dias e um JSON que o Bazar se recusa a ler.

Referências

- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*. https://www.rfc-editor.org/rfc/rfc8446
- RFC 5280, *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*. https://www.rfc-editor.org/rfc/rfc5280
- RFC 9525, *Service Identity in TLS*. https://www.rfc-editor.org/rfc/rfc9525
- RFC 6066, *TLS Extensions* (SNI). https://www.rfc-editor.org/rfc/rfc6066
- RFC 7301, *TLS Application-Layer Protocol Negotiation Extension*. https://www.rfc-editor.org/rfc/rfc7301
- RFC 6962, *Certificate Transparency*. https://www.rfc-editor.org/rfc/rfc6962

- RFC 8659, *DNS Certification Authority Authorization (CAA) Resource Record*. https://www.rfc-editor.org/rfc/rfc8659
- RFC 8555, *Automatic Certificate Management Environment (ACME)*. https://www.rfc-editor.org/rfc/rfc8555
- CA/Browser Forum, *Baseline Requirements*. https://cabforum.org/working-groups/server/baseline-requirements/requirements/
- Let's Encrypt, "Ending OCSP Support in 2025". https://letsencrypt.org/2024/12/05/ending-ocsp/
- Let's Encrypt, "DST Root CA X3 Expiration (September 2021)". https://letsencrypt.org/docs/dst-root-ca-x3-expiration-september-2021/
- TechCrunch, "Here's what caused yesterday's O2 and SoftBank outages" (2018). https://techcrunch.com/2018/12/07/heres-what-caused-yesterdays-o2-and-softbank-outages

CAPÍTULO 11

Contratos de API

"É só um campo", ou como um JSON educado derrubou um marketplace inteiro

A mensagem de erro tinha 214 caracteres, um colchete sobrando e a assinatura inconfundível do Java:

```
com.fasterxml.jackson.databind.exc.UnrecognizedPropertyException:
Unrecognized field "parcelas_detalhe" (class br.com.bazar.pagamentos.Cobranca),
not marked as ignorable (9 known properties: "id", "valor", "estado", ...])
```

Terça, 11h02. O gerente de produto pôs na resposta da cobrança um campo novo, `parcelas_detalhe`. "É só um campo, quem não usa ignora." O Bazar Bem-te-vi não ignora. A integração dele foi escrita por uma consultoria em Java, com Jackson, cujo padrão (`FAIL_ON_UNKNOWN_PROPERTIES = true`) trata campo desconhecido como invasão de domicílio.

Todo `GET /v1/cobrancas/{id}` e todo webhook virou exceção no Bazar. O endpoint de webhooks dele respondia `500`, e a Lastro, que ainda manda webhook uma vez só (capítulo 4), seguia a vida. Às 11h32, o job do Bazar que cancela pedido sem pagamento confirmado em 30 minutos fez o trabalho dele com zelo: 312 compradores pagaram o Pix e receberam "pedido cancelado". "URGENTE!!!!" às 11h41, rollback às 13h40, e o financeiro passou a tarde devolvendo Pix de gente que só queria uma capinha.

Na reunião, o gerente culpou o Bazar e o Bazar culpou a Lastro. Os dois têm razão, por isso a reunião durou duas horas. A pergunta: o que é o contrato de uma API, que mudanças ele permite sem perguntar a ninguém, e como uma mudança que quebra chega ao mundo sem cancelar 312 pedidos?

O contrato é tudo o que o cliente consegue ver, inclusive o que você nunca prometeu

O **contrato** de uma API é o conjunto de comportamentos observáveis dos quais um cliente pode depender: caminhos, métodos, campos e tipos, obrigatoriedade, `null`, valores de enum, status, formato dos erros, cabeçalhos, ordem das listas e a semântica de tudo (`valor` em centavos, `criado_em` em UTC).

A documentação é só a parte escrita. Hyrum Wright, engenheiro do Google, registrou a observação conhecida como **Lei de Hyrum**: com usuários suficientes, todo comportamento observável do sistema terá alguém dependendo dele, prometido ou não. A ordem das chaves no JSON, o texto do erro, o ID ter sempre 36 caracteres. Em algum lugar existe um `if` em cima disso, escrito por alguém que já tem o template do chamado pronto.

Uma mudança é **compatível com versões anteriores** (*backward compatible*) quando todo cliente escrito para o contrato antigo continua funcionando com o servidor novo. A regra vem da teoria de tipos: a resposta nova pode ser um **subtipo** da antiga (tudo o que tinha, e mais), e a requisição nova pode aceitar um **supertipo** (tudo o que aceitava, e mais). É a variância que o mypy cobra na sobrescrita de método: retorno covariante, parâmetro contravariante. Quem inverte viola Liskov e ganha uma noite de plantão de brinde.

Mudança	Na resposta	Na requisição
Adicionar campo	Compatível se o cliente tolera campo desconhecido	Compatível se opcional
Remover ou renomear campo	Quebra	Quebra se o servidor passar a recusá-lo
Mudar tipo (`int` → `string`)	Quebra	Quebra
Campo opcional vira obrigatório	Compatível (sempre presente)	Quebra
Campo passa a admitir `null`	Quebra	Compatível
Novo valor de enum	Quebra quem trata enum como fechado	Compatível
Validação mais rígida	n/a	Quebra
Validação mais frouxa	n/a	Compatível

Mudança	Na resposta	Na requisição
Mudar significado (`valor` em reais)	Quebra sem erro nenhum	Quebra sem erro nenhum

A última linha é a pior. Trocar centavos por reais mantém o schema intacto, passa em todo validador, e o vendedor recebe R$ 0,87 por uma venda de R$ 87,01, com documentação impecável e um advogado novo.

A primeira linha tem um asterisco: adicionar campo só é compatível com um **leitor tolerante** (*tolerant reader*, nome de Martin Fowler para o padrão), que lê o que precisa e ignora o resto. O HTTP faz isso com cabeçalhos desconhecidos (RFC 9110); Protocol Buffers, por projeto. O Jackson padrão faz o oposto, e o Bazar aprendeu a diferença em 312 e-mails:

```
Gerente:  É só um campo. Quem não usa, ignora.
Jackson:  Unrecognized field "parcelas_detalhe".
Gerente:  Então ignora.
Jackson:  not marked as ignorable.
Gerente:  Marca, ué.
Jackson:  Quem marca é o Bazar. O Bazar terceirizou.
Bazar:    (cancela 312 pedidos, abre chamado "URGENTE!!!!")
```

O ancestral é o princípio da robustez de Jon Postel (RFC 761, 1980): conservador no que envia, liberal no que aceita. A RFC 9413 (2023) registra o efeito colateral, emissores tortos para sempre porque ninguém reclama, e propõe a versão estreita: ignore o que o contrato manda ignorar e recuse o resto em voz alta.

Enums merecem velório próprio. Um `match estado:` com `pendente`, `paga` e `cancelada`, sem caso padrão, explode no dia em que nasce `contestada`. O contrato precisa dizer se o enum é aberto (valores novos podem aparecer, e desconhecido é tratado como desconhecido) ou fechado. E `null` contra ausente: `{"desconto": null}` e `{}` são diferentes em JSON, metade dos desserializadores acha que são iguais, e as duas metades integram com a Lastro.

Versionar é escolher onde mora a mentira

Mudança que quebra precisa conviver com a antiga. **Versionar** é dar nome a cada contrato e deixar o cliente escolher qual recebe. O nome pode morar em quatro lugares, e cada um mente para um componente diferente.

Estratégia	Exemplo	Roteamento no proxy	Cache HTTP	Visível em log	Granularidade
Caminho	`/v1/cobrancas`	Por prefixo, trivial	Sem cuidado extra (URL diferente)	Sim	API inteira
Cabeçalho próprio	`Lastro-Versao: 2026-11-01`	Precisa ler cabeçalho	Exige `Vary`	Só se logar o cabeçalho	Por requisição
Media type	`Accept: application/vnd.lastro.v2+json`	Precisa ler cabeçalho	Exige `Vary: Accept`	Só se logar	Por recurso
Parâmetro	`?versao=2`	Por query	URL diferente	Sim	Por requisição

O caminho aparece em todo log, roteia por prefixo e separa o cache sozinho, ao custo de granularidade: `/v2` é uma API inteira nova por causa de um campo. O media type é o mais fiel ao HTTP (mesmo recurso, outra representação) e o mais traiçoeiro: um CDN sem `Vary: Accept` serve a versão 2 a quem pediu a 1, e quem esqueceu o `Vary` uma vez nunca mais esquece, porque o incidente ganha o nome dele.

A Stripe documentou em 2017 uma quinta via: **versões por data, fixadas por conta**. Cada conta fica presa à versão do dia em que integrou, e pode sobrescrever por requisição com o cabeçalho `Stripe-Version`. O código produz a resposta mais nova, e uma cadeia de transformações a reescreve para trás até a data do cliente. Caro de construir, barato por mudança.

Versão velha precisa de atestado de óbito. `Deprecation` (RFC 9745) diz desde quando o recurso está obsoleto; `Sunset` (RFC 8594), quando deixa de responder; um `Link` com `rel="deprecation"` aponta a migração.

```
HTTP/1.1 200 OK
Deprecation: @1793491200
Sunset: Mon, 01 Nov 2027 00:00:00 GMT
Link: <https://docs.lastro.com.br/migracao-v2>; rel="deprecation"
```

O `Deprecation` usa a data estruturada da RFC 9651 (segundos Unix; essa é 1º de novembro de 2026); o `Sunset`, a data HTTP clássica. Duas RFCs, dois formatos de data, mesmo velório. Ninguém lê nenhum dos dois: a versão morre no dia marcado, alguém liga desesperado, a data é adiada três meses, e o ciclo se repete até a versão velha ficar mais antiga que `temp-01` e pedir aposentadoria por tempo de serviço.

Paginação e filtros, o contrato escrito nas entrelinhas

O mecanismo do cursor está no capítulo 6; aqui interessa o que ele promete. Um contrato de paginação define a ordem (estável e total, com desempate, como `(criado_em, id)`), o limite (padrão e máximo; quem aceita `limite=1000000` acabou de publicar um endpoint de exportação de banco) e a opacidade do cursor.

O cursor da Lastro é um base64 de `(criado_em, id)`, e a Lei de Hyrum garante que algum cliente vai decodificar e montar cursores à mão. O contrato diz "string opaca", e o servidor assina o cursor para que o artesanato dê `400`. Opaco no papel e legível no fio é cantada de bar.

Filtros sofrem da mesma doença. `criado_em_ate` é inclusivo? Em que fuso? Sem resposta escrita, `2026-11-03T00:00:00` significa meia-noite UTC no servidor e meia-noite em Brasília no Bazar, e o extrato do dia some com três horas de vendas. E o filtro com typo: `?estdo=paga`, num servidor tolerante, é ignorado e devolve todas as cobranças, e o cliente informa ao vendedor um faturamento de ficção científica. Leitor tolerante é virtude no cliente lendo respostas; no servidor recebendo consultas financeiras, é cumplicidade.

Erros com formato: RFC 9457, o fim do `{"erro": "deu ruim"}`

Toda API inventa seu formato de erro, e o cliente coleciona parsers como quem coleciona traumas. Algumas devolvem `200 OK` com `{"sucesso": false}`, o primo HTTP do log `Erro: sucesso`. A RFC 9457 (2023, substitui a RFC 7807) padroniza o **problem details**: um objeto JSON com media type `application/problem+json` e cinco membros definidos.

```
{
  "type": "https://docs.lastro.com.br/erros/split-invalido",
  "title": "Split não fecha 100% do valor",
  "status": 422,
  "detail": "Os recebedores somam 10300 basis points; o máximo é 10000.",
  "instance": "/v1/cobrancas/requisicoes/0190d5f2-41aa-7b3e-8c11-2f6a9e0b7d54",
  "recebedores_excedentes": ["vend_7310"]
}
```

`type` é uma URI que identifica o tipo de problema, e é por ele que o cliente decide o que fazer; ausente, vale `about:blank` ("nada além do status"). `title` é um resumo fixo por tipo. `status` repete o código HTTP e precisa bater com ele. `detail` explica esta ocorrência, para humanos; `instance` a identifica. Membros extras são extensões, e quem não as conhece as ignora.

O cliente nunca faz parsing do `detail`. É texto para gente, e texto para gente muda quando o redator acorda inspirado. Regex em cima de `detail` transforma uma frase em API, e frase não tem versionamento, só revisor.

OpenAPI e testes de contrato, ou o contrato com firma reconhecida

Contrato que mora na cabeça do gerente muda quando o gerente almoça. A **OpenAPI** (a 3.1 é alinhada ao JSON Schema 2020-12) descreve a API em YAML ou JSON: caminhos, parâmetros, schemas, respostas por status. O FastAPI gera o documento a partir dos modelos Pydantic: o contrato publicado é exatamente o código, e qualquer refatoração inocente de modelo altera o contrato sem ninguém perceber até o "URGENTE!!!!".

Versionado no repositório, o documento vira diff, e comparadores (o oasdiff é um exemplo) classificam cada diferença pela tabela de compatibilidade. A CI recusa o merge que quebra. O diff vê o schema e é cego para o comportamento.

Para o comportamento existem os **testes de contrato dirigidos pelo consumidor** (*consumer-driven contract testing*; o Pact é o exemplo mais conhecido). Cada consumidor registra num teste as interações de que depende ("este `GET` devolve `200` com `estado` string"), publica o contrato num repositório central (o *broker*), e o provedor verifica todos os contratos contra o próprio código antes de subir. O provedor descobre quais campos cada um usa e remove o resto sem medo. O custo é o consumidor escrever os testes: entre serviços da mesma empresa, funciona; com a consultoria Java do Bazar, leve lanche e um padre.

As regras da Lastro, por escrito e com testemunha

O requisito: a API é integrada por equipes externas, com stacks desconhecidas e o ritmo de deploy de quem sobe versão quando o estagiário volta das férias. Mudança aditiva sai sem coordenação; mudança que quebra pede meses de convivência.

Compatibilidade, no guia de integração e no contrato comercial: o cliente ignora campos desconhecidos em respostas e webhooks (no Jackson, `FAIL_ON_UNKNOWN_PROPERTIES = false`); enums de resposta (`estado`, `metodo`, `tipo` de evento) são abertos; ausente e `null` significam "sem valor". Sem versão nova, a Lastro pode adicionar campo em resposta, campo opcional em requisição, valor de enum e endpoint, e afrouxar validação, sempre com 30 dias de changelog. O resto quebra.

Versionamento: major no caminho (`/v1`), sem versões menores. O requisito é operacional: proxy reverso em duas máquinas (capítulo 10), cache com `ETag` (capítulo 9), logs que já mostram o caminho. O prefixo roteia sem ler cabeçalho, dispensa `Vary` e aparece em qualquer `grep` . A granularidade grossa (toda quebra vira `/v2` inteira) é paga quebrando quase nunca. A `/v1` convive com a sucessora por 12 meses, com `Deprecation` e `Sunset` . Com centenas de marketplaces e quebras trimestrais, versões por data no estilo da Stripe pagariam a cadeia de transformações; com dois clientes, seriam um framework para servir duas pessoas e um consultor.

Erros: RFC 9457 em todo `4xx` e `5xx` , um `type` por regra (`saldo-insuficiente` , `split-invalido` , `estado-invalido`), lista aberta como qualquer enum. Validação de corpo devolve a extensão `erros` com o JSON Pointer (RFC 6901) de cada campo. Recusa do emissor segue `201` com estado `recusada` (capítulo 9).

Listagens: cursor assinado, `limite` padrão 50 e máximo 200, ordem `(criado_em, id)` , filtros semiabertos `[de, ate)` em RFC 3339 com fuso obrigatório, parâmetro desconhecido devolve `400` com `type` `parametro-desconhecido` .

Testes: o OpenAPI gerado é commitado e a CI bloqueia diff incompatível com o da `main` . E o sandbox injeta em toda resposta e todo webhook um campo `_sandbox_campo_novo` com valor aleatório. O campo-isca só existe para quebrar no sandbox a integração que quebraria em produção. O Bazar teria descoberto o Jackson numa terça qualquer, sem o financeiro chorando no estorno.

Webhooks, cujo contrato até hoje era "um POST com um JSON e fé":

```
{
  "id": "evt_0190d61a-9c03-7f12-a4e8-6b1d0c2e9f77",
  "tipo": "cobranca.paga",
  "versao_api": "v1",
  "criado_em": "2026-11-03T14:22:07.481Z",
  "dados": {
    "id": "0190c3a1-7e2b-7c41-9d0f-5b2a8e6f1c33",
    "estado": "paga",
    "valor": 259800,
    "versao": 4
  }
}
```

Envelope fixo; `dados` carrega a cobrança inteira, na representação do `GET` da versão em `versao_api` .

Cláusula	Por quê
`id` único por evento	O cliente deduplica: a entrega será "pelo menos uma vez" (capítulos 16 e 17)

Cláusula	Por quê
`dados.versao` (a coluna do `ETag`)	Evento com `versao` menor que a já processada é velho e vai para o lixo
Sem garantia de ordem	`cobranca.estornada` pode chegar antes de `cobranca.paga`; o `versao` desempata
Cabeçalho de assinatura criptográfica	Nome reservado; o algoritmo é do capítulo 33
Inscrição por tipo de evento	Tipo novo só chega a quem se inscreveu nele
Resposta `2xx` em até 10 s	Qualquer outra coisa é falha de entrega

A inscrição por tipo desarma o enum mais perigoso da API: se `cobranca.contestada` chegasse a todo mundo, cada `match tipo:` sem caso padrão viraria `500`. E o `parcelas_detalhe` volta em 30 dias, pelo changelog e pelo sandbox. A consultoria do Bazar trocou uma linha de configuração e cobrou 40 horas.

Armadilhas e casos reais

> **Armadilha:** o `ETag` da Lastro vem de `cobrancas.versao`, que muda quando a cobrança muda. Com `parcelas_detalhe`, a cobrança ficou igual e a representação mudou. O Hornero, que faz polling com `If-None-Match` (capítulo 9), recebeu `304` e seguiu com a representação velha, sem o campo. O `ETag` identifica a representação, então agora combina `versao` com o identificador do schema de resposta. O Hornero foi o único cliente poupado do incidente, e graças a um bug.
>
> **Na vida real:** em 2010, o Twitter passou a gerar IDs de tweet de 64 bits com o Snowflake. Acima de 2^53, o `JSON.parse` do JavaScript arredonda o inteiro (capítulo 1) e o cliente lê um ID que não existe. A correção foi aditiva: um campo `id_str`, o mesmo número como string, ao lado do `id`, que nunca pôde sair. Dois IDs por tweet, para sempre, porque remover o errado quebraria quem se acostumou com ele.

Schema de resposta fechado. `additionalProperties: false` num schema de resposta faz geradores de cliente produzirem modelos que recusam campo novo. Uma linha de YAML distribuiria o Jackson estrito a todos os clientes, em todas as linguagens, com a assinatura da Lastro. Os schemas publicados declaram `true`.

A Lei de Hyrum dentro de casa. Na semana seguinte, o Carlos corrigiu um typo no `detail` de saldo insuficiente: "saldo insuficinte" virou "saldo insuficiente". O Hornero tinha uma regex procurando "insuficinte", e por quatro horas todo saque recusado virou "erro desconhecido" no painel dele. O Carlos derrubou produção acertando a ortografia, feito inédito numa carreira dedicada a derrubar produção errando. O `type` `saldo-insuficiente` estava lá o tempo todo, sem typo, esperando alguém ler.

Mudança semântica. Trocar a captura padrão de automática para manual não altera um byte do OpenAPI. Só o teste dirigido pelo consumidor pega; para o diff, está tudo lindo enquanto o dinheiro apodrece em autorização.

Resumo

- Contrato é todo comportamento observável; pela Lei de Hyrum, alguém depende até do que nunca foi prometido.
- Compatibilidade segue variância: resposta pode crescer, requisição pode aceitar mais. Remover, renomear, mudar tipo, endurecer validação e mudar significado quebram.
- Campo novo exige leitor tolerante; enum precisa ser declarado aberto ou fechado; `null` e ausente precisam de regra.
- Versão no caminho, cabeçalho, media type ou data por conta: cada uma paga em cache, roteamento, visibilidade ou manutenção. `Deprecation` (RFC 9745) e `Sunset` (RFC 8594) assinam o atestado de óbito.
- Paginação promete ordem estável, limite máximo e cursor opaco; filtro define intervalo e fuso, e filtro desconhecido é erro.
- Problem details (RFC 9457): o cliente decide pelo `type`, nunca pelo `detail`.
- Diff de OpenAPI na CI protege o schema; testes dirigidos pelo consumidor protegem o comportamento.
- `ETag` identifica a representação e muda quando o schema muda.

A Lastro agora

Regras de compatibilidade publicadas: cliente ignora campo desconhecido, enums abertos, mudança aditiva com 30 dias de changelog e estágio no sandbox com o campo-isca. Versão no caminho, 12 meses de convivência. Erros na RFC 9457; cursor assinado; OpenAPI com diff na CI. Webhooks com envelope, `id` para deduplicar, `versao` para descartar evento velho, inscrição por tipo e prazo de 10 segundos. O `ETag` inclui o schema.

O gerente leu as regras e voltou com três pedidos: GraphQL para o painel dos marketplaces, "porque é só uma consulta"; o checkout sabendo em tempo real que o Pix foi pago, sem polling de 2 segundos; e o novo fornecedor de antifraude, que só fala gRPC. Nenhum dos três é JSON sobre `GET`, e o gerente jura que todos são aditivos. É só um campo. É sempre só um campo.

Referências

- RFC 9457, Problem Details for HTTP APIs: https://www.rfc-editor.org/rfc/rfc9457
- RFC 7807, Problem Details for HTTP APIs (substituída): https://www.rfc-editor.org/rfc/rfc7807
- RFC 9110, HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110
- RFC 8594, The Sunset HTTP Header Field: https://www.rfc-editor.org/rfc/rfc8594
- RFC 9745, The Deprecation HTTP Response Header Field: https://www.rfc-editor.org/rfc/rfc9745
- RFC 9651, Structured Field Values for HTTP: https://www.rfc-editor.org/rfc/rfc9651
- RFC 9413, Maintaining Robust Protocols: https://www.rfc-editor.org/rfc/rfc9413
- RFC 761 (princípio da robustez): https://www.rfc-editor.org/rfc/rfc761
- RFC 3339, Date and Time on the Internet: https://www.rfc-editor.org/rfc/rfc3339
- RFC 6901, JSON Pointer: https://www.rfc-editor.org/rfc/rfc6901
- OpenAPI Specification: https://spec.openapis.org/oas/latest.html
- JSON Schema 2020-12: https://json-schema.org/draft/2020-12
- Pact, documentação: https://docs.pact.io/
- Hyrum's Law: https://www.hyrumslaw.com/
- Martin Fowler, Tolerant Reader: https://martinfowler.com/bliki/TolerantReader.html
- Stripe, APIs as infrastructure: future-proofing Stripe with versioning (2017): https://stripe.com/blog/api-versioning
- Jackson databind, Deserialization Features: https://github.com/FasterXML/jackson-databind/wiki/Deserialization-Features

CAPÍTULO 12

Estilos de comunicação

Três pedidos, três protocolos e um gerente que acha tudo igual

O gerente de produto entrou com três pedidos e a convicção de que eram um pedido escrito três vezes.

O primeiro veio do Mercadito Hornero. O painel deles mostra as vendas do dia com vendedor, split e estornos, e cada tela custa 201 chamadas à API: uma listagem, cem de vendedor, cem de estornos. Com 100 ms de RTT até a Virgínia (capítulo 8), a tela leva mais de 20 segundos. O N+1 do capítulo 6 emigrou: agora são 201 viagens por um cabo no fundo do Atlântico, e cada uma paga pedágio para os tubarões. O Hornero pediu GraphQL, "como todo mundo".

O segundo veio do checkout, que pergunta a cada 2 segundos se o Pix foi pago. No pico, 600 checkouts abertos geram 300 requisições por segundo, e 99,6% recebem a mesma resposta: `pendente`. A Lastro responde "ainda não" um milhão de vezes por hora, o que faz dela a ex mais requisitada do país. O Carlos propôs polling de 100 ms, "pra ficar tempo real": 6.000 requisições por segundo. O Carlos chamou de tempo real; o PostgreSQL chamaria de ataque, se sobrevivesse para chamar alguma coisa.

O terceiro veio da Vigia, o novo fornecedor de antifraude, que avalia cada transação de cartão antes da autorização e só oferece gRPC. A documentação tem uma frase: "não temos JSON, temos princípios".

```
Gerente:  É tudo API. É só chamar.
Hornero:  Quero escrever a minha consulta.
Checkout: Quero que me avisem.
Vigia:    Quero bytes, HTTP/2 e um deadline.
Carlos:   Quero polling de 100 ms.
Todos:    Não.
```

Estilo de comunicação é marketing até alguém olhar o tráfego. O que separa REST, gRPC, GraphQL, WebSocket, SSE e webhooks no fio, e qual propriedade de cada canal da Lastro decide o estilo dele?

REST, o estilo que já pagava as contas

Cinco perguntas separam os estilos: quem inicia, quantas mensagens por chamada (uma de cada lado, a chamada unária, ou uma sequência, o *streaming*), o que trafega, quem guarda cache e o que a infraestrutura no caminho suporta, do proxy ao firewall corporativo de 2009. REST é o estilo descrito por Roy Fielding na tese dele, em 2000: recursos identificados por URL, interface uniforme, nenhum estado de sessão no servidor. No mercado, é o que a Lastro já faz, JSON sobre a semântica da RFC 9110 do capítulo 9. O cliente inicia, a chamada é unária, o cache HTTP funciona completo (o `GET` é seguro e a URL é a chave), e qualquer proxy da face da Terra entende, inclusive o que ninguém sabe quem configurou.

O custo: o servidor decide a forma da resposta. Precisando de menos, o cliente recebe campos que joga fora (*over-fetching*); precisando de mais, faz outra chamada (*under-fetching*), e cada chamada custa um RTT. Para o Bazar, a 2 ms, isso é ruído. Para o Hornero, a 100 ms, é o motivo de o analista financeiro deles ter aprendido crochê enquanto o painel carrega.

gRPC, um contrato binário com HTTP/2 obrigatório e nenhum ser humano no meio

gRPC é o framework de RPC publicado pelo Google em 2015. Serviços e mensagens ficam num arquivo `.proto`, um gerador produz cliente e servidor em cada linguagem, e o código chama um método remoto que parece local. A rede continua lá, só que agora com cara de função.

Protocol Buffers no fio

O formato é o Protocol Buffers. O schema numera cada campo, e é o número que trafega, nunca o nome:

```
message Transacao {
  int64  valor = 1;
  string moeda = 2;
}

valor = 150, moeda = "BRL"

protobuf: 08 96 01 12 03 42 52 4c            (8 bytes)
JSON:     {"valor":150,"moeda":"BRL"}        (27 bytes)
```

Cada campo começa com uma *tag*, `(número << 3) | tipo_no_fio`. O `08` é o campo 1, tipo 0 (*varint*); `96 01` é 150 em varint, 7 bits por byte com o bit alto avisando que vem mais; `12` é o campo 2, tipo 2 (tamanho prefixado), com tamanho `03` e os bytes de

"BRL". O JSON manda a palavra "valor" em toda mensagem, por toda a eternidade, como quem rubrica cada página do contrato.

Número desconhecido é pulado (o tipo no fio diz quantos bytes) e, desde o protobuf 3.5, guardado para reserializar: o leitor tolerante do capítulo 11 vem de fábrica, e o Jackson da consultoria do Bazar teria um colapso nervoso. O número é o contrato: renomear campo é inofensivo, reutilizar o número de um removido faz o cliente antigo ler `moeda` onde agora vai `pais`. Por isso existe `reserved 3;`, a lápide que impede enterrar outro campo na mesma cova.

O gRPC sobre o HTTP/2

O gRPC exige HTTP/2. Cada chamada é um stream: `POST` para `/pacote.Servico/Metodo`, cada mensagem com 1 byte de flag de compressão e 4 de tamanho, e o resultado nos *trailers*, cabeçalhos que chegam depois do corpo:

```
:method: POST
:path: /vigia.v1.Avaliacao/Avaliar
content-type: application/grpc
te: trailers
grpc-timeout: 120m

(corpo: 00 00 00 00 2a + 42 bytes de protobuf)

trailers -> grpc-status: 0
```

Num stream, o servidor só sabe se deu certo depois de mandar as mensagens, daí o status no trailer: 17 códigos (`OK`, `DEADLINE_EXCEEDED`, `UNAVAILABLE`...), com o HTTP em `200` quase sempre. É o único protocolo que responde "200, morreu" sem ironia.

O `grpc-timeout: 120m` é o *deadline*: "m" minúsculo é mili, maiúsculo é minuto, e alguém já confundiu os dois em produção. O servidor desiste quando o prazo passa e repassa o restante às chamadas que fizer (capítulo 15).

Como cada chamada é um stream, há quatro formas: unária, streaming do servidor, streaming do cliente e bidirecional, em que os dois lados falam quando querem, sem esperar a vez, como em toda reunião de postmortem.

Os custos: sem cache HTTP, é tudo `POST`; o navegador não expõe trailers ao JavaScript e precisa do gRPC-Web com um proxy tradutor; o log vira binário ilegível sem o `.proto`; e a conexão HTTP/2 longa engana o balanceador de camada 4 (ver armadilhas).

GraphQL, ou o cliente escreve a consulta e você paga a conta

O GraphQL nasceu no Facebook em 2012, para os apps móveis, e saiu em 2015. O servidor publica um schema de tipos e relações; o cliente manda a forma exata da resposta, num endpoint só, via `POST`:

```
query VendasDoDia {
  cobrancas(primeiros: 100, de: "2026-11-03T00:00:00-03:00") {
    nos {
      id
      valor
      vendedor { nome documento }
      estornos { valor estado }
    }
    proximoCursor
  }
}
```

Uma viagem, nem um campo a mais. O Hornero leu e chorou.

Resolvers, ou o N+1 com certificado de origem

Cada campo tem um *resolver*, a função que produz o valor dele a partir do objeto pai. O de `cobrancas` devolve 100 cobranças; o executor chama o de `vendedor` 100 vezes e o de `estornos` mais 100. Do jeito óbvio, são 201 consultas. O GraphQL tirou o N+1 da rede do Hornero e o reinstalou no seu banco, com nota fiscal.

A saída é o padrão *DataLoader*: o resolver só registra o ID que quer e devolve uma promessa; o loader junta os IDs da mesma rodada do event loop numa consulta (`WHERE id = ANY(...)`), com cache por requisição. As 201 viram 3, o número do `selectinload` do capítulo 6, porque a matemática do N+1 não muda de nacionalidade.

O custo de uma consulta que o cliente escreveu

No REST, você escreve as consultas e sabe o pior caso. No GraphQL, o cliente escreve, e o pior caso é criatividade:

```
{ cobrancas(primeiros: 100) { nos {
    vendedor { cobrancas(primeiros: 100) { nos {
      vendedor { cobrancas(primeiros: 100) { nos { id } } } } } } } } } }
```

São 100 × 100 × 100 = 1 milhão de nós, numa requisição de menos de 200 bytes. As defesas rodam antes da execução: limite de profundidade, paginação obrigatória com teto em toda lista e uma análise de custo estática que multiplica os tetos aninhados e recusa o que passar do orçamento. Sem isso, sua API pública vira um gerador de produto cartesiano com o cartão de crédito de outra pessoa.

O cache HTTP morre, porque tudo é `POST` na mesma URL (consultas persistidas, chamadas por hash via `GET`, ressuscitam o cache, mas exigem conhecer os clientes). E erro de execução volta, por convenção, com status `200`, `data` parcial e uma lista `errors`: o monitoramento que conta `5xx` jura que está tudo bem enquanto metade dos campos volta `null`.

Empurrar do servidor: WebSocket, SSE e webhooks

Até aqui, o servidor só fala quando perguntado. Para ele tomar a iniciativa, há três mecanismos.

WebSocket

O WebSocket (RFC 6455) começa como HTTP/1.1 e troca de protocolo no meio da conversa:

```
GET /chat HTTP/1.1
Host: servidor.example
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

O `Accept` é o SHA-1 da chave com um GUID fixo da RFC, em Base64: prova que o servidor entende WebSocket e ninguém está ecoando cabeçalhos por educação. Depois do `101`, acabou o HTTP: o TCP vira um canal bidirecional de frames, e os do cliente vão mascarados com uma chave de 32 bits, para nenhum proxy antigo confundir bytes do usuário com uma requisição e envenenar o próprio cache.

Somem status por mensagem, cache e reconexão automática. A conexão tem estado e mora num processo: o proxy precisa repassar o `Upgrade`, o balanceador aguenta conexões de horas, e o deploy derruba todo mundo junto. É conexão persistente no sentido mais carente da expressão.

Server-Sent Events

O SSE, do HTML Living Standard, é uma resposta HTTP que nunca termina, com `Content-Type: text/event-stream`:

```
retry: 3000
```

```
id: 7
event: estado
data: {"estado":"pendente"}

: ping

id: 8
event: estado
data: {"estado":"paga"}
```

Linha em branco separa eventos; linha com `:` é comentário, o *heartbeat* que impede o proxy de matar a conexão ociosa. O `EventSource` do navegador reconecta após `retry` milissegundos e manda o último `id` em `Last-Event-ID`.

É HTTP, com o TLS e a autenticação que já existem. Os limites: uma direção, só texto (binário vai em Base64, chorando), e o `EventSource` não aceita cabeçalhos customizados, detalhe que vai custar uma reunião inteira na Lastro.

O terceiro, o webhook (capítulo 11), é o servidor virando cliente e exige endpoint HTTPS público no destinatário, coisa que o navegador do comprador, felizmente, não tem.

A tabela que o gerente vai imprimir e não vai ler

Estilo	Quem inicia	Mensagens	No fio	Cache HTTP	Exige da infraestrutura
REST	cliente	unária	JSON	completo	só HTTP
gRPC	cliente	unária e streaming	protobuf	não	HTTP/2 ponta a ponta, balanceamento por requisição
GraphQL	cliente	unária	JSON	com consultas persistidas	análise de custo, DataLoader
WebSocket	os dois	bidirecional	frames próprios	não	`Upgrade`, conexões longas com estado
SSE	servidor	streaming	texto	não	proxy sem buffer, heartbeat
Webhook	servidor	unária	JSON	não	endpoint público no destino

Na Lastro, cada canal recebe o estilo que pede

Os comandos da API pública continuam REST, apoiados no `201`, no `202` e no `If-Match` do capítulo 9, e os webhooks continuam iguais. Os três pedidos novos recebem três decisões.

O painel do Hornero: GraphQL só de leitura

Opção	Viagens por tela (Hornero, 100 ms)	Custo para a Lastro
REST atual	201, mais de 20 s	nenhum
REST com `expandir=vendedor,estornos`	1, ~140 ms	um parâmetro por relação, formas fixas
GraphQL	1, ~140 ms	limites de custo, DataLoader, autorização por resolver

O `expandir` resolve a tela do Hornero, mas cada marketplace pede outra combinação de cobrança, split, estornos, vendedor e lançamentos. Com `expandir`, cada combinação nova é trabalho da Lastro; com GraphQL, é consulta do cliente. A Lastro publica `POST /v1/graphql`, só com `query`; escritas ficam no REST, com `201`, `If-Match` e, em breve, idempotência.

Limites verificados antes da execução: profundidade 5; `primeiros` obrigatório, máximo 100; custo estático (objeto vale 1, listas multiplicam pelo teto) até 5.000 pontos; `statement_timeout` de 3 segundos. Toda relação passa por DataLoader, e a CI conta o SQL de cada consulta de exemplo, com a raiva do `lazy="raise"` do capítulo 6. Paginação usa o cursor assinado do capítulo 11; erros trazem em `extensions.type` as URIs da RFC 9457; o schema evolui por adição e `@deprecated`. Cada resolver confere o marketplace da credencial, e o capítulo 31 volta nisso com sangue.

Com poucos clientes e telas fixas, `expandir` custaria menos e manteria o cache HTTP; com dezenas desenhando painéis próprios, a conta inverte.

O checkout: SSE, porque só um lado tem algo a dizer

O checkout espera um fato só: a cobrança mudou de estado. O navegador não tem nada a dizer durante a espera, exceto "e aí?", que o polling diz 300 vezes por segundo.

Opção	Requisições no pico (600 checkouts)	Atraso até o comprador ver "paga"	Peças novas
Polling 2 s (atual)	300/s	1 s em média, mais o RTT	nenhuma
Polling 100 ms (Carlos)	6.000/s	50 ms, até o banco cair	um novo emprego
WebSocket	600 conexões abertas	milissegundos após o commit	`Upgrade` no proxy, protocolo próprio

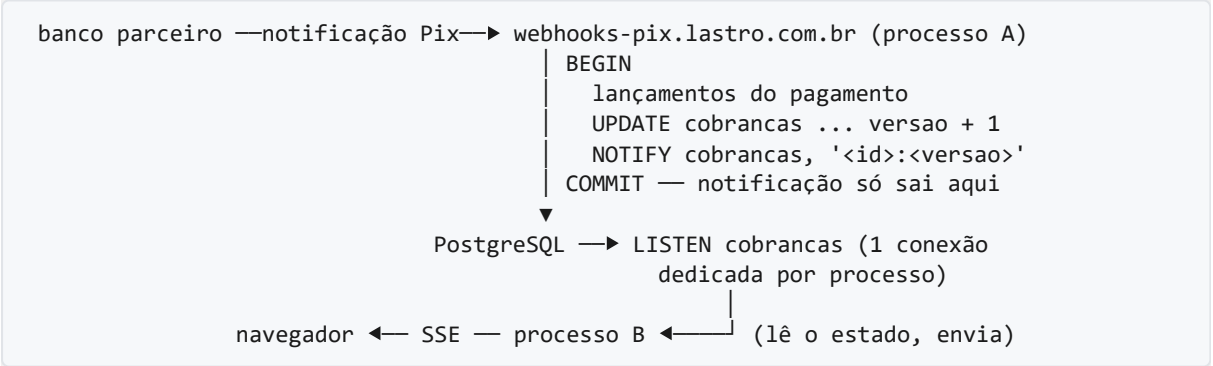
Opção	Requisições no pico (600 checkouts)	Atraso até o comprador ver "paga"	Peças novas
SSE	600 conexões abertas	milissegundos após o commit	proxy sem buffer, heartbeat

O WebSocket entregaria uma direção que ninguém usa e cobraria a perda do HTTP por ela; o SSE passa pelo proxy, pelo TLS e pelo CORS que já existem. WebSocket seria a escolha se o checkout mandasse dados continuamente, como num chat com o vendedor.

Aí veio a reunião inteira. O capítulo 9 pôs o token do checkout no `Authorization`, fora da URL e dos logs, e o `EventSource` não manda cabeçalhos customizados (a API `WebSocket` do navegador também não). O checkout lê o `text/event-stream` com `fetch`, aos pedaços, e reimplementa a reconexão com `retry` e `Last-Event-ID`. O Carlos sugeriu token na URL "só desta vez". O log do proxy guardaria o token por 90 dias, que é quanto dura "só desta vez" no ramo de vazamentos.

O endpoint é `GET /v1/checkout/cobrancas/{id}/eventos`; o `id` do evento é a `versao` da cobrança, e na reconexão a versão maior que o `Last-Event-ID` sai na hora. `: ping` a cada 15 segundos, proxy sem buffer com 60 segundos de tolerância, stream fechado na mudança de estado ou na expiração do QR Code.

Falta o problema de verdade: a notificação do Pix chega a um dos 6 processos (3 em `temp-01`, 3 em `api-01`) e o stream do comprador está em outro. A Lastro usa o `LISTEN` / `NOTIFY` do PostgreSQL, sem peça nova:



O `NOTIFY` dentro da transação só sai no `COMMIT`; se ela aborta, a notificação morre junto, e o comprador nunca vê "paga" antes de o tabelião registrar o lançamento. Cada processo tem uma conexão fora do pool para o `LISTEN`: 6 a mais, ociosas, longe do teto de 17 a 20 ativas do capítulo 4. O stream que espera não segura conexão do pool; se segurasse, 600 checkouts disputariam 15 conexões e o `QueuePool limit` voltaria em formato de Pix.

O `NOTIFY` também não guarda nada: processo desconectado perde a notificação. Por isso o handler faz `LISTEN` antes de ler o estado, e a reconexão relê. A notificação acorda; a verdade mora no banco.

A Vigia: gRPC unário com deadline

Aqui quem escolheu foi o fornecedor. Cada autorização de cartão chama `Avaliar` antes do adquirente, unária, com `grpc-timeout` de 120 ms (a Vigia promete p99 de 60 ms); o que fazer quando estoura fica para o capítulo 15. Cada processo mantém um canal, uma conexão HTTP/2 com TLS, como a do adquirente. O `.proto` entra no repositório com versão fixada, e os campos da decisão usam `optional` (ver armadilhas).

Armadilhas e casos reais

> **Armadilha:** no proto3, campo escalar sem `optional` confunde "ausente" com "zero". Se a Vigia não mandar `score`, a Lastro lê `score = 0`, "risco nenhum" no contrato deles. É o `null` × ausente do capítulo 11, só que aqui o default aprova a fraude. Com `optional` (protobuf 3.15+), o código gerado ganha `HasField`.
>
> **Na vida real:** o blog do Kubernetes descreveu em 2018 o gRPC atrás de um Service comum, que balanceia por conexão (camada 4), enquanto o cliente gRPC manda tudo por uma conexão HTTP/2. Uma réplica recebe todo o tráfego, as outras recebem o salário. A correção é balancear por requisição, num proxy de camada 7 ou no cliente (capítulo 27).
>
> **Armadilha:** proxy com buffer de resposta (o `proxy_buffering` do nginx vem ligado) segura os eventos SSE até encher o buffer ou a resposta acabar. É polling com etapas extras. Desligue o buffer na rota ou responda `X-Accel-Buffering: no`, no caso do nginx.
>
> **Armadilha:** aliases do GraphQL (`a1: cobranca(id: 1)`, `a2: cobranca(id: 2)` ...) põem 10.000 tentativas numa requisição só, e o rate limit por requisição dá desconto de atacado ao atacante. A análise de custo conta cada alias.
>
> **Na vida real:** a API GraphQL do GitHub exige `first` / `last` de 1 a 100 em toda conexão e recusa consultas acima de 500.000 nós.

Resumo

- Estilos se separam por quem inicia, mensagens por chamada, formato no fio, cache e exigências de infraestrutura.
- REST: semântica e cache do HTTP; cada chamada a mais custa um RTT.
- gRPC: protobuf com campos por número; HTTP/2, status nos trailers, deadline em `grpc-timeout`; sem cache HTTP nem navegador direto.

- GraphQL: o cliente define a forma; DataLoader contra o N+1; limites de custo antes de executar.
- WebSocket: bidirecional após `101`. SSE: resposta HTTP infinita com `Last-Event-ID`. Webhook: servidor como cliente.
- `NOTIFY` só sai no commit e se perde sem ouvinte conectado.

A Lastro agora

A Lastro fala quatro dialetos: REST para comandos; `POST /v1/graphql` só de leitura para os painéis, com profundidade 5, listas de até 100 e 5.000 pontos; SSE no checkout, lido por `fetch` com o token no `Authorization` e acordado por `NOTIFY` no commit do pagamento; gRPC unário de 120 ms para a Vigia. 300 polls por segundo viraram 600 conexões quietas; o Hornero vê o painel em 140 ms. O analista deles largou o crochê com o casaco pela metade.

O Carlos integrou a Vigia com o stub síncrono do gRPC dentro de um `async def`, "porque é só uma chamada", ao lado do cliente do adquirente, que sempre foi síncrono. Cada autorização trava o processo por quase um segundo, e os 100 checkouts dele congelam junto, educadamente, sem erro. O polling pelo menos falhava com status.

Referências

- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures*, 2000: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
- RFC 9110, HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110
- Protocol Buffers, Encoding: https://protobuf.dev/programming-guides/encoding/
- gRPC over HTTP/2: https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-HTTP2.md
- gRPC-Web: https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-WEB.md
- GraphQL Specification: https://spec.graphql.org/
- DataLoader: https://github.com/graphql/dataloader
- GitHub, Rate limits and node limits for the GraphQL API: https://docs.github.com/en/graphql/overview/rate-limits-and-node-limits-for-the-graphql-api
- RFC 6455, The WebSocket Protocol: https://www.rfc-editor.org/rfc/rfc6455
- HTML Living Standard, Server-sent events: https://html.spec.whatwg.org/multipage/server-sent-events.html

- PostgreSQL, `NOTIFY`: https://www.postgresql.org/docs/current/sql-notify.html
- Kubernetes Blog, gRPC Load Balancing on Kubernetes without Tears (2018): https://kubernetes.io/blog/2018/11/07/grpc-load-balancing-on-kubernetes-without-tears/

CAPÍTULO 13

Processos, threads e I/O assíncrono

A CPU a 4% e ninguém atendendo

Às 20h03 de uma quinta-feira, a Lastro teve o incidente mais relaxado da sua história: as duas máquinas com a CPU a 4%, memória sobrando, rede bocejando, e nenhuma requisição sendo atendida. Um servidor parado por excesso de trabalho você entende. Um servidor parado por excesso de descanso precisa de terapia.

O Bazar tinha lançado às 20h a campanha "12x sem juros em tudo, até em capinha", e as autorizações de cartão saíram de 1 para 7 por segundo. Cada uma chama a Vigia (uns 60 ms) e o adquirente (uns 800 ms) com clientes síncronos, dentro de um `async def`, porque o Carlos achou que "é só uma chamada". Era. Uma chamada que segura o processo inteiro por 0,9 segundo.

Seis processos (3 em `temp-01`, 3 em `api-01`), cada um fazendo uma coisa por vez durante 0,9 segundo, dão um teto de 6 / 0,9 ≈ 6,7 autorizações por segundo. Chegavam 7. O `POST /v1/cobrancas` foi de 0,9 para 14 segundos, o Bazar desistia aos 10 (o destino da cobrança abandonada é assunto do capítulo 15), e os checkouts com SSE pararam de receber o `: ping`, até o proxy cortar todos aos 60 segundos. O Pix, que nem passa pelo adquirente, morreu junto, por solidariedade.

O URGENTE!!!! chegou às 20h11. O Carlos propôs `--workers 64`. O fundador lembrou que no tempo dele era um processo e funcionava.

Por que seis processos com a CPU a 4% não dão conta de sete pedidos por segundo, e como dimensionar um servidor cujo trabalho é 99% esperar?

Processo e thread, ou a árvore genealógica mais mórbida da computação

Um **processo** é um espaço de endereçamento virtual próprio, uma tabela de descritores de arquivo (sockets inclusos) e pelo menos uma thread. Uma **thread** é um fluxo de execução (registradores, pilha) que compartilha memória e descritores com as outras do processo.

No Linux, os dois são *tasks* criadas por `clone()`: com `CLONE_VM` e companhia, a task nova divide memória com a mãe e vira thread; sem, vira processo. O `fork()` cria o filho com *copy-on-write*: as páginas ficam compartilhadas até alguém escrever nelas. Fork de um processo de 2 GB custa milissegundos. Até a primeira escrita.

E aí começa a novela. O filho que termina vira **zumbi**: fica na tabela de processos até o pai chamar `wait()` e ler o código de saída. Pai que não lê atestado de óbito acumula zumbis. O filho cujo pai morre primeiro vira **órfão** e é adotado pelo PID 1, que passa o dia recolhendo cadáver alheio. Sistemas operacionais foram batizados por alguém que precisava conversar com um profissional.

A pilha de uma thread reserva 8 MB de endereço virtual (o padrão de `ulimit -s` na maioria das distribuições), mas só ocupa RAM a página tocada: uma thread ociosa custa dezenas de KB. Um processo Python com FastAPI, SQLAlchemy e gRPC carregados ocupa de 100 a 200 MB de RSS; na Lastro, 160 MB por worker. Thread é barata. Processo Python tem pensão alimentícia.

A troca de contexto, ou o kernel que vive de interromper

O *scheduler* do kernel (o EEVDF, desde o Linux 6.6) decide qual task roda em cada núcleo. Uma task está **executável** ou **dormindo**, à espera de um evento: dado no socket, timer, lock.

Uma **troca de contexto** salva os registradores de uma task e carrega os da outra: voluntária quando a task bloqueia numa chamada de sistema, involuntária quando a fatia de tempo acaba. Entre processos, troca também a tabela de páginas. O custo direto é de microssegundos; o indireto, cache e TLB frios, não aparece em gráfico nenhum, como todo custo indireto da sua vida.

O kernel conta as duas por task:

```
$ grep ctxt /proc/$(pgrep -of "uvicorn.*worker")/status
voluntary_ctxt_switches:     1843021
nonvoluntary_ctxt_switches:  312
```

Essa saída, de um worker de `temp-01` às 20h05, é a assinatura do incidente: quase dois milhões de saídas voluntárias (dorme esperando rede) e 312 expulsões (ninguém disputa CPU com ele). Passa o dia dormindo e nunca é interrompido. Chegou ao estilo de vida do fundador.

I/O bloqueante e não bloqueante, ou a arte de não esperar sentado

Um `recv()` num socket sem dado, no modo padrão, **bloqueia**: o kernel põe a thread para dormir na fila de espera do socket e a acorda quando os bytes chegarem. Simples de programar, caro de escalar: cada conexão em espera prende uma thread. O worker do Carlos tinha uma thread só, e ela estava esperando o adquirente.

Com `O_NONBLOCK`, o mesmo `recv()` retorna na hora com `EAGAIN`. A thread não dorme, e também não sabe quando tentar de novo. Tentar em laço é *busy waiting*: a CPU a 100% perguntando "chegou?" milhões de vezes por segundo, o equivalente computacional da criança no banco de trás. Falta o kernel avisar quando um descritor fica pronto, a **notificação de prontidão**:

Mecanismo	Como registra os descritores	Custo por chamada	Limite
`select`	bitmask copiado a cada chamada	O(n) descritores vigiados	`FD_SETSIZE`, 1.024
`poll`	vetor copiado a cada chamada	O(n) descritores vigiados	nenhum fixo
`epoll`	lista de interesse no kernel, registrada uma vez	O(k) descritores prontos	memória

No `epoll`, `epoll_ctl()` registra o descritor uma vez e `epoll_wait()` dorme até algum ficar pronto, devolvendo só os prontos. Com 10.000 conexões e 12 com dado, o `select` varre 10.000 bits e o `epoll` devolve 12 entradas. No modo padrão, **level-triggered**, cada `epoll_wait` avisa de novo enquanto houver dado no buffer; no **edge-triggered** (`EPOLLET`), o aviso vem só na transição, e quem leu metade e foi embora nunca mais é avisado. É o modo para quem gosta de emoção.

O `kqueue` (BSD, macOS) também é de prontidão. O IOCP do Windows e o `io_uring` do Linux (desde o 5.1) são de **conclusão**: o kernel faz a leitura e avisa que terminou. Arquivo comum fica fora da festa: o `epoll_ctl` recusa arquivo regular com `EPERM`, porque disco, para o kernel, está sempre pronto. Por isso o `asyncio` lê arquivo e resolve DNS (`getaddrinfo`, bloqueante) num pool de threads, escondido nos fundos.

Em 1999, o texto "The C10K problem", de Dan Kegel, perguntou como atender 10 mil conexões numa máquina. A resposta foi essa tabela, e o nginx, o Node.js e o `asyncio` descendem dela.

O event loop, ou uma thread que não pode piscar

Um **event loop** é um laço em volta do `epoll_wait`: calcula quanto falta para o próximo timer, dorme no `epoll_wait` até esse prazo, executa os callbacks dos descritores prontos e dos agendados, e recomeça. Uma thread, milhares de conexões.

No `asyncio`, cada requisição é uma **corrotina** dentro de uma **Task**; no `await` num socket sem dado, a Task devolve o controle ao loop, que registra o descritor no `epoll` e atende outra. A multitarefa é **cooperativa**: a troca só acontece num `await` de algo pendente. Código que não faz `await` nunca solta o loop, e o resto do processo espera, como numa reunião em que alguém compartilhou a tela e saiu para almoçar.

```
import asyncio, time

async def bloqueia():
    time.sleep(0.8)           # a thread dorme; o loop dorme junto

async def coopera():
    await asyncio.sleep(0.8)  # a corrotina dorme; o loop segue atendendo

# asyncio.gather com 100 de cada:
# bloqueia: 80,1 s     coopera: 0,8 s
```

Cem chamadas que bloqueiam viram 80 segundos em série; cem que cooperam terminam juntas em 0,8. O `requests.post` e o stub síncrono do gRPC são `time.sleep` com currículo. Às 20h03, eram sete por segundo, cada uma dormindo 0,9 segundo com o loop no colo.

O modo debug do `asyncio` denuncia o crime: callback acima de `loop.slow_callback_duration` (100 ms por padrão) gera aviso.

```
WARNING:asyncio:Executing <Task pending name='Task-48213'
  coro=<RequestResponseCycle.run_asgi() running at ...>
  wait_for=<Future pending ...>> took 0.913 seconds
```

Novecentos e treze milissegundos sem atender ninguém: nove vezes o limite de vergonha que o próprio Python configurou.

Código bloqueante inevitável vai por `asyncio.to_thread()` (3.9+) ou `loop.run_in_executor()`: a chamada roda numa thread do pool e a corrotina faz `await` no resultado. A thread fica presa, o loop fica livre.

O GIL, ou o cadeado que o Python pôs no próprio pé

Se thread resolve espera, por que não usar threads em tudo e esquecer o `asyncio`? Por causa do **GIL**, o Global Interpreter Lock do CPython: um mutex que só deixa uma thread por interpretador executar bytecode Python por vez.

O GIL é liberado em toda chamada de sistema bloqueante (`recv`, `sleep`) e por extensões em C que trabalham sem tocar em objetos Python (o `hashlib` com entradas grandes, o NumPy). Uma thread que só executa Python é convidada a soltá-lo a cada 5 ms (`sys.getswitchinterval()`). Threads Python dão concorrência para I/O, porque quem espera rede solta o GIL, e nenhum paralelismo para CPU: quatro threads calculando em 4 núcleos levam o mesmo tempo que uma thread fazendo as quatro contas, mais a briga pelo lock. Paralelismo de CPU, com GIL, vem de processos. E o `+=` do capítulo 2 continua sem atomicidade: o GIL protege o interpretador. Você ele deixa morrer.

O build sem GIL (*free-threaded*, PEP 703) é suportado desde o 3.14 (PEP 779), ainda opcional. A Lastro roda o padrão, com GIL.

Modelos de servidor, ou quantas requisições cabem num processo

Modelo	Exemplos	Requisições simultâneas por processo	Limite que aparece primeiro
Prefork síncrono	Gunicorn `sync`, Apache `prefork`	1	número de processos × memória
Workers com threads	Gunicorn `gthread`, Apache `worker`	número de threads	threads × memória; GIL para CPU
Event loop	nginx, Node.js, Uvicorn	milhares	CPU de um núcleo; código que bloqueia
Híbrido	Uvicorn `--workers N`, Gunicorn com workers Uvicorn	milhares × N	CPU total; limites dos parceiros

No híbrido, um mestre abre o socket de escuta e faz fork dos workers, que herdam o descritor e disputam o `accept()` (com `SO_REUSEPORT`, cada um abre o próprio socket e o kernel distribui por hash). O mestre repõe quem morre: funeral com contratação

imediata. O Gunicorn mata o worker síncrono que passa 30 segundos sem sinal de vida (`--timeout`, padrão 30): o timeout de 30 segundos que ninguém configurou, agora com licença para matar.

O **WSGI** (PEP 3333) define a aplicação como callable síncrono que devolve a resposta inteira: um processo ou thread por requisição em andamento. O **ASGI** a define como corrotina que recebe e envia eventos, o que permite conexões longas e milhares delas num loop. O SSE do capítulo 12 só existe porque a Lastro é ASGI.

No FastAPI, rota `async def` roda no event loop; rota `def` roda num pool de threads do AnyIO, de 40 por processo. O Carlos achou a combinação perfeita: código bloqueante em `async def`, com a concorrência do modelo síncrono e a fragilidade do assíncrono. Com `def`, cada processo atenderia 40 autorizações ao mesmo tempo. Errar pela metade teria funcionado melhor.

Quantos workers, ou a conta que o fundador fez de cabeça

Chame de `C` o tempo de CPU de uma requisição e de `W` o tempo esperando I/O. Trabalho de CPU puro: um processo por núcleo; mais que isso só compra troca de contexto (o `(2 × núcleos) + 1` da documentação do Gunicorn conta com alguma espera de rede).

Espera num modelo bloqueante: cada trabalhador atende uma requisição por vez, e a vazão máxima é `trabalhadores / (C + W)`. A concorrência necessária sai da Lei de Little (capítulo 14), `L = λ × W`. Brian Goetz, em *Java Concurrency in Practice*, dá o número de threads que mantém a CPU ocupada:

```
threads = núcleos × utilização_alvo × (1 + W / C)
```

Num event loop, volta a ser um processo por núcleo, e o teto vira `núcleos / C` ou o recurso de fora que acabar primeiro: conexões do banco, limite do parceiro, memória.

A autorização da Lastro tem `C ≈ 6 ms` e `W ≈ 900 ms`: `W / C = 150`. Por Goetz, com 4 núcleos, 604 threads por máquina. Com event loop, 4 processos. O fundador configurou `--workers 3` "para aproveitar os núcleos" e acertou o número com o modelo errado. Relógio parado também acerta duas vezes por dia, e nenhum dos dois devia ter acesso de administrador.

Na Lastro

`temp-01` e `api-01` têm 4 vCPUs cada. As opções para o caminho da autorização, nas duas máquinas juntas:

Opção	Teto de autorizações/s	Custo
Como estava (síncrono em `async def`, 6 processos)	~6,7, e o resto da API para junto	nada a mudar; tudo a perder
`--workers 64` por máquina (proposta do Carlos)	~140	128 × 160 MB = 20 GB de RAM; 128 conexões HTTP/2 ao adquirente, que aceita 20; 640 conexões possíveis ao banco
Rotas `def` no pool de threads (40 por processo)	~266	240 threads disputando o GIL nos 6 ms de CPU; SSE e GraphQL continuam no loop
Clientes assíncronos, 3 processos por máquina	~750 (streams do adquirente)	trocar dois clientes; disciplina contra chamada bloqueante

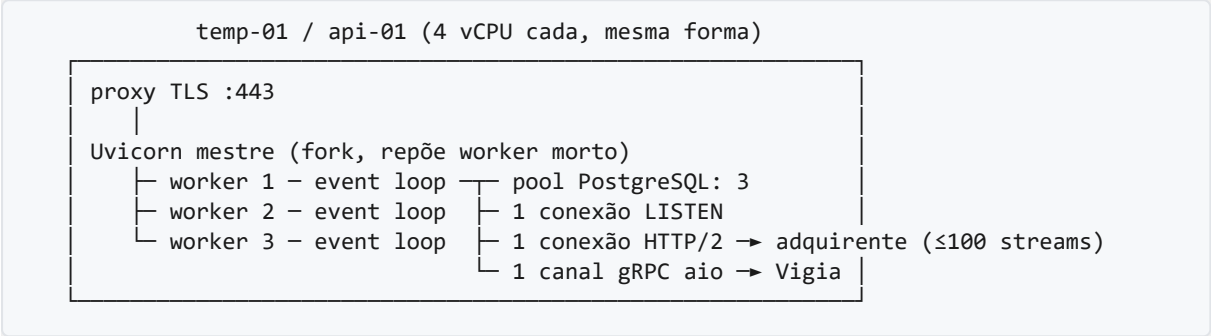
O `--workers 64` morre antes de começar: o adquirente aceita 20 conexões (capítulo 9), e cada processo abre a sua. Seriam 108 processos recusados, e o Carlos chamaria de teste de carga.

A Lastro escolhe os clientes assíncronos: `grpc.aio` para a Vigia e um cliente HTTP/2 assíncrono para o adquirente (o `httpx.AsyncClient` é um exemplo), com a mesma conexão por processo do capítulo 9. O requisito é capacidade para um trabalho 99% espera, e o event loop atende espera sem thread nem processo extra. O teto passa a ser o adquirente: 100 streams por conexão (`SETTINGS_MAX_CONCURRENT_STREAMS`), 600 nos 6 processos, 600 / 0,8 = 750 autorizações por segundo; a CPU aguentaria uns 1.000. Um semáforo de 100 por processo segura o excedente na aplicação, com prazo, em vez de estourar no parceiro.

O resto do dimensionamento:

- **Processos:** 3 por máquina. O quarto núcleo fica com o proxy TLS (capítulo 10), o sistema e, em `temp-01`, o `repasse_final_v2_agora_vai.py`, que ninguém quer ver disputando CPU com autorização.
- **Pool do banco:** 6 processos × 5 = 30 conexões possíveis, acima do teto útil de 17 a 20 do capítulo 4. O `pool_size` cai para 3: 18.

- **Nenhuma conexão do banco atravessa um `await` externo.** A cobrança é gravada numa transação curta, Vigia e adquirente são chamados sem conexão, e o resultado vai numa segunda transação curta. Segurar conexão por 0,9 segundo daria 3,3 autorizações por segundo por processo, e o `QueuePool limit` voltaria das férias.
- **CPU pesada fora do loop:** rota com mais de 20 ms de CPU vai para um processo separado. Thread não serve: com o GIL, ela paralelizaria só a culpa.



Cada worker tem uma Task que dorme 100 ms e mede quanto acordou atrasado: o **lag do event loop**, métrica com alerta acima de 50 ms. Na CI, as regras `ASYNC` do Ruff barram chamada HTTP síncrona e `time.sleep` em `async def`. Na quinta seguinte, campanha repetida: CPU a 9%, lag de 2 ms, autorização ainda em 0,9 segundo (o adquirente segue em 0,8), e o processo atendendo centenas de outras coisas em vez de olhar para o teto.

Com biblioteca de parceiro só síncrona e dezenas de requisições por processo, rotas `def` seriam a escolha; com trabalho dominado por CPU, mais processos; com tráfego pequeno, prefork síncrono, que só não aguenta 12x sem juros em capinha.

Armadilhas e casos reais

> **Armadilha:** fork depois de criar threads. O `fork()` copia só a thread que o chamou; um lock que outra thread segurava fica travado no filho para sempre, sem dono para soltar. Herança com dívida e sem inventariante. O Python 3.12 avisa com `DeprecationWarning`, e o 3.14 trocou o padrão do `multiprocessing` no Linux para `forkserver`.
>
> **Armadilha:** o driver síncrono escondido. Um `async def` com psycopg2, ou com um SDK de parceiro que usa `requests` por baixo, bloqueia o loop como o cliente do adquirente e passa no code review, porque a palavra `async` está lá, linda, mentindo para todo mundo. O lag do loop pega; os olhos, não.

> **Na vida real:** em 2 de julho de 2019, uma regra nova no WAF da Cloudflare trouxe uma expressão regular de *backtracking* catastrófico; a CPU dos núcleos que atendiam HTTP e HTTPS na rede inteira foi a 100%, e o tráfego falhou por 27 minutos. Nenhum modelo de concorrência salva CPU presa no caminho da requisição. O seu `re.match` também está ouvindo.

Resumo

- Processo tem memória e descritores próprios; threads compartilham os do processo. `fork` usa copy-on-write.
- Troca de contexto voluntária (bloqueio) ou involuntária (preempção).
- I/O não bloqueante depende de notificação de prontidão; o `epoll` custa O(prontos).
- `asyncio` é cooperativo: código bloqueante em `async def` para o processo inteiro.
- GIL: threads servem para I/O, processos para CPU.
- Prefork, threads, event loop e híbrido; WSGI síncrono, ASGI assíncrono.
- CPU: um processo por núcleo; espera bloqueante: `núcleos × (1 + W/C)` threads; event loop: teto em `núcleos / C` ou no recurso de fora.

A Lastro agora

Vigia e adquirente são chamados por clientes assíncronos, com semáforo de 100 por processo. Seguem 3 workers por máquina de 4 vCPUs, o pool caiu para 3 conexões por processo (18 no total), nenhuma conexão atravessa chamada externa, e o lag do loop tem alerta em 50 ms. O teto da autorização foi de 6,7 para 750 por segundo, com a mesma CPU que antes ficava a 4% assistindo.

O Carlos aceitou a derrota do `--workers 64` e apresentou o relatório de capacidade para a Black Friday: autorização com média de 0,9 segundo, "tranquilo". O Bazar respondeu com um contrato que fala em p99 abaixo de 1,5 segundo, e ninguém na Lastro sabe dizer qual é o p99. A média do Carlos está correta. Do mesmo jeito que a média de temperatura de um paciente com um pé no forno e outro no freezer.

Referências

- Dan Kegel, "The C10K problem". http://www.kegel.com/c10k.html
- Linux man-pages, `epoll(7)`. https://man7.org/linux/man-pages/man7/epoll.7.html

- Python, "Developing with asyncio". https://docs.python.org/3/library/asyncio-dev.html
- PEP 703, "Making the Global Interpreter Lock Optional in CPython". https://peps.python.org/pep-0703/
- PEP 779, "Criteria for supported status for free-threaded Python". https://peps.python.org/pep-0779/
- PEP 3333, "Python Web Server Gateway Interface v1.0.1". https://peps.python.org/pep-3333/
- ASGI Specification. https://asgi.readthedocs.io/en/latest/specs/main.html
- Gunicorn, "Design". https://docs.gunicorn.org/en/stable/design.html
- FastAPI, "Concurrency and async / await". https://fastapi.tiangolo.com/async/
- Ruff, regras flake8-async. https://docs.astral.sh/ruff/rules/#flake8-async-async
- Brian Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006, seção 8.2.
- Cloudflare, "Details of the Cloudflare outage on July 2, 2019". https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/

CAPÍTULO 14

Desempenho mensurável

A média do Carlos

O Carlos entrou na reunião com o Bazar levando um slide com um número só, em fonte 96: "0,9 s", a média da autorização. O Bazar levou um contrato. Cláusula 7.2: na Black Friday, o p99 da autorização fica abaixo de 1,5 segundo no pico projetado, com capacidade comprovada para o dobro do pico. Multa por hora fora.

Ninguém na Lastro sabia o p99. O fundador perguntou se p99 era o modelo novo de celular. O Carlos garantiu que tinha feito um teste de carga: 50 usuários virtuais, dez minutos, "aguenta 55 autorizações por segundo com p99 de 1,1 segundo". A projeção do Bazar para o pico é de 60. Faltavam 5, e o Carlos achou que era arredondamento.

O log do proxy discordou:

```
autorizações:  21.418     média:  0,93 s
p50:  0,87 s    p90:  1,04 s    p99:  1,31 s    p99,9:  2,9 s
```

A média estava certa. O p99 estava dentro, por 190 ms. E o teste de carga do Carlos tinha medido, com toda a precisão do mundo, o limite do próprio teste. Como transformar "rápido" num número que cabe num contrato, e como provar, antes de novembro, que a Lastro cumpre esse número no dobro do pico?

Latência, throughput e por que a média mente na cara dura

Latência é o tempo de uma operação entre dois pontos que você precisa nomear: do cliente ao cliente inclui rede e fila no kernel; do proxy ao proxy, não. **Throughput** (vazão) é quantas operações terminam por unidade de tempo. Um lote noturno tem vazão alta e latência horrível; o worker do capítulo 13 tinha latência boa e vazão de um pedido por vez.

Latência é **tempo de serviço** mais **tempo de espera** na fila de algum recurso (CPU, conexão, lock, o adquirente). O serviço é estável. A espera depende de quem chegou antes, e é ela que estraga a sua noite.

Distribuição de latência tem cauda longa: quase tudo rápido, uns poucos muito lentos (uma pausa, um retry, uma fila momentânea). A média esconde a cauda. Cem requisições de 100 ms e uma de 10 segundos dão média de 198 ms, um número que nenhuma requisição teve. A média é a única estatística capaz de descrever com exatidão um paciente com um pé no forno e o outro no freezer: 36,5 °C, estável, pode dar alta.

Um **percentil** p é o valor abaixo do qual estão p% das medidas. O p50 é a mediana: metade das requisições foi mais rápida. O p99 é o valor que só 1% ultrapassa. O p99,9, 1 em mil. Em 21.418 autorizações por hora, o 1% acima do p99 são 214 compradores por hora olhando a tela girar. Num cliente que faz 50 chamadas por página, a chance de pelo menos uma passar do p99 é 1 − 0,99^50 ≈ 39%. O seu p99 é a experiência típica de quem usa muito o sistema. O p99,9 é a dos seus melhores clientes, que são os que mais reclamam, com razão e com URGENTE!!!!.

> **Na vida real:** em "The Tail at Scale" (2013), Jeffrey Dean e Luiz André Barroso mostram o efeito em sistemas com fan-out: um servidor com p99 de 1 segundo, consultado em paralelo com outros 99 iguais por cada requisição, faz 63% das requisições do usuário passarem de 1 segundo. A cauda de um vira o corpo de todos.

Percentil não se soma nem se tira média: na média dos p99 de seis processos, o processo doente que carrega a cauda é diluído pelos cinco saudáveis. O que se agrega são **histogramas**: cada processo conta requisições por faixa (0–50 ms, 50–100 ms, ...), os contadores se somam, e o percentil sai do histograma somado, com o erro da largura da faixa. É o que fazem o HdrHistogram e os histogramas do Prometheus (capítulo 38). Média de percentil tem cara de estatística e alma de horóscopo.

A Lei de Little, ou a equação que entrega o teste do Carlos

Para qualquer sistema estável (o que entra é o que sai, em média, no período):

```
L = λ × W
L: itens dentro do sistema, em média
λ: taxa de chegada (= vazão, no estado estável)
W: tempo médio de cada item lá dentro
```

John Little provou em 1961 que isso vale sem hipótese sobre a distribuição das chegadas ou do serviço, nem sobre a ordem de atendimento: requisições num servidor, conexões num pool, streams no adquirente.

No teste do Carlos, o gerador usava 50 usuários num **modelo fechado**: cada um manda, espera a resposta e só então manda a próxima. Logo L ≤ 50 e, com W = 0,9 s, λ ≤ 50 / 0,9 = 55,6 por segundo, com qualquer servidor do universo. O "aguenta 55" era a Lei de Little medindo o gerador. O servidor nem foi apresentado à Black Friday.

A mesma lei dimensiona recursos: no pico, 60 × 0,9 = 54 autorizações em andamento no adquirente. É a primeira conta de qualquer dimensionamento e a última defesa contra um relatório de teste de carga.

Utilização e filas, ou por que 90% de CPU é uma emergência

Utilização (ρ) é a fração do tempo em que um recurso está ocupado: taxa de chegada vezes tempo de serviço, dividido pelo número de servidores. Um núcleo que recebe 200 requisições por segundo de 3 ms está 60% ocupado.

A intuição diz que um recurso a 90% ainda tem 10% de folga. A teoria de filas diz que ele está a um espirro do colapso. No modelo M/M/1 (chegadas de Poisson, serviço exponencial, um servidor), o tempo médio no sistema é:

```
W = S / (1 − ρ)        S: tempo de serviço médio
```

ρ	W, em múltiplos de S
0,5	2
0,7	3,3
0,8	5
0,9	10
0,95	20
0,99	100

De 50% para 90%, a vazão cresce 80% e a latência média, 400%. De 90% para 99%, a vazão cresce 10% e a latência, dez vezes. É uma hipérbole com a assíntota em ρ = 1, e o seu sistema sobe por ela de olhos fechados. No M/M/1, o tempo no sistema é exponencial, e o p99 fica em ln(100) ≈ 4,6 vezes a média: com ρ = 0,9, o p99 é 46 vezes o tempo de serviço.

Sistemas reais têm várias filas e distribuições piores. A aproximação de Kingman, para filas G/G/1, mostra o outro ingrediente: a espera é proporcional a ρ / (1 − ρ) e à variabilidade das chegadas e do serviço (a média dos quadrados dos coeficientes de

variação). Serviço irregular piora a fila na mesma utilização. Num event loop, uma consulta GraphQL de 15 ms de CPU entre GETs de 1,5 ms é exatamente essa variabilidade: todo mundo atrás dela espera.

Recurso de que a latência depende se dimensiona bem abaixo de ρ = 1 no pico, e o teste de carga acha o joelho. Banco "a 95% para não desperdiçar" economiza um servidor e paga em latência com juros de agiota.

A Lei de Amdahl, ou otimizar o que não importa com muita competência

Gene Amdahl, em 1967, sobre acelerar uma parte de um trabalho:

```
aceleração = 1 / ((1 − p) + p / s)
p: fração do tempo que pode ser acelerada
s: quanto essa fração fica mais rápida
```

Com s infinito, a aceleração máxima é 1 / (1 − p). Se 95% de um job paraleliza, nem infinitos núcleos o fazem mais de 20 vezes mais rápido: os 5% seriais viram o job inteiro.

A lei vale para latência também. Da autorização de 0,9 s, o código da Lastro (validar, gravar, serializar) gasta uns 50 ms; o resto é Vigia e adquirente. p ≈ 0,056: deixar esse trecho infinitamente rápido dá 1 / 0,944 ≈ 1,06. Seis por cento. O Carlos tinha aberto um épico "migrar autorização para Rust". Amdahl fechou o épico quase seis décadas antes de ele ser aberto.

Números de latência para fazer conta de cabeça

A lista popularizada por Jeff Dean ("Numbers Everyone Should Know", 2009, atualizada por terceiros desde então) dá ordens de grandeza aproximadas, que variam com o hardware e servem para comparar, nunca para citar em contrato:

Operação	Ordem de grandeza
Referência ao cache L1	~1 ns
Acesso à memória principal	~100 ns
Leitura aleatória em SSD	dezenas a centenas de µs
Ida e volta na mesma zona de um datacenter	~0,5 ms
Consulta simples ao PostgreSQL da Lastro, rede inclusa (capítulo 6)	~1 ms
Ida e volta São Paulo–Virgínia (capítulo 8)	~100 ms

Operação	Ordem de grandeza
Autorização no adquirente	~800 ms

De cima para baixo, nove ordens de grandeza: oito milhões de acessos à memória cabem numa autorização. Discutir se um dicionário Python é mais rápido que uma dataclass no caminho da autorização é discutir o troco da passagem de um voo que atrasou seis horas.

Teste de carga, ou como o gerador mente para você

No **modelo fechado**, um número fixo de usuários manda, espera a resposta (e um tempo de pensar, se houver) e manda de novo. Servidor lento, usuários mandam menos: a carga se ajusta à capacidade e a fila nunca cresce de verdade. Serve para população limitada que espera, como 40 atendentes usando um sistema interno.

No **modelo aberto**, as requisições chegam a uma taxa definida, independente das respostas. Servidor lento, a fila cresce. É o comportamento de milhares de compradores independentes, e a Black Friday é um modelo aberto com o agravante de que o comprador, quando demora, clica de novo.

O modelo fechado ainda carrega a **coordinated omission**, nome dado por Gil Tene: o gerador espera a resposta antes de mandar a próxima e, sem querer, combina com o servidor de não medir justamente as requisições que teriam sofrido a lentidão. No teste do Carlos, o simulador do adquirente (escrito em Java, por motivos de herança) parou 12 segundos numa pausa de coleta de lixo. Veja a conta:

	Modelo fechado (50 usuários)	Modelo aberto (55/s)
Requisições em 10 min	~33.000	33.000
Afetadas pela pausa de 12 s	50 (uma por usuário)	~660 (as que chegaram durante a pausa)
Fração afetada	0,15%	2%
p99 medido	1,1 s	~6,9 s

No fechado, os 50 usuários ficaram parados juntos esperando, e cada um registrou uma requisição lenta. As 610 requisições que deveriam ter saído nesses 12 segundos simplesmente não existiram. Um comprador de verdade teria existido, e teria esperado. O p99 do Carlos tinha omitido 660 pessoas, coordenadamente, como uma ata de reunião de condomínio.

A correção: carga em modelo aberto, a taxa constante, com cada latência medida a partir do instante em que a requisição *deveria* ter saído. O wrk2 faz isso por padrão; o k6 tem executores de taxa de chegada (`constant-arrival-rate`); o Gatling tem injeção aberta.

O resto do teste também precisa ser honesto: hardware e volume de dados de produção (um ledger de 1.000 linhas cabe em cache e mente sobre índice), a mistura de requisições do pico, duração maior que os ciclos que atrapalham (autovacuum, coleta de lixo), aquecimento descartado e o gerador monitorado, porque gerador a 100% de CPU mede a si mesmo.

Na Lastro

O número. A latência da autorização passa a ser medida no proxy, do recebimento da requisição ao último byte da resposta, num histograma por processo, agregado a cada minuto. Cada etapa (Vigia, adquirente, as duas transações) ganha o seu histograma. A definição formal como SLI e a meta como SLO ficam para o capítulo 39; o contrato de 1,5 s já usa esta medida.

O orçamento. O p99 de 1,5 s é dividido entre as etapas:

Etapa	p99 medido	Orçamento
Proxy, TLS, fila do event loop	4 ms	20 ms
Validação e primeira transação	9 ms	20 ms
Vigia (timeout de 120 ms, capítulo 12)	58 ms	120 ms
Adquirente	1.180 ms	1.200 ms
Segunda transação e resposta	7 ms	20 ms
Folga		120 ms
Total		**1.500 ms**

Somar p99 de etapas não dá o p99 da soma. Com etapas independentes, a soma costuma ficar acima do p99 real, porque raramente todas estão no pior ao mesmo tempo, mas sem garantia; o teste de carga confere. O que a tabela mostra: 80% do orçamento é do adquirente, e a Lastro controla 60 ms. Amdahl avisou.

A projeção. O pico da Black Friday, somando Bazar e Hornero: 60 autorizações de cartão, 140 cobranças Pix, 900 GETs e 40 consultas GraphQL por segundo. CPU medida por requisição: 6, 4, 1,5 e 15 ms. Demanda: 0,36 + 0,56 + 1,35 + 0,60 = 2,87 segundos de

CPU por segundo. Com 6 workers (um núcleo cada), ρ = 0,48 no pico e 0,96 no dobro do pico. A cláusula do dobro mata o arranjo atual: a 0,96, a fila do event loop sozinha come a folga.

A Lastro acrescenta `api-02`, igual a `api-01`: 4 vCPUs, 3 workers, proxy TLS, certificado próprio por ACME (capítulo 10). Com 9 workers, ρ = 0,32 no pico e 0,64 no dobro. A regra adotada: ρ de CPU por worker até 0,7 no dobro do pico. O resto das contas, por Little:

- **Pool do banco:** 9 processos × 3 = 27 conexões, acima do teto útil de 17 a 20 (capítulo 4). O `pool_size` cai para 2: 18. A demanda no pico é de 3,2 conexões ocupadas no total (soma de taxa × tempo de posse de cada transação), 6,4 no dobro: cabe.
- **Adquirente:** 54 autorizações em andamento no pico, 108 no dobro; 9 conexões HTTP/2 × 100 streams = 900, e 9 das 20 conexões contratadas.
- **Contas quentes:** a comissão do Bazar recebe 200 créditos por segundo no pico; 16 baldes com posse de 2 ms (capítulo 4) dão teto de 8.000.

Mais uma máquina, e não máquinas maiores: núcleos extras só ajudam com mais workers, e a conta do pool é a mesma; a terceira máquina ainda dá isolamento de falha (perdida uma, 6 workers aguentam o pico a ρ = 0,48, não o dobro). Máquinas de 8 vCPUs seriam a escolha se o custo fosse por máquina; autoscaling fica para o capítulo 29.

O teste. Modelo aberto, taxa constante, latência do instante planejado, mistura da projeção, volume do ledger copiado. Vigia e adquirente viram simuladores que sorteiam a latência dos histogramas reais de produção, porque a sandbox do adquirente responde em 40 ms e faria qualquer número da Lastro parecer gênio. O coletor de lixo do simulador agora é monitorado. Três patamares de 30 minutos:

Carga	6 workers (p99 autorização)	9 workers (p99 autorização)
1× (60/s)	1,33 s	1,32 s
1,5×	1,41 s	1,33 s
2× (120/s)	2,7 s	1,36 s

Com 6 workers, o dobro estoura o contrato por 1,2 segundo; com 9, sobra folga. A hipérbole apareceu onde a conta disse, e a equipe sentiu algo próximo de fé.

Armadilhas e casos reais

> **Armadilha:** dashboard com média de p99 por host. O processo doente vira um sexto de um número e some. Agregue histogramas, ou mostre o pior host.
>
> **Armadilha:** medir só dentro da aplicação. O cronômetro do FastAPI começa quando o event loop pega a requisição; o tempo na fila do `accept` do kernel e no proxy fica de fora, e é exatamente onde a fila mora quando o loop está ocupado. Medir no ponto mais externo que você controla.
>
> **Armadilha:** a sandbox do parceiro como alvo do teste. Responde rápido, sem cauda, e às vezes com um limite que vira o seu gargalo. Ou o parceiro liga perguntando por que você mandou 216 mil autorizações numa terça.
>
> **Na vida real:** o wrk2, de Gil Tene, nasceu de um fork do wrk para gerar carga a taxa constante e corrigir a coordinated omission; o README compara a cauda medida pelos dois geradores contra o mesmo servidor, e a diferença chega a ordens de grandeza. Medir mal produz números melhores, e por isso tanta gente mede mal.

Resumo

- Latência é serviço mais espera, entre pontos nomeados; vazão é conclusões por tempo.
- Média esconde a cauda; percentis a mostram e se agregam por histograma.
- Little: `L = λ × W`. Filas: `W = S / (1 − ρ)`, explosão perto de ρ = 1.
- Amdahl: acelerar a parte p limita o ganho a `1 / (1 − p)`.
- Teste de carga em modelo aberto, latência do instante planejado, sem coordinated omission.

A Lastro agora

A autorização tem um número: p99 de 1,31 s, medido no proxy, com histograma por etapa e orçamento de 1,5 s em que o adquirente fica com 1,2. Entrou `api-02` (4 vCPUs, 3 workers), totalizando 9 workers; o pool caiu para 2 conexões por processo (18); a regra é ρ de CPU até 0,7 no dobro do pico. O teste de carga em modelo aberto, com simuladores alimentados pelos histogramas reais, sustentou 120 autorizações por segundo com p99 de 1,36 s. O Carlos refez o slide: agora são três números em fonte 72, e um deles tem vírgula.

Toda a folga, porém, depende de o adquirente manter o p99 de 1,18 s na Black Friday dele também. Na sexta, às 21h40, o p99 do adquirente foi para 4 segundos, e alguém configurou um timeout de 1,3 segundo para proteger o contrato. O timeout disparou 3.000 vezes em dez minutos, a Lastro respondeu "recusada" para todas, e o financeiro descobriu no arquivo do dia seguinte que 412 delas tinham sido aprovadas. O cronômetro funcionou. O dinheiro não leu o contrato.

Referências

- John D. C. Little, "A Proof for the Queuing Formula: L = λW", *Operations Research* 9(3), 1961.
- Gene Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities", AFIPS Spring Joint Computer Conference, 1967.
- J. F. C. Kingman, "The single server queue in heavy traffic", *Mathematical Proceedings of the Cambridge Philosophical Society* 57(4), 1961.
- Jeffrey Dean e Luiz André Barroso, "The Tail at Scale", *Communications of the ACM* 56(2), 2013. https://cacm.acm.org/research/the-tail-at-scale/
- Jeff Dean, "Designs, Lessons and Advice from Building Large Distributed Systems", LADIS 2009 (lista "Numbers Everyone Should Know").
- Gil Tene, wrk2 (README sobre coordinated omission). https://github.com/giltene/wrk2
- HdrHistogram. https://hdrhistogram.github.io/HdrHistogram/
- Grafana k6, "Open and closed models". https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/
- Bianca Schroeder, Adam Wierman e Mor Harchol-Balter, "Open Versus Closed: A Cautionary Tale", NSDI 2006.

PARTE III

Dinheiro entre sistemas

CAPÍTULO 15

Falhas parciais

Recusada, segundo o cronômetro

Sexta-feira de Black Friday, 21h40, e o adquirente da Lastro resolveu ter a Black Friday dele também: o p99 das autorizações foi de 1,18 para 4 segundos. Na Lastro, alguém tinha configurado um timeout de 1,3 segundo na chamada ao adquirente, "para proteger o contrato". Esse alguém tem nome, crachá e um histórico de commits que o RH guarda numa pasta à parte. Foi o Carlos.

O contrato exige p99 abaixo de 1,5 s, então o que passasse de 1,3 s virava `recusada`. Em dez minutos, foram 3.000. O p99 da Lastro ficou lindo. O Bazar cancelou 3.000 pedidos.

No sábado, o financeiro cruzou o arquivo do adquirente com a planilha: das 3.000 "recusadas", 412 tinham sido aprovadas. 412 compradores com o limite reservado, sem pedido e com um e-mail do Bazar dizendo "tente novamente". Muitos tentaram. O financeiro desfez as 412 reservas uma por uma no portal do adquirente, numa planilha nova chamada `bf_desfazer_FINAL.xlsx`, que agora divide com a planilha de conciliação o título de banco de dados mais confiável da empresa.

O cronômetro funcionou perfeitamente. O erro foi o que a Lastro fez com a exceção: transformou "não sei" em "não". Quando uma chamada a um parceiro estoura o timeout, o comprador foi cobrado ou não? E o que um sistema de pagamentos faz com uma pergunta que não tem resposta?

As formas criativas de uma chamada remota dar errado

Uma chamada de função tem dois finais: retorna ou levanta exceção. Pela rede aparece um terceiro: **resultado desconhecido**. É a **falha parcial**: parte do sistema funciona, parte não, e um lado não consegue saber com certeza o estado do outro. As falhas se classificam pelo comportamento visível:

- **Crash (parada):** o componente para de responder, ou volta reiniciado e sem memória. Se os outros detectam a parada com certeza, chama-se *fail-stop*. Quase nunca detectam.

- **Omissão:** uma mensagem some, na ida ou na volta.
- **Falha de tempo:** a resposta vem certa e tarde demais. A do adquirente na sexta.
- **Bizantina:** o componente responde coisa errada, por bug ou malícia. Com parceiros confiáveis e TLS, sobra o bug, que já é bizantino o suficiente.

Do lado de quem chamou, crash, omissão e lentidão produzem o mesmo sintoma: silêncio. E silêncio tem quatro explicações que você não consegue distinguir:

```
1. a requisição se perdeu antes de chegar          -> nada aconteceu
2. o parceiro está processando, devagar            -> vai acontecer
3. o parceiro processou e a resposta se perdeu     -> aconteceu
4. o parceiro morreu no meio                       -> boa sorte
```

A Lastro, na sexta, apostou na explicação 1 para todas as 3.000. Acertou 2.588 e errou 412, uma taxa de acerto que seria excelente para um meteorologista e é demissão para um sistema de pagamentos.

E há a **falha cinzenta** (*gray failure*), descrita por pesquisadores da Microsoft em 2017: saudável para o monitoramento (health check `200`), doente para os clientes. O monitoramento diz que está tudo bem. O Bazar diz URGENTE!!!!. Só um dos dois paga a sua fatura.

Nenhum timeout distingue um parceiro morto de um lento. Ele só diz que você parou de esperar, e o resto do capítulo trata do que fazer depois.

Timeout, ou o momento exato de desistir de alguém

Um timeout é uma decisão de negócio disfarçada de número. Uma chamada HTTP tem dois que importam. O de **conexão** limita TCP + TLS até a conexão existir, e tem uma propriedade preciosa: quando estoura, a requisição **com certeza não foi enviada**. É a explicação 1, garantida. O de **leitura** limita a espera pela resposta e estoura com a requisição já entregue: "não sei". Há ainda o de escrita e o total, que limita a operação inteira.

Curto demais, ele dispara com o parceiro saudável: com p99,9 de 2,7 s num dia normal, um timeout de 1,3 s faz, por construção, mais de 0,1% de falsas falhas, e num dia ruim muito mais. Com retry, cada uma vira carga extra no parceiro lento. O timeout curto mede a febre e aplica mais febre.

Longo demais, cada chamada presa segura semáforo, conexão e memória. Pela Lei de Little (capítulo 14), em andamento = taxa de chegada × espera. A 60 autorizações por segundo com o parceiro travado, um timeout de 8 s acumula 480 chamadas penduradas; um de 30 s, 1.800.

A regra: o timeout fica **acima da cauda saudável do parceiro** (o p99,9 de um dia normal, com folga), para disparar só em falha de verdade, e **abaixo da paciência de quem chamou você**, para a resposta chegar a alguém que ainda está ouvindo. Se as duas condições não cabem juntas, o problema está no contrato.

Não ter timeout também é uma escolha, tomada por omissão. A `requests` espera para sempre sem `timeout=`; o `httpx` usa 5 s por padrão. Uma chamada sem timeout é um parente desaparecido: o processo não pode fazer o velório nem reaproveitar o quarto.

Retry, ou insistir com quem já disse que está ocupado

Tentar de novo resolve falhas transitórias e cria duas pragas: a operação executada duas vezes e a multidão insistindo ao mesmo tempo.

Quando é seguro repetir

Uma repetição é segura em dois casos. No primeiro, você **sabe** que a primeira tentativa não teve efeito: timeout de conexão, conexão recusada, `GOAWAY` do HTTP/2 com o stream acima do último ID processado (capítulo 9), `429` ou `503` emitidos antes de processar. No segundo, a operação é **idempotente**: repeti-la tem o mesmo efeito que executá-la uma vez. Consultar uma autorização é idempotente. Autorizar um cartão, sem mecanismo extra, reserva o limite duas vezes, e o comprador ganha duas linhas na fatura e um motivo para conhecer o Procon.

Retry de operação não idempotente depois de timeout de leitura transforma "não sei se cobrei" em "talvez tenha cobrado duas vezes". O capítulo 16 mostra como tornar a autorização segura para repetir; até lá, não se repete.

Backoff exponencial e jitter

Repetir imediatamente devolve a carga, multiplicada, ao servidor que caiu por excesso dela. O **backoff exponencial** dobra a espera a cada tentativa (100 ms, 200 ms, 400 ms, com um teto). Não basta: mil clientes que falharam no mesmo instante esperam os mesmos 100 ms e voltam juntos, em ondas sincronizadas, um flash mob de DDoS com convite agendado.

O **jitter** sorteia a espera. A variante *full jitter*, analisada por Marc Brooker no blog de arquitetura da AWS em 2015, sorteia uniformemente entre zero e o valor exponencial:

```
espera = uniforme(0, min(teto, base × 2^tentativa))
base 0,1 s, teto 2 s:  0–0,1 | 0–0,2 | 0–0,4 | 0–0,8 | 0–1,6 s
```

Nas simulações do artigo, full jitter reduz muito o trabalho total do servidor, porque espalha as tentativas. Sem jitter, o backoff só agenda o próximo ataque.

Retry storm, ou a multiplicação dos pães, só que de requisições

Uma requisição atravessa três camadas (cliente → API → serviço → banco), e cada camada faz até 3 retries, por segurança. O banco engasga. Cada chamada ao banco vira 4 tentativas; cada chamada ao serviço, que falhou, também vira 4, cada uma com as suas 4 ao banco; e o cliente repete por cima.

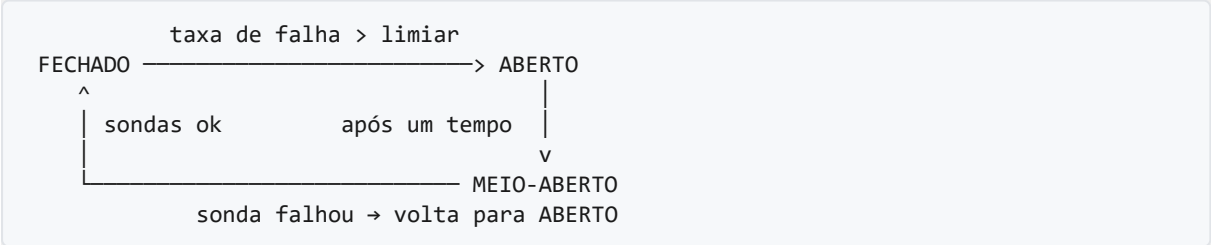
```
amplificação = (1 + retries) ^ camadas = 4 ^ 3 = 64
```

O banco engasgou com 100 consultas por segundo e recebe até 6.400. É a única técnica de recuperação conhecida que mata o paciente com o desfibrilador.

As defesas são duas: repetir **em uma camada só**, e as outras propagam o erro; e um **orçamento de retries** (*retry budget*), que limita as repetições a uma fração das requisições normais, por exemplo 10%. Quando o parceiro piora, o orçamento acaba e os retries param, justamente quando fariam mais estrago. O gRPC implementa isso como *retry throttling*: um balde de tokens que perde um token a cada falha, ganha uma fração de token a cada sucesso e, abaixo da metade, desliga os retries.

Circuit breaker e bulkhead, ou a arte de largar quem não te merece

Se o adquirente devolve timeout para 80% das chamadas, mandar a próxima é segurar um comprador por segundos para somar carga num parceiro afogado. O **circuit breaker** (disjuntor), popularizado por Michael Nygard em *Release It!*, é uma máquina de três estados em volta da chamada:



Fechado, as chamadas passam e o disjuntor conta falhas numa janela deslizante. Passou do limiar (com um mínimo de chamadas, para três falhas às 4h não abrirem o circuito), ele **abre**: as chamadas falham na hora, sem rede. Depois de um intervalo, fica **meio-aberto** e deixa passar poucas sondas; se funcionam, fecha, se falham, abre de novo. Timeout conta como falha, porque lentidão é a falha que mais consome recursos.

Quem chama recebe "não" em microssegundos em vez de "não sei" em segundos, e o parceiro doente para de apanhar e tem chance de se recuperar, uma gentileza rara em sistemas distribuídos e em ex-namorados.

O **bulkhead** (antepara) isola recursos por dependência. Se adquirente e banco do Pix dividem o mesmo semáforo, a lentidão de um ocupa tudo e o outro, saudável, morre de contágio. Com um limite por dependência, o lento esgota só o compartimento dele, e a chamada seguinte falha na hora em vez de entrar numa fila sem fim. O semáforo de 100 por processo do capítulo 13 já era um bulkhead, só que um para tudo, o que faz dele mais condomínio que antepara.

Deadlines e degradação, ou quanto tempo ainda resta de vida

Timeouts por chamada somam mal. A API dá 2 s ao serviço, o serviço dá 2 s ao Vigia e mais 2 s ao adquirente: a requisição pode levar 4 s, e o cliente desistiu aos 3. O servidor termina tudo com zelo, serializa o JSON com carinho e entrega para um socket fechado. Psicografia: resposta para quem já partiu.

Um **deadline** é um instante absoluto ("responder até t0 + 8 s"); cada etapa usa no máximo o tempo restante. Na rede, propaga-se o **tempo restante**, e não o instante, porque os relógios das máquinas discordam (capítulo 20). O `grpc-timeout` que a Lastro manda ao Vigia (capítulo 12) é exatamente isso. No HTTP não há cabeçalho padrão.

O deadline traz o **cancelamento**: quando o cliente desiste, o trabalho que só existia para ele para. Um servidor ASGI recebe o evento de desconexão, e o asyncio cancela a Task. Cancelar uma consulta ao Vigia é economia; cancelar uma autorização já enviada ao adquirente é impossível.

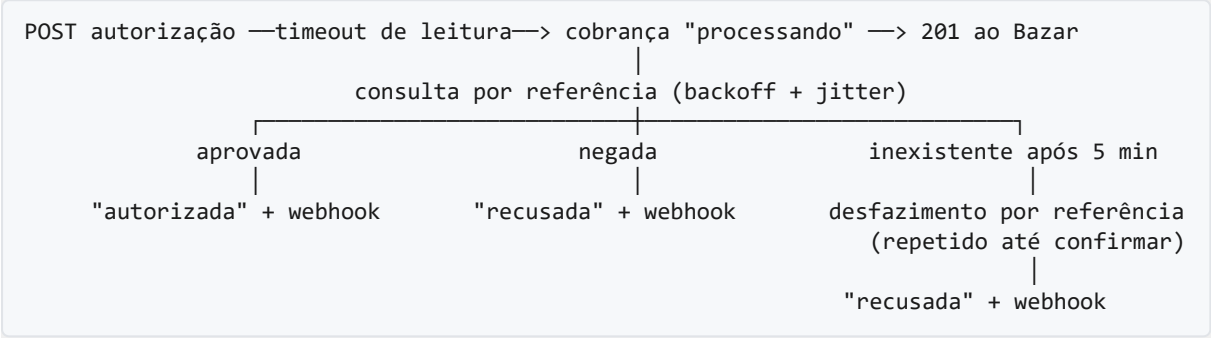
Degradação controlada é decidir antes do incidente o que o sistema faz sem cada dependência: *fail-open* (o antifraude caiu, aprova assim mesmo) ou *fail-closed* (recusa tudo). A certa depende do que custa mais, a fraude que passou ou a venda perdida. O que não pode é a postura ser decidida pelo `except` que o Carlos escreveu às 2h.

Na Lastro: um "não sei" honesto

O terceiro final ganha nome: a cobrança de cartão tem o estado `processando` (autorização enviada, resultado desconhecido). O enum de estado é aberto desde o capítulo 11, então é mudança aditiva, com 30 dias de changelog. No timeout de leitura, o

`POST /v1/cobrancas` responde `201` com `estado: processando`, e o resultado chega por webhook.

Para resolver, a Lastro usa o contrato de integração do adquirente: toda autorização leva uma referência única (o ID da cobrança), e o adquirente permite consultar e **desfazer** por referência. Um processo de resolução consulta cada `processando` com backoff exponencial e full jitter (1 s, 2 s, 4 s, até 60 s):



O ramo da direita existe porque "inexistente" ainda pode ser a explicação 2, a autorização chegando atrasada. O desfazimento, aceito mesmo para referência desconhecida, bloqueia qualquer autorização posterior com a mesma referência e fecha a porta. Consulta e desfazimento são idempotentes por referência, então repeti-los é seguro.

As alternativas para o momento do timeout:

Opção	Comprador	Bazar	Risco de dinheiro
Responder `recusada` (sexta)	Limite reservado sem pedido	Cancela pedido	Cobrança sem venda: 412 casos
Esperar sem limite	Tela girando até o Bazar desistir (10 s)	Timeout do lado dele	Mesmo "não sei", só que no Bazar
`processando` + resolução	Espera segundos a minutos	Trata 1 estado novo	Nenhum: todo caso termina autorizado ou desfeito

A Lastro fica com a terceira, porque o requisito é dinheiro sem perda nem duplicidade, e ela é a única em que todo resultado desconhecido termina num estado conhecido. Sem consulta e desfazimento por referência no adquirente, a saída honesta seria esperar o máximo possível e deixar os desconhecidos para a conciliação diária (capítulo 18).

O fluxo de autorização ganha **deadline de 8 s** a partir da chegada no proxy, abaixo dos 10 s em que o Bazar desiste (capítulo 13):

```
t0 ── validação + tx1 ── Vigia: min(120 ms, restante) ── adquirente: restante − 300 ms
── tx2 + resposta

(reserva de 300 ms)
```

O adquirente recebe cerca de 7,5 s, bem acima do p99,9 saudável dele (perto de 2,7 s no histograma por etapa do capítulo 14). Na sexta, com p99 de 4 s, quase todas as autorizações teriam só demorado. A Lastro estouraria o p99 do contrato e pagaria a multa da cláusula 7.2, um número com vírgula. Os 412 compradores custaram uma planilha, três dias do financeiro e uma reunião em que o Bazar disse "confiança" onze vezes. Entre violar um percentil e mentir sobre dinheiro, o runbook agora escolhe o percentil.

Conexão ao adquirente: 500 ms, com um retry, porque nada foi enviado. Leitura: nunca repetida.

Bulkheads por parceiro, por processo: adquirente 100, Vigia 50, banco do Pix 30. Com o do adquirente cheio, a cobrança falha na hora com `503`, `Retry-After: 2` e o erro RFC 9457 `adquirente-indisponivel`, sem enviar nada. Pela Lei de Little, 9 processos × 100 slots = 900 em andamento; com o adquirente travado em 7,5 s, isso sustenta 120 autorizações por segundo, o dobro do pico. Acima disso, a fila só aumentaria a espera de quem já não vai ser atendido a tempo.

O circuit breaker é **por parceiro e por processo**: abre com mais de 50% de falhas (timeouts incluídos) numa janela de 10 s com pelo menos 20 chamadas, fica aberto 5 s e sonda com 3. Aberto, o marketplace recebe o mesmo `503` sem chamada nenhuma ao adquirente. Cada um dos 9 processos aprende sozinho: até 9 × 20 chamadas provam o parceiro doente antes de todos abrirem, em troca de não depender de um estado compartilhado que também pode falhar.

O Vigia ganha degradação definida: com timeout de 120 ms ou circuito aberto, cobranças de até R$ 300,00 seguem com as regras locais (limite por cartão em 24 h, no PostgreSQL) e as acima são recusadas com motivo `antifraude_indisponivel`. Fail-open embaixo, fail-closed em cima, fronteira no runbook, com data e sem o Carlos.

Retries só na resolução e nos timeouts de conexão, com orçamento de 10% por parceiro. O guia de integração pede que o cliente espere pelo menos 10 s.

Armadilhas e casos reais

> **Na vida real:** em 20 de setembro de 2015, o DynamoDB em us-east-1 passou quase cinco horas com erros altos. Os servidores de armazenamento renovam periodicamente com um serviço de metadados a lista das partições que lhes cabem; os índices secundários globais tinham engordado essas listas, e depois de uma breve falha de rede as respostas passaram do tempo-limite. Quem não renovava parava de atender e tentava de novo, o serviço de metadados ficava mais lento, e os servidores

> saudáveis também falhavam. A AWS teve de pausar os pedidos de metadados para conseguir acrescentar capacidade. Um componente ficou lento, e o sistema inteiro decidiu ajudá-lo até a morte.
>
> **Armadilha:** retry em cima de retry. O Carlos pôs `@retry(tentativas=5)` na função que chama o banco do Pix, cujo cliente HTTP já repete 3 vezes. Quinze tentativas por chamada, e o banco parceiro abriu um chamado perguntando se a Lastro estava bem.
>
> **Armadilha:** timeout da aplicação maior que o do proxy. O proxy corta em 60 s (capítulo 12); um timeout de 90 s na aplicação nunca dispara, e o marketplace recebe um `504` que nenhum código trata como "não sei".
>
> **Armadilha:** health check que mede a vida do processo e ignora a do parceiro. Na sexta, os 9 processos respondiam `200` no `/saude` e "recusada" no `/v1/cobrancas`. Falha cinzenta de manual, e o Bazar soube antes, de novo.

A mais cara: o circuit breaker que conta recusa como falha. Uma recusa do emissor é o adquirente funcionando, rápido. Contar recusas abre o circuito numa noite de muita fraude, que é quando você mais precisa do adquirente, e só se descobre no postmortem, a autópsia em que o morto é você.

Resumo

- Chamada remota tem três finais: sucesso, falha e desconhecido; timeout de leitura cai no terceiro.
- Falhas: crash, omissão, tempo, bizantina; a cinzenta engana o monitoramento.
- Timeout acima da cauda saudável do parceiro e abaixo da paciência de quem chamou; Little dá o custo de esperar.
- Retry só sem efeito comprovado ou com operação idempotente; backoff exponencial com jitter, uma camada, orçamento.
- Circuit breaker falha rápido e poupa o parceiro; bulkhead isola recursos por dependência.
- Deadline propagado como tempo restante, com cancelamento; degradação decidida antes do incidente.

A Lastro agora

A cobrança de cartão tem um estado para "não sei", `processando`, resolvido por consulta e desfazimento por referência. A autorização tem deadline de 8 s, conexão de 500 ms com um retry, leitura nunca repetida, bulkheads por parceiro (100, 50, 30), circuit breaker por parceiro e processo, orçamento de retries de 10% e o Vigia degradando em R$ 300,00. A `bf_desfazer_FINAL.xlsx` foi arquivada ao lado da planilha de conciliação, que ganhou uma irmã mais nova e igualmente mais confiável que o sistema.

A consultoria Java do Bazar leu o guia e implementou exatamente o que este capítulo ensina: timeout de 10 s, três retries, backoff exponencial, full jitter, uma obra de arte. No `POST /v1/cobrancas`. Na terça em que a Lastro levou 7,8 s e o pool de conexões do próprio Bazar somou mais 2,6, a Lastro criou a cobrança, o Bazar desistiu, tentou de novo com elegância estatística, e o comprador pagou duas vezes, com jitter.

Referências

- Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2ª ed., Pragmatic Bookshelf, 2018.
- Marc Brooker, "Exponential Backoff And Jitter", AWS Architecture Blog, 2015: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Peng Huang et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems", HotOS 2017.
- Martin Fowler, "CircuitBreaker": https://martinfowler.com/bliki/CircuitBreaker.html
- gRPC, "Deadlines": https://grpc.io/docs/guides/deadlines/
- gRPC, "Retry" (retry throttling): https://grpc.io/docs/guides/retry/
- AWS, "Summary of the Amazon DynamoDB Service Disruption and Related Impacts in the US-East Region" (setembro de 2015): https://aws.amazon.com/message/5467D2/
- Requests, "Timeouts": https://requests.readthedocs.io/en/latest/user/advanced/#timeouts
- HTTPX, "Timeouts": https://www.python-httpx.org/advanced/timeouts/

CAPÍTULO 16

Idempotência e semântica de entrega

Duas vezes, com jitter

O comprador comprou um fone de R$ 349,90 no Bazar e recebeu duas notificações do banco: duas compras de R$ 349,90, mesmo estabelecimento, com 3 segundos de diferença. Ele não clicou duas vezes. Quem clicou duas vezes foi a consultoria Java do Bazar, por ele, com backoff exponencial e full jitter, exatamente como o guia mandava.

A sequência daquela terça, cronometrada pelo financeiro com a serenidade de quem já não espera nada:

```
10h14:02,0  Bazar envia POST /v1/cobrancas (pedido 88213)       [timeout 10 s]
10h14:04,6  ...pool de conexões do Bazar libera a requisição
10h14:12,4  Lastro responde 201, estado "processando"            [Bazar já desistiu]
10h14:13,1  Bazar repete: POST /v1/cobrancas (pedido 88213)
10h14:14,0  Lastro responde 201, estado "autorizada", outra cobrança
10h14:31,0  resolução: primeira cobrança aprovada -> webhook "autorizada"
```

Duas cobranças, dois IDs, duas referências no adquirente, duas autorizações. Para a Lastro, eram dois pedidos de pagamento perfeitamente válidos que, por coincidência, tinham o mesmo valor, o mesmo cartão e o mesmo `pedido_id`. Naquela terça foram 37 compradores. O Bazar abriu o chamado "URGENTE!!!! COBRANÇA DUPLICADA!!!!", com o recorde de 16 exclamações, e a consultoria cobrou 40 horas para explicar que o retry dela estava correto.

E estava. O capítulo 15 ensinou a regra: só se repete com segurança o que não teve efeito ou o que é idempotente. O retry do Bazar seguiu a regra pela metade, e a outra metade era responsabilidade da Lastro. Como uma API faz com que "tentar de novo" signifique "a mesma operação", e não "outra operação igual"?

At-most-once, at-least-once e o unicórnio

Toda comunicação por uma rede que perde mensagens, entre processos que podem morrer, escolhe uma **semântica de entrega**:

Semântica	Como se obtém	O que pode dar errado
At-most-once (no máximo uma vez)	envia uma vez, nunca repete	a mensagem se perde
At-least-once (pelo menos uma vez)	repete até receber confirmação	a mensagem chega duplicada
Exactly-once (exatamente uma vez)	?	o vendedor da ideia some com o dinheiro

A escolha entre as duas primeiras é a escolha entre perder e duplicar. Não existe terceira opção na entrega, e a razão é o **problema dos dois generais** (descrito em 1975 por Akkoyunlu, Ekanadham e Huber; o nome veio de Jim Gray, em 1978): dois processos que se comunicam por um canal que pode perder mensagens não conseguem, com nenhum protocolo finito, ter certeza de que ambos sabem o mesmo fato. A última mensagem do protocolo pode se perder, e quem a enviou nunca sabe se ela chegou. Confirmar a confirmação só empurra o problema para a próxima mensagem. É o TCP do capítulo 8 com crise de ansiedade, e a terapia não converge.

O que existe, e é o que os produtos querem dizer quando escrevem "exactly-once" na página de vendas, é **efeito exatamente uma vez**: entrega at-least-once mais um receptor que reconhece duplicatas e as descarta, ou que processa de forma que a repetição não muda nada. A mensagem chega três vezes; o dinheiro sai uma.

> **Na vida real:** o Kafka anuncia exactly-once desde a versão 0.11 (2017, KIP-98), e a garantia é precisa e restrita. O produtor idempotente recebe um ID e numera as mensagens por partição, e o broker descarta números repetidos; as transações tornam atômico o ciclo consumir, processar e produzir dentro do Kafka. Um e-mail enviado, uma chamada ao adquirente ou um `INSERT` num PostgreSQL de fora estão fora da transação e voltam a ser at-least-once. O unicórnio existe, mas não sai do curral.

Idempotência, a palavra que todo mundo usa e quase ninguém conjuga

Uma operação é **idempotente** quando aplicá-la várias vezes produz o mesmo efeito que aplicá-la uma vez: `f(f(x)) = f(x)`. A RFC 9110 (seção 9.2.2) define assim os métodos HTTP: são idempotentes os seguros (`GET`, `HEAD`, `OPTIONS`, `TRACE`) mais `PUT` e `DELETE`. O `POST` fica de fora, por definição, o que faz dele o método favorito de quem cria cobranças e de quem cria problemas.

Dois detalhes que o mercado erra:

Idempotência fala do efeito, e não da resposta. O primeiro `DELETE /v1/webhooks/123` responde `204`; o segundo, `404`. O estado do servidor é o mesmo depois de um ou de dois, então a operação é idempotente, mesmo com respostas diferentes.

Idempotência isolada não protege contra intercalação. `PUT` com `estado = pausado` é idempotente. Mas se o cliente envia A, outro cliente envia B, e o retry atrasado de A chega depois, A sobrescreve B. Cada operação é idempotente; a sequência ficou errada. Para isso existe a escrita condicional com `If-Match` e `ETag` (capítulo 9), que transforma o retry atrasado num `412`.

Algumas operações já nascem idempotentes, e vale desenhar para que nasçam:

Operação	Idempotente?	Por quê
`UPDATE contas SET estado = 'bloqueada'`	sim	valor absoluto
`UPDATE ... SET estado = 'capturada' WHERE estado = 'autorizada'`	sim	a segunda execução afeta 0 linhas
`UPDATE ... SET saldo = saldo + 100`	não	valor relativo; cada execução soma
`INSERT` com chave gerada pelo banco	não	cada execução cria uma linha nova
`INSERT` com chave natural `UNIQUE`	sim, com `ON CONFLICT DO NOTHING`	a segunda colide e não faz nada

A última linha é a ferramenta que resolve metade dos problemas de pagamento, e o PostgreSQL entrega de fábrica:

```
INSERT INTO pix_recebidos (end_to_end_id, valor)
VALUES ('E00038166202611041314abcdef12345', 34990)
ON CONFLICT (end_to_end_id) DO NOTHING;
-- primeira vez:  INSERT 0 1
-- segunda vez:   INSERT 0 0
```

`INSERT 0 0` é o banco dizendo "já tenho, obrigado", com a educação de quem não vai cobrar de novo. A chave natural do evento vira a chave de deduplicação, e a garantia mora numa restrição do banco, que não esquece, não tem pausa de GC e não depende do humor do Carlos.

A chave de idempotência, ou como dar RG para uma intenção

O `POST /v1/cobrancas` não tem chave natural. Dois pedidos de R$ 349,90 no mesmo cartão podem ser duas compras legítimas (o comprador quis dois fones; ninguém julga) ou um retry. Só o cliente sabe, e a Lastro, que não lê mentes nem o código da consultoria, precisa que ele diga.

A **chave de idempotência** é um identificador que o cliente gera **uma vez por operação lógica**, antes da primeira tentativa, e reenvia igual em todos os retries. O servidor guarda a chave junto com o resultado e, ao ver a mesma chave de novo, devolve o resultado guardado em vez de executar de novo. O Stripe popularizou o cabeçalho `Idempotency-Key`, e o IETF tem um Internet-Draft que padroniza o campo (`draft-ietf-httpapi-idempotency-key-header`, ainda em andamento):

```
POST /v1/cobrancas HTTP/1.1
Idempotency-Key: 0193f2a1-7c4e-7b55-9d1e-3f6a8b2c4d10
Content-Type: application/json

{"valor": 34990, "metodo": "cartao", "metadata": {"pedido_id": "88213"}}
```

A primeira requisição cria e responde `201`. A segunda, com a mesma chave e o mesmo corpo, recebe o mesmo `201` com o mesmo ID de cobrança, sem tocar no adquirente. O servidor precisa tratar quatro casos:

1. **Chave nova:** registra a chave como *em andamento*, executa, grava a resposta.
2. **Chave concluída, mesmo corpo:** devolve a resposta gravada, com o mesmo status.
3. **Chave em andamento** (o retry chegou antes do fim da primeira): `409`, porque executar em paralelo é exatamente o que se quer evitar.
4. **Chave conhecida, corpo diferente:** `422`. O cliente reaproveitou a chave para outra operação, e isso é bug dele que, se aceito, vira bug seu.

O draft do IETF usa esses mesmos `409` e `422`. Para o caso 4, o servidor guarda uma impressão digital da requisição (um hash do corpo canonizado e do caminho) e compara.

Três decisões definem se o mecanismo funciona ou só decora:

Escopo. A chave é única por cliente, não globalmente: `(marketplace_id, chave)`. Senão o Bazar e o Hornero colidem por acaso, ou um descobre as chaves do outro de propósito.

Retenção. A chave precisa viver mais que a janela de retry de qualquer cliente. O Stripe guarda por 24 horas. Uma chave que expira em 5 minutos protege contra o retry de 3 segundos e entrega o retry da fila de reprocessamento da madrugada.

Atomicidade. Chave e efeito precisam ser gravados juntos. Se a chave fica num Redis e a cobrança no PostgreSQL, existe um instante em que um foi gravado e o outro não, e um processo morto nesse instante deixa uma chave sem cobrança ou uma cobrança sem chave. A segunda opção cobra duas vezes. A chave mora no mesmo banco e na mesma transação do efeito.

E quando o efeito inclui uma chamada externa, que nenhuma transação de banco abraça? A operação é dividida em **fases atômicas** com pontos de recuperação, como descreveu Brandur Leach num artigo de 2017 sobre chaves no estilo do Stripe em PostgreSQL: cada fase local é uma transação que avança o estado da chave; a chamada externa fica entre duas fases e usa, ela própria, um identificador idempotente no parceiro. Se o processo morre no meio, quem retoma lê a fase gravada e continua dali, em vez de recomeçar. É a máquina de estados que o capítulo 15 já desenhou para o `processando`, agora com uma chave na frente.

Deduplicação do lado de quem recebe

Do outro lado da semântica at-least-once fica o consumidor, recebendo a mesma mensagem duas, cinco, vinte vezes. Ele tem três formas de sobreviver:

- **Processamento naturalmente idempotente:** a operação é desenhada como valor absoluto ou transição condicional (tabela acima). Zero estado extra, nenhuma janela.
- **Chave natural do domínio com `UNIQUE`:** o EndToEndId do Pix, o ID da cobrança, o ID do estorno. A restrição dura para sempre, porque a linha dura para sempre.
- **Tabela de mensagens processadas** (*inbox*): o ID de cada mensagem é gravado na mesma transação do efeito; a repetição colide e é descartada. Quando não há chave de domínio, é o que sobra.

O ponto que separa a deduplicação de verdade da de enfeite é a **janela**. Toda deduplicação por cache tem uma: o Amazon SQS FIFO, por exemplo, deduplica por 5 minutos. Uma duplicata que chegar no minuto 6 é uma mensagem nova, legítima, processada com o mesmo zelo da primeira. A janela precisa cobrir o atraso máximo possível de uma repetição, e "máximo possível" inclui o reprocessamento manual que o Carlos faz na segunda-feira.

E a regra de ouro do consumidor: **duplicata reconhecida responde sucesso**. Se o banco do Pix reenvia uma notificação já processada e a Lastro responde `409`, o banco entende falha e tenta de novo, para sempre ou até desistir e marcar a Lastro como instável. A resposta correta é `200`: "recebido, e já estava".

Na Lastro: cada efeito com um RG

O incidente teve uma causa só, e a Lastro tem várias portas de entrada com o mesmo defeito. Cada operação que produz efeito ganha um mecanismo de deduplicação ancorado no PostgreSQL:

Operação	Duplicata vem de	Mecanismo
`POST /v1/cobrancas`, `POST .../estornos`	retry do marketplace	`Idempotency-Key`, escopo marketplace, 24 h
`POST .../captura`	retry do marketplace	transição condicional `autorizada → capturada`
Autorização no adquirente	timeout de conexão, `GOAWAY`	referência única = ID da cobrança
Notificação de Pix recebido	retry do banco parceiro	EndToEndId `UNIQUE` (desde o capítulo 1)
Repasse por Pix	retry da Lastro ao banco	identificador do repasse enviado ao banco
Transação do ledger	qualquer uma acima	`(origem, referencia)` `UNIQUE` em `transacoes`
Webhook para o marketplace	retry da Lastro (capítulo 17)	`id` do evento, deduplicado pelo marketplace (capítulo 11)

A chave na API. Em `/v1`, o `Idempotency-Key` é opcional, porque torná-lo obrigatório quebraria todo cliente existente (regra do capítulo 11), e passa a ser obrigatório em `/v2`. O guia de integração diz o que a consultoria precisava ler: gere a chave (um UUID) quando o pedido decide cobrar, guarde junto do pedido, reenvie a mesma em todo retry. A tabela de chaves tem, conceitualmente, `marketplace_id`, `chave`, `hash_requisicao`, `fase`, `cobranca_id`, `status_http`, `corpo_resposta` e `expira_em`, com chave primária `(marketplace_id, chave)`. A retenção de 24 horas cobre o deadline de 8 s, a resolução do `processando` e as filas de reprocessamento dos clientes. Mesmo um dia inteiro no pico projetado de 200 cobranças por segundo daria 17 milhões de linhas, e um job apaga as expiradas em lotes.

O fluxo da cobrança de cartão com chave:

```
tx1: INSERT chave (fase = criada) + INSERT cobrança      ── COMMIT
     ── retry agora recebe 409 "requisicao-em-andamento" + Retry-After: 2
Vigia, adquirente (referência = ID da cobrança)
tx2: UPDATE cobrança + chave (fase = concluida, resposta) ── COMMIT
     ── retry agora recebe a resposta gravada, mesmo status, mesmo ID
```

```
processo morre entre tx1 e tx2?
     ── resolução do capítulo 15 consulta por referência, conclui a chave
```

Se o corpo do retry diferir do original, `422` com o `type` RFC 9457 `chave-idempotencia-reutilizada`. Se a cobrança terminou em `processando`, o retry recebe o mesmo `processando` com o mesmo ID, e não uma segunda cobrança que, por ser mais rápida, sairia `autorizada`. No incidente da terça, o retry das 10h14:13 teria devolvido a cobrança da primeira tentativa, e o fone teria sido cobrado uma vez.

O adquirente. A referência única que o capítulo 15 trouxe também é a chave de idempotência da Lastro com o adquirente: pelo contrato de integração, uma segunda autorização com referência já usada é recusada com o código de referência duplicada, sem nova reserva no cartão. Os retries de conexão e de `GOAWAY` ficam seguros mesmo se a premissa "nada foi enviado" falhar. O timeout de leitura segue sem retry, porque o deadline já acabou; quem resolve é a consulta.

O ledger. A tabela `transacoes` ganha `origem` e `referencia`, com índice único sobre o par, criado com `CONCURRENTLY` e as regras de migração do capítulo 7. Todo evento que gera lançamentos se identifica: `('pix_recebido', EndToEndId)`, `('captura', cobranca_id)`, `('estorno', estorno_id)`, `('repasse', repasse_id)`. O tabelião agora registra também o RG de quem pediu cada partida dobrada. Se dois caminhos tentarem lançar a mesma captura (um retry, um job, o `psql` do Carlos), o segundo encontra a restrição e vira um `INSERT 0 0` de uma transação inteira.

O Pix. A notificação do banco parceiro já era deduplicada pelo EndToEndId `UNIQUE` desde o capítulo 1, a única defesa da Lastro que funcionou na terça, escrita quinze capítulos atrás por quem nem sabia que estava se defendendo. A mudança é na resposta: duplicata reconhecida devolve `200`. No repasse, a Lastro envia ao banco o ID do repasse como identificador da requisição, e o banco recusa um segundo pagamento com o mesmo identificador: o retry após timeout fica seguro, e a "falha pós-débito" pendente desde o capítulo 2 ganha metade da resposta (a outra metade é do capítulo 18).

A alternativa descartada foi deduplicar por conteúdo (mesmo marketplace, `pedido_id`, valor e cartão em 10 minutos). Dispensa mudança no cliente e erra nos dois sentidos: bloqueia a segunda compra legítima e deixa passar o retry com `pedido_id` regerado. Serve só como alerta para cliente que não pode mudar a integração, nunca como regra que recusa pagamento.

Armadilhas e casos reais

> **Armadilha:** a chave gerada dentro do laço de retry. A consultoria do Bazar, na primeira versão da correção, colocou `UUID.randomUUID()` dentro do `for` das tentativas. Cada retry tinha uma chave nova, a Lastro via três operações diferentes, e a idempotência funcionou perfeitamente para garantir que cada cobrança duplicada fosse única. Mais 40 horas.
>
> **Armadilha:** devolver o estado atual em vez da resposta original. Se o replay de uma chave concluída lê a cobrança de novo, o retry de uma criação pode receber `estado: estornada`, e o cliente conclui que a cobrança nasceu estornada. O replay devolve o que a primeira resposta disse.
>
> **Armadilha:** a chave e o efeito em bancos diferentes. O Carlos propôs guardar as chaves no Redis "porque é mais rápido". É, e o processo que morre entre o `SET` no Redis e o `COMMIT` no PostgreSQL deixa uma chave que diz "concluída" para uma cobrança que não existe. A velocidade de errar também melhorou.
>
> **Na vida real:** o Amazon SQS oferece filas padrão com entrega at-least-once e filas FIFO com deduplicação por 5 minutos, pelo ID de deduplicação ou pelo hash do corpo. A documentação é explícita sobre a janela; o que costuma falhar é o produtor que repete depois dela.

Uma última: "idempotente" usado como sinônimo de "pode repetir à vontade". O `PUT` que sobrescreve com o valor antigo depois de outro cliente ter mudado é idempotente e está errado. Idempotência resolve repetição; ordem é outro problema, com outra ferramenta (o `If-Match`), e o capítulo 20 volta a ela.

Resumo

- Entrega at-most-once perde, at-least-once duplica, e exactly-once na entrega é impossível (dois generais); existe efeito exatamente uma vez = at-least-once + deduplicação.
- Idempotência: `f(f(x)) = f(x)`, sobre o efeito, e não sobre a resposta; não protege contra intercalação.
- Operações naturalmente idempotentes: valor absoluto, transição condicional, `INSERT` com chave natural e `ON CONFLICT DO NOTHING`.
- Chave de idempotência: gerada por operação, escopo por cliente, retenção maior que a janela de retry, gravada na mesma transação do efeito; `409` em andamento, `422` com corpo diferente.

- Chamada externa no meio: fases atômicas com recuperação e identificador idempotente no parceiro.
- Consumidor deduplica por chave natural ou inbox, com janela suficiente, e responde sucesso à duplicata.

A Lastro agora

A API aceita `Idempotency-Key` (opcional em `/v1`, obrigatória em `/v2`), com escopo por marketplace, retenção de 24 h, `409` em andamento e `422` para chave reutilizada. A captura é transição condicional, a referência no adquirente e o ID do repasse no banco viraram chaves de idempotência com os parceiros, e `transacoes` ganhou `(origem, referencia)` único: o ledger não aceita o mesmo fato duas vezes, venha de onde vier. O financeiro estornou as 37 cobranças duplicadas, e a consultoria, na segunda tentativa, gerou a chave fora do laço.

Toda entrada da Lastro agora resiste à repetição. A saída, nem tanto: os webhooks continuam saindo de dentro da requisição, uma vez, sem retry, depois do COMMIT. A Lastro garante efeito exatamente uma vez em tudo o que recebe e entrega at-most-once em tudo o que manda. Na quinta, o servidor de webhooks do Hornero começou a responder em 25 segundos, e a API da Lastro descobriu que esperar educadamente por um cliente lento é a forma mais idempotente de cair: cai igual, quantas vezes for.

Referências

- RFC 9110, HTTP Semantics, seção 9.2.2 (Idempotent Methods): https://www.rfc-editor.org/rfc/rfc9110#section-9.2.2
- IETF, "The Idempotency-Key HTTP Header Field" (Internet-Draft): https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/
- Stripe, "Idempotent requests": https://docs.stripe.com/api/idempotent_requests
- Brandur Leach, "Implementing Stripe-like Idempotency Keys in Postgres", 2017: https://brandur.org/idempotency-keys
- E. A. Akkoyunlu, K. Ekanadham, R. V. Huber, "Some Constraints and Tradeoffs in the Design of Network Communications", SOSP 1975.
- Jim Gray, "Notes on Data Base Operating Systems", 1978.
- KIP-98, "Exactly Once Delivery and Transactional Messaging": https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-+Exactly+Once+Delivery+and+Transactional+Messaging

- Amazon SQS, "Exactly-once processing in Amazon SQS" (FIFO): https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues-exactly-once-processing.html
- PostgreSQL 16, "INSERT … ON CONFLICT": https://www.postgresql.org/docs/16/sql-insert.html

CAPÍTULO 17

Mensageria

Quinta, 14h02: o Hornero ficou lento e a Lastro foi junto, por solidariedade

Na quinta às 14h02, o servidor de webhooks do Mercadito Hornero começou a responder em 25 segundos, e a Lastro, educadíssima, esperou cada um deles até o fim, de pé, segurando a porta.

O webhook sai depois do COMMIT (capítulo 4), mas **de dentro da requisição** que mudou o estado: o worker grava, commita, chama o marketplace e só então responde a quem pediu. O cliente HTTP dos webhooks é um por processo, limitado a 5 conexões "para não sobrecarregar os marketplaces", com um timeout de 30 segundos "para dar tempo ao cliente". As duas gentilezas têm o mesmo autor, nome, sobrenome e um comentário `não mexa` na linha de cima.

Às 14h, o Hornero gera uns 2 webhooks por segundo. A Lei de Little (capítulo 14) faz a conta sem pedir licença: 2 por segundo × 25 segundos = 50 conexões ocupadas ao mesmo tempo. Os 9 processos da API somam 45.

```
14h02  Hornero: deploy deles; webhooks passam a responder em 25 s
14h04  pool httpx dos 9 processos: 5/5 conexões ocupadas
14h05  webhooks do Bazar entram na fila do mesmo pool, atrás do Hornero
14h06  POST /v1/cobrancas (cartão), p99: 31 s
14h07  banco parceiro desiste de 212 notificações Pix após 3 tentativas
14h11  Bazar: "URGENTE!!!! A LASTRO CAIU?????"
14h12  analista do Hornero termina a manga do casaco de crochê
```

Você aí, que já chamou um serviço externo no meio de um handler "porque é rapidinho": a Lastro passou a ter o SLA do pior estagiário de cada marketplace.

O Carlos resolveu em nove minutos: trocou o `await enviar_webhook(...)` por `asyncio.create_task(enviar_webhook(...))`, a resposta parou de esperar e a API voltou às 14h31. Aplausos no canal. Só que as tasks continuaram na fila do mesmo pool, agora em silêncio, morrendo de timeout sem log e sem retry. Às 16h, o *reload* do deploy (capítulo 7) levou junto 140 tasks pendentes. E a documentação do `asyncio` avisa que o event loop guarda só uma referência fraca para cada task: sem referência forte no seu

código, o garbage collector pode recolher a task no meio da execução. O Carlos inventou o webhook que pode ser apagado pela coleta de lixo. A semântica era at-most-once (capítulo 16); virou at-most-once com boato.

A pergunta do capítulo: como a Lastro entrega uma mensagem a quem pode demorar, sumir ou morrer, sem esperar por ele e sem perder a mensagem?

O broker, ou o primeiro componente da Lastro cuja função é segurar coisa dos outros

Um **broker** de mensagens recebe mensagens de **produtores**, guarda de forma durável (disco e réplicas) e entrega a **consumidores**. O produtor depende só do broker; o consumidor pode estar lento, fora do ar ou nem ter sido escrito. A mensagem espera.

Isso desacopla o **tempo** (produzir agora, consumir depois), a **taxa** (produzir 500 por segundo, consumir 80, e o broker absorve a diferença) e a **falha** (o consumidor cai e o produtor nem fica sabendo). O preço vem parcelado, igual fatura da consultoria Java do Bazar: um sistema a mais para operar, entrega "depois" (milissegundos num dia bom, horas num dia ruim) e at-least-once como padrão.

Existem dois modelos de broker, com garantias bem diferentes, e o mercado chama os dois de "fila", o que ajuda tanto quanto chamar PostgreSQL e planilha de "banco".

A fila: entregar, esperar o ack, apagar

Numa **fila**, o broker entrega cada mensagem a **um** consumidor e espera a confirmação, o **ack** (*acknowledgment*). Com ack, apaga. Sem ack, entrega de novo, talvez a outro. Vários consumidores na mesma fila são **consumidores concorrentes** (*competing consumers*): o broker divide as mensagens e a vazão cresce com eles.

O "sem ack" vem de dois jeitos. No RabbitMQ (AMQP 0-9-1), a mensagem sem ack fica presa ao canal do consumidor; se a conexão cai, volta à fila marcada `redelivered`, e o *prefetch* (`basic.qos`) limita quantas cada consumidor segura. Numa fila gerenciada como o Amazon SQS, receber esconde a mensagem por um **visibility timeout** (padrão 30 s, máximo 12 h); apagar é o ack, e se o prazo vence ela reaparece para qualquer um.

O visibility timeout é um lease (capítulo 2) com as mesmas doenças: processamento mais longo que o prazo e são dois consumidores fazendo a mesma coisa. Estende-se o prazo durante o trabalho (`ChangeMessageVisibility`) ou ele fica acima da cauda, com o critério do capítulo 15.

Nos dois mecanismos, o consumidor pode morrer entre terminar o trabalho e mandar o ack, e a mensagem é processada outra vez. Fila com ack é at-least-once por construção; o efeito exatamente uma vez sai da deduplicação no consumidor (capítulo 16). Quem manda o ack antes de processar ganha at-most-once e passa a perder mensagem com a consciência tranquila.

E a ordem? Com consumidores concorrentes, a mensagem 1 vai para o consumidor A, a 2 para o B, o B é mais rápido e a 2 termina primeiro. Uma redelivery manda a 1 para o fim da festa. As saídas têm o mesmo preço: o SQS FIFO não libera a próxima mensagem de um `MessageGroupId` enquanto a anterior não for apagada; o RabbitMQ tem *single active consumer*. Ordem custa paralelismo, sempre.

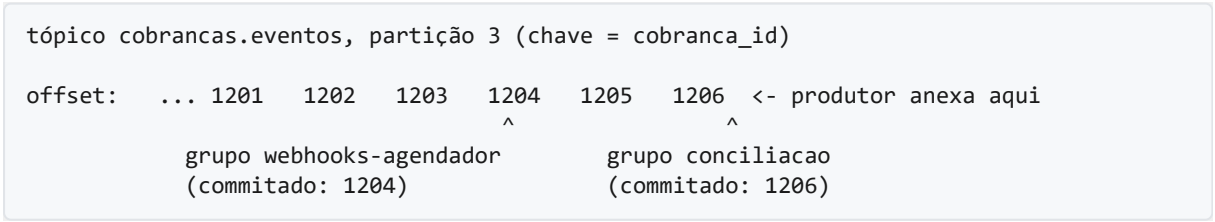
O log: gravar no fim, cada um lê de onde parou

Um **log** (o Apache Kafka é o exemplo de referência) é um arquivo append-only replicado. O produtor anexa registros no fim, cada um com um **offset**, sua posição. Ler **não apaga nada**: o registro fica pela **retenção** (no Kafka, 168 horas por padrão), lido por ninguém ou por quarenta consumidores.

O consumidor guarda o próprio progresso, **commitando** o offset até onde processou (no Kafka, no tópico interno `__consumer_offsets`), e volta dali se morrer. Commitar depois de processar dá at-least-once; antes, at-most-once. O log trocou o nome do ack e manteve o dilema, como todo rebranding.

Um tópico se divide em **partições**, logs independentes com ordem total dentro e nenhuma entre elas. A **chave** escolhe a partição: no cliente Java do Kafka, `murmur2(chave) mod número_de_partições`. Mesma chave, mesma partição, mesma ordem.

No **consumer group**, os consumidores repartem as partições entre si, e cada partição é lida por exatamente um membro do grupo. Grupos diferentes são independentes: cada um tem os próprios offsets e lê o tópico inteiro no próprio ritmo, e um grupo novo pode começar do offset zero e reprocessar a semana inteira. Isso é o **replay**, impossível numa fila, que já apagou tudo quando você teve a ideia.



As consequências da partição são todas mecânicas:

- O paralelismo de um grupo tem teto no número de partições. Doze partições e vinte consumidores: oito recebendo salário para não fazer nada.

- Uma mensagem lenta trava a partição inteira para aquele grupo, porque o offset só avança em ordem. Head-of-line blocking (capítulo 9), agora com retenção de sete dias.
- Aumentar o número de partições muda o resultado do `mod`, e a ordem por chave quebra na transição. O número se escolhe com folga no primeiro dia, igual nome de filho.
- Consumidor que entra ou sai dispara um *rebalance*: as partições afetadas ficam sem leitor por um tempo, e o que foi processado sem commit é processado de novo pelo novo dono.

Do lado do produtor, durabilidade é configuração. No Kafka, `acks=all` com `min.insync.replicas=2` e fator de replicação 3 só confirma depois que duas réplicas gravaram; com `acks=1`, o líder confirma sozinho e, se morrer antes de replicar, a mensagem confirmada some. O produtor idempotente (KIP-98, capítulo 16, padrão desde o Kafka 3.0) evita que os retries do produtor dupliquem registros numa partição.

Propriedade	Fila (RabbitMQ, SQS)	Log (Kafka)
Depois de consumida	Apagada no ack	Fica até a retenção
Consumidores independentes	Uma fila por consumidor	Um consumer group por consumidor
Replay	Não	Sim, dentro da retenção
Ordem	Perdida com concorrência; FIFO por grupo	Por partição
Paralelismo	Cresce com consumidores	Teto nas partições
Retry e atraso por mensagem	Nativos	Inexistentes: trava a partição ou vai para outro tópico

A mensagem que não quer ser processada

Uma **mensagem venenosa** (*poison message*) falha sempre: JSON truncado, schema desconhecido, a cobrança de um marketplace apagado pelo Carlos num dia que ninguém comenta. Numa fila, ela volta e falha para sempre. Num log é pior: o offset não avança e a partição inteira para atrás dela.

A saída é a **DLQ** (*dead-letter queue*): depois de N tentativas, a mensagem vai para uma fila separada e o fluxo segue. No SQS, a *redrive policy* move a mensagem quando o `ApproximateReceiveCount` passa do `maxReceiveCount`; no RabbitMQ, o `x-dead-letter-exchange` recebe o que foi rejeitado sem requeue ou expirou. No Kafka não há DLQ nativa: o consumidor publica num tópico de DLQ e commita o offset.

A DLQ é um cemitério onde cada lápide é um efeito que não aconteceu. Precisa de alarme no primeiro registro, de dono e de *redrive* depois da correção. DLQ sem alarme é aquele bucket de logs que ninguém abre, só que cheio de dinheiro.

Backlog: a dívida que a fila guarda para você

Backlog é o que entrou e ainda não foi processado. Na fila, é a profundidade; no log, é o **lag** de cada grupo, a diferença entre o último offset do log e o offset commitado, por partição (saída com as colunas de consumidor omitidas):

```
$ kafka-consumer-groups.sh --describe --group webhooks-agendador ...
TOPIC              PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
cobrancas.eventos  6          1830114         1830121         7
cobrancas.eventos  7          1702388         1750599         48211
cobrancas.eventos  8          1811920         1811925         5
```

Partição 7 com 48 mil de atraso e as vizinhas em dia: uma mensagem venenosa, num ensaio da Lastro antes de a DLQ existir. Lag somado esconde isso; o alarme olha por partição. E lag em mensagens mente sobre urgência (48 mil podem ser dez segundos ou um dia); a métrica é a **idade da mensagem mais antiga**. O tempo para zerar o backlog sai da diferença entre capacidade e chegada:

```
tempo para drenar = backlog / (vazão do consumidor − taxa de chegada)
```

Consumindo 100 por segundo e chegando 90, um backlog de 36 mil leva uma hora. Chegando 100, para sempre. Chegando 101, o backlog cresce até a retenção apagar as mais velhas, e no log isso é perder dados sem nenhum erro, só um offset que pulou. Broker não cria capacidade, compra tempo, a juros.

Na Lastro: um log para os fatos, uma fila para as entregas

Os requisitos, em números do projeto:

1. **Isolamento.** Endpoint lento ou fora do ar não afeta a API nem as entregas de outro marketplace.
2. **Retentativas longas.** Quem não responde `2xx` em 10 segundos (contrato do capítulo 11) recebe novas tentativas por dias, e o painel mostra o que morreu e permite reenviar.
3. **Ordem por cobrança.** O contrato segue dizendo que o `versao` desempata, mas a Lastro passa a entregar a cada endpoint os eventos de uma cobrança em ordem de `versao`, para a consultoria Java do Bazar parar de ver `estornada` antes de `paga` e de cobrar 40 horas para descobrir isso.

4. **Consumidores com replay.** A conciliação quer reler os eventos do dia e o Vigia pediu o fluxo de eventos para treinar regras, cada um no próprio ritmo.
5. **Volume.** Pico da Black Friday: 200 cobranças por segundo (capítulo 14), ~2,5 eventos por cobrança, 500 eventos por segundo; 1.000 no 2×.

As alternativas:

	Só fila (RabbitMQ, ou fila gerenciada)	Só log (Kafka)	Tudo no PostgreSQL (`SKIP LOCKED`)	Log + fila de entregas
Isolamento por endpoint	Fila por endpoint	Endpoint lento trava a partição	Limite na consulta	Limite na consulta
Retry com atraso de horas	TTL + DLX, visibility	Tópicos de retry; ordem quebra	Coluna de próxima tentativa	Coluna de próxima tentativa
Replay	Não	Sim	Offsets reinventados	Sim
"Entregas mortas do marketplace X"	Sem consulta	Sem consulta	Índice	Índice
Peça nova	Broker	Broker	Nenhuma; carga no banco	Broker + banco pequeno

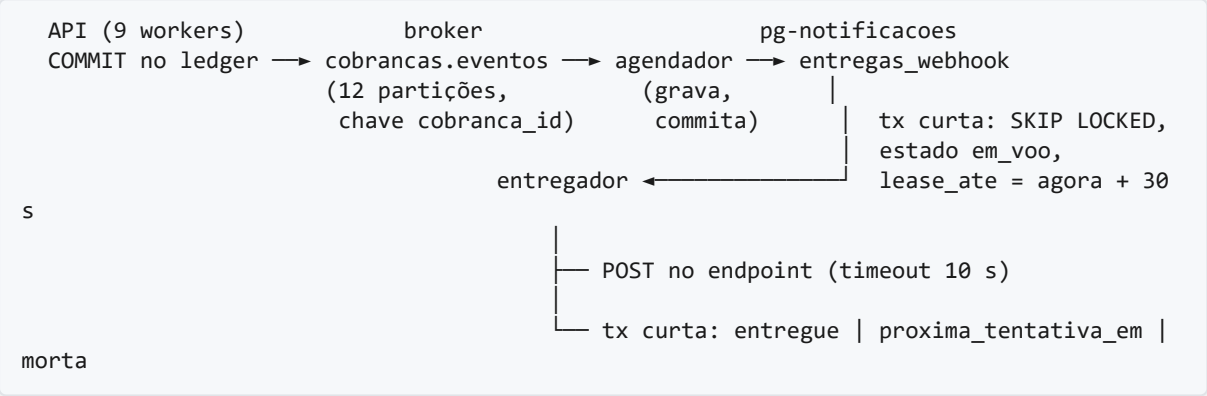
A Lastro fica com a última coluna, em dois estágios.

Os fatos vão para um log. O broker é um serviço gerenciado compatível com o protocolo do Kafka (Amazon MSK e Confluent Cloud são exemplos), escolhido pelo requisito 4: replay e consumidores independentes só o log tem. Gerenciado porque a equipe não opera Kafka, e um cluster de Kafka operado pelo Carlos é um incidente com cronograma.

O tópico `cobrancas.eventos` tem 12 partições, fator de replicação 3, `min.insync.replicas=2` e retenção de 7 dias. A chave `cobranca_id` dá ordem por cobrança (requisito 3) e espalha a carga, porque nenhuma cobrança sozinha é quente. Doze partições sobram para 1.000 eventos por segundo; o número existe para nunca precisar mudar. O produtor usa `acks=all` e o modo idempotente; o registro é o envelope do capítulo 11.

As entregas vão para uma fila no banco. O grupo `webhooks-agendador` lê o tópico e grava uma linha por inscrição na tabela `entregas_webhook`, num PostgreSQL novo e pequeno, o `pg-notificacoes`. Separado do ledger pelo orçamento de conexões do capítulo 14 (o primário já usa 18 das ~17 a 20 que aguenta) e para que o churn das entregas não divida VACUUM com os lançamentos. A unicidade de `(evento_id, inscricao_id)` com `ON CONFLICT DO NOTHING` torna o agendador idempotente: reprocessamento gera `INSERT 0 0` e mais nada. O agendador nunca faz HTTP; grava e commita o offset em milissegundos, e nenhuma partição espera marketplace.

O `webhooks-entregador` roda em `api-01` e `api-02`, fora dos workers da API, assíncrono. Cada entrega reproduz o visibility timeout dentro do banco, sem segurar conexão durante `await` externo (capítulo 13):



- A seleção pega, com `FOR UPDATE SKIP LOCKED` (capítulo 4), entregas vencidas cuja anterior da mesma inscrição e cobrança já terminou: ordem por `versao` sem travar mais nada.
- Se o entregador morre com a entrega `em_voo`, o lease vence e outro pega. O marketplace pode receber o evento duas vezes, e o contrato já manda deduplicar pelo `id`.
- Cada endpoint tem bulkhead de 10 entregas simultâneas por processo e circuit breaker com os parâmetros do capítulo 15. O Hornero lento ocupa no máximo 20 conexões, e o Bazar nem percebe.
- Falhou (timeout, `5xx`, `429`, erro de conexão)? Novas tentativas em 1 min, 5 min, 30 min, 2 h, 6 h, 12 h, 24 h e 24 h, com jitter de ±20%: nove tentativas em cerca de 69 horas, a ordem de grandeza que o Stripe documenta (até três dias). Depois, estado `morta`: a DLQ da Lastro, consultável no painel, com botão de reenvio e alarme na primeira linha.
- Evento que não desserializa vai para `cobrancas.eventos.dlq` e o agendador segue. A partição 7 do ensaio mandou um abraço.

A quinta, refeita: o Hornero gera 2 eventos por segundo e recebe 20 / 25 s = 0,8, acumulando backlog só dele, umas 4.300 entregas por hora. Quando volta a responder em 200 ms, as 20 conexões entregam 100 por segundo, e 4.300 a 100 − 2 = 98 por segundo drenam em 44 segundos. A API, enquanto isso, nem soube.

```
Entregador: Hornero, evt_0192f3..., cobrança paga.
Hornero:    (25 segundos de silêncio)
Entregador: Tudo bem. Volto em 1 minuto. Depois em 5. Depois em 30.
Hornero:    (silêncio)
Entregador: Volto amanhã. E depois de amanhã. Domingo você vira lápide.
Bazar:      O meu chegou em 80 ms. Posso abrir chamado mesmo assim?
```

Sem consumidores além dos webhooks, uma fila sozinha bastaria (o par SNS e SQS, numa só nuvem, é um exemplo); com poucas dezenas de eventos por segundo e nenhum consumidor novo, tudo no PostgreSQL.

Armadilhas e casos reais

> **Armadilha:** publicar no broker depois do COMMIT, que é o que a API da Lastro faz hoje. Processo morto entre as duas linhas: pagamento sem evento. Publicar antes: evento sem pagamento. Dois sistemas sem transação entre eles, buraco com capítulo próprio, o 18.
>
> **Armadilha:** confundir "o broker confirmou" com "a mensagem está segura". Com `acks=1`, o líder pode morrer antes de replicar; numa fila não durável do RabbitMQ, a mensagem confirmada mora na memória de um processo, o endereço mais instável da computação depois do celular do fundador.
>
> **Armadilha:** HTTP dentro do consumidor de um log. O endpoint lento de um marketplace trava os eventos de todo mundo que caiu na mesma partição. É a tarde de quinta, com retenção de sete dias.
>
> **Na vida real:** em 25 de novembro de 2020, o Amazon Kinesis Data Streams (um log gerenciado) falhou por horas em us-east-1. Segundo o relatório da AWS, ao ganhar capacidade, cada servidor da frota de front-end passou do limite de threads do sistema operacional, porque mantinha uma thread para cada outro servidor. Cognito e CloudWatch foram junto. O broker vira dependência de tudo que publica nele.

Resumo

- Broker desacopla tempo, taxa e falha, ao custo de uma peça a mais, entrega posterior e at-least-once.
- Fila: apaga no ack; sem ack (conexão caída, visibility timeout vencido), entrega de novo. Consumidores concorrentes escalam e perdem a ordem.
- Log: append-only, offsets, retenção independente de consumo; consumer groups independentes; replay.
- Ordem só por partição ou grupo de mensagens; mudar partições quebra a ordem por chave.
- Ack ou commit depois de processar: at-least-once; antes: at-most-once.
- DLQ para mensagem venenosa, com alarme, dono e redrive; sem alarme, é cemitério com mensalidade.
- Backlog: lag por partição e idade da mais antiga; drena a `backlog / (vazão − chegada)`.

A Lastro agora

Eventos em `cobrancas.eventos` (log compatível com Kafka, 12 partições, chave `cobranca_id`); entregas no `pg-notificacoes`, com lease, bulkhead por endpoint, nove tentativas em 69 horas, ordem por cobrança e DLQ no painel. A API nunca mais espera marketplace. O `create_task` do Carlos foi removido, e o garbage collector perdeu o emprego de entregador de webhook.

O Hornero ficou lento de novo na terça seguinte, e só o Hornero percebeu. A analista dele, pela primeira vez, ficou sem tempo para o crochê. Na mesma terça, o deploy das 16h reiniciou um worker exatamente entre o COMMIT de um Pix de R$ 870,00 e a publicação do evento: o ledger tem o pagamento, o tópico não tem nada, o Bazar nunca soube, e o pedido foi cancelado em 30 minutos. A Lastro aprendeu a entregar uma mensagem a qualquer um, desde que ela tenha sido enviada.

Referências

- Apache Kafka, documentação: design (log, partições, consumer groups, semântica de entrega) e configurações de produtor, consumidor e tópico. https://kafka.apache.org/documentation/
- RabbitMQ, Consumer Acknowledgements and Publisher Confirms. https://www.rabbitmq.com/docs/confirms

- RabbitMQ, Dead Letter Exchanges. https://www.rabbitmq.com/docs/dlx
- Amazon SQS Developer Guide, visibility timeout e dead-letter queues. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
- Python, documentação do `asyncio.create_task` (referência fraca às tasks). https://docs.python.org/3/library/asyncio-task.html#asyncio.create_task
- Stripe, documentação de webhooks (retentativas por até três dias). https://docs.stripe.com/webhooks
- Jay Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction", 2013. https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying
- AWS, Summary of the Amazon Kinesis Event in the Northern Virginia (US-EAST-1) Region, novembro de 2020. https://aws.amazon.com/message/11201/

CAPÍTULO 18

Transações distribuídas

Terça, 16h00min03s: o Pix existe, o evento não

Na terça às 16h00min03s, o deploy reiniciou um worker no intervalo mais caro da computação: entre o COMMIT e a linha seguinte.

O COMMIT gravou um Pix de R$ 870,00 no ledger, com split, tarifa e tudo. A linha seguinte publicaria o evento `cobranca.paga` no tópico do capítulo 17. Ela nunca rodou. O `NOTIFY` do capítulo 12 sai junto com o COMMIT, então o checkout mostrou "Pagamento confirmado!" para o comprador, com confete. O backend do Bazar, que só sabe do pagamento pelo webhook, não recebeu nada e, 30 minutos depois, cancelou o pedido por falta de pagamento. O comprador tem o comprovante, a tela de confete e um e-mail de "pedido cancelado". O chamado do Bazar: "URGENTE!!!! O CLIENTE VIU PAGO E A GENTE VIU NADA".

O Carlos propôs inverter: publicar antes do COMMIT. No ambiente de homologação, a transação de um pagamento falhou depois da publicação (o trigger de soma zero do capítulo 1 pegou um split com um centavo sobrando), o ROLLBACK apagou o pagamento, e o evento já tinha saído. O Bazar de homologação despachou 14 fones de ouvido de graça. Num ambiente de mentira, felizmente; o Carlos agora tem um prejuízo imaginário de R$ 4.898,60 no currículo.

E tem o terceiro caso, que já estava lá desde o capítulo 2. O saque debita o ledger, faz COMMIT e depois chama o banco parceiro. Na mesma terça, outro worker morreu no meio desse "depois": o vendedor 42 teve R$ 300,00 debitados e nenhum Pix enviado. Pela primeira vez na história deste livro, o vendedor 42 tem razão, e ele está aproveitando cada segundo.

A pergunta do capítulo: como dois sistemas que não compartilham uma transação concordam sobre o que aconteceu?

A escrita dupla, ou duas linhas de código que não podem ser atômicas

O problema tem nome: **escrita dupla** (*dual write*). Uma operação precisa mudar o estado de dois sistemas (banco e broker, banco e banco parceiro, banco e adquirente), e não existe uma transação que cubra os dois. Qualquer ordem que você escolha tem uma janela em que o processo pode morrer:

Ordem	Morre entre as duas escritas	Resultado
Banco, depois broker	Depois do COMMIT	Pagamento sem evento (a terça)
Broker, depois banco	Antes do COMMIT, ou o COMMIT falha	Evento sem pagamento (os fones)
Banco, depois parceiro	Depois do COMMIT	Débito sem Pix (o vendedor 42)
Parceiro, depois banco	Depois da resposta do parceiro	Pix enviado sem débito, e o 42 descobre

O retry não fecha a janela, porque o processo que ia repetir é o que morreu. O try/except também não, porque `kill -9` não executa `finally`; ele não dá direito a últimas palavras. A janela é pequena, alguns milissegundos por operação, e por isso é traiçoeira: com 25 mil cobranças por dia e um deploy diário que recicla 9 workers, ela acontece algumas vezes por mês, sempre em produção, sempre com o Bazar.

Existem duas famílias de solução. Uma tenta tornar as duas escritas atômicas de verdade: o two-phase commit. A outra desiste da atomicidade e garante que o sistema **converge**: sagas, outbox e, no fim de tudo, a conciliação.

Two-phase commit, o protocolo que faz todo mundo esperar de mão levantada

O **two-phase commit** (2PC) coloca um **coordenador** acima dos **participantes**:

```
Fase 1 (prepare)   coordenador -> participantes: "PREPARE T"
                   cada participante grava tudo de forma durável,
                   segura os locks e responde SIM ou NÃO
Fase 2 (decisão)   todos SIM  -> coordenador grava COMMIT no log dele, avisa todos
                   algum NÃO  -> coordenador grava ABORT, avisa todos
```

O ponto do protocolo é o voto: quem votou SIM perdeu o direito de desistir. Está pronto para commitar e proibido de abortar por conta própria até ouvir a decisão. Se o coordenador morre depois dos votos e antes de mandar a decisão, os participantes ficam **em dúvida** (*in doubt*): não sabem se os outros commitaram, então não podem commitar nem abortar, e seguram os locks até o coordenador voltar. É isso que significa dizer que o 2PC é **bloqueante**. A disponibilidade da operação vira o produto da disponibilidade de todos os participantes e do coordenador, e um coordenador que demora uma hora para voltar segura linhas por uma hora. Variantes como o three-phase commit evitam o bloqueio só com suposições sobre a rede (atraso limitado) que redes reais não garantem.

O PostgreSQL implementa o papel de participante: `PREPARE TRANSACTION 'id'` faz a fase 1, e `COMMIT PREPARED 'id'` ou `ROLLBACK PREPARED 'id'` faz a fase 2. Vem desligado: `max_prepared_transactions` vale 0 por padrão. Uma transação preparada sobrevive a reinício, segura os locks e impede o VACUUM de limpar qualquer tupla mais nova que ela. Uma preparada esquecida em `pg_prepared_xacts` é uma transação longa (capítulo 3) que não morre nem com reboot, um morto-vivo com atestado de imortalidade. O padrão que padroniza o protocolo entre gerenciadores de transação é o XA.

Para a Lastro, o 2PC tem um defeito anterior a todos esses: os participantes precisam implementar o protocolo. O PostgreSQL implementa. O broker de log da Lastro, não; as transações do Kafka são atômicas entre partições do próprio Kafka e não participam de XA com um banco externo. O adquirente e o banco do Pix são APIs HTTP. Ninguém vai mandar `PREPARE` para o Banco Central e esperar ele votar.

```
Coordenador: PREPARE?
PostgreSQL:  SIM. Gravei tudo, travei as linhas, aguardo.
Outro banco: SIM.
Coordenador: Então eu dec... (kill -9)
PostgreSQL:  (seis horas depois, segurando as mesmas linhas) Alguém?
Carlos:      Posso dar um ROLLBACK PREPARED?
PostgreSQL:  Pode. Se o outro commitou, você inventou a escrita dupla com mais passos.
```

> **Na vida real:** o 2PC funciona bem dentro de um banco distribuído. O Spanner, do Google, faz 2PC entre grupos de réplicas em que cada participante é replicado por Paxos, então um participante "em dúvida" raramente fica indisponível.

Sagas, ou desfazer em vez de não fazer

Uma **saga** (Garcia-Molina e Salem, 1987) divide a operação longa numa sequência de **transações locais** T1, T2, ..., Tn, cada uma commitada no seu sistema. Para cada Ti, exceto a última, existe uma **compensação** Ci, que desfaz semanticamente o efeito de Ti. Se Tk falha de vez, a saga executa Ck−1, ..., C1, na ordem inversa.

A compensação desfaz pelo significado: ninguém apaga o lançamento do débito; um lançamento novo devolve o valor. O ledger da Lastro faz isso desde o capítulo 1 (estorno é uma transação nova invertida) e, portanto, a Lastro escrevia compensações antes de saber o nome. O tabelião aprova: nada se apaga, tudo se registra de novo, ao contrário.

Três tipos de passo organizam a saga:

- **Compensáveis**: têm Ci. Ficam antes do pivô.
- **Pivô**: o ponto sem volta. Depois dele, a saga só termina para a frente. Um Pix enviado e aceito pelo banco é irrevogável; o arranjo prevê devolução, mas ela é outra operação, iniciada pelo recebedor.
- **Repetíveis**: depois do pivô, precisam dar certo eventualmente, então são idempotentes (capítulo 16) e repetidos até dar certo.

O preço da saga é o **I** do ACID. Os estados intermediários ficam visíveis para todo mundo: entre T1 e T3, outra operação pode ler o saldo já debitado e o Pix ainda não enviado. As contramedidas são de projeto. O **lock semântico** é um estado explícito (`enviando`, `capturando`) que outras operações respeitam. As **atualizações comutativas** são mudanças que podem ser aplicadas em qualquer ordem, como lançamentos que só somam. Sem elas, a saga produz as anomalias do capítulo 3, só que agora entre sistemas, sem SSI para salvar ninguém.

Quem decide o próximo passo? Duas formas:

	Orquestração	Coreografia
Lógica do fluxo	Num orquestrador, uma máquina de estados persistida	Espalhada: cada serviço reage a eventos e publica os seus
Estado da saga	Uma linha, consultável	Reconstruído juntando eventos de vários serviços
Acoplamento	Orquestrador conhece todos os passos	Serviços conhecem só eventos
Risco típico	Orquestrador vira deus de tudo	Ciclo de eventos que ninguém desenhou
Bom para	Fluxos com compensação e prazos	Reações independentes, sem desfazer nada

Coreografia com compensação é a forma mais distribuída de ninguém saber por que o dinheiro voltou.

Outbox e CDC: o evento entra na mesma transação

Sobrou a escrita dupla banco + broker. O **padrão outbox** resolve sem 2PC: em vez de publicar, a transação local **grava o evento numa tabela** (a *outbox*) junto com a mudança de estado. Um COMMIT, um sistema, atomicidade de verdade: ou existem o pagamento e o evento, ou nenhum dos dois. Um processo separado, o **relay**, lê a outbox e publica no broker. Se o relay morre depois de publicar e antes de registrar que publicou, publica de novo: at-least-once, e o consumidor deduplica pelo ID do evento (capítulo 16). A janela não sumiu; foi movida para um lugar onde o pior resultado é uma duplicata.

O relay lê a outbox de dois jeitos.

Polling: consulta periódica pelas linhas não publicadas. Simples e com uma armadilha mecânica: quem lê "tudo com `id` maior que o último que publiquei" perde eventos, porque os IDs seguem a ordem do `nextval`, e os COMMITs seguem a ordem que quiserem.

```
t1  tx A: INSERT outbox, id 100    (ainda sem COMMIT)
t2  tx B: INSERT outbox, id 101, COMMIT
t3  relay: lê id > 99, vê só 101, publica, salva "último = 101"
t4  tx A: COMMIT
t5  relay: lê id > 101. O 100 nunca mais será visto.
```

O polling correto marca cada linha (`publicado_em`) ou a apaga depois de publicar, o que custa uma escrita a mais por evento no banco de origem.

CDC (*change data capture*): o relay lê o log de escrita do banco. No PostgreSQL, com `wal_level = logical`, a **decodificação lógica** transforma o WAL em mudanças por linha, entregues por um **slot de replicação** na ordem do COMMIT, e a *publication* restringe quais tabelas saem. O Debezium é o exemplo de referência de conector, com um roteador específico para outbox. Sem consulta repetida e sem escrita a mais. O custo é operacional: o slot guarda o WAL até o consumidor confirmar, então um conector parado faz o disco do primário encher sem parar. O `max_slot_wal_keep_size` (PostgreSQL 13+) limita esse acúmulo invalidando o slot, e um slot invalidado perde o ponto de leitura. E até o PostgreSQL 16, o slot lógico não sobrevive a um failover para a réplica; o 17 passou a sincronizá-lo.

Fazer CDC direto das tabelas internas (cobranças, lançamentos) também funciona, e publica o seu schema interno como contrato. A Lei de Hyrum do capítulo 11 chega em dez dias. A outbox é o contrato explícito: o evento tem o formato que você escolheu.

Conciliação: a verificação que não confia em ninguém

Outbox e sagas fecham as janelas que a Lastro conhece. A **conciliação** pega as que ela não conhece: bug no orquestrador, parceiro que confirmou e não fez, arquivo que diz outra coisa. É uma comparação periódica entre o estado registrado pela Lastro e o estado declarado por cada parceiro, casando registro a registro por uma chave comum, e classificando o que não casa.

Ela funciona porque não depende de nenhum mecanismo que possa ter falhado: o arquivo do adquirente nem sabe que a saga existe. É a única peça que parte do princípio de que todo o resto mentiu, e nisso o financeiro e a planilha dele estavam certos desde o capítulo 1. Eles nunca vão deixar você esquecer.

Na Lastro: outbox, duas sagas e um arquivo às 6h

Outbox com CDC. Toda transação que muda o estado de uma cobrança ou de um repasse grava uma linha em `eventos_saida`, no mesmo COMMIT: `id` (o `evt_` UUIDv7 do envelope), `chave` (`cobranca_id` ou `repasse_id`), `tipo`, `payload` (o envelope do capítulo 11). Um conector CDC lê uma publication só com `eventos_saida` e publica em `cobrancas.eventos` (e no tópico novo `repasses.eventos`) com a chave da linha. A API não fala mais com o broker.

A escolha do CDC sobre o polling vem do recurso mais disputado da Lastro, o primário do ledger (capítulos 4 e 14): o polling acrescentaria uma escrita por evento, 500 por segundo no pico da Black Friday, mais consultas a cada fração de segundo, e o CDC não acrescenta nenhuma. O CDC também entrega na ordem do COMMIT, que a chave por cobrança aproveita. Os custos assumidos:

- `wal_level = logical` exige reiniciar o PostgreSQL. Foram 14 segundos de banco fora numa madrugada de quarta, com retry nos clientes (capítulo 15) e o financeiro avisado por escrito, em três vias.
- Alerta quando o slot retém mais de 5 GB de WAL; `max_slot_wal_keep_size` em 50 GB, abaixo do disco livre.
- `eventos_saida` guarda 7 dias, apagados em lotes diários (capítulo 7). Slot perdido (invalidação ou failover) se recupera republicando a outbox a partir do último evento que chegou ao tópico. Duplicatas são problema do `id`, que os consumidores já deduplicam.

Com poucos eventos por segundo e um banco sem decodificação lógica (alguns serviços gerenciados limitam isso), o polling com `publicado_em` seria a escolha, ao custo de escrita no primário.

A saga do repasse, orquestrada, com o estado na linha de `repasses`:

```
T1 (local, 1 COMMIT)  débito: disponível do vendedor -> repasses_em_transito
                      estado = enviando; evento repasse.criado na outbox
T2 (banco parceiro)   Pix com o ID do repasse (idempotente, cap. 16)
        aceito ──────► PIVÔ. T3 (local): em_transito -> conta do liquidante,
        │               estado = enviado; evento repasse.enviado
        recusado ────► C1 (local): em_transito -> disponível do vendedor,
        │               estado = falhou; evento repasse.falhou
        sem resposta ► consulta pelo ID com backoff (cap. 15); decide aceito/recusado
```

A conta `repasses_em_transito` é o lock semântico: o dinheiro saiu do disponível em T1, então o vendedor 42 não consegue sacar o mesmo valor duas vezes enquanto o banco pensa, nem com a sétima aba. O ledger soma zero em todos os estados intermediários. T3 é repetível: depois do pivô, só para a frente.

A saga do pagamento com cartão. O processo de resolução do capítulo 15 vira o orquestrador das duas sagas: escolhe com `SKIP LOCKED` as linhas com `proxima_acao_em` vencida e executa o passo seguinte. A autorização é compensável (o desfazimento do capítulo 15). O pivô é a captura. Depois dela, os lançamentos de saldo a liberar e o evento são repetíveis, num COMMIT só, com outbox. Desfazer uma cobrança capturada é o estorno: uma saga própria com um lançamento invertido local e o estorno no adquirente, cada passo com a referência da cobrança.

Orquestração porque os dois fluxos têm compensação, prazos e precisam responder "em que passo está o repasse 88.412?" com uma consulta, que é o que o chargeback e a conciliação perguntam. Coreografia fica para quem só reage: webhooks, Vigia e a própria conciliação, que consomem eventos e não desfazem nada.

A conciliação diária, às 6h, depois da chegada do arquivo do adquirente (transações e liquidações do dia anterior) e do extrato do banco liquidante:

Classe	Chave	Exemplo	Ação
Casou	referência, EndToEndId, ID do repasse	Quase tudo	Nenhuma
Só no parceiro	Referência	Captura que a Lastro registra como `recusada` (os 412 da Black Friday)	Estorno automático pela saga, com alerta
Só na Lastro	Referência	Captura sem registro no adquirente	Consulta ao adquirente; se confirmar, fila manual
Valor diferente	Referência	MDR descontado diferente do contratado	Fila manual do financeiro

Classe	Chave	Exemplo	Ação
Estado sem efeito	ID do repasse	Repasse `enviando` há mais de 24 h	Orquestrador acordado, alerta
Evento sem entrega	`evt_`	`cobranca.paga` sem entrega concluída em 3 dias	Lista no painel do marketplace

Num dia comum, com 25 mil cobranças, sobram algumas dezenas de linhas fora de "casou". A fila manual tem interface web. O financeiro exporta a fila para uma planilha.

Armadilhas e casos reais

> **Armadilha:** compensação que pode falhar. Se C1 depende de um parceiro fora do ar, a saga fica parada no meio, num estado que nenhum desenho previu. Compensações precisam ser repetíveis e idempotentes, igual aos passos depois do pivô. A da Lastro é local de propósito.
>
> **Armadilha:** saga sem estado persistido. O orquestrador que guarda "em que passo estou" em memória perde a saga no primeiro reload, e reload é o que causou este capítulo inteiro.
>
> **Armadilha:** slot de replicação esquecido. Um conector CDC desligado "só para testar" no ambiente de homologação segura o WAL até o disco acabar. Em homologação, o Carlos descobriu num sábado. O disco tinha 200 GB e durou 11 dias.
>
> **Armadilha:** achar que a outbox garante o efeito. Ela garante que o evento sai; se o Bazar fez algo com ele, só a DLQ do capítulo 17 e a conciliação contam.
>
> **Na vida real:** Pat Helland, em "Life beyond Distributed Transactions" (2007), descreveu como sistemas grandes abandonam transações distribuídas: entidades atualizadas atomicamente uma de cada vez, mensagens entre elas com entrega at-least-once, idempotência no destino e "atividades" que guardam o que cada parceiro já respondeu. É a Lastro deste capítulo, descrita antes de o fundador saber programar.

Resumo

- Escrita dupla: dois sistemas sem transação comum; toda ordem tem uma janela de falha, e retry não a fecha.
- 2PC: quem votou SIM espera a decisão segurando locks; bloqueante, e só entre participantes que implementam o protocolo.
- Saga: transações locais com compensações semânticas; compensáveis, pivô, repetíveis; sem isolamento, com lock semântico e atualizações comutativas.
- Orquestração para fluxos com compensação e prazo; coreografia para reações independentes.
- Outbox: evento na mesma transação; relay por polling (cuidado com a ordem dos IDs) ou CDC (decodificação lógica, slot que retém WAL); at-least-once com deduplicação.
- Conciliação: verificação independente contra os parceiros, que pega o que os mecanismos deixaram passar.

A Lastro agora

Eventos saem pela outbox `eventos_saida`, lida por CDC num slot lógico, com alerta de WAL retido e recuperação pela própria tabela. Repasse e pagamento com cartão são sagas orquestradas pelo antigo processo de resolução, com a conta `repasses_em_transito` como lock semântico, a captura e o Pix aceito como pivôs, e o estorno como compensação. A conciliação das 6h casa o ledger com o adquirente e o banco liquidante e manda o que não casa para uma fila com interface web. O vendedor 42 recebeu os R$ 300,00 de volta pela C1, com lançamento e tudo, e está decepcionado: ele queria uma história melhor.

O `repasse_final_v2_agora_vai.py` continua no crontab de `temp-01`, com a cópia própria do código e sem saga nenhuma, porque ninguém teve coragem de abrir o arquivo. Na sexta, o vendedor 42 viu no painel R$ 870,00 de saldo disponível, clicou em sacar e recebeu `saldo-insuficiente`. Os dois números estavam certos. Só não estavam certos ao mesmo tempo.

Referências

- Hector Garcia-Molina e Kenneth Salem, "Sagas", ACM SIGMOD, 1987. https://dl.acm.org/doi/10.1145/38713.38742

- Pat Helland, "Life beyond Distributed Transactions: an Apostate's Opinion", CIDR 2007. https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf
- PostgreSQL 16, `PREPARE TRANSACTION`. https://www.postgresql.org/docs/16/sql-prepare-transaction.html
- PostgreSQL 16, Logical Decoding. https://www.postgresql.org/docs/16/logicaldecoding.html
- PostgreSQL 17, Logical Replication Failover. https://www.postgresql.org/docs/17/logical-replication-failover.html
- Debezium, Outbox Event Router. https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html
- James C. Corbett et al., "Spanner: Google's Globally-Distributed Database", OSDI 2012. https://www.usenix.org/conference/osdi12/technical-sessions/presentation/corbett
- Chris Richardson, padrões Saga e Transactional Outbox. https://microservices.io/patterns/data/saga.html

CAPÍTULO 19

Modelos de consistência

Sexta, 17h02: dois números certos, com onze minutos de diferença

Às 17h02 de sexta, o vendedor 42 abriu o painel no notebook, viu R$ 870,00 de saldo disponível e fez o que um homem com o histórico dele faz diante de dinheiro na tela: clicou em sacar.

`saldo-insuficiente`. Ele atualizou: R$ 870,00. Clicou de novo: `saldo-insuficiente`. Na terceira, tirou um print, e print do vendedor 42 tem valor de petição inicial.

O chamado chegou pelo Bazar às 17h09: "URGENTE!!!! O SISTEMA MOSTRA O DINHEIRO E NÃO DEIXA SACAR. ISSO É CALOTE???". O financeiro achou a explicação em três segundos: às 16h58, pelo celular, o vendedor 42 tinha sacado os mesmos R$ 870,00, e o Pix já estava na conta dele. O trigger do capítulo 2 recusou o segundo saque com a frieza de um tabelião. Pela primeira vez na carreira, o vendedor 42 tentou sacar duas vezes sem querer. Ele só acreditou no painel, e o painel estava onze minutos atrasado.

O atraso tem autor. Duas semanas antes, o Carlos tirou o saldo do painel de cima do primário: 1.500 abas no pico, atualizando a cada 15 s, davam 100 `SUM`s por segundo, alguns de 0,7 s (a Capinhas Hiper, sempre ela). Agora toda transação que mexe numa conta disponível grava na outbox do capítulo 18 um evento `conta.movimentada`, publicado no tópico `contas.movimentos` (chave `conta_id`), e o consumidor `painel-saldos` o soma na tabela `saldos_painel`, no `pg-notificacoes`. Leitura de 1 ms, atraso típico de 300 ms. O Carlos chamou de "saldo em tempo real", como chama tudo, inclusive o polling de 100 ms.

Naquela sexta caiu o D+30 da Black Friday: a partir das 16h50, o job de liberação gerou 310 mil eventos. O `painel-saldos`, instância única, aplica uns 500 por segundo. O saque das 16h58 esperou na fila atrás de 300 mil liberações da Capinhas e foi aplicado às 17h13.

O gerente perguntou se não era só mostrar o número certo. O Carlos propôs fazer o saque conferir o saldo na projeção, "aí os dois batem". Batem: às 17h02 os dois diriam R$ 870,00, e o vendedor 42 receberia R$ 870,00 pela segunda vez, com tela e banco em perfeito acordo. Saque dobrado, agora com a aprovação de dois sistemas.

A pergunta deste capítulo: qual leitura precisa ver o estado de agora, qual pode ver o de alguns minutos atrás, e como garantir que a atrasada nunca contradiga o que o próprio usuário acabou de fazer?

"Consistência", a palavra que significa três coisas e briga nas três

É a terceira palavra mais abusada da computação, atrás de "simples" e de "temporário". O C do ACID (capítulo 3) é sobre invariantes: o ledger soma zero. O C do CAP é linearizabilidade, que vem já já. E "modelo de consistência" é o contrato entre um sistema e quem lê dele: dadas as escritas que aconteceram, quais valores uma leitura pode devolver.

Formalmente, um modelo é um conjunto de histórias permitidas (operações com início, fim e valor, como os clientes observaram), e é mais forte que outro quando permite menos histórias. Em português de palco: é a lista de mentiras que o sistema está autorizado a te contar. A projeção do Carlos tinha a lista mais comprida do mercado e usou o item "seu próprio saque não aconteceu".

Linearizabilidade, ou fingir que existe uma cópia só

A definição é de Herlihy e Wing (1990). Uma execução é **linearizável** quando cada operação parece ter efeito atômico em algum instante entre a invocação e a resposta, numa ordem compatível com o tempo real: se A terminou antes de B começar, A vem antes de B. Com cinquenta réplicas, o sistema se comporta, para quem está de fora, como uma cópia só.

```
tempo →
Celular:   |--- saque 870 (saldo: 870 → 0) ---|
Notebook A:                                        |-- lê --| → 870   PROIBIDO
Notebook B:              |-------- lê --------|               → 870 ou 0, ambos válidos
Notebook C:                                                 |-- lê --| → depois que
alguém leu 0,
                                                                          ninguém mais lê
870
```

O notebook A começou depois que o saque respondeu, então tem que ver 0. O B é concorrente e pode ver qualquer um dos dois valores. A terceira regra pega os sistemas espertinhos: depois que uma leitura devolveu o valor novo, nenhuma leitura que comece depois pode devolver o antigo. O saldo não fica piscando entre 0 e 870 conforme a réplica que atendeu, como letreiro de motel.

Linearizabilidade trata de operações sobre um objeto; serializability (capítulo 3), de transações sobre vários, sem exigir tempo real: em tese, um banco serializable pode ordenar a sua transação antes de uma que já tinha feito COMMIT quando a sua começou. As duas juntas formam a *strict serializability*. Confundir uma com a outra rende muito postmortem.

O custo: alguém decide a ordem, seja um único nó (o primário), seja um acordo entre réplicas a cada operação, que é consenso (capítulo 20). A operação paga pelo menos uma ida e volta até quem decide, 100 ms do outro lado do oceano (capítulo 8), e nenhuma otimização fura a velocidade da luz. O fundador sempre disse que no tempo dele era um servidor só e funcionava. Funcionava, e era linearizável de graça. Ele só não sabia o nome.

Descendo a escada: sequencial, causal, eventual

Consistência sequencial (Lamport, 1979): existe uma ordem total única de todas as operações, todos os clientes concordam com ela, e ela respeita a ordem do programa de cada cliente. Caiu o tempo real. O notebook A poderia ler 870, contanto que todo mundo concordasse que a leitura veio "antes" do saque. Viagem no tempo liberada, desde que em excursão: todos viajam juntos.

Consistência causal (Ahamad e colegas, 1995): só operações com relação de causa precisam aparecer na mesma ordem para todos. Se o Bazar leu a cobrança paga e, por isso, pediu um estorno, ninguém pode ver o estorno sem ver o pagamento; operações concorrentes podem aparecer em ordens diferentes para clientes diferentes. A relação de causa é o "aconteceu antes" de Lamport, que o capítulo 20 rastreia com relógios lógicos. Ela se mantém durante uma partição, com todos os nós respondendo, e Mahajan, Alvisi e Dahlin (2011) mostraram que nada mais forte que a variante *real-time causal* é possível num sistema que sempre responde.

Consistência eventual (termo popularizado por Werner Vogels): se as escritas pararem, as réplicas convergem para o mesmo valor. Só isso. Sem prazo e sem promessa sobre o que você lê no caminho, inclusive valor que volta a ser mais velho. É a garantia que um político daria: vai convergir, e não pergunte quando. A projeção do Carlos convergiu direitinho às 17h13, onze minutos e um chamado em caixa alta depois.

A favor dela: saldo é soma, e soma é comutativa. Aplicar os eventos de uma conta em qualquer ordem chega ao mesmo total, e a deduplicação por `transacao_id` (capítulo 16) torna a reaplicação inofensiva. É o truque dos CRDTs, tipos replicados que convergem sozinhos. A aritmética garante o fim da história; o meio da história ela deixa para o vendedor 42 descobrir.

Garantias de sessão: consistência eventual com modos

Entre o eventual puro e o causal ficam as garantias de sessão, de Terry e colegas (1994, sistema Bayou). Nada sobre o mundo; só coerência para cada cliente, dentro da sessão dele.

Garantia	Promessa	Na Lastro, quando falha
Read-your-writes	Uma leitura reflete todas as escritas anteriores da mesma sessão	O vendedor saca e o painel mostra o dinheiro ainda lá (sexta, 17h02)
Leituras monotônicas	Uma leitura nunca reflete um estado mais antigo que o de uma leitura anterior da sessão	O painel mostra R$ 0,00, atualiza e mostra R$ 870,00 de novo
Escritas seguem leituras	Uma escrita feita depois de ler X é ordenada depois de X em todas as réplicas	O estorno chega a uma réplica que ainda não viu o pagamento
Escritas monotônicas	As escritas da sessão são aplicadas na ordem em que foram feitas	A troca da chave Pix chega depois do saque que deveria usá-la

Terry acrescentou depois (2013) o *prefixo consistente*: o leitor vê um prefixo da ordem real das escritas, sem buracos. Quem lê o placar vê 0 a 0, 1 a 0, 2 a 0, nunca 0 a 2.

Toda linha da coluna da direita é um chamado do Bazar que ainda não foi aberto. A segunda, um saldo que some e reaparece, transforma qualquer vendedor num conspiracionista. O vendedor 42 já era um sem ajuda.

CAP com a definição certa, e o PACELC que diz o resto

O CAP aparece em entrevista como "escolha dois entre três", dito com a solenidade de quem cita um provérbio chinês, e está errado. A conjectura é de Eric Brewer (2000); o teorema é de Gilbert e Lynch (2002), com estas definições:

- **C** (consistência): linearizabilidade de um registrador de leitura e escrita.
- **A** (disponibilidade): toda requisição recebida por um nó que não falhou recebe resposta. Toda.
- **P** (tolerância a partição): a rede pode perder arbitrariamente muitas mensagens entre os nós.

O teorema: com a rede particionada, nenhum sistema garante C e A ao mesmo tempo. A prova cabe num guardanapo. Dois nós, a rede entre eles caiu, um cliente escreve no nó 1 e outro lê do nó 2. Se o nó 2 responde, pode devolver o valor velho e quebra C. Se espera notícia do nó 1 para não mentir, não responde e quebra A.

Ninguém escolhe deixar a partição de fora: a rede particiona quando quer. A escolha é o que fazer nessa hora, recusar para manter a linearizabilidade ou responder com o que tem. Fora de partição, o CAP não diz nada. E o A do CAP é muito mais exigente que o de um contrato: um sistema "CP" pode entregar 99,99% no SLA, e um que responde tudo com dado de ontem é "A" pelo teorema. Por isso Martin Kleppmann pediu, num artigo inteiro, que parassem de classificar bancos como CP ou AP.

Daniel Abadi completou com o PACELC (2012): se há **P**artição, escolha entre **A** e **C**; senão (**E**lse), entre latência (**L**) e **C**. A segunda metade você paga todo dia: partição é rara, latência é toda requisição. Na classificação de Abadi, Dynamo, Cassandra e Riak na configuração padrão são PA/EL; sistemas ACID completos como VoltDB e Megastore são PC/EC.

O DynamoDB coloca o PACELC na fatura: uma leitura fortemente consistente de até 4 KB consome uma unidade de capacidade de leitura; a eventualmente consistente, meia. Nos índices secundários globais, a leitura forte nem existe. Um teorema de sistemas distribuídos virou item de nota fiscal, e o financeiro vai querer conciliar.

Na Lastro: cada leitura com o contrato de que precisa

A Lastro tem um primário do ledger, sem réplica (capítulo 23). Uma leitura no primário depois que um COMMIT retornou enxerga esse COMMIT, porque o snapshot (do comando em read committed, da primeira consulta em serializable) inclui toda transação já confirmada: é a recência que a linearizabilidade pede. A projeção do Carlos foi a primeira cópia de saldo fora dele. A regra da Lastro: o modelo de cada leitura é escolhido pelo prejuízo de ela vir velha.

Leitura	Modelo	Fonte	O que uma leitura velha causaria
Saldo na checagem de débito (saque, estorno, repasse)	linearizável, dentro da transação serializable (capítulo 3)	primário	Saque dobrado
Tela de confirmação do saque ("disponível agora")	linearizável	primário	Vendedor decide em cima de número falso
Chave de idempotência (capítulo 16)	linearizável (UNIQUE)	primário	Cobrança duplicada
Saldo no cabeçalho do painel e no GraphQL	eventual + read-your-writes + leituras monotônicas	`saldos_painel`	Número atrasado na tela, nunca dinheiro fora

Leitura	Modelo	Fonte	O que uma leitura velha causaria
Webhooks aos marketplaces	eventual, ordem por cobrança, monotônico no cliente pelo campo `versao` (capítulo 11)	broker	Estado antigo descartado pelo cliente
Fechamento e conciliação	snapshot (repeatable read READ ONLY)	primário	Dia fechado pela metade

A proposta do Carlos morre na primeira linha: a leitura que autoriza a saída de dinheiro nunca vem de cópia. A projeção fica, porque o requisito que a criou continua (100 `SUM`s por segundo no pico, só para enfeitar o cabeçalho), agora com contrato de sessão.

Read-your-writes sem token mágico

A sessão, aqui, é o vendedor: o saque saiu do celular e a leitura veio do notebook, então uma garantia presa a um cookie não teria salvado ninguém. A "última escrita da sessão" é o último débito iniciado por aquele vendedor, e quem sabe disso com certeza é o primário.

```
Painel pede o saldo do vendedor 42
  │
  ├─ 1. primário: último débito iniciado por ele        (índice, < 1 ms)
  │      → transacao_id 98.114.230 (saque de 16h58)
  │
  ├─ 2. pg-notificacoes: inbox do painel-saldos já tem 98.114.230?
  │      ├─ sim → devolve saldos_painel (1 ms)
  │      └─ não → espera o NOTIFY de aplicação, até 2 s
  │               ├─ chegou → devolve saldos_painel
  │               └─ não chegou → devolve "atualizando", sem número
  │
  └─ tela de saque: sempre o primário, dentro da checagem
```

A parte barata ("qual escrita minha você precisa refletir") vai ao primário; a cara, a soma de 1,1 milhão de lançamentos, fica na projeção. O `painel-saldos` grava saldo e linha da inbox na mesma transação, então "a inbox tem a transação" e "o saldo inclui a transação" são o mesmo fato. O painel mostra o número certo ou "atualizando". Às 17h02, teria dito "atualizando" por onze minutos, o que irrita, mas não gera print com valor jurídico.

Leituras monotônicas saem por construção: `saldos_painel` vive numa única instância, e o consumidor só aplica, nunca desfaz, então toda leitura reflete um superconjunto da anterior. Vale enquanto houver uma cópia só; com réplica (capítulo 23), o cliente terá de carregar a posição mais alta que já viu. Fica anotado, porque o primeiro a esquecer vai ser quem anotou.

O `painel-saldos` passa a três instâncias (12 partições permitem até 12) e o atraso vira métrica com alerta acima de 30 s, porque "eventualmente" sem número é horóscopo. A recusa de saque leva no corpo RFC 9457 o saldo lido na própria transação e o último débito ("disponível: R$ 0,00; último débito: saque de R$ 870,00 às 16h58"), do mesmo snapshot que decidiu. O erro não discute com o ledger.

Na partição: sem o `pg-notificacoes`, o cabeçalho some e o saque continua. Sem o primário, a projeção segue mostrando saldo (A para quem só olha) e todo débito recebe `503` (C para quem quer levar dinheiro). Cada leitura escolheu seu lado do CAP antes da partição, o único momento em que dá para escolher com calma.

Com outros requisitos, a escolha seria outra: saldo servido em várias regiões com latência baixa pediria consistência causal com réplicas locais; dezenas de acessos por segundo em vez de 100 `SUM`s dispensariam a projeção, e ler do primário seria mais simples.

Armadilhas e casos reais

> **Armadilha:** check-then-act com leitura eventual. A corrida do capítulo 2 durava microssegundos; com uma cópia atrasada, dura o atraso da cópia. Onze minutos, na Lastro. Toda decisão que gasta, reserva ou libera alguma coisa lê da fonte linearizável. A proposta do Carlos era essa armadilha com laço de presente.
>
> **Armadilha:** "fortemente consistente" no material de vendas pode significar linearizável por chave e nada entre chaves, ou só no líder, ou só com uma opção desligada por padrão. Pergunte qual história ele proíbe, em qual configuração, e se há teste publicado (a página do Jepsen tem o mapa). O vendedor do banco tem um brinde.
>
> **Na vida real:** o artigo do Dynamo (DeCandia e colegas, 2007) descreve o carrinho da Amazon projetado para aceitar escrita sempre; versões divergentes são mescladas, e o artigo admite que itens removidos podem reaparecer. Num carrinho, um item ressuscitado é aceitável. Num saque, a ressurreição seria o vendedor 42 recebendo duas vezes, e ele com certeza já leu esse artigo.
>
> **Na vida real:** até dezembro de 2020, o Amazon S3 tinha consistência eventual para sobrescritas, remoções e listagens, e ferramentas de *data lake* mantinham camadas extras só para a listagem que não mostrava o arquivo recém-gravado. Naquele mês, a AWS anunciou consistência forte de leitura após escrita em todas as operações, e muita gambiarra foi demitida sem aviso prévio.

Resumo

- "Consistência": invariantes (ACID), linearizabilidade (CAP), modelo de consistência (o que uma leitura pode devolver).
- Linearizabilidade: efeito atômico entre início e fim, em ordem de tempo real; custa coordenação e um RTT até quem ordena. Serializability não exige tempo real; strict serializability junta as duas.
- Sequencial: ordem total única. Causal: só o que tem causa precisa de ordem; teto sem coordenação. Eventual: converge, sem prazo.
- Garantias de sessão: read-your-writes, leituras monotônicas, escritas seguem leituras, escritas monotônicas; prefixo consistente.
- CAP: na partição, linearizabilidade ou resposta de todo nó. PACELC: sem partição, latência contra consistência.
- O modelo de cada leitura sai do prejuízo de ela vir velha.

A Lastro agora

Débitos, tela de saque e chaves de idempotência leem o primário. O cabeçalho do painel lê `saldos_painel`, com read-your-writes pelo último débito do vendedor, "atualizando" no lugar de número velho e alerta de atraso acima de 30 s. O vendedor 42 recebeu uma explicação por escrito, leu duas vezes e mandou print para o Bazar, por garantia.

O fundador leu a seção sobre o CAP, gostou muito da palavra "disponibilidade" e, na segunda-feira, copiou o crontab de `temp-01` para `api-02`, para que o `repasse_final_v2_agora_vai.py` tivesse alta disponibilidade. Agora ele tem. Roda nas duas máquinas, à meia-noite de cada uma, e as duas discordam sobre que horas é meia-noite.

Referências

- Maurice Herlihy e Jeannette Wing, "Linearizability: A Correctness Condition for Concurrent Objects", ACM TOPLAS, 1990. https://dl.acm.org/doi/10.1145/78969.78972
- Leslie Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs", IEEE Transactions on Computers, 1979. https://www.microsoft.com/en-us/research/publication/make-multiprocessor-computer-correctly-executes-multiprocess-programs/

- Mustaque Ahamad et al., "Causal Memory: Definitions, Implementation, and Programming", Distributed Computing, 1995.
- Prince Mahajan, Lorenzo Alvisi e Mike Dahlin, "Consistency, Availability, and Convergence", UTCS TR-11-22, University of Texas at Austin, 2011.
- Werner Vogels, "Eventually Consistent", Communications of the ACM, 2009. https://dl.acm.org/doi/10.1145/1435417.1435432
- Douglas Terry et al., "Session Guarantees for Weakly Consistent Replicated Data", PDIS 1994.
- Douglas Terry, "Replicated Data Consistency Explained Through Baseball", Communications of the ACM, 2013. https://www.microsoft.com/en-us/research/publication/replicated-data-consistency-explained-through-baseball/
- Seth Gilbert e Nancy Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services", ACM SIGACT News, 2002. https://dl.acm.org/doi/10.1145/564585.564601
- Daniel Abadi, "Consistency Tradeoffs in Modern Distributed Database System Design", IEEE Computer, 2012. https://doi.org/10.1109/MC.2012.33
- Martin Kleppmann, "Please stop calling databases CP or AP", 2015. https://martin.kleppmann.com/2015/05/11/please-stop-calling-databases-cp-or-ap.html
- Jepsen, Consistency Models. https://jepsen.io/consistency
- Giuseppe DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007. https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- Amazon DynamoDB, Read consistency. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html
- AWS News Blog, "Amazon S3 Update – Strong Read-After-Write Consistency", dezembro de 2020. https://aws.amazon.com/blogs/aws/amazon-s3-update-strong-read-after-write-consistency/

CAPÍTULO 20

Tempo, ordem e consenso

Meia-noite, duas vezes, com seis minutos de intervalo

A meia-noite de terça chegou duas vezes na Lastro, e as duas pagaram.

Na segunda, o fundador tinha copiado o crontab de `temp-01` para `api-02`, para dar alta disponibilidade ao `repasse_final_v2_agora_vai.py`. Deu. O relógio de `api-02` estava 3 min 40 s adiantado: o firewall que o fundador configurou ("fecha tudo menos a 443") bloqueia a porta UDP 123, e a máquina passou seis semanas sem NTP, derivando uns 60 ppm. O relógio de `temp-01` estava 2 min 10 s atrasado, porque o daemon de NTP dele morreu em algum momento do ano passado e ninguém foi ao velório.

Às 23h56min20s reais, `api-02` achou que era meia-noite e pagou os repasses automáticos de 380 vendedores. Às 00h02min10s reais, `temp-01` achou que era meia-noite e pagou de novo.

Alguém finalmente abriu o arquivo. O script gera um UUID novo para cada repasse, manda o Pix ao banco e só depois debita o ledger. Na segunda rodada, o banco recebeu IDs inéditos e pagou; o trigger do capítulo 2 recusou cada débito por saldo insuficiente; e o script capturou a exceção com um `except: pass` assinado pelo Carlos, às 2h de uma madrugada de 2024. O ledger ficou impecável. O banco pagou R$ 186.420,00 a mais. O tabelião protegeu o livro, e o dinheiro saiu pela janela do vizinho.

A conciliação das 6h achou 380 Pix de saída sem lançamento. O financeiro passou a semana ligando para vendedores. O vendedor 42 devolveu em dez minutos, com um bilhete: "dessa vez não fui eu". Ele está gostando muito da fase de vítima.

O script também gera o fechamento do dia, com `criado_em < hoje 00:00` no relógio da máquina. Saíram dois fechamentos diferentes: o de `api-02` sem os últimos 3 min 40 s de Pix, o de `temp-01` com 2 min 10 s do dia seguinte. A planilha do financeiro tinha um terceiro número, e era o certo.

A pergunta: como garantir um único executor para um job que duas máquinas querem disparar, e como decidir o que pertence a um dia, quando nenhum relógio concorda com outro?

Relógios físicos, ou cada máquina tem a sua opinião sobre que horas são

Todo servidor conta o tempo com um cristal de quartzo, e todo cristal erra. O erro é a deriva, em partes por milhão: 1 ppm são 86 ms por dia; os 200 ppm que o Google assume como pior caso (artigo do Spanner) são 17 segundos por dia. Ela muda com a temperatura: o vizinho de rack esquentando mexe no seu.

O NTP corrige trocando quatro carimbos com um servidor de referência: o cliente envia em t1, o servidor recebe em t2 e responde em t3, o cliente recebe em t4.

```
atraso (δ)       = (t4 − t1) − (t3 − t2)
deslocamento (θ) = ((t2 − t1) + (t3 − t4)) / 2
```

A fórmula supõe que ida e volta levam o mesmo tempo. Quando a rota é assimétrica, o erro chega a δ/2, e nada no protocolo detecta isso. Na mesma rede, o NTP costuma manter as máquinas a poucos milissegundos umas das outras; pela internet, dezenas de milissegundos. Alguns milissegundos de incerteza, garantidos por uma suposição que ninguém confere: é o horóscopo da infraestrutura, só que com RFC (a 5905).

Pior que errar é corrigir. Se o deslocamento é pequeno, o daemon acelera ou freia o relógio (*slew*, no máximo 500 ppm no ntpd). Se é grande, ele pula (*step*), para frente ou para trás. Um relógio de parede pode voltar no tempo, e um `fim - inicio` medido com ele pode dar negativo. Duração se mede com o relógio monotônico (`CLOCK_MONOTONIC`; em Python, `time.monotonic()`), que nunca volta e não significa nada fora da máquina: vale para medir intervalo, nunca para comparar com outro servidor.

E tem o segundo bissexto, que acerta o UTC com a rotação da Terra e produz 23:59:60, horário que metade do software do mundo considera ficção científica (dois casos na seção de armadilhas). Grandes provedores espalham o segundo extra ao longo de horas (*leap smear*), e a CGPM decidiu em 2022 abandonar os segundos bissextos até 2035. A humanidade desistiu de acertar o relógio com o planeta. Você vai desistir de acertar o seu com o do vizinho já já.

Ordem sem relógio: Lamport e os vetores

Em 1978, Leslie Lamport fez a pergunta certa: para ordenar eventos, eu preciso mesmo do tempo? Ele definiu a relação **aconteceu antes** (→): a → b se a e b ocorreram no mesmo processo e a veio antes; ou se a é o envio de uma mensagem e b o recebimento dela; ou, por transitividade, se existe c com a → c e c → b. Quando nem a → b nem b → a,

os eventos são **concorrentes**. Concorrente quer dizer que nenhum soube do outro, e não que aconteceram "ao mesmo tempo", conceito que, como a seção anterior mostrou, ninguém consegue medir.

O **relógio de Lamport** é um contador por processo, com três regras: incrementa a cada evento; envia o valor junto de cada mensagem; ao receber, ajusta para `max(local, recebido) + 1`. Garantia: se a → b, então L(a) < L(b). A volta não vale: L(a) < L(b) pode ser só coincidência. Para uma ordem total, desempata pelo identificador do processo. É uma ordem coerente com a causalidade, sem um único cristal de quartzo envolvido, e é por isso que o Lamport tem o prêmio Turing e o fundador tem um crontab.

Para saber se dois eventos são concorrentes, entra o **vector clock** (Fidge e Mattern, 1988): cada processo mantém um vetor com um contador por processo, incrementa a própria posição a cada evento, envia o vetor e, ao receber, faz o máximo posição a posição. a → b se e só se V(a) ≤ V(b) em toda posição e o vetor é diferente; se nenhum domina o outro, são concorrentes, e alguém precisa decidir o conflito. O Dynamo usava essa estrutura para detectar versões divergentes do carrinho (aquele, do capítulo 19, com itens zumbis). O custo é o tamanho: um contador por participante, viajando em toda mensagem.

O Spanner foi pelo caminho oposto e caro: GPS e relógios atômicos em cada data center, uma API que devolve um intervalo de incerteza em vez de um instante, e escrita que espera a incerteza passar antes de confirmar. Quem tem dinheiro compra relógio atômico. Quem tem `temp-01` usa Lamport.

Consenso, ou como três máquinas concordam sem confiar no relógio de nenhuma

Escolher qual máquina roda o job é um caso de **consenso**: processos propõem valores e todos decidem o mesmo (acordo), o valor decidido foi proposto por alguém (validade), e todo processo que não falhou decide (terminação). Parece trivial até o capítulo 15 voltar: quem não responde pode estar morto, lento ou isolado.

Fischer, Lynch e Paterson provaram em 1985 (o resultado FLP) que, num sistema totalmente assíncrono, nenhum algoritmo determinístico garante consenso se um único processo puder falhar. Os algoritmos práticos contornam isso usando timeouts para suspeitar de falha: nunca decidem errado, e só deixam de decidir enquanto a rede se comporta pior que os timeouts. Segurança sempre, progresso quando dá. É o acordo pré-nupcial dos sistemas distribuídos.

O **Raft** (Ongaro e Ousterhout, 2014) é o algoritmo de consenso que se pode ler sem pedir licença médica. As regras:

- Cada nó é seguidor, candidato ou líder. O tempo é dividido em mandatos (*terms*) numerados, e cada número de mandato só ocorre uma vez, crescendo sempre.
- O líder manda *heartbeats*. O seguidor que passa um timeout aleatório (o artigo sugere 150 a 300 ms) sem ouvir nada vira candidato, incrementa o mandato e pede votos.
- Cada nó vota em no máximo um candidato por mandato, e só em quem tem o log pelo menos tão atualizado quanto o seu. Quem junta maioria vira líder. Duas maiorias de um mesmo grupo sempre se cruzam em ao menos um nó, então dois líderes no mesmo mandato são impossíveis.
- O líder acrescenta cada comando ao log e replica para os seguidores; o comando está confirmado (*committed*) quando a maioria o gravou, e só então é aplicado.
- Um nó que recebe mensagem com mandato maior que o seu desce a seguidor na hora. Um líder antigo que acorda de uma pausa descobre que foi deposto na primeira resposta que recebe.

```
mandato 7:  A (líder) ── heartbeat ──▶ B, C
            A congela (pausa de GC, 12 s)
mandato 8:  B: "faz 240 ms que não ouço ninguém" → candidato
            B ── pede voto (mandato 8) ──▶ C: "seu log está em dia, voto em você"
            B: 2 de 3 → líder
            A acorda: "sou o líder do mandato 7!"
            C: "estamos no 8, querido"   → A vira seguidor
```

São `2f + 1` nós para tolerar `f` falhas: 3 aguentam 1, 5 aguentam 2. Usam Raft o etcd (o banco de configuração do Kubernetes), o Consul e o Kafka no modo KRaft; o ZooKeeper usa o ZAB, parecido. Ninguém implementa Raft na aplicação, e quem tenta passa os anos seguintes colecionando casos de borda.

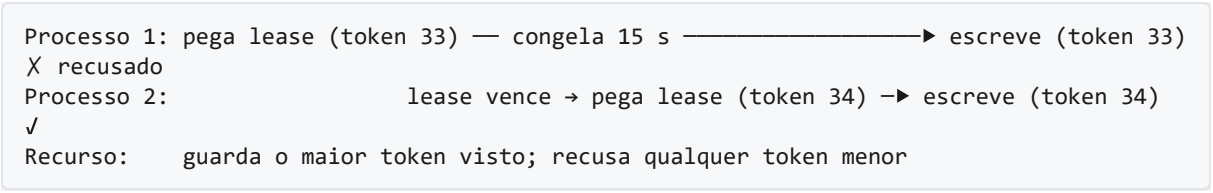
Leases e fencing tokens: o líder que não sabe que morreu

Consenso decide quem é o líder agora. O problema é que "agora" já passou quando a mensagem chega.

Um **lease** é um lock com prazo: "você é o dono até T". Se o dono morre, o lease expira e outro assume, sem ninguém precisar provar a morte. O prazo depende de os relógios andarem em ritmo parecido (a deriva, e não a hora certa), e o dono precisa ser pessimista: mede com o relógio monotônico a partir de antes de enviar o pedido, e para de agir um pouco antes do fim.

Nem assim basta. Entre "confiro que o lease vale" e "escrevo", o processo pode congelar: pausa de GC, VM migrada, `SIGSTOP`, disco que demora 40 s para devolver uma página. Ele acorda achando que ainda é dono, o lease venceu há 10 s, outro assumiu, e agora

existem dois líderes, um deles zumbi com credencial de escrita. O capítulo 2 deixou isso prometido; esta é a conta.



O **fencing token** é um número que cresce a cada concessão do lease, e quem confere é o recurso protegido, nunca o processo. O zumbi pode acreditar no que quiser; a escrita dele é recusada. Martin Kleppmann usou esse argumento em 2016 contra o Redlock: um lock sem token não protege nada contra pausas, por mais réplicas do Redis que se usem. O mandato do Raft, a revisão do etcd e o `zxid` do ZooKeeper servem de token.

Na Lastro: um dono, um token e um dia que não depende de relógio

O requisito tem duas metades que brigam. O repasse automático precisa sair todo dia mesmo se `temp-01` morrer, o que pede duas máquinas. E cada repasse precisa sair exatamente uma vez, o que pede um único executor. Lease com fencing resolve as duas.

Onde guardar o lease	Garante	Custa
Lock em Redis (`SET NX PX`)	Exclusão enquanto nada pausa	Sem token natural; peça nova; o capítulo 2 já descartou
etcd, ZooKeeper ou Consul	Consenso real, revisão como token	Cluster de 3 nós para operar; o ledger ainda precisa conferir o token
Linha no primário do ledger	Token conferido na mesma transação da escrita, um só relógio (o do banco)	Disponível só enquanto o primário estiver (já é verdade para o repasse)

A Lastro escolhe a linha no primário. O recurso a proteger é o próprio ledger, e conferir o token dentro da transação que debita é a cerca mais curta possível. A tabela `lideranca_jobs` tem `job`, `dono`, `epoca` e `expira_em`. Assumir é um `UPDATE` que só vence se o lease expirou e devolve `epoca + 1`; renovar, a cada 10 s, é um `UPDATE` que só vence se dono e época ainda são os mesmos. O prazo é 30 s, medido pelo `now()` do primário, então o relógio das máquinas não entra na conta. Quem não conseguiu renovar em 20 s, pelo monotônico, para.

E cada transação de repasse começa conferindo o token:

```
-- primeira linha de cada transação de repasse
SELECT epoca FROM lideranca_jobs WHERE job = 'repasse' FOR SHARE;
--  epoca
-- -------
--      8
-- token do processo = 7 → ROLLBACK: existe um dono mais novo
```

O `FOR SHARE` conflita com o `UPDATE` de quem quer assumir: a troca de dono espera os repasses em andamento, e nenhuma transação com token velho faz COMMIT depois dela. Com algumas centenas de repasses por dia, o lock não pesa.

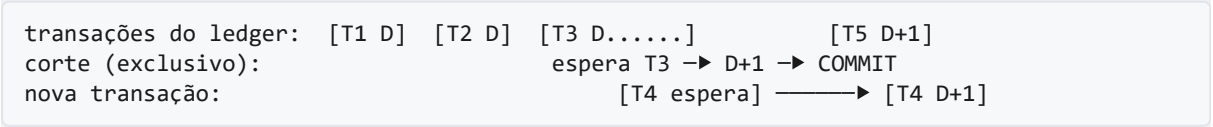
O banco parceiro não confere tokens, então o lado de fora se protege como no capítulo 16: o ID do repasse passa a ser determinístico, `(vendedor, dia contábil)`, e o banco recusa um ID repetido. O script, finalmente aberto, vira só o agendador: cria as sagas de repasse do capítulo 18, que debitam antes de enviar o Pix. O nome do arquivo ficou. Renomear quebraria o crontab, o monitoramento e a fé.

O etcd seria a escolha certa se o recurso protegido não soubesse conferir tokens numa transação, ou se vários serviços precisassem do mesmo líder (o Kubernetes, no capítulo 35, traz leases apoiados nele). E o failover do primário (capítulo 23) vai depender de consenso para não haver dois primários.

O dia contábil sem relógio de parede

O fechamento por `criado_em` tem dois defeitos. Um é o relógio da máquina. O outro sobrevive a qualquer NTP: no PostgreSQL, `now()` é o instante de início da transação. Uma transação que começa às 23h59min59,8s e faz COMMIT às 00h00min00,3s ganha carimbo de ontem, e uma consulta à meia-noite em ponto não a enxerga. Na consulta de amanhã, ela é de ontem. O lançamento fica órfão de dia, igual aos buracos de ID do capítulo 18.

A Lastro corta pela ordem dos commits. Toda transação do ledger pega um advisory lock compartilhado da época contábil e carimba nela `dia_contabil`, lido de `periodo_aberto`. O job de corte pega o mesmo advisory lock em modo exclusivo, o que espera as transações em andamento terminarem e segura as novas por alguns milissegundos, avança `periodo_aberto` e faz COMMIT. Tudo o que carimbou o dia D fez COMMIT antes do corte; tudo depois carimba D+1. O relógio decide só quando cortar. O que pertence ao dia é decidido pela ordem dos commits, e o fechamento do dia é `WHERE dia_contabil = D`.



Dentro do ledger, a ordem que importa para corretude já é explícita: o estorno referencia a transação original (capítulo 1), uma relação causal registrada, sem carimbo de hora envolvido. Com um único primário ordenando todas as escritas, a Lastro não precisa de relógios de Lamport nem de vetores. Eles entram em cena quando há mais de um nó aceitando escrita, o assunto da replicação multi-líder no capítulo 23.

Por fim, NTP (o chrony, por exemplo) em todas as máquinas, porta 123 de saída liberada para os servidores de tempo, e alerta com deslocamento acima de 100 ms. O relógio sincronizado serve a logs, certificados (capítulo 10) e à hora de disparar o corte, nunca à decisão de quem é dono ou do que é de qual dia.

Armadilhas e casos reais

> **Armadilha:** "último a escrever vence" com carimbo de relógio. Em bancos que resolvem conflito por timestamp, como o Cassandra na configuração padrão, a máquina adiantada ganha todas as discussões, inclusive com escritas que aconteceram depois dela. Um relógio 3 min 40 s adiantado vira um usuário com poder de voltar no tempo e apagar o que os outros escreveram.
>
> **Na vida real:** no segundo bissexto de 30 de junho de 2012, um bug no subsistema de timers do kernel Linux deixou processos em laço de CPU após a virada; sites grandes passaram a madrugada reiniciando servidores Java e MySQL, e a solução de emergência era acertar a hora na mão. Um único segundo extra, e o planeta inteiro de plantão.
>
> **Na vida real:** em 1º de janeiro de 2017, a Cloudflare documentou que o segundo bissexto fez o RRDNS calcular um tempo negativo e entrar em pânico ao passá-lo para `rand.Int63n`, derrubando parte das resoluções de DNS. A Go 1.9 passou a embutir uma leitura monotônica em `time.Now()`. Feliz ano-novo, e que o seu `fim - inicio` nunca dê negativo.

Resumo

- Relógios derivam (1 ppm = 86 ms/dia); o NTP erra até δ/2 e pode pular para trás. Duração: relógio monotônico.
- Aconteceu antes: ordem causal. Lamport: a → b implica L(a) < L(b). Vector clock detecta concorrência.
- Consenso: acordo, validade, terminação; FLP; algoritmos práticos são sempre seguros e progridem quando a rede colabora.

- Raft: mandatos, eleição com maioria e log atualizado, commit por maioria, `2f + 1` nós.
- Lease: lock com prazo, dependente da deriva. Fencing token: número crescente conferido pelo recurso, que recusa o dono zumbi.
- Corte por ordem de commit, nunca por carimbo de início de transação.

A Lastro agora

O `repasse_final_v2_agora_vai.py` roda em `temp-01` e `api-02` e virou agendador de sagas: só o dono do lease em `lideranca_jobs` age, com prazo de 30 s pelo relógio do banco, renovação a cada 10 s e a época conferida com `FOR SHARE` em cada transação de repasse. O ID de repasse é determinístico por vendedor e dia contábil. O ledger carimba `dia_contabil` sob advisory lock compartilhado, e o corte, exclusivo, fecha o dia pela ordem dos commits. Todas as máquinas têm NTP e alerta de deslocamento. O financeiro recuperou 377 dos 380 repasses em dobro; os outros três viraram linha nova na planilha, que continua sendo a única coisa da empresa com horário certo.

Enquanto isso, um chargeback de uma venda de setembro pede o estado exato da cobrança no dia 14, às 15h, e a Lastro só guarda o estado de agora. O Carlos sugeriu restaurar o backup do banco inteiro daquele dia, "rapidinho", para ler uma linha. Ele já reservou a sala e pediu pizza.

Referências

- Leslie Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System", Communications of the ACM, 1978. https://lamport.azurewebsites.net/pubs/time-clocks.pdf
- RFC 5905, Network Time Protocol Version 4. https://www.rfc-editor.org/rfc/rfc5905
- Colin Fidge, "Timestamps in Message-Passing Systems That Preserve the Partial Ordering", 1988; Friedemann Mattern, "Virtual Time and Global States of Distributed Systems", 1988.
- Michael Fischer, Nancy Lynch e Michael Paterson, "Impossibility of Distributed Consensus with One Faulty Process", Journal of the ACM, 1985. https://dl.acm.org/doi/10.1145/3149.214121
- Diego Ongaro e John Ousterhout, "In Search of an Understandable Consensus Algorithm", USENIX ATC 2014. https://raft.github.io/raft.pdf
- James C. Corbett et al., "Spanner: Google's Globally-Distributed Database", OSDI 2012. https://www.usenix.org/conference/osdi12/technical-

sessions/presentation/corbett

- Martin Kleppmann, "How to do distributed locking", 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Cloudflare, "How and why the leap second affected Cloudflare DNS", 2017. https://blog.cloudflare.com/how-and-why-the-leap-second-affected-cloudflare-dns/
- BIPM, Resolução 4 da 27ª CGPM (2022), sobre o uso do UTC. https://www.bipm.org/en/cgpm-2022/resolution-4
- PostgreSQL 16, Explicit Locking (advisory locks e locks de linha). https://www.postgresql.org/docs/16/explicit-locking.html
- PostgreSQL 16, Date/Time Functions (`now()` e início da transação). https://www.postgresql.org/docs/16/functions-datetime.html

CAPÍTULO 21

Event sourcing e CQRS

O comprador tem memória, a Lastro tem amnésia

A pizza do Carlos chegou antes do backup, e isso já diz tudo sobre o plano.

Um comprador do Bazar abriu um *chargeback* contra uma venda de setembro. Chargeback, ou contestação, é a devolução que o comprador pede direto ao emissor do cartão; o adquirente repassa a disputa para a Lastro, que tem um prazo para apresentar provas. No contrato com o adquirente, sete dias corridos. A Lastro gastou dois discutindo o sabor da pizza.

A venda: um notebook de R$ 2.390,00, vendedor Sabiá Eletrônicos. A alegação do comprador: "no dia 14 de setembro, às 15h, o vendedor me prometeu estorno parcial de R$ 400,00 e o estorno nunca apareceu". Para contestar, a Lastro precisa dizer o que existia naquela cobrança às 15h do dia 14. Ela consultou a tabela `cobrancas` e recebeu isto:

```
 id              | estado    | valor  | valor_estornado | versao | atualizado_em
-----------------+-----------+--------+-----------------+--------+---------------------
 0192d6e1-...    | capturada | 239000 |           40000 |      7 | 2025-09-16 10:04:11
```

Sete versões. Seis delas foram assassinadas por um `UPDATE` e enterradas sem lápide. A linha sabe que houve estorno de R$ 400,00 e sabe que mudou sete vezes. Quando, por quê e por ordem de quem, ela esqueceu. O banco guardou o resultado e jogou fora o processo, que é exatamente o contrário do que um juiz pede.

O tópico `cobrancas.eventos` e a outbox guardam sete dias (capítulos 17 e 18). E o plano do Carlos esbarra num detalhe: o backup diário é das 3h, o comprador falou das 15h, e restaurar o banco inteiro leva seis horas para ler uma linha que, às 3h, ainda nem tinha mudado.

Quem salvou a contestação foi, claro, a planilha do financeiro. Linha 4.812: "14/09 14h53 estorno Sabiá FALHOU saldo insuficiente, ver segunda". O financeiro não disse nada. Só girou o monitor.

Então a pergunta: como um sistema responde "qual era o estado disto no instante T" sem restaurar o mundo e sem depender de alguém que anota tudo numa planilha por trauma?

Event sourcing, ou parar de dar UPDATE na própria biografia

A cobrança da Lastro segue o padrão de todo CRUD: uma linha por entidade, cada mudança sobrescreve a linha, e o passado é lixo que o `UPDATE` recolhe de graça.

Event sourcing inverte isso. A fonte da verdade passa a ser a sequência de **eventos**: fatos imutáveis, no passado (`cobranca.criada`, `estorno.solicitado`). O estado atual é **derivado** aplicando os eventos em ordem:

```
estado_atual = fold(aplicar, eventos, estado_inicial)
```

`aplicar(estado, evento)` é uma função pura que recebe o estado e um evento e devolve o estado seguinte. Quem já escreveu um reducer no React escreveu exatamente isso: o reducer é `aplicar`, as actions são os eventos, e o estado da tela é o fold de tudo o que aconteceu. A diferença é que no event sourcing as actions ficam gravadas para sempre, inclusive aquela que você disparou duas vezes sem querer em 2019.

Daí sai a consulta que a Lastro não conseguia fazer: o estado no instante T é o fold dos eventos registrados até T. Sem backup, sem pizza.

Três definições sustentam o resto:

- **Agregado** (*aggregate*): a unidade de consistência. Uma cobrança é um agregado; toda regra que precisa valer de uma vez (não estornar mais do que foi capturado) vale dentro dele.
- **Stream**: a sequência de eventos de um agregado, numerada de 1 em diante. A cobrança `0192d6e1` tem um stream com versões 1, 2, 3 e assim por diante.
- **Comando** contra **evento**: o comando (`estornar 400`) é validado contra o estado atual e pode ser recusado; se aceito, gera eventos. O evento já aconteceu e não se recusa. Se `aplicar` lança exceção, o bug está no código, e o evento continua tendo acontecido, igual àquele deploy de sexta.

O evento diz o que aconteceu no domínio. `cobranca.atualizada` com o diff dos campos é o `UPDATE` fantasiado de evento: diz que `valor_estornado` foi de 0 para 40000, sem dizer se foi o marketplace, um chargeback ou o Carlos no `psql` às 2h.

Duas datas, e as duas mentem de jeitos diferentes

Todo evento carrega o momento em que foi **registrado** (o `now()` do banco) e, às vezes, o momento em que **ocorreu** no mundo (o horário que o banco parceiro informa para um Pix, que pode chegar minutos depois). As duas linhas do tempo formam o modelo

bitemporal: "o que a Lastro sabia às 15h" contra "o que tinha acontecido às 15h". Num chargeback, as respostas podem diferir, e o advogado do adquirente vai querer as duas.

A ordem dentro do stream vem da versão, nunca do relógio. Da última vez que a Lastro confiou no relógio de parede, 380 vendedores receberam duas vezes (capítulo 20).

O event store, um banco que só aceita confissão

O **event store** é o armazenamento dos streams. As operações que ele precisa oferecer são poucas e exigentes:

1. **Anexar** eventos ao fim de um stream, atomicamente (vários eventos de um comando entram juntos ou nenhum entra).
2. **Ler** um stream inteiro, ou a partir de uma versão, em ordem.
3. **Controle de concorrência otimista**: anexar com uma **versão esperada**. Se outro comando gravou no stream nesse meio-tempo, o anexo falha.
4. **Assinar** todos os eventos em uma ordem global, para quem constrói visões (as projeções, adiante).

O item 3 é o coração da coisa. Dois comandos leem a cobrança na versão 4, os dois validam "dá para estornar 400", e os dois tentam gravar a versão 5. Num PostgreSQL, a chave primária `(cobranca_id, versao)` faz o trabalho sozinha:

```
INSERT INTO eventos_cobranca (cobranca_id, versao, tipo, dados)
VALUES ('0192d6e1-7c3e-7d4e-9f3a-2b8c6d1e0f47', 5,
        'estorno.solicitado', '{"valor": 40000}');
```

```
ERROR:  duplicate key value violates unique constraint "eventos_cobranca_pkey"
DETAIL:  Key (cobranca_id, versao)=(0192d6e1-7c3e-7d4e-9f3a-2b8c6d1e0f47, 5) already
exists.
```

O perdedor relê o stream, revalida e tenta de novo, ou devolve `409`. É o controle otimista do capítulo 4, com a versão sendo a posição do próprio fato na história. O banco recusa a segunda versão da mesma verdade com a frieza de um cartório que já tem firma reconhecida.

O item 4 é onde os event stores caseiros sangram. Uma posição global `BIGSERIAL` é atribuída no INSERT, mas fica visível no COMMIT: a 1.001 pode aparecer antes da 1.000, e o leitor que avançou para 1.001 nunca vê a 1.000. É o furo de IDs da outbox do capítulo 18, e as saídas são as mesmas: ler pelo CDC, na ordem do COMMIT, ou serializar os anexos e pagar em vazão.

Existem bancos feitos só para isso (o EventStoreDB é um exemplo). E existe a tentação de sempre: o Kafka como event store. Ele resolve o item 4 com sobra e falha nos itens 2 e 3: não há anexo condicional por chave, e ler o stream de uma cobrança exige varrer a partição inteira. Com retenção de sete dias, ainda é um cartório que incinera as escrituras toda segunda.

Projeções e snapshots, ou como ler o passado sem reler a vida inteira

Uma **projeção** (*read model*) é um consumidor que lê os eventos em ordem e mantém uma estrutura pronta para uma consulta: uma tabela, um índice de busca, um contador. O `saldos_painel` do capítulo 19 é uma projeção que ainda não sabia o próprio nome. Propriedades:

- **Descartável.** Apagar e reconstruir a partir do evento 1 dá o mesmo resultado; mudar uma tela vira criar a projeção nova, alimentar desde o começo e trocar o ponteiro. É o único dado da empresa que você apaga sem avisar o jurídico.
- **Idempotente.** Guarda até onde aplicou (checkpoint) e ignora o que já viu, porque a entrega é at-least-once (capítulo 16).
- **Inline ou assíncrona.** A **inline** é atualizada na mesma transação do evento: leitura linearizável, custo na escrita. A **assíncrona** consome depois: escrita barata, leitura atrasada, e os truques de sessão do capítulo 19 voltam a ser problema seu.

Um **snapshot** é o estado de um agregado na versão N, salvo para que a leitura faça o fold só de N+1 em diante. É cache: apagável sem perda, e invalidado quando `aplicar` muda, porque carrega as regras do código velho. Só compensa em stream longo. Fold de 8 eventos custa microssegundos; snapshot para isso é contratar segurança armado para vigiar um pote de sal.

Versionar eventos, porque o passado não aceita migração

No CRUD, mudar formato é migração (capítulo 7). No event sourcing, o `estorno.solicitado` de 2025 vai existir com o formato de 2025 enquanto a empresa existir, e o seu código de 2031 vai ler isso resmungando. As técnicas, da mais barata para a mais cara:

Técnica	Como funciona	Custo
Esquema tolerante	Campo novo opcional; leitor ignora o que não conhece e trata ausente como padrão (o leitor tolerante do capítulo 11)	Quase nenhum; só serve para mudanças aditivas
Upcasting	Cada evento leva `versao_schema`; ao ler, uma função converte v1 → v2 → v3 antes de `aplicar`	Uma função por salto, para sempre, testada contra eventos reais antigos
Tipo novo	O significado mudou: `estorno.solicitado.v2` convive com o velho, e `aplicar` conhece os dois	Código que trata os dois tipos até o fim dos tempos
Copiar e transformar	Reescreve o store num novo, convertendo tudo	Quebra a imutabilidade; exige parar escritas ou rodar em paralelo; perde a prova de que o dado original era aquele

A última linha é a favorita de quem nunca explicou a um perito por que o passado mudou de formato numa terça.

E tem o dado pessoal. A LGPD (Lei 13.709/2018) prevê, em certas hipóteses, a eliminação de dados pessoais, e um log imutável com CPF dentro descumpre isso para sempre. As saídas: o evento referencia o comprador por ID e o cadastro mora numa tabela que aceita `DELETE`, ou o dado é cifrado com uma chave por titular que se apaga quando for preciso esquecer (*crypto-shredding*, KMS no capítulo 33). O evento continua lá, intacto e ilegível, igual ao código do fundador.

CQRS, a sigla que a Lastro já praticava escondida

Bertrand Meyer descreveu em 1988 o princípio CQS (*Command-Query Separation*): um método ou muda estado ou devolve dado, nunca as duas coisas. Greg Young levou a ideia do método para a arquitetura e chamou de **CQRS** (*Command Query Responsibility Segregation*): o modelo que recebe escritas é separado do modelo que serve leituras.

O modelo de escrita valida regras e mantém invariantes: normalizado, transações curtas. O modelo de leitura serve uma consulta: desnormalizado, já agregado, às vezes em outro banco. A ponte entre os dois costuma ser um fluxo de eventos.

CQRS não exige event sourcing: uma view materializada para relatório já é CQRS em miniatura. O contrário quase sempre vale, porque listar "as cobranças pendentes do Hornero" fazendo fold de cada stream é inviável sem projeção. CQRS também não exige

dois bancos; a separação é de modelos. O custo fica na leitura assíncrona, que é eventual: com CQRS, o vendedor 42 olhando um saldo que ainda não sabe do próprio saque (capítulo 19) deixa de ser acidente e vira cláusula.

A Lastro já fazia isso escondida. O `saldos_painel` é um modelo de leitura. E o ledger, desde o capítulo 1, é event sourcing de cabo a rabo: lançamentos imutáveis, saldo derivado por `SUM` (um fold com `+` no lugar de `aplicar`), estorno como fato novo, baldes de `saldos_parciais` como projeção inline. Pat Helland pôs isso num título de seção: contadores não usam borracha. O tabelião do capítulo 1 fazia event sourcing antes de o Carlos saber soletrar a expressão.

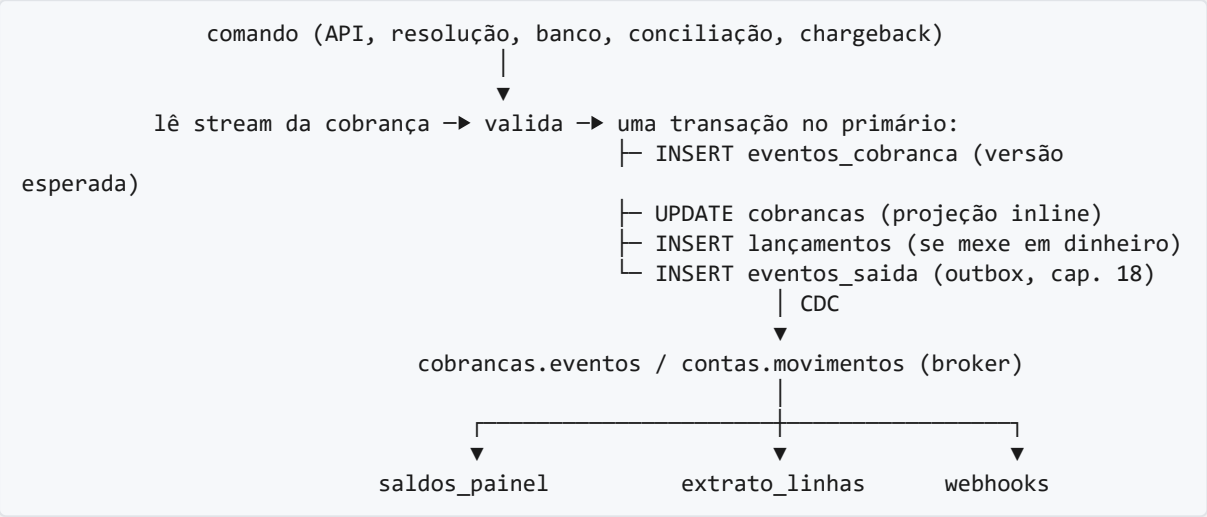
O que a Lastro guarda como evento (e o que continua sendo foto)

O requisito vem da contestação e da conciliação: para cada cobrança, o estado em qualquer instante, o fato que causou cada mudança e quem pediu (marketplace, adquirente, banco parceiro, conciliação), em segundos e com valor de evidência.

Alternativa	O que guarda	Responde "estado em T"?	Responde "por quê"?	Custo
Backup + restauração	Foto do banco às 3h	Só às 3h, em ~6 h	Não	Sala, pizza e o Carlos
Tabela de histórico por trigger	Cópia da linha a cada `UPDATE`	Sim	Não: guarda o diff, sem intenção nem autor	Baixo; o código de escrita não muda
Tabelas versionadas do SQL:2011	Igual, com sintaxe `FOR SYSTEM_TIME AS OF`	Sim	Não	O PostgreSQL não implementa; exigiria extensão
Event sourcing do agregado	Fatos com autor e causa	Sim, pelo fold	Sim	Reescrever o caminho de escrita da cobrança; versionar eventos para sempre

A tabela de histórico responde "qual era o estado" e falha na pergunta que decidiu a contestação: `valor_estornado` igual a zero antes e depois das 14h53 esconde um estorno pedido às 14h52 e recusado por saldo um minuto depois.

A Lastro adota event sourcing **para o ciclo de vida da cobrança e só para ele**:



As decisões e os motivos:

- **Event store no PostgreSQL do ledger**: `eventos_cobranca`, chave `(cobranca_id, versao)`, com `tipo`, `versao_schema`, `dados` (`jsonb`), `autor`, `registrado_em`, `ocorrido_em`. O evento entra no mesmo COMMIT dos lançamentos e da outbox; em outro sistema, volta a escrita dupla do capítulo 18.
- **`cobrancas` vira projeção inline.** A coluna `versao`, que já alimentava o `ETag` (capítulo 9) e os webhooks (capítulo 11), passa a ser a versão do stream. A leitura segue linearizável (capítulo 19), e nenhum dos 40 trechos de código que leem `cobrancas` mudou. O Carlos contou 40; a CI contou 43.
- **Eventos de domínio ficam internos.** O webhook público segue o contrato do capítulo 11; publicar o store direto congelaria o formato interno pela Lei de Hyrum.
- **Volume:** ~25 mil cobranças/dia × ~5 eventos × ~1 KB ≈ 45 GB por ano, conta que vai para o capítulo 24. O maior stream tem 31 eventos (14 estornos parciais; ninguém quer conhecer esse comprador). **Sem snapshots**: o pior fold leva menos de 1 ms.
- **Migração:** cada cobrança antiga ganha o evento sintético `cobranca.importada`, com `autor = migracao`, para nenhum perito confundir o começo do stream com o começo da cobrança.

A consulta da contestação, refeita sobre o stream, passa a ter esta cara:

```
v  registrado_em        tipo                 autor          efeito
1  10/09 19h22min04s    cobranca.criada      api:bazar      pendente
2  10/09 19h22min05s    cobranca.autorizada  adquirente     autorizada
3  10/09 19h22min05s    cobranca.capturada   api:bazar      capturada
4  14/09 14h52min10s    estorno.solicitado   api:bazar      R$ 400,00
5  14/09 14h53min02s    estorno.falhou       ledger         saldo-insuficiente
6  16/09 10h04min09s    estorno.solicitado   api:bazar      R$ 400,00
7  16/09 10h04min11s    estorno.concluido    adquirente     estornado 40000
-- fold até 14/09 15h00: capturada, estorno pedido e recusado, nada devolvido
```

O extrato do vendedor, que lia o primário (capítulos 5 e 6), vira a projeção assíncrona `extrato_linhas`: cada linha precisa mostrar a história ligada a ela (venda, estorno recusado, valor bloqueado por chargeback), e montar isso no primário junta lançamentos, cobranças e stream a cada página. O evento `conta.movimentada` (capítulo 19) passa a carregar a descrição pronta, e o consumidor grava no `pg-notificacoes`, que virou o banco de leitura da Lastro e só não mudou de nome por respeito ao crontab. Leitura com o read-your-writes do capítulo 19; reconstrução a partir dos ~60 milhões de lançamentos numa tabela paralela, ~5 mil linhas/s, ~3h20, troca de ponteiro no fim.

Continuam foto, com histórico por trigger: cadastro de vendedores, inscrições de webhook, configurações de tarifa e split. Conciliação e fechamento seguem no ledger do primário, porque o financeiro não aceita "atualizando" como saldo. Se a Lastro só precisasse de auditoria, a tabela de histórico bastaria para tudo; event sourcing se paga quando a intenção de cada mudança precisa sobreviver a ela.

Armadilhas e casos reais

> **Armadilha:** event sourcing em tudo. O cadastro do vendedor como stream de `vendedor.telefone_alterado` cobra projeção, versionamento e consistência eventual para responder uma pergunta que ninguém faz. O resultado é um CRUD com custo de banco central e utilidade de diário de adolescente.
>
> **Armadilha:** projeção que lê o relógio ou chama uma API dentro de `aplicar`. A reconstrução de amanhã dá resultado diferente da de hoje, e "descartável" morre na primeira tentativa. Tudo que a projeção precisa tem de estar no evento. E cronometre a reconstrução: 3h20 para refazer o extrato é um RTO de 3h20 para aquela tela.
>
> **Na vida real:** a LMAX, bolsa de câmbio de Londres, roda a lógica de negócio num único thread em memória, com cerca de 6 milhões de pedidos por segundo, segundo o artigo de Martin Fowler de 2011. Os eventos de entrada vão para um journal; se o processo cai, volta de um snapshot e reaplica o resto. O banco de dados virou um arquivo de log e um botão de replay.

O erro mais frequente: eventos que descrevem tabelas, como `cobranca.atualizada` e `cobranca.salva`. O time paga o preço inteiro do event sourcing e recebe de volta a amnésia do `UPDATE`, agora em JSON.

Resumo

- Event sourcing: eventos imutáveis são a fonte da verdade; estado = fold; estado em T = fold até T.
- Comando pode ser recusado; evento já aconteceu. Agregado = unidade de consistência; stream = seus eventos, numerados.
- Event store: anexo atômico, leitura por stream, versão esperada, assinatura em ordem global.
- Bitemporal: registro e ocorrência; ordem pela versão, nunca pelo relógio.
- Projeções descartáveis e idempotentes, inline ou assíncronas; snapshot é cache para stream longo.
- Versionamento: esquema tolerante, upcasting, tipo novo, copiar e transformar; dado pessoal fora do evento.
- CQRS: modelos separados de escrita e leitura, sem exigir event sourcing nem dois bancos.

A Lastro agora

O ciclo de vida da cobrança é um stream em `eventos_cobranca`, no mesmo COMMIT dos lançamentos e da outbox; `cobrancas` virou projeção inline linearizável e o extrato, a projeção assíncrona `extrato_linhas`. Cadastros e configurações seguem como tabela com histórico por trigger. A contestação do Sabiá foi respondida no sexto dia, com a planilha do financeiro anexada como "evidência complementar"; agora o próprio stream responde em 4 ms, e o financeiro, pela primeira vez desde o capítulo 1, apagou uma aba.

O Carlos comeu a pizza sozinho, porque o backup terminou às 2h. Já o gerente de produto notou que a criação de cobrança lê a tabela de tarifas e a configuração de split do marketplace onze vezes por requisição, as mesmas onze vezes, com os mesmos valores, que mudam uma vez por mês. O Carlos ouviu a palavra "Redis" numa conversa de corredor e já propôs guardar tudo lá, inclusive o saldo do saque, "porque é só um cache".

Referências

- Martin Fowler, "Event Sourcing" (2005): https://martinfowler.com/eaaDev/EventSourcing.html
- Martin Fowler, "CQRS" (2011): https://martinfowler.com/bliki/CQRS.html
- Martin Fowler, "The LMAX Architecture" (2011): https://martinfowler.com/articles/lmax.html

- Greg Young, "CQRS Documents" (2010): https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf
- Pat Helland, "Immutability Changes Everything", CIDR 2015: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper16.pdf
- Bertrand Meyer, *Object-Oriented Software Construction*, Prentice Hall, 1988 (princípio CQS).
- Lei nº 13.709/2018 (LGPD): https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm

PARTE IV

Escala

CAPÍTULO 22

Cache

A tarifa por sorteio

Enquanto o Redis não chegava, o Carlos resolveu as onze leituras de tarifa com uma linha de Python, e essa linha passou três dias cobrando cada cliente do Hornero por sorteio.

A linha era um decorador:

```
@functools.lru_cache
def tarifa_do_marketplace(marketplace_id: int) -> Tarifa: ...

tarifa_do_marketplace.cache_info()
# CacheInfo(hits=204467, misses=1, maxsize=128, currsize=1)
```

Um miss por processo, em cada um dos nove workers de temp-01, api-01 e api-02. Depois disso, cada um guardou a sua cópia da tarifa do Hornero para sempre, porque `lru_cache` não tem expiração. O primário parou de receber ~2.200 consultas por segundo no pico projetado, e o Carlos ganhou um emoji de foguete no canal.

Na segunda-feira, o comercial fechou uma tarifa Pix nova com o Mercadito Hornero: de 0,99% para 0,79%, vigente a partir da meia-noite. O banco foi atualizado. Os nove workers continuaram com 0,99% na memória, cada um com a convicção de um tio no grupo da família.

O que salvou parcialmente o Hornero foi um acidente: anos atrás, o fundador mandou reciclar cada worker a cada 50 mil requisições, para conter um vazamento de memória que ninguém achou. Os workers foram renascendo um por um, em horários aleatórios, e cada renascido cobrava 0,79%. Durante três dias, a tarifa dependeu de qual processo atendia. 6.100 cobranças saíram com 0,99%. A tarifa aplicada fica gravada na cobrança (capítulo 1), então o financeiro achou todas e devolveu a diferença em lançamentos novos, com o olhar de quem já não tem mais o que perder.

No meio do incidente, o Carlos abriu um PR chamado "cache do saldo do saque no Redis (write-through, seguro)". O PR foi fechado. O Carlos está de luto.

A pergunta: o que pode ir para o cache, por quanto tempo, como apagar o que mudou, e por que o saldo do saque nunca vai?

Por que o cache funciona, e por que ele mente

Um cache é uma cópia de um dado, guardada num lugar mais rápido ou mais barato de ler do que a origem. Ele funciona por dois fatos empíricos sobre acesso: poucas chaves concentram a maior parte das leituras, e a mesma chave tende a ser lida de novo logo. A tarifa do Hornero é lida em cada cobrança do Hornero e muda uma vez por trimestre. É o paciente ideal.

A conta que importa é a da taxa de acerto (*hit ratio*, `h`):

```
latência efetiva   = h × t_cache + (1 − h) × t_origem
carga na origem    = (1 − h) × leituras
```

A segunda linha é a que paga a infraestrutura. Subir a taxa de acerto de 90% para 99% parece uma melhoria de 9 pontos e corta a carga na origem por dez. A recíproca também vale, e é ela que derruba sistemas: um cache que cai de 99% para 90% multiplica por dez a carga num banco que foi dimensionado contando com o cache. Nesse dia, o banco descobre que trabalhava meio período.

O preço é a mentira. A cópia pode estar velha, e o cache não tem como saber, porque ninguém o avisa quando a origem muda. Ele é uma réplica sem protocolo de replicação. Pelos modelos do capítulo 19, a melhor garantia que um cache comum dá é consistência eventual com atraso limitado pelo TTL, e sem read-your-writes: você grava no banco, lê do cache e recebe o seu passado. O resto do capítulo é sobre controlar o tamanho dessa mentira.

Os padrões de leitura e escrita, ou quem tem a obrigação de mentir

Padrão	Leitura	Escrita	Risco principal
Cache-aside	App lê o cache; no miss, lê a origem e grava no cache	App grava na origem e apaga a chave	Corrida que regrava valor velho
Read-through	App lê do cache; o próprio cache busca na origem no miss	Igual ao cache-aside	O cache precisa conhecer a origem
Write-through	Do cache	Grava no cache e na origem, de forma síncrona	Escrita dupla; valor gravado que ninguém lê
Write-behind	Do cache	Grava no cache e confirma; a origem recebe depois, em lote	Perda do que estava só no cache

O **cache-aside** é o mais comum porque o cache pode ser burro: um mapa de chaves com expiração. O **read-through** move a lógica do miss para dentro da camada de cache, concentra o código e mantém as mesmas garantias.

O **write-through** é a escrita dupla do capítulo 18 com nome bonito: o banco confirma, o processo morre antes de gravar no cache, e o cache serve o valor antigo até o TTL. O **write-behind** confirma a escrita quando ela só existe na memória do cache; se o cache cai antes de descarregar, a escrita confirmada some. Para contador de visualização, aceitável. Para dinheiro, é o caminho mais curto entre a Lastro e o noticiário policial.

A corrida que regrava o passado

O cache-aside tem uma corrida clássica, e ela acontece em produção, sim:

```
leitor:   miss na chave tarifa:hornero
leitor:   SELECT tarifa → 0,99%          (lê antes da mudança)
escritor: UPDATE tarifa = 0,79%; COMMIT
escritor: DEL tarifa:hornero              (apaga o que nem existia ainda)
leitor:   SET tarifa:hornero 0,99%        (grava o passado no cache)
```

O escritor fez tudo certo e o cache ficou errado até o TTL. Sem TTL, para sempre. As defesas conhecidas:

- **TTL sempre**, como teto da mentira. Uma chave sem TTL num cache-aside é uma aposta de que nunca vai haver corrida.
- **Valor versionado**: o valor carrega a versão da origem, e a escrita no cache só substitui se a versão for maior (no Redis, um script Lua faz a comparação e o `SET` atomicamente).
- **Leases**: no miss, o cache entrega ao leitor um token; se a chave for apagada antes de o leitor gravar, o token deixa de valer e o `SET` é recusado. O Facebook descreveu esse mecanismo no artigo sobre o memcache em 2013.
- **Apagar depois do COMMIT**, nunca antes. Apagar antes abre uma janela em que outro leitor repõe o valor velho, que ainda é o valor commitado.

Invalidação, a coisa difícil que tem nome de coisa fácil

A frase atribuída a Phil Karlton diz que só há duas coisas difíceis em computação: invalidação de cache e dar nome às coisas. O Carlos conseguiu errar as duas no mesmo PR, que se chamava `cache_simples`.

As estratégias, em ordem de quanto a origem precisa colaborar:

1. **TTL puro.** Nada é invalidado; tudo expira. Simples, e o atraso máximo é o TTL inteiro. Serve quando o atraso é aceitável pelo requisito.
2. **Invalidação explícita.** Quem escreve na origem apaga a chave. Exige que todo caminho de escrita conheça o cache, inclusive o `psql` do Carlos, que nunca conheceu ninguém.
3. **Invalidação por evento.** A origem publica a mudança (CDC, outbox, `NOTIFY`) e os caches apagam ao receber. Pega todo caminho de escrita, porque sai do banco; o atraso é o do fluxo de eventos, e evento perdido vira valor velho até o TTL.
4. **Chave versionada.** A chave inclui a versão (`tarifa:hornero:v14`). Valor imutável nunca fica velho; o que muda é qual chave se procura. O problema passa a ser descobrir a versão atual, que é uma leitura pequena e quente.

Sistemas sérios combinam uma invalidação com um TTL de segurança, que ninguém planeja usar e todo mundo agradece quando usa.

E o **cache negativo**: guardar "não existe" poupa a origem de chaves inexistentes (o DNS faz isso, capítulo 8) e também guarda o "não existe" de algo criado um segundo depois. O marketplace cadastrado às 10h00 que recebe "marketplace não encontrado" até as 10h05 foi vítima de um TTL negativo de cinco minutos.

Despejo, ou quem morre quando a memória acaba

O cache tem tamanho fixo; a política de despejo (*eviction*) escolhe a vítima.

- **LRU** (*least recently used*): despeja quem foi acessado há mais tempo. Bom para recência; uma varredura única (um relatório que lê um milhão de chaves uma vez) expulsa as chaves quentes de verdade.
- **LFU** (*least frequently used*): despeja quem é menos acessado. Resiste à varredura; precisa envelhecer os contadores, senão a chave que foi popular em 2023 vive para sempre por mérito histórico.
- **Por TTL**: despeja primeiro quem expira mais cedo.

O Redis implementa LRU e LFU aproximados: sorteia algumas chaves (`maxmemory-samples`, padrão 5) e despeja a pior da amostra; o LFU usa um contador logarítmico de 8 bits com decaimento. O padrão de `maxmemory-policy` é `noeviction`: memória cheia, escrita recusada com erro. Num cache, é um jeito criativo de descobrir que ninguém configurou o despejo; num Redis com dado que não pode sumir, é o comportamento certo. As variantes `allkeys-*` escolhem entre todas as chaves; as `volatile-*`, só entre as que têm TTL.

O `lru_cache` do Carlos tem `maxsize=128` e nenhum TTL: despeja por tamanho, nunca por idade, e com dezenas de marketplaces nunca despejaria nada. A tarifa velha tinha estabilidade de servidor público.

Cache stampede, ou a manada no portão

Uma chave muito lida expira. Nos milissegundos seguintes, todas as requisições que chegam dão miss ao mesmo tempo e vão à origem ao mesmo tempo, para calcular o mesmo valor. Se o valor custa 2 segundos de banco e chegam 200 requisições por segundo, 400 cálculos idênticos disputam o banco, que fica lento, o que estende o cálculo, o que junta mais requisições. Isso é o *cache stampede* (ou *dogpile*), e ele tem o formato exato da explosão de filas do capítulo 14.

As soluções:

- **Coalescência** (*single-flight*): só um cálculo por chave; os outros esperam o resultado ou recebem o valor velho. Dentro de um processo, um `asyncio.Lock` por chave. Entre processos, um lock no próprio cache (`SET chave:lock 1 NX PX 5000`). Esse lock só evita trabalho duplicado, que é o único uso que o capítulo 2 deixou para lock no Redis: se dois calcularem, o pior que acontece é desperdício.
- **Servir velho enquanto recalcula** (*stale-while-revalidate*): entre o prazo "fresco" e o "aceitável", devolve o velho e recalcula em segundo plano. No HTTP, é a diretiva da RFC 5861.
- **Expiração antecipada probabilística**: cada leitor decide, com probabilidade que cresce perto do vencimento, recalcular antes da hora. O algoritmo XFetch (Vattani, Chierichetti e Lowenstein, VLDB 2015) faz isso sem coordenação nenhuma.
- **Jitter no TTL**: TTL de 30 s ± 20% para que mil chaves criadas juntas não expirem juntas. É o full jitter do capítulo 15, aplicado à data de morte.

Camadas, e por que o atraso soma

Camada	Latência típica	Compartilhado entre	Invalidação	Custo escondido
Memória do processo	~100 ns	um processo	só local; cada processo é uma ilha	N cópias, N misses, N versões da verdade

Camada	Latência típica	Compartilhado entre	Invalidação	Custo escondido
Cache distribuído (Redis/Valkey)	~0,2–0,5 ms (1 RTT na rede local)	todos os processos	`DEL` atinge todos	mais uma peça para operar e falhar
Cache HTTP no cliente ou proxy (capítulo 9)	0 a 1 RTT	depende de `private` / `public`	por TTL ou validação condicional	dado privado em cache compartilhado
CDN	1 RTT até a borda	todos os usuários de uma região	purga por API, segundos a minutos	servir uma página de um usuário para outro

As camadas empilham as mentiras. Um valor com TTL de 60 s no Redis, lido por um processo que guarda 60 s em memória e entrega com `max-age=60` para uma CDN, pode chegar ao usuário com até 3 minutos de atraso, e cada camada jura que respeitou o próprio TTL. Todas respeitaram. Ninguém respeitou o usuário.

O que a Lastro põe em cache

O critério vem do capítulo 19: cada leitura recebe a garantia de que precisa, e o cache só pode servir leituras que aceitam atraso. A tabela de decisões:

Dado	Leitura	Mudança	Decisão
Tarifas e configuração de split	11 por cobrança, ~2.200/s no pico	~1 vez por trimestre por marketplace	Memória do processo, com vigência
Resumo do dia no painel (GraphQL)	40 abas por marketplace, refresh 15 s	contínua	Redis, TTL 30 s, coalescência
`GET /v1/moedas`, JS do checkout	todo mundo	quase nunca	HTTP e CDN
Cobrança na API, chave de idempotência	linearizável (capítulo 19)	toda hora	Sem cache
Saldo do saque	linearizável	a cada lançamento	Nunca

Tarifas e split: vigência em vez de invalidação. A tarifa do Hornero falhou porque tinha efeito imediato. Agora toda tarifa e configuração de split tem `vigente_desde` e é imutável (versão nova em vez de `UPDATE`); a vigência começa na próxima hora cheia, com

pelo menos 10 minutos de antecedência. Cada processo guarda todas as vigências (dezenas de linhas, menos de 1 MB), com TTL de 60 s ± 20%, e escolhe a vigente pelo `now()` da transação no banco, nunca pelo relógio da máquina (capítulo 20). Com TTL de 60 s e 10 minutos de antecedência, toda versão chega a todos os processos antes de valer. O cache pode estar atrasado; o resultado nunca está errado.

Para correção urgente (vírgula no lugar errado), a gravação dispara `NOTIFY config_tarifas`, e a conexão `LISTEN` que cada processo mantém desde o capítulo 12 apaga o cache local; notificação perdida cai no TTL. A versão aplicada vai para a cobrança, e a conciliação confere. Redis aqui custaria um salto de rede de 0,3 ms em cada uma das onze leituras para guardar menos de 1 MB.

Painel: aqui entra o Redis. O resumo de vendas do dia custa ~15 ms de CPU e ~40 ms de banco, e um marketplace grande tem 40 abas abertas. Em memória de processo, seriam nove misses por marketplace a cada expiração; um Redis (ou Valkey) gerenciado, compartilhado, faz uma conta por marketplace a cada 30 s. Cache-aside, chave `painel:resumo:{marketplace}:{dia}`, TTL 30 s ± 20%, coalescência com `SET NX PX`, valor velho servido enquanto recalcula, e a tela mostra "atualizado às 14h32min10s". `allkeys-lru` com `maxmemory` fixo, porque tudo ali pode sumir. Com o Redis fora do ar, cada processo calcula com coalescência local: no máximo nove cálculos por marketplace a cada 30 s, conta feita antes de o Redis entrar.

HTTP e CDN: `GET /v1/moedas` segue com `public, max-age=86400` (capítulo 9). O JavaScript do checkout vai para uma CDN (o Amazon CloudFront e o Cloudflare são exemplos) com o hash do conteúdo no nome do arquivo e `Cache-Control: public, max-age=31536000, immutable`: versão nova é arquivo novo, e invalidação deixa de existir. Resposta com dado de cobrança continua `private`.

O saldo do saque fica fora. O write-through do Carlos é escrita dupla (capítulo 18): processo morto entre o COMMIT e o `SET` deixa o Redis com saldo antigo. O Redis replica de forma assíncrona, e um failover pode perder a última atualização confirmada. E, com tudo certo, a leitura no Redis continua sem o lock da conta e sem o snapshot da transação de débito (capítulos 2 e 3). A única fonte linearizável do saldo é o `SUM` sob lock, no primário. O painel já tem a sua cópia eventual, `saldos_painel` (capítulo 19), um cache com invalidação embutida; Redis na frente dela empilharia atraso para economizar 1 ms numa leitura de 1 ms.

Outra escolha seria a certa se as tarifas mudassem várias vezes por dia (invalidação por evento via broker) ou se o painel tivesse milhares de marketplaces pouco acessados, com acerto baixo demais para pagar o Redis.

Armadilhas e casos reais

> **Armadilha:** cache que vira dependência de capacidade sem ninguém perceber. O banco foi dimensionado com 99% de acerto, o cache reinicia vazio, e a primeira hora é de 0%. Todo cache precisa de uma resposta escrita para "e se ele acordar vazio no pico".
>
> **Na vida real:** em 23 de setembro de 2010, o Facebook ficou fora do ar por cerca de duas horas e meia. Um sistema automático trocava valores de configuração inválidos no cache pelo valor do banco; uma mudança no valor do banco foi interpretada como inválida, e todos os clientes foram ao banco ao mesmo tempo para "corrigir". Os erros do banco sobrecarregado também eram lidos como valor inválido, e o ciclo só parou com o site desligado. Um stampede com autocorreção, que corrigiu o site até ele sumir.
>
> **Na vida real:** no Natal de 2015, uma mudança na configuração de cache da Steam durante um ataque de negação de serviço fez páginas personalizadas serem guardadas e servidas para outros usuários. A Valve informou que dados de cerca de 34 mil usuários foram expostos. Resposta privada em cache compartilhado é o vazamento mais barato de produzir.

E a da própria Lastro: um cache sem TTL é um banco de dados sem backup que discorda do banco de verdade.

Resumo

- Cache é cópia sem replicação: eventual, atraso limitado pelo TTL.
- Carga na origem = (1 – h) × leituras.
- Cache-aside, read-through, write-through (escrita dupla), write-behind (perde escrita).
- Corrida do cache-aside: TTL, valor versionado, leases, apagar após o COMMIT.
- Invalidação: TTL, explícita, por evento, chave versionada; cache negativo.
- Despejo LRU, LFU, por TTL; Redis aproximado por amostragem, padrão `noeviction`.
- Stampede: coalescência, stale-while-revalidate, expiração probabilística, jitter.
- Camadas somam atraso.

A Lastro agora

Tarifas e split viraram versões imutáveis com vigência, cacheadas em cada processo (TTL 60 s, `NOTIFY` para urgência); o primário deixou de receber ~2.200 consultas por segundo no pico. Um Redis gerenciado entrou para o resumo do painel, o JavaScript do checkout foi para uma CDN, e o saldo do saque continua no primário, sob lock. O financeiro devolveu as 6.100 diferenças do Hornero e pediu que a palavra "sorteio" fosse banida do canal de incidentes.

O Carlos abriu outro PR: os relatórios mensais dos marketplaces, que rodam no primário e levam 40 segundos cada, disputando I/O com os lançamentos no dia 1º às 9h. A proposta dele é "cachear o relatório por 30 dias". O relatório é gerado uma vez por mês. A taxa de acerto prevista é zero, e o PR já tem dois aprovadores.

Referências

- Nishtala et al., "Scaling Memcache at Facebook", NSDI 2013: https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala
- Vattani, Chierichetti, Lowenstein, "Optimal Probabilistic Cache Stampede Prevention", VLDB 2015: https://www.vldb.org/pvldb/vol8/p886-vattani.pdf
- RFC 5861, HTTP Cache-Control Extensions for Stale Content: https://www.rfc-editor.org/rfc/rfc5861
- RFC 9111, HTTP Caching: https://www.rfc-editor.org/rfc/rfc9111
- Redis, "Key eviction": https://redis.io/docs/latest/develop/reference/eviction/
- Python, `functools.lru_cache`: https://docs.python.org/3/library/functools.html#functools.lru_cache
- Robert Johnson, "More Details on Today's Outage", Facebook Engineering, 23 set. 2010.
- Valve, comunicado sobre o incidente de cache da Steam em 25 dez. 2015.

CAPÍTULO 23

Replicação

O PR do Carlos para "cachear o relatório mensal por 30 dias" foi aprovado, foi para produção e, no dia 1º às 9h, atingiu a taxa de acerto prevista: zero. Os 41 relatórios dos marketplaces rodaram no primário do ledger, agora em paralelo, porque o Carlos aproveitou o PR para "acelerar a geração": 41 processos lendo um mês de lançamentos num PostgreSQL de 8 núcleos que também registrava o Pix das 9h de uma segunda-feira.

Os relatórios expulsaram do cache do banco as páginas quentes dos índices do ledger. O INSERT de lançamento, que levava 0,13 ms (capítulo 5), passou a esperar disco e chegou a 9 ms, e a posse do lock nos baldes das contas quentes (capítulo 4) foi junto. A autorização subiu de p99 1,3 s para 2,4 s, e o Bazar abriu o chamado às 9h07, com uma exclamação por relatório ainda rodando.

No meio do incidente, o fundador perguntou: "e se esse servidor morrer?". A resposta oficial era o backup das 3h, restaurado em seis horas (capítulo 21): até um dia de lançamentos perdidos e uma manhã sem Pix. O financeiro foi buscar um copo d'água, que ninguém acredita que era água.

Os dois problemas pedem mais de uma cópia do banco. O que custa mantê-las iguais, e o que acontece com o dinheiro enquanto não estão?

Primário e réplica, ou quem manda e quem copia

Replicação é manter cópias dos mesmos dados em mais de uma máquina, para escalar leitura, sobreviver à perda de uma máquina ou ler perto do usuário. Na **replicação com líder**, o **primário** aceita todas as escritas e envia as mudanças às **réplicas** (no PostgreSQL, *standbys*), que só leem. O mercado chamou isso de *master* e *slave* por décadas e trocou os nomes. A relação continua a mesma: um manda, o outro repete tudo, na mesma ordem, sem direito a opinião.

A questão técnica é o que o primário envia:

- **Comandos** (*statement-based*): o SQL. Barato, até o comando ser não determinístico. O MySQL classifica como inseguros para esse modo um `UUID()` ou

um `UPDATE ... LIMIT` sem `ORDER BY`: cada réplica gera os seus UUIDs, escolhe as suas linhas, e as cópias divergem em silêncio. Replicação com fé.

- **Log físico**: o **WAL**, o log que ele já grava para sobreviver a uma queda (capítulo 3). Descreve bytes em páginas, e a réplica vira uma cópia idêntica, página por página.
- **Log lógico**: o WAL decodificado em mudanças por linha, como o CDC do capítulo 18 já faz para a outbox.

No PostgreSQL, a física é a **streaming replication**: o `walsender` envia o WAL por TCP, o `walreceiver` grava, e o `startup` reaplica como na recuperação de uma queda. A réplica é um banco se recuperando de um crash para sempre. Com `hot_standby = on` (padrão desde a versão 10), aceita leituras durante essa convalescença eterna.

Cada posição no WAL é um **LSN** (*log sequence number*), um deslocamento em bytes escrito como `0/3A2B1C8`. É o relógio lógico do banco (capítulo 20), e a distância entre o LSN do primário e o aplicado na réplica é o atraso dela.

A cópia física leva o cluster inteiro, exige a mesma versão principal e recusa qualquer escrita. A **replicação lógica** (publication e subscription, PostgreSQL 10+) leva tabelas escolhidas, entre versões diferentes, para um banco que também escreve, mas até o PostgreSQL 17 não replica DDL nem o valor das sequências. Esqueça a sequência e, quando o destino virar primário, o próximo `INSERT` tenta o ID 1. O tabelião do capítulo 1 pede demissão.

Síncrona ou assíncrona, ou quanto tempo o COMMIT espera pela réplica

Perder dinheiro numa falha depende de quando o primário responde "COMMIT".

Na **replicação assíncrona**, o primário grava o WAL no próprio disco, responde e envia para a réplica quando der. Se morrer entre responder e enviar, a réplica promovida nunca ouviu falar da transação. O cliente tem recibo de uma transação que o banco novo desconhece: perda de dado com papel timbrado.

Na **replicação síncrona**, o primário só responde depois que pelo menos uma réplica confirma ter recebido. Ele não termina enquanto o parceiro não confirma que também chegou lá. Cada COMMIT paga uma ida e volta de rede, e o primário passa a depender da réplica para escrever.

No PostgreSQL, `synchronous_standby_names` diz quais réplicas contam:

```
synchronous_standby_names = 'ANY 1 (pg_ledger_2, pg_ledger_3)'
```

`ANY 1` é o **commit por quórum** (PostgreSQL 10+): basta uma confirmar. E `synchronous_commit`, ajustável por transação, diz o que ela confirma:

`synchronous_commit`	O COMMIT espera	Perde transação confirmada se
`off`	nada, nem o disco local (capítulo 3)	o primário cai
`local`	disco do primário	o primário cai antes de enviar
`remote_write`	réplica entregou o WAL ao sistema operacional	primário e sistema operacional da réplica caem juntos
`on`	réplica gravou o WAL no disco (fsync)	primário e réplica perdem o disco juntos
`remote_apply`	réplica aplicou: a transação já é visível lá	idem `on`

Se nenhuma réplica síncrona responde, o PostgreSQL não troca para assíncrono: o COMMIT espera para sempre. É o C do CAP (capítulo 19) sem frescura. E se você cancelar a espera:

```
WARNING:  canceling wait for synchronous replication due to user request
DETAIL:  The transaction has already committed locally, but might not
         have been replicated to the standby.
```

A transação já está no disco do primário; cancelar só a torna visível sem a garantia que você pagou para ter. "Eu fiz, só não posso provar." Com uma réplica síncrona só, cada manutenção dela congela as escritas; daí o quórum com duas.

Com a réplica a 1 ms e fsync de ~0,5 ms, o COMMIT ganha ~1,5 ms, e os locks só são liberados depois da espera: toda conta quente fica 1,5 ms mais quente.

Lag de replicação, ou a réplica que vive no passado

Mesmo na síncrona com `on`, a réplica aplica o WAL depois de gravá-lo: é o **lag de replicação**, em geral milissegundos. Mas o `startup` aplica num processo só, o primário escreve com dezenas de conexões, e qualquer rajada (backfill, `CREATE INDEX`, limpeza da outbox) vira fila. O primário escreve com 40 mãos e a réplica copia com uma.

O `pg_stat_replication` mostra o lag de cada réplica:

```
 application_name |   state   | sync_state |   flush_lag     |   replay_lag
------------------+-----------+------------+-----------------+-----------------
 pg_ledger_2      | streaming | quorum     | 00:00:00.00114  | 00:00:00.00187
```

```
 pg_ledger_3      | streaming | quorum     | 00:00:00.00121  | 00:00:00.00203
 pg_ledger_relat  | streaming | async      | 00:00:00.00098  | 00:04:12.84117
```

A terceira linha vive quatro minutos no passado, por motivos que o Carlos vai explicar.

O VACUUM do primário remove versões mortas de linha (capítulo 3) e manda a remoção pelo WAL. Se uma consulta longa na réplica ainda precisa delas, o `startup` espera, e o lag cresce, até `max_standby_streaming_delay` (30 s por padrão). Depois, executa a consulta:

```
ERROR:  canceling statement due to conflict with recovery
DETAIL:  User query might have needed to see row versions that must be removed.
```

Com `hot_standby_feedback = on`, a réplica avisa ao primário quais versões ainda usa, e o VACUUM passa a respeitar consultas de outra máquina: a consulta sobrevive e o primário incha pelo tempo que ela durar. As vítimas possíveis são a consulta, a atualidade da réplica ou o tamanho do primário. Não existe "nenhuma das anteriores", e o Carlos procurou.

Ler o que você acabou de escrever

O **read-your-writes** do capítulo 19 some com uma réplica atrasada, sem ninguém mexer no código: o vendedor saca, a tela seguinte lê na réplica, e lá o saque ainda não aconteceu. O vendedor 42 lê isso como convite. Três técnicas:

1. **Ler do primário por alguns segundos após escrever.** Falha quando o lag passa do prazo, justamente quando importa.
2. **`remote_apply` na escrita.** Vale só na réplica síncrona, com o COMMIT esperando o replay.
3. **Token de posição.** Após o COMMIT, a aplicação guarda `pg_current_wal_lsn()` do primário como posição mínima que o usuário precisa ver, e confere a réplica antes de ler:

```
SELECT pg_last_wal_replay_lsn() >= '0/3A2B1C8'::pg_lsn AS pode_ler;
-- pode_ler: f  (a réplica ainda não viu o saque; espere ou leia do primário)
```

O token também dá **leituras monotônicas**: o cliente guarda o maior LSN que já viu e recusa réplica abaixo dele. Sem isso, duas réplicas seguidas fazem o saque aparecer e sumir, e o saldo pisca como o letreiro de motel do capítulo 19.

Failover, split-brain e como aposentar um primário à força

Failover é promover uma réplica quando o primário morre. O comando (`pg_promote()`, PostgreSQL 12+) é a parte fácil. O processo:

1. **Detectar** que o primário parou, ou que a rede até ele parou, o que daqui é idêntico (capítulo 15).
2. **Escolher** a réplica com o maior LSN recebido.
3. **Promover** e **redirecionar os clientes**: DNS com TTL baixo (capítulo 8), IP virtual, proxy ou cliente com vários hosts.
4. **Impedir que o antigo volte a aceitar escrita.** O passo que derruba empresas.

Se o antigo estava só isolado, há dois primários aceitando escritas que o outro não vê: o **split-brain**. Dois ledgers legítimos, e nenhum `merge` decide qual saque vale. O banco ganha um segundo cérebro, e os dois gastam o mesmo dinheiro.

Decidir o primário é consenso (capítulo 20), e com dois nós, numa partição, cada lado acha que o outro morreu. O padrão é guardar a liderança num armazenamento com consenso (etcd, Consul ou ZooKeeper). No PostgreSQL, o exemplo de referência é o Patroni: o primário renova uma chave de líder com TTL (30 s por padrão) e, se não consegue, rebaixa a si mesmo; um watchdog reinicia a máquina se o Patroni travar. É o lease do capítulo 20: quem perde o lease se mata antes que alguém precise matá-lo. A literatura chama a última instância de STONITH, *shoot the other node in the head*. Alta disponibilidade sempre teve humor de pistoleiro.

O primário antigo volta com WAL que o novo nunca recebeu. O `pg_rewind` descarta os blocos divergentes e o religa como réplica. Se algum cliente recebeu COMMIT por aquelas transações, você perdeu dado confirmado e vai saber pela conciliação.

Multi-líder e sem líder, ou democracia na escrita

Na **replicação multi-líder**, vários nós aceitam escrita (um por região ou por dispositivo offline) e trocam mudanças de forma assíncrona. O preço são **conflitos**: duas regiões alteram a mesma linha ao mesmo tempo. *Last write wins* descarta uma escrita confirmada com base num relógio que deriva (capítulo 20); fusão exige estruturas que comutam, como CRDTs (capítulo 19); rotear cada registro sempre ao mesmo líder reinventa o líder único com mais passos. Num ledger, conflito resolvido por relógio é saque dobrado com carimbo de data.

Na **replicação sem líder** (o Dynamo da Amazon, seguido por Cassandra e Riak), o cliente escreve em várias réplicas. Com **N** réplicas por chave, a escrita vale quando **W** confirmam, e a leitura consulta **R** e fica com a versão mais nova. Se `W + R > N`, todo conjunto de leitura cruza todo conjunto de escrita: é o **quórum**. Com N = 3, W = 2 e R = 2, uma réplica pode estar fora.

Réplicas atrasadas se atualizam por **read repair** (a leitura que achou versão velha regrava a nova) e por **anti-entropia** (comparação de árvores de Merkle). Com o **quórum relaxado** do Dynamo, a escrita vai para réplicas que não são donas da chave (*hinted handoff*), e o `W + R > N` deixa de garantir interseção. E nem o quórum estrito dá linearizabilidade: escritas concorrentes resolvidas por relógio e uma escrita que falhou numa réplica e venceu em outra cabem dentro dele. É uma democracia em que todo mundo vota e cada um conta os votos do seu jeito.

Na Lastro: três cópias, um primário e nenhuma dúvida sobre quem manda

Os requisitos: nenhum lançamento confirmado some num failover; o Pix é 24x7, então a troca de primário leva segundos, e não as seis horas do backup; e os relatórios do dia 1º não tocam no primário.

Opção	Perda num failover	Custo no COMMIT	Operação
Física assíncrona	últimos ms a s	nenhum	baixa
Física síncrona, 1 réplica	zero	~1,5 ms	réplica parada congela o ledger
Física síncrona, `ANY 1` de 2	zero	~1,5 ms	3 máquinas + consenso
Lógica	últimos ms a s + sequências	nenhum	DDL à mão
Multi-líder / sem líder	conflito resolvido por relógio	baixo	sem ordem única

Multi-líder e sem líder saem porque o lock da conta, o serializable e o dia contábil dependem de uma ordem única (capítulos 2, 3 e 20); a assíncrona, pelo primeiro requisito. Fica o quórum síncrono, com as zonas (partes isoladas da região, capítulo 36) a ~1 ms:

```
            etcd (3 nós, 1 por zona) guarda a chave de líder
                 |
```

```
  zona A        |         zona B              zona C
+--------------------+  +--------------+  +--------------+
| pg-ledger-1        |  | pg-ledger-2  |  | pg-ledger-3  |
| PRIMÁRIO           |->| standby      |  | standby      |
| Patroni            |  | síncrono*    |  | síncrono*    |
+--------------------+  +--------------+  +--------------+
       |          * ANY 1: basta um confirmar
       +--> pg-ledger-relat (zona A, assíncrono, nunca promovido)
```

- **Quórum síncrono** `ANY 1` com `synchronous_commit = on`: todo COMMIT está em disco em duas zonas antes de o cliente saber dele, e uma réplica pode parar sem congelar o ledger.
- **Modo estrito** (`synchronous_mode_strict` do Patroni): sem nenhuma réplica, o Patroni não rebaixa para assíncrono, e débitos e pagamentos recebem `503`. A escolha do capítulo 19: o painel pode mentir um pouco, o ledger não.
- **Failover automático** só para réplica síncrona, em 30 a 40 s. A aplicação usa a lista de hosts do libpq com `target_session_attrs=read-write`, que procura o primário.
- **Cerca de graça**: um primário antigo isolado não confirma nada, porque nenhuma réplica responde mais a ele. O modo estrito vira o fencing token do capítulo 20, sem uma linha de código.
- **Contas quentes**: a posse do lock no balde sobe de ~2 para ~3,5 ms, e o teto por conta cai de 8.000 para ~4.500 créditos por segundo, contra ~200 da comissão do Bazar no pico (capítulo 14).
- **CDC da outbox**: no PostgreSQL 16, o slot lógico se perde no failover (recuperação do capítulo 18), e a decodificação pode publicar uma transação que nenhuma réplica confirmou: se o primário morrer nesse instante, o evento existe e o lançamento não. O PostgreSQL 17 fecha isso com `synchronized_standby_slots`; a migração para o 17, por replicação lógica, entra no planejamento, e até lá a conciliação das 6h pega o caso.

Os relatórios foram para `pg-ledger-relat`, assíncrona e marcada `nofailover`, com a configuração padrão. Aos 30 segundos, o `startup` cancelou 41 relatórios de 40 segundos; o agendador repetiu, e às 9h05 eram 380 cancelamentos, Sísifo com cron. O Carlos ligou o `hot_standby_feedback`, e o primário começou a inchar. A configuração final, só nessa réplica: `max_standby_streaming_delay = 15min`, feedback desligado, rodapé "dados até 09h03min12s" (`pg_last_xact_replay_timestamp()`) e alerta com lag acima de 20 minutos.

O painel GraphQL (capítulo 12) passou a ler nas réplicas síncronas. No segundo dia, com o replay 4 s atrasado pela limpeza da outbox, o vendedor 42 sacou R$ 300,00, "Meus repasses" veio sem o saque, e ele sacou de novo. O trigger recusou por saldo, mas o print com "saque sumiu" chegou ao Bazar antes do fim do lag.

A regra agora: depois do COMMIT de uma escrita do vendedor, a API grava o LSN em `posicao:vendedor:{id}` no Redis (TTL 5 min) e o devolve ao painel, que guarda o maior que já viu. A leitura usa uma réplica que já aplicou o maior dos dois ou, depois de 200 ms, o primário. O Redis guarda posição, nunca dado, e dá read-your-writes entre aparelhos; o painel dá as leituras monotônicas.

Em outras condições, a escolha mudaria: relatórios cruzando vários sistemas pediriam um banco analítico (capítulo 25); usuários em vários continentes, réplicas regionais com token; perda de segundos aceitável, assíncrona pura.

Armadilhas e casos reais

> **Na vida real:** em 21 de outubro de 2018, uma manutenção de rede isolou por 43 segundos o datacenter principal do GitHub na Costa Leste. O Orchestrator promoveu primários MySQL na Costa Oeste, cada lado ficou com escritas que o outro não tinha, e o GitHub passou 24 horas e 11 minutos degradado. Quarenta e três segundos de partição, um dia de conserto: a taxa de juros do split-brain.
>
> **Na vida real:** em 31 de janeiro de 2017, recriando uma réplica atrasada do PostgreSQL do GitLab.com, alguém apagou o diretório de dados do servidor errado: o primário. Cerca de 300 GB sumiram, e a restauração veio de um snapshot de seis horas antes. Réplica protege contra máquina que morre, e replica em milissegundos o comando que mata.
>
> **Armadilha:** tratar réplica como backup. Um `DELETE` sem `WHERE` chega às réplicas síncronas antes de você tirar o dedo do Enter. Dado apagado se recupera com point-in-time recovery (capítulo 40).
>
> **Armadilha:** monitorar só se a réplica está conectada. Uma réplica em `streaming` com 40 minutos de `replay_lag` está viva e inútil, como plantonista dormindo de crachá.

Resumo

- Replicação física (WAL, cópia idêntica) ou lógica (por linha, sem DDL nem sequências).
- Assíncrona perde transação confirmada no failover; síncrona paga uma ida e volta por COMMIT; quórum `ANY 1` evita travar com uma réplica fora.
- Consulta longa na réplica: cancelamento, lag ou inchaço do primário.

- Read-your-writes e leituras monotônicas com réplicas pedem token de posição (LSN).
- Failover inclui cercar o primário antigo; errar é split-brain.
- Multi-líder troca ordem por conflitos; sem líder usa `W + R > N`, que não garante linearizabilidade.

A Lastro agora

O ledger tem três cópias em três zonas, quórum síncrono, Patroni sobre etcd e failover em 30 a 40 s sem perder lançamento confirmado, a ~1,5 ms por COMMIT. Os relatórios moram em `pg-ledger-relat`; o painel lê nas réplicas síncronas, guiado pelo LSN. O PR "cachear por 30 dias" foi revertido, e o Carlos defendeu que a taxa de acerto tinha sido exatamente a prevista.

No dia 1º seguinte, com os relatórios em paz, alguém olhou o que eles liam: com 41 marketplaces, `lancamentos` tem 420 milhões de linhas, recebe 1,5 milhão por dia e, com a retenção de dez anos da Lastro, caminha para mais de cinco bilhões. O índice do extrato já não cabe na memória de nenhuma das três cópias, e o fundador, satisfeito, comentou que agora o problema estava replicado com perfeição: três bancos idênticos, igualmente grandes demais.

Referências

- PostgreSQL 16, High Availability, Load Balancing, and Replication: https://www.postgresql.org/docs/16/high-availability.html
- PostgreSQL 16, Replication (parâmetros `synchronous_standby_names`, `max_standby_streaming_delay`, `hot_standby_feedback`): https://www.postgresql.org/docs/16/runtime-config-replication.html
- PostgreSQL 16, Hot Standby (conflitos de consulta): https://www.postgresql.org/docs/16/hot-standby.html
- PostgreSQL 16, Logical Replication: https://www.postgresql.org/docs/16/logical-replication.html
- PostgreSQL 17, Logical Replication Failover: https://www.postgresql.org/docs/17/logical-replication-failover.html
- Patroni, documentação: https://patroni.readthedocs.io/
- MySQL 8.0, Determination of Safe and Unsafe Statements in Binary Logging: https://dev.mysql.com/doc/refman/8.0/en/replication-rbr-safe-unsafe.html
- DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007: https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf

- GitHub, "October 21 post-incident analysis", 2018: https://github.blog/2018-10-30-oct21-post-incident-analysis/
- GitLab, "Postmortem of database outage of January 31", 2017: https://about.gitlab.com/blog/2017/02/10/postmortem-of-database-outage-of-january-31/

CAPÍTULO 24

Particionamento e sharding

A Capinhas Hiper pediu um estorno de R$ 19,90 numa terça às 15h, e o PostgreSQL levou 1,9 segundo para descobrir se ela tinha R$ 19,90. O trigger de débito (capítulos 2 e 3) soma os lançamentos da conta para conferir o saldo, e a Capinhas, a maior vendedora do Bazar, acumula 9 milhões de lançamentos. O `SUM` é index-only, bem planejado, impecável, e lê 9 milhões de entradas de índice com o lock da conta na mão. Cada estorno dela faz fila atrás do anterior. Ser o melhor cliente da Lastro virou uma punição com complexidade O(n).

O resto do ledger vai pelo mesmo caminho. `lancamentos` tem 420 milhões de linhas e recebe 1,5 milhão por dia. O índice do extrato não cabe mais na memória de nenhuma das três cópias (capítulo 23). O VACUUM da tabela leva horas, e cada página que ele congela vira WAL para três réplicas. Um índice novo, mesmo com `CONCURRENTLY` (capítulo 7), seria uma maratona de 420 milhões de linhas.

As propostas chegaram rápido. O fundador quer "uma máquina maior". O Carlos quer apagar os lançamentos com mais de um ano "e guardar num arquivo", o tabelião do capítulo 1 recusou o `DELETE` antes de ele terminar a frase, e o financeiro teve que se sentar.

Como se divide uma tabela que não pode perder uma linha, e o que acontece com uma transferência quando as duas contas moram em máquinas diferentes?

Particionar sem sair da máquina

Particionamento é dividir uma tabela em pedaços menores, as **partições**, dentro do mesmo banco. Para a aplicação, continua uma tabela só: o `INSERT` vai para `lancamentos` e o banco põe a linha na partição certa. O PostgreSQL faz isso de forma declarativa (versão 10+), por três critérios:

- **Faixa** (`RANGE`): um intervalo de valores por partição. O caso clássico é tempo: uma partição por mês.
- **Lista** (`LIST`): valores enumerados, como uma partição por moeda ou por região.

- **Hash** (`HASH`): o resto da divisão do hash da chave, para espalhar linhas sem ordem natural.

```
CREATE TABLE lancamentos (...) PARTITION BY RANGE (criado_em);
CREATE TABLE lancamentos_2027_01 PARTITION OF lancamentos
    FOR VALUES FROM ('2027-01-01') TO ('2027-02-01');
-- o limite superior é exclusivo: meia-noite do dia 1º já é do mês seguinte
```

O ganho vem de três mecanismos. O **partition pruning**: uma consulta com `criado_em` no `WHERE` só toca as partições que podem conter a resposta, decidido no planejamento ou, com parâmetros, na execução. A **manutenção local**: VACUUM, `REINDEX` e estatísticas rodam por partição, e uma partição de três anos atrás, congelada uma vez, nunca mais dá trabalho. E a **retenção barata**: `DETACH PARTITION` tira um mês inteiro da tabela mexendo só no catálogo, sem gerar uma linha morta. É o `DELETE` de 45 milhões de linhas que custa o mesmo que renomear uma variável.

As regras que mordem. Cada partição tem os próprios índices, e não existe índice global, então `PRIMARY KEY` e `UNIQUE` precisam incluir a chave de partição: o `id` sozinho deixa de ser garantidamente único, e a chave vira `(id, criado_em)`. Consulta sem a chave de partição no filtro visita todas as partições, uma por uma, e milhares de partições custam tempo de planejamento e memória por sessão (a documentação dedica uma seção de boas práticas a isso). E `CREATE INDEX CONCURRENTLY` não funciona na tabela particionada: cria-se o índice com `ON ONLY` na tabela-mãe, inválido, depois um índice concorrente em cada partição, e cada um é anexado com `ALTER INDEX ... ATTACH PARTITION`. Quando a última partição se anexa, o índice da mãe fica válido sozinho. Um ritual com mais etapas que despacho de importação, mas sem lock de escrita.

Tudo isso continua numa máquina só. Particionar organiza o problema; não aumenta o disco, a memória nem a vazão de escrita do servidor.

Sharding, ou como transformar um banco em vários problemas independentes

Sharding é particionar entre máquinas: cada **shard** é um banco independente, com seu primário e suas réplicas, guardando uma fração das linhas. A escrita escala com o número de shards, e cada um tem memória, disco e WAL próprios. O preço é tudo o que o banco fazia por você entre linhas que agora moram em lugares diferentes: `JOIN`, transação, unicidade, sequência, consistência de leitura. O PostgreSQL de cada shard não sabe que os outros existem. Você ganhou dez bancos e perdeu o único que sabia de tudo.

Alguém precisa saber em qual shard está cada chave. As estratégias:

Por faixa: cada shard guarda um intervalo da chave (marketplaces de 1 a 1000 no shard 1, de 1001 a 2000 no shard 2). Consultas por intervalo continuam baratas. O defeito é a chave crescente: com faixas de tempo ou IDs sequenciais, toda escrita nova cai no último shard, e os outros viram museu. Você comprou dez máquinas e uma trabalha.

Por hash: `shard = hash(chave) mod N`. Distribuição uniforme, consultas por intervalo viram consulta a todos. E o `mod N` esconde uma bomba: ao passar de N para N + 1 shards, quase toda chave muda de lugar. Uma chave fica onde está só se `h mod N = h mod (N + 1)` ; de 10 para 11 shards, cerca de 91% dos dados atravessam a rede. Crescer 10% de capacidade custa uma mudança de casa quase inteira.

Hashing consistente (Karger et al., 1997) resolve isso. Chaves e nós são mapeados no mesmo anel de hashes, e cada chave pertence ao primeiro nó encontrado no sentido horário. Um nó novo assume só o arco entre ele e o antecessor: em média, só 1/N das chaves se move. Com poucos nós, os arcos saem desiguais, e um nó pega metade do anel pela mesma sorte que escolhe o plantonista do Natal. Por isso cada nó físico ocupa centenas de posições no anel, os **nós virtuais**, e a carga se espalha. Cassandra usa essa ideia.

Shards lógicos fixos: a chave vai para um de muitos baldes fixos (milhares), e uma tabela pequena diz em que máquina mora cada balde. O Redis Cluster usa 16.384 *hash slots*. Rebalancear vira mover baldes inteiros e atualizar a tabela, e nenhuma chave muda de balde. Escolha um número de baldes muito maior do que o número de máquinas que você algum dia terá, porque esse número não muda.

Diretório: uma tabela de roteamento mapeia cada chave (ou grupo de chaves) para um shard, com a liberdade de pôr qualquer chave em qualquer lugar. O custo é uma consulta a mais por requisição, que vira cache, e um componente que, se cair, deixa o sistema inteiro sem saber onde está nada.

Hot spots, ou o hash que espalha as chaves e não a carga

O hash distribui chaves de forma uniforme. A carga por chave, não. Se uma chave recebe 30% do tráfego, o shard dela recebe pelo menos 30%, e não existe função de hash que divida uma chave só. Isso é um **hot spot**: a conta de comissão do capítulo 4, agora com endereço de entrega.

As saídas são as mesmas do capítulo 4, em escala de máquina. Dividir a chave quente em subchaves com sufixo (`comissao#0` a `comissao#15`), que caem em shards diferentes e se somam na leitura: os baldes do capítulo 4, finalmente promovidos a infraestrutura. Ou isolar o cliente gigante num shard dedicado, o que o diretório permite e o hash puro não.

Entre shards: consultas, transações e mudanças de casa

Consulta que não traz a chave de sharding vira **scatter-gather**: a pergunta vai a todos os shards, cada um responde com o seu pedaço, e a aplicação junta. A latência é a do shard mais lento, e a cauda se multiplica: com 20 shards, cada um com p99 de 50 ms, a chance de uma consulta pegar ao menos um p99 é `1 - 0,99^20`, cerca de 18% (capítulo 14). O "quanto vendemos hoje?" passa a esperar o shard que, naquele segundo, está no meio de um VACUUM. Ordenação e paginação globais pedem que cada shard mande o seu topo e alguém faça o merge, que é o `ORDER BY` do banco reescrito à mão, com bugs novos.

Transação entre shards é transação distribuída (capítulo 18). O 2PC funciona e segura locks durante a rede inteira, travando se o coordenador morrer no meio. A saga troca atomicidade por estados intermediários visíveis e compensações. Unicidade global também sai do banco: um `UNIQUE` vale só dentro do shard, então IDs precisam ser globalmente únicos por construção, como os UUIDv7 que a Lastro usa desde o capítulo 1.

Rebalancear é mover dados entre shards com o sistema no ar: copiar o snapshot do balde, acompanhar as mudanças que chegam durante a cópia (replicação lógica, capítulo 23), bloquear escritas daquele balde por segundos, aplicar o resto, atualizar o roteamento e liberar. Cliente com roteamento em cache antigo escreve no lugar errado, então o shard de origem precisa recusar escrita de balde que já não é dele. O fencing token do capítulo 20, de novo, porque todo problema distribuído acaba sendo o mesmo problema com outra roupa.

E a **chave de sharding**: a que mantém cada transação inteira dentro de um shard, aparece em quase toda consulta e espalha a carga. As três quase nunca vêm juntas, e a ordem de prioridade decide a arquitetura.

Na Lastro: o tempo particiona, o marketplace fragmenta

Particionamento por mês, já

`lancamentos` passa a ser particionada por faixa mensal de `criado_em`. O extrato e a conciliação filtram por período e são podados; VACUUM e índices passam a trabalhar no mês corrente, cerca de 45 milhões de linhas e 2,3 GB de índice do extrato, que cabem no cache. A chave primária vira `(id, criado_em)`; nada referencia `lancamentos` por FK, então ninguém sente falta do `id` sozinho, já que o `BIGINT` identity continua único por construção.

A migração dos 420 milhões de linhas não move nenhuma linha. A tabela atual vira `lancamentos_legado`, com um `CHECK (criado_em < '2026-12-01')` criado `NOT VALID` e validado sem lock de escrita (capítulo 7) e o índice único `(id, criado_em)` criado

concorrentemente. Num deploy, sob `lock_timeout`, a tabela é renomeada, a tabela particionada nasce com o nome antigo e a legada é anexada como a partição `[MINVALUE, '2026-12-01')`. O `CHECK` válido prova a restrição da partição, e o `ATTACH` pula a varredura. Segundos de lock, contra uma cópia de 420 milhões de linhas. Os triggers do tabelião e do débito são recriados na tabela particionada e valem para todas as partições. Um job cria as partições com três meses de antecedência e alerta se faltar a do mês seguinte: lançamento sem partição é `INSERT` recusado, e Pix recusado à meia-noite de virada de mês é o tipo de incidente que vira lenda.

O saldo deixa de somar a história inteira. A cada mês encerrado, um job grava `saldos_fechados (conta_id, mes, saldo)`. O mês está encerrado quando nenhuma transação aberta começou nele: como `criado_em` é o `now()` do início da transação (capítulo 20), o job espera o menor `xact_start` das sessões de cliente em `pg_stat_activity` passar da virada antes de somar. Lançamentos são imutáveis, então o saldo fechado nunca invalida. O trigger de débito passa a somar o último saldo fechado com os lançamentos do mês corrente, e o estorno da Capinhas cai de 1,9 s para 4 ms. É o snapshot do capítulo 21: no event sourcing, snapshot era cache; aqui é cache que nunca envelhece, porque o passado do ledger não muda.

Retenção: dez anos (política da Lastro). Partição com mais que isso sai com `DETACH`, é exportada para armazenamento de arquivo e conferida pela soma antes de apagada. O Carlos ganhou o "guardar num arquivo" dele, com dez anos de atraso.

Sharding: a chave escolhida, a máquina ainda não

Hoje o primário aguenta: o pico da Black Friday foi de ~800 lançamentos por segundo (200 pagamentos de quatro lançamentos, capítulo 14), bem abaixo do que o teste de carga mediu. O sharding entra quando a vazão de escrita do primário passar de 50% da capacidade medida em teste de carga ou o mês corrente deixar de caber no cache, e o capítulo 29 vai fazer essa conta com o marketplace dez vezes maior. A chave, porém, é escolhida agora, porque toda tabela nova do ledger precisa nascer com ela.

Chave	Transação de pagamento	Saldo de uma conta	Carga	Problema
`conta_id` (hash)	4 contas em até 4 shards	1 shard	uniforme	toda venda vira transação distribuída
`transacao_id` (hash)	1 shard	todos os shards	uniforme	todo saldo vira scatter-gather
`criado_em` (faixa)	1 shard	todos os shards	tudo no último	hot spot permanente

Chave	Transação de pagamento	Saldo de uma conta	Carga	Problema
`marketplace_id` (diretório)	1 shard	1 shard	desigual	contas globais e marketplace gigante

A Lastro escolhe `marketplace_id`, com diretório. Um pagamento com split toca o vendedor, a comissão do marketplace, o liquidante e a receita da Lastro (capítulo 3), e os dois primeiros já pertencem ao marketplace. Os dois últimos são globais, e são resolvidos como o capítulo 4 resolveu as contas quentes: cada shard tem a sua subconta de liquidante e de receita, e o saldo global é a soma das subcontas, conferida pela conciliação contra o extrato do banco liquidante. Assim, todo pagamento, estorno e saque é uma transação local de um shard, com o lock da conta, o serializable e a soma zero do ledger funcionando como hoje.

O diretório, em vez de hash, porque são 41 marketplaces com tamanhos que variam mais de mil vezes: o Bazar e o gigante do capítulo 29 ganham shard próprio, e os pequenos dividem. Hashing consistente seria a escolha com milhões de chaves de tamanho parecido; aqui ele espalharia 41 chaves de forma uniforme e a carga de forma nenhuma. O diretório fica no primário de configuração, em cache em cada processo como as tarifas (capítulo 22), e mudar um marketplace de shard segue o rebalanceamento acima, com época no diretório como fencing.

A transferência entre shards

O gerente de produto trouxe o primeiro caso: um grupo que é dono de dois marketplaces quer mover saldo de um para o outro. "É só um botão." O botão atravessa shards.

```
shard A (marketplace origem)          shard B (marketplace destino)
T1: débito conta origem      ---->    T2: crédito conta destino
    crédito transito_saida_A              débito transito_entrada_B
    transferência = 'enviada'             (referência = id transferência, UNIQUE)
falha definitiva em T2 -> C1 no shard A: estorno de T1, 'falhou'
```

É a saga do capítulo 18, orquestrada pelo mesmo agendador com lease (capítulo 20). Cada shard continua com soma zero em cada transação, porque o dinheiro passa pelas contas de trânsito, e T2 é idempotente pela referência única (capítulo 16). Entre T1 e T2, o dinheiro está visivelmente em trânsito, e a conciliação confere que a soma das contas de trânsito de todos os shards é zero ao fim do dia. O 2PC ficou de fora: travaria a conta de origem durante uma ida e volta entre shards e, com o coordenador morto, deixaria o lock pendurado até alguém resolver à mão (capítulo 18).

Armadilhas e casos reais

> **Na vida real:** a Notion fragmentou o PostgreSQL em 2021 pelo ID do workspace, com 480 shards lógicos distribuídos em 32 bancos físicos. O número 480 foi escolhido por ter muitos divisores, o que permite redistribuir os shards lógicos entre várias quantidades de máquinas sem sobra. Toda tabela com dados de workspace ganhou a chave de partição, e a migração teve escrita dupla, backfill e verificação antes da troca. A matemática do número de baldes é a parte que ninguém pode mudar depois.
>
> **Armadilha:** chave de partição ausente da consulta. Um relatório sem `criado_em` no `WHERE` visita 120 partições; uma consulta sem `marketplace_id` vira scatter-gather em todos os shards. O plano parece correto e a conta chega na latência.
>
> **Armadilha:** esquecer de criar a partição do mês seguinte. O PostgreSQL recusa a linha sem destino com `no partition of relation "lancamentos" found for row`, e a virada do ano é a data favorita desse erro, porque o job de partições do Carlos só olhava o mês.
>
> **Armadilha:** fragmentar cedo. Cada shard é mais um primário com réplicas, failover e backup, e cada consulta global vira código. Particionamento resolve tamanho; sharding resolve vazão de escrita. Sharding para resolver tamanho é comprar dez apartamentos porque o armário encheu.

Resumo

- Particionamento divide a tabela dentro do banco (faixa, lista, hash): pruning, manutenção por partição e retenção com `DETACH`; PK e UNIQUE incluem a chave de partição.
- Sharding divide entre máquinas e escala a escrita, perdendo `JOIN`, transação e unicidade entre shards.
- Faixa preserva intervalos e concentra chaves crescentes; hash `mod N` move quase tudo ao crescer; hashing consistente move ~1/N; shards lógicos e diretório movem unidades inteiras.
- Hash espalha chaves, não carga: hot spot pede subchaves ou shard dedicado.
- Entre shards: scatter-gather com cauda multiplicada, saga ou 2PC, IDs únicos por construção, rebalanceamento com cópia, replicação e fencing.
- A chave de sharding mantém a transação num shard, aparece nas consultas e espalha a carga.

A Lastro agora

`lancamentos` é particionada por mês, com a legada anexada sem cópia, partições criadas com três meses de antecedência e retenção de dez anos por `DETACH`. O saldo soma `saldos_fechados` com o mês corrente, e o estorno da Capinhas voltou a 4 ms. O sharding ainda não existe, mas a chave sim: `marketplace_id` por diretório, subcontas de liquidante e receita por shard, transferência entre shards como saga com contas de trânsito. O tabelião aprovou tudo, porque nenhuma linha foi apagada, e cobrou um carimbo por partição.

Com os relatórios na réplica e o ledger organizado, sobrou tempo para o antifraude. O Vigia avisou que precisa de contadores por cartão e por dispositivo em milissegundos, de um grafo de contas ligadas para achar fraude em anel e de agregações que nenhum índice B-tree entrega. O Carlos já criou a tabela `grafo_de_fraude (de, para)` no ledger e escreveu um `WITH RECURSIVE` que, segundo ele, "termina".

Referências

- PostgreSQL 16, Table Partitioning: https://www.postgresql.org/docs/16/ddl-partitioning.html
- PostgreSQL 16, `ALTER TABLE` (ATTACH e DETACH PARTITION): https://www.postgresql.org/docs/16/sql-altertable.html
- PostgreSQL 16, `CREATE INDEX` (índices em tabelas particionadas): https://www.postgresql.org/docs/16/sql-createindex.html
- Karger et al., "Consistent Hashing and Random Trees", STOC 1997: https://dl.acm.org/doi/10.1145/258533.258660
- Redis, Cluster specification (hash slots): https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
- Notion, "Herding elephants: lessons learned from sharding Postgres at Notion", 2021: https://www.notion.so/blog/sharding-postgres-at-notion

CAPÍTULO 25

Modelos não relacionais

O Carlos tinha razão: a consulta dele termina. Termina aos 8 segundos, quando o deadline da autorização (capítulo 15) cancela a coitada, e termina três vezes por segundo, porque estava ligada em 5% das autorizações de cartão por uma feature flag chamada `sinal_anel_beta`.

O Vigia lançou um `Avaliar` (capítulo 12) que aceita "sinais" calculados pela Lastro, que vê 41 marketplaces: tentativas do cartão em 10 minutos, cartões distintos por dispositivo, ligação do comprador com um anel que já tomou chargeback. O Carlos criou `grafo_de_fraude (de, para)` no primário do ledger, com arestas sem tipo entre compradores, cartões, dispositivos e IPs, e um `WITH RECURSIVE` de até quatro saltos.

Numa quarta às 15h20, no pico de 60 autorizações por segundo da campanha de aniversário do Bazar, um comprador entrou pelo IP de um CGNAT de operadora móvel, o NAT do capítulo 8 em escala industrial, compartilhado por 11 mil celulares. Do comprador ao IP, do IP a 11 mil contas, delas aos cartões, dos cartões aos dispositivos: 40 milhões de caminhos, 30 GB de arquivos temporários. Cada consulta segurava uma das 2 conexões do pool do worker (capítulo 14) por 8 segundos. Lei de Little: 3 por segundo × 8 s = 24 conexões, contra 18 no total. Às 15h22 o Pix parou de confirmar pagamento; às 15h23 o Bazar abriu chamado "URGENTE!!!!" com o corpo "vocês estão investigando a gente?".

Contadores em milissegundos, um grafo e agregações sobre anos de dados: qual modelo de dados responde cada pergunta, e por que o ledger fica exatamente onde está?

O padrão de acesso escolhe o modelo, nunca o contrário

O modelo relacional começa pelas entidades. Você normaliza (capítulo 1), cria as tabelas, escreve as consultas depois, e o planejador (capítulo 6) responde qualquer pergunta que o SQL expresse, inclusive as imprevistas. O preço são junções na leitura e um único formato de armazenamento, linhas, para tudo.

Os não relacionais invertem: primeiro as perguntas, com frequência e latência, e os dados gravados já no formato da resposta. Pergunta fora da lista fica entre cara e impossível. A modelagem atende o relatório que o gerente pediu e quebra no que ele vai pedir depois.

Cada família abaixo vem com modelo, operações, garantias e o que não faz, que é a parte que o vendedor do banco esquece.

Chave-valor

Um `dict` do Python distribuído: chave opaca, valor, e operações de ler, gravar e apagar por chave em tempo constante. "Todas as chaves cujo valor passa de 100" exige varrer tudo.

O Redis (ou o Valkey) acrescenta estruturas ao valor, e aí ele fica útil para contar. `INCR` incrementa atomicamente, porque o servidor executa comandos numa thread única. `EXPIRE` faz a chave sumir sozinha. O HyperLogLog conta distintos em no máximo 12 KB por chave, com erro padrão de 0,81%: ótimo para contar cartões, péssimo para contar centavos.

```
127.0.0.1:6379> INCR sinal:cartao:c9f2:tentativas:202611041520
(integer) 7
127.0.0.1:6379> EXPIRE sinal:cartao:c9f2:tentativas:202611041520 7200
(integer) 1
127.0.0.1:6379> PFADD sinal:disp:a17e:cartoes:2026110415 c9f2 b011 77d3
(integer) 1
127.0.0.1:6379> PFCOUNT sinal:disp:a17e:cartoes:2026110415
(integer) 3
```

Sétima tentativa do mesmo cartão no mesmo minuto. Nem a estagiária da Capinhas compra capinha sete vezes por minuto.

As garantias são a parte mórbida. O Redis guarda tudo em memória e persiste por snapshot (RDB) ou por log de comandos (AOF); com `appendfsync everysec`, uma queda perde até cerca de um segundo de escritas. A replicação é assíncrona: o failover promove uma réplica que pode não ter os últimos incrementos, e o `WAIT` só conta quantas réplicas confirmaram, sem tornar nada linearizável. Chave-valor gerenciado (o Amazon DynamoDB é um exemplo) traz durabilidade replicada e escrita condicional, e cobra em modelagem: toda consulta precisa caber na chave.

Chave-valor não faz consulta por valor, junção nem agregação, e não garante que o contador sobreviva ao failover com o número certo. Contador de fraude errado por três unidades durante 30 segundos é tolerável. Saldo errado por três unidades durante 30 segundos é um processo com o nome do vendedor 42.

Documento

Cada documento é uma árvore (o MongoDB guarda em BSON, até 16 MB), identificada por chave e consultável por campos internos, com índices secundários. A unidade de modelagem é o agregado (capítulo 21): o que é lido e escrito junto mora junto, embutido.

O pedido com seus itens vira um documento, e a tela faz uma leitura em vez de quatro junções.

O schema continua existindo, só que no código que lê ("schema-on-read"), que precisa entender os documentos de 2019, os de 2022 e o que o Carlos gravou "rapidinho" com o campo `vlaor`.

A atomicidade natural é por documento. Transações com vários documentos chegaram no MongoDB 4.0 (2018, replica sets) e 4.2 (sharded), mais caras e com limite de duração. Junções existem (`$lookup`) e são a confissão de que o dado foi modelado para outra pergunta. Documento sofre com muitos-para-muitos e com invariantes que atravessam documentos, como "a soma de todos os lançamentos é zero".

Colunar, que são duas coisas com o mesmo nome

"Colunar" nomeia duas famílias quase sem nada em comum, e metade das reuniões sobre o tema são duas pessoas concordando sobre coisas diferentes.

A primeira é a **wide-column** (Bigtable, Cassandra, ScyllaDB). A chave de partição escolhe o nó (hash, capítulo 24), e dentro da partição as linhas ficam ordenadas pelas colunas de clustering. A consulta eficiente é "linhas da partição X entre A e B", leitura sequencial num nó só. O armazenamento é uma LSM-tree: a escrita vai para um log e para uma tabela em memória (memtable), despejada em arquivos imutáveis ordenados (SSTables) e fundida em segundo plano (compaction). Escrever é barato, porque nada é reescrito no lugar; ler pode consultar vários SSTables; apagar é gravar uma lápide (*tombstone*). O Cassandra aborta a leitura que encontra mais de 100.000 lápides (`tombstone_failure_threshold`): o banco tem cemitério com lotação máxima. A consistência é escolhida por operação, com quórum (capítulo 23), e conflito se resolve por timestamp, o last-write-wins que o capítulo 20 já autopsiou.

A segunda é o **armazenamento colunar analítico** (ClickHouse, Parquet, DuckDB, data warehouses gerenciados). Continua sendo tabela com SQL; muda o disco. Cada coluna é gravada separada e comprimida: `SELECT avg(valor)` lê só `valor`, e uma coluna de estados com quatro valores repetidos 90 milhões de vezes comprime para quase nada com dicionário e run-length. O ClickHouse ordena a tabela por uma chave e guarda uma entrada de índice a cada 8.192 linhas (`index_granularity`): um BRIN (capítulo 5) que faz academia.

O colunar analítico não busca linha por ID com pressa, não atualiza linha (vira reescrita de bloco ou mutação assíncrona) e não gosta de inserção unitária. Ele quer lotes de milhares de linhas. Inserir uma por vez no ClickHouse é crime contra a compressão.

Grafo

O grafo de propriedades tem nós e arestas, ambos com tipo e atributos. A pergunta natural é a travessia: "a partir deste nó, siga arestas destes tipos até N saltos". Bancos nativos de grafo (o Neo4j, com a linguagem Cypher, é o exemplo clássico; o Amazon Neptune é um gerenciado) guardam cada nó com ponteiros diretos para suas arestas, a *index-free adjacency*: seguir uma aresta custa um acesso a registro, sem descer B-tree. A travessia custa proporcionalmente ao trecho visitado, independente do tamanho total.

No relacional, cada salto é uma junção, uma descida de índice O(log n). Com 9 milhões de arestas a B-tree tem três níveis, e a diferença por salto é pequena. A diferença grande é de expressão: o banco de grafos escreve "caminho de tamanho variável com tipos alternados" numa linha, e o SQL escreve num `WITH RECURSIVE` que o Carlos fez sem detecção de ciclo.

```
WITH RECURSIVE anel(no, saltos) AS (
    SELECT para, 1 FROM grafo_de_fraude WHERE de = 'comprador:91822'
  UNION ALL
    SELECT g.para, a.saltos + 1
    FROM anel a JOIN grafo_de_fraude g ON g.de = a.no
    WHERE a.saltos < 4          -- "termina" (Carlos, 2026)
)
SELECT count(*) FROM anel;
```

O limite de saltos garante o término; não garante que ele aconteça nesta encarnação. O `UNION ALL` revisita o mesmo nó por todos os caminhos, e o primeiro supernó (o IP do CGNAT, com 11 mil vizinhos) multiplica tudo por 11 mil. Desde o PostgreSQL 14, a cláusula `CYCLE` do padrão SQL marca caminhos que repetem nó: resolve o ciclo, ignora o supernó. Banco de grafos sofre igual, porque 11 mil arestas custam 11 mil acessos em qualquer motor. Supernó se trata como dado ruim e sai da travessia.

Grafo vai mal em agregação sobre o conjunto inteiro e em particionamento. Cortar um grafo em shards sem cortar as travessias é NP-difícil, e você tem reunião às 16h.

Séries temporais

O modelo é (série, instante, valor), e a série é um nome com rótulos (`aprovacoes{marketplace="bazar", metodo="cartao"}`). O acesso é inserção no fim, leitura por janela, agregação por intervalo e descarte por retenção. Bancos de séries temporais (Prometheus, InfluxDB, o TimescaleDB como extensão do PostgreSQL) particionam por tempo (o capítulo 24 no automático), apagam blocos inteiros e fazem *downsampling*: depois de 30 dias, um ponto por minuto vira um por hora.

A compressão aproveita a regularidade. O Gorilla, do Facebook (VLDB 2015), grava o delta do delta dos timestamps (pontos a cada 15 s dão zero, que cabe em um bit) e o XOR de cada valor com o anterior: 1,37 byte por ponto, contra 16 sem compressão.

O limite é a cardinalidade: cada combinação de rótulos é uma série nova, com índice próprio. `cobranca_id` como rótulo cria 25 mil séries por dia, de um ponto cada, e o banco morre com a memória cheia delas. É o jeito mais caro de reinventar uma tabela.

Modelo	Acesso que faz bem	Consistência típica	O que não faz
Chave-valor	ler, gravar, incrementar por chave; TTL	por chave; réplica assíncrona (Redis) ou configurável	consulta por valor, junção, agregação
Documento	ler e gravar o agregado inteiro	atômica por documento; multi-documento mais cara	muitos-para-muitos, invariantes entre documentos
Wide-column	faixa ordenada dentro de uma partição; escrita em volume	quórum por operação; last-write-wins	junção, consulta fora da chave, apagar barato
Colunar analítico	varrer e agregar poucas colunas de muitas linhas	lotes; sem transação de linha	busca pontual, UPDATE, inserção unitária
Grafo	travessia de tamanho variável	depende do motor (o Neo4j é ACID numa instância)	agregação global, sharding
Séries temporais	janela de tempo, agregação, retenção	inserção no fim; pontos tardios custam	cardinalidade alta de rótulos

Na Lastro: três perguntas, três modelos, um ledger intocado

Primeira decisão: nada disso mora no primário do ledger. O Carlos desligou a flag às 15h24 e pediu para registrar o horário; `grafo_de_fraude` saiu do `pg-ledger-1` às 15h31.

Contadores: Redis separado, com baldes de minuto

O orçamento do capítulo 14 dá 20 ms para validação e primeira transação, e os sinais cabem ali ou não existem. São oito contadores por autorização: tentativas e valor por cartão em 10 minutos, 1 hora e 24 horas, mais cartões e contas distintos por dispositivo em 24 horas, a 60 autorizações por segundo no pico. O cartão entra por um identificador derivado, nunca pelo número (o dado de cartão é assunto do capítulo 33).

Alternativa	Latência por autorização	Carga no ledger	Garantia
`count` / `sum` em `cobrancas` no primário	5–30 ms, 8 consultas	+480 consultas/s no pico	exata
Mesma consulta na réplica síncrona	5–30 ms + lag	nenhuma no primário	exata com atraso
Redis, baldes por minuto/hora	~1 ms, 1 ida e volta em pipeline	nenhuma	aproximada; perde segundos no failover

A Lastro escolheu o Redis porque o sinal precisa de ~1 ms e tolera erro pequeno: o Vigia decide com dezenas de variáveis, e três tentativas a menos durante um failover mudam uma probabilidade, enquanto o saldo continua intocado no ledger. Os contadores ficam numa instância **separada** do cache do painel (capítulo 22), `redis-sinais`, com `noeviction`: contador de fraude despejado por LRU para abrir espaço para resumo de painel é a pior troca do mercado. Toda chave tem TTL, e a memória fica limitada pelo próprio esquema, cerca de 2 GB para 24 horas de baldes.

A janela deslizante é aproximada por baldes: 1 hora soma 60 baldes de minuto (um `MGET`), 24 horas soma 24 baldes de hora, e distintos usam HyperLogLog por hora, unidos por `PFCOUNT` com várias chaves. Leitura e incremento da tentativa atual vão no mesmo pipeline, antes do Vigia, e contam as tentativas recusadas, que são justamente as que importam.

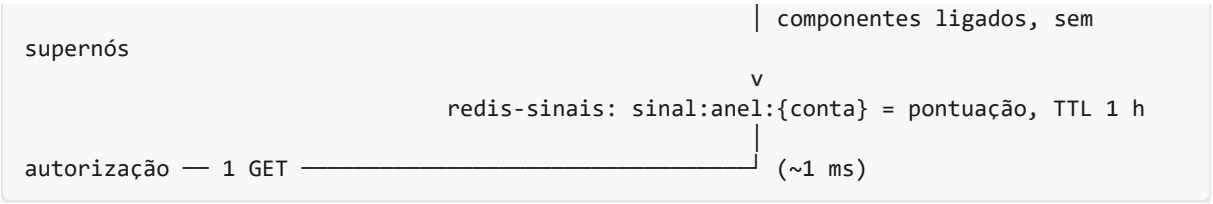
Se o `redis-sinais` cair, os sinais vão ausentes no `Avaliar` (campos `optional`, capítulo 12) e a regra local do capítulo 15 segue valendo, com o limite por cartão em 24 horas lido no PostgreSQL. A degradação de lá previa o Vigia fora; agora prevê o Vigia cego de um olho.

O anel: grafo fora do caminho da autorização

O grafo entrou por um requisito exploratório: a analista de risco, contratada depois do chargeback do capítulo 21, pergunta coisas como "contas a até 4 saltos que compartilham dispositivo ou conta de repasse com quem tomou chargeback", com tipos de aresta novos toda semana. São ~2 milhões de nós (compradores, vendedores, cartões, dispositivos, contas de repasse) e 9 milhões de arestas.

O fluxo:





A autorização nunca faz travessia: lê uma pontuação pronta por chave, e o grafo responde com paciência de analista. Nós com mais de 500 arestas (o CGNAT, o Wi-Fi do shopping, o celular de demonstração da loja) viram atributo e saem da travessia. O supernó deixou de ser vizinho e virou fofoca.

O custo é mais um sistema para operar e até 5 minutos de atraso na pontuação. Uma tabela de arestas tipadas no `pg-notificacoes`, com `CYCLE` e corte de supernó, seria a escolha certa se as perguntas fossem fixas e de até dois saltos, sem sistema novo. A Lastro escolheu o banco de grafos porque as perguntas mudam toda semana e a analista escreve Cypher sem pedir PR a ninguém.

Agregações: colunar alimentado pelo log

A terceira pergunta é o relatório semanal de risco: taxa de chargeback por vendedor, faixa de valor, BIN e hora do dia, sobre dois anos de cobranças. A `pg-ledger-relat` (capítulo 23) leva 90 segundos lendo linhas inteiras para usar cinco colunas. Um banco colunar analítico (o ClickHouse é um exemplo; Parquet num armazenamento de objetos com um motor de consulta também serve) recebe `cobrancas.eventos` em lotes de 10 segundos, ordenados por `(marketplace_id, criado_em)`, e responde em menos de 1 segundo. Como UPDATE ali é caro, o chargeback vira linha nova e a consulta pega a versão mais recente de cada cobrança: o event sourcing do capítulo 21, agora num banco que gosta dele.

Séries temporais ficam para o capítulo 38, com as métricas. O gerente de produto registrou a ausência delas como "atraso no roadmap".

Por que o ledger continua relacional

O ledger precisa exatamente do que essas famílias abrem mão: transação atômica sobre várias linhas (o split do capítulo 3), restrições no banco (soma zero, FK, o tabelião), serializable para débitos e consulta ad hoc para a conciliação. Num banco de documentos, a transação com seus lançamentos caberia num documento atômico, mas o saldo soma lançamentos de milhões de documentos, e a verificação do capítulo 2 vira transação multi-documento. No relacional, "não perder um centavo" é restrição verificada pelo banco; fora dele, vira esperança da aplicação.

Armadilhas e casos reais

> **Armadilha:** usar o Redis do cache como banco de contadores. Com `allkeys-lru` (capítulo 22), ele apaga contadores sob pressão de memória, sem erro, e o cartão que tentou 40 vezes aparece com 3. O ataque passou porque o resumo do painel do Hornero precisava de espaço.
>
> **Na vida real:** o Discord guardava mensagens no MongoDB e migrou para o Cassandra em 2017, quando dados e índices deixaram de caber em memória. A chave de partição virou `(channel_id, bucket)`, com o bucket como janela fixa de tempo, para nenhuma partição crescer sem limite. A lição do capítulo 24, num banco sem `ATTACH PARTITION`.
>
> **Na vida real:** a análise do Jepsen sobre o MongoDB 4.2.6 (2020) encontrou anomalias nas transações multi-documento, inclusive com os níveis mais fortes de read concern e write concern. Garantia se lê na especificação e se confere em teste; a página de marketing segue um modelo de consistência próprio, chamado otimismo.
>
> **Armadilha:** HyperLogLog onde se exige número exato. Erro de 0,81% em cartões distintos por dispositivo é ruído; em contagem de cobranças para faturar tarifa, é o financeiro processando o Redis.

Resumo

- Modelagem não relacional parte do padrão de acesso; pergunta não prevista fica cara.
- Chave-valor: operações por chave, contadores atômicos, TTL; o Redis perde escritas no failover.
- Documento: agregado embutido, atomicidade por documento; sofre com muitos-para-muitos.
- Wide-column: partição, clustering, LSM-tree, lápides, quórum. Colunar analítico: colunas comprimidas, varredura, lotes.
- Grafo: travessia custa o trecho visitado; supernó é problema em qualquer motor.
- Séries temporais: partição por tempo, retenção, downsampling; o limite é a cardinalidade.
- Dinheiro precisa de transação entre linhas e restrição no banco: fica no relacional.

A Lastro agora

Os sinais do Vigia saem do `redis-sinais` (separado, `noeviction`, TTL em tudo) em ~1 ms: oito contadores em baldes e HyperLogLog para distintos. O grafo mora num banco de grafos alimentado pelos tópicos, sem supernós, e só a pontuação de anel chega à autorização, a cada 5 minutos. O relatório de risco roda num colunar em menos de 1 segundo. O ledger, o tabelião e a planilha do financeiro continuam relacionais, cada um à sua maneira.

A consulta do Carlos foi preservada num comentário com a anotação "termina", e ele segue tecnicamente correto, que é o tipo de correto que derruba o Pix. A Lastro agora é um monólito que fala com dois PostgreSQL, dois Redis, um log, um banco de grafos e um colunar. O deploy da semana que vem é o relatório novo de risco, que sobe no mesmo processo que autoriza cartão. O que pode dar errado num deploy de relatório?

Referências

- Redis, documentação de persistência (RDB e AOF): https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- Redis, HyperLogLog: https://redis.io/docs/latest/develop/data-types/probabilistic/hyperloglogs/
- PostgreSQL 16, `WITH` queries (`SEARCH` e `CYCLE`): https://www.postgresql.org/docs/16/queries-with.html
- MongoDB, Transactions: https://www.mongodb.com/docs/manual/core/transactions/
- Chang et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006: https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/
- O'Neil et al., "The Log-Structured Merge-Tree (LSM-Tree)", Acta Informatica, 1996: https://doi.org/10.1007/s002360050048
- Pelkonen et al., "Gorilla: A Fast, Scalable, In-Memory Time Series Database", VLDB 2015: https://www.vldb.org/pvldb/vol8/p1816-teller.pdf
- ClickHouse, MergeTree: https://clickhouse.com/docs/engines/table-engines/mergetree-family/mergetree
- Discord, "How Discord Stores Billions of Messages", 2017: https://discord.com/blog/how-discord-stores-billions-of-messages
- Jepsen, "MongoDB 4.2.6", 2020: https://jepsen.io/analyses/mongodb-4.2.6

CAPÍTULO 26

Decomposição de sistemas

O relatório de risco subiu numa quinta às 9h30 e às 10h05 derrubou a autorização de cartão, o que é uma conquista para um relatório: ele nem tinha permissão para autorizar nada.

A analista de risco clicou em "exportar 2 anos". O endpoint novo, no mesmo monólito que autoriza cartão, puxou 14 milhões de linhas do colunar (capítulo 25) para um DataFrame dentro do worker. Foram 40 segundos de CPU no event loop, o pecado exato que o capítulo 13 proibiu, com o RSS do processo indo de 160 MB a 7 GB. O OOM killer do Linux encerrou o worker, e com ele as 212 autorizações em andamento naquele processo, que viraram `processando` e foram resolvidas pela consulta ao adquirente (capítulo 15). A analista achou que tinha travado e clicou mais duas vezes. O painel interno espalhou os cliques pelas três máquinas, e cada uma perdeu um worker do mesmo jeito: um terço da capacidade de autorização evaporou em 90 segundos, por um CSV.

O Bazar abriu chamado às 10h09. O assunto era "URGENTE!!!!" e o corpo era "quem é essa analista".

A correção óbvia foi o rollback. O deploy das 9h30 levava 14 mudanças de cinco pessoas, entre elas a correção de um bug no Pix que o Hornero esperava havia uma semana. O rollback levou o relatório embora e o conserto do Pix junto. O Hornero recebeu o bug de volta às 10h20, sem ter pedido.

Por que um relatório pode matar uma autorização, e como se divide um sistema para que isso não aconteça sem criar uma dúzia de problemas novos?

Três formas de empacotar o mesmo código

As três arquiteturas se distinguem por três fronteiras: a de deploy (o que sobe junto), a de processo (o que morre junto) e a de dados (quem pode escrever em qual tabela).

Um **monólito** tem uma unidade de deploy, um processo (replicado) e um banco. Chamar outro módulo é uma chamada de função: nanossegundos, mesma transação, mesma stack trace. Refatorar uma fronteira é um PR. O custo é o destino compartilhado:

qualquer código pode consumir a memória, a CPU e o pool de conexões de qualquer outro, e todo mundo sobe e desce no mesmo deploy.

Um **monólito modular** mantém o deploy e o processo únicos, e impõe fronteiras dentro do código. Cada módulo expõe uma interface pública e esconde o resto. No Python, ferramentas de análise de imports (o import-linter é um exemplo) quebram a CI quando `pagamentos` importa algo interno de `ledger`. Cada módulo é dono das suas tabelas, de preferência num schema próprio do PostgreSQL, e nenhum outro faz `SELECT` nelas. Resolve o acoplamento de código; não resolve o destino compartilhado. O DataFrame de 7 GB do relatório mata o worker do mesmo jeito, só que agora com imports organizados.

Microsserviços separam as três fronteiras: cada serviço tem deploy, processo e banco próprios, e fala com os outros pela rede. O relatório pode morrer de OOM à vontade, sozinho, num processo que só serve relatório. O preço é tudo o que o resto deste capítulo descreve.

```
                  deploy      processo     dados        chamada entre partes
monólito          1           1            1 banco      função (ns)
monólito modular  1           1            schemas      função, via interface
microsserviços    N           N            N bancos     rede (ms) + falha parcial
```

Existe uma quarta forma, que ninguém escolhe e muita gente constrói: o **monólito distribuído**. Serviços separados que precisam subir juntos, compartilham o mesmo banco e se chamam em cadeia síncrona. Ele tem a latência dos microsserviços, o acoplamento do monólito e o orçamento de infraestrutura dos dois. É o pior dos mundos com SLA de cada um deles.

Acoplamento, coesão e a pergunta que decide a fronteira

Acoplamento é o quanto uma mudança numa parte obriga mudança em outra. Ele aparece em formas diferentes, e uma divisão em serviços elimina algumas e cria outras:

Tipo	Exemplo	O que a separação em serviços faz
De código	`import` de função interna	elimina (vira contrato de API)
De dados	dois módulos lendo a mesma tabela	elimina só se cada serviço tiver banco próprio
Temporal	A só responde se B responder agora	piora: vira dependência de rede

Tipo	Exemplo	O que a separação em serviços faz
De deploy	A e B precisam subir juntos	elimina se os contratos forem compatíveis (capítulo 11)
Semântico	os dois precisam concordar sobre o que é "pago"	não muda nada, só muda o lugar da briga

Coesão é o quanto as coisas dentro de uma parte mudam pelos mesmos motivos. A heurística útil para achar fronteira é olhar o histórico do git: arquivos que mudam no mesmo commit, repetidamente, pertencem ao mesmo lugar. Separar em dois serviços duas coisas que sempre mudam juntas transforma cada PR em dois PRs, dois deploys e uma ordem de deploy, e você vai errar a ordem numa sexta-feira.

A fronteira mais barata segue três critérios técnicos: dados que precisam de consistência transacional entre si ficam do mesmo lado; perfis de recurso muito diferentes (CPU pesada, memória grande, latência apertada) ficam de lados diferentes; e requisitos de disponibilidade diferentes ficam de lados diferentes, para que o menos crítico não arraste o mais crítico. O relatório da quinta-feira violou os dois últimos ao mesmo tempo, o que é um tipo de eficiência.

Um banco por serviço, e o que você perde com ele

Dois serviços que leem a mesma tabela formam um serviço só, com dois deploys e o dobro de plantão. O schema da tabela vira o contrato entre eles, um contrato sem versão, sem changelog e com a Lei de Hyrum (capítulo 11) em vigor: qualquer coluna, qualquer índice, até a ordem implícita das linhas, alguém usa. Renomear uma coluna exige deploy coordenado, que é exatamente o que a separação queria evitar.

Banco por serviço significa que só o dono escreve e lê suas tabelas; os outros perguntam pela API ou recebem eventos. Pode ser um servidor por serviço ou um servidor com bancos e credenciais separados; a fronteira é a permissão, a máquina é decisão de custo.

O que se perde, e onde o livro já tratou de cada perda:

- **JOIN entre domínios.** Vira composição na aplicação (duas chamadas e uma junção em memória) ou projeção local alimentada por eventos (capítulos 19 e 21), com consistência eventual.
- **Transação entre domínios.** Vira saga com outbox (capítulo 18). Atomicidade dá lugar a compensação, e a conciliação vira a verificação final.
- **Integridade referencial.** A FK de `cobrancas.vendedor_id` para `vendedores` deixa de existir. O serviço guarda o ID e confia que ele existia quando foi gravado.

O critério que sobra é direto: se duas entidades precisam de uma restrição verificada no mesmo COMMIT, elas moram no mesmo banco, e portanto no mesmo serviço. Separar e depois reconstruir a atomicidade por saga custa uma saga para cada invariante, e você tem invariantes demais para isso.

Quanto custa trocar uma função por uma requisição

Uma chamada de função em Python custa dezenas de nanossegundos. Uma chamada a outro serviço na mesma zona custa um RTT de rede (~0,5 ms, pelos números de referência do capítulo 14), mais serialização e desserialização do payload, mais TLS se houver, mais a fila do servidor do outro lado. Na prática, entre 1 e 5 ms no p50 e muito mais na cauda.

Uma chamada dessas se aguenta. O estrago vem da multiplicação. Uma interface fina ("me dá a tarifa", "me dá o split", "me dá o vendedor", onze vezes por cobrança, capítulo 22) que custava 11 chamadas de função passa a custar 11 requisições em série: 11 a 55 ms, onze timeouts para escolher (capítulo 15), onze pontos de falha parcial e onze lugares para o Carlos colocar `@retry(5)`. A regra que sai do mecanismo: interface entre serviços é grossa. Uma chamada carrega tudo o que o chamador precisa para aquela operação.

E a chamada pode simplesmente não voltar. Toda a Parte III deste livro existe porque uma chamada de rede tem um terceiro resultado além de sucesso e erro. Um monólito tem um problema de falha parcial por parceiro externo. Um sistema com seis serviços tem esse problema em cada aresta do grafo de chamadas.

Disponibilidade composta, ou por que cinco noves de vizinhos dão menos que um

Se uma requisição precisa que os componentes A e B respondam, e eles falham de forma independente, a disponibilidade do caminho é o produto:

```
série:     A = A1 × A2 × ... × An
paralelo:  A = 1 − (1 − A1) × (1 − A2) × ... × (1 − An)
```

Cinco serviços em série, cada um com 99,9% (43,8 minutos fora por mês), dão $0{,}999^5 \approx 99{,}50\%$: cerca de 219 minutos fora por mês. Cada serviço novo no caminho síncrono pega o seu orçamento de falha e soma ao dos outros. Você não comprou cinco componentes com 99,9%; comprou um caminho com 99,5%, e pagou cinco faturas.

Duas réplicas de um componente com 99% em paralelo dão 1 − 0,01² = 99,99%, desde que falhem de forma independente. E aí está a letra miúda: a fórmula supõe independência. Réplicas na mesma zona, com o mesmo deploy e o mesmo bug, falham juntas, e o produto vira ficção otimista. O incidente desta quinta é um exemplo perfeito: três workers "redundantes", em três máquinas, morreram pela mesma causa, no mesmo minuto e meio, clicada pela mesma pessoa.

Há duas saídas da multiplicação. A primeira é tirar componentes do caminho síncrono: o que pode acontecer depois (notificar, contabilizar, atualizar projeção) sai por um log ou fila e deixa de entrar no produto. A segunda é degradação definida (capítulo 15): um componente com fallback deixa de ser série, porque a falha dele não derruba a requisição.

Lei de Conway, a observação que ninguém revogou

Em 1968, Melvin Conway publicou que organizações que projetam sistemas tendem a produzir projetos que copiam a estrutura de comunicação da própria organização. É uma observação empírica; estudos posteriores sobre a chamada hipótese do espelhamento encontraram evidência compatível com ela em projetos de software, sem transformá-la em lei física. A consequência técnica é mensurável: interfaces entre partes mantidas pelas mesmas pessoas ficam informais e acopladas (todo mundo sabe o que o outro módulo faz, então ninguém documenta), e interfaces entre partes de grupos diferentes ficam formais, versionadas e mais lentas de mudar.

Para a arquitetura, isso significa que uma fronteira de serviço que ninguém mantém de forma separada tende a voltar a acoplar com o tempo. Seis serviços mantidos pelas mesmas poucas pessoas viram, em dois anos, um monólito distribuído com seis repositórios e o mesmo Carlos em todos eles.

Na Lastro: a fronteira segue a transação

A decisão saiu de três requisitos: a autorização não pode compartilhar destino com cargas de outro perfil (o incidente); um deploy não pode levar mudanças de domínios alheios junto (o Pix do Hornero); e o ledger precisa continuar com transações atômicas e restrições no banco (capítulos 1 a 3).

Serviço	Dono dos dados	Perfil	No caminho síncrono da autorização?
pagamentos	cobranças, `eventos_cobranca`, chaves de idempotência, outbox (`pg-pagamentos`)	I/O, latência apertada	sim

Serviço	Dono dos dados	Perfil	No caminho síncrono da autorização?
antifraude	sinais (`redis-sinais`), grafo, chamada ao Vigia	I/O, latência apertada	sim, com degradação
ledger	contas, lançamentos, saldos (`pg-ledger`)	escrita transacional	não
repasses	sagas de repasse, lease do job	lote, chamadas ao banco do Pix	não
notificações	inscrições, entregas (`pg-notificacoes`)	I/O, volume alto	não
relatórios	nada; lê o colunar e a `pg-ledger-relat`	CPU e memória pesadas	não

A mudança mais séria é a fronteira entre pagamentos e ledger, e ela muda uma decisão do capítulo 21. Lá, `eventos_cobranca` e os lançamentos iam no mesmo COMMIT do primário do ledger. Agora, pagamentos grava o evento e a outbox no `pg-pagamentos`, e o ledger consome `cobrancas.eventos` e registra a transação contábil com `referencia` UNIQUE (capítulo 16): at-least-once mais deduplicação, efeito único. O motivo é a disponibilidade: com o ledger fora do caminho síncrono, a autorização deixa de depender dele. O custo é uma janela de milissegundos a segundos em que a cobrança está paga e o saldo ainda não. A conciliação ganhou uma classe nova, "cobrança paga sem transação no ledger após 5 minutos", e o alerta de atraso do consumidor fica em 30 segundos.

O débito continua síncrono e transacional. Repasses chama a API do ledger para debitar, e o lock, o serializable e o trigger dos capítulos 2 e 3 continuam dentro de um banco só. O ledger é o único serviço que escreve em `pg-ledger`, e o tabelião agora tem um cliente só, o que ele considera uma promoção.

A conta do caminho da autorização, com disponibilidades por componente assumidas como hipótese de projeto (o SLO formal fica para o capítulo 39):

```
                      proxy   pagam.  banco   antifr. sinais  ledger  pg-ledger adquir.
tudo síncrono         .9999 × .9995 × .9995 × .999 × .999 × .9995 × .9995 × .999  ≈
99,49%  (~223 min/mês)
ledger assíncrono,
antifraude degradável .9999 × .9995 × .9995 ×  -   ×  -   ×   -   ×   -    × .999  ≈
99,79%  (~92 min/mês)
```

A segunda linha vale para cobranças de até R$ 300,00; acima disso, o antifraude fora ainda recusa (capítulo 15) e volta ao produto. O monólito de hoje (proxy, monólito, banco, adquirente) também dá ~99,79%. A divisão bem feita preserva a disponibilidade

do caminho e ganha isolamento; a divisão ingênua, com todo mundo chamando todo mundo, dobraria os minutos fora. A diferença entre as duas linhas é uma decisão sobre o que pode acontecer depois.

Pagamentos chama o antifraude uma vez por autorização, com interface grossa: `avaliar` recebe a cobrança e devolve a decisão, e o antifraude calcula os sinais e chama o Vigia por dentro. O deadline (capítulo 15) é propagado, e o antifraude responde com a degradação do capítulo 15 se estourar. Um salto de rede a mais, ~1 ms, sai da folga de 120 ms do orçamento do capítulo 14.

Mais processos significa mais conexões. O capítulo 4 adiou o PgBouncer para cá, e ele entra em modo transaction na frente de `pg-ledger` e `pg-pagamentos`. As conexões de `LISTEN` (capítulo 12) continuam diretas, porque `LISTEN` precisa de sessão, e os advisory locks do dia contábil (capítulo 20) ficam no nível da transação (`pg_advisory_xact_lock_shared`), que o modo transaction suporta; um advisory lock de sessão sobreviveria ao COMMIT preso a uma conexão que o PgBouncer já entregou a outro cliente.

A ordem da migração é a do risco: primeiro o monólito vira modular, com schemas por módulo e import-linter na CI; depois saem relatórios (sem dados próprios, risco zero), notificações (já tinha banco próprio desde o capítulo 17), antifraude e repasses; a separação dos bancos de pagamentos e ledger vai por último, com replicação lógica e fencing (capítulo 24). Cada serviço roda em processos próprios nas mesmas três máquinas, por enquanto, e o relatório tem limite de memória, então morre sozinho.

A alternativa que fica registrada: se a Lastro tivesse um único perfil de carga e um deploy por semana sem incidentes cruzados, o monólito modular resolveria o acoplamento de código sem pagar rede, sagas e PgBouncer. Ele é o primeiro passo da migração justamente porque continua sendo uma arquitetura válida se o resto parar no meio.

Armadilhas e casos reais

> **Armadilha:** dividir por entidade em vez de por transação. Um "serviço de vendedores", um "serviço de contas" e um "serviço de saldos" separados fazem do saque uma saga de três passos, e o vendedor 42 vai achar a janela entre o segundo e o terceiro antes do monitoramento.
>
> **Armadilha:** o serviço que só repassa a chamada. Um "serviço de tarifas" que recebe a requisição, chama o banco e devolve o resultado adiciona um salto de rede, um fator no produto da disponibilidade e um plantão, em troca de nada. A tarifa continua em cache dentro de pagamentos (capítulo 22), versionada.

> **Na vida real:** em 2023, a equipe do Prime Video publicou que juntou num único processo uma ferramenta de monitoramento de qualidade de vídeo construída antes com componentes distribuídos (funções serverless e uma máquina de estados), porque o custo de passar cada quadro entre componentes pela rede e por armazenamento intermediário dominava o sistema; o relato fala em redução de custo de infraestrutura acima de 90%. O caso mostra o custo de uma fronteira posta onde o dado trafega em volume, e diz pouco sobre microsserviços em geral, que é como a internet resolveu ler.
>
> **Armadilha:** relógios, IDs e configuração "compartilhados" por cópia. Seis serviços copiando a tabela de tarifas em seis caches, com seis TTLs diferentes, fazem o sorteio de tarifas do capítulo 22 renascer em escala. Configuração tem um dono, e os outros recebem eventos dele.

Resumo

- Monólito, monólito modular e microsserviços diferem nas fronteiras de deploy, de processo e de dados.
- Acoplamento tem tipos (código, dados, temporal, deploy, semântico); serviços eliminam alguns e pioram o temporal.
- Banco por serviço: o schema deixa de ser contrato; JOIN vira composição ou projeção, transação vira saga, FK desaparece.
- Chamada de rede custa milissegundos e falha parcial; interfaces entre serviços são grossas.
- Disponibilidade em série é o produto; em paralelo, 1 – produto das indisponibilidades; ambas supõem independência.
- Assíncrono e degradação tiram componentes do produto.
- Lei de Conway: observação empírica de que a estrutura do sistema acompanha a de comunicação de quem o mantém.

A Lastro agora

A Lastro tem seis serviços (pagamentos, antifraude, ledger, repasses, notificações, relatórios), cada um com deploy próprio e dono único dos seus dados. O ledger saiu do caminho da autorização: consome os eventos de cobrança com referência única e só atende débitos síncronos de repasses. O caminho da autorização segue em ~99,79% na

conta de projeto, o PgBouncer finalmente chegou, e o relatório de risco morre agora num processo com limite de memória, sem levar ninguém junto. O Hornero recebeu a correção do Pix num deploy só dele, com uma mudança, de uma pessoa.

Falta um detalhe. Seis serviços em três máquinas, cada um com várias réplicas, e por enquanto cada chamador tem uma lista de IPs e portas num arquivo de configuração que o Carlos mantém à mão, "porque são só seis". Na primeira vez que uma réplica de pagamentos morrer, o antifraude vai continuar mandando requisições para ela com a fidelidade de um cachorro esperando na estação.

Referências

- Conway, M. E., "How Do Committees Invent?", Datamation, abril de 1968: https://www.melconway.com/Home/Committees_Paper.html
- import-linter, documentação: https://import-linter.readthedocs.io/
- PgBouncer, documentação (modos de pool e recursos suportados): https://www.pgbouncer.org/features.html
- Prime Video Tech, "Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%", 2023: https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90

CAPÍTULO 27

Balanceamento e proxies

O processo do antifraude em `api-02` morreu numa quarta às 14h10, e durante 47 minutos o serviço de pagamentos continuou mandando um terço das avaliações para ele, com a lealdade de quem não lê obituário.

A causa da morte foi um segfault, assunto de outro postmortem. O supervisor tentou reiniciar cinco vezes em dez segundos e desistiu, que é o comportamento configurado e também o do Carlos às sextas. Cada processo de pagamentos lia, na partida, a variável que o Carlos mantém à mão desde o capítulo 26:

```
ANTIFRAUDE_HOSTS=10.0.1.10:7100,10.0.1.11:7100,10.0.1.12:7100
# não mexa. são só seis serviços. (Carlos)
```

O cliente fazia round robin na lista. Uma chamada em cada três ia para `10.0.1.12:7100`, onde ninguém escutava. O kernel de `api-02`, educadíssimo, respondia RST em menos de um milissegundo, e o pagamentos caía na degradação do capítulo 15: até R$ 300,00, regras locais; acima, `recusada` com `antifraude_indisponivel`.

O circuit breaker ficou fechado o tempo todo. Ele abre com mais de 50% de falhas, e a falha era de 33%, uma taxa de erro desenhada com precisão para passar por baixo do radar. Foram 1.128 cartões recusados. O Bazar abriu chamado às 14h31, assunto "URGENTE!!!!", corpo "vocês têm alguma coisa contra notebook?". O Carlos tirou o IP da variável às 14h57 e reiniciou os nove processos de pagamentos, um a um.

Quem decide para onde vai cada requisição, e como esse alguém percebe que uma réplica morreu sem concluir, por conta própria, que morreram todas?

Camada 4 e camada 7, ou quanto da conversa o balanceador tem coragem de ler

Um **balanceador de carga** recebe tráfego endereçado a um serviço e escolhe, para cada unidade de trabalho, uma das instâncias que o atendem (o *backend*). Qual unidade ele enxerga depende da camada (capítulo 8).

Um **balanceador de camada 4** decide por conexão TCP (ou fluxo UDP), olhando só a quíntupla. Pode reescrever o destino dos pacotes (NAT, com a resposta voltando por ele), entregar o pacote e deixar o backend responder direto ao cliente (DSR, útil quando a resposta é cem vezes maior que a pergunta) ou terminar a conexão e abrir outra até o backend (proxy TCP). Ele não lê HTTP nem precisa das chaves TLS, e segura milhões de conexões com pouca CPU. O preço: balanceia conexões, sem saber o que é uma requisição. Com HTTP/2 e gRPC, uma conexão carrega tudo, e o capítulo 12 já mostrou o resultado: uma réplica afogada e as outras de folga, recebendo salário.

No modo proxy TCP, o backend vê o IP do balanceador e o cliente some do log. O **PROXY protocol**, criado no HAProxy, devolve a origem: antes do primeiro byte do cliente, o balanceador escreve uma linha de texto (versão 1) ou um cabeçalho binário (versão 2):

```
PROXY TCP4 198.51.100.22 203.0.113.50 51234 443\r\n
<aqui começa o ClientHello do TLS, intocado>
```

Backend que aceita esse prefixo de qualquer origem deixa qualquer um na internet escolher o próprio IP, o que é ótimo para quem quer culpar outra pessoa pelo ataque.

Um **balanceador de camada 7** termina a conexão e o TLS, lê a requisição e escolhe um backend **por requisição**, com conexões próprias até cada instância. Roteia por `Host`, caminho ou cabeçalho, repete o que falhou ao conectar, impõe timeout por rota e balanceia cada stream HTTP/2. Custa CPU, a chave privada guardada nele e um parser HTTP a mais, capaz de entender a requisição diferente do backend (o *request smuggling* do capítulo 9). O L4 deixa passar sem perguntar nada; o L7 abre a mochila, lê o diário, corrige a ortografia e só então decide a sala.

Os algoritmos, ou como dividir trabalho sem saber quanto trabalho é

Round robin percorre a lista em ordem (o ponderado repete as instâncias maiores) e assume que toda requisição custa igual; a instância que pegou três autorizações de 900 ms seguidas recebe a quarta com a mesma alegria. **Aleatório** dispensa estado compartilhado, mas com `n` requisições em `n` instâncias a mais carregada recebe da ordem de `ln n / ln ln n`, longe da média 1. **Menos conexões** (*least connections*) e **menos requisições pendentes** (*least outstanding requests*) mandam para quem tem menos trabalho em andamento, uma medida direta de carga que se adapta a custo variado.

Com uma armadilha de humor negro embutida. A instância que responde erro em 1 ms (porta fechada, `503` imediato) nunca tem nada pendente, e para o least connections ela é a mais saudável do grupo:

```
Balanceador: Quem está livre?
Réplica 1:   Tenho 14 em andamento.
Réplica 2:   Tenho 12.
Réplica 3:   Zero! Termino tudo na hora!
Balanceador: Que funcionária exemplar. (manda 80% para a 3)
Réplica 3:   (devolve 503 em todas, na hora, como prometido)
```

Na quarta-feira da Lastro, com least connections, o defunto teria recebido quase tudo, e o Bazar perguntaria se a Lastro tem algo contra cartão em geral. Algoritmo por carga só funciona junto com detecção de falha.

Duas escolhas aleatórias (*power of two choices*, P2C) sorteia duas instâncias e manda para a menos carregada. Pelo resultado de Azar, Broder, Karlin e Upfal (1994), a carga máxima cai de `ln n / ln ln n` para `ln ln n / ln 2` mais uma constante, uma melhora exponencial por olhar um número a mais. E como a escolha é sorteada, balanceadores com informação atrasada não correm todos para a mesma instância vazia. O `LEAST_REQUEST` do Envoy faz P2C por padrão.

Hash manda a mesma chave (IP, cabeçalho, recurso) sempre à mesma instância, o que serve a cache local; o **hash consistente** (capítulo 24) e o **Maglev** do Google (NSDI 2016) movem só ~1/N das chaves quando uma instância sai.

Health checks, ou o exame médico que pode matar o paciente e a família

Na **checagem ativa**, o balanceador chama um endpoint de cada instância a cada intervalo e a tira de rotação após `k` falhas seguidas; acha morto sem tráfego, com atraso de até `intervalo × k`. Na **checagem passiva** (*outlier detection*), ejeta por tempo crescente a instância com erros reais seguidos; detecta em milissegundos e pega parte das falhas cinzentas do capítulo 15, em que o health check responde lindo e as requisições morrem.

O que o endpoint verifica tem dois sentidos, que o capítulo 35 reencontra no Kubernetes. **Vivacidade** (*liveness*): o processo travou sem conserto? Reinicie. **Prontidão** (*readiness*): a instância consegue atender agora? Se não, pare de mandar tráfego, sem matar ninguém.

Vocês já repararam que todo dev, no primeiro health check, faz um check-up completo? Consulta o banco, pinga o Redis, chama o parceiro e, se der tempo, mede a pressão. Acompanhe a falha: o Redis faz failover e some por 20 segundos, todas as instâncias consultam o mesmo Redis, todas falham o check no mesmo intervalo, e o balanceador, obediente, tira todas de rotação. Uma dependência que o sistema tolerava em parte vira 100% de `503`. É o único exame médico da história que mata o paciente, a família e o médico.

Por isso a prontidão verifica só o que é **local** à instância; dependências compartilhadas por todas as réplicas são tratadas na requisição, com timeout, breaker e degradação (capítulo 15). Duas proteções completam: o **limite de ejeção** impede a checagem passiva de tirar mais que uma fração das instâncias, e o **limiar de pânico** (50% por padrão no Envoy) faz o balanceador ignorar os health checks e distribuir para todos quando sobram poucas saudáveis, em vez de matar as sobreviventes de cansaço.

Na **drenagem** (*connection draining*), a instância falha a prontidão, termina o que está em andamento e só então recebe o sinal de encerramento (capítulo 34). E o tempo ocioso do keep-alive do backend precisa ser maior que o do balanceador: se o backend fecha primeiro, o balanceador escreve numa conexão recém-morta e devolve `502` sem que o backend leia um byte. É o motivo dos 65 s do Uvicorn contra os 60 s dos clientes no capítulo 8; um proxy na frente é só mais um cliente obrigado a ir embora antes de ser expulso.

Sticky sessions, o casamento por conveniência

Afinidade de sessão (*sticky session*) manda o mesmo cliente sempre à mesma instância, por cookie do balanceador, hash do IP ou cabeçalho, porque a instância guarda estado em memória (sessão, WebSocket, cache local). A carga fica desigual: hash de IP com o CGNAT do capítulo 25 manda 11 mil celulares para uma réplica só, e o supernó volta como problema de balanceamento. A instância morre e leva o estado; a drenagem dura a sessão mais longa. É um relacionamento que só funciona enquanto ninguém morre, e no backend alguém sempre morre. A alternativa é tirar o estado da instância (capítulo 29).

Reverse proxy, API gateway, service mesh e CDN, o mesmo programa com quatro crachás

Os quatro são proxies de camada 7 em posições diferentes. O **reverse proxy** fica na frente de uma aplicação: termina TLS e lê a requisição inteira antes de ocupar um worker, protegendo-o de cliente lento (a Lastro tem um por máquina desde o capítulo 10). O **API gateway** acrescenta política de API: autentica o cliente (capítulo 30), limita taxa por cliente (capítulo 28), roteia por caminho e versão para serviços diferentes e carimba um ID de requisição. O risco conhecido é a regra de negócio migrar para lá, uma transformação de payload por vez, até o gateway virar um monólito escrito em YAML, a forma mais cruel de monólito já inventada.

A **service mesh** põe um proxy ao lado de cada instância (o *sidecar*) e um **plano de controle** que distribui rotas e certificados; ganha-se mTLS, retries, balanceamento por requisição e telemetria sem mexer no código. Custa dois saltos de proxy por chamada, memória de um proxy por instância e um plano de controle que, configurado errado, quebra todas as chamadas ao mesmo tempo. Istio e Linkerd são exemplos.

A **CDN** espalha reverse proxies perto dos usuários: cache, TLS terminado perto do cliente e absorção de ataque volumétrico. Custa um terceiro lendo o tráfego em claro e o risco de uma chave de cache mal definida entregar o extrato de um vendedor a outro, um vazamento de dados com latência de borda.

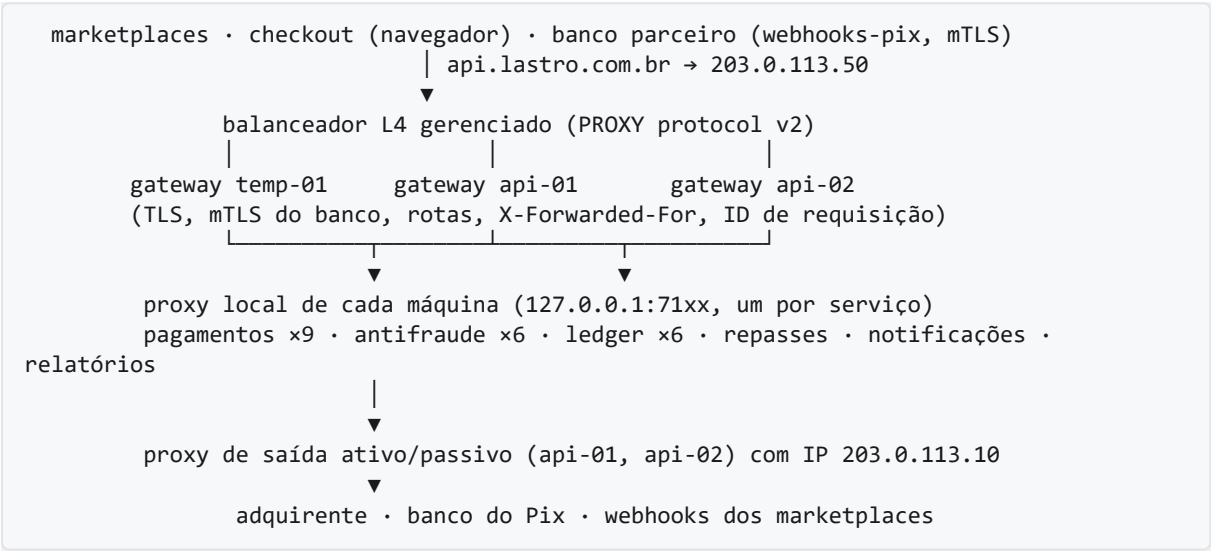
Na Lastro: uma porta de entrada, um proxy por máquina e um IP para os parceiros

Hoje `api.lastro.com.br` responde com dois registros A, `api-01` e `api-02`. DNS round robin não sabe quem está vivo, os clientes guardam a resposta por 300 s (capítulo 8), e `temp-01` ainda atende a integração do Bazar que resolveu o nome há mais de 200 dias e nunca mais perguntou. Os requisitos: tirar réplica morta de rotação em segundos; nenhuma falha de dependência compartilhada pode tirar todas as réplicas juntas (a degradação do antifraude, capítulo 15, não pode ser anulada pelo balanceador); autenticação e limite de taxa por marketplace num ponto só (capítulos 28 e 30); verificar o certificado cliente do banco em `webhooks-pix.lastro.com.br` (capítulo 10); e um IP de saída fixo, porque o adquirente, na renovação do contrato, passou a exigir allowlist de no máximo dois IPs, como o banco do Pix já exige desde o capítulo 8.

Opção	Saber quem está vivo	Política por marketplace	mTLS do banco	Saltos L7
DNS round robin (hoje)	não	em cada máquina	no proxy local	1

Opção	Saber quem está vivo	Política por marketplace	mTLS do banco	Saltos L7
L7 gerenciado + gateway próprio	sim	no gateway	depende do produto	2
L4 gerenciado + gateway próprio	sim	no gateway	no gateway	1

A Lastro fica com a terceira. O gateway próprio é obrigatório pela política por marketplace, e um L7 gerenciado na frente dele seria uma segunda terminação TLS e um segundo lugar para o mTLS do banco dar errado. O L4 entrega IP público estável, health check dos gateways e o IP do comprador pelo PROXY protocol v2, aceito só da faixa privada do balanceador. Isso muda a previsão do capítulo 10 (certificado gerenciado no balanceador): o ACME por máquina continua, nos gateways. Um L7 gerenciado que verificasse a CA privada do banco e autenticasse os marketplaces tornaria a segunda linha a certa.



Sim, `temp-01` virou gateway. Agora atende o mundo e recebe health check a cada 2 segundos. É a primeira vez que alguém mede a saúde dele, e ninguém quer ver o resultado do exame.

Os gateways roteiam por `Host` e prefixo para cada serviço (`webhooks-pix` só com certificado cliente válido). Na criação de cobrança, o timeout do gateway é 9 s, acima do deadline de 8 s do capítulo 15, para que responda a aplicação, com `processando`, e nunca o gateway, com um `504` que não diz se o cartão foi cobrado. Nenhuma rota usa afinidade: nenhum serviço guarda estado de cliente em memória, e o SSE do checkout reconecta em qualquer instância com `Last-Event-ID` (capítulo 12).

No tráfego interno, cada máquina roda um proxy local (o Envoy é um exemplo), com uma porta em `127.0.0.1` por serviço, que conhece as réplicas das três máquinas pela lista gerada no deploy. A variável do Carlos foi aposentada com honras. Pagamentos chama `127.0.0.1:7100`, e o proxy aplica P2C por requisição (inclusive streams gRPC); checagem ativa de `/pronto` a cada 2 s, fora após 2 falhas; ejeção passiva após 5 falhas de conexão ou `5xx` seguidos, por 30 s dobrando a cada reincidência, no máximo um terço das instâncias; pânico em 50%; e repete uma vez, em outra instância, só a falha de conexão, em que a requisição comprovadamente não saiu (capítulo 15).

`/pronto` verifica o que é local: tarifas carregadas, processo fora de drenagem, lag do loop abaixo de 50 ms. O Carlos abriu um PR com `/saude` testando PostgreSQL, Redis, Vigia e adquirente "por segurança". A conta da revisão: um soluço de 3 s do Vigia derruba o check das seis réplicas do antifraude ao mesmo tempo, e o balanceador remove o serviço que o capítulo 15 ensinou a sobreviver sem o Vigia, por causa do Vigia. PR fechado, com a conta no comentário.

Com essas regras, a quarta-feira vira uns quinze RSTs repetidos em outra réplica, ejeção em cada proxy local na quinta falha, confirmação ativa em 4 s e nenhum cartão recusado.

Na saída, todo HTTPS para parceiros e webhooks passa por um proxy de saída (*forward proxy*) com `CONNECT`: o TLS vai de ponta a ponta pelo túnel, e o proxy só confere o destino contra uma lista permitida (o capítulo 32 vai cobrá-la). Roda ativo em `api-01` e passivo em `api-02`, com o IP `203.0.113.10`, que o banco já cadastrou no capítulo 8, reassociado no failover em ~30 s (hipótese de projeto). O formulário com firma reconhecida para tirar o IP de `temp-01` da allowlist do banco foi protocolado: 10 dias úteis. O capítulo 36 leva a saída para NAT na rede.

A service mesh fica de fora: com seis serviços em três máquinas, os proxies locais já entregam o que ela daria, sem plano de controle; vira a escolha com dezenas de serviços, mTLS interno obrigatório ou Kubernetes (capítulo 35). A API fica fora de CDN: nenhuma resposta pode ir para cache (`private`, capítulo 22), e TLS terminado num terceiro levaria o escopo do PCI DSS junto (capítulo 33).

Armadilhas e casos reais

> **Armadilha:** retry no balanceador para qualquer erro. Um L7 que repete `POST /v1/cobrancas` após timeout de leitura cria a segunda cobrança do capítulo 16 sem ajuda de nenhuma consultoria Java.

> **Armadilha:** confiar no primeiro valor de `X-Forwarded-For`. O cliente escreve o começo da lista; o confiável é o que o seu último proxy acrescentou, contado da direita. Quem lê o da esquerda deixa o rate limiting do capítulo 28 limitar o IP que o atacante achou mais engraçado.
>
> **Armadilha:** health check respondido pelo próprio gateway ou servido de cache. Fica verde para sempre, inclusive no velório, enquanto o Bazar abre o chamado.
>
> **Na vida real:** em 8 de junho de 2021, um bug que a Fastly tinha posto em produção em 12 de maio foi disparado pela mudança de configuração válida de um cliente, e cerca de 85% da rede passou a devolver erros. A causa foi identificada em cerca de um minuto, e 95% da rede voltou em 49 minutos. Um componente compartilhado na frente de todo mundo transformou um defeito em falha simultânea de milhares de sites que não tinham mudado nada.

Resumo

- L4 decide por conexão, sem ler HTTP; L7 decide por requisição, com rota e retry, ao custo de CPU, chave e um parser a mais. PROXY protocol devolve o IP do cliente.
- Least connections ama a instância que falha rápido; P2C derruba a carga máxima para `ln ln n / ln 2`; hash consistente e Maglev servem afinidade por chave.
- Checagem ativa e passiva; prontidão só com o que é local; limite de ejeção e limiar de pânico; drenagem; keep-alive do backend maior que o do balanceador.
- Sticky session paga estado em memória com carga desigual.
- Reverse proxy, API gateway, service mesh e CDN: proxies L7 em posições diferentes.

A Lastro agora

`api.lastro.com.br` aponta para um balanceador L4 gerenciado que distribui entre três gateways; entre serviços, proxies locais com P2C, `/pronto` e ejeção passiva; na saída, um proxy `CONNECT` com o IP `203.0.113.10`, o único que os parceiros conhecem. A quarta-feira de 47 minutos virou quinze RSTs que ninguém viu, e `temp-01`, promovido a borda, passa por exame médico a cada dois segundos e, para surpresa de todos, passa.

Agora existe uma porta só, e ela atende qualquer um com a mesma educação. Na segunda, o Hornero sobe uma integração cujo laço de reconciliação perdeu a condição de parada e consulta `GET /v1/cobrancas/{id}` quatro mil vezes por segundo. O balanceador distribui as quatro mil com perfeição, e as cobranças dos outros marketplaces ficam na fila, atrás de um `while True` muito bem balanceado.

Referências

- Azar, Y.; Broder, A.; Karlin, A.; Upfal, E., "Balanced Allocations", STOC 1994 (SIAM Journal on Computing, 1999): https://doi.org/10.1137/S0097539795288490
- Mitzenmacher, M., "The Power of Two Choices in Randomized Load Balancing", IEEE TPDS, 2001: https://doi.org/10.1109/71.963420
- Eisenbud, D. E. et al., "Maglev: A Fast and Reliable Software Network Load Balancer", NSDI 2016: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud
- HAProxy, "The PROXY protocol" (versões 1 e 2): https://www.haproxy.org/download/1.8/doc/proxy-protocol.txt
- Envoy, balanceadores de carga (least request com P2C): https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers
- Envoy, limiar de pânico: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/panic_threshold
- Envoy, outlier detection: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/outlier
- RFC 7239, Forwarded HTTP Extension: https://www.rfc-editor.org/rfc/rfc7239
- Fastly, "Summary of June 8 outage", 2021: https://www.fastly.com/blog/summary-of-june-8-outage

CAPÍTULO 28

Controle de carga

Na segunda às 11h02, o Mercadito Hornero subiu uma versão nova da integração, e às 11h03 a Lastro descobriu que um `while True` sem condição de parada, do outro lado do continente, é um teste de carga que ninguém agendou e ninguém pode cancelar.

O laço de reconciliação do Hornero passou a consultar `GET /v1/cobrancas/{id}` em todas as cobranças do mês, sem parar, sem `If-None-Match`, por 40 conexões: 4.000 requisições por segundo, que o balanceador do capítulo 27 distribuiu com perfeição entre os nove processos de pagamentos. Cada GET custa ~1,5 ms de CPU (capítulo 14), então cada event loop passou a gastar dois terços do tempo servindo o Hornero. A autorização de cartão, que divide esses loops, foi de p99 1,3 s para 4,8 s, e a criação de Pix do Bazar começou a estourar o timeout de 10 s do próprio Bazar.

O Bazar abriu chamado às 11h09: "URGENTE!!!! O Pix de vocês está mais lento que o meu" (o Pix dele é o mesmo). Às 11h20, o Carlos bloqueou no firewall o IP de saída do Hornero. Funcionou. O Hornero parou de consultar, e parou também de criar cobranças, de capturar cartão e de receber Pix dos compradores, porque tudo sai do mesmo IP. A analista do Hornero passou 25 minutos sem vender nada e, pela primeira vez, sem tempo para o crochê, porque estava abrindo chamado.

Como uma API recusa o excesso de um cliente sem punir os outros e sem matar o próprio cliente, e o que ela sacrifica quando o excesso vem de todos ao mesmo tempo?

Por que todo sistema tem um limite e quase nenhum sabe qual é

O capítulo 14 deu a fórmula: com utilização ρ, o tempo de resposta cresce com `S / (1 - ρ)`. A 70%, a latência é 3,3 vezes o tempo de serviço; a 90%, 10 vezes; a 99%, 100 vezes. Acima de 100%, a fila cresce sem limite, e a latência também, até alguém desistir.

E alguém desiste. O cliente tem timeout (capítulo 15), corta a conexão e manda de novo. O servidor, que não ficou sabendo, termina a requisição abandonada, gasta CPU numa resposta que ninguém vai ler e põe a nova tentativa no fim da fila. A partir de um ponto, tudo que termina já passou do prazo de quem pediu: a **vazão útil** (*goodput*) vai a zero

com a CPU no talo. O dashboard de requisições processadas nunca esteve tão bonito, e nenhum cliente foi atendido. É o sistema mais produtivo e mais inútil da empresa, e ele concorre com o comitê de arquitetura.

Controle de carga mantém o sistema abaixo desse ponto, em três lugares: a **limitação de taxa** (*rate limiting*) define quanto cada cliente pode pedir, mesmo com o sistema folgado; a **contrapressão** (*backpressure*) faz quem está cheio desacelerar quem está antes; e o **descarte de carga** (*load shedding*) recusa parte do trabalho na entrada quando o servidor passa do limite, escolhendo qual parte.

Os algoritmos de limitação, ou quatro jeitos de contar até cem

Todo limitador pergunta se esta requisição, deste cliente, cabe em N por período. Muda o que ele guarda e quanto erra.

Janela fixa

Um contador por cliente por janela de calendário (`hornero:2026-11-09T11:03`), incrementado a cada requisição e zerado na virada. Uma chave, um `INCR`, um `EXPIRE`. O defeito está na fronteira:

```
limite: 100 por minuto
11:03:59.5   100 requisições   (janela 11:03: 100/100, aceitas)
11:04:00.1   100 requisições   (janela 11:04: 100/100, aceitas)
             200 em 0,6 s, e o limitador acha que está tudo certo
```

O cliente pode mandar o dobro do limite em um instante, desde que tenha um relógio. E todo cliente tem um relógio, inclusive o `repasse_final_v2_agora_vai.py`, que tem dois e discorda dos dois.

Janela deslizante

O **registro deslizante** (*sliding log*) guarda o horário de cada requisição e conta as dos últimos 60 s. É exato e custa memória proporcional ao limite por cliente: para 1.000 por minuto, mil timestamps por chave.

O **contador deslizante** (*sliding window counter*) guarda só dois contadores, a janela atual e a anterior, e estima o total ponderando a anterior pela fração que ainda cai dentro dos últimos 60 s:

```
estimativa = anterior × (1 − decorrido/janela) + atual
ex.: anterior 84, atual 36, 15 s da janela atual decorridos
     84 × 0,75 + 36 = 99   →   aceita a 100ª, recusa a 101ª
```

A estimativa assume que a janela anterior foi uniforme, e erra pouco (ver o caso da Cloudflare nas armadilhas). Dois inteiros por cliente, quase o registro exato: raro caso em que a aproximação é mais honesta que o planejamento de sprint.

Balde de fichas e balde furado

O **balde de fichas** (*token bucket*) tem capacidade `b` e recebe fichas à taxa `r` por segundo, até encher. Cada requisição gasta uma ficha; sem ficha, recusa. A taxa média fica limitada a `r`, e uma rajada de até `b` passa de uma vez. Na implementação, ninguém agenda o gotejamento: o balde guarda `(fichas, último_acesso)` e, a cada requisição, soma `r × (agora − último_acesso)` antes de descontar. Dois números por cliente, rajada configurável separada da taxa, e por isso é o algoritmo dos gateways e das APIs públicas.

O **balde furado** (*leaky bucket*) tem dois sentidos na literatura. Como **fila**, as requisições entram num buffer de tamanho `b` e saem à taxa constante `r`, alisando o tráfego ao custo de espera. Como **medidor**, é o balde de fichas visto do outro lado: o nível sobe a cada requisição, vaza à taxa `r`, e o que transborda é recusado. O **GCRA** (*Generic Cell Rate Algorithm*, das redes ATM) é esse medidor guardando um único número por cliente, o horário teórico da próxima chegada, e cabe num script atômico de Redis.

Algoritmo	Estado por cliente	Rajada	Erro
Janela fixa	1 contador	até 2× na fronteira	grande na virada
Registro deslizante	1 timestamp por requisição	nenhuma além do limite	zero
Contador deslizante	2 contadores	pequena	~0,003% medido
Balde de fichas / GCRA	2 números / 1 número	até `b`, explícita	zero para a regra que define
Balde furado (fila)	fila de até `b`	absorvida como espera	zero, com latência

Limitar com vários gateways, onde o contador mora

Com um gateway, o estado fica na memória dele. Com três, há três opções, e cada uma erra de um jeito.

Contador central (Redis, por exemplo): todo gateway atualiza o mesmo estado num comando atômico (script Lua com o GCRA, ou `INCR` com `EXPIRE`). É exato, custa uma ida e volta por requisição e põe uma dependência no caminho de toda chamada. Se o

Redis cai, alguém decide entre deixar passar tudo (*fail open*) ou recusar tudo (*fail closed*), e recusar tudo porque o contador sumiu é derrubar a API para protegê-la.

Contador local com o limite dividido por N: zero latência e zero dependência, e só funciona se o tráfego de cada cliente se espalhar igual entre os gateways. O capítulo 27 explica por que não se espalha: o balanceador de camada 4 distribui conexões, e um cliente com duas conexões persistentes cai em um ou dois gateways e recebe um terço ou dois terços do contratado, conforme o sorteio de um balanceador que nunca leu o contrato.

Híbrido: cada gateway decide localmente e sincroniza com o contador central a cada intervalo (100 ms, por exemplo), aceitando estourar o limite por uma fração do intervalo.

A resposta: 429, Retry-After e um corpo que explica

A recusa por limite de taxa tem status próprio, o `429 Too Many Requests` (RFC 6585), e o `Retry-After` (RFC 9110) diz quando tentar de novo, em segundos ou como data. Sobrecarga do servidor é `503 Service Unavailable`, também com `Retry-After`. O `429` diz "você pediu demais"; o `503` diz "eu estou cheio, e nada pessoal". Um rascunho do IETF (*RateLimit header fields for HTTP*) padroniza cabeçalhos com a política e a cota restante; até virar RFC, os nomes podem mudar.

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 1

{"type": "https://api.lastro.com.br/erros/limite-de-taxa",
 "title": "Limite de requisições excedido",
 "status": 429,
 "detail": "Classe 'leitura': 200 requisições/s. Tente em 1 s.",
 "classe": "leitura"}
```

O cliente educado espera o `Retry-After`, recua com backoff e jitter (capítulo 15) e tem orçamento de retentativas. O cliente mal-educado trata o `429` como erro transitório e repete na hora, o que transforma o limitador num gerador de tráfego com dupla contagem. O livro de SRE do Google descreve a defesa do lado do cliente: cada cliente mede as próprias requisições e aceitações recentes e recusa localmente, sem nem enviar, com probabilidade `max(0, (requisições − K × aceitas) / (requisições + 1))`, com `K = 2` como padrão. Quando o servidor recusa muito, o cliente para de insistir sozinho.

Contrapressão e filas limitadas

Toda fila sem limite é uma promessa que o sistema não sabe que fez. A fila do event loop, o backlog do `listen`, a espera do pool, a fila do proxy: sem teto, absorvem a sobrecarga como latência, que vira timeout lá na frente, onde ninguém sabe de onde veio. Uma fila limitada transforma excesso em recusa rápida e visível.

Contrapressão é a fila cheia se propagando para trás. No TCP, a janela de recepção (capítulo 8) faz o remetente parar. No log (capítulo 17), o consumidor puxa no próprio ritmo e o excesso vira atraso no broker. Em HTTP, é o `429` / `503` com `Retry-After` e um cliente que respeita. No resto, o projeto constrói: `asyncio.Queue(maxsize=...)` que bloqueia o produtor, semáforo, bulkhead (capítulo 15).

O critério de corte que melhor acompanha a experiência do cliente é o **tempo na fila**, mais que o tamanho dela. O **CoDel** (RFC 8289), desenhado para roteadores, descarta quando o menor atraso de fila num intervalo fica acima de um alvo; o Facebook descreveu em 2015 (*Fail at Scale*) o uso desse critério em servidores, junto com **LIFO adaptativo**: sob fila longa, atender primeiro a requisição mais nova, porque a mais velha provavelmente já foi abandonada pelo cliente. É cruel com a requisição velha. Ela também é a única que já morreu e ainda não sabe.

Descarte de carga, ou quem sai do bote primeiro

Se todos os clientes ficam dentro do limite ao mesmo tempo (a Black Friday, o dia 1º), a soma ainda passa da capacidade. O **descarte de carga** recusa trabalho quando o próprio servidor está no limite, por um sinal local: requisições em andamento, lag do event loop, tempo na fila. Recusa **cedo**, antes de autenticar e abrir transação; por limite fixo de concorrência ou por **limite adaptativo**, que ajusta a concorrência pela latência observada, à moda do controle de congestionamento do TCP (a biblioteca *concurrency-limits* da Netflix é um exemplo); e por **prioridade**, começando pelas classes menos importantes.

A prioridade só funciona se a classe sacrificada consumir o mesmo recurso que a protegida. Recusar relatório para salvar autorização só ajuda se os dois disputam a mesma CPU, a mesma fila ou o mesmo banco. Senão, você jogou fora o relatório e a autorização continua lenta, agora sem relatório.

Na Lastro: limites por marketplace e uma ordem de sacrifício

Os requisitos: o excesso de um marketplace não pode afetar os outros (o p99 do Bazar está em contrato, capítulo 14); o limite é por marketplace e por classe de operação, nunca por IP, para que o Hornero em loop continue recebendo Pix; e, sob sobrecarga geral, autorização e Pix são os últimos a sofrer.

Limitação no gateway

O gateway identifica o marketplace pela credencial (o capítulo 30 detalha) e aplica um balde de fichas (GCRA) por `(marketplace, classe)`:

Classe	Rotas	Padrão (`r` / `b`)	Bazar
escrita	criar cobrança, captura, estorno, saque	50/s / 100	300/s / 600
leitura	GET de cobranças, estornos, listagens	200/s / 400	1.000/s / 2.000
painel	GraphQL, relatórios	20/s / 40	60/s / 120
sem limite	`webhooks-pix` (banco parceiro)	–	–

Os valores ficam a cerca de dez vezes o pico medido de cada marketplace, e os do Bazar acima da projeção de Black Friday do capítulo 14. As classes são separadas para que o laço do Hornero gaste a cota de leitura e ele continue criando cobranças pela de escrita. As notificações do banco ficam fora: recusar uma é arriscar perder a confirmação de um Pix (o banco desiste após três tentativas, capítulo 10).

O estado mora no Redis de cache do capítulo 22: ~120 chaves com expiração, consultadas a cada requisição, as últimas que um LRU despejaria, a ~0,4 ms de ida e volta. Divisão local por três foi descartada pelo sorteio de conexões da seção anterior. Se o Redis cair, cada gateway aplica `limite / 3` localmente: impreciso por alguns minutos, e nunca um *fail closed* que derruba a API porque um contador sumiu.

A resposta é `429` com `Retry-After` e o `type` `limite-de-taxa` (capítulo 11). Status novo em rota existente é mudança de comportamento: os limites entraram no changelog com 30 dias de antecedência, com a obrigação de respeitar o `Retry-After`. Para incidente ativo, há um limite de emergência por marketplace, ajustável sem deploy, que o Carlos deveria ter usado às 11h20 no lugar do firewall.

A segunda-feira das 11h02 se repete assim: o Hornero recebe 200 respostas por segundo e 3.800 `429`, os nove loops gastam 3% do tempo com ele, ninguém mais percebe, e um e-mail automático avisa o Hornero porque a taxa de `429` dele passou de 50% por cinco minutos. A analista volta ao crochê.

Descarte por prioridade em pagamentos

O limite por marketplace não protege da soma. Cada processo de pagamentos classifica a requisição antes de qualquer trabalho:

- **crítica**: autorização de cartão, criação de Pix, notificação do banco;
- **normal**: captura, estorno, GET de cobrança;
- **descartável**: GraphQL do painel e listagens longas.

O sinal é o lag do event loop (medido desde o capítulo 13): acima de 50 ms, as descartáveis recebem `503` com `Retry-After: 5`; acima de 150 ms, também as normais; as críticas só esbarram nos bulkheads do capítulo 15. O corte acontece no middleware de entrada, antes de banco ou transação, e custa microssegundos. As três classes dividem os mesmos event loops, então recusar uma alivia a outra.

Relatórios, no próprio serviço desde o capítulo 26, têm fila limitada: dois em execução por processo, cinco esperando; o sexto recebe `503` com `Retry-After: 60`. A fila sem limite que o Carlos propôs ("uma hora sai") acumularia, num dia 1º, os 41 relatórios do capítulo 23 mais cada clique impaciente, e andaria para um passado em que ninguém mais está esperando.

No gateway, cada rota tem concorrência máxima até o serviço, e requisição que esperou mais de 1 s na fila recebe `503`, porque o deadline de 8 s da autorização (capítulo 15) não melhora parado. O ledger, que consome `cobrancas.eventos`, tem contrapressão de graça: o pico vira atraso de consumidor, com o alerta de 30 s do capítulo 26.

Armadilhas e casos reais

> **Armadilha:** limitar por IP numa API de servidores. Um IP de saída costuma ser um NAT com a empresa inteira atrás, e o limite pune o cliente pelo tamanho. O bloqueio do Carlos foi o caso extremo: tirou do ar a operação inteira de um marketplace para conter uma rota dele.
>
> **Armadilha:** retentativa sem respeitar `Retry-After`. O cliente que repete o `429` na hora, com três camadas de retry (capítulo 15), multiplica o tráfego justo quando o servidor pediu menos. O limitador precisa recusar barato, porque também vai receber o dilúvio.
>
> **Armadilha:** descartar depois do trabalho caro. O `503` que sai depois da consulta ao banco gastou o recurso escasso para anunciar que ele está escasso, com a eficiência de um comunicado do RH.

> **Armadilha:** health check sujeito ao descarte. O `/pronto` (capítulo 27) recebe `503` do próprio descartador, o balanceador tira a instância, as outras herdam a carga e descartam o health check também. Em minutos, o sistema que descartava 20% para sobreviver descarta 100%. O health check fica fora do descarte.
>
> **Na vida real:** a Cloudflare publicou em 2017 a análise do contador deslizante do seu limitador: em 400 milhões de requisições de 270 mil origens, 0,003% foram aceitas ou recusadas de forma errada.
>
> **Na vida real:** a Stripe descreveu em 2017 quatro mecanismos em produção: limitador de taxa por usuário (balde de fichas em Redis), limitador de requisições concorrentes e dois descartadores, um que reserva parte da frota para tráfego crítico e outro por utilização dos workers.

Resumo

- Acima da capacidade, a vazão útil vai a zero com a CPU no talo.
- Janela fixa deixa passar 2× na fronteira; contador deslizante aproxima com dois números; balde de fichas e GCRA separam taxa de rajada.
- Contador central é exato e custa uma ida e volta; local dividido por N falha com balanceamento por conexão.
- `429` com `Retry-After` para o cliente acima do limite; `503` para o servidor cheio.
- Filas limitadas, contrapressão e corte por tempo na fila.
- Descarte cedo, por sinal local e por prioridade, entre classes que disputam o mesmo recurso.

A Lastro agora

O gateway limita cada marketplace por classe com GCRA no Redis, devolve `429` com `Retry-After` e cai para `limite / 3` local se o Redis sumir; as notificações do banco não têm limite. Pagamentos corta descartáveis e depois normais pelo lag do event loop, relatórios têm fila de cinco, e o gateway descarta o que esperou mais de 1 s. O Hornero em loop custa 3% de CPU e um e-mail automático, e o Carlos ganhou um botão de limite de emergência, que ele chama de "firewall com pontaria".

Na sexta, o comercial fechou contrato com um marketplace de um grupo de varejo, dez vezes maior que o Bazar, com migração em quatro meses. Todos os números deste capítulo foram calculados sobre o pico do Bazar. O gerente de produto perguntou se dá

para "só multiplicar tudo por dez", e o PostgreSQL do ledger, que não foi consultado, começou a suar.

Referências

- RFC 6585, Additional HTTP Status Codes (429): https://www.rfc-editor.org/rfc/rfc6585
- RFC 9110, HTTP Semantics (Retry-After, 503): https://www.rfc-editor.org/rfc/rfc9110
- RFC 8289, Controlled Delay Active Queue Management: https://www.rfc-editor.org/rfc/rfc8289
- IETF, RateLimit header fields for HTTP (rascunho): https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/
- Google, Site Reliability Engineering, "Handling Overload": https://sre.google/sre-book/handling-overload/
- Maurer, B., "Fail at Scale", ACM Queue, 2015: https://queue.acm.org/detail.cfm?id=2839461
- Cloudflare, "Counting things, a lot of different things...", 2017: https://blog.cloudflare.com/counting-things-a-lot-of-different-things/
- Stripe, "Scaling your API with rate limiters", 2017: https://stripe.com/blog/rate-limiters
- Netflix, concurrency-limits: https://github.com/Netflix/concurrency-limits

CAPÍTULO 29

Escalabilidade e capacidade

Na terça às 15h12, o Carlos ligou o *autoscaling* do ambiente de homologação com máximo de 64 processos, "por segurança", e às 15h19 a Lastro aprendeu que segurança, no vocabulário do Carlos, é uma unidade de medida de incidente.

O contrato era com o Tuiuiú, marketplace de um grupo de varejo, dez vezes maior que o Bazar, migração em quatro meses. O gerente apresentou o plano de capacidade num slide: tudo vezes dez, inclusive o PostgreSQL. O fundador sugeriu "a maior máquina que tiver", com a convicção de quem foi ignorado no capítulo 24. O Carlos quis provar com um teste de carga no volume do Tuiuiú.

O autoscaling funcionou lindamente. Subiu 64 processos de pagamentos, cada um com a sua conexão HTTP/2 para a homologação do adquirente. Só que a allowlist do adquirente aceita dois IPs (capítulo 27), o segundo ainda é o do `temp-01`, e homologação sai pelo mesmo proxy de produção, com o mesmo `203.0.113.10`. O adquirente limita 20 conexões por origem (capítulo 9), viu 73, classificou como abuso e bloqueou o IP por 15 minutos. Produção foi junto, de carona.

Foram 2.730 autorizações de cartão devolvidas com `503` e `adquirente-indisponivel`, nenhum comprador cobrado (falha de conexão é certeza de não envio, capítulo 15). O Bazar abriu chamado às 15h22: "URGENTE!!!! vocês estão escalando ou caindo?". As duas coisas, Bazar. Em paralelo, que é o jeito escalável.

Quanto um cliente dez vezes maior exige de cada recurso, quais partes crescem com mais máquinas, quais não crescem com nenhuma, e como fazer as máquinas aparecerem sozinhas sem atropelar um limite que ninguém tinha olhado?

Multiplicar tudo por dez, ou a arte de errar a conta com confiança

Estimativa de capacidade é aritmética de ordem de grandeza: transformar um número de negócio no consumo de cada recurso (CPU, conexões, escritas, armazenamento, banda) com erro de até 2× ou 3×, curta o bastante para caber num guardanapo e certa o bastante para dizer se o plano é uma máquina ou cinquenta.

As conversões que fazem quase todo o trabalho:

```
1 dia                 = 86.400 s ≈ 10^5 s
1 ano                 ≈ 3,15 × 10^7 s
1 req/s constante     ≈ 86 mil/dia ≈ 2,6 milhões/mês
1 milhão/dia          ≈ 12/s de média
1 KB × 1.000/s        = 1 MB/s = 8 Mbit/s
concorrência          = taxa × tempo de permanência   (Little: L = λW)
```

A média diária não dimensiona nada. O **fator de pico** (pico dividido pela média) se mede no histórico do próprio tráfego, e é ele que multiplica tudo. Sistema dimensionado pela média cai todo dia no mesmo horário, com uma pontualidade que o resto da empresa nunca conseguiu.

O método é uma cadeia: negócio → taxa de pico por operação → custo medido por operação (ms de CPU, linhas, bytes) → limite de cada recurso, com a folga da teoria de filas (acima de ~70% a latência explode, capítulo 14).

O defeito do slide é supor que todo recurso cresce linearmente. A CPU dos serviços sem estado cresce. Alguns recursos nem crescem, porque são o limite fixo de alguém que não estava na reunião: as 20 conexões do adquirente, os dois IPs da allowlist, as 12 partições de um tópico. O gerente multiplicou por dez até o adquirente, que pelo visto vinha com volume ilimitado incluso no plano.

> **Na vida real:** no lançamento do Pokémon GO, em julho de 2016, o tráfego chegou a 50 vezes a meta que a Niantic tinha planejado com o Google Cloud e a 10 vezes a estimativa de pior caso, segundo o próprio Google. O pior caso tinha um pior caso.

Vertical, horizontal e o pedágio que cada máquina nova cobra

Escalar **verticalmente** é trocar a máquina por uma maior. O código e a consistência não mudam. Os limites: o catálogo tem uma maior máquina, o preço por núcleo sobe perto do topo, a troca exige reiniciar, e continua havendo um único domínio de falha. O fundador, que acredita em máquina maior com a fé que outros reservam para santo, está certo até o dia em que o catálogo acaba.

Escalar **horizontalmente** é pôr máquinas iguais atrás de um balanceador (capítulo 27). Sem teto, e perder uma réplica de vinte custa 5%. Em troca, qualquer réplica precisa atender qualquer requisição, e a coordenação cresce com o número de réplicas.

Esse custo tem modelo. A **Lei Universal de Escalabilidade** (USL), de Neil Gunther, dá a capacidade relativa com N nós:

```
C(N) = N / (1 + α(N − 1) + βN(N − 1))

α = contenção   (fração serializada: lock, recurso único, fila comum)
β = coerência   (custo de os nós ficarem de acordo entre si, par a par)
```

Com β = 0, é a Lei de Amdahl do capítulo 14: teto em 1/α. Com β > 0, a curva passa por um máximo em `N* = √((1 − α)/β)` e depois **desce**, porque a coerência cresce com N^2 e a capacidade com N. É a escalabilidade retrógrada, que explica por que uma reunião de 30 pessoas decide menos que uma de quatro.

Com α = 0,05 e β = 0,001, dez nós entregam 6,5 vezes um nó, 31 nós (o máximo) 9,0 vezes, e cem nós 6,3 vezes: noventa máquinas a mais para ficar pior que com dez, e o consultor cobra por máquina. Os coeficientes saem do ajuste a testes de carga com 1, 2, 4 e 8 nós. Serviço sem estado tem α e β perto de zero; o α mora no que as réplicas compartilham: banco, parceiro, lock.

Stateless, ou onde você escondeu o corpo

Um serviço é *stateless* (sem estado) quando nenhum dado que importa para a próxima requisição vive só dentro do processo: matar uma réplica perde, no máximo, o que estava em andamento nela (é o fator VI do Twelve-Factor App). Na prática, é uma auditoria:

O que o processo guarda	Pode morrer junto?
Estado da requisição	Sim; o cliente repete com chave de idempotência
Cache local (tarifas, cap. 22)	Sim, se reconstruível na partida
Sessão de usuário em memória	Não; vai para banco, Redis ou token assinado (cap. 30)
Conexão longa (SSE, `LISTEN`)	Sim, se o cliente reconecta e retoma
Trabalho em andamento	Só com lease ou resolução que retome
Limite local (bulkhead, pool, conexão HTTP/2)	Sim, mas o total muda com N
Papel único (job com lease, cap. 20)	Não escala horizontalmente
Arquivo em disco local	Não; vai para armazenamento de objetos

A linha traiçoeira é a do limite local: limite por processo vezes N cresce sozinho, e o recurso do outro lado não. 64 processos × 1 conexão = 64 conexões para quem aceita 20. Ninguém mudou uma linha de código. Mudou N, e N não passa por revisão de PR.

E todo sistema tem uma máquina que só existe no singular. Na Lastro é o `temp-01`, que não tem imagem nem script de criação: ele é a imagem. Se morrer, a Lastro escala para zero, com a elegância de um autoscaling que só sabe descer.

Autoscaling, um laço de controle com atraso

Autoscaling é um laço de controle: mede, compara com uma meta, calcula quantas réplicas deveriam existir, cria ou destrói, espera e mede de novo. O Horizontal Pod Autoscaler do Kubernetes (capítulo 35) usa a regra proporcional que a maioria dos autoscalers segue:

```
réplicas_desejadas = ceil(réplicas_atuais × métrica_atual / métrica_alvo)

6 réplicas, CPU média 63%, alvo 35%   →   ceil(6 × 63 / 35) = ceil(10,8) = 11
```

O HPA ignora desvios de até 10% e, por padrão, só reduz olhando a maior recomendação dos últimos 300 s.

Qual métrica

A métrica precisa subir antes da latência e cair quando se acrescenta réplica: CPU para serviço limitado por CPU; concorrência em andamento ou lag do event loop para quem espera rede (capítulo 13); backlog ou idade da mensagem mais antiga para consumidores (capítulo 17). Latência p99 sozinha é a pior escolha: sobe também quando o lento é o banco ou o parceiro, e o laço responde do único jeito que conhece:

```
Banco:        Tô lento.
Autoscaler:   Entendido. Mandando mais 40 clientes pra você.
Banco:        Tô MAIS lento.
Autoscaler:   Entendido. Mandando mais 80.
```

Consumidores de log têm um teto que métrica nenhuma fura: num consumer group, cada partição é lida por um único consumidor (capítulo 17). Com 12 partições, o 13º consumidor fica parado, recebendo salário, igual ao cargo de vice-presidente de inovação.

Histerese

Laço com atraso e sem histerese oscila: sobe réplicas, a CPU cai, remove, a CPU sobe, sobe de novo. É o *flapping*. A **histerese** separa os gatilhos: subir quando a métrica passa do alvo por alguns minutos; descer só bem abaixo dele, por bem mais tempo, um passo por vez, com intervalo de espera (*cooldown*) após cada ação. Réplica a mais custa dinheiro; réplica a menos custa cliente.

Tempo de reação

O autoscaling reage ao que já aconteceu. A soma dos atrasos do laço é o tempo em que a carga nova bate só nas réplicas antigas:

```
t = 0      pico começa
+60 s      fecha a janela da métrica (média do último minuto)
+120 s     segunda medição confirma (não reage a ruído)
+125 s     decisão; pedido de máquinas novas
+215 s     máquina liga a partir da imagem (~90 s)
+245 s     processos sobem, carregam tarifas, abrem conexões
+255 s     /pronto responde; registro e proxies atualizam a lista
≈ 4 min    capacidade nova recebe tráfego
```

Nesses quatro minutos, quem segura a carga é a folga das réplicas existentes. Daí a regra: o alvo deixa folga para o maior crescimento possível dentro do tempo de reação. Se a carga pode dobrar em quatro minutos e o máximo tolerável é 70%, o alvo é 35%. Pico mais rápido fica com o controle de carga do capítulo 28 e com o **escalonamento agendado**, que sobe réplicas antes do pico com hora marcada.

O máximo do grupo sai do recurso mais escasso que cada réplica consome: conexões do banco ou do parceiro, partições do log. O de 64 do Carlos saiu de "por segurança", a mesma fonte dos 1.024 baldes do capítulo 4. E a réplica que vai morrer sai do balanceador e termina o que está em andamento (drenagem, capítulo 27); senão, devolve a pergunta do capítulo 15, agora automatizada.

A conta do Tuiuiú

Primeira correção do slide: o Bazar é metade do pico da Lastro. Dez Bazares são cinco Lastros, e a Lastro com o Tuiuiú é seis vezes a de hoje. O gerente errou para mais, o erro que só custa dinheiro. O do Carlos custou o adquirente. O pico de referência do capítulo 14 (2,87 s de CPU por segundo) vezes seis:

Recurso	Pico com Tuiuiú	Limite que importa	Cresce com réplicas?
CPU de pagamentos	17,2 s/s	$\rho \le 0{,}7$ no 2× do pico	Sim
Autorizações em andamento	324 (648 no 2×)	adquirente: 20 conexões	Não
Lançamentos no ledger	~4.800/s	primário: ~7.000/s medidos	Não
Créditos na comissão do Tuiuiú	~2.000/s	16 baldes: ~4.500/s	Não

Recurso	Pico com Tuiuiú	Limite que importa	Cresce com réplicas?
Eventos de cobrança	3.000/s	12 partições	Até 12 consumidores
Webhooks ao Tuiuiú	~2.500/s	bulkhead 10 por endpoint	Não

A primeira linha é a única que obedece ao slide: pela regra do capítulo 14, pagamentos precisa de `2 × 17,2 / 0,7 ≈ 50` workers, contra 9. CPU de serviço sem estado escala com α perto de zero, e é aí que entra o autoscaling. As outras linhas são o que o slide não viu.

Adquirente. Uma conexão por processo seriam 50 conexões contra 20: o incidente de terça, versão produção. A Lastro desacopla: as chamadas ao adquirente vão para um serviço novo, `conector-adquirente`, com 10 processos fixos (5 em `api-01`, 5 em `api-02`), uma conexão e 100 streams cada: 1.000 streams para 648 autorizações em andamento, 10 conexões mais 2 de homologação. Breaker, bulkhead e deadline (capítulo 15) mudam junto. Custa ~1 ms de salto interno, pago com a folga de 120 ms do orçamento. Negociar um limite maior seria a escolha certa com número de processos estável; com autoscaling, manteria a amarra entre N e o parceiro.

Ledger. O gatilho do capítulo 24 era escrita no primário acima de 50% da capacidade medida. O teste em `pg-ledger-1` mediu ~7.000 lançamentos/s até o p99 do COMMIT síncrono passar de 10 ms; 4.800 é 69%. Disparou. O Tuiuiú ganha shard próprio pelo diretório, e o shard 0 fica com ~800/s. O do Tuiuiú, ~4.000/s, nasce com o dobro de núcleos e mediu ~11.000/s (36%): 1,6 vez a menor, não 2, porque flush do WAL e espera pela réplica síncrona são o α da USL, e núcleo nenhum acelera fibra entre zonas. A comissão dele, ~4.000 créditos/s no dobro do pico contra ~4.500/s de teto, passa de 16 para 32 baldes. O Carlos queria 1.024. Continua não ganhando.

Partições. Com 12 partições, o consumidor do ledger teria 5 ms por evento no dobro do pico, e a posse de balde sozinha gasta ~3,5. O tópico vai para 48. Acrescentar partição muda `murmur2 mod N` das chaves existentes e pode desordenar eventos da mesma cobrança, mas o único produtor é o relay do CDC (capítulo 18): pausa o relay, a outbox acumula, o lag zera, acrescentam-se as partições, retoma. A outbox, que nasceu para não perder evento, agora segura a fila enquanto a Lastro faz obra.

Webhooks. Dois entregadores com bulkhead de 10 por endpoint dão 20 entregas simultâneas; com o endpoint do Tuiuiú em ~150 ms, Little dá `20 / 0,15 ≈ 133`/s, contra 2.500/s. Uma hora de Black Friday deixaria 8,5 milhões de webhooks na fila, e o Tuiuiú

descobriria os pagamentos da sexta na terça. A concorrência passa a ser por inscrição, por Little com folga de 2×: `2.500 × 0,15 × 2 = 750`, arredondado para 800. A banda dá ~72 Mbit/s. O gargalo era uma constante 10 num arquivo de configuração.

Armazenamento. O ledger vai de 1,5 para 9 milhões de lançamentos por dia, a ~170 bytes cada com índices (o mês corrente ocupa ~7,5 GB para ~45 milhões de linhas):

```
9 × 10^6 lançamentos/dia × 170 B  ≈ 1,5 GB/dia
                                  ≈ 46 GB/mês ≈ 560 GB/ano
10 anos de retenção (cap. 23)     ≈ 5,6 TB e ~33 bilhões de lançamentos
× 4 cópias (primário, 2 standbys, réplica de relatórios)
```

Tudo online seriam mais de 22 TB de disco de banco. A Lastro mantém 24 meses online (~1,1 TB por cópia) e manda o resto, por `DETACH`, para o arquivo do capítulo 24; `eventos_cobranca` (~270 GB/ano) passa a ser particionada por mês. O tabelião vai carimbar 33 bilhões de partidas dobradas e já pediu um estagiário.

O autoscaling da Lastro

Nem todo serviço escala, e a tabela diz por quê:

Serviço	Métrica de escala	Réplicas	O máximo vem de
pagamentos	CPU por worker, alvo 35%	12–60	conexões `LISTEN`
antifraude	CPU, alvo 35%	6–30	conexões do `redis-sinais`
entregador de webhooks	idade da entrega mais antiga, alvo 5 s	2–20	concorrência por endpoint
ledger (consumidor)	lag por partição	6–48	48 partições
conector-adquirente	não escala	10	20 conexões do adquirente
repasses	não escala	lease	papel único (cap. 20)
relatórios	não escala	fixo	memória (cap. 28)

O máximo de pagamentos é uma conta: cada processo mantém uma conexão `LISTEN` direto no `pg-pagamentos` (capítulo 12), fora do PgBouncer (capítulo 26), e 60 mais as 18 do PgBouncer dão 78, abaixo das 100 de `max_connections`. O alvo de 35% sai do tempo de reação medido, 4 min 20 s; crescimento acima de 2× nesse intervalo fica com o descarte do capítulo 28, que já sabe sacrificar relatórios antes de autorizações.

Pagamentos sobe 50% das máquinas com CPU acima de 35% por 2 minutos ou lag do event loop acima de 50 ms por 1 minuto. Desce uma máquina a cada 5 minutos com CPU abaixo de 20% por 15 minutos: ela sai do registro, espera 15 s (deadline de 8 s com margem), fecha os SSE com `retry` para o checkout reconectar em outra réplica com `Last-Event-ID`, e desliga. O contrato do Tuiuiú obriga aviso de campanha com 48 h, e uma hora antes o mínimo sobe para 50.

Até o Kubernetes do capítulo 35, a execução usa grupos de máquinas com autoscaling do provedor (o Auto Scaling group da AWS e o managed instance group do Google Cloud são exemplos), imagem imutável, 3 workers por máquina de 4 vCPU. A lista estática do capítulo 27 dá lugar a um registro de serviços: a máquina entra quando `/pronto` responde, sai ao drenar, e proxies e gateways releem a lista a cada 5 s. De 12 workers de madrugada a 50 na Black Friday, o laço se paga; carga estável se dimensiona fixa.

`temp-01`, `api-01` e `api-02` continuam fixos. A homologação ganhou 2 conexões no conector e máximo de 6 processos, numa conta que cabe num post-it, colado no monitor do Carlos.

Armadilhas e casos reais

> **Armadilha:** homologação que compartilha IP de saída, cota de parceiro, banco ou broker com produção é produção com outro nome e menos alarme. O teste do Carlos foi perfeito, inclusive em derrubar produção.

Partida a frio com estouro de manada: vinte máquinas novas carregam cache juntas, o stampede do capítulo 22 disparado pelo autoscaler.

Descer sem drenar. Cada descida vira uma pequena queda programada, que aparece no p99 como o mistério de toda hora cheia (o sinal de encerramento no processo é tema do capítulo 34).

Fator de pico emprestado. O do Tuiuiú vem de 12 meses de dados dele; o do slide veio da intuição do gerente, que também acha que o 49º consumidor de um tópico com 48 partições "ajuda em alguma coisa".

Resumo

- Estimativa: negócio → taxa de pico → custo por operação → limite; pelo pico; limite fixo de terceiro não multiplica.

- Vertical: teto de catálogo, um domínio de falha. Horizontal: exige serviço sem estado e paga coordenação (USL).
- Stateless é auditoria do que o processo guarda; limite por processo vezes N cresce sozinho.
- Autoscaling: laço com atraso; métrica de saturação, histerese, alvo com folga para o tempo de reação, agenda, máximo do recurso mais escasso, descida com drenagem.

A Lastro agora

A Lastro tem a conta do Tuiuiú: seis vezes o pico, 50 workers, shard próprio do ledger em máquina dobrada, 32 baldes, 48 partições, webhooks por Little, 24 meses online. Pagamentos, antifraude e entregador escalam sozinhos com alvo de 35%; o `conector-adquirente` fica em 10 processos fixos, e o adquirente voltou a atender o IP da Lastro. O fundador ganhou a máquina maior que pediu, para um shard só, e chamou isso de vitória.

Enquanto isso, a consultoria Java do Bazar publicou um repositório de exemplo para "ajudar a comunidade de integradores da Lastro". Está tudo lá: retry com full jitter, `Idempotency-Key` fora do laço (finalmente) e, no `application.properties`, a chave de API de produção do Bazar, com 40 horas de consultoria e nenhuma de revisão.

Referências

- Neil J. Gunther, *Guerrilla Capacity Planning*, Springer, 2007 (Lei Universal de Escalabilidade).
- Kubernetes, Horizontal Pod Autoscaling: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
- The Twelve-Factor App, VI. Processes: https://12factor.net/processes
- Apache Kafka, Modifying topics (efeito de acrescentar partições): https://kafka.apache.org/documentation/#basic_ops_modify_topic
- Google Cloud Blog, "Bringing Pokémon GO to life on Google Cloud" (2016): https://cloud.google.com/blog/products/gcp/bringing-pokemon-go-to-life-on-google-cloud

PARTE V

Segurança

CAPÍTULO 30

Autenticação

No sábado às 3h41, a Lastro recebeu a requisição mais educada da sua história, com chave de API válida, `Idempotency-Key` gerada fora do laço e retry com full jitter, e ela vinha de um ladrão.

A chave era a do Bazar, publicada na quinta às 18h no repositório de exemplo da consultoria Java. Em 33 horas, alguém achou a chave e leu a documentação da Lastro com mais atenção que a própria consultoria. Às 3h41, listou dois anos de cobranças. Às 3h44, cadastrou uma inscrição de webhook apontando para um servidor dele, que passou a receber cada cobrança nova com nome e CPF do comprador. Às 3h52, trocou a chave Pix de repasse de 18 vendedores e pediu um saque para cada um.

O limite do gerente para chave Pix cadastrada há menos de 24 h (capítulo 3), R$ 500,00 por dia, segurou o resto: saíram R$ 9.000,00 de R$ 212.400,00 pedidos. O rate limiting do capítulo 28 não viu nada, porque o ladrão respeitou os limites do Bazar com uma disciplina que o Bazar nunca teve. O Bazar abriu chamado às 9h12: "URGENTE!!!! vendedores dizendo que a chave Pix mudou sozinha". O vendedor 42 estava entre os 18 e jurou que dessa vez nem ele sabia.

Às 9h30, o Carlos gerou uma chave nova para o Bazar. A antiga parou na hora, e a integração do Bazar também: a chave estava em 14 servidores que só a consultoria sabia mexer. Foram 1h50 sem cobranças do Bazar, faturadas pela consultoria em 6 horas.

A Lastro autenticava cada requisição. Por que isso não protegeu nada, e o que muda quando a credencial vaza, como toda credencial um dia vaza?

Quem é você, e por que o servidor acredita

Autenticação é estabelecer quem fez a requisição; **autorização**, o que esse alguém pode fazer, é o capítulo 31. A autenticação se apoia em fatores: algo que você sabe (senha), algo que você tem (chave privada, celular) e algo que você é (biometria, que no WebAuthn só destrava uma chave local e nunca sai do aparelho).

A distinção que mais pesa no backend é sobre a credencial que viaja. Uma credencial **ao portador** (*bearer*) autentica quem a apresenta: chave de API, cookie de sessão, access token comum. Quem copia, vira você. Uma credencial com **prova de posse** obriga o cliente a provar, a cada uso, que tem uma chave privada que nunca trafega: mTLS (capítulo 10), DPoP (RFC 9449), WebAuthn. Copiar o que passou no fio não serve para nada.

A chave do Bazar era ao portador, sem prazo, sem escopo e única. Quem leu o GitHub virou o Bazar, com todos os direitos, inclusive o de derrubar o Bazar.

Sessão com cookie, a tecnologia que ninguém acha moderna e quase todo mundo deveria usar

No login, o servidor gera um identificador aleatório de pelo menos 128 bits, guarda no banco ou no Redis a linha `(sessao_id, usuario, criada_em, ultimo_uso)` e devolve o identificador num cookie. Cada requisição traz o cookie, o servidor busca a linha, e o usuário está autenticado.

Os atributos do cookie fazem metade da segurança. `HttpOnly` esconde o cookie do JavaScript, e um XSS não consegue lê-lo. `Secure` só manda por HTTPS. `SameSite=Lax` ou `Strict` impede o navegador de mandá-lo em requisições disparadas por outro site, a defesa principal contra CSRF. O prefixo `__Host-` obriga `Secure`, `Path=/` e nenhum `Domain`, e um subdomínio comprometido não consegue sobrescrevê-lo.

Revogar é apagar a linha. Logout, troca de senha, "sair de todos os aparelhos": um `DELETE`, a única vez na carreira em que um `DELETE` em produção é a coisa certa, e mesmo assim o Carlos pediu dois aprovadores. O custo é uma consulta por requisição, que um Redis responde em menos de um milissegundo.

Tokens opacos e JWT, ou onde mora a verdade

Um **token opaco** é uma string aleatória sem significado, como o ID de sessão, que o servidor consulta numa tabela ou num endpoint de introspecção (RFC 7662). Revogação imediata; uma consulta por uso.

Um **JWT** (RFC 7519) carrega os dados dentro dele, assinado. São três partes em base64url separadas por ponto: cabeçalho, payload e assinatura (JWS, RFC 7515). Decodificado:

```
cabeçalho   {"alg":"ES256","typ":"JWT","kid":"2026-11"}
payload     {"iss":"https://auth.lastro.com.br","sub":"mkt_tuiuiu",
             "aud":"https://api.lastro.com.br","scope":"cobrancas:escrita",
```

```
              "iat":1793000000,"exp":1793000600,"jti":"tok_01JC8..."}
assinatura  ECDSA P-256 sobre base64url(cabeçalho) + "." + base64url(payload)
```

Quem valida confere a assinatura com a chave pública do emissor (publicada num JWKS, escolhida pelo `kid`) e checa `exp`, `nbf`, `iss` e `aud` sem consultar ninguém. Esse é o ganho: validação local, sem estado, em qualquer réplica. O payload é só codificado, não cifrado: qualquer um lê. JWT guarda identidade e permissões, nunca segredo, e quem põe CPF no payload publica CPF com assinatura digital.

O `alg` decide o modelo de chave. HS256 é HMAC com segredo compartilhado: quem valida também consegue emitir, então serve só quando emissor e validador são o mesmo sistema. ES256, EdDSA e RS256 são assinaturas assimétricas: o validador tem só a chave pública e não forja nada.

O preço da validação local é a revogação. Um JWT assinado vale até o `exp`, e o servidor que não consulta nada não tem como saber que ele foi roubado. As saídas: prazo curto (5 a 15 minutos) com um **refresh token** opaco, guardado no servidor, que renova o access token e pode ser revogado; rotação do refresh token a cada uso, com detecção de reuso (se o antigo aparecer de novo, alguém copiou, e a família inteira é revogada); e, para emergência, uma lista de `jti` ou de `sub` bloqueados, consultada pelo validador. A lista reintroduz estado, só que pequeno e só para os condenados. JWT de 30 dias sem lista é a credencial que o próprio dono não consegue cassar, a estabilidade de servidor público em formato base64.

OAuth 2.0 e OpenID Connect

O **OAuth 2.0** (RFC 6749) é um protocolo de delegação: um **cliente** obtém de um **servidor de autorização** um access token para chamar um **servidor de recursos**, em nome de um **dono do recurso** ou de si mesmo. Ele define como o token é obtido, e quase nada do formato do token. Os fluxos atuais, consolidados na RFC 9700 (o BCP de segurança do OAuth, de janeiro de 2025):

- **Authorization code com PKCE:** para usuário num navegador ou app. O usuário se autentica no servidor de autorização, que devolve ao cliente um código de uso único, trocado por tokens num canal direto.
- **Client credentials:** máquina falando com máquina, sem usuário. O cliente se autentica no endpoint de token e recebe um access token.
- **Device authorization** (RFC 8628): aparelho sem teclado; o usuário aprova em outro.

O fluxo implícito (token na URL de retorno) e o de senha (cliente recebe a senha do usuário) estão descontinuados pela RFC 9700: o primeiro vaza token em histórico e log; o segundo treina o usuário a digitar a senha em qualquer formulário que peça com educação.

O **PKCE** (RFC 7636) amarra o código a quem o pediu:

```
code_verifier   = 43 a 128 caracteres aleatórios, fica no cliente
code_challenge  = BASE64URL(SHA-256(code_verifier)), vai no pedido de autorização
troca do código = código + code_verifier; o servidor recalcula o hash e compara
```

Quem intercepta o código no redirecionamento não tem o `code_verifier` e não troca nada. A RFC 9700 exige PKCE de clientes públicos e o recomenda também aos que têm segredo.

No client credentials, o cliente pode se autenticar com segredo compartilhado (`client_secret`, ao portador), com um JWT assinado pela própria chave privada (`private_key_jwt`, RFC 7523) ou com certificado (mTLS, RFC 8705). A RFC 8705 ainda permite amarrar o access token ao certificado: o servidor de recursos recusa o token se a conexão não vier com o mesmo certificado. Token roubado sem a chave privada vira papel de parede.

O **OpenID Connect** (OIDC) é uma camada de autenticação sobre o OAuth. O OAuth diz "este cliente pode ler as cobranças"; o OIDC diz "quem está no navegador é a Maria, autenticada às 9h02 com MFA". Ele acrescenta o **ID token**, um JWT para o cliente (`aud` = cliente) com `sub`, `auth_time`, `amr` e `nonce` (contra replay), e a descoberta em `/.well-known/openid-configuration`. Access token vai para a API; ID token fica com o cliente. Inverter os dois é a confusão mais popular da área e já rendeu login com o token de outro aplicativo.

MFA e passkeys, ou como parar de confiar na memória humana

Senha é um segredo que o usuário escolhe mal, repete em vinte sites e digita em qualquer página parecida com a certa. O armazenamento correto (Argon2id) é do capítulo 33. O resto do problema se resolve com um segundo fator.

TOTP (RFC 6238) gera o código com `HMAC(segredo, ⌊tempo / 30 s⌋)` truncado em 6 dígitos, a partir de um segredo compartilhado no QR code do cadastro. Funciona sem rede e tem um defeito estrutural: uma página falsa pede o código e o repassa ao site legítimo em segundos. **SMS** soma a isso a troca fraudulenta de chip (*SIM swap*). **Push** soma a fadiga: mande quarenta notificações às 2h e alguém aprova uma só para dormir.

As **passkeys** são credenciais WebAuthn (W3C) com FIDO2: no cadastro, o autenticador (celular, notebook, chave física) gera um par de chaves para aquele site, guarda a privada e entrega a pública ao servidor, que a armazena com o ID da credencial. No login, o servidor manda um desafio aleatório; o autenticador, depois de verificar o usuário (biometria ou PIN, local), assina o desafio junto com a **origem** que o navegador informa. O servidor confere assinatura, desafio e origem.

A origem é o ponto. O navegador escreve a origem real nos dados assinados, e a credencial de `lastro.com.br` não assina nada para `lastro-login.com.br`. O phishing deixa de funcionar por construção, sem depender de o usuário ler a barra de endereço às 23h. O servidor não guarda segredo nenhum: quem rouba a tabela de chaves públicas leva uma coleção de cadeados, sem nenhuma chave.

As credenciais dos marketplaces

As opções para máquina falando com a API:

Credencial	Revogação	Se vazar	Custo
Chave de API estática	Consulta por requisição	Vale até alguém notar	Mínimo
Client credentials + `client_secret`	Token curto; segredo revogável	Ladrão emite tokens até a revogação	Endpoint de token
Client credentials + `private_key_jwt`	Idem; chave pública removível	Nada trafega para ser copiado	Cliente assina JWT
mTLS com token amarrado (RFC 8705)	Certificado e token	Token inútil sem a chave	Operar certificados

A Lastro fica com duas, por versão da API, porque o `/v1` não pode quebrar clientes (capítulo 11).

No `/v1`, a chave continua e muda de natureza. O formato passa a ser reconhecível por máquina:

```
lastro_live_  +  43 caracteres base62 (256 bits aleatórios)  +  _  +  CRC32 em base62
└─ prefixo ─┘                                                        └─ checksum ─┘
```

O prefixo e o checksum servem aos programas de varredura de segredos (o *secret scanning* do GitHub é um exemplo): a Lastro cadastra o padrão, e uma chave publicada gera aviso em minutos, com a confirmação local pelo checksum filtrando falso positivo.

Aviso de chave `live` revoga automaticamente e manda e-mail ao marketplace. Revogar derruba a integração, mas a chave pública já é do ladrão, e 33 horas foi quanto o Bazar demorou sem aviso.

A chave é mostrada uma vez e guardada só como SHA-256. Para 256 bits aleatórios, um hash rápido basta: não há dicionário para atacar, e o Argon2 do capítulo 33 serve para segredo de baixa entropia, como senha. Cada marketplace pode ter **duas chaves ativas**, com data de último uso: gera a nova, troca nos servidores, revoga a antiga quando o último uso dela parar. A rotação de sábado teria sido sem queda.

Cada chave tem **escopos**. A padrão não inclui `repasses:escrita` nem `webhooks:escrita`, as duas operações que o ladrão usou; trocar chave Pix de repasse e cadastrar inscrição de webhook exigem uma chave separada, que pode ser restrita a IPs de origem, e geram e-mail ao marketplace e ao vendedor afetado. O gateway, que já identificava o marketplace pela credencial (capítulo 28), mantém em memória o hash das ~200 chaves ativas e recebe revogações por notificação: chave revogada morre nos gateways em segundos.

No `/v2`, a credencial é OAuth client credentials em `auth.lastro.com.br/token`, com `private_key_jwt` ou mTLS, e nenhum segredo compartilhado. O access token é o JWT acima: ES256, `aud` da API, escopos, 10 minutos. O gateway valida localmente pelo JWKS, sem consulta por requisição, o que pesa a 6.840 requisições por segundo com o Tuiuiú (capítulo 29). Revogação de emergência: o cliente desativado não obtém token novo, e o `sub` entra numa lista no gateway até o último `exp`. O Tuiuiú entra direto no `/v2` com mTLS e tokens amarrados ao certificado, porque o contrato dele exige credencial que não possa ser copiada de um repositório. O `/v2` com chave estática seria a escolha certa só para clientes sem capacidade de assinar um JWT, e ainda não apareceu nenhum.

O token curto do checkout (capítulo 9) vira um JWT com `aud` do checkout, escopo de leitura de uma única cobrança e `exp` igual à expiração do QR code. Vazou? Mostra o status de um Pix por meia hora.

O painel dos vendedores

O painel usava e-mail, senha e um cookie de sessão sem prazo. O vendedor 42 tinha a mesma senha no painel e no fórum de capinhas.

A sessão continua no servidor, em Redis, com cookie `__Host-sessao`, `HttpOnly`, `Secure`, `SameSite=Lax`, ociosidade de 30 minutos e prazo absoluto de 12 horas. JWT no `localStorage` foi descartado: qualquer XSS o lê, e revogar exigiria a lista de bloqueio,

que é uma sessão no servidor com passos extras. Login com **passkey** como opção principal, TOTP como alternativa, e segundo fator obrigatório. SMS não entra.

As operações que movem dinheiro pedem **reautenticação recente**: trocar chave Pix de repasse ou sacar acima de R$ 1.000,00 exige passkey ou TOTP nos últimos 5 minutos, mesmo com sessão válida. Troca de senha, de chave Pix ou de fator apaga todas as sessões do vendedor. Os usuários dos marketplaces entram no painel deles por OIDC com o provedor de identidade do próprio marketplace, exigência do Tuiuiú, e o `amr` do ID token tem que mostrar MFA.

Armadilhas e casos reais

> **Na vida real:** em 2015, Tim McLean mostrou que várias bibliotecas de JWT aceitavam `"alg": "none"` (token sem assinatura) e que outras, configuradas com chave pública RSA, aceitavam um token HS256 assinado usando essa chave pública como segredo HMAC. A biblioteca deixava o token escolher o algoritmo pelo qual seria validado: o réu escolhia o juiz. A defesa é fixar o algoritmo esperado na validação.

Não validar `aud`. Um token emitido para o serviço de relatórios, aceito pelo de pagamentos, transforma uma credencial de leitura em credencial de tudo. Todo validador confere `iss`, `aud`, `exp` e o algoritmo.

Comparar segredo com `==`. A comparação de strings para no primeiro byte diferente e vaza o prefixo pelo tempo de resposta. `hmac.compare_digest` compara em tempo constante; comparar hashes também resolve.

Logar o cabeçalho `Authorization`. O log vira um repositório público com controle de acesso pior que o GitHub (capítulo 38).

Fadiga de MFA. Em setembro de 2022, a Uber relatou que o invasor obteve a senha de um terceirizado e mandou pedidos de aprovação repetidos até um ser aceito. Push sem número para conferir é um botão "deixa entrar" com vibração.

Segredo em repositório. Em outubro de 2022, a Toyota informou que um terceirizado tinha publicado no GitHub, em dezembro de 2017, código com uma chave de acesso a um servidor de dados do T-Connect, exposta por quase cinco anos, com até 296.019 e-mails de clientes. O estudo "How Bad Can It Git?" (NDSS 2019) achou segredos vazando em mais de 100 mil repositórios públicos, com milhares de segredos novos por dia. A defesa que escala é a credencial que avisa quando vaza, e a que não vale nada fora da máquina dona.

Resumo

- Autenticação estabelece quem; autorização, o que pode (cap. 31). Credencial ao portador vale para quem a copia; prova de posse exige chave privada que não trafega.
- Sessão no servidor: ID aleatório, cookie `HttpOnly`, `Secure`, `SameSite`, `__Host-`; revogação imediata.
- JWT: assinado, legível, validado localmente; `alg` fixo, `aud`, `exp`; revogação por prazo curto, refresh token rotativo e lista de bloqueio.
- OAuth 2.0: authorization code com PKCE, client credentials, device; implícito e senha descontinuados (RFC 9700). `private_key_jwt` e mTLS dispensam segredo compartilhado. OIDC acrescenta o ID token.
- TOTP e SMS são phishable; passkeys (WebAuthn) assinam a origem e resistem a phishing.
- Chave de API: prefixo e checksum para varredura, hash, duas ativas para rotação, escopos.

A Lastro agora

O `/v1` tem chaves `lastro_live_` com checksum, cadastradas em varredura de segredos, guardadas como hash, duas por marketplace e com escopos que separam repasses e webhooks; o `/v2` usa client credentials com `private_key_jwt` ou mTLS e JWT de 10 minutos validado no gateway, e o Tuiuiú entra assim. O painel tem sessão no servidor, passkey ou TOTP obrigatórios e reautenticação para mover dinheiro. O Bazar ganhou a segunda chave e, pela primeira vez, trocou uma sem abrir chamado. A consultoria cobrou 12 horas para aprender a usar duas chaves.

Na segunda, a analista do Hornero, devidamente autenticada com passkey e MFA, abriu uma cobrança no painel, trocou um caractere do ID na URL por curiosidade e viu uma cobrança do Bazar, com o nome do comprador. A Lastro agora sabe exatamente quem ela é. Só não perguntou o que ela pode ver.

Referências

- RFC 6749, The OAuth 2.0 Authorization Framework: https://www.rfc-editor.org/rfc/rfc6749
- RFC 9700, Best Current Practice for OAuth 2.0 Security: https://www.rfc-editor.org/rfc/rfc9700

- RFC 7636, Proof Key for Code Exchange (PKCE): https://www.rfc-editor.org/rfc/rfc7636
- RFC 7519, JSON Web Token: https://www.rfc-editor.org/rfc/rfc7519
- RFC 7515, JSON Web Signature: https://www.rfc-editor.org/rfc/rfc7515
- RFC 7523, JWT Profile for OAuth 2.0 Client Authentication: https://www.rfc-editor.org/rfc/rfc7523
- RFC 8705, OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens: https://www.rfc-editor.org/rfc/rfc8705
- RFC 9449, OAuth 2.0 Demonstrating Proof of Possession (DPoP): https://www.rfc-editor.org/rfc/rfc9449
- RFC 7662, OAuth 2.0 Token Introspection: https://www.rfc-editor.org/rfc/rfc7662
- RFC 8628, OAuth 2.0 Device Authorization Grant: https://www.rfc-editor.org/rfc/rfc8628
- RFC 6238, TOTP: https://www.rfc-editor.org/rfc/rfc6238
- OpenID Connect Core 1.0: https://openid.net/specs/openid-connect-core-1_0.html
- W3C, Web Authentication Level 3: https://www.w3.org/TR/webauthn-3/
- GitHub Docs, Secret scanning partner program: https://docs.github.com/en/code-security/secret-scanning/secret-scanning-partnership-program/secret-scanning-partner-program
- Michael Meli, Matthew R. McNiece, Bradley Reaves, "How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories", NDSS 2019.
- Tim McLean, "Critical vulnerabilities in JSON Web Token libraries", Auth0, 2015.

CAPÍTULO 31

Autorização e multitenancy

A analista do Hornero fez o *pen test* mais barato da história da Lastro: zero reais, uma tecla e o tempo de esfriar o café.

Na segunda às 10h40, ela viu na barra de endereço do painel `/painel/cobrancas/18734512` e, por curiosidade, trocou o 2 final por 3. Apareceu um fone de R$ 349,90 vendido pelo Bazar, com nome, CPF e e-mail do comprador. Trocou por 4: uma capinha do Bazar. Ao lado, um botão "Estornar", ativo, laranja, implorando. Ela não clicou. Por educação, e porque o crochê estava no colo.

Às 10h52, ela abriu chamado. O Bazar soube às 11h30 e abriu o seu: "URGENTE!!!! O CONCORRENTE ESTÁ VENDO NOSSAS VENDAS??????". Estava. O painel usava o ID interno `BIGINT` da cobrança, sequencial, e a rota de detalhe, escrita pelo Carlos "para o suporte interno", carregava a cobrança pelo ID e nada mais. Depois virou a tela dos marketplaces, com a mesma consulta e um logotipo novo.

A auditoria da tarde achou mais um. No GraphQL do painel (capítulo 12), a consulta de topo `cobranca(id:)` conferia o marketplace. O campo aninhado `vendedor { cobrancas }` não. A Capinhas Hiper vende no Bazar e no Hornero com o mesmo CNPJ, `UNIQUE` desde o capítulo 1, e o painel do Hornero listava as vendas dela no Bazar, com paginação por cursor. A Lastro tinha construído uma integração entre concorrentes melhor que a de qualquer cliente.

A autenticação do capítulo 30 funcionou: a Lastro sabia, com passkey e MFA, exatamente quem estava vendo o que não devia. Onde mora a pergunta "e ela pode?", e como garantir que nenhuma consulta esquecida faça de novo um marketplace ler o outro?

Autorização, a pergunta que vem depois do crachá

Autorização é a decisão sobre uma tupla: um **sujeito** (usuário, chave, serviço) quer executar uma **ação** sobre um **recurso**, num **contexto** (horário, origem, autenticação recente). Sem regra aplicável, a resposta é negar. *Deny by default* é a única configuração em que esquecer uma regra gera um chamado de "não consigo ver", e não um chamado do Bazar.

A literatura separa quem decide de quem obedece. O **ponto de decisão** (PDP, *policy decision point*) avalia a política; o **ponto de aplicação** (PEP, *policy enforcement point*) é o código que pergunta e barra. Os nomes vêm do XACML, padrão em XML que pouca gente usa e todo mundo cita. A aplicação precisa estar em cada caminho até o dado, e é ali que mora o bug. O painel da Lastro tinha política. Faltava aplicação em duas rotas.

RBAC, ou como cinco papéis viram quatrocentos

No **RBAC** (*role-based access control*), permissões são atribuídas a papéis, e papéis a usuários (Ferraiolo e Kuhn, 1992; ANSI INCITS 359-2004). A checagem é uma pertinência: algum papel do usuário contém `estorno:criar`? Barata, auditável ("quem pode estornar?" é um `JOIN`) e cabe num token: o `scope` do capítulo 30 é RBAC de pobre.

O RBAC responde sobre tipos de ação, e não sobre objetos. "Suporte pode estornar cobranças do próprio marketplace até R$ 500,00" tem dois atributos do recurso na regra, e o RBAC puro só expressa isso criando papéis: `suporte_bazar_ate_500`, `suporte_hornero_ate_500_exceto_domingo`. É a **explosão de papéis**, que termina com mais papéis que funcionários e o Carlos pedindo `super_suporte` "por segurança".

ABAC e ReBAC, ou a política que olha o objeto

No **ABAC** (*attribute-based access control*, NIST SP 800-162), a política é uma expressão sobre atributos do sujeito, do recurso, da ação e do ambiente:

```
permitir estorno:criar se
    sujeito.marketplace_id == recurso.marketplace_id
  e "suporte" em sujeito.papeis
  e recurso.valor <= 50000                    -- centavos
  e ambiente.agora - sujeito.ultimo_fator <= 5 min
```

Expressa qualquer coisa, e cobra por isso: o PDP não decide antes de carregar a cobrança, a política vira um programa com bugs de programa, e "quem pode estornar esta cobrança?" exige avaliar a expressão para todo mundo. Linguagens dedicadas (Rego, do Open Policy Agent, e Cedar, da AWS, são exemplos) tornam a política testável fora da aplicação.

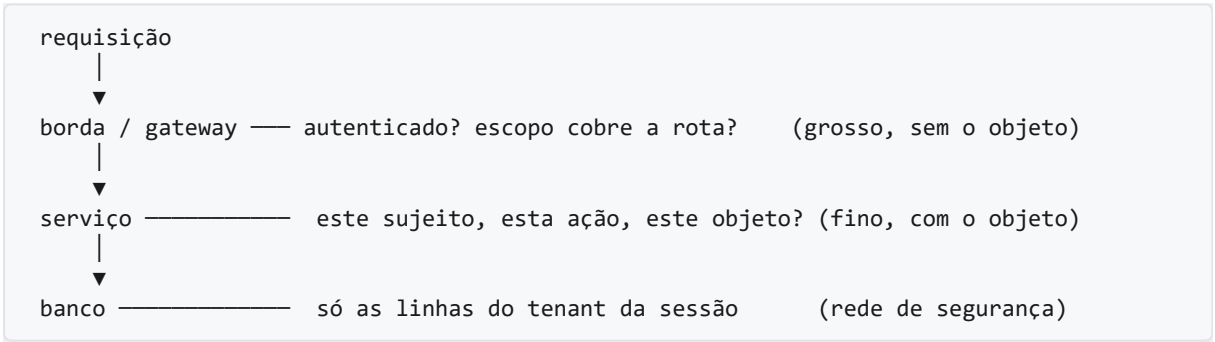
No **ReBAC** (*relationship-based access control*), a permissão deriva de relações: a cobrança pertence ao marketplace, o usuário é membro do marketplace, logo pode ler a cobrança. A referência é o Zanzibar, do Google (USENIX ATC 2019): tuplas `objeto#relação@sujeito`, checagem como alcançabilidade num grafo.

O paper dedica uma seção ao **problema do novo inimigo**: Alice tira o ex da pasta compartilhada e depois põe uma foto nova nela. Se a checagem ler uma réplica onde a remoção ainda não chegou e a foto já, o ex vê a foto. O Google resolveu com o *zookie*, um token que amarra a checagem a um instante pelo menos tão novo quanto a última mudança de permissão: o read-your-writes do capítulo 19 aplicado ao ex, no único paper de sistemas distribuídos com motivação de novela.

O custo do ReBAC é um serviço com estado próprio na frente de toda leitura, tão disponível quanto o caminho mais crítico. E há a pergunta difícil, dos três modelos: "liste o que eu posso ver". Filtrar 9 milhões de cobranças exige traduzir a política num `WHERE`, ou o sistema lê tudo e descarta.

Modelo	Decide com	Pergunta "quem pode?"	Custo principal
RBAC	papéis do sujeito	barata	explosão de papéis
ABAC	atributos de sujeito, recurso, ambiente	cara	política vira programa; precisa do recurso
ReBAC	grafo de relações	razoável	serviço com estado e consistência própria
## Onde a checagem mora			

A política pode ser perfeita e ainda vazar, porque a aplicação acontece em camadas, e cada uma enxerga uma parte do problema.



O gateway só decide o que cabe na credencial. O serviço é o único lugar com sujeito e objeto ao mesmo tempo. O banco filtra linhas por tenant mesmo quando o código esquece.

Três regras saem daí. **Listagens filtram na consulta**, nunca depois: buscar 50 e descartar as alheias devolve páginas com 3 itens e um cursor que vaza o resto. **Objeto de outro tenant responde `404`**, igual ao inexistente: um `403` confirma que a cobrança

existe, e enumerar é metade do roubo. **O tenant vem da credencial**, nunca da requisição: um `?marketplace_id=` que o cliente consegue mandar é uma sugestão do atacante.

A última regra vale entre serviços. Se o ledger aceita com permissão total tudo que vem de pagamentos, pagamentos vira um **confused deputy**: um intermediário com privilégio que executa, para quem não tem, o que este não poderia fazer sozinho. O termo é de Norm Hardy (1988), sobre um compilador que sobrescreveu o arquivo de faturamento do sistema porque o usuário pediu com jeitinho. A defesa é propagar o sujeito original num formato que o intermediário não consiga forjar.

IDOR, o ataque que cabe numa tecla

O que a analista fez é **IDOR** (*insecure direct object reference*), que a lista de API da OWASP de 2023 chama de **BOLA** (*broken object level authorization*) e põe em primeiro lugar. A rota recebe um identificador, carrega o objeto e não verifica a relação do sujeito com ele. A URL é um formulário que o usuário pode editar, e o usuário edita.

Trocar o ID sequencial por UUID dificulta a enumeração, e só. Um UUIDv7 tem 74 bits aleatórios, mas IDs vazam em webhooks, logs, links de e-mail e no print que o vendedor 42 mandou para o Bazar. Com um ID alheio em mãos, a rota sem checagem abre do mesmo jeito. UUID é ofuscação; a correção é a checagem.

O ID sequencial ainda conta: cria uma cobrança na segunda, outra na terça, subtrai, e o concorrente sabe o volume diário da Lastro. Na Segunda Guerra, estatísticos aliados estimaram pelos números de série capturados uma produção de 246 tanques alemães por mês, contra 1.400 da inteligência convencional; os registros alemães mostraram 245. ID sequencial é relatório de faturamento com acesso público.

Isolamento entre tenants

Um **tenant** é um cliente cujos dados e operações precisam ficar isolados dos outros clientes no mesmo sistema. Na Lastro, cada marketplace. Há quatro modelos de isolamento, do mais separado ao mais compartilhado:

Modelo	Isolamento	Custo por tenant	Migração de schema	Consulta entre tenants
Banco por tenant	físico, inclusive recursos	alto (instância, réplicas, conexões)	N vezes	entre bancos

Modelo	Isolamento	Custo por tenant	Migração de schema	Consulta entre tenants
Schema por tenant	lógico, mesmo processo	médio (catálogo cresce)	N vezes	`UNION` de schemas
Coluna de tenant	só na consulta	~zero	1 vez	um `WHERE`
Coluna + row-level security	consulta, aplicado pelo banco	~zero	1 vez	papel autorizado

O **banco por tenant** isola até o vizinho barulhento, ao custo de instância, réplicas e pool por cliente e da mesma migração N vezes, com N chances de uma falhar no meio. O **schema por tenant** isola nomes, e nada de CPU ou I/O; depende de `search_path` por sessão, que um pool em modo transaction mistura entre clientes. A **coluna de tenant** é a mais barata e a que vazou na segunda: o isolamento é tão bom quanto a consulta mais desleixada do sistema, e o sistema tem o Carlos.

Row-level security, o WHERE que o Carlos não consegue esquecer

O **row-level security** (RLS) do PostgreSQL (desde a versão 9.5) acrescenta a cada consulta, dentro do planejador, um predicado definido numa política da tabela. A aplicação esquece o `WHERE`; o banco põe.

```
ALTER TABLE cobrancas ENABLE ROW LEVEL SECURITY;
ALTER TABLE cobrancas FORCE ROW LEVEL SECURITY;

CREATE POLICY por_marketplace ON cobrancas
  USING (marketplace_id =
         nullif(current_setting('app.marketplace_id', true), '')::bigint);
```

`USING` filtra as linhas visíveis; `WITH CHECK`, que omitido repete o `USING`, valida o que se grava. Com RLS ligado e nenhuma política, a tabela não mostra nada a ninguém: deny by default de fábrica. A consulta da segunda-feira, com o tenant do Hornero na sessão:

```
lastro=> BEGIN;
lastro=> SET LOCAL app.marketplace_id = '7';   -- Hornero
lastro=> SELECT count(*) FROM cobrancas WHERE id = 18734513;
 count
-------
     0
```

Os detalhes que separam RLS de teatro:

Quem escapa. Superusuários e papéis com `BYPASSRLS` ignoram as políticas; o dono da tabela também, sem `FORCE ROW LEVEL SECURITY`. Aplicação que conecta como dona das tabelas tem RLS só no diagrama.

Pool em modo transaction. O PgBouncer (capítulo 4) entrega a sessão do servidor ao próximo cliente, e um `SET` sem `LOCAL` deixa o tenant do Hornero para a transação seguinte, que pode ser do Bazar: o motel do capítulo 4 agora entrega o quarto com a bagagem do hóspede anterior. `SET LOCAL` termina com a transação. Depois disso, `current_setting` devolve string vazia, e não `NULL`; sem o `nullif`, a conversão de `''` para `bigint` derruba a consulta. Fecha sem vazar, ao menos.

Views executam com os privilégios do dono, e herdam o desprezo dele pelo RLS; desde o PostgreSQL 15, `security_invoker = true` aplica as políticas de quem consulta.
Restrições `UNIQUE` e chaves estrangeiras enxergam todas as linhas: uma violação de unicidade revela que outro tenant tem aquele valor, e a defesa é pôr o tenant em toda chave única de negócio.

Desempenho. O predicado entra no plano como um `WHERE`: precisa de índice, e a coluna do tenant no índice para o index-only scan sobreviver. Tabelas particionadas aplicam as políticas da mãe quando consultadas por ela; partição acessada direto usa as próprias, que começam vazias.

Na Lastro: marketplaces, vendedores e quem pode o quê

Os requisitos: nenhum marketplace lê dado de outro, mesmo com consulta esquecida; os 42 marketplaces (41 mais o Tuiuiú) diferem em tamanho por mais de mil vezes; a conciliação lê todos; o vendedor pode pertencer a vários.

Modelo de tenant. Coluna `marketplace_id` em toda tabela de dado de cliente, com RLS como segunda camada, em `pg-pagamentos` e em cada shard de `pg-ledger`. Banco por tenant daria 42 primários com três réplicas e 42 migrações por mudança, para um isolamento de recursos que o ledger já tem onde importa (o Tuiuiú tem shard próprio desde o capítulo 29); seria o certo com poucos clientes enormes e exigência contratual de separação física. Schema por tenant traria o custo de migração sem isolar recurso. Os lançamentos herdam o `marketplace_id` da transação, inclusive nas contas da Lastro (liquidante e receita).

Papéis no banco.

Papel	RLS	Usado por
`dono_schema`	dono das tabelas, `FORCE` ativo	só migrações

Papel	RLS	Usado por
`app_pagamentos`, `app_ledger`	aplica; sem `BYPASSRLS`	serviços, `SET LOCAL` no início de toda transação
`conciliacao`	política própria `USING (true)`, só `SELECT`	conciliação das 6h, relatórios internos
`quebra_vidro`	`BYPASSRLS`, temporário	emergência, com aprovação de duas pessoas, 1 hora, auditado

O fundador perdeu o superusuário que tinha desde `temp-01`, a pendência do capítulo 1, quando o tabelião descobriu que tinha um chefe capaz de desligá-lo. Ganhou o `quebra_vidro`, como todo mundo, e disse que no tempo dele o banco confiava nas pessoas. Confiava, e a planilha do financeiro tem uma aba só para isso.

Contexto propagado. O gateway autentica (capítulo 30), descarta cabeçalhos internos vindos de fora e emite um **token de contexto**: JWT ES256 de 60 segundos com `marketplace_id`, sujeito, papéis e instante do último fator. Pagamentos o repassa ao ledger e ao antifraude, que tiram o tenant dele, nunca de um parâmetro. No painel, o tenant vem do emissor OIDC configurado para o marketplace, e não de uma claim: o IdP é do cliente, e com claim um marketplace emitiria para si um token dizendo ser o Bazar, com a Lastro assinando embaixo.

Política. RBAC para o tipo de ação, ABAC para o objeto, relações em tabela:

Papel (usuário do marketplace)	Pode
`leitura`	ver cobranças, extratos e vendedores do marketplace
`suporte`	+ estornar até R$ 500,00
`financeiro`	+ estornar sem limite, com fator nos últimos 5 minutos
`admin`	+ gerir usuários, chaves e inscrições de webhook

Os papéis vêm dos grupos do IdP do marketplace. Vendedores entram pela relação `afiliacoes (vendedor_id, marketplace_id)` e veem os próprios recebíveis, nunca a cobrança inteira. Um serviço estilo Zanzibar foi descartado: as relações têm dois níveis e cabem em tabelas; ele faria sentido com compartilhamento arbitrário ou hierarquias profundas.

Aplicação. A política mora numa biblioteca dos serviços. Rota sem ação declarada responde `403`, e a CI falha se achar uma. Os repositórios só oferecem buscas com tenant, `buscar(marketplace_id, id)`; a versão sem tenant deixou de existir. O DataLoader do

GraphQL passou a ter chave `(marketplace_id, vendedor_id)`. Objeto alheio responde `404` RFC 9457, `type` `nao-encontrado`. O painel trocou o ID interno pela UUIDv7 pública: a checagem corrigiu o IDOR, e a troca só fechou a contagem de tanques.

Mais de 20 negações para o mesmo sujeito em 5 minutos dispara alerta, porque enumeração lenta passa por baixo do rate limiting do capítulo 28. O índice do extrato no ledger ganhou `marketplace_id` no `INCLUDE`; sem ele, a soma do trigger de débito perdia o index-only scan (0,6 ms a mais em homologação).

A equipe da Lastro acessa um marketplace por **personificação**: leitura no tenant, vinculada a um chamado, 2 horas, CPF mascarado e um registro que o marketplace vê no painel. O Carlos pediu personificação com escrita, por segurança. Está no backlog, ao lado dos 1.024 baldes.

Armadilhas e casos reais

> **Na vida real:** em maio de 2019, Brian Krebs noticiou que a First American Financial, seguradora de títulos imobiliários dos Estados Unidos, expunha cerca de 885 milhões de documentos de clientes, de 2003 em diante, com contas bancárias e documentos de identidade. Bastava mudar o número na URL de um documento. Sem login. O IDOR mais caro dos livros não precisou nem do crochê.

Checar na tela e não na API: esconder o botão "Estornar" no React não impede ninguém de mandar o `POST`. Checar o pai e esquecer o filho: `GET /cobrancas/{id}/estornos/{estorno_id}` que carrega o estorno sem conferir se pertence àquela cobrança é IDOR com sobrenome. Tenant fora da chave de cache: `painel:resumo:{dia}` serve o resumo do Bazar ao Hornero (o Carlos tinha criado uma).

Função `SECURITY DEFINER` no caminho: executa com os privilégios do dono, que ignora RLS, e abre a tabela inteira para quem puder chamá-la.

Resumo

- Autorização decide sobre sujeito, ação, recurso e contexto, negando por padrão; o bug costuma estar no ponto de aplicação.
- RBAC explode quando a regra depende do objeto; ABAC precisa do recurso; ReBAC exige serviço com consistência própria (novo inimigo).
- Borda decide o grosso, serviço checa o objeto, banco filtra o tenant. Listagem filtra na consulta, objeto alheio é `404`, tenant vem da credencial, sujeito propagado contra o

confused deputy.

- IDOR/BOLA: UUID não substitui checagem; ID sequencial vaza volume.
- Isolamento por banco, schema, coluna ou coluna com RLS.
- RLS: `FORCE`, `BYPASSRLS`, `SET LOCAL` com pool, `security_invoker`, `UNIQUE` vaza existência, tenant no índice.

A Lastro agora

Cada marketplace é um tenant tirado da credencial, com `marketplace_id` em toda tabela e RLS em `pg-pagamentos` e nos shards do ledger. Os serviços conectam sem `BYPASSRLS` e fazem `SET LOCAL` do tenant em cada transação; só a conciliação lê tudo, e só lê. Um token de contexto de 60 segundos atravessa os serviços, e os papéis do marketplace vêm do IdP dele. O fundador entrou na fila do `quebra_vidro`, e o tabelião, pela primeira vez desde o capítulo 1, não tem ninguém acima dele com um interruptor.

A analista do Hornero recebeu agradecimento formal e voltou ao crochê. Na quarta, o `admin` de um marketplace novo, autenticado com passkey, autorizado pelo papel certo, no tenant certo, com RLS conferindo cada linha, cadastrou uma inscrição de webhook. A política permitiu, corretamente. Ninguém perguntou para onde apontava a URL `http://169.254.169.254/latest/meta-data/`.

Referências

- David Ferraiolo, Richard Kuhn, "Role-Based Access Controls", 15th National Computer Security Conference, 1992.
- ANSI INCITS 359-2004, Role Based Access Control.
- NIST SP 800-162, Guide to Attribute Based Access Control (ABAC) Definition and Considerations: https://csrc.nist.gov/pubs/sp/800/162/upd2/final
- Ruoming Pang et al., "Zanzibar: Google's Consistent, Global Authorization System", USENIX ATC 2019: https://www.usenix.org/conference/atc19/presentation/pang
- Norm Hardy, "The Confused Deputy", ACM SIGOPS Operating Systems Review, 1988.
- OWASP API Security Top 10 2023, API1:2023 Broken Object Level Authorization: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/
- PostgreSQL 16, Row Security Policies: https://www.postgresql.org/docs/16/ddl-rowsecurity.html

- PostgreSQL 16, CREATE VIEW (`security_invoker`): https://www.postgresql.org/docs/16/sql-createview.html
- Open Policy Agent, Rego: https://www.openpolicyagent.org/docs/latest/policy-language/
- Cedar Policy Language: https://docs.cedarpolicy.com/
- Richard Ruggles, Henry Brodie, "An Empirical Approach to Economic Intelligence in World War II", Journal of the American Statistical Association, 1947.
- Brian Krebs, "First American Financial Corp. Leaked Hundreds of Millions of Title Insurance Records", KrebsOnSecurity, 24 maio 2019: https://krebsonsecurity.com/2019/05/first-american-financial-corp-leaked-hundreds-of-millions-of-title-insurance-records/

CAPÍTULO 32

Vulnerabilidades de backend

O marketplace mais novo da Lastro passou na análise comercial porque o CNPJ existia, e ficou 14 minutos como cliente antes de virar o atacante mais bem-sucedido da história da empresa.

Na quarta às 14h05, o `admin` da Pechincha Relâmpago, autenticado com passkey e autorizado pelo papel certo no tenant certo (capítulo 31), cadastrou uma inscrição de webhook com a URL `http://169.254.169.254/latest/meta-data/iam/security-credentials/`. Às 14h06, apertou "Testar endpoint", um botão que o Carlos fez no ano passado para reduzir chamados: ele faz um `GET` na URL a partir do servidor e mostra no painel o status e os primeiros 2 KB da resposta. A resposta foi uma linha, `lastro-maquina`, o nome do papel de nuvem das máquinas. Às 14h07, o admin cadastrou a mesma URL com `lastro-maquina` no fim, testou de novo e recebeu no painel, com formatação bonita, um JSON com chave de acesso, segredo e token de sessão.

O endereço `169.254.169.254` é o serviço de metadados da instância, que no exemplo da AWS (o Google Cloud e o Azure usam o mesmo IP) entrega às máquinas as credenciais temporárias do papel delas. Ele só responde a quem está dentro da máquina. O botão do Carlos estava dentro da máquina.

As máquinas do grupo de autoscaling usam a mesma imagem e o mesmo papel, porque "`temp-01` é a imagem" (capítulo 29), e o papel podia ler o bucket dos relatórios exportados, porque o serviço de relatórios precisava. Às 14h19, de um IP fora da nuvem, as credenciais listaram o bucket e baixaram 3 relatórios mensais de 2 marketplaces, com CPF dos compradores. Às 14h31, o detector de anomalias do provedor alertou que credenciais de máquina estavam sendo usadas de fora da nuvem. Às 14h40, o Carlos invalidou as sessões emitidas antes daquele minuto e desativou o botão, que ele defendeu como "a feature mais usada do painel". Era, por uma margem de 2 KB.

A autenticação e a autorização funcionaram. O ataque entrou por uma funcionalidade legítima, usada por um cliente legítimo, do jeito que ela foi construída. Quantas outras portas assim o backend da Lastro tem, e como achar todas antes da próxima Pechincha?

As listas da OWASP, ou o ranking anual das mesmas dez formas de apanhar

A OWASP (Open Worldwide Application Security Project) publica duas listas que servem de mapa. O **OWASP Top 10** classifica riscos de aplicações web a partir de dados de testes; a edição de 2025 tem, em ordem, controle de acesso quebrado, configuração insegura, cadeia de suprimentos de software, falhas criptográficas, injeção, design inseguro, autenticação, integridade de software e dados, log e alerta, e tratamento de condições excepcionais. O SSRF, categoria própria em 2021, foi absorvido pelo primeiro item: servidor que acessa o que o usuário não poderia é controle de acesso quebrado com intermediário.

O **OWASP API Security Top 10** (2023) olha só para APIs, e a lista muda de cara:

API 2023	Risco	Onde este livro trata
API1	autorização por objeto (BOLA)	capítulo 31
API2	autenticação quebrada	capítulo 30
API3	autorização por propriedade (BOPLA)	aqui
API4	consumo irrestrito de recursos	capítulo 28 e aqui
API5	autorização por função (BFLA)	aqui
API6	acesso irrestrito a fluxos de negócio sensíveis	capítulos 3 e 30 (limites de saque)
API7	SSRF	aqui
API8	configuração insegura	aqui
API9	inventário de APIs desatualizado	capítulo 11
API10	consumo inseguro de APIs de terceiros	aqui

As listas mudam de ordem a cada edição e o conteúdo mal mexe, porque os erros são os mesmos desde que o primeiro estagiário concatenou uma string num SQL.

SSRF, o servidor que faz o que o atacante pede

SSRF (*server-side request forgery*) acontece quando o servidor faz uma requisição para um destino controlado pelo usuário. O atacante não alcança a rede interna; o servidor alcança, e o atacante empresta o servidor. Toda funcionalidade que recebe URL é candidata: webhook, importação por link, preview de imagem, "testar conexão".

Os alvos de sempre:

- o serviço de metadados da nuvem, que entrega credenciais;
- `localhost`, onde moram painéis de administração, a porta de métricas e o Redis sem senha que "só escuta local";
- serviços internos sem autenticação, protegidos pela crença de que ninguém de fora chega neles;
- outros esquemas: `file:///etc/passwd`, `gopher://`, conforme a biblioteca.

Validar a URL por texto falha de quatro jeitos. **Codificações alternativas**: `http://2852039166/` é `169.254.169.254` em decimal, e `http://[::ffff:169.254.169.254]/` é o mesmo IPv4 embrulhado em IPv6. **Redirecionamentos**: a URL validada responde `302` para o metadado, e a biblioteca segue (o `requests` segue por padrão; o `httpx`, não). **DNS rebinding**: o nome resolve para um IP público na validação e, com TTL zero, para `127.0.0.1` na conexão, segundos depois. **Nome que aponta para dentro**: `meta.atacante.com` com registro A `169.254.169.254`, sem truque nenhum.

A defesa que fecha os quatro trabalha com o endereço, e não com o texto: resolver o nome, rejeitar qualquer endereço fora do espaço global (RFC 1918, loopback, link-local `169.254.0.0/16` e `fe80::/10`, CGNAT `100.64.0.0/10`, IPv6 ULA `fc00::/7`, IPv4 mapeado em IPv6 depois de desembrulhado), **conectar no IP validado** em vez de resolver de novo, mandar o nome original no `Host` e no SNI, e não seguir redirecionamento. O módulo `ipaddress` do Python faz a classificação; o resto é disciplina.

Na rede, a defesa é a segunda camada: o processo que faz requisições para URLs de terceiros roda num lugar de onde a rede interna e o metadado não são alcançáveis. No metadado, a versão 2 do serviço da AWS (IMDSv2, de 2019) exige um token obtido por `PUT` com cabeçalho próprio e permite limitar o TTL do pacote de resposta a um salto, o que derruba a maioria dos SSRF simples, que só controlam a URL de um `GET`. O Google Cloud exige o cabeçalho `Metadata-Flavor: Google` com o mesmo objetivo.

E a regra que o botão do Carlos violou com entusiasmo: a resposta de uma requisição feita a mando do usuário não volta para o usuário. SSRF sem resposta visível (*blind*) ainda permite varrer portas e disparar ações, mas não lê credencial em JSON formatado.

Injeção, a mais velha da casa

Injeção é dado do usuário interpretado como código por algum interpretador: SQL, shell, template, LDAP, cabeçalho HTTP. O mecanismo é sempre a concatenação.

```
# vulnerável: o valor vira parte do SQL
cur.execute(f"SELECT id FROM vendedores WHERE nome = '{nome}'")
```

```
# nome = "x' OR '1'='1"  →  devolve todos os vendedores

# parametrizado: o valor viaja separado do texto da consulta
cur.execute("SELECT id FROM vendedores WHERE nome = %s", (nome,))
```

Com psycopg 3, a consulta parametrizada usa o protocolo estendido do PostgreSQL: texto e valores viajam em campos separados, e o servidor nunca analisa o valor como SQL. Nome de coluna e direção de ordenação (`?ordem=valor desc`) não são parametrizáveis: vão por lista de permitidos ou por `psycopg.sql.Identifier`. ORMs protegem até alguém usar `text()` com f-string "só desta vez", que é a frase que antecede todo incidente de injeção desde 1998.

As outras injeções seguem o mesmo padrão. `subprocess.run(f"convert {arquivo}", shell=True)` entrega o shell para quem nomeou o arquivo `x; rm -rf /`; a lista de argumentos sem `shell=True` resolve. Template montado com entrada do usuário no Jinja2 executa Python (SSTI). E existe a injeção que atravessa sistemas: um campo que começa com `=`, exportado para CSV e aberto numa planilha, vira fórmula. A planilha do financeiro, a fonte da verdade desde o capítulo 1, também é um interpretador.

Deserialização insegura, ou o pickle que executa

Deserializar é reconstruir objetos a partir de bytes. Formatos que reconstroem objetos arbitrários executam código durante a reconstrução. O `pickle` do Python é o exemplo canônico, e a documentação dele avisa no topo, em caixa de alerta, que o módulo é inseguro contra dados maliciosos:

```
import os, pickle

class Presente:
    def __reduce__(self):
        return (os.system, ("id",))

carga = pickle.dumps(Presente())   # o atacante manda isto
pickle.loads(carga)                # o servidor abre o presente
```

```
uid=1000(app) gid=1000(app) groups=1000(app)
```

O `__reduce__` diz ao `pickle` qual chamável usar para reconstruir o objeto, e `pickle.loads` chama. Validação depois chega ao velório: o código já rodou. O mesmo vale para `yaml.load` com o carregador completo do PyYAML (o seguro é `yaml.safe_load`) e para `jsonpickle`. Assinar o pickle com HMAC só troca o problema de lugar: quem tiver a chave executa código em todos os consumidores.

Dado que atravessa fronteira de confiança vai em formato de dados (JSON, Protocol Buffers) e é validado contra um schema na entrada. O Pydantic, que o FastAPI já usa, faz isso por padrão nas requisições; o perigo mora nos lugares que ninguém chama de entrada: mensagens do broker, cache no Redis, arquivos de parceiros, colunas `jsonb`.

Autorização por propriedade e por função

O capítulo 31 fechou a autorização por objeto. A OWASP separa mais duas.

BOPLA (*broken object property level authorization*) junta dois erros. **Atribuição em massa** (*mass assignment*): o endpoint aceita o objeto inteiro e grava todo campo que vier, inclusive os que o cliente não deveria mexer. **Exposição excessiva**: a resposta serializa o objeto inteiro e confia que o front-end esconda o resto. A defesa é ter modelos diferentes para entrada e saída, com listas explícitas de campos: o modelo de entrada do Pydantic com `extra="forbid"` recusa o campo inesperado, e o `response_model` do FastAPI corta o que não estiver declarado na saída.

É o endpoint que devolve o vendedor com o hash da senha "porque o React só mostra o nome". O React mostra só o nome. O `curl` mostra tudo, e o `curl` não tem gerente de produto.

BFLA (*broken function level authorization*) é a rota administrativa sem checagem de papel, achada porque o caminho é previsível (`/admin`) ou porque está no OpenAPI público, documentada com exemplo e botão "Try it out".

Dependências, o código que você não escreveu e que roda com seus privilégios

Um serviço FastAPI típico tem dezenas de dependências diretas e mais de cem transitivas, todas com as permissões do processo. O risco vem por três caminhos. **Vulnerabilidade conhecida**: o PyPA Advisory Database (consultado pelo `pip-audit`) lista o CVE, e o serviço segue com a versão afetada porque "está funcionando". **Pacote malicioso**: nome parecido com um popular (*typosquatting*) ou mantenedor comprometido. **Confusão de dependência**: um pacote interno `lastro-utils` sem índice privado bem configurado, e alguém publica `lastro-utils` versão 99 no PyPI; o instalador escolhe a versão mais alta.

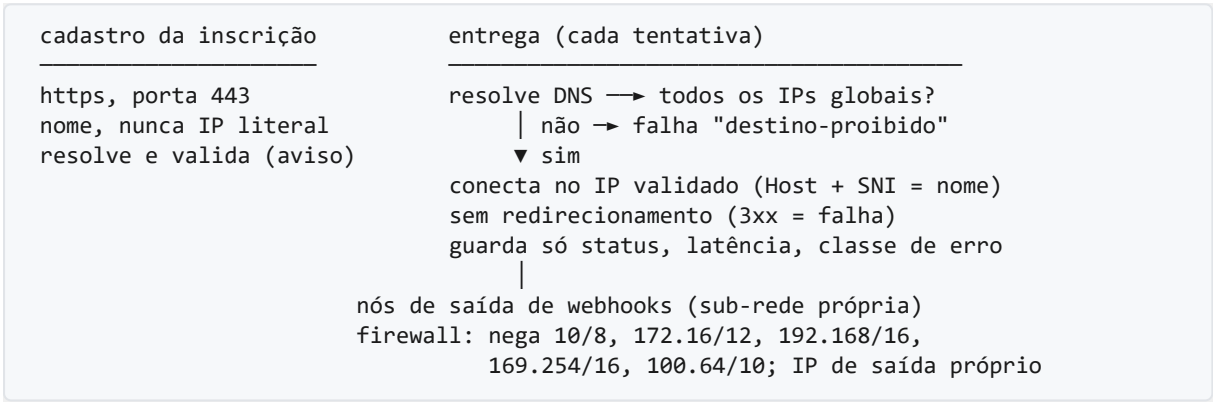
As defesas são mecânicas. Lock com versões exatas e hashes (`pip install --require-hashes`, ou o lock do uv ou do Poetry): a build instala os bytes revisados, e pacote trocado no índice falha na verificação. Índice privado como fonte única. Auditoria de CVEs que

falha a build. Um SBOM (lista de componentes) por imagem, para responder em minutos "onde usamos a biblioteca X?" quando sair o próximo CVE com logotipo.

Na Lastro: a superfície de ataque revisada

A revisão percorreu, contra as duas listas, cada ponto por onde dado externo entra ou a Lastro sai. O requisito: nenhum dado controlado por cliente, comprador ou parceiro alcança credencial, rede interna ou interpretador sem uma checagem que independa da boa vontade de quem mandou.

Webhooks, a porta que a Pechincha abriu



A validação no cadastro dá erro amigável; a que protege é a da entrega, porque o DNS muda depois. O "Testar endpoint" voltou como um evento `webhook.teste` por `POST`, pelo caminho das entregas, mostrando só status e latência. A resposta, o marketplace lê no log do próprio servidor, onde ela sempre deveria estar.

Os entregadores foram para **nós de saída** dedicados, numa sub-rede sem rota para a rede interna nem para o metadado, com IP de saída próprio, separado do `203.0.113.10` liberado pelo adquirente e pelo banco (capítulos 27 e 29): uma inscrição maliciosa que faça um destino bloquear a Lastro bloqueia os webhooks, e não as autorizações. O desenho das sub-redes é do capítulo 36.

Todas as máquinas passaram a exigir token no metadado, com limite de um salto, e cada grupo de máquinas ganhou papel próprio e mínimo: o dos entregadores não lê bucket nenhum. O fundador perguntou por que a máquina precisava de papel, se no tempo dele a chave ficava num arquivo. O arquivo está no `temp-01`, e o capítulo 33 vai visitá-lo.

O resto da superfície

Entrada ou saída	Risco (lista)	Decisão
API pública e GraphQL	BOPLA (API3)	modelos de entrada `extra="forbid"`, saída por `response_model`; achado: `PATCH` de vendedor no painel aceitava `tarifa_pix` porque reusava o modelo inteiro
Rotas internas e de operação	BFLA (API5)	prefixo `/interno` só no proxy local, nunca no gateway; ausente do OpenAPI público
Ordenação e filtros	injeção (A05)	colunas por lista de permitidos; CI recusa f-string dentro de `execute` e de `text()`
Eventos do broker, Redis, `jsonb`	deserialização (A08)	só JSON com schema Pydantic na leitura; `pickle` proibido por regra de lint; achado: cache do painel no Redis usava `pickle`
Arquivos da conciliação, respostas de parceiros	consumo inseguro (API10)	resposta do adquirente validada por schema e tamanho máximo; CSV lido como texto, sem avaliação
Exportação para planilha	injeção de fórmula	célula que começa com `=`, `+`, `-`, `@`, tab ou CR ganha um apóstrofo na frente
Corpo de requisição, upload, XML	consumo de recursos (API4)	corpo até 1 MB no gateway; XML de parceiro só com parser sem entidades externas (`defusedxml`)
Dependências	cadeia de suprimentos (A03)	lock com hashes, índice privado único, `pip-audit` na CI, SBOM por imagem
Erros	configuração (A02), exceções (A10)	RFC 9457 sem stack trace nem SQL; erro em checagem de segurança nega (fail closed)

O achado da planilha: um vendedor com o nome `=HYPERLINK("http://...","Clique aqui")` aparecia no relatório de repasses que o financeiro exporta toda segunda. Era o vendedor 42, testando "se o sistema aceitava fórmula" com a desenvoltura de quem já sacou dinheiro de cinco abas. Aceitava.

O último item da tabela pegou um clássico do capítulo 20: um `except: pass` do Carlos em volta da verificação de assinatura das notificações do banco parceiro. Assinatura inválida levantava exceção, a exceção era engolida, e a notificação seguia. Fail open, com data de 2024 e comentário `# temporario`.

Armadilhas e casos reais

> **Na vida real:** em julho de 2019, o Capital One anunciou o vazamento de dados de cerca de 100 milhões de pessoas nos Estados Unidos e 6 milhões no Canadá. Um firewall de aplicação mal configurado, rodando numa máquina da AWS, permitiu um SSRF até o serviço de metadados, que entregou as credenciais do papel da máquina; o papel lia os buckets onde os dados estavam. O roteiro da Pechincha, com seis zeros a mais. A AWS lançou o IMDSv2 em novembro do mesmo ano.
>
> **Na vida real:** em 2017, a Equifax expôs dados de cerca de 147 milhões de pessoas por uma vulnerabilidade do Apache Struts (CVE-2017-5638) cuja correção tinha sido publicada em março; a invasão começou em maio. A dependência estava no inventário de risco de todo mundo, menos no dela.

Em 2021, Alex Birsan publicou em índices públicos pacotes com os nomes de dependências internas de grandes empresas (Apple, Microsoft e PayPal entre elas) e conseguiu execução de código nas redes delas, recebendo recompensas de bug bounty em vez de processos. Em março de 2024, um mantenedor do xz-utils, depois de dois anos de contribuições, inseriu um backdoor no código de release que atingia o `sshd` em algumas distribuições Linux (CVE-2024-3094). Foi descoberto por um engenheiro que estranhou meio segundo a mais no login. Lock com hashes não pega mantenedor malicioso; só garante que todo mundo instale o mesmo veneno, de forma reproduzível.

Bloquear o metadado por string (`169.254.169.254`) e esquecer o decimal, o IPv6 e o DNS do atacante: a lista para o único atacante que não precisava ser parado.

Resumo

- OWASP Top 10 (2025) e API Security Top 10 (2023) como lista de verificação; SSRF agora em controle de acesso.
- SSRF: validar o endereço resolvido e conectar nele, sem redirecionamento; saída isolada; metadado com token; resposta não volta ao usuário.
- Injeção: parâmetros, identificadores por lista, argumentos sem shell; CSV vira fórmula.
- Deserialização: `pickle` e `yaml.load` executam código; JSON com schema na fronteira.
- BOPLA e BFLA: modelos explícitos de entrada e saída; rotas internas fora da borda.
- Dependências: lock com hashes, índice privado, auditoria na CI, SBOM.

A Lastro agora

Webhooks saem de nós dedicados, sem rota para dentro, conectando só em IPs globais validados na hora da entrega, sem seguir redirecionamento e sem devolver a resposta a ninguém. Cada grupo de máquinas tem papel próprio e mínimo, o metadado exige token, e a CI recusa f-string em SQL, `pickle` e dependência sem hash. O financeiro abriu a planilha de repasses e viu, pela primeira vez, uma fórmula no nome do vendedor 42 aparecer como texto. Ficou desconfiado mesmo assim.

Para descobrir por onde as credenciais da Pechincha tinham passado, o Carlos fez uma busca nos logs de todos os serviços por trechos do token roubado. Não achou o token. Achou, no log do `conector-adquirente`, uma linha de depuração esquecida desde a Black Friday com o corpo inteiro da requisição de autorização. Dezesseis dígitos, validade e nome do titular, repetidos algumas centenas de milhares de vezes, em texto puro, num bucket que o papel antigo de todas as máquinas conseguia ler.

Referências

- OWASP Top 10:2025: https://top10.owasp.org/2025/
- OWASP API Security Top 10 2023: https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- OWASP, Server Side Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP, CSV Injection: https://owasp.org/www-community/attacks/CSV_Injection
- AWS, "Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 Instance Metadata Service", novembro 2019: https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/
- Google Cloud, About VM metadata: https://cloud.google.com/compute/docs/metadata/overview
- Python, `pickle` (aviso de segurança): https://docs.python.org/3/library/pickle.html
- Python, `ipaddress`: https://docs.python.org/3/library/ipaddress.html
- Psycopg 3, Passing parameters to SQL queries: https://www.psycopg.org/psycopg3/docs/basic/params.html
- pip, Secure installs (hash-checking mode): https://pip.pypa.io/en/stable/topics/secure-installs/
- pip-audit: https://github.com/pypa/pip-audit

- Capital One, "Information on the Capital One Cyber Incident", 2019: https://www.capitalone.com/digital/facts2019/
- U.S. Government Accountability Office, "Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach", GAO-18-559, 2018: https://www.gao.gov/products/gao-18-559
- Alex Birsan, "Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies", 2021: https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610
- NVD, CVE-2024-3094 (xz-utils): https://nvd.nist.gov/vuln/detail/CVE-2024-3094

CAPÍTULO 33

Criptografia aplicada e segredos

O Carlos foi procurar um token roubado nos logs e achou um cartão de crédito. Depois outro. Quando parou de contar, eram 318 mil autorizações de 204 mil cartões, com número, validade e nome, em texto puro, num bucket que toda máquina da Lastro lia até a semana passada.

A linha vinha do `conector-adquirente`: um `logger.debug(corpo)` ligado na Black Friday "só pra ver o que o adquirente estava recebendo". O adquirente estava recebendo cartões. O bucket também. O CVV escapou porque o log cortava o campo, o único acerto do sistema, cometido sem querer por uma biblioteca.

O PCI DSS, o padrão de segurança da indústria de cartões, cobre todo sistema que armazena, processa ou transmite dados de cartão e todo sistema conectado a eles. O número do cartão (PAN, *primary account number*) passava pelo gateway, por pagamentos, pelo conector, pelo coletor de logs e pelo bucket: o escopo da auditoria virou quase a Lastro inteira, `temp-01` incluído. O auditor pediu o inventário de software do `temp-01`. O fundador mandou uma foto do monitor.

Como guardar e transportar o que não pode vazar, para que o vazamento entregue só lixo?

Hash de senha, ou a arte de ser lento de propósito

Senha não se guarda, nem cifrada: guarda-se um hash, e o login compara hashes. Cifrar senha significa existir uma chave que devolve todas elas, morando no mesmo servidor, numa variável chamada `CHAVE_SENHAS`.

Mas não qualquer hash. Uma GPU de jogo calcula SHA-256 na casa das dezenas de bilhões por segundo, e todas as variações de `Capinha@2024` caem antes do café esfriar. Hash de senha usa **sal** aleatório por usuário (os milhões de `123456` ficam diferentes e tabela pré-calculada morre), **custo de tempo** ajustável e **custo de memória**, porque GPU tem milhares de núcleos e pouca RAM por núcleo.

O **bcrypt** (1999) tem custo exponencial (`cost` 12 = 2^12 rodadas) e uma surpresa: só os primeiros 72 bytes da senha contam. O **Argon2id** venceu a Password Hashing Competition em 2015, está na RFC 9106 e tem três parâmetros: memória `m`, passadas `t`, paralelismo `p`. A RFC sugere `m` = 64 MiB, `t` = 3, `p` = 4 para pouca memória; o mínimo da OWASP é `m` = 19 MiB, `t` = 2, `p` = 1. É a primeira função da história projetada para ser lenta e gorda de propósito, e o fundador se sentiu representado. O resultado carrega tudo:

```
$argon2id$v=19$m=65536,t=3,p=4$Wm9yM2tOcFhxQmM0ZE1aVw$q8H3vYp0mK2tR7sLxN4cWbE9jF1uGzAiDo
SeTkQhVnU
          versão  parâmetros     sal (base64)          hash (base64)
```

Subir o custo é gradual: cada login bem-sucedido regrava com os parâmetros novos. Existe ainda a **pimenta** (*pepper*), um segredo fora do banco aplicado como HMAC antes do hash: se só o banco vazar, nenhum hash é atacável. Sim, hash de senha leva sal e pimenta. O Carlos perguntou se tinha sobremesa, e a resposta é o Argon2, que leva 64 MiB e meia hora de explicação.

AES-GCM, ou cifrar sem autenticar é deixar a porta encostada

Dado que precisa voltar a ser lido, como o PAN na hora de autorizar, usa criptografia simétrica: AES (FIPS 197), chave de 256 bits. Quem decide se presta é o **modo de operação**. O ECB cifra cada bloco de 16 bytes isoladamente, e blocos iguais viram cifras iguais: o pinguim do Linux cifrado em ECB continua um pinguim, exposto na Wikipédia há anos como réu primário. CBC e CTR escondem padrão, mas não detectam adulteração: em CTR, inverter um bit da cifra inverte o mesmo bit do texto, e o atacante muda um valor sem saber a chave.

Daí o **AEAD** (*authenticated encryption with associated data*), o mesmo do TLS 1.3 (capítulo 10). O **AES-GCM** (NIST SP 800-38D) cifra em modo contador e calcula uma etiqueta de 128 bits sobre a cifra e os **dados associados** (AAD), um contexto público amarrado à mensagem. Um bit alterado em qualquer dos dois e a decifração falha inteira:

```python
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

aes = AESGCM(AESGCM.generate_key(bit_length=256))
nonce = os.urandom(12)                                  # 96 bits, nunca repetir
cifra = aes.encrypt(nonce, b"4111111111111111", b"tok_7Hq2")
aes.decrypt(nonce, cifra, b"tok_9Xz1")                  # AAD de outro registro
```

```
cryptography.exceptions.InvalidTag
```

O AAD é o identificador do registro: quem copia no banco a cifra do cartão A para a linha do cartão B ganha um `InvalidTag`. Sem AAD, o sistema decifraria feliz o cartão errado para a cobrança errada.

O preço do GCM é o *nonce*, 96 bits que nunca se repetem com a mesma chave. O nome é *number used once*, a especificação inteira cabe no nome, e mesmo assim tem gente que usa duas vezes. Nonce repetido entrega o XOR dos dois textos e a chave de autenticação, com a qual o atacante forja mensagens. Com nonce aleatório, a SP 800-38D limita a 2^32 cifragens por chave; com contador, basta um processo reiniciar e recomeçar do zero, coisa que processos fazem toda vez que o kernel os mata. O ChaCha20-Poly1305 (RFC 8439) é o AEAD alternativo, com a mesma regra.

Assimétrica e assinaturas, ou a chave que pode ir para o outdoor

Na assimétrica, a privada fica guardada e a pública pode ir para um outdoor. Ela serve para **troca de chaves** (o ECDH do capítulo 10) e para **assinatura**: só a privada assina, qualquer um verifica, e o verificador não ganha poder de assinar. Dado grande não se cifra com ela, lenta e limitada em tamanho: ela protege uma chave simétrica, que carrega o peso. Ed25519 (RFC 8032) e ECDSA P-256 são as opções comuns. O ECDSA tem a fragilidade do GCM com outro nome: cada assinatura usa um número secreto `k`, e dois `k` iguais revelam a chave privada com álgebra de ensino médio. O Ed25519 deriva o `k` da própria mensagem e protege o engenheiro de si mesmo, que é o único inimigo que nunca tira férias.

HMAC, ou como provar que foi você sem mostrar o RG

Com segredo compartilhado, assinatura assimétrica é exagero. O **HMAC** (RFC 2104) é `H((K ⊕ opad) ‖ H((K ⊕ ipad) ‖ m))`: só quem tem `K` produz o código de `m`. Os dois níveis têm motivo: o ingênuo `SHA-256(segredo ‖ m)` sofre *length extension*, e quem conhece esse hash calcula o de `segredo ‖ m ‖ enchimento ‖ qualquer_coisa` sem saber o segredo. Em 2009, Thai Duong e Juliano Rizzo forjaram assim assinaturas da API do Flickr, que usava MD5 nesse formato. O atacante assinou embaixo do que o Flickr tinha assinado, e o Flickr reconheceu firma.

A comparação é em tempo constante (`hmac.compare_digest`): o `==` para no primeiro byte diferente, e medindo milhões de respostas o atacante acha o código byte a byte. É lento e chato, como todo ataque que funciona.

Hash com chave também identifica um cartão sem guardar o número. O `SHA-256(PAN)` parece anônimo até a conta:

```
PAN de 16 dígitos  = BIN (6) + conta (9) + verificador Luhn (1)
painel mostra      = BIN + 4 últimos   (o máximo que o PCI DSS permite exibir)
falta descobrir    = 6 dígitos, e o Luhn elimina 9 em cada 10 candidatos
tentativas         = 100.000 SHA-256 por cartão  →  milissegundos numa CPU
```

Hash sem chave de espaço pequeno é só um jeito demorado de escrever o número, com mais etapas e o mesmo crime. Por isso o PCI DSS 4.0 exige hash de PAN com chave: HMAC com uma chave que o atacante não tem.

Envelope encryption e KMS, ou chave dentro de chave dentro de caixa

Cifrar desloca o problema para a chave. Chave num arquivo ao lado do banco: o atacante leva os dois e agradece.

Um **KMS** (*key management service*; o AWS KMS e o Cloud KMS do Google são exemplos) guarda chaves mestras em HSMs (*hardware security modules*) e nunca as exporta; cada operação passa por política e entra na auditoria. Como ele limita taxa e custa uma ida e volta de rede, a **envelope encryption** tira o volume de dentro dele:

1. A aplicação pede uma **chave de dados** (DEK) e recebe a DEK em claro e a DEK cifrada pela **chave mestra** (KEK).
2. Cifra localmente com a DEK, guarda a DEK cifrada ao lado e descarta a DEK em claro.
3. Para ler, manda a DEK cifrada ao KMS e decifra localmente com o que voltar.

Quem rouba o banco leva lixo. Rotacionar a KEK recifra as DEKs (kilobytes), e não os dados (terabytes). Apagar uma DEK torna ilegível tudo o que ela cifrou: o *crypto-shredding* prometido no capítulo 21. A chave mestra é a única coisa na Lastro que nunca saiu de casa, nem para uma reunião de alinhamento.

Segredos, ou onde a senha do banco passa as noites

Segredo no repositório deu o capítulo 30. Em variável de ambiente, é herdado por todo processo filho, legível em `/proc/<pid>/environ` pelo mesmo usuário e despachado junto no relatório de erro que "manda o ambiente para ajudar a depurar". O lugar dele é um **gerenciador de segredos** (o HashiCorp Vault ou o serviço equivalente do provedor): o

processo se autentica com a identidade da máquina (o papel do capítulo 32, sem "segredo zero" em disco), recebe o segredo na partida, guarda em memória, e cada leitura fica auditada.

Rotação é o que todo mundo adia, porque trocar a senha do banco com 60 processos conectados derruba alguma coisa. O padrão sem queda é o das chaves de API do capítulo 30, duas credenciais válidas: os processos usam o usuário A, a rotação troca a senha do B, espera todos relerem e só então troca a do A. Gerenciadores mais completos emitem **segredos dinâmicos**, um usuário criado na hora para aquele processo e apagado no prazo. Segredo vencido que vaza vale o mesmo que cupom de pizzaria de 2019.

Tokenização, ou o cartão no programa de proteção a testemunhas

Tokenização troca o dado sensível por um substituto aleatório: nenhuma conta leva do token ao PAN, e o único caminho é perguntar a um **cofre**, que guarda o mapeamento para o PAN cifrado e só responde a quem tem autorização. PAN cifrado continua sendo dado de cartão, e quem alcança a cifra e a chave fica no escopo; token fora do cofre é só uma string, e quem só vê token sai da auditoria. (Token com formato preservado, por cifra FPE do NIST SP 800-38G, traz a chave de volta ao escopo.)

Duas regras do PCI DSS 4.0 completam o quadro: dado de autenticação sensível (CVV, trilha, PIN) não pode ser guardado depois da autorização, nem cifrado; e o PAN exibido mostra no máximo o BIN e os quatro últimos dígitos. O cartão entra no programa de proteção a testemunhas, ganha nome novo, e quem pergunta pelo nome antigo precisa de crachá.

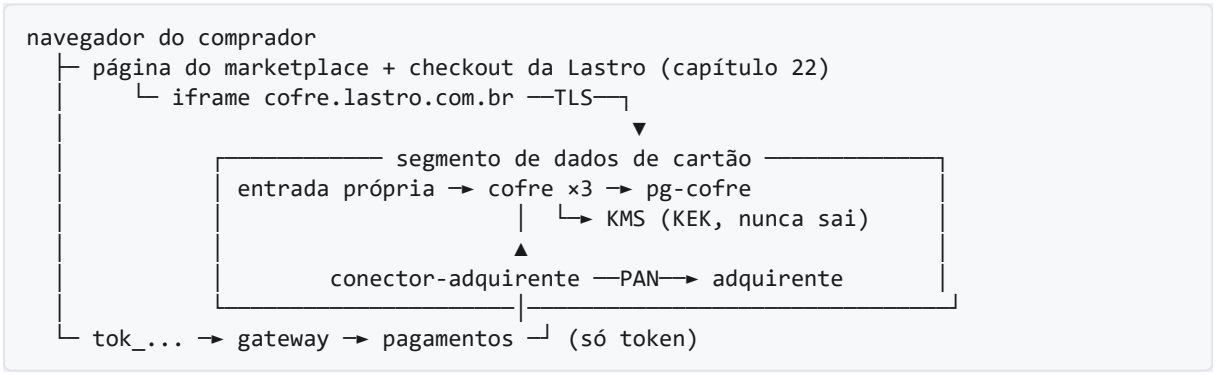
Na Lastro: o cofre, a fronteira e o resto do lixo

Primeiro, a limpeza: no bucket versionado, apagar criou só marcadores de exclusão, e as versões antigas saíram uma a uma, depois o índice do coletor e o backup semanal. A resposta seguiu o plano combinado com o adquirente, que fala com as bandeiras. O vendedor 42 achou o próprio cartão na amostra: tinha comprado capinha de si mesmo para testar o saque.

As alternativas para o cartão:

Opção	O que fica no escopo da Lastro	Cartão salvo	Troca de adquirente	Custo principal
Repassar sem guardar	gateway, pagamentos, conector, logs	não	sem impacto	o escopo de hoje
Token do adquirente	página de captura e conector	sim	cartões ficam presos ao adquirente	dependência
Cofre de terceiro	quase nada	sim	sim	taxa por transação e uma ida e volta externa na autorização
Cofre próprio	cofre, conector, a entrada deles	sim	sim	operar um segmento auditado

O contrato do Tuiuiú pede cartão salvo para recompra em um clique e cartões que sobrevivam a uma troca de adquirente, contingência que a Lastro precisa prever desde o bloqueio por IP do capítulo 29. Sobram as duas últimas linhas, e o cofre de terceiro põe uma chamada externa de 20 a 40 ms (hipótese) na folga de 120 ms do orçamento da autorização (capítulo 14), a 360 autorizações por segundo no pico. A Lastro fica com o **cofre próprio**. Com volume pequeno e sem equipe para operar um ambiente auditado, o de terceiro seria o certo.



O formulário de cartão roda num iframe de `cofre.lastro.com.br`, com entrada e TLS próprios, fora do alcance dos scripts da página do marketplace (outra origem). Gateway, pagamentos, broker, logs e `temp-01` passam a ver `tok_` mais 128 bits aleatórios. O `temp-01` saiu do escopo: primeira auditoria em que ele passa, e foi por não comparecer.

No `pg-cofre`, cada cartão guarda PAN e validade cifrados com AES-256-GCM (token no AAD), a DEK cifrada, BIN, quatro últimos e `marketplace_id` com RLS (capítulo 31). Cada processo pede uma DEK por hora: a 360 cifragens por segundo, cerca de 1,3 milhão por chave, longe das 2^32. DEKs decifradas ficam 5 minutos em memória; a KEK rotaciona por ano no KMS, com as DEKs recifradas em lote. O CVV nunca toca disco: Redis exclusivo do cofre, sem persistência, cifrado, 10 minutos, apagado na autorização.

O `conector-adquirente` muda para máquinas próprias no segmento e, a cada autorização, troca o token pelo PAN no cofre, monta a requisição e esquece: 2 ms a mais, medidos em homologação, pagos pela folga. Do cofre saem dois HMAC do PAN com chaves diferentes: a **impressão**, que responde "este cartão já está salvo neste marketplace?", e o identificador do antifraude prometido no capítulo 25, para os contadores e o grafo. A chave do antifraude, que fica fora do segmento, rotaciona por ano com os dois identificadores em paralelo por 24 horas, o prazo dos contadores.

O resto da casa:

- **Webhooks**: o cabeçalho reservado no capítulo 11 virou `Lastro-Assinatura: t=1762352400,v1=<hex>`, HMAC-SHA256 sobre `t + "." + corpo bruto`, no formato que a Stripe usa. Segredo por inscrição, prefixo `lastro_whsec_` (varredura do capítulo 30), `t` aceito por 5 minutos, dois `v1` durante a rotação. Ed25519 seria o certo se alguém precisasse provar a terceiros quem assinou.
- **Senhas do painel** (capítulo 30): Argon2id `m` = 64 MiB, `t` = 3, `p` = 4, ~50 ms por login, semáforo de 4 cálculos por processo, porque 40 logins juntos pediriam 2,5 GiB.
- **Segredos**: gerenciador lido pela identidade da máquina; senhas dos bancos em pares A/B, rotação a cada 30 dias.
- **Logs**: mascaramento no logger e detector no coletor de 13 a 19 dígitos que passam no Luhn e caem numa faixa de BIN (capítulo 38).

E o arquivo do fundador: `/root/senhas.txt` no `temp-01`, cabeçalho "NÃO APAGAR (backup)", com a credencial do adquirente, a senha da conta do DNS (inclusive a zona ACME do capítulo 10) e uma chave permanente da conta da nuvem criada em 2019, a única identidade da Lastro com mais poder que o fundador. Tudo foi rotacionado. Ele ainda quis passar `shred`, que em SSD grava a sobrescrita noutro bloco físico e deixa o original, como a própria documentação avisa. O `shred` foi feito mesmo assim, como missa de sétimo dia.

Armadilhas e casos reais

> **Na vida real:** em 2013, a Adobe vazou cerca de 150 milhões de registros com senhas cifradas em 3DES no modo ECB, sem sal, ao lado das dicas de senha em texto puro. Senhas iguais tinham cifras iguais, e a dica de um usuário entregava a senha de todos com a mesma cifra. O maior jogo de palavras cruzadas da história, com 150 milhões de jogadores involuntários.

> **Na vida real:** em dezembro de 2010, no 27C3, o grupo failoverflow mostrou que a Sony assinava o software do PlayStation 3 com ECDSA usando o mesmo `k` em toda assinatura. Duas assinaturas bastaram para calcular a chave privada da plataforma. Um console inteiro protegido por uma constante.
>
> **Armadilha:** verificar a assinatura do webhook depois de desserializar e serializar o JSON de novo. Ordem e espaços mudam, o HMAC não bate, e a verificação é desligada "temporariamente". A consultoria Java do Bazar já pediu 8 horas para entender por que o HMAC só falha quando o nome do vendedor tem acento.
>
> **Armadilha:** criptografia caseira: CBC com HMAC "na ordem certa", nonce gerado com `random` em vez de `secrets`. Em criptografia, "eu mesmo implementei" vai para o boletim de ocorrência, e não para o currículo.

Resumo

- Senha: hash lento com sal e custo de memória, Argon2id (RFC 9106) ou bcrypt (72 bytes); parâmetros no próprio hash.
- Simétrica é AEAD (AES-GCM, ChaCha20-Poly1305): AAD amarra o contexto, nonce nunca se repete.
- Assimétrica troca chaves e assina; `k` repetido no ECDSA entrega a chave privada.
- HMAC autentica com segredo compartilhado, sem length extension, comparado em tempo constante; hash sem chave de PAN é reversível.
- Envelope encryption: DEK local, KEK no KMS, rotação recifra só DEKs.
- Segredos num gerenciador, pela identidade da máquina, com rotação em pares.
- Tokenização tira do escopo do PCI DSS tudo o que só vê o token.

A Lastro agora

O cartão entra por um iframe do cofre, vira `tok_` e só volta a ser número dentro do segmento, por 2 ms, no `conector-adquirente`. Os webhooks saem assinados e o escopo do PCI DSS caiu de quase tudo para quatro componentes. O financeiro comentou que a planilha tokeniza cartões desde o primeiro dia: chama todo mundo de "cliente do final 4421".

Para subir o conector nas máquinas novas, o Carlos fez o primeiro contêiner da Lastro, com o Python iniciado por um script de shell "só pra carregar os segredos". No primeiro deploy, o `docker stop` mandou o `SIGTERM` educado, o shell não repassou, e dez segundos

depois veio o `SIGKILL` no meio de 31 autorizações. Na mesma tarde, o contêiner do painel sumiu sem uma linha de log: três workers, quatro Argon2id de 64 MiB cada, e um limite de memória de 512 MiB escolhido por segurança.

Referências

- RFC 9106, *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*: https://www.rfc-editor.org/rfc/rfc9106
- OWASP, *Password Storage Cheat Sheet*: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- NIST SP 800-38D, *Galois/Counter Mode (GCM) and GMAC*: https://csrc.nist.gov/pubs/sp/800/38/d/final
- NIST SP 800-38G, *Methods for Format-Preserving Encryption*: https://csrc.nist.gov/pubs/sp/800/38/g/final
- RFC 8439, *ChaCha20 and Poly1305 for IETF Protocols*: https://www.rfc-editor.org/rfc/rfc8439
- RFC 8032, *Edwards-Curve Digital Signature Algorithm (EdDSA)*: https://www.rfc-editor.org/rfc/rfc8032
- RFC 2104, *HMAC: Keyed-Hashing for Message Authentication*: https://www.rfc-editor.org/rfc/rfc2104
- Documentação do `cryptography`, AEAD: https://cryptography.io/en/latest/hazmat/primitives/aead/
- Documentação do Python, módulo `hmac`: https://docs.python.org/3/library/hmac.html
- PCI Security Standards Council, PCI DSS v4.0.1 (biblioteca de documentos): https://www.pcisecuritystandards.org/document_library/
- Stripe, verificação de assinatura de webhooks: https://docs.stripe.com/webhooks
- T. Duong e J. Rizzo, *Flickr's API Signature Forgery Vulnerability* (2009).
- fail0verflow, *Console Hacking 2010*, 27C3 (dezembro de 2010).
- Análises públicas do vazamento da Adobe (2013), 3DES em modo ECB e dicas de senha.

PARTE VI

Infraestrutura e operação

CAPÍTULO 34

Contêineres

O primeiro contêiner da Lastro durou onze minutos em produção e matou 31 autorizações, o que dá uma autorização a cada 21 segundos, ritmo de serial killer com metas trimestrais.

O Carlos empacotou o `conector-adquirente` para as máquinas do segmento de cartão (capítulo 33) com `CMD ./iniciar.sh`, um script que fazia `carregar_segredos && python -m conector`. No primeiro deploy, o `docker stop` mandou `SIGTERM`, esperou os dez segundos padrão sem reação e mandou `SIGKILL`. As 31 autorizações em voo ficaram em `processando` e a resolução do capítulo 15 fechou uma a uma com o adquirente. Nenhum centavo se perdeu; 31 compradores encararam um "aguarde" de quatro minutos, e o Bazar abriu chamado com o assunto "URGENTE!!!! CONTÊINER????".

Na mesma tarde, o contêiner do painel começou a perder processos sem traceback, sem `ERROR`, sem nada. O gateway devolvia `502` e o log do Python terminava no meio de uma frase sobre login, como diário de vítima em filme de terror.

O que é um contêiner, para um sinal sumir no caminho e um processo morrer calado?

Um contêiner é um processo com antolhos

Contêiner não existe no kernel do Linux; não há chamada de sistema `create_container`. Existe um processo comum, criado por `clone` (capítulo 13), com duas coisas configuradas: **namespaces**, que limitam o que ele enxerga, e **cgroups**, que limitam o que ele consome. O resto (imagem, registry, `docker run`) é tooling em volta desse processo.

Cada namespace isola um recurso global do kernel:

Namespace	O processo dentro dele vê
`pid`	a própria árvore de processos, começando no PID 1
`mnt`	a própria árvore de montagens, com a imagem como raiz
`net`	as próprias interfaces, rotas, portas e regras de firewall

Namespace	O processo dentro dele vê
`uts`	o próprio hostname
`ipc`	a própria memória compartilhada System V e filas POSIX
`user`	um mapeamento de UIDs: root dentro, usuário comum fora
`cgroup`	a própria raiz da hierarquia de cgroups
`time`	deslocamentos próprios dos relógios monotônico e de boot

O mesmo processo tem dois PIDs. Dentro do contêiner do Carlos:

```
$ docker exec conector ps -o pid,cmd
  PID CMD
    1 /bin/sh ./iniciar.sh
    9 python -m conector
```

No host, o mesmo Python é o PID 48213. Ele acha que é o segundo processo do universo. É o 48.213º, e o host sabe.

A consequência que mais importa: **os contêineres de uma máquina compartilham o mesmo kernel**, sem sistema operacional convidado como numa máquina virtual. Por isso o contêiner sobe em milissegundos e custa quase só o processo; e por isso um bug no kernel ou no runtime é fuga para o host. A defesa é reduzir o que o processo pode pedir ao kernel: usuário não root, *capabilities* removidas, seccomp e raiz somente leitura.

cgroups, ou o kernel cobrando a conta no ato

Os **cgroups** (na versão 2, a hierarquia única em `/sys/fs/cgroup`) aplicam limites e contabilidade a grupos de processos. O contêiner é um cgroup, e os arquivos que importam são poucos e brutais.

Memória. `memory.max` é o teto rígido. Quando o grupo passa dele e o kernel não consegue recuperar páginas (o cache de arquivos conta, mas é recuperável; o heap do Python, sem swap, fica), o OOM killer age dentro do grupo: escolhe o processo de maior pontuação, em geral o de maior RSS, e manda `SIGKILL`. `SIGKILL` não pode ser capturado, então não existe `except`, `finally` ou `atexit` que rode. O processo não tem últimas palavras. O kernel registra o óbito por ele:

```
Memory cgroup out of memory: Killed process 51877 (python) total-vm:1489240kB,
anon-rss:498112kB, file-rss:10240kB, shmem-rss:0kB, UID:10001 oom_score_adj:0
```

E o contador `oom_kill` em `memory.events` do cgroup sobe um. Se o processo morto era o PID 1, o contêiner inteiro termina com código 137, que é 128 + 9, o número do `SIGKILL`. Esse é o "sem log" do painel: a única testemunha do assassinato é o próprio assassino, e ele escreve no log do kernel, que ninguém da aplicação lê.

CPU. `cpu.max` define cota e período: `200000 100000` dá 200 ms de CPU a cada 100 ms, o equivalente a 2 núcleos. A cota é por período e somada entre todas as threads. Um processo com 8 threads ativas gasta os 200 ms em 25 ms de relógio e fica **congelado** os 75 ms restantes do período (*throttling*), com fila de requisições crescendo e o p99 subindo. O `cpu.stat` mostra `nr_throttled` e `throttled_usec`, e quase ninguém olha. Já `cpu.weight` só define proporção quando há disputa: com a máquina ociosa, o grupo usa o que quiser.

A pegadinha mais cara: o processo dentro do contêiner não sabe dos próprios limites a menos que leia o cgroup. `os.cpu_count()` devolve os núcleos do host. Uma configuração que calcula workers como `2 × núcleos + 1` num host de 64 núcleos, com limite de 2 CPUs, sobe 129 workers que dividem 2 núcleos. O Carlos tinha chegado a 64 workers por puro instinto no capítulo 13; o contêiner chegaria a 129 por aritmética.

Imagens e camadas, ou o sistema de arquivos que nunca esquece

Uma imagem segue a **OCI Image Specification**: um *manifest* em JSON que aponta para uma configuração (comando, variáveis, usuário) e para uma lista ordenada de **camadas**, cada uma um tar com as diferenças do sistema de arquivos. Cada peça é identificada pelo SHA-256 dos próprios bytes, e camada base repetida entre imagens é guardada e baixada uma vez.

Na execução, um sistema de arquivos de união (overlayfs) empilha as camadas como diretórios somente leitura e põe por cima uma camada gravável do contêiner. Alterar um arquivo de baixo copia o arquivo inteiro para cima (*copy-up*). Apagar um arquivo cria um *whiteout*, um marcador que o esconde. O arquivo continua inteiro na camada de baixo.

Cada instrução `RUN`, `COPY` e `ADD` do Dockerfile gera uma camada. Daí a armadilha que fecha com o capítulo 33:

```
COPY .env requirements.txt /app/
RUN . /app/.env && pip install --index-url "$INDICE_PRIVADO" -r /app/requirements.txt \
    && rm /app/.env                                  # "apaguei, tá seguro"
```

O `rm` criou um whiteout numa camada nova. O `.env`, com a credencial do índice privado do capítulo 32, segue inteiro na camada do `COPY`, que viaja para todo registry e toda máquina que baixar a imagem. Segredo em imagem é segredo publicado com controle de versão.

O cache de build segue a mesma lógica: mudou a instrução ou os arquivos que ela copia, a camada e todas as seguintes são refeitas. Instalar dependências antes de copiar o código faz a mudança de uma linha refazer só a última camada.

Build em múltiplos estágios, ou a obra sem o entulho

A imagem que constrói precisa de compilador, cabeçalhos e ferramentas; a que roda precisa de Python, das dependências instaladas e do código. O **build em múltiplos estágios** usa vários `FROM` num Dockerfile e copia para a imagem final só o resultado:

```
FROM python:3.12-slim AS build
RUN python -m venv /venv
COPY requirements.txt .
RUN /venv/bin/pip install --require-hashes -r requirements.txt

FROM python:3.12-slim
COPY --from=build /venv /venv
COPY app/ /app/
USER 10001
ENTRYPOINT ["/venv/bin/python", "-m", "app"]
```

O estágio `build` é descartado. O compilador e o cache do pip não vão para produção, e o `--require-hashes` é o lock com hashes do capítulo 32.

A escolha da base define tamanho e superfície:

Base	Tamanho aproximado	Shell e gerenciador de pacotes	Custo
`python:3.12` (Debian completo)	perto de 1 GB	sim	centenas de pacotes que o scanner de CVE vai listar
`python:3.12-slim`	pouco mais de 100 MB	sim	pacotes compilados precisam do estágio de build
Alpine	dezenas de MB	sim	musl em vez de glibc: wheels diferentes e diferenças de comportamento
distroless	dezenas de MB	não	sem shell para depurar dentro do contêiner

Menos pacotes é menos CVE para triar e menos ferramenta para quem invadir: sem shell, o atacante com execução de código não tem `sh`, `curl` nem `apt` e precisa trazer tudo de casa. Você também, às 3h da manhã: depurar distroless exige anexar um contêiner de depuração ao lado.

PID 1, ou o processo que o kernel não deixa morrer de susto

O PID 1 de um namespace tem duas regras próprias no kernel.

A primeira: **sinal sem tratador instalado é ignorado**. Num processo comum, `SIGTERM` sem tratador mata pela ação padrão. No PID 1, o kernel descarta o sinal, para proteger o init do sistema de morrer por acidente. Só `SIGKILL` e `SIGSTOP` vindos de fora do namespace passam.

A segunda: o PID 1 adota todo órfão do namespace e precisa colher os filhos que morrem com `wait`. Quem não colhe acumula zumbis na tabela de processos (capítulo 13), mortos que ninguém enterrou porque o pai não leu o atestado de óbito.

Agora o Dockerfile do Carlos. `CMD ./iniciar.sh` está na *forma shell*, que o Docker executa por `/bin/sh -c`, e o PID 1 acabou sendo um shell. O script tinha `&&`, então o shell precisava continuar vivo para rodar o segundo comando, e o Python virou filho. O `docker stop` mandou `SIGTERM` ao PID 1. O `sh` não tinha tratador, o kernel jogou o sinal fora, o Python nunca soube de nada, e dez segundos depois veio o `SIGKILL` para todos. O Python morreu no meio de 31 autorizações sem ter sido avisado. O shell morreu como viveu: sem fazer nada.

```
docker stop        SIGTERM ──► PID 1 (sh, sem tratador) ──► descartado
  10 s depois      SIGKILL ──► todo o namespace       ──► 31 autorizações em voo
```

As correções são três, combináveis. A **forma exec** (`ENTRYPOINT ["/venv/bin/python", "-m", "app"]`) não passa por shell, e o Python é o PID 1. Script de entrada que termina com `exec python ...` substitui o shell pelo Python no mesmo PID. E um init mínimo como PID 1 (o `tini`, ou `docker run --init`, que usa o mesmo princípio) repassa os sinais ao filho e colhe zumbis.

Receber o sinal é metade do trabalho. O processo precisa saber morrer. O **encerramento gracioso** é uma sequência:

1. Ao receber `SIGTERM`, sair do balanceamento: o endpoint de prontidão passa a falhar e o proxy para de mandar requisições novas.
2. Parar de aceitar conexões e terminar as requisições em voo, dentro de um prazo.

3. Fechar o que é externo: pools de banco, conexões HTTP/2 (com `GOAWAY`, capítulo 9), consumidores do broker (devolvendo partições, capítulo 17).
4. Sair antes do `SIGKILL`.

O prazo de parada precisa ser maior que a soma. Fora do PID 1, um Python sem tratador de `SIGTERM` morre na hora, sem `finally`; o Uvicorn instala tratadores e faz o passo 2, com `--timeout-graceful-shutdown` como limite.

Registries e a tríade OCI

A OCI padroniza três peças: a **Image Spec** (o formato acima), a **Distribution Spec** (a API HTTP de um registry, que guarda blobs e manifests por digest) e a **Runtime Spec** (como um runtime, o `runc` por exemplo, cria o processo com namespaces e cgroups a partir de um `config.json`). Ferramentas que falam OCI trocam imagens entre si.

No registry, a **tag** (`pagamentos:v42`) é um ponteiro mutável, que alguém pode reapontar amanhã. O **digest** (`pagamentos@sha256:9f1c...`) é o hash do manifest e só significa uma imagem. Deploy por tag roda o que a etiqueta apontar na hora do pull; por digest, roda exatamente o que foi testado. E `latest` é só uma tag com nome otimista, igual ao `repasse_final_v2_agora_vai.py`.

Na Lastro: imagens, sinais e limites

A Lastro já tinha artefato imutável: a imagem de máquina do autoscaling (capítulo 29). O contêiner custa mais uma camada de software por máquina e se paga por três requisitos: o tempo de reação do autoscaling (4 min 20 s, 2 min deles montando a máquina), limite de memória por serviço e o mínimo de software no segmento de cartão. Com um serviço por máquina e sem segmento, a imagem de máquina bastaria.

As decisões:

Item	Decisão	Por quê
Processos por contêiner	1 processo Uvicorn	escala por contêiner; os números do capítulo 29 já contavam processos
PID 1	`tini` + Python em forma exec	sinal chega, zumbi é colhido
Base	`python:3.12-slim`, multiestágio; cofre e conector em distroless	cofre e conector ficam sem shell no segmento
Usuário	UID 10001, raiz somente leitura, sem capabilities	execução de código não vira root

Item	Decisão	Por quê
Identificação	deploy por digest; tag `serviço:data-commit` só para humanos	o que foi testado é o que roda
Segredos	lidos do gerenciador pelo próprio Python na partida (capítulo 33)	nada em camada, nada em variável de ambiente
Tamanho	pagamentos de 1,2 GB para 160 MB	pull na máquina nova de 38 s para 6 s

No encerramento de pagamentos e do conector, o `/pronto` do capítulo 27 falha no `SIGTERM` e o proxy local tira o processo após 2 checagens de 2 s; as requisições em voo têm o deadline de 8 s (capítulo 15); o SSE fecha com `retry` (capítulo 29). Total: até 15 s, a drenagem que o autoscaling já reservava. O prazo de parada é **20 s**, e um contador de "processos encerrados por `SIGKILL`" alarma acima de zero. Os consumidores do broker (ledger, entregador) param de buscar, terminam o lote, confirmam offsets e saem.

Os limites saíram de medição, e não de chute:

Serviço	Pico de RSS medido	`memory.max`	`cpu.max`
pagamentos	180 MiB	320 MiB	2 núcleos
conector-adquirente	140 MiB	256 MiB	2 núcleos
painel (login com Argon2id)	160 MiB + 4 × 64 MiB	640 MiB	2 núcleos
relatórios	1,4 GiB	2 GiB	2 núcleos

O painel morria porque o Carlos tinha empacotado os três workers num contêiner de 512 MiB: 3 × (160 + 4 × 64) MiB, perto de 1,2 GiB no pior caso, e o OOM killer resolvia pela maior conta. Com um processo por contêiner e o semáforo de 4 do capítulo 33, o pior caso é 416 MiB, e o limite tem folga de 50%. O relatório de risco do capítulo 26, que derrubou a autorização com 7 GB de DataFrame, agora morre sozinho dentro dos 2 GiB dele, em vez de levar a máquina junto. É o mesmo crime com a cena isolada.

A cota de 2 núcleos para um processo com GIL parece generosa. Ela existe para as threads em C (o gRPC do antifraude, o `to_thread` do capítulo 13) não serem congeladas por *throttling* no meio de uma autorização; `nr_throttled` acima de zero vira alerta. Nenhum serviço calcula workers por `os.cpu_count()`: o número é fixo e igual a um.

E a morte silenciosa ganhou testemunha: o agente de cada máquina lê `memory.events` dos cgroups e o código de saída dos contêineres, e 137 com `oom_kill` incrementado vira evento com nome do serviço e pico de memória. O `temp-01` não roda contêiner: o kernel dele é anterior à hierarquia única de cgroups v2 e ninguém aceita reiniciá-lo para descobrir se volta.

Armadilhas e casos reais

> **Na vida real:** a JVM, antes do JDK 10 (e do 8u191, que recebeu o recurso por *backport*), não lia os limites do cgroup e dimensionava o heap pela memória do host. Um contêiner com limite de 1 GB numa máquina de 64 GB ganhava heap padrão de 16 GB e era morto pelo OOM killer muito antes de o garbage collector achar que estava na hora de limpar alguma coisa. A consultoria Java do Bazar descobriu isso em produção e cobrou o aprendizado em horas.
>
> **Na vida real:** a CVE-2019-5736, de fevereiro de 2019, permitia a um contêiner malicioso sobrescrever o binário `runc` do host e ganhar root na máquina na próxima execução. O ataque exigia root dentro do contêiner: quem rodava com usuário comum assistiu de camarote.
>
> **Armadilha:** cache em memória sem `maxsize` cresce até o `memory.max`, e o OOM killer escolhe o maior culpado, que é ele. O `lru_cache` do capítulo 22 tinha 128 entradas, e foi a única coisa naquele PR que estava certa.
>
> **Armadilha:** `print` em contêiner. Sem terminal, o `stdout` do Python é bufferizado em blocos, e o processo morto por `SIGKILL` leva o buffer junto. O módulo `logging` faz *flush* a cada registro; `PYTHONUNBUFFERED=1` cobre o resto.

Resumo

- Contêiner é um processo com namespaces (o que vê) e cgroups (o que consome), sobre o kernel do host.
- `memory.max` estourado: OOM killer, `SIGKILL`, código 137, registro só no kernel e em `memory.events`.
- `cpu.max` é cota por período somada entre threads; excesso vira *throttling*, e o processo não conhece os limites sem ler o cgroup.
- Imagem OCI: camadas endereçadas por SHA-256; apagar depois não remove o arquivo.
- Múltiplos estágios e bases mínimas reduzem tamanho e superfície.
- PID 1 ignora sinais sem tratador e colhe órfãos: forma exec ou init mínimo.
- Encerramento gracioso: sair do balanceamento, drenar, fechar conexões, sair antes do `SIGKILL`.
- Tag é ponteiro mutável; digest identifica a imagem.

A Lastro agora

Cada serviço virou uma imagem multiestágio, com um processo Uvicorn atrás do `tini`, usuário sem privilégio, segredos lidos na partida e deploy por digest. O prazo de parada é 20 s contra 15 s de drenagem, os limites de memória vêm de pico medido mais folga, e um `SIGKILL` ou um OOM agora deixam nome, sobrenome e pico de memória no alerta. Pagamentos caiu de 1,2 GB para 160 MB, e o fundador perguntou para onde foi o resto. Foi o `.git`, os testes, o compilador e uma pasta `backup_antigo/` com 400 MB de CSV do financeiro.

Agora são 14 imagens e uns 90 contêineres espalhados pelas máquinas, iniciados por scripts de deploy em bash. Para não depender de ninguém acordado, o Carlos escreveu um laço que reinicia todo contêiner cujo `/saude` não responde em 1 s e chamou de orquestrador. Na primeira sexta de campanha, sob carga, o lag do event loop passou de 1 s, o laço reiniciou um contêiner saudável, a carga dele foi para os vizinhos, que também passaram de 1 s, e o orquestrador do Carlos derrubou, com eficiência exemplar, todos os processos de pagamentos que estavam funcionando.

Referências

- Linux man-pages, `namespaces(7)`: https://man7.org/linux/man-pages/man7/namespaces.7.html
- Linux man-pages, `pid_namespaces(7)` (PID 1 e sinais): https://man7.org/linux/man-pages/man7/pid_namespaces.7.html
- Documentação do kernel Linux, *Control Group v2*: https://docs.kernel.org/admin-guide/cgroup-v2.html
- Open Container Initiative, especificações de imagem, distribuição e runtime: https://opencontainers.org/
- Docker, *Multi-stage builds*: https://docs.docker.com/build/building/multi-stage/
- Docker, referência do Dockerfile (formas shell e exec): https://docs.docker.com/reference/dockerfile/
- Docker, `docker container stop`: https://docs.docker.com/reference/cli/docker/container/stop/
- tini: https://github.com/krallin/tini
- Uvicorn, configurações (`--timeout-graceful-shutdown`): https://www.uvicorn.org/settings/
- CVE-2019-5736 (runc): https://nvd.nist.gov/vuln/detail/CVE-2019-5736

- JDK-8146115, suporte a contêineres na JVM: https://bugs.openjdk.org/browse/JDK-8146115

CAPÍTULO 35

Kubernetes

O orquestrador do Carlos tinha 41 linhas de bash, um `sleep 5` e o temperamento de um segurança de boate que expulsa o cliente por ter piscado devagar.

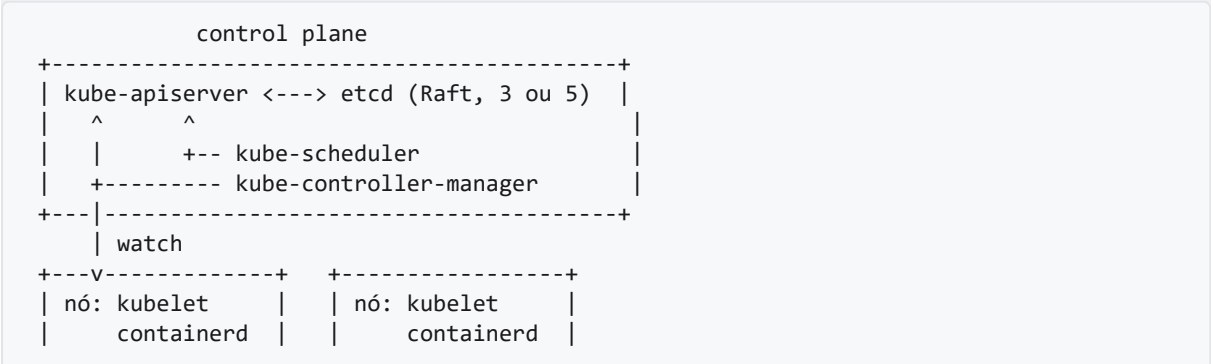
Sexta, 20h14, abertura da "Semana da Capinha" do Bazar. Um contêiner de pagamentos passou de 1 s de lag no event loop e o laço mandou `docker restart`, que dá 10 s de prazo contra os 15 s de drenagem do capítulo 34. O processo morreu no meio de 37 autorizações, a carga foi para os outros 23, e o lag deles passou de 1 s. Em quatro minutos o laço reiniciou 22 dos 24, inclusive um que ainda subia, por não responder enquanto subia: 640 autorizações em `processando` (resolvidas pelo processo do capítulo 15) e seis minutos de `503` no Pix. O Bazar abriu chamado às 20h19: "URGENTE!!!! VOCÊS DESLIGARAM DE PROPÓSITO???". Foi de propósito só no sentido de que alguém escreveu um programa para fazer exatamente isso.

A ideia do laço estava certa; o resto, não. Quem decide onde cada um dos 90 contêineres roda, quem o reinicia, quem o tira do balanceamento e quem troca a versão sem derrubar autorização?

O control plane, ou um banco de dados cercado de laços infinitos

O Kubernetes é um banco de estado desejado cercado de programas em `while True` comparando o desejado com o real. O laço do Carlos era um controlador. Só que sem estado desejado, sem histerese e com licença para matar.

O **control plane** guarda o estado e decide; os **nós** rodam os contêineres:



```
|     kube-proxy  |   |     kube-proxy  |
+-----------------+   +-----------------+
```

O **kube-apiserver** é uma API REST sobre objetos (Pod, Deployment, Service) e o único componente que fala com o **etcd**, um chave-valor replicado com o Raft do capítulo 20: 3 membros sobrevivem à perda de 1; 5, à perda de 2. Sem maioria, o cluster continua rodando o que já rodava e para de aceitar mudança. Vivo e catatônico.

Todo o resto conversa com o apiserver por **watch**: uma requisição longa que recebe cada alteração de um tipo de objeto, retomável por `resourceVersion`. O **kube-scheduler** vê pods sem nó, filtra os nós que cabem, dá nota aos que sobraram e grava o vencedor no pod. O **kube-controller-manager** roda dezenas de controladores (o de ReplicaSet cria ou apaga pods até bater o número). O **kubelet** de cada nó pede ao runtime (containerd ou CRI-O) os pods atribuídos a ele, com os namespaces e cgroups do capítulo 34.

Ninguém manda ordem para ninguém. Cada um observa, compara com o desejado e age; se morre e volta, relê tudo e continua. Isso é **reconciliação**: o script de deploy executa passos uma vez e reza; o controlador repete até o mundo concordar com ele, com a paciência de quem nunca teve plantão.

Pods, ou o menor imóvel que o Kubernetes aluga

O Kubernetes agenda **pods**: um ou mais contêineres que dividem o namespace de rede (mesmo IP, mesmo `localhost`), podem dividir volumes e caem juntos no mesmo nó.

O modelo de rede tem uma regra: **todo pod tem um IP próprio e alcança todo outro pod sem NAT**, cumprida pelo plugin de rede (CNI). O IP é efêmero, e o pod também: morre, o controlador cria outro com nome e IP novos, e ninguém faz velório. Guardou o IP do pod num arquivo de configuração? Reinventou o `ANTIFRAUDE_HOSTS` do capítulo 27, agora com rotatividade de minutos.

O *sidecar* nativo (`initContainer` com `restartPolicy: Always`, estável desde a 1.33) sobe antes dos contêineres principais e morre depois deles.

Deployments e rolling update, ou trocar o pneu com o carro andando e 37 autorizações no banco de trás

Um **Deployment** declara o modelo do pod (imagem por digest, capítulo 34) e o número de réplicas. Ele cria um **ReplicaSet** por versão do modelo, que mantém aquele número de pods vivos, achando-os por *labels* (pares chave-valor consultados por seletores). O

Kubernetes amarra objetos assim, sem chave estrangeira; dois Deployments com o mesmo seletor apagam os pods um do outro para sempre, com zelo.

Mudar a imagem cria um ReplicaSet novo, e o **rolling update** troca os pods em lotes, limitado por `maxSurge` (pods acima do desejado, padrão 25%) e `maxUnavailable` (pods abaixo dele, padrão 25%). Com 40 réplicas e os padrões, 10 pods caem de cara: 25% a menos de capacidade, notado só no deploy do pico. O pod novo só conta como disponível depois de passar na readiness, e `kubectl rollout undo` reescala o ReplicaSet anterior.

A parte que mata autorização é o encerramento. Quando o pod é apagado, duas coisas começam **ao mesmo tempo**, sem esperar uma pela outra:

```
pod marcado Terminating
 |-- kubelet: roda preStop, depois manda SIGTERM, espera até
 |            terminationGracePeriodSeconds (30 s), depois SIGKILL
 '-- controlador de endpoints: tira o pod do Service
       '-- kube-proxy de cada nó atualiza regras... quando der
```

O processo recebe `SIGTERM` e fecha o socket, e por um ou dois segundos outros nós ainda mandam conexões para ele, porque o kube-proxy de lá não leu a mudança: conexão recusada no meio do deploy, culpa de ninguém, sintoma de todos. O remédio é um `preStop` que só espera alguns segundos antes do `SIGTERM`. E o prazo conta desde o início do `preStop`: um `preStop` de 5 s come 5 s da drenagem.

Services e Ingress, ou um IP que não pertence a ninguém

O **Service** dá endereço estável a pods que trocam de IP: o ClusterIP, um IP virtual sem interface, por quem ninguém responde ARP, que existe só como regra de NAT.

Um controlador mantém **EndpointSlices** com os IPs dos pods que casam com o seletor do Service **e estão prontos**. O kube-proxy de cada nó os lê e escreve regras no kernel (iptables, IPVS ou nftables) que trocam o ClusterIP por um pod sorteado; o CoreDNS resolve `pagamentos.lastro.svc.cluster.local` para o ClusterIP. É camada 4 (capítulo 27), decidida **por conexão**: uma conexão HTTP/2 que dura a vida do processo fica a vida inteira no mesmo pod, como o capítulo 12 mostrou com gRPC. O Service distribui conexões com justiça perfeita; as requisições dentro delas, ele nem vê. O tipo `LoadBalancer` pede ao provedor um balanceador externo.

Para HTTP, o **Ingress** (e a sucessora, a **Gateway API**, v1.0 em outubro de 2023) declara rotas por host e caminho, aplicadas por um controller; sem controller, é YAML decorativo.

Secrets, ou base64 com crachá de criptografia

ConfigMap guarda configuração; **Secret** guarda segredo. A diferença entre os dois é menor do que o nome sugere:

```
$ kubectl get secret adquirente -o jsonpath='{.data.senha}' | base64 -d
s3nh4-qu3-0-c4rl0s-c0l0u-n0-ch4t
```

O valor fica em base64 e, por padrão, **em claro no etcd**. Cifrar em repouso exige configurar o apiserver com um provedor de cifra, de preferência o de KMS (o envelope encryption do capítulo 33). Mesmo cifrado, quem pode criar pod num namespace monta qualquer Secret dele e lê pelo pod. RBAC que nega `get secrets` e permite `create pods` é porta trancada com a chave no tapete. Como variável de ambiente, ainda caem no `/proc/<pid>/environ` do capítulo 33 e só mudam quando o contêiner é recriado, com a teimosia do worker convicto do capítulo 22.

Requests, limits e a ordem de quem morre primeiro

Cada contêiner declara **requests** e **limits** de CPU e memória:

Campo	Quem usa	Vira no cgroup (cap. 34)
request de CPU	scheduler (soma por nó)	`cpu.weight` (fatia sob disputa)
limit de CPU	kernel	`cpu.max` (throttling)
request de memória	scheduler, despejo	nada que limite
limit de memória	kernel	`memory.max` (OOM killer)

O scheduler só soma requests: um nó de 7,5 núcleos alocáveis aceita pods até a soma chegar lá, mesmo com todos dormindo, e recusa o seguinte com a CPU real a 3%. Request baixo lota o nó de pods que depois disputam CPU; request alto paga máquina para guardar ar.

As combinações definem a **classe de QoS**: *Guaranteed* (request igual a limit em CPU e memória, em todos os contêineres), *Burstable* (algum valor declarado) e *BestEffort* (nada). Sem memória no nó, o kubelet despeja primeiro quem mais passou do request, começando pelos BestEffort. Pod sem request é o primeiro jogado do barco, e com razão: nunca disse quanto pesava. CPU além do limit deixa lento; memória além do limit mata, e por isso memória com request igual a limit reserva exatamente o que o kernel vai cobrar.

Probes, ou o médico com licença para matar

O kubelet faz três tipos de exame, por HTTP, TCP, gRPC ou comando:

- **startup**: enquanto não passa, as outras duas nem começam; para processo que demora a subir;
- **readiness**: falhou, o pod sai dos EndpointSlices e para de receber tráfego; passou, volta;
- **liveness**: falhou, o kubelet **mata o contêiner** e o recria, com espera crescente entre reinícios (10 s, 20 s, 40 s... até 5 min: o `CrashLoopBackOff`).

Os padrões são `periodSeconds: 10`, `timeoutSeconds: 1`, `failureThreshold: 3`. Leia o segundo de novo. Um segundo de timeout numa liveness é o orquestrador do Carlos com suporte oficial e documentação em 15 idiomas. Em homologação, sob a carga da Semana da Capinha:

```
Warning  Unhealthy  kubelet  Liveness probe failed: Get "http://10.42.3.17:8000/saude":
                             context deadline exceeded (Client.Timeout exceeded while
awaiting headers)
Normal   Killing    kubelet  Container pagamentos failed liveness probe, will be
restarted
```

A regra: **readiness responde "devo receber tráfego agora?" e liveness responde "este processo está irrecuperável?"**. Readiness pode ser sensível, porque o castigo é reversível. Liveness só pode falhar quando reiniciar resolve (deadlock, laço travado), e reiniciar não resolve carga alta, banco lento nem parceiro fora. Liveness que consulta o banco mata todos os pods quando o banco engasga; eles voltam, consultam o banco engasgado e morrem de novo. O health check que mata paciente, família e médico (capítulo 27), agora em todos os nós.

E o Service não tem o **modo pânico** do proxy do capítulo 27: readiness falhando em todos os pods deixa o Service sem endpoint, recusando tudo. Se ninguém está pronto, ninguém trabalha, inclusive os que estavam só cansados.

HPA, ou o autoscaler do capítulo 29 com outro crachá

O **HorizontalPodAutoscaler** aplica a regra proporcional do capítulo 29 (`ceil(réplicas × atual/alvo)`, tolerância de 10%, 300 s de estabilização para descer) a cada 15 s, com uma pegadinha: a CPU é medida **em relação ao request**. Request de 1 núcleo com alvo de 35% são 0,35 núcleo por pod. O campo `behavior` define políticas separadas de subida e descida.

O HPA cria pods, não máquinas. Pod sem nó fica `Pending`, e um *autoscaler de nós* (Cluster Autoscaler e Karpenter são exemplos) pede máquinas ao provedor, que levam minutos. Para ter capacidade ligada, roda-se um Deployment de pods de enchimento com prioridade negativa: o pod de verdade chega, o scheduler expulsa (*preempção*) o de enchimento, que fica pendente e puxa um nó novo. Em inglês, *balloon pods*. Na Lastro, por razões óbvias, pods de lastro. A empresa levou 35 capítulos para ter lastro, e ele é feito de pod vazio.

Na Lastro: o cluster

O Kubernetes substitui o laço do Carlos, os scripts de deploy e a lista do proxy local, pelos requisitos do capítulo 34: reação do autoscaling, limites por serviço, deploy sem perder autorização. Custa um sistema distribuído a mais para atualizar (três versões menores por ano, ~14 meses de suporte cada). Com máquinas fixas, os scripts aguentariam; com autoscaling em três zonas, a Lastro estava escrevendo um Kubernetes em bash, e mal.

O desenho:

Item	Decisão	Requisito
Control plane	gerenciado (EKS, GKE e AKS são exemplos)	etcd sem plantão da Lastro
Nós	8 vCPU, 16 GiB, zonas A, B e C, 9 a 36 nós	perder uma zona tira ≤ 1/3 (espalhamento por zona)
Drenagem de nó	PodDisruptionBudget `maxUnavailable: 10%`	atualizar nó sem afundar a autorização
Segmento de cartão	**cluster separado**	escopo PCI (capítulo 33)
Bancos, Redis, broker	**fora do cluster**	ver abaixo

Namespace do Kubernetes separa nomes, e não kernel nem control plane. Com nós dedicados (*taints*) e NetworkPolicy, o apiserver que agenda o cofre agendaria também o relatório de risco e entraria no escopo do PCI DSS, levando junto quem administra o cluster. Um cluster a mais custa um control plane gerenciado; o compartilhado custava a auditoria olhar para tudo de novo.

Pagamentos fica assim, por pod:

Campo	Valor	Por quê
CPU request / limit	1 / 2 núcleos	GIL usa ~1 núcleo; threads em C (cap. 34)
memória request = limit	320 MiB	pico medido 180 MiB (cap. 34)
HPA	12–60, alvo 35% do request ou lag > 50 ms	ρ ≤ 0,7 no 2× pico; teto do banco (cap. 29)
`behavior`	+50%/min; −3 pods/5 min após 15 min	regras do cap. 29
rolling update	`maxSurge: 20%`, `maxUnavailable: 0`	capacidade nunca cai
encerramento	`preStop` `sleep` 5 s; prazo 25 s	propagação + 20 s do cap. 34

O `preStop` usa a ação `sleep` nativa, e não `sh -c "sleep 5"`: o conector é distroless, sem shell, e o `preStop` sem binário falha direto no `SIGTERM` que existia para adiar.

As probes:

```
startupProbe:
  httpGet: {path: /vivo, port: 8000}
  periodSeconds: 2
  failureThreshold: 30     # até 60 s para segredos e tarifas
readinessProbe:
  httpGet: {path: /pronto, port: 8000}
  periodSeconds: 2
  failureThreshold: 2      # o /pronto local do capítulo 27
livenessProbe:
  httpGet: {path: /vivo, port: 8000}
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 6      # 60 s seguidos sem resposta
```

O `/vivo` não consulta nada e responde do próprio event loop: com lag de 1 s, responde em 1 s, dentro dos 5 s; com o loop travado, falha seis vezes e o kubelet reinicia, a única situação em que reiniciar ajuda. A readiness segue o `/pronto` do capítulo 27 (tarifas, drenagem, lag < 50 ms). Ficar lento tira de circulação; travar mata. O laço do Carlos matava por lentidão.

Entre serviços, o ClusterIP não basta: gRPC ao antifraude e a API do ledger usam HTTP/2 de longa duração, e a Lastro depende de P2C, ejeção e modo pânico (capítulo 27). O proxy local vira **service mesh** com sidecar por pod (Istio e Linkerd são exemplos), alimentado pelos EndpointSlices, com mTLS por identidade de

ServiceAccount, que o segmento de cartão usa para aceitar só o pagamentos. Custo medido: ~50 MiB por pod e +0,4 ms por salto, tirados da folga de 120 ms (capítulo 14). O sidecar é nativo: a drenagem de 15 s acontece com o proxy vivo.

Segredos continuam no gerenciador do capítulo 33: o pod troca o token da ServiceAccount, projetado pelo kubelet, por credencial do provedor (identidade de carga de trabalho). O papel por grupo de máquinas do capítulo 32 vira papel por ServiceAccount, e uma NetworkPolicy bloqueia `169.254.169.254` em todos os pods. O botão "Testar endpoint" foi enterrado; a NetworkPolicy é a lápide.

O ledger fica fora do cluster. PostgreSQL no Kubernetes existe (StatefulSet, volumes persistentes e um operador, o CloudNativePG é um exemplo), mas o ledger já tem Patroni, réplica síncrona e fencing (capítulo 23) para a exigência de nenhum lançamento confirmado perdido. Dentro do cluster, cada atualização de nó drenaria um pod de banco com failover de 30 a 40 s. Com dezenas de bancos pequenos e um time dono do operador, a conta muda. Redis e broker, gerenciados, também ficam fora.

O `repasse_final_v2_agora_vai.py` virou Deployment de 2 réplicas, nome mantido. O Kubernetes tem objeto de eleição de líder (Lease, em `coordination.k8s.io`), mas o lease do capítulo 20 continua no PostgreSQL: o Lease diria quem é o líder; só o banco, na transação que debita, pode recusar o débito do ex-líder.

Armadilhas e casos reais

> **Na vida real:** no Pi Day de 2023, o Reddit ficou 314 minutos fora após atualizar um cluster da 1.23 para a 1.24, que removeu o label `node-role.kubernetes.io/master` (agora `control-plane`). Os *route reflectors* do Calico selecionavam nós pelo label antigo, e a rede dos pods parou. O mercado aposentou "master" por bons motivos, e o "master" se vingou em produção.
>
> **Armadilha:** o `resolv.conf` do pod vem com `ndots:5`. Um nome com menos de cinco pontos, como `api.adquirente.com.br`, é testado antes com cada domínio de busca do cluster, cada tentativa voltando `NXDOMAIN`. O conector fazia isso a cada conexão nova. Nome com ponto final (`api.adquirente.com.br.`) ou `dnsConfig` resolve; o CoreDNS mandou flores.
>
> **Armadilha:** liveness com `initialDelaySeconds` no lugar de startup probe mata, num dia lento, o processo antes de ele terminar de subir. O orquestrador do Carlos fazia isso de graça.

Resumo

- Estado desejado no etcd, servido pelo apiserver; controladores reconciliam por watch.
- Pod: contêineres que dividem rede e nó, com IP efêmero.
- Rolling update limitado por `maxSurge` e `maxUnavailable`; no encerramento, `preStop` cobre a corrida com o Service.
- Service: IP virtual, só pods prontos, L4 por conexão.
- Secret é base64; criar pod é ler Secret.
- Requests agendam, limits viram cgroup, QoS decide o despejo.
- Readiness tira do tráfego; liveness mata, e só deve falhar quando reiniciar resolve.
- HPA mede CPU em relação ao request; nós vêm de outro autoscaler.

A Lastro agora

Os serviços stateless da Lastro rodam num cluster gerenciado em três zonas, de 9 a 36 nós, e o segmento de cartão tem cluster próprio. Pagamentos tem 1 núcleo de request, 320 MiB garantidos, HPA de 12 a 60, rolling update sem perder capacidade e encerramento de 25 s. A readiness tira o lento do tráfego; a liveness só mata quem passou 60 s mudo. Mesh no lugar do proxy local, segredos por ServiceAccount, bancos, Redis e broker fora. O laço do Carlos foi apagado com honras e substituído por dezenas de controladores que fazem a mesma coisa, só que olhando antes.

Fora do cluster ficou o de sempre: o proxy de saída com o IP `203.0.113.10` em api-01 e api-02, e o `temp-01`, com o IP que a integração antiga do Bazar ainda usa e um kernel que não roda contêiner nem a pau. Aí o auditor do PCI DSS olhou o diagrama, apontou uma máquina sem dono na mesma rede privada do cofre e perguntou o que era. O fundador respondeu "é provisório". O auditor anotou a data de criação: seis anos atrás.

Referências

- Kubernetes. *Cluster Architecture*. https://kubernetes.io/docs/concepts/architecture/
- Kubernetes. *Pod Lifecycle*. https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
- Kubernetes. *Configure Liveness, Readiness and Startup Probes*. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

- Kubernetes. *Deployments*. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Kubernetes. *Sidecar Containers*. https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/
- Kubernetes. *Service*. https://kubernetes.io/docs/concepts/services-networking/service/
- Kubernetes Gateway API. https://gateway-api.sigs.k8s.io/
- Kubernetes. *Good practices for Kubernetes Secrets*. https://kubernetes.io/docs/concepts/security/secrets-good-practices/
- Kubernetes. *Resource Management for Pods and Containers*. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
- Kubernetes. *Horizontal Pod Autoscaling*. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
- Kubernetes. *Releases*. https://kubernetes.io/releases/
- Reddit Engineering. *You Broke Reddit: The Pi-Day Outage* (março de 2023), em r/RedditEng. https://www.reddit.com/r/RedditEng/

CAPÍTULO 36

Redes em nuvem e infraestrutura como código

O auditor do PCI DSS abriu uma sessão SSH no `temp-01` com a chave do fundador, num café, pela internet, e digitou um comando só:

```
temp-01$ nc -vz 10.0.7.41 5432
Connection to 10.0.7.41 5432 port [tcp/postgresql] succeeded!
```

O `10.0.7.41` é o `pg-cofre`, onde moram os cartões cifrados do capítulo 33. O `temp-01` é a máquina de seis anos, kernel sem cgroups v2, porta 22 aberta para a internet desde que o fundador quis "acessar de casa". Entre os dois, nada. O cofre tinha cluster próprio, KMS, AAD e rotação anual, e estava a um `nc` da máquina mais velha e menos vigiada da empresa. Fechadura de cofre suíço, parede de drywall.

O relatório saiu com uma frase sublinhada: "ambiente sem segmentação de rede; escopo do PCI DSS = rede inteira". Pior foi o inventário. A rede foi criada pelo fundador no console e cresceu por cliques: 11 sub-redes, todas com rota para a internet, 31 grupos de firewall com 214 regras, e um grupo `default` com todas as máquinas, liberando todo tráfego entre elas. Quem criou a regra 147? Ninguém sabe. Ela libera a porta 6379 para `0.0.0.0/0` e tem a descrição "teste".

Qual o jeito de dividir a rede para o cofre ficar inalcançável de tudo que não precisa dele? E de provar, daqui a um ano, que ela continua dividida?

Regiões, zonas e a rede virtual, ou o datacenter que você aluga em fatias

O provedor organiza a infraestrutura em **regiões** (datacenters numa área geográfica, como São Paulo) e, dentro delas, **zonas de disponibilidade**: datacenters com energia, refrigeração e rede independentes, a 1–2 ms um do outro. As zonas A, B e C do capítulo 23 são isso. Zona é domínio de falha; região é domínio de falha maior, assunto do capítulo 40.

A **rede virtual** (VPC na AWS e no Google Cloud, VNet no Azure) é uma rede privada definida por software numa região: você escolhe um CIDR da RFC 1918 (capítulo 8), como `10.20.0.0/16`, e ganha 65.536 endereços que só existem para você. O pacote entre duas máquinas suas é encapsulado pelo hipervisor, e a rede do provedor descarta pacote cujo remetente não seja o IP atribuído àquela interface. Falsificar IP de origem na VPC tem a eficácia de falsificar firma na frente do tabelião.

As **sub-redes** são fatias do CIDR; na AWS e no Azure, cada uma vive numa zona; no Google Cloud, são regionais. O provedor reserva endereços de cada sub-rede (cinco, na AWS), e quem deu uma `/28` aos nós do Kubernetes descobriu que 11 endereços acabam antes do almoço.

Tabelas de rota, ou onde "pública" e "privada" são decididas

O que faz uma sub-rede pública ou privada é a **tabela de rotas** associada a ela, e nada mais: nem nome, nem etiqueta, nem fé. Cada tabela tem rotas `destino → alvo`, e o pacote segue a rota de **prefixo mais longo** que casa com o destino:

```
tabela "publica"                    tabela "privada-a"
10.20.0.0/16  -> local              10.20.0.0/16  -> local
0.0.0.0/0     -> gateway-internet   0.0.0.0/0     -> nat-a
```

A rota `local` cobre a rede inteira, é criada pelo provedor e, na AWS, não pode ser removida: **toda sub-rede alcança toda sub-rede da mesma VPC**, e o isolamento interno fica por conta dos firewalls. A sub-rede **pública** tem rota padrão para o *gateway de internet*, que faz NAT um para um entre o IP privado da máquina e o IP público dela. A **privada** não tem: uma máquina lá com IP público continua inalcançável, porque a resposta para a internet não casa com rota nenhuma e é descartada. IP público pago, usado zero vezes, que é o tipo de eficiência que o fundador chama de "reserva estratégica".

Para a privada falar com o mundo, entra o **gateway NAT**: fica numa sub-rede pública, tem IP público fixo, reescreve a origem dos pacotes que saem (SNAT, capítulo 8) e só deixa entrar respostas de conexões abertas de dentro. O IP de saída é o do NAT, por mais máquinas que o autoscaling crie, e é ele que vai na allowlist do parceiro. E o NAT vive numa zona: se ela cai, as sub-redes que apontam para ele perdem a internet. Um NAT por zona dá resiliência e um IP por zona; um NAT só dá um IP e um ponto único de falha.

Cada conexão gasta uma porta de origem do IP do NAT, então há teto: na AWS, cerca de 55 mil conexões simultâneas por destino único para cada IP. Abrir conexão nova por requisição, o pecado do capítulo 8, agora tem limite e mensagem de erro.

Tráfego para serviços do provedor (armazenamento de objetos, KMS, gerenciador de segredos) dispensa NAT e internet: **endpoints privados** (PrivateLink na AWS e Private Service Connect no Google Cloud são exemplos) põem uma interface do serviço na sua sub-rede. O cofre fala com o KMS sem rota para a internet, e o auditor gosta disso quase tanto quanto gosta de dizer não.

Firewalls com estado e sem estado, ou a diferença entre ter memória e ter regulamento

O **firewall com estado** (o *security group* da AWS, o *network security group* do Azure, as regras de firewall da VPC do Google Cloud) fica na interface de cada máquina e mantém uma tabela de conexões (*connection tracking*): liberou a entrada na 443, a resposta sai sozinha. No modelo da AWS, as regras são só de permissão, e a origem pode ser outro grupo: "aceita 5432 de quem estiver no grupo `nos-app`". A regra acompanha as máquinas que entram e saem do grupo, sem lista de IPs. É o que o `ANTIFRAUDE_HOSTS` do Carlos queria ser quando crescesse.

O **firewall sem estado** (a *network ACL* da AWS) fica na fronteira da sub-rede e avalia cada pacote isolado, em ordem numérica, com regras de permitir **e** negar. Sem memória, exige regra para a volta: a resposta da 443 sai para a porta efêmera do cliente, entre 1024 e 65535 (no Linux, 32768 a 60999, capítulo 8). Liberou a 443 e esqueceu as efêmeras? Construiu um servidor que escuta tudo e não consegue responder nada. O servidor perfeito para reunião de diretoria.

	Com estado	Sem estado
Onde	interface de cada máquina	fronteira da sub-rede
Resposta	volta sozinha	precisa de regra explícita
Regras	só permitir (AWS)	permitir e negar, em ordem
Origem por grupo	sim	só CIDR
Serve para	política por papel	barreira grossa, negar faixas

Os dois se somam: o pacote passa pela ACL da sub-rede e pelo grupo da interface. Um erro em um deles não abre a porta sozinho. Defesa em profundidade, sem precisar de palestra.

Infraestrutura como código, ou o fim do "quem clicou nisso?"

Nenhum firewall resolve o outro problema do auditor: ninguém sabia o que existia, nem por quê. Infraestrutura criada no console é estado sem histórico. **Infraestrutura como código** descreve os recursos em arquivos versionados, revisados em pull request e aplicados por uma ferramenta. O Terraform é a referência (sob licença BSL desde agosto de 2023; o OpenTofu é o fork aberto, na Linux Foundation, com a mesma linguagem).

O Terraform é **declarativo**: o código, em HCL, diz que recursos devem existir, e a ferramenta calcula as chamadas de API. *Providers* traduzem recursos para a API de cada provedor. Referências entre recursos formam um grafo: a sub-rede referencia a VPC, que é criada antes e destruída depois; o que não depende entre si sai em paralelo.

O coração do mecanismo é o **estado**: um arquivo JSON que liga cada endereço do código (`module.rede.aws_subnet.app["a"]`) ao ID real do recurso no provedor, com os atributos da última leitura. Três consequências:

- **o Terraform só gerencia o que está no estado**. Um recurso criado no console é invisível para ele até ser importado (blocos `import`, desde a 1.5);
- **o estado guarda atributos em claro**, inclusive senhas geradas e chaves. Arquivo de estado em repositório git é vazamento com controle de versão, o mesmo crime do `.env` apagado na camada do capítulo 34;
- **o estado precisa de trava**: dois `apply` simultâneos corrompem o arquivo, e o backend remoto (armazenamento de objetos com versionamento, cifra e trava) os serializa. É o `SELECT ... FOR UPDATE` da infraestrutura.

O ciclo é **plan** e **apply**. O `plan` lê o estado, consulta o provedor (*refresh*), compara com o código e mostra o diff; o `apply` executa aquele plano. Uma das linhas do plano merece medo:

```
  # module.rede.aws_subnet.dados["a"] must be replaced
-/+ resource "aws_subnet" "dados" {
      ~ cidr_block = "10.20.32.0/24" -> "10.20.32.0/23" # forces replacement
        id         = "subnet-0a1b2c3d4e5f60718"
    }

Plan: 1 to add, 0 to change, 1 to destroy.
```

`forces replacement` quer dizer que o atributo não muda no lugar: o Terraform destrói e recria. Numa sub-rede com três bancos dentro, quem não lê o `plan` descobre no `apply`, na frente do canal inteiro. `lifecycle { prevent_destroy = true }` faz o Terraform recusar qualquer plano que destrua o recurso.

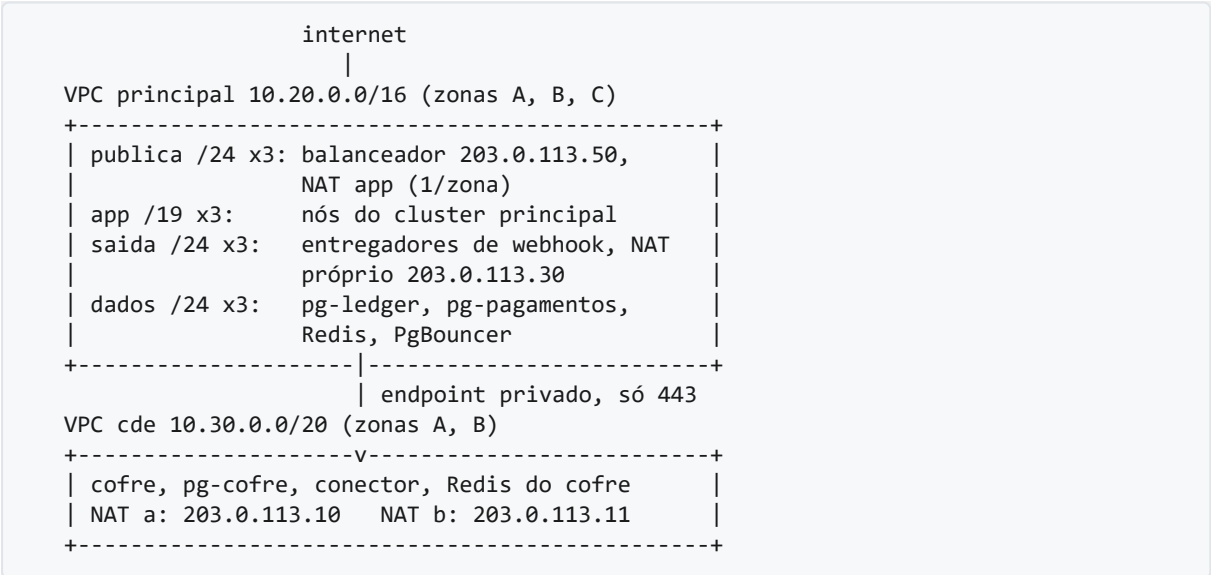
Drift é a diferença entre estado e realidade causada por mudança fora do Terraform. O `plan` a detecta nos recursos que gerencia, e `terraform plan -detailed-exitcode` sai com código 2 quando há diferença, o que permite um job agendado que alarma. O próximo `apply` desfaz o drift, porque o código é a fonte da verdade. Quem abriu porta pelo console à meia-noite a vê fechada pelo pipeline às 9h, e aprende infraestrutura declarativa do jeito que se aprende `git push --force`.

Módulos são diretórios com variáveis e saídas, reaproveitados: um módulo `zona` cria sub-redes, rotas e NAT de uma zona, e três chamadas criam três zonas. Para repetir recursos, `for_each` com chaves estáveis (`"a"`, `"b"`, `"c"`) evita o problema do `count`: remover o primeiro item renumera os outros, e o Terraform planeja recriar todos, um dominó em que cada peça é uma sub-rede de produção.

Na Lastro: a rede refeita, e o enterro

A rede nova nasceu em Terraform, numa VPC nova (`10.20.0.0/16`), porque refazer a antiga no lugar seria `forces replacement` em cascata em sub-redes com bancos dentro. O CIDR não se sobrepõe ao antigo `10.0.0.0/16`, o que permitiu ligar as duas por *peering* na migração: com CIDRs sobrepostos, o roteador não tem como saber em qual das duas redes mora o `10.0.1.10`.

O segmento de cartão ficou numa **VPC separada** (`10.30.0.0/20`), sem peering com a principal. O único caminho entre elas é um endpoint privado: pagamentos chega na entrada do cofre por uma interface na própria sub-rede, só na 443, e o cofre não tem rota nenhuma para a rede principal. O requisito é o escopo do PCI DSS: o que não tem caminho de rede até o ambiente de dados de cartão sai do escopo, e "não tem caminho" se prova com tabela de rotas, coisa que um auditor lê em minutos.



Os IPs de saída resolveram uma charada que vinha do capítulo 27. O adquirente aceita no máximo dois IPs na allowlist, e quem fala com ele agora é só o conector, dentro do segmento. Por isso o segmento tem duas zonas, A e B, com um NAT em cada: o `203.0.113.10`, transferido do proxy de saída de api-01 (é um IP reservado transferível, capítulo 8), e o `203.0.113.11`, novo, no lugar que era do `temp-01` na allowlist. Três zonas pediriam três IPs; o contrato do adquirente permite dois, e o cofre ficou com três réplicas em duas zonas. Perder uma zona deixa o segmento com metade da capacidade e o mesmo IP de saída da outra metade.

O banco do Pix recebe chamadas de pagamentos e repasses, da rede principal. Os três NATs da rede de aplicação ganharam IPs novos, o formulário do banco (dez dias úteis, firma reconhecida) foi enviado antes de tudo, e só depois da confirmação o tráfego mudou de caminho e o forward proxy de api-01 e api-02 foi desligado. Na sub-rede de saída dos webhooks (capítulo 32), a rota `local`, que não sai, mostrou o limite: o "sem rota interna" virou uma ACL sem estado que **nega** `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` e `169.254.0.0/16` e permite só 443 para fora e efêmeras de volta. Negar faixas inteiras é o que o grupo com estado, só de permissão, não faz.

Os firewalls com estado seguem um grupo por papel, com origem por grupo:

Destino	Porta	Origem permitida
nós do cluster (gateways)	8443	grupo do balanceador
PgBouncer do ledger e de pagamentos	6432	grupo dos nós do cluster
PostgreSQL	5432	grupo do PgBouncer, grupo de replicação
Redis de cache	6379	grupo dos nós do cluster
entrada do cofre (endpoint)	443	grupo dos nós de pagamentos

São 47 regras no lugar de 214, e cada uma tem um motivo escrito no pull request que a criou. Ninguém tem porta 22 aberta: o acesso a máquinas é pela sessão gerenciada do provedor, autenticada pela identidade do capítulo 30 e gravada. O grupo `default` ficou vazio, sem regra nenhuma, e um teste na CI recusa qualquer recurso que o use.

O Terraform tem um estado por camada (`rede`, `cde`, `dados`, `clusters`, `dns`), cada um com backend e trava, para que um erro no plano do DNS não alcance um banco. A rede antiga foi importada antes de desmontada, para a demolição também ter `plan`. O pipeline faz `plan` em todo pull request e `apply` só da `main`, com aprovação; humanos têm console somente leitura, com a quebra de vidro do capítulo 31. Os IPs reservados do adquirente e do banco têm `prevent_destroy`, porque perder um deles custa dez dias úteis e um formulário com firma.

Na primeira semana, o job noturno de drift alarmou às 3h: uma regra liberando 5432 para o IP de casa do Carlos, criada pelo console às 23h40 com a descrição "temporária, debug". O `apply` das 9h a removeu. O Carlos abriu um pull request com a mesma regra, a revisão perguntou o motivo, e o motivo era um `SELECT` que a réplica de relatórios responderia sem regra nenhuma.

E chegou o dia do `temp-01`. O tráfego dele foi medido por 30 dias: a integração antiga do Bazar, que resolvia DNS uma vez na vida e guardava o IP efêmero `198.51.100.7`, era a última cliente. A consultoria Java do Bazar reiniciou o servidor (quatro horas faturadas), o tráfego zerou e ficou zerado por 14 dias. Então tiraram um snapshot do disco, por via das dúvidas, e o `temp-01` foi parado. Parar uma máquina com IP efêmero devolve o IP ao provedor, sem volta: a decisão era irreversível no instante do clique, e foi tomada assim mesmo. Trinta dias depois, sem nenhum chamado, a máquina foi destruída. Uptime final: seis anos, dois meses e onze dias.

Armadilhas e casos reais

> **Na vida real:** em junho de 2014, a Code Spaces, empresa de hospedagem de código, teve o console da conta de nuvem invadido. O invasor exigiu resgate e, quando a empresa tentou retomar o controle, apagou máquinas, buckets, snapshots e backups, todos na mesma conta. A empresa anunciou o encerramento das operações dias depois. Credencial de console com poder de apagar tudo, inclusive o backup, é um único ponto de falha com senha.
>
> **Armadilha:** um NAT só, numa zona, para a rede inteira. Funciona até a zona cair, e aí as outras duas zonas, saudáveis, perdem o adquirente junto. O tráfego entre zonas até o NAT também é cobrado à parte pela maioria dos provedores; a conta chega antes do incidente.
>
> **Armadilha:** `apply` com estado local. O notebook vai para a manutenção e a produção vira órfã: os recursos existem, nenhuma ferramenta sabe que são dela. Adoção por `import`, um por um, com papelada.
>
> **Armadilha:** confiar no `plan` sem ler o `forces replacement`. Renomear um recurso no código também planeja destruir e recriar; o bloco `moved` diz ao Terraform que é o mesmo objeto com endereço novo. Sem ele, refatorar o código é demolir a produção.

Resumo

- Região reúne zonas de disponibilidade, domínios de falha independentes a 1–2 ms um do outro.
- A rede virtual é um CIDR privado; sub-redes são fatias dele, por zona na maioria dos provedores.
- Pública ou privada é a tabela de rotas: rota padrão para o gateway de internet ou para um NAT.
- A rota local liga toda a VPC; isolamento interno é firewall, e isolamento forte é outra rede.
- NAT dá IP de saída fixo e só deixa entrar respostas; vive numa zona e tem limite de portas.
- Firewall com estado devolve a resposta e aceita grupos como origem; sem estado avalia pacote a pacote e permite negar.
- Terraform: código declarativo, estado com trava e segredos, plan antes de apply, drift detectado e desfeito.
- `forces replacement`, `count` renumerado e renomeação sem `moved` destroem recursos.

A Lastro agora

A Lastro tem duas redes em Terraform: a principal, em três zonas, com sub-redes públicas, de aplicação, de saída e de dados, e a do segmento de cartão, em duas zonas, ligada à principal só por um endpoint privado na 443. O adquirente vê os IPs `203.0.113.10` e `203.0.113.11`, ambos de NATs do segmento; o banco vê os NATs da rede de aplicação; os webhooks saem pelo `203.0.113.30`, por uma sub-rede que nega a rede interna. São 47 regras com motivo, nenhuma porta 22, drift verificado toda noite e console somente leitura. O auditor refez o `nc` do `temp-01` e não conseguiu, porque o `temp-01` não existe mais. O financeiro fez um minuto de silêncio, e a planilha ganhou uma aba "in memoriam".

Com a infraestrutura inteira em código revisado, o gerente de produto concluiu que qualquer mudança agora é segura, desde que passe pela CI. Na terça seguinte, uma mudança no cálculo da tarifa passou pela CI, por dois aprovadores e pelo deploy, foi para todas as réplicas de pagamentos de uma vez e passou uma hora cobrando um centavo a mais em cada cobrança. A CI estava verde. O financeiro, não.

Referências

- RFC 1918: *Address Allocation for Private Internets*. https://www.rfc-editor.org/rfc/rfc1918
- AWS. *How Amazon VPC works* (sub-redes, tabelas de rotas, gateways). https://docs.aws.amazon.com/vpc/latest/userguide/how-it-works.html
- AWS. *NAT gateways*. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
- AWS. *Compare security groups and network ACLs*. https://docs.aws.amazon.com/vpc/latest/userguide/infrastructure-security.html
- Google Cloud. *VPC networks*. https://cloud.google.com/vpc/docs/vpc
- HashiCorp. *Terraform Language Documentation* (state, plan, `import`, `moved`, `lifecycle`). https://developer.hashicorp.com/terraform/language
- HashiCorp. *Sensitive data in state*. https://developer.hashicorp.com/terraform/language/state/sensitive-data
- OpenTofu. https://opentofu.org/
- PCI Security Standards Council. *Guidance for PCI DSS Scoping and Network Segmentation*. https://www.pcisecuritystandards.org/

CAPÍTULO 37

CI/CD e estratégias de deploy

O pull request que não mudava nada

O pull request se chamava "refatora cálculo de tarifa (sem mudança de comportamento)", e os dois aprovadores aprovaram o título. Era terça, 9h40. O gerente de produto queria mostrar a tarifa no checkout antes do pagamento ("é só um número na tela, não?"), e o Carlos extraiu o cálculo para uma função compartilhada. No caminho, trocou o modo de arredondamento. Com um comentário, para ninguém dizer que ele não documenta:

```
>>> from decimal import Decimal, ROUND_HALF_EVEN, ROUND_UP
>>> bruto = Decimal("34990") * Decimal("0.0099")  # R$ 349,90 a 0,99%, em centavos
>>> bruto
Decimal('346.4010')
>>> bruto.quantize(Decimal("1"), ROUND_HALF_EVEN)  # o que o capítulo 1 decidiu
Decimal('346')
>>> bruto.quantize(Decimal("1"), ROUND_UP)  # "pra nunca cobrar menos"
Decimal('347')
```

`ROUND_UP`, no `decimal` do Python, arredonda para longe do zero: qualquer fração vira um centavo inteiro. Os testes passaram, porque todos os fixtures usavam R$ 100,00, cuja tarifa dá 99 centavos exatos. A CI ficou verde. A ferramenta mais rigorosa do pipeline continuava sendo o título do PR.

O deploy saiu às 10h02. O rolling update do capítulo 35 trocou os 30 pods de pagamentos em quatro minutos, sem perder uma autorização. Os pods passaram em todas as probes: saudáveis, rápidos e cobrando um centavo a mais de quase todo mundo, com latência excelente.

Às 10h51, o vendedor 42, que confere cada venda numa planilha própria desde o saque duplo, achou R$ 0,01 de diferença. Às 10h58, o Bazar abriu chamado: "URGENTE!!!! VOCÊS ESTÃO COBRANDO CENTAVO ESCONDIDO???". O rollback terminou às 11h06. Saldo: 131 mil das 140 mil cobranças da hora com tarifa um centavo acima, R$ 1.310,00 tirados de vendedores, em 131 mil lançamentos imutáveis que nenhum rollback desfaz. A conciliação das 6h teria achado no dia seguinte. O vendedor 42 achou em 49 minutos, de graça, e pela primeira vez na vida estava do lado certo de uma divergência.

Como uma mudança chega à produção expondo pouca gente primeiro, com a métrica certa olhando, e o que se faz com o dinheiro que a versão errada já gravou?

O pipeline: compile uma vez, promova o mesmo artefato

Integração contínua (CI) é integrar cada mudança à branch principal com build e testes automáticos. **Entrega contínua** (CD) é manter essa branch sempre implantável, com a ida à produção dependendo de uma decisão; **implantação contínua** dispensa a decisão. O pipeline é a sequência de estágios entre commit e produção, cada um barrando uma classe de defeito:

```
commit ─► build ─► testes ─► análise ─► publica ─► homologação ─► produção
          (imagem   (unidade,  (lint,     (registry,  (mesmo digest)  (mesmo digest,
          OCI)      contrato,  pip-audit, digest,                     canary)
                    integração) SBOM)     assinatura)
```

A regra que segura o pipeline inteiro é o **artefato imutável**: a imagem é construída uma vez, identificada pelo digest (capítulo 34), e esse mesmo digest passa por homologação e chega à produção. Reconstruir por ambiente parece inofensivo até o `pip install` da produção baixar uma dependência que a homologação nunca viu. Aí você testou um binário e implantou o primo dele, aquele que ninguém convida para o churrasco.

Por isso o que muda entre ambientes fica fora da imagem: configuração e segredos (capítulo 33). Migrações são estágio próprio, na ordem do capítulo 7: *expand* antes da versão que depende dele, *contract* num deploy posterior.

Por último, a **proveniência**: o pipeline assina a imagem e registra de que commit ela saiu (o SLSA descreve níveis disso; Sigstore é um exemplo de assinatura), e o cluster só admite imagem assinada e por digest. Sem isso, o Kubernetes executa o que aparecer no registry com a obediência de um estagiário no primeiro dia.

Credenciais de curta duração: o pipeline sem chave no bolso

O pipeline publica imagens, altera infraestrutura e implanta em produção: tem mais poder que o fundador desde o capítulo 31, e por anos foi autenticado com uma chave permanente nas variáveis secretas da CI. A chave vazava por log, por dependência comprometida ou pelo fornecedor da CI, e funcionava para quem a achasse. Chave de CI é o único funcionário que nunca tira férias, nunca é desligado e nunca passou por entrevista.

A alternativa é **federação por OIDC**. O fornecedor da CI atua como provedor de identidade: em cada job, emite um ID token JWT (capítulo 30) assinado por ele, descrevendo quem está pedindo. No GitHub Actions, por exemplo:

```
{
  "iss": "https://token.actions.githubusercontent.com",
  "sub": "repo:lastro/pagamentos:environment:producao",
  "aud": "sts.amazonaws.com",
  "ref": "refs/heads/main",
  "sha": "4e1f0c2a9b...",
  "exp": 1793872920
}
```

O job apresenta o token ao serviço de credenciais da nuvem (o AWS STS é um exemplo; Google Cloud e Azure têm federação equivalente). A nuvem busca as chaves públicas do emissor pelo JWKS, valida assinatura, `iss`, `aud` e `exp`, e confere a **política de confiança** do papel, que só aceita um `sub` exato. Passou, devolve credenciais que expiram em minutos. Não há segredo guardado na CI; o que vaza num log morre antes do café esfriar.

A segurança inteira mora na condição sobre o `sub`. Uma política que aceita `repo:lastro/*` entrega a produção a qualquer branch de qualquer repositório da organização, inclusive o `teste-do-estagiario-nao-apagar`. O OIDC troca o problema "guardar a chave" pelo problema "escrever a condição", e a condição, ao contrário da chave, aparece num `plan` revisado.

Estratégias de deploy: quanto tráfego a versão nova machuca primeiro

Estratégia	Versões juntas	Capacidade extra	Exposição antes da decisão	Tempo de rollback
Recreate (para tudo, sobe tudo)	nenhuma	nenhuma	100%, com indisponibilidade	um deploy inteiro
Rolling update	2, por minutos	`maxSurge`	cresce até 100% sem critério além das probes	outro rolling (minutos)
Blue-green	2 ambientes, 1 recebe	100%	100% de uma vez	segundos (troca de rota)
Canary	2, por dezenas de minutos	pequena	1%, 5%, 25%... com análise em cada passo	segundos (peso 0)

Recreate derruba a versão velha e sobe a nova, com indisponibilidade no meio. Exige janela de manutenção, e o Pix 24x7 não deu janela nem para o velório do `temp-01`.

Rolling update é o que o Kubernetes já faz: troca pods em lotes e só avança quando o pod novo passa na readiness. O critério é "o processo responde", e um processo que cobra errado responde perfeitamente. O rolling protege contra versão que não sobe. Contra

versão que sobe e erra, ele é um distribuidor com logística de primeiro mundo.

Blue-green mantém dois ambientes completos; o verde recebe a versão nova e, num instante, o balanceador aponta tudo para ele. Rollback é apontar de volta, em segundos. Custa o dobro da capacidade durante a troca, e a exposição é total: o bug atinge todo mundo ao mesmo tempo, só que com um botão de desfazer melhor. É o paraquedas reserva de quem pula sem olhar a altura.

Canary manda uma fração pequena do tráfego para a versão nova, compara as métricas dela com as da versão estável (a *baseline*) no mesmo período e só aumenta a fração se empatarem. Mesmo tráfego, mesma hora, mesma carga: acabou o "às 10h é sempre assim". O nome vem do canário que os mineiros levavam para a mina, e a ideia continua a mesma, só que o bicho agora é um pod e ninguém fica triste quando ele morre.

O canary é um teste estatístico com tráfego de verdade

A pergunta de cada degrau é se a diferença entre canary e baseline é maior que o ruído. Com aprovação de cartão de 85%, o desvio padrão da proporção medida em `n` autorizações é `√(0,85 × 0,15 / n)`. Com 100 autorizações, 3,6 pontos percentuais: uma queda de 5 pontos some no ruído. Com 2.000, 0,8 ponto, e a queda aparece com folga. Por isso o canary tem degraus: 1% pega o desastre grosso (erro 500, crash), 5% e 25% pegam as quedas sutis. Canary de cinco minutos com 1% do tráfego mede a sorte, e a sorte tem variância alta.

Com o Service do Kubernetes, a fração é o número de pods (1 novo em 30 dá 3,3%); um mesh com pesos por requisição, como o do capítulo 35, manda 1% com quantos pods existirem. Controladores de entrega progressiva (Argo Rollouts e Flagger são exemplos) sobem os degraus, consultam as métricas e voltam o peso a zero sozinhos quando a análise reprova. O controlador não tem chefe, não tem medo de postmortem e não acha que "deve ser só ruído".

E a mesma cobrança pode ser criada pela versão nova e capturada pela velha: a matriz de compatibilidade do capítulo 7 passa a valer por uma hora, e não por quarenta segundos.

Feature flags: deploy é uma coisa, lançamento é outra

Uma **feature flag** é uma condição no código cujo valor vem de configuração em tempo de execução. Ela separa **deploy** (o código novo está nos processos) de **lançamento** (o comportamento novo está ligado para alguém): o código vai desligado, liga para um marketplace, depois para todos, sem deploy no meio.

Pelo tempo de vida, há flags de lançamento (dias), operacionais ou *kill switch* (permanentes, desligam uma peça cara sob carga), de experimento e de permissão. A avaliação é em memória, com a configuração empurrada aos processos; um serviço de flags consultado por requisição vira dependência síncrona do caminho do dinheiro, com a disponibilidade composta do capítulo 26 cobrando a conta.

O custo é combinatório: dez flags booleanas dão 1.024 comportamentos, e a CI testa uns cinco. Flag de lançamento que não morre vira código morto com interruptor, um zumbi esperando alguém reaproveitar o nome dele. A seção de armadilhas tem um zumbi que custou mais de 460 milhões de dólares.

Em sistema de dinheiro, a decisão tomada sob uma flag fica registrada junto do efeito. Se a cobrança foi tarifada pelo cálculo novo, a cobrança diz isso, e a conciliação refaz a conta com a regra certa daqui a três anos, quando a flag já tiver sido apagada e o autor dela estiver vendendo curso.

Rollback, roll forward e o dinheiro que já foi gravado

Rollback é voltar à versão anterior; **roll forward** é corrigir e implantar uma nova. O rollback é mais rápido (peso zero em segundos, `rollout undo` em minutos) e só exige reconhecer o bug. O roll forward é o caminho quando voltar quebra algo: a versão nova gravou um estado que a antiga não sabe ler. Pipeline de onze minutos torna o roll forward viável; pipeline de duas horas transforma cada bug em duas horas de bug, mais o tempo de alguém achar o crachá.

E nenhum dos dois desfaz efeitos. Rollback volta o código; os lançamentos continuam no ledger, os eventos no tópico, os webhooks entregues e o Pix no banco de destino. Rollback é pedir desculpas ao vivo: interrompe a ofensa, não desdiz nada. Num ledger imutável (capítulo 1), dado errado se corrige com transação compensatória, referenciando a original, idempotente como no capítulo 16. O rollback estanca a sangria; a correção dos dados é um segundo projeto, com dono, contagem e conferência.

Na Lastro

A limpeza da terça

A correção foram 131 mil transações de ajuste: débito na receita da Lastro, crédito de um centavo ao vendedor (a liberar, no cartão; disponível, no Pix), origem `ajuste`, referência `ajuste-tarifa:{cobranca_id}` UNIQUE (capítulo 16), para o job parar e recomeçar sem duplicar centavo. O extrato mostra "Ajuste de tarifa (devolução)"; o evento novo

`cobranca.tarifa_ajustada` é mudança aditiva (capítulo 11). O financeiro conferiu pela soma: R$ 1.310,00. O tabelião registrou 131 mil pedidos de desculpas com firma reconhecida.

Pagamentos: canary com métricas de negócio

Blue-green dobraria os 30 a 60 pods a cada troca e exporia 100% das cobranças de uma vez; seria a escolha se as versões não pudessem conviver, o que o expand/contract já evita. Probe não sabe quanto custa uma tarifa. A Lastro escolheu canary para pagamentos pelo requisito de que um erro de cálculo atinja uma fração pequena das cobranças antes de ser detectado, ao custo de cerca de uma hora por deploy até 100%. O mesh faz a divisão por peso; um controlador de entrega progressiva conduz os degraus:

Degrau	Peso	Duração	Cobranças na versão nova (manhã típica, ~39/s)
1	1%	10 min	~230
2	5%	15 min	~1.750
3	25%	15 min	~8.800
4	50%	15 min	~17.500
5	100%		

Toda cobrança grava `versao_servico` (o digest da imagem), e as métricas se dividem por versão. A análise volta o peso a zero se o canary piorar em relação à baseline em: taxa de `5xx`; p99 da autorização (orçamento do capítulo 14); fração de cartão em `processando`; aprovação de cartão, a partir do degrau 2, quando a amostra tem poder estatístico; e recusas do consumidor do ledger para cobranças daquela versão.

A métrica que teria pegado a terça é o **resíduo de arredondamento**, `tarifa_cobrada - valor × taxa` em centavos: com half-even, fica entre −0,5 e +0,5, média zero; com `ROUND_UP`, entre 0 e 1, média 0,5. O desvio padrão de uma uniforme de largura 1 é ~0,29; com as 230 cobranças do degrau 1, o erro padrão da média é $0{,}29/\sqrt{230} \approx 0{,}02$, e o deslocamento vira 25 erros padrão. O bug do Carlos morreria aos dez minutos, com uns 2 reais de prejuízo. O vendedor 42 nem teria tido tempo de abrir a planilha.

Consumidores (ledger, entregador) não recebem requisição: neles, comportamento novo vai por flag, com deploy rolling e as mesmas métricas.

Flags com recibo

As flags são versionadas no banco de configuração e chegam aos processos pelo caminho das tarifas do capítulo 22: cache em memória, TTL de 60 s, `NOTIFY` para urgência. Toda mudança de regra de dinheiro (tarifa, split, arredondamento, antifraude) vai atrás de flag

por marketplace, ligada primeiro para a "Lastro Interna", um marketplace de produção da própria empresa que faz Pix reais de R$ 1,00 (o único cliente que nunca abriu chamado URGENTE), depois para um marketplace pequeno, depois para todos. A cobrança grava `regras_aplicadas` (versão da tabela de tarifas e versão do cálculo), e a conciliação das 6h refaz a conta com as duas. Flag de lançamento nasce com data de expiração, a CI falha se o código usa flag vencida, e nome de flag morta não se reaproveita: vai para a lápide, como os campos `reserved` do capítulo 12.

O pipeline sem chave permanente

A CI usava uma chave permanente de administrador chamada `ci-temporario`, criada em 2021 e compartilhada por todos os repositórios: temporária no mesmo sentido em que o `temp-01` era provisório. Foi apagada e trocada por papéis federados por OIDC, um por função, com `sub` exato:

Papel	Pode	`sub` aceito
`ci-publica-imagem`	push no registry, assinar	`repo:lastro/<serviço>:ref:refs/heads/main`
`ci-terraform-plan`	leitura	qualquer PR dos repositórios de infra
`ci-terraform-apply`	escrita na camada	`repo:lastro/infra:environment:producao`

As credenciais duram 15 minutos. O deploy no cluster é por pull: o pipeline grava o digest novo num repositório de manifestos e um controlador dentro do cluster reconcilia (o princípio do capítulo 35; Argo CD e Flux são exemplos). A CI não tem credencial do cluster, e um vazamento dela não alcança os pods. O segmento de cartão tem pipeline e papéis próprios, para que uma credencial que publica no cofre não arraste o escopo do PCI DSS para tudo o que a usa. Do merge ao degrau 1 são 11 minutos, o que mantém o roll forward como opção real.

Armadilhas e casos reais

> **Na vida real:** em 1º de agosto de 2012, a Knight Capital implantou código novo de roteamento de ordens em oito servidores, e um deles ficou com a versão antiga. A versão nova reaproveitava uma flag que, no código antigo, ativava uma funcionalidade abandonada havia anos, a "Power Peg". No servidor esquecido, a flag ressuscitou o código morto, que passou a enviar ordens sem parar. Em cerca de 45 minutos, a empresa perdeu mais de 460 milhões de dólares, segundo a SEC. Deploy manual sem conferir se todas as máquinas convergiram, mais um nome de flag reaproveitado: duas linhas deste capítulo, cobradas em nove dígitos.

> **Na vida real:** em janeiro de 2023, a CircleCI informou que um malware no notebook de um engenheiro roubou uma sessão autenticada, usada para acessar segredos de clientes, e recomendou que todos girassem tudo o que guardavam lá. Quem usava OIDC girou menos: não guardava nada.
>
> **Armadilha:** configuração também é deploy. Tabela de tarifas, regra de antifraude ou flag mudada para 100% de uma vez tem a exposição de um recreate, sem pipeline nenhum. Na Lastro, flag também sobe em degraus, com a mesma análise.
>
> **Armadilha:** canary com a métrica errada. Taxa de `5xx` e latência aprovam com louvor um código que cobra errado. Sem métrica de negócio, o canary é um rolling update com cerimônia de formatura.

Resumo

- Artefato imutável promovido pelo digest; configuração fora da imagem; imagem assinada.
- OIDC na CI: token por job, `sub` exato, credenciais de minutos.
- Estratégias diferem em versões juntas, capacidade extra, exposição e tempo de rollback.
- Canary é teste estatístico em degraus, com baseline e métrica de negócio.
- Flags separam deploy de lançamento, expiram e deixam recibo no dado.
- Rollback estanca; dado gravado se corrige com transação compensatória.

A Lastro agora

Pagamentos sai em canary de cinco degraus, com métricas por digest, inclusive o resíduo de arredondamento. Regras de dinheiro vão atrás de flag por marketplace, começando pela Lastro Interna, com `regras_aplicadas` conferido pela conciliação. A CI não guarda credencial: OIDC, 15 minutos, deploy por pull, imagem assinada. Os 131 mil centavos voltaram, e o vendedor 42 conferiu o dele antes do financeiro.

O PR seguinte do Carlos se chamou "ajusta arredondamento (COM mudança de comportamento)", em maiúsculas, e foi o primeiro título de PR da Lastro a dizer a verdade. O canary agora vigia cada métrica que alguém pensou em medir. O problema são as que ninguém pensou: na sexta seguinte, a taxa de aprovação vai cair sem nenhum deploy, e quem vai perceber primeiro, de novo, é o Bazar.

Referências

- Humble, J.; Farley, D. *Continuous Delivery*. Addison-Wesley, 2010.
- Kubernetes. *Deployments*. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- GitHub. *About security hardening with OpenID Connect*. https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/about-security-hardening-with-openid-connect
- OpenID Foundation. *OpenID Connect Core 1.0*. https://openid.net/specs/openid-connect-core-1_0.html
- SLSA. *Supply-chain Levels for Software Artifacts*. https://slsa.dev/
- Sigstore. https://www.sigstore.dev/
- Argo Rollouts. https://argoproj.github.io/argo-rollouts/
- Python. *decimal: Decimal fixed point and floating point arithmetic*. https://docs.python.org/3/library/decimal.html
- SEC. *In the Matter of Knight Capital Americas LLC*, Release No. 70694, 2013. https://www.sec.gov/litigation/admin/2013/34-70694.pdf
- CircleCI. *CircleCI incident report for January 4, 2023 security incident*. https://circleci.com/blog/jan-4-2023-incident-report/

CAPÍTULO 38

Observabilidade

O dashboard verde e o Bazar vermelho

Às 19h25 de uma sexta, o Bazar abriu chamado com o assunto "URGENTE!!!! APROVAÇÃO CAIU, O QUE VOCÊS FIZERAM???", e a resposta honesta era "nada", que é a pior resposta possível. Não houve deploy. O canary do capítulo anterior vigiava a taxa de aprovação com rigor estatístico, durante deploys. Fora deles, a aprovação podia despencar à vontade, sem plateia.

O Carlos abriu o dashboard. Taxa de `5xx`: zero. p99 da autorização: 1,1 s, melhor que o normal. CPU: 31%. Lag do event loop: 2 ms. Tudo verde, num tom de verde que só se vê em painel de empresa que está perdendo dinheiro. O Bazar, olhando a própria conversão, via a aprovação de cartão sair de 85% para 72% desde as 18h52.

Às 20h10, a equipe estava fazendo `grep` em logs de 60 pods, um por um, em texto livre, sem saber o que procurar. Às 21h05 alguém achou, entre dois milhões de linhas iguais, esta:

```
WARNING antifraude: falha ao ler sinais, seguindo sem
```

O `redis-sinais` do capítulo 25 tinha batido o `maxmemory` às 18h52. Configurado com `noeviction`, ele continuava aceitando leituras e recusava escritas com `OOM command not allowed when used memory > 'maxmemory'`. O pipeline de leitura e incremento falhava inteiro, o antifraude tratava a falha como "Redis fora" e seguia sem sinais, exatamente como o capítulo 25 projetou. O Vigia, sem os contadores, recusava mais. A degradação controlada funcionou com perfeição, e ninguém sabia que ela estava ligada. Em 2h20, umas 16 mil autorizações a mais foram recusadas. Nenhum centavo errado no ledger; só vendas que não aconteceram, que é o tipo de prejuízo que não aparece em conciliação nenhuma.

O sistema sabia de tudo: cada processo tinha escrito a falha no log, dois milhões de vezes, num lugar que ninguém abria desde a visita do auditor. Como se constrói um sistema em que a pergunta "por que a aprovação caiu?" se responde em minutos, e não em duas horas de `grep`?

Monitorar o que você sabe, observar o que você não sabe

Monitoramento responde perguntas feitas com antecedência: a CPU passou de 80%? O `5xx` subiu? Cada dashboard e cada alerta é uma pergunta que alguém pensou em fazer. **Observabilidade** é a propriedade de um sistema cujos sinais permitem responder perguntas que ninguém fez antes, sem mudar código e sem deploy novo. O termo vem da teoria de controle (Kálmán, 1960), onde mede se o estado interno pode ser inferido das saídas.

A sexta-feira foi uma pergunta nova: "o que mudou nas autorizações recusadas desde as 18h52?". O monitoramento da Lastro respondia com perfeição a todas as perguntas antigas. Todo dashboard é o postmortem do incidente anterior, emoldurado, e o incidente seguinte, por educação, nunca repete o figurino.

Os sinais clássicos são três: **logs** (eventos discretos com contexto), **métricas** (números agregados no tempo) e **traces** (o caminho de uma requisição pelos serviços). Cada um responde um tipo de pergunta a um custo diferente, e nenhum substitui os outros, apesar de o Carlos querer os três num `print`.

Logs estruturados: o evento com campos

Log em texto livre é escrito para um humano lendo um processo. Com 60 pods, ninguém lê; consulta-se. **Log estruturado** é um registro com campos nomeados, em geral JSON por linha, que um sistema de busca indexa e filtra:

```
{"ts": "2027-03-12T21:52:07.114Z", "nivel": "WARNING", "servico": "antifraude",
 "versao_servico": "sha256:7c1e...", "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
 "evento": "sinais.indisponiveis", "causa": "redis_oom", "marketplace_id": 7,
 "cobranca_id": "0195f2a4-..."}
```

A diferença é a consulta: `evento = "sinais.indisponiveis" agrupado por causa` devolve uma tabela em segundos, onde o `grep` devolvia um fim de semana. O campo `evento` é um nome estável, que código e consulta compartilham; a mensagem livre muda a cada refatoração do Carlos.

Os custos de log são volume e vazamento. Volume, porque cada linha é paga na ingestão, no índice e na retenção, e um `WARNING` por requisição a 400 requisições por segundo dá 35 milhões de linhas por dia, quase todas idênticas. Vazamento, porque o log é a cópia de dados mais esquecida da empresa: o capítulo 33 achou 204 mil cartões num log de depuração, e a solução de lá (mascaramento na origem e detector de PAN no coletor) continua valendo aqui. Corpo de requisição, token e CPF ficam fora do log, e quem discorda pode reler o capítulo 33, de joelhos, no milho.

Métricas: contadores, gauges, histogramas e a conta da cardinalidade

Uma **métrica** é uma série temporal: um nome, um conjunto de rótulos e valores amostrados no tempo. O custo não depende do tráfego; um contador que soma um milhão de eventos por segundo ocupa o mesmo espaço que um que soma um. É o sinal mais barato, e por isso alerta se faz com ele.

Os tipos, no modelo do Prometheus (que virou referência e é o formato que o OpenTelemetry também exporta):

- **Contador**: só cresce, zera quando o processo reinicia. O valor bruto não significa nada; o que se consulta é a taxa, `rate(x[5m])`, que já trata a volta a zero.
- **Gauge**: sobe e desce. Memória usada, conexões abertas, tamanho de fila.
- **Histograma**: contadores por faixa de valor (*buckets*), cumulativos, mais soma e contagem. É o tipo que dá percentis.

```
autorizacao_duracao_segundos_bucket{le="0.5"} 1210
autorizacao_duracao_segundos_bucket{le="1.0"} 8433
autorizacao_duracao_segundos_bucket{le="2.5"} 9902
autorizacao_duracao_segundos_bucket{le="+Inf"} 10000
autorizacao_duracao_segundos_count 10000
```

O p99 é a posição 9.900, que cai no bucket entre 1,0 e 2,5 s. `histogram_quantile` interpola linearmente dentro do bucket e estima ~2,5 s, mesmo que o p99 real seja 1,3 s. O percentil de um histograma só é tão bom quanto os buckets perto do valor que importa, então os buckets se escolhem em volta da meta (para a Lastro, 1,5 s do contrato do capítulo 14), e não pelo padrão da biblioteca. A vantagem que compensa: buckets se somam entre processos, e percentis não (a média de percentis do capítulo 14, a de "alma de horóscopo", continua inválida).

Cada combinação de rótulos é uma série separada, e o número de séries é o produto das cardinalidades. `metodo` (2) × `marketplace` (42) × `bandeira` (5) × `motivo_recusa` (12) dá 5.040 séries, tranquilo. Acrescente `cobranca_id` como rótulo e você terá uma série por cobrança, com um ponto só, guardada como série temporal: o banco de métricas explode de memória e o boleto da observabilidade passa o da nuvem. Identificador de alta cardinalidade vai para log e trace, onde cada evento já é uma linha. Esse é o ponto do capítulo 25: série temporal comprime bem justamente porque as séries são poucas e longas.

Tracing distribuído: a cobrança com rastro

Um **trace** é a árvore de operações de uma requisição. Cada operação é um **span**, com nome, início, duração, atributos e o ID do span pai. O trace inteiro compartilha um `trace_id`. O que transforma isso em sistema distribuído é a **propagação de contexto**: cada chamada de saída leva o `trace_id` e o ID do span atual, e o serviço do outro lado cria spans filhos dele.

O formato padrão é o **W3C Trace Context**, num cabeçalho HTTP:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

Versão, `trace-id` de 16 bytes, `parent-id` de 8 bytes e flags (o bit final diz se o trace foi amostrado). O gRPC leva o mesmo valor nos metadados. Mensagens assíncronas levam o contexto no cabeçalho do registro do broker; o trace atravessa a fila e o consumidor continua a árvore, ou, quando processa um lote de mensagens de traces diferentes, cria **links** para cada uma, porque um span só tem um pai. O tracing é monogâmico; o lote é uma relação aberta, e o link é o registro de cada caso.

Guardar todos os spans custa caro, então se amostra. **Amostragem na cabeça** decide no primeiro serviço, por sorteio, e propaga a decisão no bit de flags: barata, e cega, porque decide antes de saber se a requisição vai dar erro. É escolher quem sai nas fotos do casamento antes de saber quem vai cair bêbado na piscina. O Dapper do Google, artigo de 2010 que popularizou a técnica, amostrava 1 em 1.024. **Amostragem na cauda** guarda tudo temporariamente num coletor e decide quando o trace termina: fica com 100% dos erros e lentos e uma fração do resto. Custa memória no coletor e exige que todos os spans de um trace cheguem ao mesmo coletor.

OpenTelemetry: um padrão para os três

O **OpenTelemetry** (projeto da CNCF, fusão do OpenTracing com o OpenCensus em 2019, o raro caso da computação em que dois padrões concorrentes viraram um, e não três) padroniza a produção de telemetria, sem ser backend de armazenamento. Tem uma API que o código e as bibliotecas chamam, um SDK que implementa amostragem e exportação, instrumentação automática para frameworks comuns (FastAPI, httpx, psycopg, gRPC, Kafka), as **convenções semânticas** (nomes padronizados como `http.response.status_code`) e o protocolo **OTLP**.

O **Collector** é um processo separado com pipelines de *receivers*, *processors* e *exporters*: recebe OTLP dos serviços, amostra na cauda, filtra atributos, mascara, e exporta para o backend que a empresa usar (Prometheus, Jaeger, Grafana Tempo e Loki são exemplos

de código aberto; há dezenas de serviços comerciais). Trocar de backend vira configuração do Collector, sem reinstrumentar os serviços. O vendor lock-in muda de endereço, mas pelo menos muda para um YAML, que é onde todo mundo já espera sofrer.

Exemplars fecham o ciclo: um ponto de histograma carrega o `trace_id` de uma requisição que caiu naquele bucket. Do pico no gráfico de p99 vai-se direto a um trace lento.

RED, USE e a métrica que o negócio sente

Dois métodos dizem o que medir em cada coisa. O **RED** (Tom Wilkie) vale para serviços que atendem requisições: *Rate* (requisições por segundo), *Errors* (falhas por segundo) e *Duration* (distribuição de latência). O **USE** (Brendan Gregg) vale para recursos: *Utilization* (fração do tempo ocupado), *Saturation* (trabalho esperando) e *Errors*. Para um pool de conexões, utilização é conexões em uso sobre o tamanho, saturação é quem espera na fila do pool, erro é timeout de aquisição. Os "quatro sinais dourados" do livro de SRE do Google (latência, tráfego, erros, saturação) cobrem os dois.

A sexta-feira passou por RED e USE com nota dez. RED dos serviços: taxa normal, zero erros, latência ótima. USE do Redis: ninguém tinha aplicado; memória a 100% do `maxmemory` e escritas recusadas são, respectivamente, U e E de manual. E faltou a terceira camada: **métricas de negócio**, contadas como qualquer contador, que dizem se o sistema faz o que a empresa vende. Uma API que responde `201` rapidamente para uma autorização recusada pelo motivo errado é, para o RED, um serviço exemplar.

Na Lastro

Os três sinais, de ponta a ponta

Todos os serviços usam o SDK do OpenTelemetry com instrumentação automática de FastAPI, httpx, gRPC, psycopg e do cliente Kafka, e exportam OTLP para um Collector por nó (DaemonSet), que encaminha a um Collector central com amostragem na cauda. O segmento de cartão tem Collector próprio no cluster dele, e só spans com atributos de uma lista permitida saem de lá: nada de PAN, BIN ou token do cofre, com o detector de Luhn do capítulo 33 descartando e contando qualquer atributo suspeito. O backend fica em armazenamento gerenciado, separado por sinal; a escolha do produto é configuração do Collector.

Um trace de cobrança de cartão, como o Collector monta:

```
POST /v1/cobrancas  trace 4bf92f35...                      0 ms ──────────── 912 ms
├─ gateway: limite + token                                  2 ms
├─ pagamentos: tx1 (cobrança + chave idempotência)          4 ms
├─ antifraude.avaliar                                       9 ms
│  ├─ redis-sinais: MGET + INCR (pipeline)                  1 ms
│  └─ Vigia.Avaliar (gRPC)                                  6 ms
├─ conector-adquirente.autorizar (segmento cartão)        871 ms
│  └─ cofre.detokenizar                                     2 ms
└─ pagamentos: tx2 (estado + evento na outbox)              5 ms
   ┄┄┄ link ┄┄┄ ledger-consumidor: lançamentos            +340 ms
   ┄┄┄ link ┄┄┄ webhooks-entregador: POST ao marketplace  +1,2 s
```

O contexto atravessa a parte assíncrona assim: a linha de `eventos_saida` (a outbox do capítulo 18) ganhou a coluna `traceparent`, o CDC a copia para um cabeçalho do registro no `cobrancas.eventos`, e o consumidor do ledger e o agendador de webhooks continuam o trace. O entregador envia ao marketplace o cabeçalho `traceparent` com um `trace_id` que o suporte pode consultar. Como o webhook é a única parte que o marketplace vê, o painel mostra o `trace_id` de cada entrega; o Bazar agora pode colar no "URGENTE!!!!" um identificador útil, o que só dá para torcer.

A amostragem na cauda guarda 100% dos traces com erro, com estado `processando` ou `recusada`, e com duração acima de 2 s; do resto, 5%. São ~39 cobranças por segundo de manhã e cerca de 25 spans por cobrança, uns mil spans por segundo antes da amostragem.

Logs com contrato

Logs em JSON, uma linha por evento, com campos obrigatórios: `ts`, `nivel`, `servico`, `versao_servico` (o digest do capítulo 37), `trace_id`, `span_id`, `evento` e, quando houver, `marketplace_id` e `cobranca_id`. O formatador recusa campo fora de uma lista por serviço, para que o próximo `logger.debug(corpo)` falhe na CI, e não no relatório do auditor. Repetição vira métrica: um evento que acontece em toda requisição é contador, com no máximo um log de amostra por minuto. Retenção de 30 dias pesquisável e um ano em armazenamento frio; os logs do segmento de cartão têm retenção e acesso próprios.

Métricas: RED, USE e o negócio

Camada	Métrica (exemplos)	Rótulos permitidos
RED	requisições, erros, histograma de duração por rota	`servico`, `rota`, `status`, `versao_servico`

Camada	Métrica (exemplos)	Rótulos permitidos
USE	pool e PgBouncer (uso, espera), memória do Redis / `maxmemory`, comandos recusados, lag de consumidor, `nr_throttled`	`instancia`
Negócio	autorizações por resultado e motivo; aprovação; Pix confirmados por minuto; webhooks entregues; resíduo de arredondamento	`marketplace`, `metodo`, `bandeira`, `motivo`
Degradação	`antifraude_sinais_ausentes_total`, breaker aberto, descarte de carga, retry consumido	`servico`, `causa`

Os buckets do histograma de autorização são 0,25, 0,5, 0,75, 1, 1,25, 1,5, 2, 3, 5 e 8 s: densos perto de 1,5 s e com o último no deadline de 8 s do capítulo 15. O Collector descarta qualquer rótulo fora da lista e conta o descarte, porque a primeira pessoa a pôr `vendedor_id` num rótulo vai ser, estatisticamente, o Carlos. São cerca de 400 mil séries ativas no total.

A regra nova é a da última linha da tabela: **todo caminho de degradação incrementa um contador**, e todo contador de degradação tem alerta. O código do capítulo 25 seguia sem sinais em silêncio; agora conta, por causa (`redis_oom`, `timeout`, `conexao`). Na sexta, esse contador teria saído de zero para 15 por segundo às 18h52.

A aprovação por marketplace e método é comparada com a mesma janela da semana anterior; queda de mais de 5 pontos por 10 minutos alerta, com o detalhamento por motivo de recusa no próprio alerta. A memória do `redis-sinais` alerta a 80% do `maxmemory`, e o `maxmemory` foi recalculado pelo volume do Tuiuiú, com a mesma conta de ordem de grandeza do capítulo 29. Como esses alertas se transformam em metas com orçamento de erro é o assunto do capítulo 39.

O diagnóstico da sexta, refeito com os sinais novos: alerta de aprovação do Bazar às 19h02; motivo dominante `antifraude_recusa`; o contador de sinais ausentes com causa `redis_oom`; o gauge de memória do `redis-sinais` em 100%. Quatro cliques, dez minutos depois do início, 23 minutos antes do Bazar.

Armadilhas e casos reais

> **Na vida real:** o artigo do Dapper (Sigelman et al., Google, 2010) descreve o tracing em produção que inspirou o Zipkin, o Jaeger e o próprio OpenTelemetry, com propagação de contexto e amostragem uniforme de 1 em 1.024 requisições. A observação que sobreviveu: o rastreamento só funciona se for automático e

onipresente, porque o serviço que não propaga o contexto corta o trace no meio, e o pedaço que falta é sempre o que você precisava ver. Trace com um serviço sem instrumentação é autópsia com um órgão faltando: dá para assinar o laudo, só não dá para dizer do que o paciente morreu.

Armadilha: degradação silenciosa. Fallback, retry, breaker e cache stale existem para esconder falhas do usuário, e escondem também do operador. Cada um precisa de contador próprio, senão a falha só aparece quando a última camada de proteção cede, e a primeira testemunha é o cliente.

Armadilha: dashboard de vaidade. Trinta gráficos de CPU e memória por pod, nenhum de negócio. A tela mais bonita da Lastro estava 100% verde enquanto a aprovação caía 13 pontos.

Armadilha: contexto perdido na fronteira assíncrona. Instrumentação automática cobre HTTP e gRPC; outbox, CDC e jobs próprios não sabem de trace nenhum até alguém pôr o `traceparent` no dado. Sem isso, todo trace termina no `COMMIT` e o webhook vira outra história.

Armadilha: log em nível `DEBUG` ligado em produção "só por hoje". O volume decuplica, a fatura chega um mês depois e o conteúdo, como o capítulo 33 provou, pode incluir o que nunca deveria estar escrito.

Resumo

- Monitoramento responde perguntas conhecidas; observabilidade permite responder as novas sem deploy.
- Logs estruturados: campos nomeados, `evento` estável, `trace_id`; custo de volume e risco de vazamento.
- Métricas: contador (consulta por taxa), gauge, histograma (buckets agregáveis, percentil interpolado); séries = produto das cardinalidades.
- Trace: spans com pai, propagação W3C `traceparent`, links em lotes, amostragem na cabeça ou na cauda.
- OpenTelemetry: API, SDK, convenções semânticas, OTLP e Collector; backend intercambiável.
- RED para serviços, USE para recursos, métricas de negócio e contadores de degradação para o que os dois não veem.

A Lastro agora

Toda cobrança tem um trace da API ao webhook, com o `traceparent` atravessando a outbox, o CDC e o broker, e amostragem na cauda que guarda todos os erros, recusas e lentos. Logs são JSON com campos fixos e lista permitida; métricas seguem RED, USE e negócio, com rótulos controlados pelo Collector e um contador para cada caminho de degradação. O segmento de cartão tem Collector próprio e só deixa sair atributos permitidos. O `redis-sinais` ganhou `maxmemory` recalculado e alerta de memória.

O Vigia, que no capítulo 25 ficou cego de um olho, passou a sexta cego dos dois sem que ninguém notasse, e agora tem até contador para isso. Os alertas novos disparam quando algo muda; ninguém ainda definiu quanto a Lastro pode falhar antes de doer. O novo contrato vai definir isso pela Lastro, com cláusula de SLA e multa, e o advogado do cliente não aceita "o dashboard estava verde" como evidência.

Referências

- Sigelman, B. et al. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google, 2010. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/
- W3C. *Trace Context*. https://www.w3.org/TR/trace-context/
- OpenTelemetry. *Documentation*. https://opentelemetry.io/docs/
- OpenTelemetry. *Semantic Conventions*. https://opentelemetry.io/docs/specs/semconv/
- Prometheus. *Metric types* e *Histograms and summaries*. https://prometheus.io/docs/concepts/metric_types/ e https://prometheus.io/docs/practices/histograms/
- Gregg, B. *The USE Method*. https://www.brendangregg.com/usemethod.html
- Wilkie, T. *The RED Method: key metrics for microservices architecture*. Grafana Labs. https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/
- Beyer, B. et al. *Site Reliability Engineering*, cap. 6, "Monitoring Distributed Systems". O'Reilly, 2016. https://sre.google/sre-book/monitoring-distributed-systems/
- Redis. *Key eviction*. https://redis.io/docs/latest/develop/reference/eviction/

CAPÍTULO 39

SLI, SLO e SLA

O anexo 3, ou como um nove a mais virou cláusula de multa

O contrato definitivo do Tuiuiú chegou com 84 páginas, e o gerente de produto respondeu "de acordo" em onze minutos, ou seja, leu o assunto do e-mail e a assinatura.

Na página 61 mora o anexo 3: disponibilidade mensal mínima de 99,99%, multa de 10% da mensalidade abaixo disso, 25% abaixo de 99,9% e 50% abaixo de 99,5%. O gerente justificou: "a gente já tem 99,9, é só um nove a mais". Ninguém sabia de onde saiu o 99,9. Estava no slide de vendas desde 2023, ao lado da foto de um homem apertando a mão de um servidor.

A migração do Tuiuiú termina em seis semanas, e o jurídico dele quer saber qual foi a disponibilidade da Lastro no mês passado. O Carlos respondeu "100%", porque nenhum Deployment ficou com zero réplicas. O fundador respondeu "o temp-01 ficou seis anos no ar", frase que só se diz em velório. O financeiro abriu uma aba nova chamada "SLA" e já calculou a multa.

Você foi aos contadores da borda (capítulo 38). Contando como falha todo `5xx` e toda requisição sem resposta no gateway, os últimos três meses deram 99,971%, 99,938% e 99,989%. Com o anexo 3 como está, multa nos três. A Lastro viraria uma empresa que fatura o Tuiuiú e devolve 10% por esporte.

O que a Lastro deve medir, quanto pode prometer, e como saber, numa terça às 15h, que está gastando a indisponibilidade que não tem?

Três siglas, três níveis de compromisso

O mercado usa as três siglas como sinônimos, do mesmo jeito que usa "deploy na sexta" como sinônimo de "fim de semana livre".

O **SLI** (*service level indicator*) é uma medição de um aspecto do serviço visto pelo usuário. A forma mais útil é a razão entre eventos bons e válidos:

```
SLI = eventos bons / eventos válidos

ex.: requisições POST /v1/cobrancas respondidas sem 5xx
     ------------------------------------------------  = 0,99971
     requisições POST /v1/cobrancas válidas
```

O **SLO** (*service level objective*) é uma meta para um SLI numa janela: "99,97% das autorizações sem `5xx`, nos últimos 30 dias". É uma promessa que você faz a si mesmo, sem advogado. O **SLA** (*service level agreement*) é um contrato: um SLO, uma regra de medição aceita pelas duas partes e uma consequência financeira. Se ninguém paga nada quando a meta falha, você tem um SLO, por mais que o slide diga SLA.

O SLO precisa ser mais rígido que o SLA: é o alarme que toca antes da multa. SLO igual ao SLA é detector de fumaça que dispara quando a casa termina de queimar.

O que é um evento bom

Válido decide o denominador. Um `422` por JSON malformado foi culpa do cliente e fica fora; um `429` por limite de taxa (capítulo 28) também. Um `503` do load shedding entra, como ruim: a Lastro descartou a requisição do comprador para salvar outras. Load shedding salva o sistema e gasta orçamento, as duas coisas ao mesmo tempo.

Bom decide o numerador. Latência se escreve como proporção, "respostas em até 1,5 s / respostas válidas", em vez de "p99 < 1,5 s". As duas dizem o mesmo, mas a proporção soma entre processos e minutos, e percentil não soma. Média de percentil, como no capítulo 14, tem alma de horóscopo.

Os tipos de SLI que cobrem quase todo backend:

Tipo	Evento bom	Exemplo
Disponibilidade	resposta sem falha do servidor	`POST` sem `5xx`
Latência	resposta abaixo do limite	resposta final em ≤ 1,5 s
Qualidade	resposta completa, sem degradação	autorização avaliada com todos os sinais
Frescor	dado atualizado dentro do prazo	webhook iniciado ≤ 10 s após o pagamento
Correção	resultado certo	lançamento confere com o arquivo do adquirente
Durabilidade	dado gravado continua legível	lançamento confirmado sobrevive a falha

A linha de qualidade existe por causa da sexta-feira do capítulo 38: zero `5xx`, gráficos verdes, aprovação despencando. A disponibilidade deu nota 10 a um serviço que recusava cliente bom por falta de memória no Redis. O SLI mede o que você mandou medir, com o zelo de um fiscal que confere se a obra tem porta e nunca abre a porta.

Onde medir

No **servidor**, você tem detalhe e não vê o que morreu antes dele: o balanceador que derrubou a conexão, o certificado vencido. Na **borda** (gateway), você vê tudo que chegou, inclusive o que nenhum pod respondeu. No **cliente**, vê a experiência inteira e não controla nada. Na **sonda sintética**, um robô de fora faz uma requisição conhecida a cada tantos segundos.

Agora o bit do capítulo 10. Domingo, 3h12, certificado expirado, nenhum handshake completa. O gateway não recebe requisição HTTP nenhuma, logo não registra falha nenhuma:

```
eventos válidos:  0
eventos bons:     0
SLI:              0 / 0  -> o painel mostra "sem dados", ou pior, 100%
```

Duas horas e 35 minutos fora do ar, e o SLI tranquilo: é a disponibilidade do morto, zero reclamações, 100% de satisfação no túmulo. Por isso o SLI por requisição precisa de uma sonda externa medida por tempo, em que cada tentativa falha conta como intervalo ruim.

Os noves, ou quanto tempo você pode passar morto por mês

Uma meta em noves é o tempo fora que a janela tolera, escrito de forma curta:

Meta	Indisponível em 30 dias	Indisponível por ano
99%	7,2 h	3,65 dias
99,9%	43,2 min	8,76 h
99,95%	21,6 min	4,38 h
99,97%	~13 min	~2,6 h
99,99%	4,32 min	52,6 min
99,999%	~26 s	~5,3 min

Cada nove divide o orçamento por dez. Isso é aritmética, e é a aritmética mais cruel do livro.

Cinco noves dão 26 segundos por mês. O failover do Patroni no ledger leva de 30 a 40 s (capítulo 23). Uma falha de máquina, tratada com perfeição, estoura o mês. Cinco noves exigem que o primário nunca morra, requisito que só o temp-01 cumpriu, e por teimosia.

Quatro noves dão 4 minutos e 19 segundos: um rolling update torto que leva três minutos para voltar consome 70% do mês. O "só um nove a mais" do gerente é um corte de 90% no tempo que a Lastro tem para errar, vendido como adesivo novo no notebook.

Dependências cobram juros

Serviços em série multiplicam disponibilidades (capítulo 26). Se o adquirente entrega 99,9% no mês, a autorização da Lastro não passa disso, por mais noves que tenha por dentro. Prometer 99,99% sobre uma dependência de 99,9% é assinar a multa de outra empresa com a sua caneta.

As saídas custam. Degradar sem a dependência (o antifraude abaixo de R$ 300,00, capítulo 15) troca falha por qualidade menor. Dois adquirentes em paralelo (`1 - 0,001²`) dobram integração e conciliação. Excluir a dependência do SLA protege o caixa e nada muda para o comprador cujo cartão não passou. O comprador não lê o anexo 3.

Error budget, ou a licença para quebrar

Com meta de 99,97%, os 0,03% restantes são o **error budget** (orçamento de erro): os eventos ruins que a janela aceita. Com uns 50 milhões de cobranças por mês, 15 mil eventos ruins, ou 13 minutos de indisponibilidade total.

Com orçamento, um incidente vira gasto com valor: aquele failover consumiu 4% do mês, aquele deploy ruim, 30%. Serviço que termina todo mês com 95% sobrando tem meta frouxa ou muda devagar demais. E o orçamento tem uso automático: o canary (capítulo 37) pode alongar os degraus quando ele está curto. O sistema passa a ter medo proporcional ao histórico recente, mais maturidade emocional do que boa parte da profissão alcança.

A janela pode ser **de calendário** (zera no dia 1º) ou **deslizante** (os últimos 30 dias). Contratos usam calendário, porque a fatura é mensal. SLOs internos usam a deslizante, porque o calendário é estúpido: no dia 29, com orçamento sobrando, todo risco parece barato, e no dia 1º o passado é perdoado como promessa de ano-novo de entrar na academia.

Alertas por burn rate

O alerta ingênuo é "erro acima de 0,03% por 5 minutos, acorda alguém". Um pico de 0,05% por 5 minutos gasta quase nada do mês e acorda alguém às 3h; um vazamento de 0,02% nunca dispara e come 67% do orçamento em silêncio, até o jurídico do Tuiuiú perguntar.

A métrica certa é a **taxa de consumo** (*burn rate*): quão rápido o orçamento está sendo gasto em relação ao ritmo que o esgotaria exatamente no fim da janela.

```
burn rate = taxa de erro observada / (1 − SLO)

SLO 99,97%  ->  orçamento 0,03%
taxa de erro 0,03%  -> burn rate 1    (acaba em 30 dias, no limite)
taxa de erro 0,432% -> burn rate 14,4 (acaba em ~50 horas)

tempo até esgotar = janela / burn rate
fração gasta em t = burn rate × t / janela
```

O livro de exercícios de SRE da Google propõe um ponto de partida para 30 dias: chamar alguém se 2% do orçamento sumir em 1 hora (burn rate 14,4) ou 5% em 6 horas (burn rate 6), e abrir chamado sem acordar ninguém se 10% sumirem em 3 dias (burn rate 1). Confere: 2% de 720 horas em 1 hora dá 14,4.

Uma janela só demora a esquecer: depois de um pico de 10 minutos resolvido, a de 1 hora fica acima do limiar por mais 50 minutos, gritando sobre um incidente que já virou ata. A correção é exigir também uma janela curta de 1/12 da longa (5 minutos para 1 hora, 30 para 6 horas). A longa prova que o gasto é relevante; a curta, que ainda está acontecendo. Em PromQL, com nomes genéricos:

```
(
  sum(rate(http_requests_total{rota="criar_cobranca", status=~"5.."}[1h]))
  / sum(rate(http_requests_total{rota="criar_cobranca"}[1h]))
) / (1 - 0.9997) > 14.4
and
(
  sum(rate(http_requests_total{rota="criar_cobranca", status=~"5.."}[5m]))
  / sum(rate(http_requests_total{rota="criar_cobranca"}[5m]))
) / (1 - 0.9997) > 14.4
```

No incidente do capítulo 35, 100% de falha no Pix por 6 minutos, a janela de 1 hora cruza 0,432% em uns 16 segundos e o alerta sai em menos de um minuto. Naquela noite, quem avisou foi o Bazar. Um vazamento de 0,2% (burn rate 6,7) dispara o alerta de 6 horas em cerca de 5,4 horas, com 5% do orçamento gasto e o mês intacto.

Tráfego baixo engana: às 4h, com 30 requisições em 5 minutos, uma falha é burn rate 111, e o plantonista acorda por um comprador insone. Pouco volume pede mínimo de eventos por janela e sonda para o resto.

Na Lastro: o que medir, quanto prometer

Os SLIs

O SLI é calculado no gateway, que já exporta contadores RED e o histograma do capítulo 38, com a sonda por fora.

SLI	Válidos	Bom	SLO (30 dias deslizantes)
Autorização: disponibilidade	`POST /v1/cobrancas` cartão, exceto 4xx de cliente e `429`	sem `5xx`, com resposta	99,97%
Autorização: latência	as mesmas, respondidas	estado final em ≤ 1,5 s (`processando` conta como ruim)	99%
Autorização: qualidade	autorizações avaliadas	antifraude respondeu com todos os sinais	99,9%
Criação de Pix	`POST /v1/cobrancas` Pix	sem `5xx`, em ≤ 500 ms	99,97%
Notificação do banco	`POST` em `webhooks-pix`	`2xx` em ≤ 1 s	99,97%
Webhook aos marketplaces	eventos de cobrança	1ª tentativa iniciada em ≤ 10 s do COMMIT	99,9%
Consultas	`GET /v1/cobrancas/{id}`	sem `5xx`, em ≤ 300 ms	99,95%
Sonda externa	tentativas a cada 30 s	sucesso	99,95%

Recusa do emissor é evento bom: o comprador é que não tinha limite. Avaliação sem sinais é ruim no SLI de qualidade, que lê o `antifraude_sinais_ausentes_total` do capítulo 38. A sexta do Vigia cego teria queimado esse orçamento em 40 minutos, e o alerta sairia antes do Bazar. O Bazar segue a sonda sintética mais confiável do país, mas agora tem concorrência.

O webhook mede o início da entrega, e não o `2xx` do marketplace: o endpoint do Hornero de 25 s (capítulo 17) é problema do Hornero.

A sonda roda em dois provedores fora da rede da Lastro, cria um Pix de R$ 1,00 no marketplace Lastro Interna (capítulo 37) e o consulta, pegando DNS, TLS, balanceador, gateway e banco de uma vez. O certificado do capítulo 10 viraria alerta no primeiro minuto, e o financeiro teria dormido um domingo, coisa que neste livro aconteceu uma vez.

O Tuiuiú tem SLIs próprios, filtrados por `marketplace_id`; os outros 41 marketplaces entram no agregado, somando bons e válidos, nunca tirando média.

SLO interno e SLA do contrato

A contraproposta saiu dos três meses medidos:

```
              medido      99,99%      99,95%
mês 1        99,971%     multa       ok
mês 2        99,938%     multa       multa (10%)
mês 3        99,989%     multa       ok

orçamento:   99,95% -> 21,6 min/mês     SLO interno 99,97% -> ~13 min
```

O SLA ficou em 99,95% mensal de calendário para criação e consulta de cobranças: um minuto é indisponível se mais de 5% das requisições válidas do Tuiuiú falharam na borda ou se a sonda falhou duas vezes seguidas. Latência: 99% das autorizações com estado final em até 2 s. Multa de 10% abaixo de 99,95%, 25% abaixo de 99,9% e 50% abaixo de 99,5%. Ficam fora as falhas de adquirente e banco do Pix comprovadas por relatório do parceiro. A Lastro abriu mão da exclusão por manutenção programada, porque não faz manutenção programada (o Pix é 24x7, e a equipe também, contra a vontade de todos).

O SLO interno é mais rígido em tudo: 99,97% contra 99,95%, 1,5 s contra 2 s, janela deslizante, sem exclusão de parceiro. Quando ele esgota, sobram uns 9 minutos antes da multa. Adquirente fora do ar gasta o orçamento interno, porque o comprador que não pagou não quer saber de quem foi a culpa. Os alertas carregam o rótulo `causa`, para o plantonista saber às 3h se o problema é dele ou do adquirente. Muda o tom do palavrão.

Alertas e orçamento

Cada SLO ganhou as três regras de burn rate (14,4 em 1 h e 5 min; 6 em 6 h e 30 min; 1 em 3 dias e 6 h), avaliadas a cada 30 s, com mínimo de 100 eventos na janela curta; abaixo disso, vale a sonda. Alertas de limiar fixo foram apagados; os de causa do capítulo 38 viraram chamado, sem acordar ninguém.

O canary de pagamentos lê o orçamento da autorização antes de cada degrau. Abaixo de 25%, cada degrau dura o dobro, inclusive nas flags de regra de dinheiro. Abaixo de zero, a promoção automática para; o rollback segue liberado, porque devolve orçamento.

Armadilhas e casos reais

> **Armadilha:** SLO de 100%. Orçamento zero faz de todo deploy uma violação, e o único jeito de cumprir é nunca mudar nada, o plano de carreira do temp-01. O livro de SRE da Google observa que o usuário chega por celular, rede doméstica e provedor de internet, todos bem menos confiáveis que cinco noves: acima de certo ponto, ele não percebe a diferença, e você paga por ela.
>
> **Na vida real:** o mesmo livro conta o caso do Chubby, o serviço de locks distribuídos da Google. A instância global era tão estável que equipes passaram a depender dela contando que nunca caísse. Nos trimestres em que a disponibilidade real ficava muito acima da meta, a equipe de SRE a derrubava de propósito, para gastar o orçamento e expor quem dependia do impossível. O único lugar do mundo onde sobrar orçamento é tratado como incidente.

O medidor dentro do medido é o erro clássico. Em 28 de fevereiro de 2017, um comando com parâmetro digitado errado removeu servidores demais no Amazon S3 da região us-east-1, fora do ar por cerca de quatro horas, e o painel de status da própria AWS não conseguia mostrar a falha, porque dependia do S3. Por isso a sonda da Lastro roda fora, contra o desejo do Carlos de hospedá-la "no cluster, que é mais barato".

- **SLI que mede a coisa errada.** O "100%" do Carlos media pods vivos. Pod vivo atrás de certificado vencido atende o mesmo número de clientes que pod morto.
- **SLA mais rígido que o SLO.** A multa chega antes do alarme. Era o que o "de acordo" de onze minutos teria produzido.
- **Contrato sem regra de medição.** "99,95%" sem fonte vira duas planilhas brigando. O anexo 3 revisado diz que a fonte é a borda mais a sonda, contestável em 15 dias. O financeiro já criou a aba de contestação.

Resumo

- SLI é medição (bons sobre válidos), SLO é a meta interna numa janela, SLA é o contrato com consequência financeira.
- `4xx` de cliente e `429` saem do denominador; load shedding entra como ruim; latência vira proporção abaixo de um limite.
- Borda, servidor, cliente e sonda veem falhas diferentes; SLI por requisição é cego quando nada chega.
- Cada nove divide por dez o tempo tolerado; dependências em série limitam os noves.
- Error budget = 1 − SLO; janela deslizante para operar, calendário para contrato.

- Burn rate = erro / (1 − SLO); alertas em duas janelas (14,4 em 1 h, 6 em 6 h, 1 em 3 dias).
- O SLO interno é mais rígido que o SLA e não exclui o que o contrato exclui.

A Lastro agora

A Lastro tem oito SLIs medidos na borda e uma sonda externa que cria um Pix de R$ 1,00 no Lastro Interna a cada 30 segundos. Os SLOs internos são 99,97% para autorização, Pix e notificações do banco, 99% de autorizações finalizadas em 1,5 s e 99,9% com sinais completos do antifraude, em janela deslizante de 30 dias, com alertas por burn rate. O Tuiuiú assinou SLA de 99,95% mensal, com exclusão de falha comprovada de parceiro e multa de 10%, 25% e 50%. O canary lê o orçamento antes de cada degrau. O gerente aprendeu que "um nove a mais" é 90% a menos, e pediu a estimativa do décimo nove em horas.

São 13 minutos de orçamento por mês, e o financeiro reservou uma linha da planilha para cada um. Nenhum deles cobre a API respondendo `200` com perfeição sobre uma tabela de contas destruída. Treze minutos dão para muita coisa, menos para restaurar um banco depois de um `UPDATE` sem `WHERE`, e menos ainda se, no mesmo mês, a região inteira resolver tirar folga junto.

Referências

- Beyer, B. et al. *Site Reliability Engineering*, capítulo 3, "Embracing Risk". Google, 2016. https://sre.google/sre-book/embracing-risk/
- Beyer, B. et al. *Site Reliability Engineering*, capítulo 4, "Service Level Objectives" (inclui o caso do Chubby). Google, 2016. https://sre.google/sre-book/service-level-objectives/
- Beyer, B. et al. *The Site Reliability Workbook*, "Implementing SLOs". Google, 2018. https://sre.google/workbook/implementing-slos/
- Beyer, B. et al. *The Site Reliability Workbook*, "Alerting on SLOs". Google, 2018. https://sre.google/workbook/alerting-on-slos/
- AWS. "Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region", março de 2017. https://aws.amazon.com/message/41926/

CAPÍTULO 40

Backup e recuperação de desastres

O mês em que tudo aconteceu duas vezes

O Carlos tinha duas aprovações, um chamado legítimo e a melhor das intenções, que é exatamente o kit completo de todo desastre em banco de dados.

Terça, 14h03. A analista de risco pediu o bloqueio da conta 8812, ponta de um anel de fraude achado pelo grafo do capítulo 25. A coluna `contas.bloqueada` existe desde então, e o trigger de débito recusa saque de conta bloqueada. O Carlos abriu o papel `quebra_vidro` (capítulo 31: uma hora, duas aprovações, auditado), escreveu o comando em duas linhas no cliente gráfico, selecionou a primeira com o mouse e apertou Ctrl+Enter:

```
lastro_shard0=> UPDATE contas SET bloqueada = true
UPDATE 61300
-- a segunda linha, "WHERE id = 8812;", segue no editor até hoje, intacta, inocente
```

Autocommit ligado. Em 1,5 ms, as duas réplicas síncronas do capítulo 23 receberam o erro e o confirmaram com a diligência com que confirmariam um pagamento. Nenhuma zona de disponibilidade ficou sem o seu próprio desastre.

Às 14h11 o Bazar abriu "URGENTE!!!! BLOQUEARAM GERAL???", dois minutos depois de o vendedor 42 tentar sacar. Os alertas por burn rate do capítulo 39 ficaram quietos: saque recusado por conta bloqueada é um `422` impecável, fora do SLI. O Bazar venceu o monitoramento de novo, por regulamento.

O Carlos propôs `UPDATE contas SET bloqueada = false`, também sem `WHERE`, por simetria. Só que 212 contas já estavam bloqueadas antes, por ordem judicial e por fraude. A aba "bloqueados" da planilha do financeiro tinha 209. O backup diário das 3h tinha as 212 e restaurava em 6 horas. A Lastro restaurou numa instância separada, achou nos logs do capítulo 38 os três bloqueios da manhã e corrigiu à mão. Saques parados das 14h03 às 20h05.

Dez dias depois, sábado 9h20, a região do provedor onde a Lastro inteira mora teve uma falha de rede entre zonas e no plano de controle, por 5 horas e 40 minutos. Zonas, etcd, cluster, broker, bucket de backups: tudo na mesma região, que é onde se guarda tudo quando se tem fé. O banco do Pix desistiu de 3.400 notificações. O SLA do Tuiuiú tolera

21,6 minutos por mês; foram 340, 99,21%, multa de 50%. A exclusão cobre adquirente e banco do Pix. O provedor de nuvem não estava nela, porque ninguém achou que ele fosse um parceiro. Ele também não achou.

O fundador disse "no meu tempo era só um servidor e funcionava", e desta vez ninguém conseguiu discordar com números.

Quanto dado a Lastro pode perder em cada caso, quanto tempo pode ficar parada, e o que precisa existir antes do desastre para que essas duas respostas sejam verdade?

RPO e RTO, ou quanto você perde e quanto você espera

São duas metas, definidas antes do incidente.

O **RPO** (*recovery point objective*) é a perda máxima aceitável, medida em tempo: o ponto de recuperação mais antigo que você tolera. RPO de 24 horas significa que voltar com os dados de ontem é aceitável. RPO zero significa que nenhuma transação confirmada pode sumir. O **RTO** (*recovery time objective*) é o tempo máximo entre a falha e o serviço funcionando de novo.

RPO é propriedade do dado; RTO, do serviço. O cache do Redis tem RPO infinito, porque perdê-lo é aceitável por definição. O ledger tem RPO zero, porque lançamento perdido é dinheiro sem explicação. Baixar o RPO custa latência em toda escrita, todos os dias, para um desastre que talvez nunca venha. Baixar o RTO custa infraestrutura ociosa esperando o fim do mundo, o modelo de negócio das seguradoras e do bunker do fundador.

Replicação copia o erro com perfeição

Réplica protege contra falha física: disco, máquina, zona. Contra erro lógico (o `UPDATE` sem `WHERE`, o deploy que grava tarifa errada, o ransomware que cifra tudo com o seu usuário), a réplica síncrona é cúmplice: replica a destruição em milissegundos, com garantia de durabilidade. É a única parte do sistema que nunca atrasa quando você mais queria que atrasasse.

Erro lógico exige uma cópia do passado: backup, réplica atrasada ou histórico versionado. A pergunta é "como estava às 14h02?", e nenhuma réplica em tempo real sabe responder.

Tipos de backup

Um **backup lógico** (`pg_dump`) exporta o conteúdo a partir de um snapshot MVCC consistente. É portável entre versões e restaura reconstruindo tudo, índices inclusive: para um ledger de terabytes, um feriado prolongado.

Um **backup físico** copia os arquivos do banco (`pg_basebackup` , ou ferramentas como pgBackRest e WAL-G). Os arquivos copiados durante a escrita ficam inconsistentes; na restauração, o PostgreSQL reaplica o WAL gerado durante a cópia até um ponto consistente. A cadência:

Tipo	O que copia	Restauração
Completo	tudo	só ele
Diferencial	o que mudou desde o último completo	completo + último diferencial
Incremental	o que mudou desde o último backup de qualquer tipo	completo + todos os incrementais da cadeia
Contínuo (arquivamento de WAL)	cada segmento de WAL, assim que fecha	base + WAL até o instante desejado

O PostgreSQL 17 trouxe incremental nativo no `pg_basebackup` , com sumarização de WAL e o `pg_combinebackup` para montar a cadeia. Incremental economiza disco e faz a restauração depender de todos os elos: um elo corrompido e o resto da cadeia vira peça de museu.

O contínuo é o que importa para dinheiro. Com um backup base e todo o WAL desde ele arquivado, você reconstrói o banco em qualquer instante entre a base e o último segmento arquivado. Isso é o **point-in-time recovery** (PITR):

```
# postgresql.conf da instância restaurada + arquivo recovery.signal
restore_command           = '<ferramenta de backup> busca %f para %p'
recovery_target_time      = '2026-10-20 14:03:00-03'
recovery_target_inclusive = off      # para antes do commit do desastre
recovery_target_action    = 'pause'  # espera você conferir antes de abrir
```

O RPO do backup contínuo é o atraso do arquivamento: o segmento de WAL (16 MB) vai para o repositório quando fecha, e o `archive_timeout` força o fechamento em tráfego baixo. O RTO é a cópia da base mais a reaplicação do WAL desde ela: reaplicar 23 horas de WAL leva horas, durante as quais o seu currículo também está sendo reaplicado.

A **réplica atrasada** é uma standby com `recovery_min_apply_delay = '1h'`: recebe o WAL na hora e só o aplica uma hora depois. Diante de um erro lógico, você pausa a reaplicação (`pg_wal_replay_pause()`), lê o passado e copia o que precisa. É PITR já restaurado, esperando o desastre com a paciência de quem sabe que ele vem.

A regra 3-2-1 e a cópia que ninguém apaga

Três cópias, em dois tipos de mídia, uma fora do local (a formulação é atribuída ao fotógrafo Peter Krogh). Hoje, "fora do local" é outra região e, de preferência, outra conta: quem tem as credenciais de produção apaga a produção e, se puder, o backup, como na Code Spaces do capítulo 36. A defesa é a cópia imutável: armazenamento de objetos com retenção travada (*object lock*, ou WORM), numa conta separada, em que a produção escreve e ninguém apaga antes do prazo. Nem o fundador. Principalmente o fundador.

Estratégias de recuperação de desastres

Para perda de região, há quatro arranjos clássicos (o documento de recuperação de desastres da AWS usa estes nomes, e eles valem para qualquer provedor):

Estratégia	Na região secundária	RPO típico	RTO típico	Custo
Backup e restauração	só os backups	horas	horas a dias	armazenamento
Pilot light	dados replicados, computação desligada	segundos a minutos	dezenas de minutos	bancos de réplica
Warm standby	tudo rodando em escala reduzida	segundos ou zero	minutos	uma fração da produção
Ativo-ativo	tudo rodando e servindo tráfego	zero ou quase	perto de zero	produção dobrada + conflitos

Ativo-ativo parece a resposta óbvia até a primeira escrita. Duas regiões aceitando débitos da mesma conta são o capítulo 23 de novo: multi-líder com conflito, e conflito em saldo se chama saque dobrado. O vendedor 42 abriria uma aba em cada região. Ativo-ativo funciona para dado que admite fusão, ou com cada registro tendo dono numa região só.

Na Lastro

RPO e RTO por dado

Dado	RPO	RTO (região)	Mecanismo
Ledger (shards) e `pg-pagamentos`	zero	30 min	síncrona nas zonas e na região secundária; réplica atrasada 1 h; PITR 35 dias
`pg-cofre`	≤ 5 s	30 min	assíncrona para o CDE secundário
`pg-notificacoes`	≤ 1 min	1 h	assíncrona; projeções reconstruíveis (capítulo 21)
Broker	reconstruído	30 min	republicação da outbox (7 dias), dedupe por `id`
Redis (cache, sinais, sessões)	sem cópia	aquecimento	degradação dos capítulos 15 e 25; vendedores refazem login
Colunar e grafo	24 h	1 dia	backup diário na região secundária

O RPO zero do ledger vem do requisito do capítulo 23, agora estendido à região: nenhum lançamento confirmado pode sumir. `pg-pagamentos` entra junto porque é a origem dos eventos que o ledger consome; perder a cauda dele é perder pagamentos que o banco já considera notificados. Só replicação síncrona entrega RPO zero, e o custo é aritmética: a região secundária fica a ~400 km, com RTT medido de ~8 ms (hipótese do projeto, coerente com 1 ms por 100 km de fibra do capítulo 8, mais roteamento). Todo COMMIT ganha 8 ms. A conta quente do Tuiuiú, que segurava o balde por ~3,5 ms, passa a segurar por ~11,5 ms; com 32 baldes, o teto cai para ~2.800 créditos/s, abaixo dos ~4.000 do dobro do pico. O shard do Tuiuiú vai a 64 baldes (~5.600/s). O Carlos pediu 1.024 no capítulo 4, recebeu 64 trinta e seis capítulos depois e trata isso como vitória moral. A autorização paga 16 ms das duas transações curtas em `pg-pagamentos`, dentro da folga de 120 ms.

A regra de confirmação passa a exigir a réplica da outra região mais uma das zonas:

```
synchronous_standby_names = 'FIRST 2 (pg-dr, pg-ledger-2, pg-ledger-3)'
```

Com `FIRST 2`, o COMMIT espera as duas de maior prioridade conectadas: normalmente `pg-dr` e `pg-ledger-2`. Se o enlace entre regiões cair, `pg-ledger-3` toma o lugar e as escritas continuam com RPO zero só dentro da região; um alerta dispara se `pg-dr` ficar fora do conjunto por mais de 60 s. É uma escolha explícita: piscada de rede entre regiões não para pagamento, e por esses minutos a perda da região custaria segundos de dados, que a conciliação recupera.

`pg-cofre` fica assíncrono porque a perda é recuperável: um cartão salvo nos últimos segundos some, e o comprador digita de novo.

Contra o erro humano

Cada shard do ledger e `pg-pagamentos` ganharam:

- **Arquivamento contínuo de WAL** com base completa diária e PITR de 35 dias, num repositório na região secundária, conta de backup separada, retenção travada e chave própria no KMS. A cópia mensal vai para o armazenamento frio de outro provedor: três cópias, dois provedores, fora do local.
- **Réplica atrasada** de 1 hora, `nofailover`, em cada shard. A terça-feira teria levado uns 15 minutos: pausa, `SELECT id FROM contas WHERE bloqueada` na réplica, correção com `WHERE id IN (...)`.
- **Histórico em `contas`**, pelo mesmo trigger que vendedores e tarifas já têm (capítulo 21).
- **`quebra_vidro` somente leitura por padrão** (`ALTER ROLE ... SET default_transaction_read_only = on`): escrever exige `BEGIN READ WRITE` explícito, e o `UPDATE 61300` aparece antes do COMMIT, quando ainda dá para dizer ROLLBACK em vez de palavrão.

Contra a perda da região

A região secundária roda em warm standby: os bancos acima replicando, um cluster Kubernetes com 25% da capacidade mínima, pods de lastro (capítulo 35) e as mesmas imagens, criados pelos mesmos módulos Terraform com `for_each` por região. Ativo-ativo foi descartado pelo motivo do vendedor 42. Pilot light foi descartado porque subir o cluster do zero estoura os 30 minutos.

Failover de região é decisão humana, com até 10 minutos entre o alerta e a declaração. Falha de região quase nunca é limpa: metade das zonas responde, alguns marketplaces alcançam a região e outros não. Promover automaticamente nesse estado é o split-brain do capítulo 23 em escala continental. O etcd do Patroni passa a ter cinco nós: dois na principal, dois na secundária e um numa terceira região testemunha. Um primário isolado não consegue renovar a liderança e o Patroni o rebaixa em 30 s: o fencing do capítulo 20, entre regiões. O custo é que uma partição longa entre regiões derruba as escritas da principal saudável. Para dinheiro, a Lastro prefere parar de escrever a escrever duas vezes.

O que mais precisa existir antes, porque não está em backup nenhum:

- **IPs liberados nos parceiros.** O adquirente aceitava até 2 IPs (capítulo 36). Um aditivo contratual levou o limite a 4, e os NATs do CDE secundário estão cadastrados.

O banco do Pix recebeu o formulário com firma reconhecida dos IPs novos, prazo de 10 dias úteis.

- **Certificados mTLS** do banco para as máquinas da região secundária, com os mesmos alertas de 60, 30 e 14 dias.
- **Chaves do KMS multirregião.** Backup do cofre cifrado com uma chave que só existe na região perdida é crypto-shredding involuntário.
- **Segredos, imagens e estado do Terraform** replicados. Estado do Terraform guardado só no bucket da região que caiu deixa você com o código da infraestrutura e sem saber o que ele já criou.
- **DNS** de `api` e `webhooks-pix` com TTL de 300 s (capítulo 8), trocado por Terraform da região secundária.

Depois do failover: a conciliação

O failover termina quando o dinheiro fecha. Com RPO zero no ledger e em `pg-pagamentos`, o que fica para trás são as conversas interrompidas com parceiros:

```
1. relay CDC na secundária republica a outbox da última hora; consumidores descartam
`id` repetido
2. resolução (cap. 15) retoma cobranças `processando` consultando o adquirente
3. saga de repasse (cap. 18) consulta o banco sobre todo repasse `enviando`
4. conciliação extraordinária D0 com o extrato do banco do Pix, por EndToEndId:
   Pix recebido sem notificação aceita -> credita pelo fluxo normal (referência UNIQUE)
5. fechamento do dia contábil só depois do item 4 zerar
```

O item 4 é o que teria salvo as 3.400 notificações abandonadas pelo banco no sábado. A conciliação do capítulo 18 virou procedimento de recuperação de desastre, e a planilha do financeiro ganhou status de plano de contingência.

Testes de restauração

Backup que nunca foi restaurado é uma hipótese com custo de armazenamento. A Lastro testa três coisas:

- **Toda semana, automático:** restaura a base mais recente de cada banco numa conta isolada, aplica PITR até um instante sorteado dos últimos 7 dias e confere soma zero por transação e moeda, `saldos_fechados` contra o ledger e contagem por partição contra o primário no mesmo LSN. Mede o tempo. Alerta se passar de 4 horas ou se uma conferência falhar.
- **Todo mês:** pausa a réplica atrasada e recupera uma linha escolhida.
- **Todo semestre:** troca planejada de região em produção, madrugada de domingo. Na troca planejada, as escritas pausam até a secundária confirmar o último LSN, e a pausa medida é de ~3 minutos, dentro dos 13 do orçamento do capítulo 39.

O primeiro teste de região levou 47 minutos: um módulo Terraform com as zonas da principal escritas à mão, um gateway sem certificado do banco e a cota de máquinas da secundária que ninguém pediu para aumentar. O segundo levou 26.

Armadilhas e casos reais

> **Na vida real:** em 10 de março de 2021, um incêndio destruiu o datacenter SBG2 da OVHcloud em Estrasburgo e danificou parte do SBG1 vizinho. Clientes que guardavam o backup no mesmo complexo perderam produção e backup no mesmo incêndio. O "fora do local" da regra 3-2-1 existe por causa de manhãs assim.
>
> **Na vida real:** durante a produção de *Toy Story 2*, em 1998, um `rm` recursivo apagou a maior parte dos arquivos do filme num servidor da Pixar, e os backups vinham falhando sem que ninguém soubesse. O filme foi salvo por uma cópia que uma pessoa da equipe mantinha em casa para trabalhar remotamente. O plano de recuperação de desastres de uma das maiores produções da época foi um computador doméstico.
>
> **Armadilha:** medir o RTO no slide. O RTO real inclui decidir e achar a senha do console.

- **Backup sem as chaves.** Backup cifrado com chave apagada ou presa na região morta é ruído aleatório com metadados.
- **Restaurar por cima da produção.** A terça precisava de 212 linhas, e não do banco de 14h02: voltar a produção inteira apagaria horas de pagamentos legítimos para desfazer uma coluna. Restaura-se ao lado e copia-se o necessário.

Resumo

- RPO é a perda máxima aceitável, por dado; RTO, o tempo máximo até voltar, por serviço.
- Replicação protege contra falha física e copia erro lógico.
- Backup físico com WAL arquivado permite PITR; réplica atrasada é PITR pronto.
- 3-2-1 com uma cópia imutável em outra conta e outra região.
- Backup e restauração, pilot light, warm standby e ativo-ativo trocam custo por RTO e RPO.
- RPO zero entre regiões custa o RTT em todo COMMIT.

- Failover de região exige fencing e o que não está em backup: IPs, certificados, chaves, estado.
- Restauração só existe se testada, com tempo medido.

A Lastro agora

O ledger e `pg-pagamentos` confirmam cada COMMIT numa zona e na região secundária, a ~8 ms, com RPO zero, e o shard do Tuiuiú tem 64 baldes para pagar essa conta. Cada shard tem réplica atrasada de uma hora, WAL arquivado com PITR de 35 dias num repositório imutável em outra conta e outra região, e cópia mensal em outro provedor. A região secundária roda em warm standby, com etcd em três regiões, IPs cadastrados nos parceiros, certificados do banco, chaves multirregião e RTO medido de 26 minutos. Depois de um failover, a conciliação fecha o que ficou no ar. `contas` tem histórico, e o `quebra_vidro` começa somente leitura.

O segundo teste semestral de região rodou num domingo às 3h, com três minutos de escrita pausada. Nenhum lançamento perdido, nenhuma notificação abandonada, nenhum centavo de diferença na planilha, que o financeiro conferiu às 3h40 por esporte. Na segunda de manhã, pela primeira vez em quarenta capítulos, chegou um chamado do Bazar sobre um incidente que ele não percebeu. Assunto: "URGENTE!!!! tá tudo bem aí? tá quieto demais".

Referências

- PostgreSQL. "Continuous Archiving and Point-in-Time Recovery (PITR)". https://www.postgresql.org/docs/current/continuous-archiving.html
- PostgreSQL. "Write Ahead Log" (arquivamento e alvos de recuperação). https://www.postgresql.org/docs/current/runtime-config-wal.html
- PostgreSQL. `pg_basebackup` e `pg_combinebackup` (backup incremental, versão 17). https://www.postgresql.org/docs/current/app-pgbasebackup.html
- PostgreSQL. "Replication" (`synchronous_standby_names` , `recovery_min_apply_delay`). https://www.postgresql.org/docs/current/runtime-config-replication.html
- Patroni. Documentação. https://patroni.readthedocs.io/
- AWS. "Disaster Recovery of Workloads on AWS: Recovery in the Cloud" (estratégias de DR). https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html

- Data Center Knowledge. "Fire Has Destroyed OVH's Strasbourg Data Center (SBG2)", março de 2021. https://datacenterknowledge.com/uptime/fire-has-destroyed-ovh-s-strasbourg-data-center-sbg2